book 14

knowledge discovery

from preprocessing to self-organizing maps in c23, with a go and duckdb capstone

Raeliosol

version 2.0, compiled 10 2026

Contents

1. the knowledge discovery process 7
   1.1. the process, not the step 7
   1.2. one filter at a time 8
   1.3. what survives, and the order it keeps 9
   1.4. four questions, ten arcs 10
   1.5. the c lane and its gate 11
2. data cleaning and imputation 12
   2.1. the hole, and the column around it 12
   2.2. three fills, one invariant 13
   2.3. one bad apple, measured in sigmas 14
   2.4. fences that need no mean 15
3. binning, smoothing, and discretization 17
   3.1. equal width, four shelves 17
   3.2. equal frequency, same series 18
   3.3. smoothing by means and medians 18
   3.4. smoothing by boundaries 19
4. normalization and standardization 21
   4.1. min-max, two targets 21
   4.2. z-scores, the population way 22
   4.3. the robust z, median and MAD 23
   4.4. agreement on symmetric data 24
5. distance metrics 26
   5.1. the minkowski family 26
   5.2. exact roots and integer witnesses 27
   5.3. mahalanobis, distance with a metric 28
   5.4. the shrinking ball, and the name 29
6. similarity measures 31
   6.1. sets: intersection over union 31
   6.2. vectors: one sqrt, not two 32
   6.3. strings: five edits, one alignment 33
   6.4. correlation: cosine after centering 34
7. sampling 36
   7.1. the engine: park-miller minstd 36
   7.2. five of twelve, fully traced 37
   7.3. stratified: hamilton’s leftover seat 38
   7.4. progressive: stop when the gain dies 39
8. feature selection and chi-square 41
   8.1. a score that hates redundancy 41
   8.2. best-first, and when to stop 42
   8.3. the 2x2, and yates 44
   8.4. a 2x3 that stays under 46
9. principal component analysis 48
   9.1. five points, one covariance 48
   9.2. the eigenvalues, before the solver 49
   9.3. the jacobi solver, and the handshake 50
   9.4. projection, and the variance that comes back 52
10. the haar wavelet transform 55
   10.1. averages and differences 55

10.2. the inverse, bit for bit 56
10.3. keep 2 of 8 57
10.4. energy, accounted in fractions 59
11. data cubes and olap 62
11.1. the star, and the twelve facts 62
11.2. the cube lattice, 46 cells of 64 63
11.3. roll-up and drill-down 64
11.4. slice, dice, and the pivot 65
12. simple linear regression 68
12.1. the least squares line 68
12.2. a slope the cpu cannot hold 69
12.3. residuals, and the sum that must be zero 70
12.4. the decomposition, and R squared 71
13. decision trees: id3 74
13.1. the fourteen rows, and one impure root 74
13.2. every branch has an entropy 75
13.3. information gain picks the root 76
13.4. recursion, subtrees, and a byte-exact tree 77
13.5. the tree on rows it has never seen 79
14. decision trees: cart and continuous splits 81
14.1. gini, impurity without logarithms 81
14.2. the delta criterion, and one tree twice 82
14.3. midpoints on a continuous axis 84
14.4. binary, the real cart 85
15. rules from trees and sequential covering 87
15.1. one rule per leaf 87
15.2. grow, one condition at a time 88
15.3. the laplace prune, and it fires 89
15.4. the ordered list, and the row that goes wrong 91
16. overfitting and pruning 93
16.1. a fixture planted to overfit 93
16.2. three collapses, then a stop 94
16.3. what the pruning cost, and what it bought 96
16.4. the same tree by another road 97
17. classifier evaluation 99
17.1. the confusion matrix and its five ratios 99
17.2. holdout, three draws and a pinned stream 100
17.3. three folds, one mean 102
17.4. bootstrap and the out-of-bag third 103
18. roc and auc 105
18.1. one threshold per step, seven points 105
18.2. eight of nine pairs 106
18.3. the area, by thirds 107
18.4. two degenerate curves 108
19. naive bayes 110
19.1. one query, four conditionals, two classes 110
19.2. the zero cell and the laplace rescue 111
19.3. three words, two mailboxes 112
19.4. when scores tie 113
20. nearest neighbor 115
20.1. the sorted list is the model 115

20.2. five ks, two ties ... 116
20.3. every neighbor weighs 1/d2 ... 117
20.4. the census, and the region it paints ... 118
20.5. the cell that votes against itself ... 120
21. neural networks ... 121
21.1. six epochs to a clean pass ... 121
21.2. xor breaks every line ... 123
21.3. a hidden layer buys xor ... 125
21.4. one gradient step, exactly ... 126
21.5. backpropagation, one number at a time ... 127
22. support vector machines ... 130
22.1. the hard margin, solved by hand ... 130
22.2. the soft margin and its flat edge ... 132
22.3. the kernel trick, a circle by a plane ... 134
22.4. smo, three updates from zero ... 136
23. bagging, boosting, and random forests ... 140
23.1. five resamples, one vote ... 140
23.2. three rounds of reweighting ... 144
23.3. the vote as an integer comparison ... 145
23.4. random forests and measured importance ... 146
24. class imbalance ... 150
24.1. the paradox of 99 percent ... 150
24.2. pricing a miss at 20 false alarms ... 151
24.3. the balanced set by duplication ... 153
24.4. the balanced set by subtraction ... 155
25. levelwise apriori ... 157
25.1. support and the law named a priori ... 157
25.2. join, prune, and the killing subsets ... 158
25.3. two ways to stop ... 159
25.4. thirteen itemsets, three implementations ... 161
26. vector apriori ... 163
26.1. one column per item ... 163
26.2. compound tidlists ... 165
26.3. bits and popcounts ... 166
26.4. the agreement, all thirteen ... 168
27. maximal itemsets and rule generation ... 170
27.1. the lattice and its two borders ... 170
27.2. witnesses and what each border keeps ... 171
27.3. six rules from one triple ... 172
27.4. the minconf cut, by cross multiplication ... 174
28. fp-growth ... 176
28.1. two passes and the F-list ... 176
28.2. the tree: shared prefixes, counted edges ... 177
28.3. conditional pattern bases ... 178
28.4. the recursion, and the third agreement ... 180
29. interestingness, multilevel, and quantitative rules ... 183
29.1. lift and conviction ... 183
29.2. the chi-square verdict ... 184
29.3. one hierarchy, two levels ... 186
29.4. quantitative rules by binning ... 187
30. sequential patterns ... 189

30.1. elements, order, and the containment test 189
30.2. the k=2 census, twelve candidates 190
30.3. eight frequent, two maximal 191
30.4. the match trace, greedily 193
31. k-means 195
31.1. two moves, one pass 195
31.2. the elbow against the exhaustive optimum 198
31.3. bad init buys a split local minimum 201
31.4. ties, exact halves, and the go bridge 203
32. hierarchical clustering 205
32.1. one matrix, six leaves, no ties 205
32.2. single chains, complete drags 206
32.3. average and unavoidable thirds 208
32.4. lance-williams, two ways to the same fraction 210
33. dbscan 212
33.1. eps balls and the core census 212
33.2. expansion, borders, and noise 214
33.3. density reachability 217
33.4. the border with two claims 218
34. cluster validation 221
34.1. the partition table, priced exactly 221
34.2. silhouettes as reduced fractions 224
34.3. two metrics, two answers 226
34.4. a negative width and two borrowed rows 228
35. cure and chameleon 230
35.1. the fixture and the erring centroid lane 230
35.2. cure: shrink and survive 232
35.3. partition then label 235
35.4. chameleon: the graph lane 237
36. spectral clustering 240
36.1. the laplacian and its integer spectrum 240
36.2. the handshake and the compensated zero 243
36.3. the fiedler vector cuts the ring 245
36.4. ratio cut against normalized cut 248
37. statistical anomaly detection 251
37.1. the known parameters, and the strict rule 251
37.2. one ellipse beats two rulers 252
37.3. fences that need no mean 253
37.4. masking, and the robust catch 254
38. proximity and density anomalies 257
38.1. the k-distance neighborhood 257
38.2. reachability, and the local density 258
38.3. the ratio that flags 259
38.4. what clustering sees instead 260
39. hypothesis testing and p-values 263
39.1. eight heads of ten 263
39.2. what five percent actually buys 264
39.3. four sevens and a two 265
39.4. counting the false alarms 267
40. multiple testing: bonferroni and fdr 269
40.1. one threshold for ten tests 269

40.2. holm's shrinking bar ... 271
40.3. the largest k that holds ... 272
40.4. the staircase, and a stubborn list ... 273
41. rough sets: indiscernibility and approximations ... 276
41.1. one table, seven partitions ... 276
41.2. lower, upper, boundary ... 278
41.3. rough membership ... 280
41.4. the crisp contrast ... 281
42. rough sets: dependency and reducts ... 283
42.1. the positive region and the degree ... 283
42.2. reducts, the minimal full-dependency subsets ... 285
42.3. the core and the full set that is not a reduct ... 287
42.4. monotone along the lattice ... 288
43. rough sets: discernibility and rule induction ... 289
43.1. the matrix of separating attributes ... 289
43.2. the function, and the empty cell it drops ... 290
43.3. prime implicants, brute force, two ways ... 291
43.4. rules, and the one rule the table refuses ... 293
44. self-organizing maps ... 296
44.1. a 2x3 lattice, a ring, and one seed ... 296
44.2. the kernel, truncated so it stays honest ... 298
44.3. five epochs, one dead unit ... 299
44.4. the update, one operation at a time ... 301
45. som analysis and visualization ... 303
45.1. quantization error and SSE ... 303
45.2. topographic error ... 304
45.3. the u-matrix and the component planes ... 305
45.4. k-means contrast, and the ring census ... 306
46. som variants ... 310
46.1. batch: no learning rate, frozen winners ... 310
46.2. growing grid: add units where it hurts ... 312
46.3. neural gas: rank instead of lattice ... 314
46.4. LVQ1: prototypes with a teacher ... 316
47. the bridge: c to go ... 319
47.1. the port and its gates ... 319
47.2. chapter 04, normalization ... 319
47.3. chapter 05, distance ... 321
47.4. chapter 18, roc ... 322
47.5. chapter 31, k-means ... 324
47.6. the agreement table ... 325
47.7. what the bridge buys ... 326
48. the capstone: a kdd pipeline on duckdb, with coverage appendices ... 327
48.1. one module, two lanes ... 327
48.2. the generator plants its own truth ... 328
48.3. the store: load, clean, transform ... 329
48.4. selection, mining, and the planted-truth evaluation ... 330
48.5. the results panel ... 332
48.6. verification: the golden digest ... 332
48.7. topic coverage ... 333
48.8. pinned sources ... 335

1. the knowledge discovery process

One sample carries the chapter: `pipeline.c` pushes a six-record fixture through a four-stage cleaning chain and asserts every stage count, every dropped id, and the surviving rows, 22 checks in all. The chapter makes 5 moves: what knowledge discovery is as a process rather than an algorithm, the chain as code, what survives it and in what order, the four task families the book's 48 chapters mine, and the c23 lane every later chapter rides. Every behavioral claim below is one of the 22 checks of chapter 01′s sample or a sentence quoted from Fayyad, Piatetsky-Shapiro, and Smyth fetched 2026-09-22. The chapters that follow each own one stage of the process in depth, starting with book 14, ch 2 (knowledge discovery, data cleaning and imputation), and book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices) reassembles the whole chain in go over duckdb.

1.1. the process, not the step

Fayyad, Piatetsky-Shapiro, and Smyth, fetched 2026-09-22: "KDD is the nontrivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data." The load-bearing word is process. "The term process implies that KDD comprises many steps, which involve data preparation, search for patterns, knowledge evaluation, and refinement, all repeated in multiple iterations." Mining, the step this book's middle spends its weight on, is one stage inside that loop: "KDD refers to the overall process of discovering useful knowledge from data, and data mining refers to a particular step in this process." The article walks the loop in stages, and its one-sentence compression names them all: "The KDD process involves using the database along with any required selection, preprocessing, subsampling, and transformations of it; applying data-mining methods (algorithms) to enumerate patterns from it; and evaluating the products of data mining to identify the subset of the enumerated patterns deemed knowledge."

The dry run: the chapter fixture is 6 records of (id, age, score), age −1 marking a missing cell. The sample runs the preprocessing lane over all 6 and prints the census first: `stage inputs:  6 5 4 3`, then `stage outputs: 5 4 3 2`. Each stage sees one fewer record than the last and hands one fewer on, 6 in and 2 out, which is the cleaning story of book 14, ch 2 (knowledge discovery, data cleaning and imputation) compressed into two printed lines.

```
  for (int s = 0; s < 4; s++) {
    in_count[s] = n;
    int nd = 0;
    int drop[4];
    switch (s) {
    case 0:
      n = stage_dedupe(table, n, drop, &nd);
      break;
    case 1:
      n = stage_drop_missing(table, n, drop, &nd);
      break;
    case 2:
      n = stage_range(table, n, drop, &nd);
      break;
    default:
      n = stage_format(table, n, drop, &nd);
      break;
    }
    drops_per_stage[s] = nd;
    for (int i = 0; i < nd; i++) {
      drop_ids[total_drops++] = drop[i];
    }
    out_count[s] = n;
  }
```

listing 1: the four-stage chain, census arrays filled per stage (source: kdd/samples/src/Ch01/pipeline.c, lines 116-139)

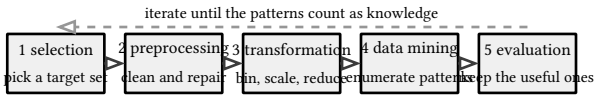


figure 1: the kdd loop of Fayyad et al. 1996, mining is stage 4 of 5

PITFALL: the mining step is the small one

The same article, fetched 2026-09-22: “The additional steps in the KDD process, such as data preparation, data selection, data cleaning, incorporation of appropriate prior knowledge, and proper interpretation of the results of mining, are essential to ensure that useful knowledge is derived from the data.” Ten of this book’s 48 chapters, 01 through 10, never leave those additional steps, and the sample of this chapter lives entirely inside stage 2.

1.2. one filter at a time

The chain is four filters in series over an in-place table: dedupe by id keeping the first occurrence, drop rows whose age cell is missing, range filter $0 \le \text{age} \le 120$, and format filter $0 \le \text{score} \le 10$. Each stage compacts the survivors with the same two-pointer move, copying keepers down with `r[m++] = r[i]` and recording every ejected id, so the census and the drop log fall out of one pass.

The dry run: stage 1 reads 6 rows. The third row (1,34,8) repeats id 1, so dedupe keeps the first copy, output count 5, and check “ch01 stage 1 drops 1 (duplicate id)” fires. Stage 2 sees those 5 and drops id 2, whose age is −1, output 4. Stage 3 drops id 4, age 200 fails the range gate, output 3. Stage 4 drops id 5, score 12 fails the $[0, 10]$ format, output 2. The printed order reads `dropped ids in order: 1 2 4 5`, and each stage’s single drop is one of the four census checks.

```c
/* stage 1: dedupe by id, keep the first occurrence, order preserved */
static int stage_dedupe(rec_t *r, int n, int *drop, int *ndrop) {
  int m = 0;
  for (int i = 0; i < n; i++) {
    bool seen = false;
    for (int j = 0; j < m; j++) {
      if (r[j].id == r[i].id) {
        seen = true;
        break;
      }
    }
    if (seen) {
      drop[(*ndrop)++] = r[i].id;
    } else {
      r[m++] = r[i];
    }
  }
  return m;
}
```

listing 2: stage 1, dedupe by id, keep first, order preserved (source: kdd/samples/src/Ch01/pipeline.c, lines 31-49)

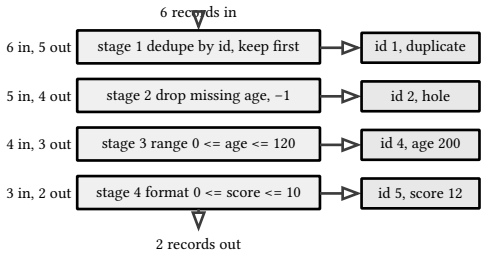


figure 2: six records through four filters, one drop per stage

1.3. what survives, and the order it keeps

A filter chain is only trustworthy when both facts survive it: which rows live, and what order they landed in. The sample checks membership per id and the order of the final table separately, because compaction could easily keep the right ids in the wrong sequence.

The dry run: the last data line prints `final survivors: (1,34,8)(6,29,7)`. Id 1 passed all four gates on its first occurrence, id 6 passed on age 29 and score 7, and ids 2, 4, 5 each failed exactly one gate. The membership checks fire as “ch01 id 1 survives”, “ch01 id 2 dropped”, “ch01 id 4 dropped”, “ch01 id 5 dropped”, “ch01 id 6 survives”, and the order check, “ch01 relative order preserved”, pins that id 1 still precedes id 6, the input order carried through every compaction.

```
/* row: final survivor set -- (1,34,8), (6,29,7), count 2 */
CHECK(table[0].id == 1 && table[0].age == 34 && table[0].score == 8,
      "ch01 survivor row 1 is (1,34,8)");
CHECK(table[1].id == 6 && table[1].age == 29 && table[1].score == 7,
      "ch01 survivor row 2 is (6,29,7)");
CHECK(survivors == 2, "ch01 final count 2");

/* row: per-id final membership -- 1 and 6 live, 2, 4, 5 dropped */
CHECK(survived(table, survivors, 1), "ch01 id 1 survives");
CHECK(!survived(table, survivors, 2), "ch01 id 2 dropped");
CHECK(!survived(table, survivors, 4), "ch01 id 4 dropped");
CHECK(!survived(table, survivors, 5), "ch01 id 5 dropped");
CHECK(survived(table, survivors, 6), "ch01 id 6 survives");
CHECK(table[0].id == 1 && table[1].id == 6, "ch01 relative order preserved");
```

listing 3: the survivor rows and the per-id membership checks (source: kdd/samples/src/Ch01/pipeline.c, lines 173-186)

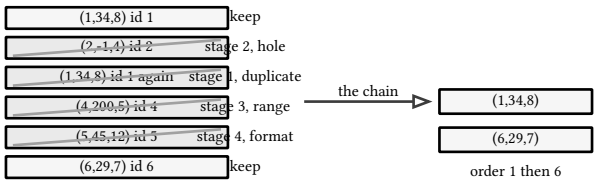


figure 3: the six input rows, four struck out, two survivors

NOTE: keep-first is a policy, not a law

Duplicate ids could resolve by newest, by most complete row, or by merge. The fixture pins keep-first because it is the cheapest deterministic rule, order preserving and one pass. Whatever rule a real pipeline picks, it belongs in the drop census: an id that silently survives twice poisons every count downstream, which is exactly what check “ch01 dropped ids in order 1,2,4,5” guards against.

1.4. four questions, ten arcs

Once the data is clean, the mining step answers one of four questions. Classify: which label does this row take, the arc from book 14, ch 13 (knowledge discovery, decision trees: id3) through book 14, ch 23 (knowledge discovery, bagging, boosting, and random forests). Associate: which items co-occur, book 14, ch 25 (knowledge discovery, levelwise apriori) through book 14, ch 30 (knowledge discovery, sequential patterns). Cluster: which rows group without labels, book 14, ch 31 (knowledge discovery, k-means) through book 14, ch 36 (knowledge discovery, spectral clustering). Flag: which rows are anomalous, book 14, ch 37 (knowledge discovery, statistical anomaly detection) and book 14, ch 38 (knowledge discovery, proximity and density anomalies). Three quieter arcs sit between them, the statistics pair book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) and book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr), the rough set triple, and the self-organizing map trilogy that closes the c23 lane.

The dry run: the fixture already separates the four questions’ raw material. Id 4 with age 200 is a range violation, dead at stage 3 of this chapter, but the same row read against a fitted distribution is the business of book 14, ch 37 (knowledge discovery, statistical anomaly detection), and a row that merely sits far from every cluster is the business of book 14, ch 38 (knowledge discovery, proximity and density anomalies). The census line `dropped ids in order: 1 2 4 5` says the filter caught it here, 47 chapters before a model ever sees it.

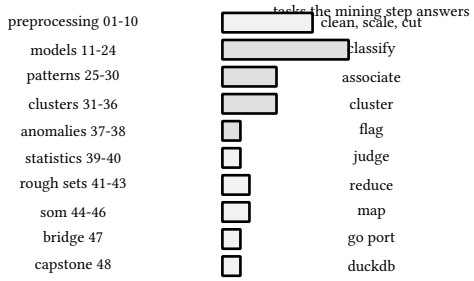


figure 4: the 48 chapters in ten arcs, four of them the task families

1.5. the c lane and its gate

Every theory chapter, 01 through 46, pins its claims the same way: one standalone c23 program per algorithm family, a CHECK macro that prints one ok line per assertion and stops the run on the first failure, fixtures witnessed by a generator script, and a gate that compiles, runs, and formats the file before it can be committed. No clocks, no ambient randomness, no filesystem, so a check that passes once passes every time, the property that lets this book quote sample output as evidence.

The dry run: the gate over this chapter prints `Ch01/pipeline 22 checks` from the compile-and-run leg, then the summary `verify-c: 1 files, 22 checks, 0 ir/objdump/roundtrip assertions, format clean, asan clean`. One file, 22 ok lines, and a format pass that must leave the file untouched, because later chapters pin line ranges into these sources and a reformat would strand every pin.

```c
static int checks = 0;
#define CHECK(cond, name)                                   \
  do {                                                      \
    if (cond) {                                             \
      checks++;                                             \
      printf("ok %d %s\n", checks, name);                   \
    } else {                                                \
      printf("FAIL %s (check %d)\n", name, checks + 1);     \
      return 1;                                             \
    }                                                       \
  } while (0)
```

listing 4: the check harness, one ok line per surviving assertion (source: kdd/samples/src/Ch01/pipeline.c, lines 11-21)

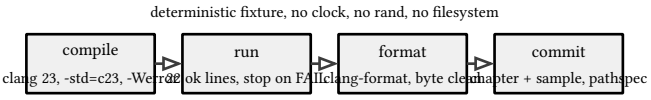


figure 5: the gate every sample passes before its chapter compiles

NOTE: the determinism classes, D0 through D3

Every pinned value in this book belongs to one of four classes. D0: exact integers and strings, compared with ==. D1: doubles written as an integer over a power of two, exact in binary floating point and asserted with ==. D2: prints pinned at a stated tolerance, the class every rounded or irrational double lands in. D3: a double-run identity pin, the sample runs its whole procedure a second time and asserts the two runs match value for value. The tags ride chapter headers and check names, so the check itself says how hard the pin is.

sources: Usama Fayyad, Gregory Piatetsky-Shapiro, Padhraic Smyth, “From Data Mining to Knowledge Discovery in Databases”, AI Magazine 17-3, ojs.aaai.org/index.php/aimagazine/article/view/1230, fetched 2026-09-22. Fixture and all 22 pinned values witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s1.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch01`, 22 checks in chapter 01 of the kdd suite.

2. data cleaning and imputation

Two samples carry the chapter: `impute.c` repairs a 5-row table with 2 holes and asserts 19 checks, `outlier.c` flags the planted outlier of a 9-value series two independent ways and asserts 16. The chapter makes 4 moves: column statistics over observed cells only, the three classic fills and the one invariant mean filling keeps, the z-score flag with its exact fraction arithmetic, and the IQR fences that need no mean and no variance at all. Every behavioral claim below is one of the 35 checks of chapter 02′s samples or a cited sentence from Tukey 1977. Chapter 01′s chain dropped the row with the missing age, book 14, ch 1 (knowledge discovery, the knowledge discovery process) this chapter repairs instead of dropping, and book 14, ch 4 (knowledge discovery, normalization and standardization) scales what survives.

2.1. the hole, and the column around it

The fixture is 5 people by 2 columns, age and income, with one hole in each: p2′s age and p3′s income are missing. Repair starts with statistics over what is observed, because every fill policy is a function of those statistics, and the observed sets are small enough to read off: age (30, 40, 50, 30) and income (40, 50, 60, 40), 4 values each, both an even count.

The dry run: age sums to 150 over 4 observed, mean 37.5. Sorted age reads (30, 30, 40, 50), the even split averages its two middle entries, median (30 + 40)/2 = 35. The value 30 appears twice, the other two once, so the mode is 30 with count 2. Income sums to 190 over 4, mean 47.5; sorted (40, 40, 50, 60) gives median 45, mode 40 with count 2. The sample prints both lines: `age: mean 37.5 median 35 mode 30 x2` and `income: mean 47.5 median 45 mode 40 x2`, and checks ok 1 through ok 8 pin all eight values.

```c
/* median over observed cells only: sort a copy, even count averages the two
 * middle entries (both columns here observe an even 4 values) */
static double col_median(const cell_t *c, int n) {
  double obs[NROWS];
  int seen = 0;
  for (int i = 0; i < n; i++) {
    if (c[i].ok) {
      obs[seen++] = c[i].v;
    }
  }
  for (int i = 1; i < seen; i++) { /* insertion sort */
    double k = obs[i];
    int j = i - 1;
    while (j >= 0 && obs[j] > k) {
      obs[j + 1] = obs[j];
      j--;
    }
    obs[j + 1] = k;
  }
  if (seen % 2 == 1) {
    return obs[seen / 2];
  }
  return (obs[seen / 2 - 1] + obs[seen / 2]) / 2.0;
}
```

listing 5: median over observed cells, insertion sort, even count averaged (source: kdd/samples/src/Ch02/impute.c, lines 42-65)

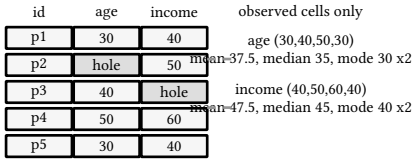


figure 6: the 5-row table, 2 holes, and the observed sets each column keeps

NOTE: missing is a fact, not a zero

A hole is recorded as a flag on the cell, `ok == false`, never as the value 0. Writing 0 into an empty age cell would drag the observed mean to 30 and invent 5 people who died at birth. The sample's `cell_t` carries the flag and the value separately, and the integrity checks count 1 hole per column before any fill and 0 after, checks ok 10 and ok 11.

2.2. three fills, one invariant

Mean, median, and mode filling each replace the hole with one statistic of the observed cells: 37.5 and 47.5 for the mean, 35 and 45 for the median, 30 and 40 for the mode. The choice is a bias decision. The mean keeps the column average but lets every row's estimate drift toward the center of everything else. The mode keeps values legal, income brackets and product tiers are countable things, but ignores magnitudes. The median ignores magnitude almost entirely and resists outliers.

The dry run: fill p2's age with the mean and the age column reads (30, 37.5, 40, 50, 30), sum 187.5 over 5, still 37.5. The mean fill preserves the column mean exactly, and the sample asserts it twice, one per column, as "ch02 post-impute age mean 37.5" and "ch02 post-impute income mean 47.5". The median and mode fills do not preserve it: the median-filled age column averages (30 + 35 + 40 + 50 + 30)/5 = 37, which no check claims and no check needs. All three fills land as checks ok 12 through ok 19.

```c
static void fill(cell_t *dst, const cell_t *src, int n, double v) {
  for (int i = 0; i < n; i++) {
    dst[i] = src[i];
    if (!dst[i].ok) {
      dst[i].v = v;
      dst[i].ok = true;
    }
  }
}
```

listing 6: one fill function serves all three policies (source: kdd/samples/src/Ch02/impute.c, lines 98-106)

```c
  /* row: mean imputation -- fill 37.5 / 47.5, column means unchanged */
  CHECK(age_m[1].v == 37.5, "ch02 mean-impute p2.age 37.5");
  CHECK(col_mean(age_m, NROWS) == 37.5, "ch02 post-impute age mean 37.5");
  CHECK(inc_m[2].v == 47.5, "ch02 mean-impute p3.income 47.5");
  CHECK(col_mean(inc_m, NROWS) == 47.5, "ch02 post-impute income mean 47.5");

  /* row: median imputation -- fill 35 / 45 */
  CHECK(age_d[1].v == 35.0, "ch02 median-impute p2.age 35");
  CHECK(inc_d[2].v == 45.0, "ch02 median-impute p3.income 45");

  /* row: mode imputation -- fill 30 / 40 */
  CHECK(age_o[1].v == 30.0, "ch02 mode-impute p2.age 30");
  CHECK(inc_o[2].v == 40.0, "ch02 mode-impute p3.income 40");
```

listing 7: the three fills checked per hole, mean invariant pinned twice (source: kdd/samples/src/Ch02/impute.c, lines 159-171)

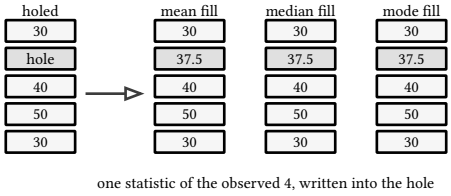


figure 7: the age hole filled three ways, the filled cell shaded

2.3. one bad apple, measured in sigmas

The second fixture is a 9-value series with one planted outlier, E = (10, 12, 14, 16, 18, 20, 22, 24, 44). The z-score asks how many standard deviations a value sits from the mean, and the sample keeps the whole computation in integers until the final division: sum of squares first, then the flag rule as an exact fraction comparison rather than a floating-point subtraction.

The dry run: the sum is 180 over 9, mean exactly 20. The deviations read (-10, −8, −6, −4, −2, 0, 2, 4, 24), they sum to 0, and their squares sum 100 + 64 + 36 + 16 + 4 + 0 + 4 + 16 + 576 = 816. The sample variance divides by n - 1 = 8, giving 102. The outlier's squared score is 24^2/102 = 576/102 = 96/17 after reduction, and the flag rule $|z| > 2$ is equivalent to $z^2 > 4$, checked exactly as 96 > 68. The next-largest squared score is 100/102 = 50/51, from the value 10, well under 4. The printed display value reads `z(44) = 2.3763541031440183`, a print-only row pinned at tolerance 1e-12, and the final count check fires as "ch02 z-score flags only 44".

```
/* row: z-score flags, SAMPLE variance (divisor n-1 = 8) */
double sum = 0.0;
for (int i = 0; i < NE; i++) {
  sum += e[i];
}
const double mean = sum / (double)NE;
int dev_sum = 0, ss = 0;
for (int i = 0; i < NE; i++) {
  int d = (int)e[i] - (int)mean;
  dev_sum += d;
  ss += d * d;
}
const double svar = (double)ss / (double)(NE - 1);
printf("mean %.17g devs sum %d SS %d sample var %.17g\n", mean, dev_sum, ss,
       svar);
CHECK(mean == 20.0, "ch02 mean 20");
CHECK(dev_sum == 0, "ch02 deviations sum 0");
CHECK(ss == 816, "ch02 sum of squares 816");
CHECK(svar == 102.0, "ch02 sample variance 102 (divisor n-1)");
```

listing 8: mean and sum of squares in integers, sample variance divisor 8 (source: kdd/samples/src/Ch02/outlier.c, lines 47-65)

```
/* z^2 of the planted outlier 24 = 24^2/102 = 576/102 = 96/17, kept as a
 * reduced integer pair so the flag rule num > 4*den is exact arithmetic */
int znum = 24 * 24, zden = (int)svar;
const int g1 = gcd(znum, zden);
znum /= g1;
zden /= g1;
printf("z^2(44) = %d/%d, z^2 > 4 iff %d > 4*%d = %d\n", znum, zden, znum,
       zden, 4 * zden);
CHECK(znum == 96 && zden == 17, "ch02 z^2(44) reduces to 96/17");
CHECK(znum > 4 * zden, "ch02 flag rule fires on 44 (|z| > 2)");
```

listing 9: the flag rule as a reduced fraction, exact arithmetic (source: kdd/samples/src/Ch02/outlier.c, lines 67-76)

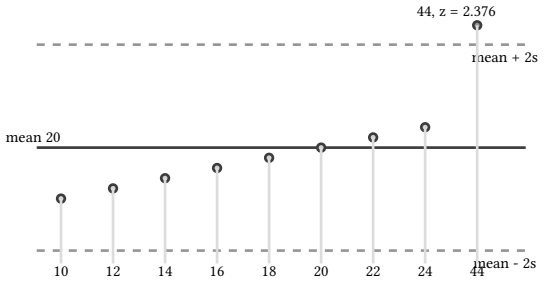


figure 8: the series against the mean and the 2-sigma band, only 44 escapes

NOTE: the divisor is a convention, and this book pins it

The z-score of this chapter divides the sum of squares by n - 1, the sample variance, pinned by the contract sheet and asserted as “ch02 sample variance 102 (divisor n-1)”. The standardization chapter divides by n, the population variance, over its own fixtures. Both are correct under their own convention and the difference is real, 816/8 against 816/9 on this very series, so any pipeline mixing chapters must say which divisor it uses where. The sheet, kdd-contract-s1s2.md, states the convention per row.

2.4. fences that need no mean

Tukey’s schematic plot flags outliers with quartiles alone: fences at $Q_1 - 1.5 \cdot \mathrm{IQR}$ and $Q_3 + 1.5 \cdot \mathrm{IQR}$, where IQR is the interquartile range. The constant 1.5 is a convention Tukey chose for schematic summaries, not a derived threshold, and the rule never computes a mean or a variance, which is its whole point: no outlier can inflate the yardstick that would measure it.

The dry run: 9 sorted values, odd count, so the middle value is excluded and the quartiles come from the halves. Lower half (10, 12, 14, 16), its even split averages the middle pair, Q1 = (12 + 14)/2 = 13. Upper half (20, 22, 24, 44), Q3 = (22 + 24)/2 = 23. IQR = 10, fences 13 - 15 = −2 and 23 + 15 = 38. The sample prints `Q1 13 Q3 23 IQR 10 fences -2 38`, every value is a check, and the final count fires as “ch02 IQR flags only 44”, the same single flag the z-score raised.

```c
/* row: IQR fence flags. odd n = 9: exclude the median, quartiles come
 * from the lower half (10,12,14,16) and upper half (20,22,24,44) */
const double q1 = median_sorted(e, NE / 2);
const double q3 = median_sorted(e + NE - NE / 2, NE / 2);
const double iqr = q3 - q1;
const double lo = q1 - 1.5 * iqr;
const double hi = q3 + 1.5 * iqr;
printf("Q1 %.17g Q3 %.17g IQR %.17g fences %.17g %.17g\n", q1, q3, iqr, lo,
       hi);
CHECK(q1 == 13.0, "ch02 Q1 13");
CHECK(q3 == 23.0, "ch02 Q3 23");
CHECK(iqr == 10.0, "ch02 IQR 10");
CHECK(lo == -2.0, "ch02 lower fence -2");
CHECK(hi == 38.0, "ch02 upper fence 38");

int iflags = 0;
double iflagged = 0;
for (int i = 0; i < NE; i++) {
  if (e[i] < lo || e[i] > hi) {
    iflags++;
    iflagged = e[i];
  }
}
CHECK(iflags == 1 && iflagged == 44.0, "ch02 IQR flags only 44");
```

listing 10: quartiles by excluded-median halves, fences, and the flag count (source: kdd/samples/src/Ch02/outlier.c, lines 118-141)

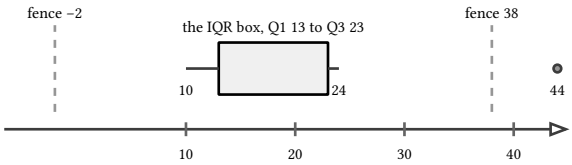


figure 9: the box between the fences, 44 sits outside both

Two methods, one verdict. The z-score needed the mean, the variance, and a divisor convention; the fences needed three order statistics and a constant. That the two agree on this fixture is a property of the planted outlier, and book 14, ch 37 (knowledge discovery, statistical anomaly detection) revisits the disagreement when they don't.

sources: John W. Tukey, Exploratory Data Analysis, Addison-Wesley 1977, schematic plots and the 1.5 fence convention, verified via en.wikipedia.org/wiki/Box_plot, accessed 2026-09-22. All 35 pinned values witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s1.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch02`, 19 + 16 checks in chapter 02 of the kdd suite.

3. binning, smoothing, and discretization

Two samples carry the chapter: `bin.c` cuts a sorted 12-value series into bins two ways and asserts 17 checks, `smooth.c` replaces the classic 9-value textbook series with per-bin statistics and asserts 13. The chapter makes 4 moves: equal-width bins and their half-open edges, equal-frequency bins on the same series, smoothing by bin means and medians, and smoothing by boundaries with its tie rule. Every behavioral claim below is one of the 30 checks of chapter 03's samples. Binning is the first transformation stage move of book 14, ch 1 (knowledge discovery, the knowledge discovery process): it turns a noisy scale into a small set of shelves that book 14, ch 8 (knowledge discovery, feature selection and chi-square) can count against and book 14, ch 4 (knowledge discovery, normalization and standardization) can leave alone.

3.1. equal width, four shelves

Equal-width binning asks only for the range and k: width w = (max - min)/k, and bin k covers $[L + kw, L + (k + 1)w)$, half open, except the last bin which closes on the right so the maximum lands somewhere. The fixture is the sorted series S12 = (2, 5, 9, 13, 14, 20, 25, 26, 33, 38, 44, 50) with k = 4.

The dry run: w = (50 - 2)/4 = 12, so the shelves are [2, 14), [14, 26), [26, 38), and [38, 50]. Walking the series fills B1 with (2, 5, 9, 13), B2 with (14, 20, 25), B3 with (26, 33), B4 with (38, 44, 50), and the sample prints `width 12 counts 4 3 2 3`. The boundary probes print `boundaries: 14→1 26→2 38→3 50→3`, zero-based indexes: 14, 26, and 38 each open the next shelf, and 50, equal to the top edge, lands in the last one because it closes. The 1-based checks read "ch03 boundary 14 opens bin 2" through "ch03 boundary 50 closes bin 4 (last bin)".

```c
/* half-open bins [lo + k*w, lo + (k+1)*w), last bin closed: x == hi lands
 * in k-1. widths and values in this file are dyadic so the divide is exact */
static int bin_of(double x, double lo, double w, int k) {
  if (x ≥ lo + (double)k * w) {
    return k - 1;
  }
  return (int)((x - lo) / w);
}
```

listing 11: the half-open bin index, last bin closed on the right (source: kdd/samples/src/Ch03/bin.c, lines 28-35)

```c
  /* row: width boundary cases -- 14, 26, 38 open left edges, 50 the closed
   * right edge of the last bin */
  printf("boundaries: 14→%d 26→%d 38→%d 50→%d\n", bin_of(14, lo, w, k),
         bin_of(26, lo, w, k), bin_of(38, lo, w, k), bin_of(50, lo, w, k));
  CHECK(bin_of(14, lo, w, k) == 1, "ch03 boundary 14 opens bin 2");
  CHECK(bin_of(26, lo, w, k) == 2, "ch03 boundary 26 opens bin 3");
  CHECK(bin_of(38, lo, w, k) == 3, "ch03 boundary 38 opens bin 4");
  CHECK(bin_of(50, lo, w, k) == 3, "ch03 boundary 50 closes bin 4 (last bin)");
```

listing 12: the four boundary probes and their checks (source: kdd/samples/src/Ch03/bin.c, lines 76-83)

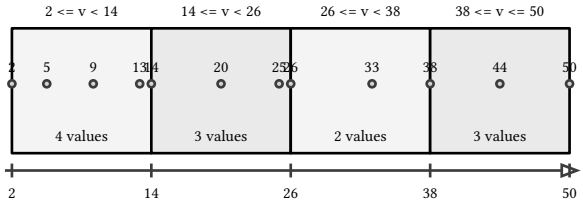


figure 10: S12 in four equal-width shelves, width 12 over 2 to 50

NOTE: the last bin closes because the width says so

> With half-open bins of width 12 over [2, 50], the value 50 satisfies no membership test: it is the excluded left edge of a fifth bin that does not exist. The convention closes the last bin on the right, and the sample asserts the consequence as “ch03 boundary 50 closes bin 4 (last bin)”. Every equal-width implementation has to answer this question somewhere, and the answer belongs in a check, not in a comment.

3.2. equal frequency, same series

Equal-frequency binning, also called equal depth, ignores geometry and cuts the sorted series into k bins of n/k values each, so every shelf holds the same count. On S12 with k = 3 the arithmetic is clean, 12 values into 3 bins of 4.

The dry run: the sorted series simply falls in order, B1 (2, 5, 9, 13), B2 (14, 20, 25, 26), B3 (33, 38, 44, 50), counts all 4, printed as `freq counts 4 4 4`. The two rules disagree about where the cuts go: width puts 26 with 33 in a 2-value bin and leaves 38, 44, 50 together, depth puts 26 with 14 and 20 and 25. The same disagreement shows on the classic 9-value series C9 = (4, 8, 15, 21, 21, 24, 25, 28, 34): 3 equal-width bins over [4, 34] have width 10 and counts (2, 3, 4), printed as `C9 width 10 counts 2 3 4`, with 24 and 34 both landing in bin 3, while equal depth forces (3, 3, 3), the bins the next section smooths.

```c
/* row: equal-frequency membership, 3 bins of 4 over the sorted series
 * (k divides n, no remainder to spread) */
const int kf = 3, per = NS / kf;
int fcounts[3] = {0};
int fmembers[3][NS];
for (int i = 0; i < NS; i++) {
  int b = i / per;
  fmembers[b][fcounts[b]++] = s12[i];
}
printf("freq counts %d %d %d\n", fcounts[0], fcounts[1], fcounts[2]);
CHECK(fcounts[0] == 4 && fcounts[1] == 4 && fcounts[2] == 4,
      "ch03 equal-frequency counts (4,4,4)");
const int want_f1[4] = {2, 5, 9, 13};
const int want_f2[4] = {14, 20, 25, 26};
const int want_f3[4] = {33, 38, 44, 50};
CHECK(eq_arr(fmembers[0], want_f1, 4), "ch03 freq B1 (2,5,9,13)");
CHECK(eq_arr(fmembers[1], want_f2, 4), "ch03 freq B2 (14,20,25,26)");
CHECK(eq_arr(fmembers[2], want_f3, 4), "ch03 freq B3 (33,38,44,50)");
```

listing 13: equal frequency by position in the sorted series, k divides n (source: kdd/samples/src/Ch03/bin.c, lines 85-102)

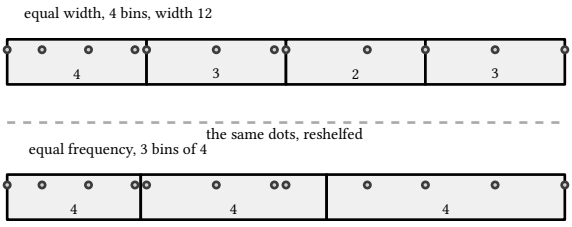


figure 11: the same 12 values cut by width into 4 and by depth into 3

3.3. smoothing by means and medians

Smoothing replaces each value in a bin with a statistic of the bin, trading resolution for stability. The fixture is Zaki and Meira’s equal-depth walkthrough series C9 in 3 bins of 3, (4, 8, 15), (21, 21, 24), (25, 28, 34). Means take the average, medians the middle order statistic, and because the bins are sorted and hold 3 values, the median is simply the middle entry.

The dry run: bin sums are 27, 66, 87, so the means are 9, 22, 29, all exact in binary, printed as `bin means: 9 22 29`, and the smoothed series reads (9, 9, 9, 22, 22, 22, 29, 29, 29). The medians are the middle entries 8, 21, 28, printed as `bin medians: 8 21 28`, giving (8, 8, 8, 21, 21, 21, 28, 28, 28). The mean erases the within-bin spread by construction, the median additionally resists a wild value inside a bin, the difference between the two is the whole of book 14, ch 2 (knowledge discovery, data cleaning and imputation) replayed one level down.

```
/* row: smoothing by bin means -- 27/3=9, 66/3=22, 87/3=29, all dyadic */
double msmooth[NB][PER];
for (int i = 0; i < NB; i++) {
  const double m = bin_mean(bins[i], PER);
  for (int j = 0; j < PER; j++) {
    msmooth[i][j] = m;
  }
}
printf("bin means: %.17g %.17g %.17g\n", msmooth[0][0], msmooth[1][0],
       msmooth[2][0]);
CHECK(msmooth[0][0] == 9.0, "ch03 bin 1 mean 9");
CHECK(msmooth[1][0] == 22.0, "ch03 bin 2 mean 22");
CHECK(msmooth[2][0] == 29.0, "ch03 bin 3 mean 29");
CHECK(msmooth[0][1] == 9.0 && msmooth[0][2] == 9.0,
      "ch03 smooth by means bin 1 -> (9,9,9)");
CHECK(msmooth[1][1] == 22.0 && msmooth[1][2] == 22.0,
      "ch03 smooth by means bin 2 -> (22,22,22)");
CHECK(msmooth[2][1] == 29.0 && msmooth[2][2] == 29.0,
      "ch03 smooth by means bin 3 -> (29,29,29)");
```

listing 14: every value of a bin replaced by the bin mean (source: kdd/samples/src/Ch03/smooth.c, lines 52-70)

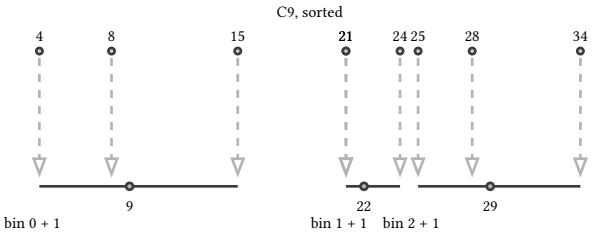


figure 12: each value pulled to its bin mean, 9, 22, 29

3.4. smoothing by boundaries

Boundary smoothing keeps the bin's own extremes instead of its center: each value snaps to the nearer of the bin's first and last values, so the output vocabulary is the input vocabulary. The contract sheet, kdd-contract-s1s2.md, the wave's fixture ledger, pins the tie rule: an equal distance to both boundaries takes the lower one, and values already sitting on a boundary stay put.

The dry run: bin (4, 8, 15). The value 8 sits 4 away from 4 and 7 away from 15, so it snaps down to 4, giving (4, 4, 15). Bin (21, 21, 24) already lives on its boundaries, 24 is its own upper bound, so it passes through unchanged as (21, 21, 24). Bin (25, 28, 34): 28 sits 3 from 25 and 6 from 34, so it snaps down to 25, giving (25, 25, 34). The sample prints all three lines and the concatenation check pins the full sequence (4, 4, 15, 21, 21, 24, 25, 25, 34) as "ch03 boundary-smoothed full sequence".

```c
/* nearest boundary; a tie (equal distance to both) takes the lower one */
static int nearest_boundary(int v, int lo, int hi) {
  if (v - lo ≤ hi - v) {
    return lo;
  }
  return hi;
}
```

listing 15: nearest boundary, a tie takes the lower one (source: kdd/samples/src/Ch03/smooth.c, lines 37-43)

```c
  {
    /* 8 sits 4 away from 4 and 7 away from 15 -> 4; 28 sits 3 from 25 and
     * 6 from 34 -> 25; the pinned sequence is the concatenation */
    const int want[9] = {4, 4, 15, 21, 21, 24, 25, 25, 34};
    bool ok = true;
    for (int i = 0; i < NB; i++) {
      for (int j = 0; j < PER; j++) {
        ok = ok && bsmooth[i][j] == want[i * PER + j];
      }
    }
    CHECK(ok,
          "ch03 boundary-smoothed full sequence (4,4,15,21,21,24,25,25,34)");
  }
```

listing 16: the full 9-value smoothed sequence as one check (source: kdd/samples/src/Ch03/smooth.c, lines 107-119)

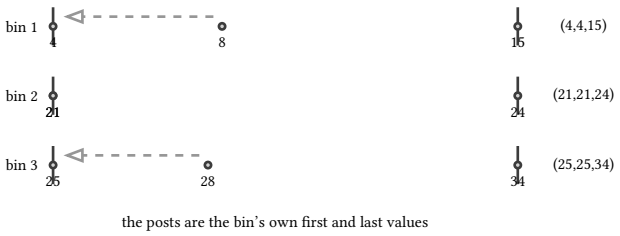


figure 13: each bin's values snap to the nearer of its two boundary posts

Two smoothers, two vocabularies. Means and medians invent values no record ever had, which is fine for scale noise and wrong for codes. Boundaries keep the vocabulary and move the counts, which is why discretized features downstream in book 14, ch 8 (knowledge discovery, feature selection and chi-square) prefer them when the values are grades, tiers, or labels.

sources: Mohammed J. Zaki and Wagner Meira Jr., Data Mining and Machine Learning: Fundamental Concepts and Algorithms, 2nd ed, ch 3 categorical attributes, the equal-depth binning walkthrough of the same 9-value series, cs.rpi.edu/ zaki/DMML/slides/pdf/ychap3.pdf, accessed 2026-09-22. All 30 pinned values witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s1.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch03`, 17 + 13 checks in chapter 03 of the kdd suite.

4. normalization and standardization

One sample carries the chapter: `znorm.c` runs three rescalings over three short series chosen so every result is dyadic, an integer over a power of two and therefore exact in binary floating point, and asserts 44 checks with exact == comparisons. The chapter makes 4 moves: min-max to [0,1] and to [−1,1], the z-score with the population divisor, the robust z built from median and MAD, and the symmetric series where the two z-scores agree element for element. Every behavioral claim below is one of the 44 checks of chapter 04′s sample. Scaling does not add information, it changes units, and book 14, ch 5 (knowledge discovery, distance metrics) consumes those units directly. Every row here is D4-eligible, the fixtures replay in go in book 14, ch 47 (knowledge discovery, the bridge: c to go).

4.1. min-max, two targets

Min-max scaling is the affine map that sends the observed minimum and maximum to the ends of a chosen interval. To [0,1] it is $(v - L)/r$ with $r = \max - \min$; to [−1,1] it is the image of that composed with $2u - 1$. The fixture is M = (2, 6, 8, 10, 14, 18), whose range 16 is a power of 2, so every division below is exact in binary.

The dry run: with L = 2 and r = 16 the images are (2 - 2)/16 = 0, 4/16 = 0.25, 6/16 = 0.375, 8/16 = 0.5, 12/16 = 0.75, 16/16 = 1, printed as `range 16 minmax01: 0 0.25 0.375 0.5 0.75 1`. The [−1,1] ruler applies 2u - 1 to each, printed as `minmax11: -1 -0.5 -0.25 0 0.5`
1. The map is strictly monotone, asserted by “ch04 minmax01 monotone 0..1”, and it is affine, so equal gaps stay equal gaps: the 2-unit gap between 6 and 8 is a 0.125 gap on both scaled rulers.

```
const double range = mx - mn;
double mm0[6];
for (int i = 0; i < 6; i++) {
  mm0[i] = (m[i] - mn) / range;
}
printf("range %.17g minmax01:", range);
for (int i = 0; i < 6; i++) {
  printf(" %.17g", mm0[i]);
}
printf("\n");
CHECK(range == 16.0, "ch04 M range 16");
CHECK(mm0[0] == 0.0 && mm0[1] == 0.25, "ch04 minmax01 (0, 0.25)");
CHECK(mm0[2] == 0.375, "ch04 minmax01 0.375");
CHECK(mm0[3] == 0.5, "ch04 minmax01 0.5");
CHECK(mm0[4] == 0.75, "ch04 minmax01 0.75");
CHECK(mm0[5] == 1.0, "ch04 minmax01 1");
CHECK(mm0[0] == 0.0 && mm0[5] == 1.0 &&
          (mm0[1] < mm0[2] && mm0[2] < mm0[3] && mm0[3] < mm0[4]),
      "ch04 minmax01 monotone 0..1");
```

listing 17: range, the 0..1 map, and the monotone assertion (source: kdd/samples/src/Ch04/znorm.c, lines 96-114)

```
/* row: min-max to [-1,1], formula 2v - 1 */
double mm1[6];
for (int i = 0; i < 6; i++) {
  mm1[i] = 2.0 * mm0[i] - 1.0;
}
printf("minmax11:");
for (int i = 0; i < 6; i++) {
  printf(" %.17g", mm1[i]);
}
printf("\n");
CHECK(mm1[0] == -1.0, "ch04 minmax11 -1");
CHECK(mm1[1] == -0.5, "ch04 minmax11 -0.5");
CHECK(mm1[2] == -0.25, "ch04 minmax11 -0.25");
CHECK(mm1[3] == 0.0, "ch04 minmax11 0");
CHECK(mm1[4] == 0.5, "ch04 minmax11 0.5");
CHECK(mm1[5] == 1.0, "ch04 minmax11 1");
```

listing 18: the −1..1 ruler as 2v - 1, checked elementwise (source: kdd/samples/src/Ch04/znorm.c, lines 116-131)

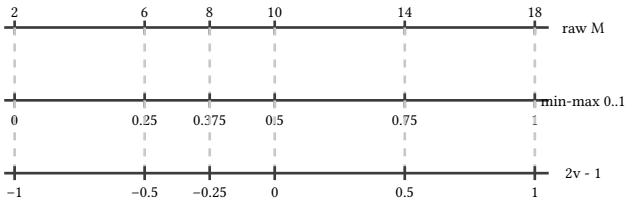


figure 14: three rulers, same spacing, the affine map at work

4.2. z-scores, the population way

The z-score standardizes by standard deviation instead of range: $z = (v - \mu)/s$, with μ the mean and s the root of the population variance, divisor n. The fixture is A = (4, 6, 6, 6, 10, 16), one modest outlier riding five tight values, and every statistic of it is an integer.

The dry run: the mean is 48/6 = 8. The deviations read (-4, −2, −2, −2, 2, 8), their squares sum 16 + 4 + 4 + 4 + 4 + 64 = 96, the population variance is 96/6 = 16, and $s = \sqrt{16} = 4$, exact because 16 is a perfect square. Dividing deviations by 4 gives z = (-1, −0.5, −0.5, −0.5, 0.5, 2), printed as `A mean 8 SS 96 var 16 s 4 z: -1 -0.5 -0.5 -0.5 0.5 2`. The divisor is pinned by the check name itself, “ch04 A population variance 16 (divisor n)”, and it is not the same convention as book 14, ch 2 (knowledge discovery, data cleaning and imputation), which divided by n - 1.

```c
/* row: z-score on A with POPULATION variance (divisor n = 6) */
const double a[6] = {4, 6, 6, 6, 10, 16};
const double amu = mean_of(a, 6);
const double ass = ss_of(a, 6, amu);
const double avar = ass / 6.0;
const double as = sqrt(avar); /* sqrt(16) is IEEE exact */
double az[6];
for (int i = 0; i < 6; i++) {
  az[i] = (a[i] - amu) / as;
}
printf("A mean %.17g SS %.17g var %.17g s %.17g z:", amu, ass, avar, as);
for (int i = 0; i < 6; i++) {
  printf(" %.17g", az[i]);
}
printf("\n");
CHECK(amu == 8.0, "ch04 A mean 8");
CHECK(ass == 96.0, "ch04 A sum of squares 96");
CHECK(avar == 16.0, "ch04 A population variance 16 (divisor n)");
CHECK(as == 4.0, "ch04 A s 4 (sqrt exact)");
CHECK(az[0] == -1.0, "ch04 z(4) -1");
CHECK(az[1] == -0.5, "ch04 z(6) -0.5 (first)");
CHECK(az[2] == -0.5, "ch04 z(6) -0.5 (second)");
CHECK(az[3] == -0.5, "ch04 z(6) -0.5 (third)");
CHECK(az[4] == 0.5, "ch04 z(10) 0.5");
CHECK(az[5] == 2.0, "ch04 z(16) 2");
```

listing 19: mean, sum of squares, population variance, and the six z values (source: kdd/samples/src/Ch04/znorm.c, lines 140-164)

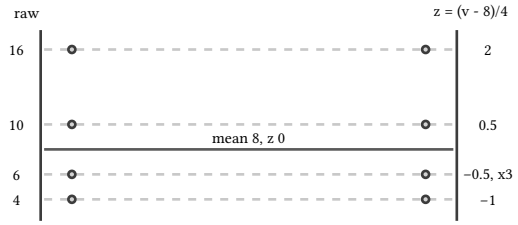


figure 15: the same six values on the raw ruler and the z ruler

4.3. the robust z, median and MAD

The classic z has a blind spot documented by Leys, Allen, Fisher, and Lumley in a paper whose title is the instruction: “Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median”. The robust z divides deviation from the median by the MAD, the median absolute deviation, and this chapter uses their raw definition, $(v - \text{median})/\text{MAD}$ with no 0.6745 consistency constant.

The dry run: sorted A reads (4, 6, 6, 6, 10, 16), even count, median (6
1. 6)/2 = 6. Absolute deviations from 6 are (2, 0, 0, 0, 4, 10), sorted

(0, 0, 0, 2, 4, 10), whose even middle averages to MAD = (0 + 2)/2 = 1. The robust z is then just the deviation from 6: (-2, 0, 0, 0, 4, 10), printed as `A median 6 MAD 1 robust z: -2 0 0 0 4 10`. The point 16 tells the story: on the classic ruler it sits 2 standard deviations out, on the robust ruler 10 deviations out, asserted together as “ch04 contrast classic z(16) = 2” and “ch04 contrast robust z(16) = 10”. The outlier inflated its own yardstick: the squared deviations of the single value 16 contribute 64 of the 96 sum of squares.

```c
/* row: MAD standardization (robust z) on A */
const double amed = median_of(a, 6);
const double amad = mad_of(a, 6, amed);
double arz[6];
for (int i = 0; i < 6; i++) {
  arz[i] = (a[i] - amed) / amad;
}
printf("A median %.17g MAD %.17g robust z:", amed, amad);
for (int i = 0; i < 6; i++) {
  printf(" %.17g", arz[i]);
}
printf("\n");
CHECK(amed == 6.0, "ch04 A median 6");
CHECK(amad == 1.0, "ch04 A MAD 1");
CHECK(arz[0] == -2.0, "ch04 robust z(4) -2");
CHECK(arz[1] == 0.0, "ch04 robust z(6) 0 (first)");
CHECK(arz[2] == 0.0, "ch04 robust z(6) 0 (second)");
CHECK(arz[3] == 0.0, "ch04 robust z(6) 0 (third)");
CHECK(arz[4] == 4.0, "ch04 robust z(10) 4");
CHECK(arz[5] == 10.0, "ch04 robust z(16) 10");

/* row: z vs robust z contrast at the point 16 */
CHECK(az[5] == 2.0, "ch04 contrast classic z(16) = 2");
CHECK(arz[5] == 10.0, "ch04 contrast robust z(16) = 10");
```

listing 20: median, MAD, robust z, and the two-ruler contrast at 16 (source: kdd/samples/src/Ch04/znorm.c, lines 166-189)

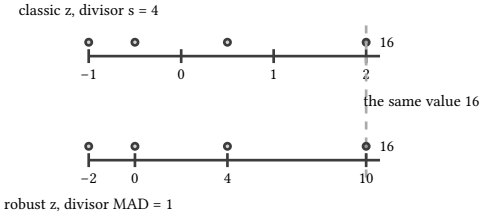


figure 16: value 16 measured on the classic ruler and the robust ruler

NOTE: which deviation constant is a per-book pin

Some texts scale MAD by 1/0.6745 so that it estimates the standard deviation under normality, Leys et al. discuss the choice and this book follows their raw ratio, pinned by the contract sheet as “robust z = (x - median)/MAD (Leys et al. 2013 definition, no 0.6745 scaling)”. The constant cancels in outlier ranking, all robust z values scale together, so the pin is a readability choice, not a modeling one.

4.4. agreement on symmetric data

The two standardizations disagree exactly when the data is lopsided, and they agree element for element on symmetric data. The fixture B = (3, 5, 7, 9, 11, 13, 15) is an arithmetic ladder, perfectly symmetric about its middle.

The dry run: the mean and the median both land on 9. The deviations (-6, −4, −2, 0, 2, 4, 6) have squares summing 36 + 16 + 4 + 0 + 4 + 16 + 36 = 112, population variance 112/7 = 16, so s = 4. The absolute deviations from 9 are (6, 4, 2, 0, 2, 4, 6), sorted (0, 2, 2, 4, 4, 6, 6), and with odd count the middle one is MAD = 4. Both divisors equal 4 and both centers equal 9, so z and robust z are the same list (-1.5, −1, −0.5, 0, 0.5, 1, 1.5), printed as `B mean 9 median 9 var 16 s 4 MAD 4` and asserted by seven checks, one per element, each named “ch04 B z=... equals robust z”.

```c
/* row: MAD matches z on symmetric B -- s = MAD = 4 forces z = robust z */
const double b[7] = {3, 5, 7, 9, 11, 13, 15};
const double bmu = mean_of(b, 7);
const double bmed = median_of(b, 7);
const double bvar = ss_of(b, 7, bmu) / 7.0;
const double bs = sqrt(bvar);
const double bmad = mad_of(b, 7, bmed);
printf("B mean %.17g median %.17g var %.17g s %.17g MAD %.17g\n", bmu, bmed,
       bvar, bs, bmad);
CHECK(bmu == 9.0 && bmed == 9.0, "ch04 B mean = median = 9");
CHECK(bvar == 16.0, "ch04 B population variance 16");
CHECK(bmad == 4.0, "ch04 B MAD 4");
const double bz_want[7] = {-1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5};
for (int i = 0; i < 7; i++) {
  const double bz = (b[i] - bmu) / bs;
  const double brz = (b[i] - bmed) / bmad;
  char name[64];
  snprintf(name, sizeof name, "ch04 B z=%.17g equals robust z", bz_want[i]);
  CHECK(bz == bz_want[i] && bz == brz, name);
}
```

listing 21: the symmetric ladder, both divisors 4, seven equality checks (source: kdd/samples/src/Ch04/znorm.c, lines 191-210)

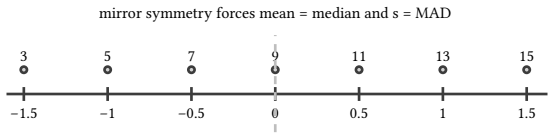


figure 17: the symmetric ladder, raw values above, z values below

Two standardizations, one lesson. Rescaling is a units decision with consequences downstream: book 14, ch 5 (knowledge discovery, distance metrics) computes raw gaps between whatever units it is handed, min-max stretches axes by range, the z family by spread, and the choice changes which pairs count as close. The fixtures of this chapter replay bit for bit in go in book 14, ch 47 (knowledge discovery, the bridge: c to go).

sources: Christophe Leys, Christy Ley, Olivier Klein, Philippe Bernard, Laurent Licata, “Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median”, Journal of Experimental Social Psychology 49(4), 764-766, 2013, banked by kdd-contract-s1s2.md, the same sheet that pins the no-0.6745 spelling used here. All 44 pinned values witnessed by the sheet and playground/kdd-matrix/gen_s1.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch04`, 44 checks in chapter 04 of the kdd suite.

5. distance metrics

One sample carries the chapter: `distance.c` computes the Minkowski family on one difference vector, exact euclidean distances on pythagorean pairs, and Mahalanobis distances under a diagonal covariance, 28 checks in all. The chapter makes 4 moves: the L_p family and its monotone shrink, exact roots and the integer witness discipline, Mahalanobis as distance with a metric, and the unit balls that picture the whole family. Every behavioral claim below is one of the 28 checks of chapter 05's sample or a pinned note of the contract sheet. Distances feed everything downstream: book 14, ch 20 (knowledge discovery, nearest neighbor) votes by them, book 14, ch 31 (knowledge discovery, k-means) minimizes them, and book 11, ch 10 (the icpc world finals, icpc world finals 2017) chains them. Every row here is D4-eligible, the fixtures replay in go in book 14, ch 47 (knowledge discovery, the bridge: c to go).

5.1. the minkowski family

The Minkowski distance of order p between vectors x and y is $d_p = \left(\sum_i |x_i - y_i|^p\right)^{1/p}$, one family containing the manhattan spelling at $p = 1$ and the euclidean at $p = 2$. The fixture is $x = (1, 1, 1)$ against $y = (4, 5, 6)$, difference $(3, 4, 5)$, chosen so every inner sum is a small integer.

The dry run: at $p = 1$ the distance is $3 + 4 + 5 = 12$, printed as `d1 = 12`. At $p = 2$ the squared distance is $9 + 16 + 25 = 50$, asserted as an integer, and the root prints `d2 = 7.0710678118654755`, the print-only row at tolerance 1e-12, exactly $5\sqrt{2}$. At $p = 3$ the cubes sum $27 + 64 + 125 = 216$, and $216 = 6^3$, so $d_3 = 6$ exactly, the sheet's hand note calling (3, 4, 5) the smallest integer triple whose coordinate cubes sum to a cube. At $p = 4$ the fourth powers sum $81 + 256 + 625 = 962$ and the root prints `d4 = 5.5692122278237566`. The four distances strictly decrease, printed as `monotone: 12 > 7.0710678118654755 > 6 > 5.5692122278237566` and asserted as "ch05 d1 > d2 > d3 > d4 strict".

```c
/* row: minkowski p=1, the manhattan spelling */
int d1 = 0;
for (int i = 0; i < 3; i++) {
  d1 += diff[i] < 0 ? -diff[i] : diff[i];
}
printf("d1 = %d\n", d1);
CHECK(d1 == 12, "ch05 minkowski p=1 distance 12");

/* row: minkowski p=2 -- squared sum 50 asserted as an integer, the root
 * is the print-only D2 row (irrational, 5*sqrt(2)) */
int p2sq = 0;
for (int i = 0; i < 3; i++) {
  p2sq += diff[i] * diff[i];
}
const double d2 = sqrt((double)p2sq);
printf("d2^2 = %d, d2 = %.17g (print-only, D2 1e-12)\n", p2sq, d2);
CHECK(p2sq == 50, "ch05 p=2 squared sum 50");
CHECK(fabs(d2 - 7.0710678118654755) <= 1e-12,
      "ch05 p=2 root 7.0710678118654755 within 1e-12");
CHECK(fabs(d2 - 5.0 * sqrt(2.0)) <= 1e-15, "ch05 p=2 root equals 5*sqrt(2)");
```

listing 22: p = 1 as a sum, p = 2 as an integer square plus a d2 root (source: kdd/samples/src/Ch05/distance.c, lines 48-67)

```
  /* row: minkowski p=4 -- 81+256+625 = 962, no integer root, print-only */
  int p4 = 0;
  for (int i = 0; i < 3; i++) {
    const int q = diff[i] * diff[i];
    p4 += q * q;
  }
  const double d4 = sqrt(sqrt((double)p4));
  printf("d4^4 = %d, d4 = %.17g (print-only, D2 1e-12)\n", p4, d4);
  CHECK(3 * 3 * 3 * 3 == 81 && 4 * 4 * 4 * 4 == 256 && 5 * 5 * 5 * 5 == 625,
        "ch05 p=4 per-coordinate fourth powers 81,256,625");
  CHECK(p4 == 962, "ch05 p=4 fourth-power sum 962");
  CHECK(fabs(d4 - 5.569212227823757) <= 1e-12,
        "ch05 p=4 root 5.569212227823757 within 1e-12");
```

listing 23: p = 4 fourth powers and its print-only root (source: kdd/samples/src/Ch05/distance.c, lines 82-94)

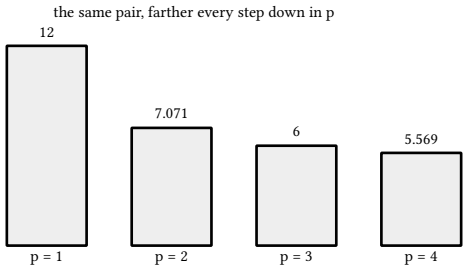


figure 18: four distances from one difference vector, shrinking in p

5.2. exact roots and integer witnesses

Two numerics rules do all the work in this sample, both pinned by the contract sheet. `sqrt` on a perfect square is IEEE exact, so the euclidean distances on pythagorean pairs assert with ==. The $p = 3$ and $p = 4$ roots are not bit-exact through libm `pow` or `cbrt`, so $p = 3$ asserts through an integer witness instead: a bounded search for the largest r with $r^3 \le n$, never touching the library.

The dry run: the pair (3, 4) has $3^2 + 4^2 = 25 = 5^2$, and `sqrt(25.0)` returns 5.0 exactly, checked as "ch05 euclidean(3,4) = 5 exact". The pair (5, 12) has $25 + 144 = 169 = 13^2$, `sqrt(169.0)` is exactly 13.0, printed on one line as `euclid(3,4) = 5 euclid(5,12) =`
13. The witness on 216 finds $r = 6$ because $6^3 = 216$ and $7^3 = 343 > 216$, printed as `d3^3 = 216, integer cube root = 6` and asserted three ways: sum 216, root 6 unique, and $6^3 = 216$ as the closing witness.

```
/* largest r with r^3 ≤ n, by bounded integer search: never touches cbrt */
static int icbrt(int n) {
  int r = 0;
  while ((r + 1) * (r + 1) * (r + 1) ≤ n) {
    r++;
  }
  return r;
}
```

listing 24: integer cube root by bounded search, libm untouched (source: kdd/samples/src/Ch05/distance.c, lines 35-42)

```
/* row: minkowski p=3 -- 27+64+125 = 216 = 6^3, the smallest integer
 * triple whose coordinate cubes sum to a cube; the root is asserted
 * through the integer witness only */
int p3 = 0;
for (int i = 0; i < 3; i++) {
  p3 += diff[i] * diff[i] * diff[i];
}
const int r3 = icbrt(p3);
printf("d3^3 = %d, integer cube root = %d\n", p3, r3);
CHECK(p3 == 216, "ch05 p=3 cube sum 216");
CHECK(r3 == 6 && 7 * 7 * 7 > 216, "ch05 p=3 integer root 6 unique");
CHECK(r3 * r3 * r3 == p3, "ch05 p=3 witness 6^3 == 216");
```

listing 25: the p = 3 row through its integer witness (source: kdd/samples/src/Ch05/distance.c, lines 69-80)

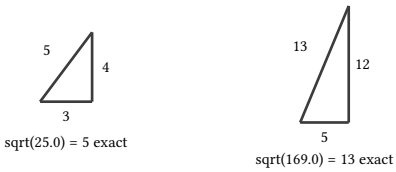


figure 19: two pythagorean pairs, both roots exact

5.3. mahalanobis, distance with a metric

Plain euclidean distance treats every axis alike, which is wrong when the axes carry different spread. The Mahalanobis distance divides each direction by its covariance, $D^2 = (p-\mu)^T S^{-1}(p-\mu)$, so a step along a noisy axis costs less than the same step along a quiet one. The fixture is 5 points on 2 features, $f_1 = (-2,-1,0,1,2)$ and $f_2 = (2,-4,0,4,-2)$, with population covariance, divisor n as book 14, ch 4 (knowledge discovery, normalization and standardization) pinned it.

The dry run: both features have mean 0. The variance of f_1 is $(4+1+0+1+4)/5 = 2$, of f_2 is $(4+16+0+16+4)/5 = 8$, and the cross term $-4+4+0+4-4 = 0$, printed as `var f1 2 var f2 8 cross 0`. The covariance is diagonal, so $D^2 = p_1^2/2 + p_2^2/8$. The point (3, 6) then sits at $9/2 + 36/8 = 4.5 + 4.5 = 9$, distance 3, printed as `pt(3,6): D^2 9 D 3`. The other four points land at $D = 0, 1, 2, 2, 3$, every D^2 an integer and every D the exact square root of one.

```
/* fixture 3: 5 points, 2 features; population covariance (divisor n = 5)
 * is diagonal: var 2 and 8 with zero cross term, so the inverse
 * covariance is diag(1/2, 1/8) and D^2 = p1^2/2 + p2^2/8 */
const int f1[5] = {-2, -1, 0, 1, 2};
const int f2[5] = {2, -4, 0, 4, -2};
double s1 = 0, s2 = 0, sc = 0;
for (int i = 0; i < 5; i++) {
  s1 += (double)f1[i] * f1[i];
  s2 += (double)f2[i] * f2[i];
  sc += (double)f1[i] * f2[i];
}
const double v1 = s1 / 5.0, v2 = s2 / 5.0, c12 = sc / 5.0;
printf("var f1 %.17g var f2 %.17g cross %.17g\n", v1, v2, c12);
CHECK(v1 == 2.0, "ch05 population var(f1) = 2");
CHECK(v2 == 8.0, "ch05 population var(f2) = 8");
CHECK(c12 == 0.0, "ch05 cov(f1,f2) = 0 (diagonal)");
```

listing 26: the 2-feature fixture and its diagonal population covariance (source: kdd/samples/src/Ch05/distance.c, lines 111-126)

```
/* row: mahalanobis distances from the mean (0,0): (0,0),(1,2),(2,4),
 * (2,-4),(3,6) -> D^2 0,1,4,4,9 (integers) and D 0,1,2,2,3 (sqrt of
 * perfect squares, exact). hand check of (3,6): 9/2 + 36/8 = 4.5+4.5 = 9 */
const double pts[5][2] = {{0, 0}, {1, 2}, {2, 4}, {2, -4}, {3, 6}};
const double want_d2[5] = {0, 1, 4, 4, 9};
const double want_d[5] = {0, 1, 2, 2, 3};
for (int i = 0; i < 5; i++) {
  const double dq = pts[i][0] * pts[i][0] / v1 + pts[i][1] * pts[i][1] / v2;
  const double d = sqrt(dq);
  char name1[64], name2[64];
  snprintf(name1, sizeof name1, "ch05 mahalanobis D^2 pt%d = %.17g", i,
           want_d2[i]);
  snprintf(name2, sizeof name2, "ch05 mahalanobis D pt%d = %.17g", i,
           want_d[i]);
  printf("pt(%g,%g): D^2 %.17g D %.17g\n", pts[i][0], pts[i][1], dq, d);
  CHECK(dq == want_d2[i], name1);
  CHECK(d == want_d[i], name2);
}
```

listing 27: five points, five squared distances, five exact roots (source: kdd/samples/src/Ch05/distance.c, lines 128-145)

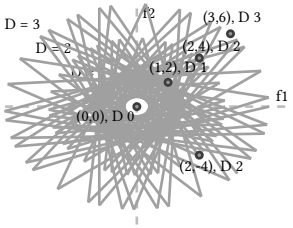


figure 20: iso-distance contours, each point pinned to its own contour

NOTE: the f2 axis is stretched, and that is the point

The contour of $D = 1$ is an ellipse, 2 units tall in f_1 and 8 wide in variance terms, so in the plot the f_2 axis reaches $\sqrt{8}$ while f_1 reaches $\sqrt{2}$. A point 4 units out on f_2 and a point 2 units out on f_1 are the same distance from the center, both $D = 2$, which is exactly what the two planted points (2, 4) and (2, −4) assert.

5.4. the shrinking ball, and the name

As p grows the distance shrinks toward the largest single coordinate gap, the monotonicity asserted at the top of the chapter, and the unit ball $\sum_i |x_i|^p = 1$ swells from the $p = 1$ diamond to the $p = 2$ circle toward the axis-aligned square. The balls are the picture to keep: which ball a miner uses decides which pairs it calls close, the diamond forgives nothing on any axis, the square forgives everything but the worst axis.

The dry run: the monotone line prints `monotone: 12 > 7.0710678118654755 > 6 > 5.5692122278237566` over the one difference vector (3, 4, 5), the same numbers the bars of the first figure drew.

```
/* row: lp monotonicity, 12 > 7.071... > 6 > 5.569... */
const double d3 = (double)r3;
printf("monotone: %d > %.17g > %.17g > %.17g\n", d1, d2, d3, d4);
CHECK((double)d1 > d2 && d2 > d3 && d3 > d4, "ch05 d1 > d2 > d3 > d4 strict");
```

listing 28: strict monotonicity of the family, one check (source: kdd/samples/src/Ch05/distance.c, lines 96-99)

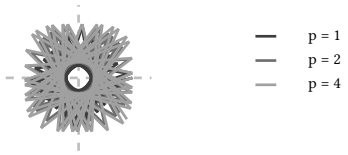


figure 21: unit balls for p = 1, 2, 4, all at distance 1 from the center

NOTE: minkowski distance is not the minkowski sum

This chapter owns the Minkowski distance, the L_p metric $\sqrt[p]{\sum_i |x_i - y_i|^p}$ of this sample. The Minkowski sum of convex geometry is a different object entirely, the set $A + B = \{a + b : a \in A, b \in B\}$ built by sliding one shape around the boundary of another, and it measures no distance at all. The sum has its own chapter in book 10, ch 37 (practical data structures and algorithms, computational geometry ii), and the two share only Hermann Minkowski's name.

sources: all 28 pinned values and both numerics notes (sqrt exactness on perfect squares, the integer cube-root witness discipline) witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s1.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch05`, 28 checks in chapter 05 of the kdd suite.

6. similarity measures

One sample carries the chapter: `similarity.c` scores three set pairs by Jaccard, two vector pairs by cosine, three string pairs by unit-cost edit distance, and two perfectly linear ladders by Pearson r, 23 checks in all. The chapter makes 4 moves: set overlap as a reduced fraction, vector alignment with exactly one sqrt call, string distance in its mining spelling, and correlation as cosine on centered data. Every behavioral claim below is one of the 23 checks of chapter 06′s sample or a cited alignment from Jurafsky and Martin. book 14, ch 5 (knowledge discovery, distance metrics) measured how far apart two things sit; this chapter turns the question around and measures how much they share, the quantity book 14, ch 31 (knowledge discovery, k-means) averages into centroids and book 14, ch 8 (knowledge discovery, feature selection and chi-square) scores features by before keeping any of them.

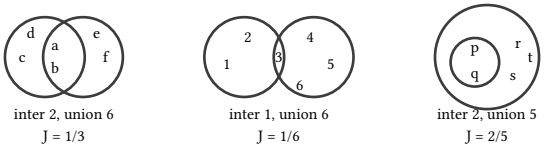
6.1. sets: intersection over union

The Jaccard similarity of two sets is the size of their intersection over the size of their union, $J(A, B) = |A \cap B| \ / \ |A \cup B|$, a fraction in $[0, 1]$ that is 0 for disjoint sets and 1 for identical ones. It needs no order, no magnitudes, and no dimensions, which is why it is the default similarity for baskets, tags, and any record reduced to a set of items. The fixture is three set pairs chosen to exercise three regimes: heavy overlap, a single shared item, and containment.

The dry run: pair 1 is $\{a, b, c, d\}$ against $\{a, b, e, f\}$, sharing a and b, so inter 2 and union $4 + 4 - 2 = 6$, printed as `pair1 {abcd} vs {abef}: inter 2 union 6 → 1/3`. Pair 2 is $\{1, 2, 3\}$ against $\{3, 4, 5, 6\}$, sharing only 3, inter 1 union 6, reduced to 1/6. Pair 3 is $\{p, q, r, s, t\}$ against $\{p, q\}$, containment, inter 2 union 5, and 2/5 is already reduced. Six checks pin the three integer pairs and the three reduced fractions, the reduction done by the same `gcd_i` loop that chapter 02 used, and every value stays an integer until the fraction is named.

```
/* row: jaccard over 3 set pairs, inter/union as integers plus the
 * reduced fraction pair */
const char *js[3][2] = {{"abcd", "abef"}, {"123", "3456"}, {"pqrst", "pq"}};
const int want_iu[3][2] = {{2, 6}, {1, 6}, {2, 5}};
const int want_frac[3][2] = {{1, 3}, {1, 6}, {2, 5}};
for (int k = 0; k < 3; k++) {
  const int inter = inter_size(js[k][0], js[k][1]);
  const int la = (int)strlen(js[k][0]), lb = (int)strlen(js[k][1]);
  const int union_ = la + lb - inter;
  const int g = gcd_i(inter, union_);
  char n1[64], n2[64];
  snprintf(n1, sizeof n1, "ch06 jaccard pair%d inter/union = (%d,%d)", k + 1,
           want_iu[k][0], want_iu[k][1]);
  snprintf(n2, sizeof n2, "ch06 jaccard pair%d reduced = %d/%d", k + 1,
           want_frac[k][0], want_frac[k][1]);
  printf("pair%d {%s} vs {%s}: inter %d union %d -> %d/%d\n", k + 1, js[k][0],
         js[k][1], inter, union_, inter / g, union_ / g);
  CHECK(inter == want_iu[k][0] && union_ == want_iu[k][1], n1);
  CHECK(inter / g == want_frac[k][0] && union_ / g == want_frac[k][1], n2);
}
```

listing 29: three set pairs, intersection counted, union by inclusion-exclusion (source: kdd/samples/src/Ch06/similarity.c, lines 128-147)



one shared vocabulary per panel, letters are the only data

figure 22: three set pairs, the shared letters sit in the lens of each panel

6.2. vectors: one sqrt, not two

The cosine similarity of two vectors is their dot product over their lengths, $\cos(u, v) = (u \cdot v)/(|u|\ |v|)$, the cosine of the angle between them. Unlike a distance it ignores magnitude entirely: scaling u by 1000 changes nothing, only direction counts, which is the right attitude for documents represented by word counts and users by item ratings. The fixture is two pairs, both built so the norms are perfect squares and the quotient is exact.

The dry run: pair 1 is u = (1, 1, 0) against v = (1, 0, 1). The dot product is 1, both squared norms are 2, and $1/\sqrt{2 \cdot 2} = 1/2$, printed as `cos1: dot 1 |u|^2 2 |v|^2 2 cos 0.5` and asserted by the exact comparison "ch06 cosine1 = 1/sqrt(4) = 0.5 exact". Pair 2 is u = (2, 1) against v = (1, 2): dot $2 \cdot 1 + 1 \cdot 2 = 4$, squared norms 5 apiece, and $\sqrt{5 \cdot 5} = 5$ exactly, so cos = 4/5 = 0.8. The printed line reads `cos2: dot 4 |u|^2 5 |v|^2 5 cos 0.8000000000000004`, seventeen significant digits of the same double the check compares == against the literal 0.8, both spellings the nearest double to 4/5.

```c
/* cosine with the pinned single-sqrt spelling */
static double cosine(const int *u, const int *v, int n, int *dot, int *su,
                     int *sv) {
  int d = 0, pu = 0, pv = 0;
  for (int i = 0; i < n; i++) {
    d += u[i] * v[i];
    pu += u[i] * u[i];
    pv += v[i] * v[i];
  }
  *dot = d;
  *su = pu;
  *sv = pv;
  return (double)d / sqrt((double)pu * (double)pv);
}
```

listing 30: cosine with the pinned single-sqrt spelling (source: kdd/samples/src/Ch06/similarity.c, lines 60-73)

PITFALL: the formula pin is load-bearing, not style

The contract sheet pins the spelling `dot / sqrt(su * sv)`, one sqrt call over an exact integer product. The double-sqrt spelling `dot / (sqrt(su) * sqrt(sv))` is algebraically identical and numerically different: on pair 1, sqrt(2) is irrational, both copies round, their product lands just above 2, and the quotient becomes 0.4999999999999999, which fails the exact `c == 0.5`. With one sqrt the radicand 4 is a perfect square, sqrt is IEEE exact, and the division is correctly rounded. The same pin holds for Pearson below, where sqrt(400.0) = 20 exactly. Determinism here is a property of the written formula, and the checks freeze it.

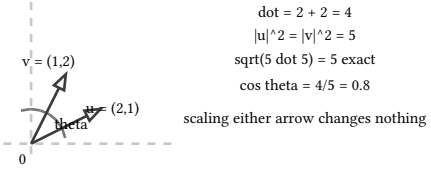


figure 23: pair 2 in the plane, equal lengths, cos of the angle 0.8

6.3. strings: five edits, one alignment

Edit distance counts the cheapest single-character operations that turn one string into the other. This chapter's pin is unit-cost Levenshtein, insert 1, delete 1, substitute 1, no transposition, and the sample keeps only the mining framing: strings are attributes too, product codes and names and addresses, and record linkage scores their pairs exactly this way. The full dynamic programming treatment, recurrence proofs and space-rolling and eight languages of implementation, belongs to book 10, ch 17 (practical data structures and algorithms, dynamic programming) here the three canonical pairs are the payload.

The dry run: kitten to sitting takes 3, substitute k to s, substitute e to i, insert g. flaw to lawn takes 2, delete f and insert n, and 2 is optimal because the longest common subsequence law has length 3, and $4 + 4 - 2 \cdot 3 = 2$ is exactly the edit distance when only insertions and deletions pay. intention to execution takes 5, the Jurafsky and Martin alignment: i against nothing, n to e, t to x, e against e, n to c, nothing against u, then t, i, o, n all matching, one delete, three substitutions, one insert. The sample prints `edit: kitten→sitting 3, flaw→lawn 2, intention→execution 5` and asserts each distance as its own check.

```c
/* unit-cost levenshtein: ins 1, del 1, sub 1, NO transposition */
static int lev(const char *a, const char *b) {
  const int m = (int)strlen(a), n = (int)strlen(b);
  int prev[64], cur[64];
  for (int j = 0; j <= n; j++) {
    prev[j] = j;
  }
  for (int i = 1; i <= m; i++) {
    cur[0] = i;
    for (int j = 1; j <= n; j++) {
      const int del = prev[j] + 1;
      const int ins = cur[j - 1] + 1;
      const int sub = prev[j - 1] + (a[i - 1] == b[j - 1] ? 0 : 1);
      int best = del;
      if (ins < best) {
        best = ins;
      }
      if (sub < best) {
        best = sub;
      }
      cur[j] = best;
    }
    for (int j = 0; j <= n; j++) {
      prev[j] = cur[j];
    }
  }
  return prev[n];
}
```

listing 31: two rolling rows, the classic space O(min(m, n)) spelling (source: kdd/samples/src/Ch06/similarity.c, lines 75-102)

```c
  /* row: edit distance, 3 pairs */
  CHECK(lev("kitten", "sitting") == 3, "ch06 edit kitten/sitting = 3");
  CHECK(lev("flaw", "lawn") == 2, "ch06 edit flaw/lawn = 2");
  CHECK(lev("intention", "execution") == 5,
        "ch06 edit intention/execution = 5");
  printf("edit: kitten→sitting %d, flaw→lawn %d, intention→execution %d\n",
         lev("kitten", "sitting"), lev("flaw", "lawn"),
         lev("intention", "execution"));
```

listing 32: the three pairs and their checks (source: kdd/samples/src/Ch06/similarity.c, lines 174-181)

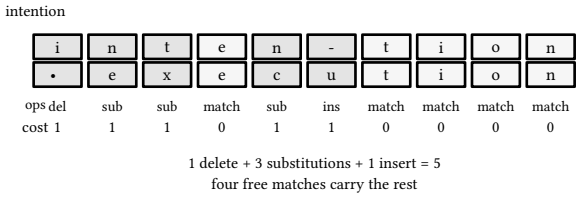


figure 24: the intention to execution alignment, five paid cells

6.4. correlation: cosine after centering

The Pearson correlation coefficient of two equal-length series is $r = S_{xy}/\sqrt{S_{xx}S_{yy}}$ with $S_{xy} = \sum(x_i - |x|)(y_i - |y|)$, and it is exactly the cosine similarity of the two mean-centered series. That is the whole connection between this facet and the last one: Pearson is cosine with the means subtracted first, so pure offset changes nothing and only linear co-movement counts.

The dry run: over x = 1..5 and y = 2x, the x deviations from mean 3 are (-2, −1, 0, 1, 2), the y deviations are doubled, and $S_{xy} = 8 + 2 + 0 + 2 + 8 = 20$, $S_{xx} = 10$, $S_{yy} = 4 \cdot 10 = 40$. One sqrt: sqrt(400.0) = 20 exactly, so r = 20/20 = 1, printed as `pearson+: Sxy 20 Sxx 10 Syy 40 r 1` and asserted by “ch06 pearson+ r = 1.0 exact”. Flipping y to 12 - 2x flips every y deviation, so Sxy becomes −20 and r becomes −1, printed as `pearson-: Sxy -20 r -1`. Both fixtures are perfect lines, and r only ever spans $[-1, 1]$, the two endpoints pinned by two checks.

```c
/* pearson r with the pinned single-sqrt spelling */
static double pearson(const int *x, const int *y, int n, double *sxy,
                      double *sxx, double *syy) {
  double mx = 0, my = 0;
  for (int i = 0; i < n; i++) {
    mx += x[i];
    my += y[i];
  }
  mx /= n;
  my /= n;
  double xy = 0, xx = 0, yy = 0;
  for (int i = 0; i < n; i++) {
    const double dx = x[i] - mx, dy = y[i] - my;
    xy += dx * dy;
    xx += dx * dx;
    yy += dy * dy;
  }
  *sxy = xy;
  *sxx = xx;
  *syy = yy;
  return xy / sqrt(xx * yy);
}
```

listing 33: pearson r, deviations first, the same single-sqrt pin (source: kdd/samples/src/Ch06/similarity.c, lines 104-125)

```c
/* row: pearson on perfect linear y = 2x: deviations of x from 3 are
 * (-2,-1,0,1,2), y deviations doubled, so Sxy = 8+2+0+2+8 = 20,
 * Sxx = 10, Syy = 4*10 = 40, r = 20/sqrt(400) = 20/20 = 1 */
{
  const int x[5] = {1, 2, 3, 4, 5}, y[5] = {2, 4, 6, 8, 10};
  double sxy, sxx, syy;
  const double r = pearson(x, y, 5, &sxy, &sxx, &syy);
  printf("pearson+: Sxy %.17g Sxx %.17g Syy %.17g r %.17g\n", sxy, sxx, syy,
         r);
  CHECK(sxy == 20.0, "ch06 pearson+ Sxy 20");
  CHECK(sxx == 10.0, "ch06 pearson+ Sxx 10");
  CHECK(syy == 40.0, "ch06 pearson+ Syy 40");
  CHECK(r == 1.0 && sqrt(400.0) == 20.0, "ch06 pearson+ r = 1.0 exact");
}
```

listing 34: the perfect line, Sxy 20, Sxx 10, Syy 40, r exactly 1 (source: kdd/samples/src/Ch06/similarity.c, lines 183-196)

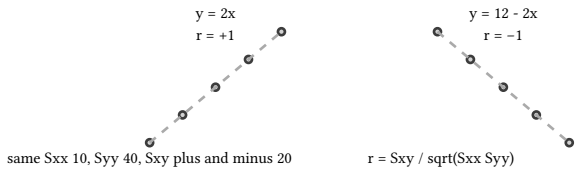


figure 25: two perfect lines, one rising, one falling, r at both endpoints

Four similarity families, one discipline. Sets, vectors, strings, and series each got a score that lives in a known range, is computed from integers or exact dyadic squares, and is pinned by an exact check. book 14, ch 8 (knowledge discovery, feature selection and chi-square) sharpens the question from how similar two values are to how much a whole column matters, and chi-square answers it with the same integer-first arithmetic.

sources: Daniel Jurafsky and James H. Martin, Speech and Language Processing, 3rd ed. draft, ch 2, minimum edit distance, the intention/execution alignment, web.stanford.edu/ jurafsky/slp3/, banked by kdd-contract-s1s2.md. All 23 pinned values, both formula pins (the single-sqrt cosine and pearson spellings), and the flaw/lawn LCS optimality note witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s2.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch06`, 23 checks in chapter 06 of the kdd suite.

7. sampling

One sample carries the chapter: `sampling.c` builds a deterministic sampling toolkit on the Park-Miller MINSTD generator, draws a simple random sample without replacement with a full recorded trace, allocates a stratified sample by Hamilton's largest remainder, and walks a progressive sampling schedule to its stop rule, 29 checks in all. The chapter makes 4 moves: the engine and why its arithmetic must be 64-bit, the partial Fisher-Yates draw traced to the last swap, the leftover-seat allocation, and the gain-based stop rule. Every behavioral claim below is one of the 29 checks of chapter 07's sample or the banked Park and Miller citation. book 14, ch 1 (knowledge discovery, the knowledge discovery process) quoted Fayyad et al. naming subsampling as part of the loop itself; this chapter is that stage, and book 14, ch 24 (knowledge discovery, class imbalance) and book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) both consume its output later.

7.1. the engine: park-miller minstd

A linear congruential generator is one line of arithmetic, $s' = (16807s) \bmod (2^{31} - 1)$, and the pin is the MINSTD generator of Park and Miller's 1988 CACM paper, multiplier 16807, modulus $2^{31} - 1 = 2147483647$ which is prime, seed 1. The single trap is width: the intermediate product $16807 \cdot (2^{31} - 1)$ is about 3.6×10^{13}, far past 2^{32}, so the multiply must run in `uint64_t` or it silently wraps and every downstream number in the chapter turns to a different, wrong, but equally confident stream.

The dry run: from seed 1 the states read 16807, 282475249, 1622650073, 984943658, 1144108930, printed as `state1 = 16807` through `state5 = 1144108930`, each one asserted as its own check. Every value is one multiply and one remainder, recomputable by hand, which is the property the rest of the chapter leans on: a sampler whose stream anyone can audit with pencil arithmetic.

```c
#define MINSTD_M 2147483647ULL

static unsigned long long minstd_next(unsigned long long s) {
  return (16807ULL * s) % MINSTD_M;
}
```

listing 35: the minstd recurrence, uint64 intermediates or it wraps (source: kdd/samples/src/Ch07/sampling.c, lines 36-40)

```c
  /* row: LCG state stream from seed 1, 5 draws (D0 values, D3 pin) */
  const unsigned long long want_state[5] = {
      16807ULL, 282475249ULL, 1622650073ULL, 984943658ULL, 1144108930ULL};
  unsigned long long s = 1;
  for (int i = 0; i < 5; i++) {
    s = minstd_next(s);
    char name[64];
    snprintf(name, sizeof name, "ch07 minstd state%d = %llu", i + 1,
             want_state[i]);
    printf("state%d = %llu\n", i + 1, s);
    CHECK(s == want_state[i], name);
  }
```

listing 36: five states from seed 1, each pinned (source: kdd/samples/src/Ch07/sampling.c, lines 64-75)

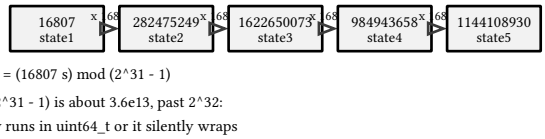


figure 26: the state chain from seed 1, one multiply and one mod per hop

7.2. five of twelve, fully traced

A simple random sample without replacement is k draws from n items where no item repeats and every subset of size k is equally likely. The sample implements it as a partial Fisher-Yates: keep the ids 0..11 in an array, at step i pick $j = i + (s_i \bmod (12 - i))$ from the pool still untouched, swap positions i and j, and emit the value now sitting at position i. Exactly k = 5 steps run, the tail of the array is never touched, and the trace of every (i, j, state) is recorded as it happens.

The dry run: the hand remainders drive everything. 16807 mod 12 = 7, so j = 7 and the swap (0, 7) emits 7. 282475249 mod 11 = 1, j = 2, swap (1, 2) emits 2. 1622650073 mod 10 = 3, j = 5, swap (2, 5) emits 5. For the fourth state the digit sum of 984943658 is 56, and casting out nines leaves 2, so 984943658 mod 9 = 2, j = 5, and the swap (3, 5) emits the 1 that earlier swaps had parked at position 5. 1144108930 mod 8 = 2, j = 6, swap (4, 6) emits 6. The printed trace reads `trace i=0 j=7 state=16807` through `trace i=4 j=6 state=1144108930`, the draws print as `draw1 = 7` through `draw5 = 6`, and the closing lines read `sorted = 1 2 5 6 7` and the distinctness check, all five different. The D3 check runs the entire procedure a second time and asserts every draw, every j, every state matches, “ch07 srs double-run identical (D3)”.

```
static void srs(int n, int k, unsigned long long seed, int *draw, int *js,
                unsigned long long *states) {
  int a[64];
  for (int i = 0; i < n; i++) {
    a[i] = i;
  }
  unsigned long long s = seed;
  for (int i = 0; i < k; i++) {
    s = minstd_next(s);
    states[i] = s;
    const int j = i + (int)(s % (unsigned long long)(n - i));
    js[i] = j;
    const int t = a[i];
    a[i] = a[j];
    a[j] = t;
    draw[i] = a[i];
  }
}
```

listing 37: partial Fisher-Yates, j from the shrinking pool, trace recorded (source: kdd/samples/src/Ch07/sampling.c, lines 44-61)

```
  for (int i = 0; i < 5; i++) {
    char name[96];
    snprintf(name, sizeof name, "ch07 srs trace i=%d: j=%d state=%llu", i,
             want_j[i], want_state[i]);
    printf("trace i=%d j=%d state=%llu\n", i, js[i], states[i]);
    CHECK(js[i] == want_j[i] && states[i] == want_state[i], name);
  }
```

listing 38: the five trace rows, each asserted (source: kdd/samples/src/Ch07/sampling.c, lines 139-145)

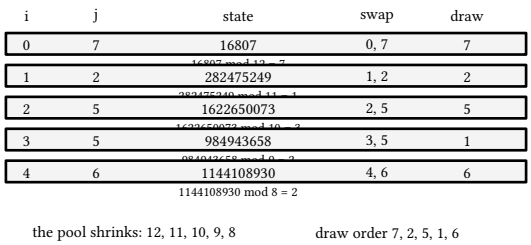


figure 27: the trace ladder, one row per draw, arithmetic on the left, outcome on the right

NOTE: determinism is what makes the sample auditable

A sampling chapter has no business printing a number nobody can reproduce. The LCG stream is D0 integer arithmetic, the draw is recomputed from it in exact integers, and the sample closes the loop with a double-run assertion, "ch07 srs double-run identical (D3)", that runs the whole procedure a second time and compares every draw, every j, and every state. The pinned trace table is therefore not a snapshot of one lucky run, it is a theorem about this binary.

7.3. stratified: hamilton's leftover seat

When classes differ wildly in size, a simple random sample underrepresents the small ones by construction. Stratified sampling splits n by class weight, and the sample implements Hamilton's largest remainder method: take each class's exact quota, keep the floors, and hand the leftover seats to the classes with the largest fractional remainders. The fixture is 3 classes of counts A 47, B 31, C 22 over a population of 100, with n = 10.

The dry run: the exact quotas are 4.7, 3.1, and 2.2 seats. The floors are (4, 3, 2), which hands out 9 of the 10 seats, and the remainders over 100 are (70, 10, 20), printed as `floors 4 3 2 rems 70 10 20 left 1`. One leftover seat remains and A's remainder 70 is the largest, so A takes it, printed as `alloc A=5 B=3 C=2 sum 10` and asserted by "ch07 stratified allocation (5,3,2)" plus the sum check against n = 10. The tie rule is pinned too, equal remainders resolve by class name order, inert on this fixture because the remainders are distinct, but living in the comparator where any real allocation would exercise it.

```
int floors[3], rems[3], left = n_s;
for (int i = 0; i < 3; i++) {
  floors[i] = counts[i] * n_s / total;
  rems[i] = counts[i] * n_s % total;
  left -= floors[i];
}
printf("floors %d %d %d rems %d %d %d left %d\n", floors[0], floors[1],
       floors[2], rems[0], rems[1], rems[2], left);
CHECK(floors[0] == 4 && floors[1] == 3 && floors[2] == 2,
      "ch07 stratified floors (4,3,2)");
CHECK(rems[0] == 70 && rems[1] == 10 && rems[2] == 20,
      "ch07 stratified remainders (70,10,20)");
CHECK(left == 1, "ch07 stratified 1 leftover seat");
```

listing 39: quotas, floors, remainders, and the count of leftover seats (source: kdd/samples/src/Ch07/sampling.c, lines 155-167)

```
int alloc[3];
for (int i = 0; i < 3; i++) {
  alloc[i] = floors[i];
}
for (int i = 0; i < left; i++) {
  alloc[order[i]]++;
}
int sum = alloc[0] + alloc[1] + alloc[2];
printf("alloc %s=%d %s=%d %s=%d sum %d\n", cls[0], alloc[0], cls[1],
       alloc[1], cls[2], alloc[2], sum);
CHECK(order[0] == 0, "ch07 stratified leftover goes to A (rem 70)");
CHECK(alloc[0] == 5 && alloc[1] == 3 && alloc[2] == 2,
      "ch07 stratified allocation (5,3,2)");
CHECK(sum == 10, "ch07 stratified allocation sums to n = 10");
```

listing 40: leftover seats to the largest remainders, final (5,3,2) (source: kdd/samples/src/Ch07/sampling.c, lines 182-195)

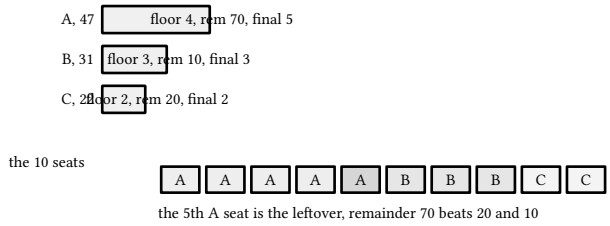


figure 28: the 10 seats: floors first, the leftover seat lands on A

7.4. progressive: stop when the gain dies

Progressive sampling grows the sample and watches the model's accuracy, paying for data only while accuracy still climbs. The schedule is sizes (25, 50, 100, 200, 400) with pinned accuracies (5/8, 11/16, 3/4, 25/32) for the first four stops, every one a dyadic fraction so the doubles subtract exactly. The stop rule is the whole point: stop the first time the gain is at or under 1/32.

The dry run: the gains print as `gain1 = 0.0625`, `gain2 = 0.0625`, `gain3 = 0.03125`, that is 1/16, 1/16, 1/32. The third gain equals the threshold, the rule fires, and the summary line reads `stop_idx 2 evals 4 chosen 200`: four evaluations happened, sizes 25 through 200, and the chosen sample size is 200. Size 400 is never evaluated, never paid for, and the checks "ch07 progressive stops after 4 evaluations" and "ch07 progressive chosen size 200" pin both facts. The gains halving in lockstep is the fixture's geometry, real curves flatten irregularly, but the rule reads the same either way.

```
const int sizes[5] = {25, 50, 100, 200, 400};
const double acc[4] = {0.625, 0.6875, 0.75, 0.78125};
const double want_gain[3] = {0.0625, 0.0625, 0.03125};
const double threshold = 0.03125; /* 1/32 */
double gains[3];
for (int i = 0; i < 3; i++) {
  gains[i] = acc[i + 1] - acc[i];
  char name[80];
  snprintf(name, sizeof name, "ch07 progressive gain%d = %.17g", i + 1,
           want_gain[i]);
  printf("gain%d = %.17g\n", i + 1, gains[i]);
  CHECK(gains[i] == want_gain[i], name);
}
```

listing 41: dyadic accuracies, gains as exact doubles (source: kdd/samples/src/Ch07/sampling.c, lines 203-215)

```
int stop_idx = -1;
for (int i = 0; i < 3; i++) {
  if (gains[i] ≤ threshold) {
    stop_idx = i;
    break;
  }
}
const int evals = stop_idx + 2;
const int chosen = sizes[stop_idx + 1];
printf("stop_idx %d evals %d chosen %d\n", stop_idx, evals, chosen);
CHECK(evals == 4, "ch07 progressive stops after 4 evaluations");
CHECK(chosen == 200, "ch07 progressive chosen size 200");
```

listing 42: first gain at or under 1/32 stops, chosen size 200 (source: kdd/samples/src/Ch07/sampling.c, lines 216-227)

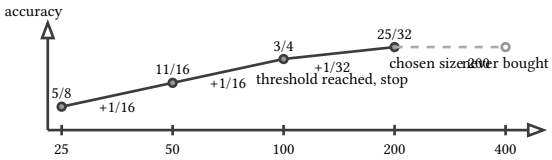


figure 29: the accuracy staircase, the third gain hits the threshold and 400 is never bought

Four samplers, one thread. Every draw traces back to one multiply and one remainder over a prime modulus, every allocation to integer division with a named tie rule, every stop to a comparison against a threshold that is itself a dyadic double. book 14, ch 8 (knowledge discovery, feature selection and chi-square) keeps columns instead of rows, and book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) eventually asks what a sample can claim about the population it came from.

sources: Stephen K. Park and Keith W. Miller, “Random number generators: good ones are hard to find”, Communications of the ACM 31(10), 1192-1201, 1988, the MINSTD generator, banked by kdd-contract-s1s2.md. All 29 pinned values, the uint64-intermediate note, the mod-arithmetic hand checks, and the stratified tie-break pin witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s2.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch07`, 29 checks in chapter 07 of the kdd suite.

8. feature selection and chi-square

Two samples carry the chapter: `subset.c` scores all 15 non-empty feature subsets under a redundancy penalty and searches them best-first, 20 checks, and `chisq.c` runs the chi-square test of independence on a 2x2 and a 2x3 table in exact integer arithmetic, 19 checks. The chapter makes 4 moves: a subset score that subtracts redundancy, the best-first search and its stop rule, chi-square on the 2x2 with the Yates contrast, and the 2x3 that stays under its threshold. Every behavioral claim below is one of the 39 checks of chapter 08's samples or the banked NIST critical values. book 14, ch 3 (knowledge discovery, binning, smoothing, and discretization) built the categorical shelves, this chapter decides which columns earn their keep, and book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) owns the testing machinery these tables are a first taste of.

8.1. a score that hates redundancy

Filter selection needs a number per subset, and the fixture fixes one: four features with weights w = (A 4, B 4, C 2, D 1), two redundant pairs (A, B) and (C, D), and the score $S = \sum w - 2$ per enclosed redundant pair. The penalty is the entire lesson. {D} alone is worth 1, and {C, D} is worth $2 + 1 - 2 = 1$ as well, the pair adds a feature and gains nothing. The redundancy pair says the second member of a pair carries no information the first did not already bring.

The dry run: all 15 non-empty subsets print their scores, `score({A}) = 4` through `score({ABCD}) = 7`, and each is its own check, 15 in a row. The singles read 4, 4, 2, 1. The pairs read $\{A, B\} = 6$, $\{A, C\} = 6$, $\{B, C\} = 6$, $\{A, D\} = 5$, $\{B, D\} = 5$, and $\{C, D\} = 1$, the last one the penalty's showcase. The triples read $\{A, B, C\} = 8$, $\{A, B, D\} = 7$, $\{A, C, D\} = 5$, $\{B, C, D\} = 5$, and the full set $\{A, B, C, D\} = 11 - 4 = 7$ pays both penalties. The best subset is $\{A, B, C\}$ at 8, asserted by "ch08 score({ABC}) = 8": it keeps both heavy features, swallows one penalty for the (A, B) pair, and refuses D, whose only contribution would be a second penalty.

```c
static const int W[4] = {4, 4, 2, 1};            /* A B C D */
static const int PAIRS[2][2] = {{0, 1}, {2, 3}}; /* (A,B), (C,D) */
static const int PEN = 2;

static int score_of(int mask) {
  int s = 0;
  for (int i = 0; i < 4; i++) {
    if (mask & (1 << i)) {
      s += W[i];
    }
  }
  for (int p = 0; p < 2; p++) {
    if ((mask & (1 << PAIRS[p][0])) && (mask & (1 << PAIRS[p][1]))) {
      s -= PEN;
    }
  }
  return s;
}
```

listing 43: weights, redundant pairs, penalty 2, and the score function (source: kdd/samples/src/Ch08/subset.c, lines 34-51)

```
/* row: all 15 non-empty subset scores */
const int want[16] = {0, 4, 4, 6, 2, 6, 6, 8, 1, 5, 5, 7, 1, 5, 5, 7};
for (int mask = 1; mask < 16; mask++) {
  char sub[5], name[48];
  mask_to_str(mask, sub);
  snprintf(name, sizeof name, "ch08 score({%s}) = %d", sub, want[mask]);
  printf("score({%s}) = %d\n", sub, score_of(mask));
  CHECK(score_of(mask) == want[mask], name);
}
```

listing 44: all 15 non-empty subsets scored, one check each (source: kdd/samples/src/Ch08/subset.c, lines 91-99)

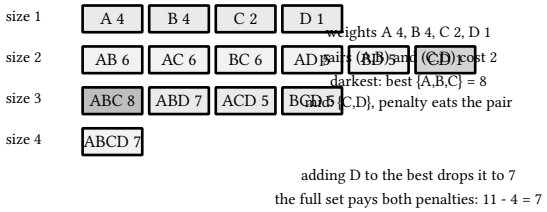


figure 30: the 15 subset scores by size, the winner keeps three features and one penalty

8.2. best-first, and when to stop

Scoring every subset is exponential; search prunes it. The sample runs best-first over an open list ordered by score first, lexicographic subset order on ties, pushing every single-feature extension of each expanded node, and stops when the best open entry can no longer beat the best subset already found. The tie rule is pinned because equal scores are everywhere in this fixture, and lexicographic order is what breaks them: $\{A\}$ pops before $\{B\}$ at score 4, and $\{A, B\}$ pops before $\{A, C\}$ at score 6.

The dry run: the expansion trail prints `expand (,0)`, `expand (A,4)`, `expand (AB,6)`, `expand (ABC,8)`, four expansions in that exact order, asserted whole as "ch08 best-first expansions (empty,A,AB,ABC) = (0,4,6,8)". Expanding $\{A, B, C\}$ pushes $\{A, B, C, D\}$ at 7, the best score already seen is 8, and no open entry can top it, so the loop breaks. The selection prints `selected {ABC} score 8`, the stop prints `open at stop: 7 entries, {AC} = 6, max {ABD} = 7`, and the checks pin both the generator's literal assertion, the open node $\{A, C\}$ at 6 below the best 8, and the stronger one, "ch08 stop rule: every open entry < best 8". The true open maximum is 7, not 6, and the header of the sample says so plainly: the pinned 6 is a specific node, the every-entry check is what actually certifies the stop.

```
  while (n_open > 0) {
    int m = 0; /* pop the minimum by (score desc, subset lex asc) */
    for (int i = 1; i < n_open; i++) {
      if (node_cmp(open_score[i], open_sub[i], open_score[m], open_sub[m]) <
          0) {
        m = i;
      }
    }
    const int s = open_score[m];
    char sub[5];
    copy_sub(sub, open_sub[m]);
    open_score[m] = open_score[n_open - 1];
    copy_sub(open_sub[m], open_sub[n_open - 1]);
    n_open--;

    const int mask = str_to_mask(sub);
    if (seen[mask]) {
      continue;
    }
    seen[mask] = 1;
    exp_mask[n_exp] = mask;
    exp_score[n_exp] = s;
    n_exp++;
```

listing 45: pop the open maximum, record the expansion, push extensions (source: kdd/samples/src/Ch08/subset.c, lines 115-137)

```
{
  /* the generator's pinned stop-rule assertion: the open node {A,C}
   * carries score 6 < 8; also print the true open maximum for the
   * record ({A,B,C,D} at 7, see header note) */
  int ac = -1, mx = -1, mxi = 0;
  for (int i = 0; i < n_stop; i++) {
    if (strcmp(stop_sub[i], "AC") == 0) {
      ac = stop_score[i];
    }
    if (stop_score[i] > mx) {
      mx = stop_score[i];
      mxi = i;
    }
  }
  printf("open at stop: %d entries, {AC} = %d, max {%s} = %d\n", n_stop, ac,
         stop_sub[mxi], mx);
  CHECK(ac == 6 && 6 < best_score,
        "ch08 stop rule: open node {A,C} at 6 < best 8");
  CHECK(mx < best_score, "ch08 stop rule: every open entry < best 8");
```

listing 46: the stop rule: one pinned node, then every open entry (source: kdd/samples/src/Ch08/subset.c, lines 189-207)

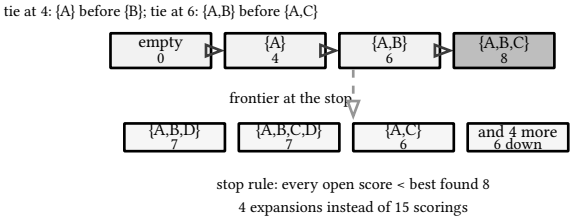


figure 31: four expansions down the spine, the frontier never tops 8

8.3. the 2x2, and yates

Chi-square tests independence: compare every cell's observed count O against its expected count E = row total times column total over n, and sum $(O-E)^2/E$. A large statistic says the rows and columns are entangled, the feature and the class travel together, which is exactly what a selected feature should do. The sample computes each cell term as an exact rational, $(100O-rc)^2/(100rc)$, so integers carry the whole computation and the comparison against the decimal critical value is a cross-multiplication.

The dry run: the table is [[20, 30], [30, 20]] with n = 100, both row totals 50 and both column totals 50. Every expected cell is $50 \cdot 50/100 = 25$, every deviation is 5, and every term is $25/25 = 1$, so chi2 = 4 exactly, printed as `2x2: E all 25, terms all 1, chi2 = 4 =
4/1`. Against the NIST critical value 3.841 at alpha 0.05 with one degree of freedom, 4 rejects independence, asserted without a single floating-point comparison as `acc_n * 1000 > 3841 * acc_d`. The Yates continuity correction subtracts 0.5 from each absolute deviation first: every corrected term becomes $(5-0.5)^2/25 = 4.5^2/25 = 0.81$, the total is $4 \cdot 0.81 = 3.24$, printed as `yates: term
0.81000000000000005 total 3.2400000000000002 (= 81/25)`, and 3.24 falls short of 3.841: on this table the correction flips the verdict.

```
/* exact rational term (O - E)^2/E with E = rt*ct/n, n = 100:
 * (100*O - rt*ct)^2 / (100*rt*ct), returned reduced */
static void cell_term(int o, int rt, int ct, long long *num, long long *den) {
  const long long d = 100LL * o - (long long)rt * ct;
  *num = d * d;
  *den = 100LL * rt * ct;
  const long long g = gcd_ll(*num, *den);
  *num /= g;
  *den /= g;
}
```

listing 47: each cell term as an exact rational, gcd-reduced (source: kdd/samples/src/Ch08/chisq.c, lines 45-54)

```
    for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
        e[i][j] = (double)rt[i] * ct[j] / n;
        long long tn, td;
        cell_term(t[i][j], rt[i], ct[j], &tn, &td);
        terms[i][j] = (double)tn / (double)td;
        const long long g = gcd_ll(acc_d, td);
        acc_n = acc_n * (td / g) + tn * (acc_d / g);
        acc_d = acc_d / g * td;
      }
    }
    const long long g = gcd_ll(acc_n, acc_d);
    acc_n /= g;
    acc_d /= g;
    const double chi = (double)acc_n / (double)acc_d;
    printf("2x2: E all %.17g, terms all %.17g, chi2 = %.17g = %lld/%lld\n",
           e[0][0], terms[0][0], chi, acc_n, acc_d);
    CHECK(rt[0] == 50 && rt[1] == 50 && ct[0] == 50 && ct[1] == 50 && n == 100,
          "ch08 2x2 row/col totals 50 and n 100");
    CHECK(e[0][0] == 25.0 && e[0][1] == 25.0 && e[1][0] == 25.0 &&
              e[1][1] == 25.0,
          "ch08 2x2 every expected cell 25");
    CHECK(terms[0][0] == 1.0 && terms[0][1] == 1.0 && terms[1][0] == 1.0 &&
              terms[1][1] == 1.0,
          "ch08 2x2 every cell term 1");
    CHECK(chi == 4.0, "ch08 2x2 chi2 = 4.0 exactly");
    CHECK(acc_n * 1000 > 3841 * acc_d, "ch08 2x2 reject: 4 > 3.841 (df 1)");
```

listing 48: expected 25 per cell, chi2 exactly 4, reject by cross-multiply (source: kdd/samples/src/Ch08/chisq.c, lines 63-89)

```
    /* row: Yates correction contrast. every |O-E| = 5, so every
     * corrected term is (5-0.5)^2/25 = 4.5^2/25 and the total is
     * 4 * 0.81 = 3.24 = 81/25 exactly */
    const double yterm = (5.0 - 0.5) * (5.0 - 0.5) / 25.0;
    const double yates = 4.0 * yterm;
    printf("yates: term %.17g total %.17g (= 81/25)\n", yterm, yates);
    CHECK(yates == 3.24, "ch08 yates chi2 = 3.24");
    CHECK(yates < 3.841, "ch08 yates would not reject at df 1");
```

listing 49: yates: 4.5 squared over 25, times 4, no longer rejects (source: kdd/samples/src/Ch08/chisq.c, lines 91-98)

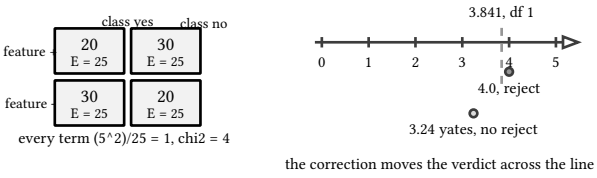


figure 32: the 2x2 table and the two verdicts against the df 1 threshold

NOTE: rationals against decimal tables

The critical values come from a printed table as decimals, 3.841 and 5.991, but the statistic itself is the exact rational the integer loop accumulated. The checks compare the two worlds by cross-multiplication, `acc_n * 1000 > 3841 * acc_d`, so no rounded double ever sits between the statistic and its threshold. The same discipline makes the Yates total assertable with ==: one term is computed and then multiplied by 4, and rounding commutes with multiplication by a power of two.

8.4. a 2x3 that stays under

One degree of freedom is the most forgiving chi-square there is, so the second fixture widens the table. Two rows, three columns, [[15, 10, 15], [15, 20, 25]], row totals (40, 60), column totals (30, 30, 40), n = 100, and two degrees of freedom, threshold 5.991.

The dry run: the expected counts print as `2x3: expected 12 12 16 18 18 24`, row 1 getting $40 \cdot 30/100, 40 \cdot 30/100, 40 \cdot 40/100$ and row 2 getting the 60-side versions. The six cell terms are the fractions 3/4, 1/3, 1/16, 1/2, 2/9, 1/24, printed as `term(0,0) = 3/4, x144 = 108` through `term(1,2) = 1/24, x144 = 6`: scaled by the common denominator 144 the terms become 108, 48, 9, 72, 32, 6, integers summing to 275, printed as `scaled sum 275, chi2 = 275/144 = 1.9097222222222223`. And 275/144 is nowhere near 5.991: the sample fails to reject independence, "ch08 2x3 fail to reject: 275/144 < 5.991 (df 2)". The feature and the class are compatible with independence on this table, and the integer pair (275, 144) is the D0 witness, the 1e-9 double only a display.

```
    /* per-cell x144 scaled terms in exact integer arithmetic */
    long long acc_n = 0, acc_d = 1, scaled_sum = 0;
    k = 0;
    for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 3; j++) {
        long long num, den;
        cell_term(t[i][j], rt[i], ct[j], &num, &den);
        /* term * 144: num*144/den must divide exactly */
        const long long s = num * 144;
        char name[64];
        snprintf(name, sizeof name, "ch08 2x3 term(%d,%d) x144 = %lld", i, j,
                 want_scaled[k]);
        printf("term(%d,%d) = %lld/%lld, x144 = %lld\n", i, j, num, den,
               s / den);
        CHECK(s % den == 0 && s / den == want_scaled[k], name);
        scaled_sum += s / den;
        const long long g = gcd_ll(acc_d, den);
        acc_n = acc_n * (den / g) + num * (acc_d / g);
        acc_d = acc_d / g * den;
        k++;
      }
```

listing 50: the six terms scaled by 144, exact integers, one check each (source: kdd/samples/src/Ch08/chisq.c, lines 127-147)

```
    }
    const long long g = gcd_ll(acc_n, acc_d);
    acc_n /= g;
    acc_d /= g;
    const double chi = (double)acc_n / (double)acc_d;
    printf("scaled sum %lld, chi2 = %lld/%lld = %.17g\n", scaled_sum, acc_n,
           acc_d, chi);
    CHECK(scaled_sum == 275, "ch08 2x3 scaled terms sum 275");
    CHECK(acc_n == 275 && acc_d == 144, "ch08 2x3 chi2 = 275/144 (D0 pair)");
    CHECK(fabs(chi - 1.9097222222222223) <= 1e-9,
          "ch08 2x3 chi2 double 1.9097222222222223 at 1e-9");
    CHECK(acc_n * 1000 < 5991 * acc_d,
          "ch08 2x3 fail to reject: 275/144 < 5.991 (df 2)");
}
```

listing 51: 275/144 as the D0 pair, the double at 1e-9, fail to reject (source: kdd/samples/src/Ch08/chisq.c, lines 148-161)

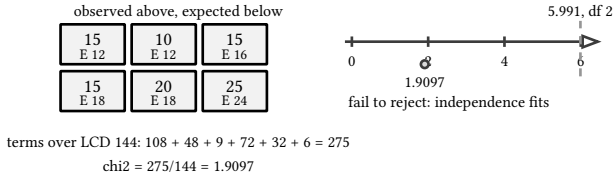


figure 33: observed against expected in six cells, the statistic far under the df 2 line

Two selection lenses, one habit. The subset score asks which columns work together and charges for saying the same thing twice; chi-square asks a single column how far its class distribution drifts from independence and prices the drift against a tabled threshold. book 14, ch 29 (knowledge discovery, interestingness, multilevel, and quantitative rules) reuses the same drift logic to score whole patterns, and book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) builds the general test machinery the tabled values abbreviate.

sources: NIST/SEMATECH e-Handbook of Statistical Methods, tabled upper-tail chi-square critical values at alpha 0.05, df 1 3.841 and df 2 5.991, itl.nist.gov/div898/handbook, accessed 2026-09-22, banked by kdd-contract-s1s2.md. All 39 pinned values, the x144 scaling, the (275, 144) pair, and the cross-multiplication comparisons witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s2.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch08`, 20 + 19 checks in chapter 08 of the kdd suite.

9. principal component analysis

One sample carries the chapter: `pca.c` builds the sample covariance of 5 centered points, recognizes its eigenvalues by hand, diagonalizes it with a cyclic 3x3 Jacobi eigensolver, certifies the factorization with the handshake $A = QDQ^T$, and projects the points onto the first component, 25 checks in all. The chapter makes 4 moves: the covariance and its divisor, the eigenvalues known before the solver runs, the Jacobi rotations and the handshake that makes them trustworthy, and the projection invariance that ties scores to eigenvalues. Every behavioral claim below is one of the 25 checks of chapter 09's sample or a pinned note of the contract sheet. The prose authority for covariance eigendecomposition is book 2, ch 8 (mathematics for programmers, matrix decompositions) this chapter owns the runnable solver and its gates. book 14, ch 10 (knowledge discovery, the haar wavelet transform) reduces by discarding detail instead of rotating axes, and the two close the transformation half of the book's first arc.

9.1. five points, one covariance

The fixture is 5 points on 3 features, integers, already centered: P = (3,0,0), (0,3,0), (0,0,3), (-2,-2,-2), (-1,-1,-1). Centering is the quiet precondition, each coordinate sums to 0, printed as `means (0, 0, 0)` and checked, which is why the scatter matrix, the plain sum of outer products, is also the covariance up to one division. The divisor is the sample one, n - 1 = 4, pinned per chapter: book 14, ch 5 (knowledge discovery, distance metrics) divided by n for its Mahalanobis fixture, and this chapter divides by n

- 1, the same per-row convention the sheet has enforced since

book 14, ch 2 (knowledge discovery, data cleaning and imputation).

The dry run: the scatter diagonal is $9 + 0 + 0 + 4 + 1 = 14$ per coordinate and every off-diagonal is $0 + 0 + 0 + 4 + 1 = 5$, asserted as "ch09 scatter diagonal 14,14,14" and three single-entry checks. Dividing by 4, the covariance is diag 3.5, off-diagonal 1.25, all dyadic, asserted with ==. The last structural check is the symmetry precondition, covariance equals its transpose, a D0 fact the solver is allowed to assume but the sample verifies anyway.

```
double means[3] = {0, 0, 0};
int scatter[3][3];
double cov[3][3];
for (int i = 0; i < 5; i++) {
  for (int j = 0; j < 3; j++) {
    means[j] += P[i][j];
  }
}
for (int j = 0; j < 3; j++) {
  means[j] /= 5.0;
}
for (int i = 0; i < 3; i++) {
  for (int j = 0; j < 3; j++) {
    int s = 0;
    for (int k = 0; k < 5; k++) {
      s += P[k][i] * P[k][j];
    }
    scatter[i][j] = s;
    cov[i][j] = (double)s / 4.0;
  }
}
```

listing 52: means, scatter as integer outer-product sums, covariance over 4 (source: kdd/samples/src/Ch09/pca.c, lines 138-158)

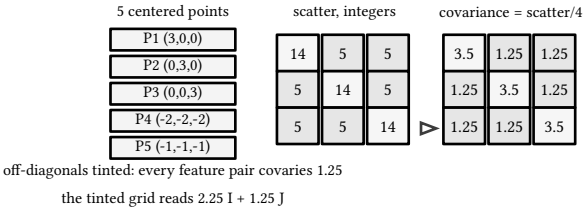


figure 34: the 5 points, their scatter, and their covariance over n minus 1

9.2. the eigenvalues, before the solver

Nothing about this covariance needs an eigensolver. It reads $2.25I + 1.25J$ with J the all-ones matrix: diagonal $2.25 + 1.25 = 3.5$, off-diagonal 1.25. J carries the vector (1,1,1) to 3 times itself and annihilates the plane $x + y + z = 0$, so $aI + bJ$ has eigenvalue $a + 3b$ on (1,1,1) and a twice on the plane. Here that is $2.25 + 3(1.25) = 6$ and 2.25, 2.25. The full theory behind that one sentence is book 2, ch 8 (mathematics for programmers, matrix decompositions) the sample's job is to witness it in arithmetic.

The dry run: with $a = 3.5 - 1.25 = 2.25$, printed as `a = 2.25`, `lambda1 = 6`, every row of the covariance sums to $3.5 + 1.25 + 1.25 = 6$, printed as `row sums (6, 6, 6)`, which is the eigenvector equation for (1,1,1) read off row by row. A plane vector, (1,-1,0), multiplies to (2.25, −2.25, 0), printed as `C*(1,-1,0) = (2.25, -2.25, 0)`, exactly 2.25 times itself. The second fixture is the stock Jacobi shape $M = [[3,2,2],[2,3,2],[2,2,3]] = 1I + 2J$: row sums all 7, plane eigenvalue 1, printed as `M row sums all 7, M*(1,-1,0) = (1, -1, 0)`, the same family one integer step over.

```c
/* row: eigenvalues. cov = aI + bJ with a = 3.5 - 1.25 = 2.25 and
 * b = 1.25: lambda1 = a + 3b = 6 on (1,1,1), lambda = a - b = 2.25
 * on the plane. */
{
  char name[80];
  const double a = cov[0][0] - cov[0][1];
  const double l1 = a + 3.0 * 1.25;
  snprintf(name, sizeof name, "ch09 lambda1 = 2.25 + 3*1.25 = %.17g", 6.0);
  printf("a = %.17g, lambda1 = %.17g\n", a, l1);
  CHECK(a == 2.25 && l1 == 6.0, name);
}
{
  double rs[3];
  int same = 1;
  for (int i = 0; i < 3; i++) {
    rs[i] = cov[i][0] + cov[i][1] + cov[i][2];
    same = same && rs[i] == 6.0;
  }
  printf("row sums (%.17g, %.17g, %.17g)\n", rs[0], rs[1], rs[2]);
  CHECK(same, "ch09 row sums all 6 confirm (1,1,1) eigenvector");
}
```

listing 53: a = diag - off-diagonal, lambda1 = a + 3b, row sums witness it (source: kdd/samples/src/Ch09/pca.c, lines 175-195)

```c
{
  /* C * (1,-1,0) = (3.5-1.25, 1.25-3.5, 0) = (2.25, -2.25, 0) */
  double mv[3];
  const double v[3] = {1, -1, 0};
  for (int i = 0; i < 3; i++) {
    mv[i] = cov[i][0] * v[0] + cov[i][1] * v[1] + cov[i][2] * v[2];
  }
  printf("C*(1,-1,0) = (%.17g, %.17g, %.17g)\n", mv[0], mv[1], mv[2]);
  CHECK(mv[0] == 2.25 && mv[1] == -2.25 && mv[2] == 0.0,
        "ch09 plane vector (1,-1,0) eigen at 2.25");
}
```

listing 54: a plane vector multiplied, eigenvalue 2.25 exact (source: kdd/samples/src/Ch09/pca.c, lines 196-206)

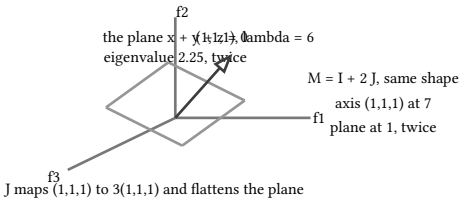


figure 35: one axis at 6, a whole plane at 2.25, the same for M at 7 and 1

9.3. the jacobi solver, and the handshake

Jacobi's method diagonalizes a symmetric matrix by planar rotations. One step picks an off-diagonal pair (p, q), computes the angle $\theta = 0.5 \operatorname{atan2}(-2a_{pq}, a_{pp} - a_{qq})$ that cancels it, and applies the rotation to columns, rows, and the accumulating Q. A sweep is the cyclic pass over pairs (0,1), (0,2), (1,2), a step fires only when $|a_{pq}| > 10^{-12}$, and the loop stops when a whole sweep fires nothing.

The dry run: on the covariance the printed summary is `jacobi cov: sweeps 2 rotations 2`, and on M it is `jacobi M: sweeps 2 rotations 2`, both D3 pins, identical across runs. Both rotations land in the first sweep, and the second sweep is the empty confirming pass that trips the stop. The eigenvalues print as `cov eigen sorted (6, 2.25, 2.2499999999999996)` and `M eigen sorted (7.0000000000000018, 1.0000000000000002, 0.99999999999999978)`: a few parts in 10^{16} off, which is why the eigenvalue pins carry 1e-9 tolerances instead of ==. The gate that matters is the handshake, $A = QDQ^T$ recomputed entry by entry: `handshake residuals: cov 8.8817841970012523e-16, M 1.3322676295501878e-15`, both asserted under 1e-12. The generator measured 4.44e-16 and 1.78e-15 on its own run, the same order, and the assertion is the bound, not the digits.

```
    for (int p = 0; p < 3; p++) {
      for (int r = p + 1; r < 3; r++) {
        if (fabs(a[p][r]) > 1e-12) {
          const double theta = 0.5 * atan2(-2.0 * a[p][r], a[p][p] - a[r][r]);
          const double c = cos(theta), sn = sin(theta);
          for (int k = 0; k < 3; k++) { /* columns p, r */
            const double akp = a[k][p], akr = a[k][r];
            a[k][p] = c * akp - sn * akr;
            a[k][r] = sn * akp + c * akr;
          }
          for (int k = 0; k < 3; k++) { /* rows p, r */
            const double apk = a[p][k], ark = a[r][k];
            a[p][k] = c * apk - sn * ark;
            a[r][k] = sn * apk + c * ark;
          }
          for (int k = 0; k < 3; k++) { /* Q accumulates the rotation */
            const double qkp = q[k][p], qkr = q[k][r];
            q[k][p] = c * qkp - sn * qkr;
            q[k][r] = sn * qkp + c * qkr;
          }
          rotated++;
        }
      }
```

listing 55: one rotation: theta from atan2, columns, rows, and Q all turned (source: kdd/samples/src/Ch09/pca.c, lines 57-79)

```
/* eigen handshake: Q * diag * Q^T must reproduce the input; returns the
 * max absolute residual */
static double handshake(const double in[3][3], const double d[3][3],
                        const double q[3][3]) {
  const double lam[3] = {d[0][0], d[1][1], d[2][2]};
  double resid = 0.0;
  for (int k = 0; k < 3; k++) {
    for (int j = 0; j < 3; j++) {
      double rec = 0.0;
      for (int i = 0; i < 3; i++) {
        rec += q[k][i] * lam[i] * q[j][i];
      }
      const double e = fabs(rec - in[k][j]);
      if (e > resid) {
        resid = e;
      }
    }
  }
  return resid;
}
```

listing 56: the handshake, Q D Q^T rebuilt and compared entrywise (source: kdd/samples/src/Ch09/pca.c, lines 96-115)

```
double dcov[3][3], qcov[3][3], dm[3][3], qm[3][3];
int sw_cov, rot_cov, sw_m, rot_m;
jacobi3(cov, dcov, qcov, &sw_cov, &rot_cov);
jacobi3(STOCK, dm, qm, &sw_m, &rot_m);
printf("jacobi cov: sweeps %d rotations %d\n", sw_cov, rot_cov);
printf("jacobi M:   sweeps %d rotations %d\n", sw_m, rot_m);
CHECK(sw_cov == 2, "ch09 jacobi covariance sweeps 2");
CHECK(rot_cov == 2, "ch09 jacobi covariance rotations 2");
CHECK(sw_m == 2, "ch09 jacobi stock M sweeps 2");
CHECK(rot_m == 2, "ch09 jacobi stock M rotations 2");

{
  double lam[3];
  eig_sorted(dcov, lam);
  printf("cov eigen sorted (%.17g, %.17g, %.17g)\n", lam[0], lam[1], lam[2]);
  CHECK(fabs(lam[0] - 6.0) ≤ 1e-9 && fabs(lam[1] - 2.25) ≤ 1e-9 &&
            fabs(lam[2] - 2.25) ≤ 1e-9,
        "ch09 cov eigen (6, 2.25, 2.25) sorted desc at 1e-9");
```

listing 57: both matrices solved, sweeps, rotations, eigenvalues pinned (source: kdd/samples/src/Ch09/pca.c, lines 210-227)

```
{
  const double resid = handshake(cov, dcov, qcov);
  const double resid_m = handshake(STOCK, dm, qm);
  printf("handshake residuals: cov %.17g, M %.17g\n", resid, resid_m);
  CHECK(resid < 1e-12, "ch09 cov handshake A = Q diag Q^T < 1e-12");
  CHECK(resid_m < 1e-12, "ch09 M handshake A = Q diag Q^T < 1e-12");
}
```

listing 58: the residual gate, under 1e-12 on both matrices (source: kdd/samples/src/Ch09/pca.c, lines 229-235)

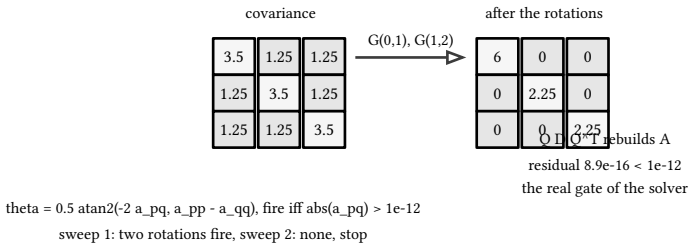


figure 36: one rotation kills an off-diagonal pair, the sweep repeats until nothing fires

PITFALL: never pin Q's entries

The eigenvalue 2.25 is repeated, so the two plane columns of Q are implementation-defined: any orthonormal basis of the plane diagonalizes the matrix equally well, and a different atan2, a different pair order, even a different compiler's sine, can rotate them into different vectors with the same handshake. The sample pins eigenvalues, sweep count, rotation count, and the residual bound, never Q itself. Repeated eigenvalues are the rule in mining data, not the exception, and this fixture makes the trap visible on purpose.

9.4. projection, and the variance that comes back

The first principal component is the eigenvector of the largest eigenvalue, here the unnormalized loading (1,1,1). Projecting each point p onto it is one dot product, $s = p \cdot (1,1,1)$, and the chapter's closing fact is the invariance: the variance of the projections onto a unit eigenvector equals the eigenvalue.

The dry run: the raw scores print as `PC1 raw scores (3, 3, 3, -6, -3)`, three axis points at 3, the two negative octant points at −6 and −3. Divided by the loading's length $\sqrt{3}$, the unit scores squared print as `unit scores squared (3,`

3, 3, 12, 3), that is $9/3, 9/3, 9/3, 36/3, 9/3$. The unit-score variance is $24/4 = 6$, exactly λ_1, printed as `var raw 18 (= 3*lambda1 18), var unit 6 (= lambda1 6)`, and the raw-score variance is 18, three times the eigenvalue because the loading's squared length is 3. Component scores inherit their scale from the loading, and pinning both variances shows the bookkeeping explicitly.

```
{
  double s[5], sq[5], sum_sq = 0.0;
  const int want_s[5] = {3, 3, 3, -6, -3};
  int same = 1;
  for (int i = 0; i < 5; i++) {
    s[i] = (double)(P[i][0] + P[i][1] + P[i][2]);
    same = same && s[i] == (double)want_s[i];
    sq[i] = s[i] * s[i] / 3.0;
    sum_sq += s[i] * s[i];
  }
  printf("PC1 raw scores (%.17g, %.17g, %.17g, %.17g, %.17g)\n", s[0], s[1],
         s[2], s[3], s[4]);
  printf("unit scores squared (%.17g, %.17g, %.17g, %.17g, %.17g)\n", sq[0],
         sq[1], sq[2], sq[3], sq[4]);
  CHECK(same, "ch09 PC1 raw scores on (1,1,1) = (3,3,3,-6,-3)");
  CHECK(sq[0] == 3.0 && sq[1] == 3.0 && sq[2] == 3.0 && sq[3] == 12.0 &&
            sq[4] == 3.0,
        "ch09 unit scores squared (3,3,3,12,3)");
```

listing 59: scores on (1,1,1), unit scores squared, sums accumulated (source: kdd/samples/src/Ch09/pca.c, lines 266-283)

```
  double lam[3];
  eig_sorted(dcov, lam);
  const double var_raw = sum_sq / 4.0;
  const double var_unit = (sq[0] + sq[1] + sq[2] + sq[3] + sq[4]) / 4.0;
  printf("var raw %.17g (= 3*lambda1 %.17g), var unit %.17g (= lambda1 "
         "%.17g)\n",
         var_raw, 3.0 * lam[0], var_unit, lam[0]);
  CHECK(var_raw == 18.0, "ch09 variance of raw scores = 18");
  CHECK(fabs(3.0 * lam[0] - 18.0) <= 1e-9,
        "ch09 raw-score variance = 3*lambda1");
  CHECK(var_unit == 6.0 && fabs(lam[0] - 6.0) <= 1e-9,
        "ch09 unit-score variance = 6 = lambda1");
```

listing 60: variance 6 equals lambda1, raw variance 18 equals 3 lambda1 (source: kdd/samples/src/Ch09/pca.c, lines 284-295)

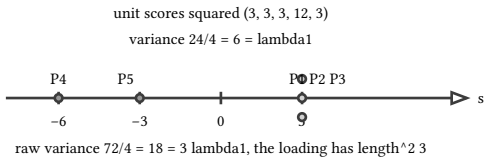


figure 37: five scores on the first component, variance 6 is the eigenvalue

One rotation family, one handshake, one invariance. The solver is fifty lines of c23, its certificate is a matrix rebuild under 1e-12, and the economics of the whole method sit in the last figure: one axis carries variance 6 of the total $6 + 2.25 + 2.25 = 10.5$, 57 percent of the spread in one number per point. book 14, ch 10 (knowledge discovery, the haar wavelet transform) next keeps two coefficients out of eight for 99 percent of the energy, a different trade on the same idea.

sources: all 25 pinned values, the sample-divisor pin (n - 1 here, n in book 14, ch 5 (knowledge discovery, distance metrics)), the never-pin-Q rule for repeated eigenvalues, and the generator-measured handshake residuals witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s2.py, run 2026-09-22, exit 0. The aI + bJ

eigenvalue derivation is the prose territory of the math book's decomposition chapter, book 2, ch 8 (mathematics for programmers, matrix decompositions). Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch09`, 25 checks in chapter 09 of the kdd suite.

10. the haar wavelet transform

One sample carries the chapter: `haar.c` runs the 3-level Haar forward and inverse transform on two 8-point signals, verifies the single-level identity, truncates to the 2 largest coefficients with a pinned tie-break, and accounts for every unit of energy as an exact fraction, 42 checks in all. The chapter makes 4 moves: the averaging pair operator and the 3-level coefficient layout, the inverse and why reconstruction is bit-exact here, keep-k truncation and its lowest-index tie-break, and energy retention as reduced fractions. Every behavioral claim below is one of the 42 checks of chapter 10's sample or a pinned note of the contract sheet. book 14, ch 9 (knowledge discovery, principal component analysis) reduced dimension by rotating axes, this chapter reduces by throwing detail away, and book 14, ch 8 (knowledge discovery, feature selection and chi-square) selects columns while this chapter recombines them.

10.1. averages and differences

The Haar transform runs one pair operator down the signal: avg = (a + b)/2 and det = (a - b)/2. The average keeps what neighboring samples agree on, the detail keeps what they disagree on, and the pair (avg, det) is just (a, b) written in a different basis, so nothing is lost, only re-shelved. Level 1 pairs the samples, level 2 pairs the averages, level 3 averages those, and an 8-point signal ends as one overall average plus one detail per scale.

The dry run: the smooth fixture is S = (4, 6, 8, 10, 20, 20, 20, 20), a ramp then a plateau. Level 1 pairs give avgs (5, 9, 20, 20) and dets (-1, −1, 0, 0). Level 2 pairs the averages: (7, 20) with dets (-2, 0). Level 3 averages those: 13.5 with det −6.5. The transform prints as `T(S) = 13.5 -6.5 -2 0 -1 -1 0 0`, eight checks, one per coefficient. The noisy fixture N = (4, 12, 4, 12, 20, 12, 20, 12), the same 8/16 step shape with a +-4 wobble riding it, prints `T(N) = 12 -4 0 0 -4 -4 4 4`: the same coarse story in the first two slots, but all four level-0 detail slots now carry +-4 wobble instead of zeros.

```c
static void haar_fwd(const int *x, int n, double *c) {
  for (int i = 0; i < n; i++) {
    c[i] = (double)x[i];
  }
  int d = n;
  while (d > 1) {
    d /= 2;
    double nxt[8];
    for (int i = 0; i < n; i++) { /* untouched positions carry over */
      nxt[i] = c[i];
    }
    for (int i = 0; i < d; i++) {
      nxt[i] = (c[2 * i] + c[2 * i + 1]) / 2.0;
      nxt[d + i] = (c[2 * i] - c[2 * i + 1]) / 2.0;
    }
    for (int i = 0; i < n; i++) {
      c[i] = nxt[i];
    }
  }
}
```

listing 61: one pass: averages into the front half, details into the back (source: kdd/samples/src/Ch10/haar.c, lines 41-60)

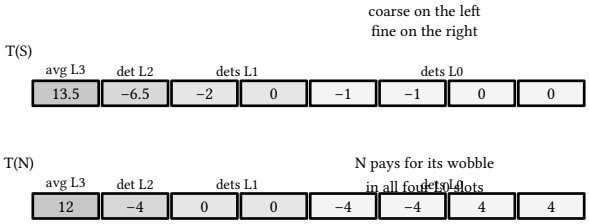


figure 38: the 8 coefficients of each signal shelved by scale

10.2. the inverse, bit for bit

The inverse operator is the same butterfly run backwards: a = avg + det, b = avg - det. Apply it level by level, doubling the working width each time, and the original signal comes back. What makes the round trip exact is arithmetic, not luck: starting from integers, every average and difference is a half-integer, halving again stays dyadic, and dyadic rationals are represented exactly in binary floating point. No rounding ever happens, so == against the original is a legitimate assertion, the same family of pin as the single-sqrt cosine of book 14, ch 6 (knowledge discovery, similarity measures).

The dry run: the single-level identity on the raw pairs prints `L1 S:`
`avgs (5, 9, 20, 20) dets (-1, -1, 0, 0)` and `L1 N: avgs (8, 8, 16, 16)`
`dets (-4, -4, 4, 4)`, and the identity checks assert avg + det and avg
- det reproduce every original sample. The full 3-level round trips are

asserted whole, “ch10 inverse reproduces S exactly” and “ch10 inverse reproduces N exactly”, all sixteen samples compared with ==. The plateau pairs are the visible freebie: (20, 20) averages to 20 with detail 0, agreement costs nothing to encode.

```
static void haar_inv(const double *c, int n, double *x) {
  for (int i = 0; i < n; i++) {
    x[i] = c[i];
  }
  int d = 1;
  while (d < n) {
    double nxt[8];
    for (int i = 0; i < n; i++) { /* untouched positions carry over */
      nxt[i] = x[i];
    }
    for (int i = 0; i < d; i++) {
      nxt[2 * i] = x[i] + x[d + i];
      nxt[2 * i + 1] = x[i] - x[d + i];
    }
    for (int i = 0; i < n; i++) {
      x[i] = nxt[i];
    }
    d *= 2;
  }
}
```

listing 62: the inverse butterfly, doubling the working width per level (source: kdd/samples/src/Ch10/haar.c, lines 62-81)

```
{
  const double want_avgs[2][4] = {{5, 9, 20, 20}, {8, 8, 16, 16}};
  const double want_dets[2][4] = {{-1, -1, 0, 0}, {-4, -4, 4, 4}};
  const int *sig[2] = {SM, NZ};
  for (int s = 0; s < 2; s++) {
    double avgs[4], dets[4];
    for (int i = 0; i < 4; i++) {
      avgs[i] = ((double)sig[s][2 * i] + sig[s][2 * i + 1]) / 2.0;
      dets[i] = ((double)sig[s][2 * i] - sig[s][2 * i + 1]) / 2.0;
    }
    printf("L1 %s: avgs (%.17g, %.17g, %.17g, %.17g) dets (%.17g, %.17g, "
           "%.17g, %.17g)\n",
           s == 0 ? "S" : "N", avgs[0], avgs[1], avgs[2], avgs[3], dets[0],
           dets[1], dets[2], dets[3]);
    int a_ok = 1, d_ok = 1, id_ok = 1;
    for (int i = 0; i < 4; i++) {
      a_ok = a_ok && avgs[i] == want_avgs[s][i];
      d_ok = d_ok && dets[i] == want_dets[s][i];
      id_ok = id_ok && avgs[i] + dets[i] == (double)sig[s][2 * i] &&
              avgs[i] - dets[i] == (double)sig[s][2 * i + 1];
    }
```

listing 63: single-level avgs and dets on both signals, identity asserted (source: kdd/samples/src/Ch10/haar.c, lines 172-192)

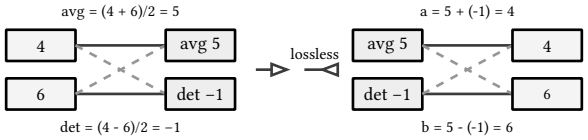


figure 39: one butterfly forward and back on the pair (4, 6)

NOTE: dyadic means the == is honest

Every coefficient of both fixtures is a dyadic rational, an integer over a power of two, so the forward transform, the inverse, and the truncated reconstruction all compute exactly representable doubles and the sample compares them with ==. The dyadics are closed under halving, which is the whole argument: from integer inputs, no level of the transform ever produces a value binary floating point cannot hold. The same closure is what let book 14, ch 4 (knowledge discovery, normalization and standardization) divide by the power-of-two range 16 and book 14, ch 6 (knowledge discovery, similarity measures) take one sqrt of a perfect square.

10.3. keep 2 of 8

Compression keeps the k largest-magnitude coefficients and zeroes the rest, and the whole trick is that the coarse shelves carry almost everything. The selection rule is pinned: sort by descending magnitude, and an equal magnitude never displaces an earlier index, so ties resolve to the lowest coefficient index.

The dry run: on T(S) = (13.5, −6.5, −2, 0, −1, −1, 0, 0) the two largest magnitudes are 13.5 and 6.5 at indices 0 and 1, printed as `keep-2 S: kept (0, 1) values (13.5, -6.5), recon 7 7 7 7 20 20 20 20`. The zeroed transform unwinds in three steps: level 3 gives (7, 20), level 2 spreads to (7, 7, 20, 20), level 1 fills all eight, four 7s and four 20s. On T(N) the top magnitudes are 12 at index 0 and then a five-way tie at magnitude 4 spanning indices 1, 4, 5, 6, 7, and the lowest-index rule keeps index 1, printed as `keep-2 N: kept (0, 1)`
`values (12, -4), recon 8 8 8 8 16 16 16 16`. The wobble lives entirely in the discarded slots, so the reconstruction is the clean step shape with every wiggle gone.

```
 * returns the kept indices in ascending order */
static void keep_k(const double *c, int n, int k, double *out, int *kept,
                   int *n_kept) {
  int keep[8] = {0};
  for (int sel = 0; sel < k; sel++) {
    int best = -1; /* ascending scan: an equal magnitude never displaces */
    for (int i = 0; i < n; i++) {
      if (keep[i]) {
        continue;
      }
      if (best == -1 || fabs(c[i]) > fabs(c[best])) {
        best = i;
      }
    }
    keep[best] = 1;
  }
```

listing 64: ascending scan, an equal magnitude never displaces (source: kdd/samples/src/Ch10/haar.c, lines 84-99)

```
  double t2sm[8], recsm[8], t2nz[8], recnz[8];
  int kept[8], n_kept;
  {
    keep_k(tsm, 8, 2, t2sm, kept, &n_kept);
    haar_inv(t2sm, 8, recsm);
    const int want_rec[8] = {7, 7, 7, 7, 20, 20, 20, 20};
    int rec_ok = 1;
    for (int i = 0; i < 8; i++) {
      rec_ok = rec_ok && recsm[i] == (double)want_rec[i];
    }
    printf("keep-2 S: kept (%d, %d) values (%.17g, %.17g), recon", kept[0],
           kept[1], t2sm[kept[0]], t2sm[kept[1]]);
    for (int i = 0; i < 8; i++) {
      printf(" %g", recsm[i]);
    }
    printf("\n");
    CHECK(n_kept == 2 && kept[0] == 0 && kept[1] == 1,
          "ch10 keep-2 S indices (0,1)");
    CHECK(t2sm[0] == 13.5 && t2sm[1] == -6.5, "ch10 keep-2 S keeps 13.5, -6.5");
    CHECK(rec_ok, "ch10 keep-2 S reconstruction (7,7,7,7,20,20,20,20)");
  }
```

listing 65: keep-2 on the smooth signal, reconstruction checked value by value (source: kdd/samples/src/Ch10/haar.c, lines 204-224)

```
{
  /* the |4| tie spans indices 1,4,5,6,7; lowest index 1 wins */
  keep_k(tnz, 8, 2, t2nz, kept, &n_kept);
  haar_inv(t2nz, 8, recnz);
  const int want_rec[8] = {8, 8, 8, 8, 16, 16, 16, 16};
  int rec_ok = 1;
  for (int i = 0; i < 8; i++) {
    rec_ok = rec_ok && recnz[i] == (double)want_rec[i];
  }
  printf("keep-2 N: kept (%d, %d) values (%.17g, %.17g), recon", kept[0],
         kept[1], t2nz[kept[0]], t2nz[kept[1]]);
  for (int i = 0; i < 8; i++) {
    printf(" %g", recnz[i]);
  }
  printf("\n");
  CHECK(n_kept == 2 && kept[0] == 0 && kept[1] == 1,
        "ch10 keep-2 N indices (0,1), |4| tie to index 1");
  CHECK(t2nz[0] == 12.0 && t2nz[1] == -4.0, "ch10 keep-2 N keeps 12, -4");
  CHECK(rec_ok, "ch10 keep-2 N reconstruction (8,8,8,8,16,16,16,16)");
}
```

listing 66: keep-2 on the noisy signal, the |4| tie resolving to index 1 (source: kdd/samples/src/Ch10/haar.c, lines 225-244)

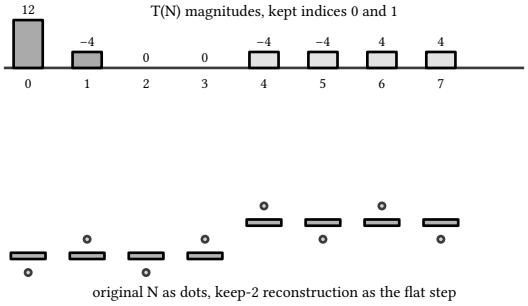


figure 40: the noisy magnitudes, two kept, and the flat step that comes back

10.4. energy, accounted in fractions

Signal energy is the sum of squares, and Parseval's identity for this orthogonal transform says the coefficients carry the same total energy as the samples. Truncation spends energy where it discards coefficients, and the sample keeps the whole accounting in integers: energies as sums of squares, retentions as reduced fraction pairs, the doubles only a display of the exact fractions.

The dry run: S's energy is $16 + 36 + 64 + 100 + 4 \cdot 400 = 1816$, the reconstruction holds $4 \cdot 49 + 4 \cdot 400 = 1796$, and the retention prints as `energies: S 1816 → 1796 = 449/454 (0.98898678414096919)`, the fraction reduced by gcd 4, 449 prime. N's energy is two 4-12 pairs and two 20-12 pairs, $2(16 + 144) + 2(400 + 144) = 320 + 1088 = 1408$, its reconstruction $4 \cdot 64 + 4 \cdot 256 = 1280$, retention 10/11, printed as `N 1408 → 1280 = 10/11 (0.90909090909090906)`. The contrast check pins the ordering, smooth beats noisy, by cross-multiplication, and the discarded wobble prints as `wobble 128/1408 = 1/11 discarded`: the noisy signal loses exactly one eleventh of its energy, all of it wobble, all of it in the detail coefficients the last facet threw away.

```
  {
    int e_sm = 0, e_nz = 0;
    for (int i = 0; i < 8; i++) {
      e_sm += SM[i] * SM[i];
      e_nz += NZ[i] * NZ[i];
    }
    const int er_sm = 4 * 7 * 7 + 4 * 20 * 20; /* hand: 4*49 + 4*400 */
    const int er_nz = 4 * 8 * 8 + 4 * 16 * 16; /* cross-checked below */
    int er_recomputed = 0;
    for (int i = 0; i < 8; i++) { /* recompute from the actual recon */
      er_recomputed += (int)(recsm[i] * recsm[i]);
    }
    CHECK(er_recomputed == er_sm, "ch10 S recon energy matches hand 1796");
    er_recomputed = 0;
    for (int i = 0; i < 8; i++) {
      er_recomputed += (int)(recnz[i] * recnz[i]);
    }
    CHECK(er_recomputed == er_nz, "ch10 N recon energy matches hand 1280");
```

listing 67: energies as integer sums of squares, recon energies recomputed (source: kdd/samples/src/Ch10/haar.c, lines 249-266)

```
    const int g_sm = gcd_i(er_sm, e_sm);
    const int g_nz = gcd_i(er_nz, e_nz);
    const double ret_sm = (double)er_sm / (double)e_sm;
    const double ret_nz = (double)er_nz / (double)e_nz;
    printf("energies: S %d -> %d = %d/%d (%.17g), N %d -> %d = %d/%d "
           "(%.17g)\n",
           e_sm, er_sm, er_sm / g_sm, e_sm / g_sm, ret_sm, e_nz, er_nz,
           er_nz / g_nz, e_nz / g_nz, ret_nz);
    CHECK(e_sm == 1816, "ch10 S energy 1816");
    CHECK(er_sm == 1796, "ch10 S keep-2 recon energy 1796");
    CHECK(er_sm / g_sm == 449 && e_sm / g_sm == 454,
          "ch10 S retention fraction 449/454");
    CHECK(ret_sm == 0.9889867841409692,
          "ch10 S retention double 0.9889867841409692");
    CHECK(e_nz == 1408, "ch10 N energy 1408");
    CHECK(er_nz == 1280, "ch10 N keep-2 recon energy 1280");
    CHECK(er_nz / g_nz == 10 && e_nz / g_nz == 11,
          "ch10 N retention fraction 10/11");
    CHECK(ret_nz == 0.9090909090909091,
          "ch10 N retention double 0.9090909090909091");
```

listing 68: retentions as reduced pairs, doubles pinned as nearest doubles (source: kdd/samples/src/Ch10/haar.c, lines 267-286)

```
    /* contrast: the smooth signal keeps more of its energy; the noisy
     * signal's wobble energy 128/1408 = 1/11 leaves with the details */
    CHECK(er_sm * e_nz > er_nz * e_sm,
          "ch10 smooth retention 449/454 > noisy 10/11");
    const int wobble = e_nz - er_nz;
    const int gw = gcd_i(wobble, e_nz);
    printf("wobble %d/%d = %d/%d discarded\n", wobble, e_nz, wobble / gw,
           e_nz / gw);
    CHECK(wobble == 128 && wobble / gw == 1 && e_nz / gw == 11,
          "ch10 discarded wobble energy 128/1408 = 1/11");
```

listing 69: the wobble 128/1408 reduced to 1/11, discarded with the details (source: kdd/samples/src/Ch10/haar.c, lines 288-297)

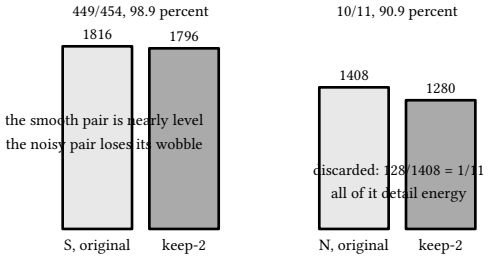


figure 41: two signals, two retentions, the wobble is exactly one eleventh

Two signals, one verdict. The smooth fixture keeps 449 of every 454 energy units with a quarter of the coefficients, the noisy one keeps 10 of 11, and both reconstructions were exact integers before any double was printed. That is the transformation arc of the book's first ten chapters closed: book 14, ch 3 (knowledge discovery, binning, smoothing, and discretization) shrank values, book 14, ch 9 (knowledge discovery, principal component analysis) rotated axes, and this chapter kept two shelves and dropped the rest, each with its arithmetic pinned to the last integer.

sources: all 42 pinned values, the avg/det convention (avg = (a + b)/2, det = (a - b)/2, inverse a = avg + det, b = avg - det), the keep-k lowest-index tie-break, and the dyadic-exactness note witnessed by kdd-contract-s1s2.md and playground/kdd-matrix/gen_s2.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch10`, 42 checks in chapter 10 of the kdd suite.

11. data cubes and olap

One sample carries the chapter: `olap.c` aggregates a 12-row fact table along three dimension hierarchies and asserts 76 checks, every one an integer comparison. The chapter makes 4 moves: the star schema and its one additive measure, the cube lattice the dimensions generate and how sparse it is, roll-up and drill-down as aggregation up and down the hierarchies, and slice, dice, and the pivot as selection and reshape. Every behavioral claim below is one of the 76 checks of chapter 11's sample or a named term from Kimball and Ross or Gray et al. The ten preprocessing chapters behind us shaped one table at a time; from here the book models, and a model consumes exactly the aggregated shapes this chapter computes, starting with book 14, ch 12 (knowledge discovery, simple linear regression) next. book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices) rebuilds this whole aggregation lane in go over duckdb.

11.1. the star, and the twelve facts

A star schema puts the measurements in the middle and the context around them: a fact table of units sold, one row per (month, product, city) triple, and a dimension table per axis, each carrying a hierarchy, month to quarter, product to category, city to country. The vocabulary is Kimball and Ross's: facts are the numeric events, dimensions are the axes you slice them along, and the one measure here, units, is additive along all three axes, which is what makes every sum in this chapter a sub-sum of one total.

The dry run: the twelve units read 4, 2, 6, 1, 3, 1, 5, 2, 6, 3, 7, 2, and their sum is the number everything else hangs off, printed as `grand total units 42` and asserted as "ch11 grand total units 42". That check is the additive anchor: every later value, group-by cell, roll-up, slice, dice, and pivot margin, is a partition or sub-sum of the same twelve numbers, and the closing margin check lands on 42 again from a completely different direction.

```
static const struct {
  const char *month, *product, *city;
  int units;
} F[NR] = {
    {"feb", "b1", "paris", 4},  {"feb", "b1", "berlin", 2},
    {"feb", "m1", "paris", 6},  {"feb", "b2", "lyon", 1},
    {"mar", "b1", "paris", 3},  {"mar", "b1", "lyon", 1},
    {"mar", "m1", "berlin", 5}, {"mar", "b2", "paris", 2},
    {"apr", "b1", "paris", 6},  {"apr", "b1", "berlin", 3},
    {"apr", "m1", "paris", 7},  {"apr", "b1", "lyon", 2},
};
```

listing 70: the fact table, 12 rows, one additive measure (source: kdd/samples/src/Ch11/olap.c, lines 28-38)

```
/* hierarchy tables, ch11 fixture: feb,mar->q1; apr->q2; b1,b2->books;
 * m1->media; paris,lyon->fr; berlin->de */
static const char *quarter(const char *m) {
  return strcmp(m, "apr") == 0 ? "q2" : "q1";
}
static const char *category(const char *p) {
  return strcmp(p, "m1") == 0 ? "media" : "books";
}
static const char *country(const char *c) {
  return strcmp(c, "berlin") == 0 ? "de" : "fr";
}
```

listing 71: the three hierarchies, quarter, category, country (source: kdd/samples/src/Ch11/olap.c, lines 44-54)

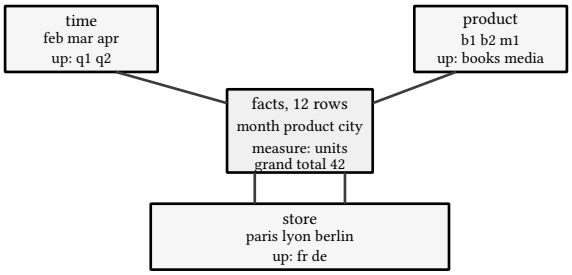


figure 42: the star: one fact table, three dimensions, three hierarchies

11.2. the cube lattice, 46 cells of 64

Gray et al's cube operator generalizes group-by to every subset of dimensions at once: one cuboid per subset, the base cuboid sitting at (month, product, city), the apex at no dimensions at all, and a lattice between them where each step up aggregates one dimension away. The sample counts nonempty cells per cuboid the direct way, marking every combination of dimension values that some fact row actually hits.

The dry run: the base cuboid holds 12 nonempty cells of the 3 x 3 x 3 = 27 possible, and the density check is exact fraction arithmetic, "ch11 base cuboid density 4/9" asserted by cross-multiplication, 12 x 9 = 4 x 27. The three 2-D cuboids print as `cuboid cells (month,product)=8 (month,city)=9 (product,city)=7` and the singles as `singles month=3 product=3 city=3 apex=1 full cube 46 of 64`. The 46 is the sum 12 + 8 + 9 + 7 + 3 + 3 + 3 + 1 across the eight cuboids, and the 64 is not a coincidence: the possible-cell total is $27 + 3 \cdot 9 + 3 \cdot 3 + 1 = (3+1)^3$, the binomial theorem counting, per dimension, its 3 values plus the choice to aggregate it away.

```
static int cells3(void) {
  bool seen[3][3][3] = {false};
  int n = 0;
  for (int i = 0; i < NR; i++) {
    seen[idx3(MON, F[i].month)][idx3(PRD, F[i].product)][idx3(CIT, F[i].city)] =
        true;
  }
  for (int a = 0; a < 3; a++)
    for (int b = 0; b < 3; b++)
      for (int c = 0; c < 3; c++) {
        n += seen[a][b][c];
      }
  return n;
}
```

listing 72: base-cuboid cells, every combination a fact row hits (source: kdd/samples/src/Ch11/olap.c, lines 112-125)

```
  /* [ch11.r04-08] 1-D cuboids, apex, and the full cube lattice */
  int m1 = cells1('m'), p1 = cells1('p'), c1 = cells1('c');
  int cube = base + mp + mc + pc + m1 + p1 + c1 + 1 /* apex */;
  printf("singles month=%d product=%d city=%d apex=1 full cube %d of %d\n", m1,
         p1, c1, cube, 4 * 4 * 4);
  CHECK(m1 == 3, "ch11 cuboid (month) 3 nonempty cells");
  CHECK(p1 == 3, "ch11 cuboid (product) 3 nonempty cells");
  CHECK(c1 == 3, "ch11 cuboid (city) 3 nonempty cells");
  CHECK(cube == 46 && 4 * 4 * 4 == 64, "ch11 full cube 46 nonempty of 64");
```

listing 73: the eight cuboids summed, 46 nonempty of 64 possible (source: kdd/samples/src/Ch11/olap.c, lines 193-201)

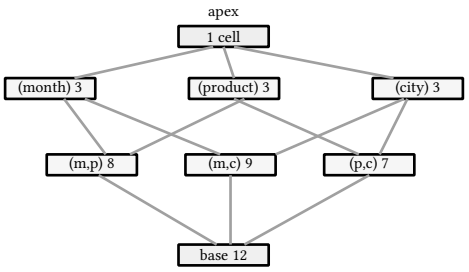


figure 43: the cube lattice, nonempty cells per cuboid, 46 of 64

PITFALL: the lattice is exponential in dimensions

Three dimensions give eight cuboids, 2^3, and the sample walks all eight. Twenty dimensions give over a million, $2^{20} = 1048576$, which no engine materializes in full. Gray et al's operator defines the whole lattice, but a warehouse computes cuboids on demand or materializes only the hot ones, and that engineering choice, picked per query pattern, is exactly what book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices) revisits on duckdb.

11.3. roll-up and drill-down

Roll-up aggregates a dimension up its hierarchy, city to country or month to quarter; drill-down walks the same ladder the other way, q1 back to feb and mar. Both are group-by underneath: the sample's one base aggregate takes a dimension index or a wildcard per axis, and the hierarchy variant matches rows at the rolled-up level instead.

The dry run: group-by month prints `sum month=feb 13`, `sum month=mar 11`, `sum month=apr 18`, the three sub-sums 4+2+6+1, 3+1+5+2, 6+3+7+2. Rolling product up one level prints `sum category=books 24` and `sum category=media 18`, and month up to quarter prints `sum quarter=q1 24` and `sum quarter=q2 18`. The mixed roll-up over (month, country) walks six cells, from `rollup feb/fr count=3 sum=11` to `rollup apr/de count=1 sum=3`, with counts 3, 1, 3, 1, 3, 1 because berlin is the only German city. Drill-down reverses direction over the same tables: `drilldown q1=24 feb=13 mar=11` splits the quarter cell, and `drilldown fr: paris=28 lyon=4; de: berlin=10` splits both countries.

```
static void agg_hier(int qi, int bi, int di, int *count, int *sum) {
  static const char *Q[2] = {"q1", "q2"};
  static const char *B[2] = {"books", "media"};
  static const char *D[2] = {"fr", "de"};
  *count = 0;
  *sum = 0;
  for (int i = 0; i < NR; i++) {
    if ((qi < 0 || strcmp(quarter(F[i].month), Q[qi]) == 0) &&
        (bi < 0 || strcmp(category(F[i].product), B[bi]) == 0) &&
        (di < 0 || strcmp(country(F[i].city), D[di]) == 0)) {
      (*count)++;
      *sum += F[i].units;
    }
  }
}
```

listing 74: the hierarchy aggregate, matching rows at quarter, category, country level (source: kdd/samples/src/Ch11/olap.c, lines 82-96)

```
{
  const int want_s[6] = {11, 2, 6, 5, 15, 3};
  const int want_c[6] = {3, 1, 3, 1, 3, 1};
  int k = 0;
  for (int mi = 0; mi < 3; mi++) {
    for (int di = 0; di < 2; di++) {
      agg_mc(mi, di, &count, &sum);
      printf("rollup %s/%s count=%d sum=%d\n", MON[mi], di ? "de" : "fr",
             count, sum);
      CHECK(sum == want_s[k], "ch11 rollup (month,country) cell sum");
      CHECK(count == want_c[k], "ch11 rollup (month,country) cell count");
      k++;
    }
  }
}
```

listing 75: roll-up (month, country), six cells, sums and counts pinned in scan order (source: kdd/samples/src/Ch11/olap.c, lines 257-271)

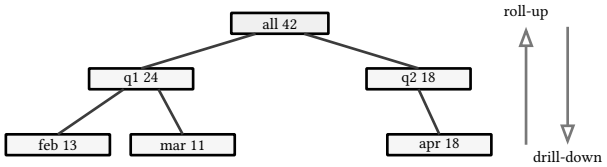


figure 44: the time hierarchy, roll-up aggregates up, drill-down refines down

PITFALL: aggregation is not invertible

The cell q1 = 24 does not remember that it was 13 + 11. Drill-down in the sample re-reads the 12-row fact table and regroups at month level; it does not unpack the quarter cell, because nothing is left to unpack. A warehouse that stores only an aggregated cuboid has burned the detail below it, which is why production schemas keep the base facts and compute roll-ups on the fly or refresh them from below.

11.4. slice, dice, and the pivot

Slice fixes one dimension to one value, dice fixes two or more, and the pivot reshapes what survives into a cross-tab with sub-totals, the cross-tab and sub-totals that Gray et al fold into the same cube operator. All three ride the one wildcard aggregate: a slice on month = feb is the group-by cell (feb) with its count kept alongside the sum.

The dry run: slice month=feb keeps 4 rows summing 13, slice city=paris keeps 6 rows summing 28, and both print through the same aggregate the group-by used. The dice month in {feb, mar} and country=de keeps 2 rows, the two berlin facts, summing 2 + 5 = 7, printed as `dice feb|mar AND de: count=2 sum=7`; the dice category=books and city=paris keeps 4 rows summing 15. The pivot prints the whole month x city matrix, `feb 10 1 2, mar 5 1 5, apr 13 2 3` over columns paris, lyon, berlin, and the margins line `margins rows 13 11 18 cols 28 4 10 grand 42` closes the loop on the chapter's first check: the grand cell and the additive anchor are the same 42.

```
{
  count = sum = 0;
  for (int i = 0; i < NR; i++) {
    if (F[i].month[0] == 'a' && strcmp(country(F[i].city), "de") == 0) {
      count++;
      sum += F[i].units;
    }
  }
  printf("dice feb|mar AND de: count=%d sum=%d\n", count, sum);
  CHECK(count == 2, "ch11 dice {feb,mar} x de count 2");
  CHECK(sum == 7, "ch11 dice {feb,mar} x de sum 7");
}
```

listing 76: dice as a two-predicate scan over the fact table (source: kdd/samples/src/Ch11/olap.c, lines 341-352)

```
int piv[3][3];
for (int mi = 0; mi < 3; mi++)
  for (int ci = 0; ci < 3; ci++) {
    agg3(mi, -1, ci, &count, &sum);
    piv[mi][ci] = sum;
  }
printf("pivot rows feb/mar/apr cols paris/lyon/berlin:\n");
for (int mi = 0; mi < 3; mi++) {
  printf("  %s %d %d %d\n", MON[mi], piv[mi][0], piv[mi][1], piv[mi][2]);
}
{
  const int want[3][3] = {{10, 1, 2}, {5, 1, 5}, {13, 2, 3}};
  for (int mi = 0; mi < 3; mi++)
    for (int ci = 0; ci < 3; ci++) {
      CHECK(piv[mi][ci] == want[mi][ci], "ch11 pivot month x city cell");
    }
```

listing 77: the pivot matrix, every cell one wildcard aggregate (source: kdd/samples/src/Ch11/olap.c, lines 370-385)

```
int rowtot[3] = {0}, coltot[3] = {0}, grand = 0;
for (int mi = 0; mi < 3; mi++)
  for (int ci = 0; ci < 3; ci++) {
    rowtot[mi] += piv[mi][ci];
    coltot[ci] += piv[mi][ci];
    grand += piv[mi][ci];
  }
printf("margins rows %d %d %d cols %d %d %d grand %d\n", rowtot[0], rowtot[1],
       rowtot[2], coltot[0], coltot[1], coltot[2], grand);
```

listing 78: margins, row totals, column totals, and the grand 42 (source: kdd/samples/src/Ch11/olap.c, lines 389-397)

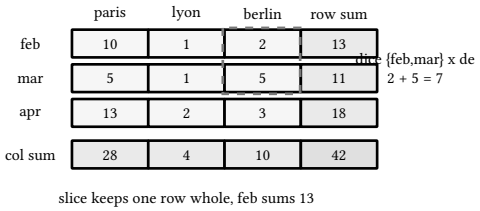


figure 45: the month x city pivot with margins, the dashed pair is the dice

The pivot is also where this chapter hands off. Everything above is one aggregate over wildcards, and a warehouse engine is that aggregate with indexes and columnar storage behind it, the lane book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices) runs in go. The next chapter fits a

line through numbers instead of summing them, book 14, ch 12 (knowledge discovery, simple linear regression), still on fixtures small enough to check by hand.

sources: Kimball & Ross, The Data Warehouse Toolkit, Wiley 2002, star schema and the slice, dice, roll-up, drill-down vocabulary. Gray, Bosworth, Layman, Pirahesh, "Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab and Sub-Totals", ICDE 1996, cube-lattice cell counting. All 76 pinned values witnessed by kdd-contract-s3.md and playground/kdd-matrix/gen_s3.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch11`, 76 checks in chapter 11 of the kdd suite.

12. simple linear regression

One sample carries the chapter: `ols.c` fits one line to each of two tiny fixtures and asserts 35 checks, exact fractions wherever the value is rational, tolerance doubles where it is not. The chapter makes 4 moves: the normal equations and the perfect fixture, the fraction arithmetic that keeps a slope like 4/5 exact, the residuals and the sum they must make, and the decomposition SSE + SSR = Syy that defines R^2. Every behavioral claim below is one of the 35 checks of chapter 12's sample or a named note of the contract sheet. Chapter 11's aggregates summarized a table, book 14, ch 11 (knowledge discovery, data cubes and olap) this chapter fits a model through it, and the whole models arc of the book opens here.

12.1. the least squares line

Given pairs (x_i, y_i), ordinary least squares picks slope m and intercept b minimizing the squared vertical errors $\sum_i (y_i - mx_i - b)^2$. Setting both partial derivatives to zero gives the normal equations, and their solution is closed form: $m = (nS_{xy} - S_x S_y)/(nS_{x2} - S_x^2)$ and $b = S_y/n - mS_x/n$. Squaring is a choice with two justifications: it makes the equations linear, and under gaussian noise the maximum likelihood line is exactly the least squares line, the derivation this book keeps in book 2, ch 19 (mathematics for programmers, statistics and inference). Fixture A is perfectly linear, the five pairs (1,3), (2,5), (3,7), (4,9), (5,11), so every quantity below is a small integer and the fit is exact.

The dry run: the five sums print as `A: n=5 Sx=15 Sy=35 Sxy=125 sumx2=55 Sxx_centered=50`, where the 50 is the cross-multiplied form $n\sum x^2 - (\sum x)^2 = 275 - 225$, five times the centered 10. The slope numerator is $5 \cdot 125 - 15 \cdot 35 = 625 - 525 = 100$, so $m = 100/50 = 2$ and $b = 35/5 - 2 \cdot 15/5 = 7 - 6 = 1$, printed as `A: slope 2/1 = 2.000000000000000, intercept 1/1 = 1.000000000000000` and asserted with plain ==, both values dyadic.

```
/* ---------- fixture A: sums, slope, intercept ---------- */
long long n = NA, Sx = 0, Sy = 0, Sxy = 0, Sxx = 0;
for (int i = 0; i < NA; i++) {
  Sx += xa[i];
  Sy += ya[i];
  Sxy += xa[i] * ya[i];
  Sxx += xa[i] * xa[i];
}
long long Sxxc = n * Sxx - Sx * Sx; /* n * centered Sxx */
printf("A: n=%lld Sx=%lld Sy=%lld Sxy=%lld sumx2=%lld Sxx_centered=%lld\n", n,
       Sx, Sy, Sxy, Sxx, Sxxc);
CHECK(n == 5, "ch12 A n 5");
CHECK(Sx == 15, "ch12 A Sx 15");
CHECK(Sy == 35, "ch12 A Sy 35");
CHECK(Sxy == 125, "ch12 A Sxy 125");
CHECK(Sxx == 55, "ch12 A sum x^2 55");
CHECK(Sxxc == 50, "ch12 A centered Sxx 50");
```

listing 79: fixture A's five sums, one pass, all integers (source: kdd/samples/src/Ch12/ols.c, lines 79-95)

```
  /* slope m = (n*Sxy - Sx*Sy)/(n*Sxx - Sx^2); intercept b = Sy/n - m*Sx/n,
   * both exact as fractions, dyadic here so the double == pins hold */
  Frac mA = frac(n * Sxy - Sx * Sy, Sxxc);
  Frac bA = frac_sub(frac(Sy, n), frac_mul(mA, frac(Sx, n)));
  printf("A: slope %lld/%lld = %.15f, intercept %lld/%lld = %.15f\n", mA.num,
         mA.den, frac_d(mA), bA.num, bA.den, frac_d(bA));
  CHECK(frac_d(mA) == 2.0, "ch12 A slope 2.0 ==");
  CHECK(frac_d(bA) == 1.0, "ch12 A intercept 1.0 ==");
```

listing 80: slope and intercept from the normal equations, exact fractions (source: kdd/samples/src/Ch12/ols.c, lines 97-104)

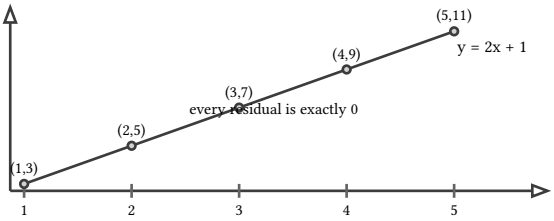


figure 46: fixture A, five points on the line y = 2x + 1, residuals zero

12.2. a slope the cpu cannot hold

Fixture B is four pairs, (1,3), (2,2), (3,4), (4,5), and its slope is 4/5, a fraction with no finite binary expansion. The sample therefore carries a reduced-fraction type, `long long` numerator and denominator, and asserts every rational result by cross-multiplication, `num * q ==`
`p * den`, exact. The decimal prints ride along as commentary, pinned separately at tolerance 1e-12, the ledger's D2 class.

The dry run: the sums are $S_x = 10$, $S_y = 14$, $S_{xy} = 39$, $\sum x^2 = 30$, so the slope numerator is $4 \cdot 39 - 10 \cdot 14 = 156 - 140 = 16$ over $4 \cdot 30 - 100 = 20$, reducing to 4/5. The intercept is $14/4 - (4/5)(10/4) = 7/2 - 2 = 3/2$. The sample prints `B: slope 4/5 = 0.800000000000000`,
`intercept 3/2 = 1.500000000000000`, asserts "ch12 B slope 4/5 exact" by cross-multiplication and "ch12 B slope double 0.8 at 1e-12", and asserts the intercept 3/2 both ways because halves are dyadic.

```
static Frac frac(long long n, long long d) {
  if (d < 0) {
    n = -n;
    d = -d;
  }
  long long g = fgcd(n, d);
  if (g > 1) {
    n /= g;
    d /= g;
  }
  return (Frac){n, d};
}
```

listing 81: the reduced-fraction constructor, gcd and sign handled (source: kdd/samples/src/Ch12/ols.c, lines 40-51)

```
Sxxc = n * Sxx - Sx * Sx; /* 20, i.e. n * 5 */
Frac m = frac(n * Sxy - Sx * Sy, Sxxc);
Frac b = frac_sub(frac(Sy, n), frac_mul(m, frac(Sx, n)));
printf("B: n=%lld Sx=%lld Sy=%lld Sxy=%lld sumx2=%lld Sxx_centered=%lld\n", n,
       Sx, Sy, Sxy, Sxx, Sxxc);
printf("B: slope %lld/%lld = %.15f, intercept %lld/%lld = %.15f\n", m.num,
       m.den, frac_d(m), b.num, b.den, frac_d(b));
CHECK(frac_is(m, 4, 5), "ch12 B slope 4/5 exact");
CHECK(near(frac_d(m), 0.8), "ch12 B slope double 0.8 at 1e-12");
CHECK(frac_is(b, 3, 2) && frac_d(b) == 1.5, "ch12 B intercept 3/2 == 1.5");
```

listing 82: fixture B's slope and intercept, exact fraction plus tolerance double (source: kdd/samples/src/Ch12/ols.c, lines 145-154)

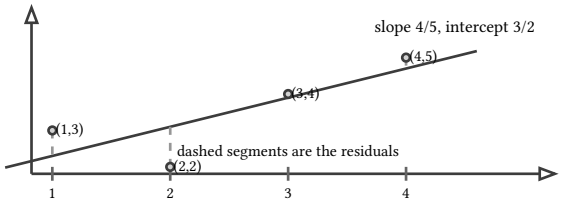


figure 47: fixture B, four points, the line y = 4x/5 + 3/2 through them

NOTE: two assertions, one number

The slope 4/5 is one number with two pins. The fraction pin cross-multiplies, exact forever; the double pin checks the printed 0.8 to 1e-12 because 0.8 is not representable in binary and the nearest double sits about 4.4 parts in 10^{17} away. The contract ledger measured the arithmetic-ordering spread across evaluation orders at 5.6 parts in 10^{17}, so the 1e-12 tolerance carries four orders of margin while staying tight enough to catch a wrong formula.

12.3. residuals, and the sum that must be zero

The residual $r_i = y_i - (mx_i + b)$ is the part of y_i the line misses. Two facts hold for every least squares fit, both forced by the normal equations rather than by luck: the residuals sum to zero, and they are orthogonal to the x values, $\sum_i r_i x_i = 0$. The sample checks the first on both fixtures and uses the second implicitly every time it reuses the same m and b.

The dry run: on fixture A the five residual lines print `A: residual x=1 0.000000000000000` through `x=5` all zero, and `A: residual sum 0.000000000000000`, asserted with == six times. On fixture B the sample scales by ten and stays in integers: the prediction at x is $(8x + 15)/10$, so it prints `B: 10*yhat(x=1) = 23` up to `B: 10*yhat(x=4) = 47`, and the residuals `B: 10*residual(x=1) = 7`, `= -11`, `= 1`, `= 3`, summing to zero, "ch12 B residual sum 0". The squared tenths sum $49 + 121 + 1 + 9 = 180$, so $SSE = 180/100 = 9/5$, printed as `B: SSE 9/5 = 1.800000000000000` and pinned both as a fraction and at 1e-12.

```
/* 10*yhat = 8x + 15 on integer arithmetic; residuals 10y - 8x - 15 */
int r10[NB], r10sum = 0;
for (int i = 0; i < NB; i++) {
  printf("B: 10*yhat(x=%d) = %d\n", xb[i], 8 * xb[i] + 15);
  r10[i] = 10 * yb[i] - 8 * xb[i] - 15;
  r10sum += r10[i];
  printf("B: 10*residual(x=%d) = %d\n", xb[i], r10[i]);
}
CHECK(8 * 1 + 15 == 23, "ch12 B 10*yhat(1) 23");
CHECK(8 * 2 + 15 == 31, "ch12 B 10*yhat(2) 31");
CHECK(8 * 3 + 15 == 39, "ch12 B 10*yhat(3) 39");
CHECK(8 * 4 + 15 == 47, "ch12 B 10*yhat(4) 47");
CHECK(r10[0] == 7, "ch12 B residual x=1 is 7");
CHECK(r10[1] == -11, "ch12 B residual x=2 is -11");
CHECK(r10[2] == 1, "ch12 B residual x=3 is 1");
CHECK(r10[3] == 3, "ch12 B residual x=4 is 3");
CHECK(r10sum == 0, "ch12 B residual sum 0");
```

listing 83: residuals in integer tenths, 10y - 8x - 15, summing to zero (source: kdd/samples/src/Ch12/ols.c, lines 156-172)

```
/* SSE = (49+121+1+9)/100 = 180/100 = 9/5 */
long long sq = 0;
for (int i = 0; i < NB; i++) {
  sq += r10[i] * r10[i];
}
Frac sse = frac(sq, 100);
printf("B: SSE %lld/%lld = %.15f\n", sse.num, sse.den, frac_d(sse));
CHECK(frac_is(sse, 9, 5), "ch12 B SSE 9/5 exact");
CHECK(near(frac_d(sse), 1.8), "ch12 B SSE double 1.8 at 1e-12");
```

listing 84: SSE from the squared tenths, 180 over 100, reduced to 9/5 (source: kdd/samples/src/Ch12/ols.c, lines 174-182)

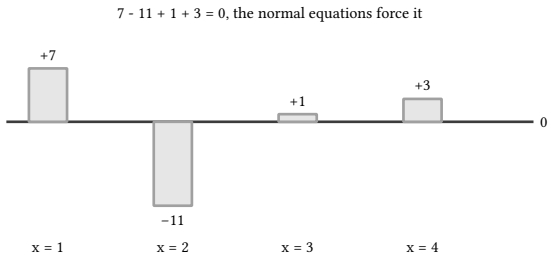


figure 48: the four fixture B residuals in tenths, positive and negative, sum zero

12.4. the decomposition, and R squared

Total scatter splits into explained and unexplained: $S_{yy} = SSR + SSE$, where SSR is the squared length the line accounts for, m^2 times the centered x scatter, and $R^2 = SSR/S_{yy}$ is the fraction of y's variance the line explains. Fixture A explains everything; fixture B explains 16/25 of it.

The dry run: fixture A prints `A: SSE 0.000000000000000 Syy 40/1 R2 1.000000000000000`, with $S_{yy} = (5 \cdot 285 - 1225)/5 = 40$ over y^2 values 9 + 25 + 49 + 81 + 121 = 285. Fixture B prints `B: Syy 5/1, SSR 16/5 = 3.200000000000000` and `B: R2 16/25 = 0.640000000000000`: $S_{yy} = (4 \cdot 54 - 196)/4 = 5$, $SSR = (16/25) \cdot 5 = 16/5$, and indeed $16/5 + 9/5 = 25/5 = 5$, the decomposition closing on itself, "ch12 B SSR 16/5 exact" and "ch12 B R2 16/25 exact" both cross-multiplied.

```
/* SSE = 0, Syy = (n*sum y^2 - Sy^2)/n = 40, R2 = 1 */
double sseA = 0.0;
long long sumy2 = 0;
for (int i = 0; i < NA; i++) {
  sseA += ra[i] * ra[i];
  sumy2 += ya[i] * ya[i];
}
Frac syyA = frac(n * sumy2 - Sy * Sy, n);
double r2A = 1.0 - sseA / frac_d(syyA);
printf("A: SSE %.15f Syy %lld/%lld R2 %.15f\n", sseA, syyA.num, syyA.den,
       r2A);
CHECK(sseA == 0.0, "ch12 A SSE 0 ==");
CHECK(frac_d(syyA) == 40.0, "ch12 A Syy 40 ==");
CHECK(r2A == 1.0, "ch12 A R2 1.0 ==");
```

listing 85: fixture A's SSE 0, Syy 40, and R2 1, all dyadic (source: kdd/samples/src/Ch12/ols.c, lines 121-134)

```
/* Syy = (4*54 - 196)/4 = 5; SSR = m^2 * Sxx_centered/n = 16/25 * 5 = 16/5 */
long long sy2 = 0;
for (int i = 0; i < NB; i++) {
  sy2 += yb[i] * yb[i];
}
Frac syy = frac(n * sy2 - Sy * Sy, n);
Frac ssr = frac_mul(frac_mul(m, m), frac(Sxxc, n));
printf("B: Syy %lld/%lld, SSR %lld/%lld = %.15f\n", syy.num, syy.den, ssr.num,
       ssr.den, frac_d(ssr));
CHECK(frac_d(syy) == 5.0, "ch12 B Syy 5 == 5.0");
CHECK(frac_is(ssr, 16, 5), "ch12 B SSR 16/5 exact");
```

listing 86: fixture B's Syy and SSR, exact fractions from the same m (source: kdd/samples/src/Ch12/ols.c, lines 184-194)

```
/* R2 = SSR/Syy = 16/25, double 0.64 at 1e-12 */
Frac r2 = frac_div(ssr, syy);
printf("B: R2 %lld/%lld = %.15f\n", r2.num, r2.den, frac_d(r2));
CHECK(frac_is(r2, 16, 25), "ch12 B R2 16/25 exact");
CHECK(near(frac_d(r2), 0.64), "ch12 B R2 double 0.64 at 1e-12");
```

listing 87: R2 as the fraction SSR over Syy, 16/25, plus the double pin (source: kdd/samples/src/Ch12/ols.c, lines 196-200)

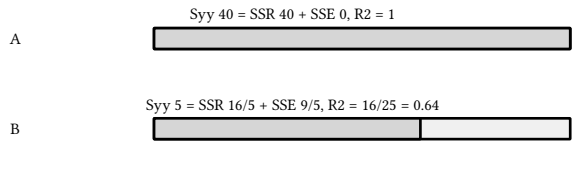


figure 49: variance explained, fixture A fills the bar, fixture B fills 16 of 25

NOTE: R2 is a report, not a grade

A perfect fit earned $R^2 = 1$ on five collinear points, and 0.64 on four noisy ones, but the number says nothing about whether the line is the right model, only how much of this sample's spread it absorbs. Two points always give $R^2 = 1$, any two, which is why the evaluation chapters judge models on held-out data, book 14, ch 17 (knowledge discovery, classifier evaluation), not on the quantity their fit minimized.

sources: all 35 pinned values, the reduced-fraction convention, and the D2 1e-12 tolerance ledger witnessed by kdd-contract-s3.md and playground/kdd-matrix/gen_s3.py, run 2026-09-22, exit 0. The gaussian-noise MLE justification for least squares is banked to the math book ch 19 (statistics) per the same sheet. Sample behavior

verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch12`, 35 checks in chapter 12 of the kdd suite.

13. decision trees: id3

One sample carries the chapter: `id3.c` grows a decision tree on the 14-row play-tennis table and asserts 43 checks, entropy by entropy, gain by gain, down to a byte-exact serialization of the finished tree. The chapter makes 5 moves: the fixture and the impurity of its root, the entropy of every attribute value's row set, information gain as the split criterion, the recursion and the tree it produces, and the tree's verdicts on rows it never saw. Every behavioral claim below is one of the 43 checks of chapter 13's sample or the banked Mitchell provenance. The same table returns in book 14, ch 14 (knowledge discovery, decision trees: cart and continuous splits) under a different impurity measure, its tree becomes rules in book 14, ch 15 (knowledge discovery, rules from trees and sequential covering), and book 14, ch 16 (knowledge discovery, overfitting and pruning) cuts one back.

13.1. the fourteen rows, and one impure root

The fixture is the play-tennis dataset Quinlan introduced in 1986, transcribed character for character, all six columns lowercased, from Mitchell's Table 3.2, page 59: 14 days, four categorical attributes (outlook, temperature, humidity, wind), one class, 9 days that play and 5 that do not. ID3, Quinlan's iterative dichotomizer, turns that table into a tree by splitting on whichever attribute most reduces the class impurity, here measured in bits, $H = -p \log_2 p - (1-p) \log_2 (1-p)$ with p the yes fraction.

The dry run: the census prints `class counts no=5 yes=9`, and the root entropy line prints the whole derivation, `H(S) = -(9/14)lg2(9/14)-(5/14)lg2(5/14) = 0.940285958670631`. That is 0.940 to the three decimals Mitchell prints in his chapter 3 walkthrough, the first of the six worked numbers this chapter reproduces. The mass fractions ride along in the print because they are the hand-checkable part, 9/14 and 5/14, while the decimal is a `log2` of non-dyadic fractions and is pinned at 1e-12.

```
/* rows: value index per attribute (header order), then class index.
 * day 1..14 exactly as Mitchell Table 3.2:
 * 1 sunny/hot/high/weak->no, 2 sunny/hot/high/strong->no,
 * 3 overcast/hot/high/weak->yes, 4 rain/mild/high/weak->yes,
 * 5 rain/cool/normal/weak->yes, 6 rain/cool/normal/strong->no,
 * 7 overcast/cool/normal/strong->yes, 8 sunny/mild/high/weak->no,
 * 9 sunny/cool/normal/weak->yes, 10 rain/mild/normal/weak->yes,
 * 11 sunny/mild/normal/strong->yes, 12 overcast/mild/high/strong->yes,
 * 13 overcast/hot/normal/weak->yes, 14 rain/mild/high/strong->no */
static const unsigned char T[NR][NA + 1] = {
    {0, 0, 0, 0, 0}, {0, 0, 0, 1, 0}, {1, 0, 0, 0, 1}, {2, 1, 0, 0, 1},
    {2, 2, 1, 0, 1}, {2, 2, 1, 1, 0}, {1, 2, 1, 1, 1}, {0, 1, 0, 0, 0},
    {0, 2, 1, 0, 1}, {2, 1, 1, 0, 1}, {0, 1, 1, 1, 1}, {1, 1, 0, 1, 1},
    {1, 0, 1, 0, 1}, {2, 1, 0, 1, 0},
```

listing 88: the 14 rows as value indices, Mitchell's table in scan order (source: kdd/samples/src/Ch13/id3.c, lines 44-58)

```
  /* [ch13.r03] root entropy, exact masses 9/14 and 5/14 alongside */
  double hs = h_bits(rows, NR);
  printf("H(S) = -(9/14)lg2(9/14)-(5/14)lg2(5/14) = %.15f\n", hs);
  CHECK(near(hs, 0.940285958670631), "ch13 root entropy 0.940285958670631");
```

listing 89: root entropy, masses printed beside the pinned decimal (source: kdd/samples/src/Ch13/id3.c, lines 225-228)

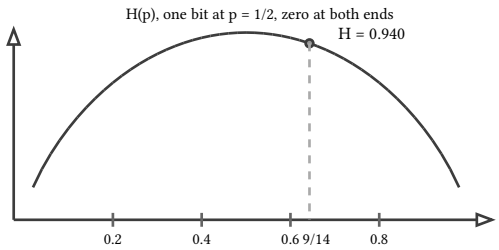


figure 50: the binary entropy curve, the root's 9 of 14 sits near the peak

NOTE: the decimals are pinned, the fractions are checked

An entropy like 0.940285958670631 contains log2 of 9/14, which no finite binary expansion holds, so the sample pins it at 1e-12 and prints the exact mass fractions, 9/14 and 5/14, on the same line. The clean cases assert harder: an empty class or a 50/50 mix gives exactly 0 or exactly 1, both dyadic, both compared with ==. The same two-tier discipline carried the OLS fractions of book 14, ch 12 (knowledge discovery, simple linear regression).

13.2. every branch has an entropy

Splitting the table on one attribute partitions the 14 rows into one row set per value of that attribute, and each set carries its own class mix and its own entropy. There are 10 such sets here, 3 for outlook, 3 for temperature, 2 for humidity, 2 for wind, and the sample counts and prices every one of them.

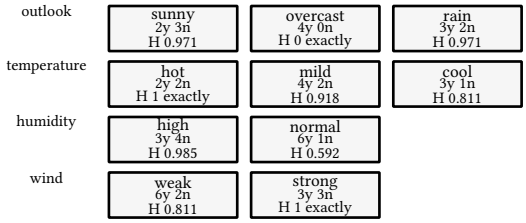
The dry run: outlook prints `split outlook=sunny: no=3 yes=2 (n=5)`, `split outlook=overcast: no=0 yes=4 (n=4)`, `split outlook=rain: no=2 yes=3 (n=5)`, and the entropies line up as `H(outlook=sunny) masses yes,no 2/5,3/5 = 0.970950594454669`, `H(outlook=overcast) masses yes,no 4/0 = 0.000000000000000`, sunny and rain tying at 0.970950594454669 because 2/5 against 3/5 is 3/5 against 2/5 worn backwards. Temperature gives `H(temperature=hot) masses yes,no 2/4,2/4 = 1.000000000000000` and strong the same perfect confusion, `H(wind=strong) masses yes,no 3/6,3/6 = 1.000000000000000`, both asserted with ==. The cheapest set in the table is `H(humidity=normal) masses yes,no 6/7,1/7 = 0.591672778582327`.

```c
/* two-class entropy in bits; zero counts skipped */
static double h_bits(const int *rows, int n) {
  int c[2] = {0, 0};
  for (int i = 0; i < n; i++) {
    c[T[rows[i]][NA]]++;
  }
  double s = 0.0;
  for (int k = 0; k < 2; k++) {
    if (c[k]) {
      double p = (double)c[k] / n;
      s -= p * log2(p);
    }
  }
  return s;
}
```

listing 90: two-class entropy in bits, zero counts skipped (source: kdd/samples/src/Ch13/id3.c, lines 61-75)

```
{
  const struct {
    int att, v;
    const char *mass;
    double pin;
    int dyadic; /* 1: assert == (0 or 1), 0: assert at 1e-12 */
  } want[10] = {
      {0, 0, "2/5,3/5", 0.970950594454669, 0},
      {0, 1, "4/0", 0.0, 1},
      {0, 2, "3/5,2/5", 0.970950594454669, 0},
      {1, 0, "2/4,2/4", 1.0, 1},
      {1, 1, "4/6,2/6", 0.918295834054490, 0},
      {1, 2, "3/4,1/4", 0.811278124459133, 0},
      {2, 0, "3/7,4/7", 0.985228136034252, 0},
      {2, 1, "6/7,1/7", 0.591672778582327, 0},
      {3, 0, "6/8,2/8", 0.811278124459133, 0},
      {3, 1, "3/6,3/6", 1.0, 1},
  };
```

listing 91: the ten pinned branch entropies, masses carried alongside (source: kdd/samples/src/Ch13/id3.c, lines 254-271)



counts are yes against no, as the sample prints them

figure 51: all ten branch sets, class mixes and entropies, overcast is free

13.3. information gain picks the root

Entropy alone does not rank attributes, because a partition into many small sets could be mostly impure yet cheap on average. Information gain weighs the children by size, $\text{Gain}(S, A) = H(S) - \sum_v (|S_v|/|S|) H(S_v)$, summing only over values present, and ID3 splits on the attribute with the largest gain.

The dry run: one line carries all four, `Gain(outlook)=0.246749819774439 Gain(temperature)=0.029222565658955 Gain(humidity)=0.151835501362342 Gain(wind)=0.048127030408270`. Hand-checked for outlook: 0.940 minus $(5/14) \cdot 0.971$ for sunny, zero for overcast, $(5/14) \cdot 0.971$ for rain, leaves 0.247. The ranking check fires as “ch13 ranking outlook > humidity > wind > temperature”. These are four more of Mitchell’s worked numbers, 0.247, 0.152, 0.048, 0.029, and the outlook root they elect is his.

```c
/* Gain(att) = H(rows) - sum over present values (global order) of
 * (|child|/n) * H(child) */
static double gain_ent(const int *rows, int n, int att) {
  double g = h_bits(rows, n);
  for (int v = 0; v < NV[att]; v++) {
    int sub[NR], m = 0;
    for (int i = 0; i < n; i++) {
      if (T[rows[i]][att] == v) {
        sub[m++] = rows[i];
      }
    }
    if (m) {
      g -= (double)m / n * h_bits(sub, m);
    }
  }
  return g;
}
```

listing 92: gain, parent entropy minus the size-weighted child entropies (source: kdd/samples/src/Ch13/id3.c, lines 84-100)

```c
  /* [ch13.r05] root information gains */
  double go = gain_ent(rows, NR, 0), gt = gain_ent(rows, NR, 1),
         gh = gain_ent(rows, NR, 2), gw = gain_ent(rows, NR, 3);
  printf("Gain(outlook)=%.15f Gain(temperature)=%.15f Gain(humidity)=%.15f "
         "Gain(wind)=%.15f\n",
         go, gt, gh, gw);
  CHECK(near(go, 0.246749819774439),
        "ch13 root gain outlook 0.246749819774439");
  CHECK(near(gh, 0.151835501362342),
        "ch13 root gain humidity 0.151835501362342");
  CHECK(near(gw, 0.048127030408270), "ch13 root gain wind 0.048127030408270");
  CHECK(near(gt, 0.029222565658955),
        "ch13 root gain temperature 0.029222565658955");

  /* [ch13.r06] gain ranking string */
  CHECK(go > gh && gh > gw && gw > gt,
        "ch13 ranking outlook > humidity > wind > temperature");
```

listing 93: the four root gains pinned and ranked in one check (source: kdd/samples/src/Ch13/id3.c, lines 290-306)

Mitchell's printed values under each bar

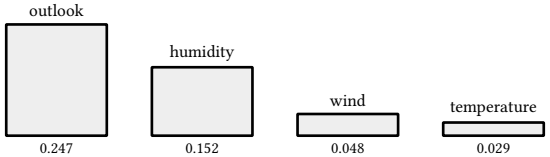


figure 52: the four root gains, outlook wins, temperature never helped

13.4. recursion, subtrees, and a byte-exact tree

The root chosen, the algorithm recurses into each row set with outlook spent. Both impure children land at entropy 0.970950594454669, and each elects a different second split. The finished tree is serialized in a fixed grammar, attribute first, branches in the order each value first appears scanning the whole table, leaves as `leaf:yes` or `leaf:no`, so the string is byte-comparable across programs.

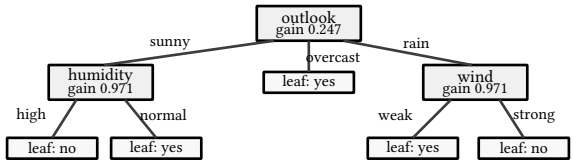
The dry run: under sunny, `Gain(humidity)=0.970950594454669` beats temperature's 0.570950594454669 and wind's 0.019973094021975, and humidity splits sunny perfectly. Under rain, wind gains 0.970950594454669 while temperature and humidity tie all the way down at 0.019973094021975, the tie itself a check, "ch13 rain tie temperature=humidity, wind wins". The recursion prints `id3 tree`
`outlook(sunny→humidity(high→leaf:no normal→leaf:yes)`
`overcast→leaf:yes rain→wind(weak→leaf:yes strong→leaf:no))`, asserted by `strcmp`, then `leaves 5 max depth 2`. This is the tree Mitchell's chapter 3 grows from the same table, in executable form.

```c
static int build(const int *rows, int n, unsigned used) {
  int id = np++;
  int c[2];
  class_counts(rows, n, c);
  if (c[0] == 0 || c[1] == 0) {
    pool[id] = (Node){.leaf = 1, .cls = c[0] ? 0 : 1};
    return id;
  }
  int best = -1;
  double bg = 0.0;
  for (int a = 0; a < NA; a++) { /* header order is the tie-break */
    if (used & (1u << a)) {
      continue;
    }
    double g = gain_ent(rows, n, a);
    if (best < 0 || g > bg) {
      best = a;
      bg = g;
    }
  }
  if (best < 0) { /* no attributes left: majority, tie -> lex smaller class */
    pool[id] = (Node){.leaf = 1, .cls = c[0] ≥ c[1] ? 0 : 1};
    return id;
  }
  Node nd = {.leaf = 0, .att = best, .nbr = 0};
```

listing 94: the recursion's fork, pure leaf, best attribute, header-order ties (source: kdd/samples/src/Ch13/id3.c, lines 109-133)

```c
/* paren grammar v1: leaf:cls | att(val->child val->child ...) */
static void ser_append(int id, char *buf) {
  size_t len = strlen(buf);
  if (pool[id].leaf) {
    sprintf(buf + len, "leaf:%s", CLS[pool[id].cls]);
    return;
  }
  len += sprintf(buf + len, "%s(", ATT[pool[id].att]);
  for (int b = 0; b < pool[id].nbr; b++) {
    if (b) {
      buf[len++] = ' ';
    }
    len += sprintf(buf + len, "%s->", VALS[pool[id].att][pool[id].bv[b]]);
    ser_append(pool[id].kid[b], buf);
    len = strlen(buf);
  }
  buf[len++] = ')';
  buf[len] = '\0';
}
```

listing 95: grammar v1 serialization, branch order fixed by first appearance (source: kdd/samples/src/Ch13/id3.c, lines 151-169)



5 leaves, max depth 2 with the root at depth 0

figure 53: the learned tree, five leaves, depth 2, gains at each split

13.5. the tree on rows it has never seen

A tree is judged by classification. Walking it is a per-node branch lookup, and two constructed rows exercise paths the growth never explicitly chose: their attribute values all exist in the table, but the combinations do not.

The dry run: on the 14 training rows the tree scores `training accuracy 14/14`, which is guaranteed for ID3 on categorical data grown to purity and is still worth the check. The held-out line prints `held-out rain/mild/high/weak -> yes; sunny/cool/high/weak ->
no`: the first row takes rain then wind weak then yes, the second takes sunny then humidity high then no, temperature never consulted in either. A row carrying a value absent at its node would return the unclassifiable marker, the sample's −1, which is the multiway tree's honest answer that it never saw that branch, and the reason book 14, ch 3 (knowledge discovery, binning, smoothing, and discretization) exists for continuous inputs and book 14, ch 14 (knowledge discovery, decision trees: cart and continuous splits) for splits that do not need categories at all.

```
static int classify(int id, const unsigned char *vals) {
  while (!pool[id].leaf) {
    int next = -1;
    for (int b = 0; b < pool[id].nbr; b++) {
      if (pool[id].bv[b] == vals[pool[id].att]) {
        next = pool[id].kid[b];
        break;
      }
    }
    if (next < 0) {
      return -1; /* value absent at this node: unclassifiable */
    }
    id = next;
  }
  return pool[id].cls;
}
```

listing 96: the walker, branch lookup per node, −1 on an absent value (source: kdd/samples/src/Ch13/id3.c, lines 171-186)

```
  {
    const unsigned char q1[NA] = {2, 1, 0, 0}; /* rain mild high weak */
    const unsigned char q2[NA] = {0, 2, 0, 0}; /* sunny cool high weak */
    const char *a1 = CLS[classify(root, q1)], *a2 = CLS[classify(root, q2)];
    printf("held-out rain/mild/high/weak -> %s; sunny/cool/high/weak -> %s\n",
           a1, a2);
    CHECK(strcmp(a1, "yes") == 0, "ch13 held-out rain mild high weak yes");
    CHECK(strcmp(a2, "no") == 0, "ch13 held-out sunny cool high weak no");
  }
```

listing 97: the two constructed rows and their verdicts (source: kdd/samples/src/Ch13/id3.c, lines 385-393)

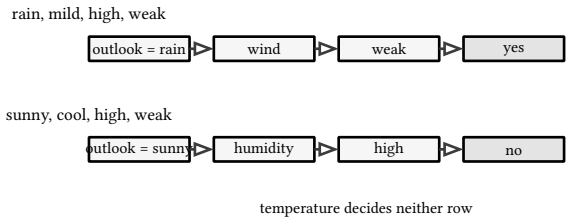


figure 54: two held-out rows walked through the tree, two different paths

PITFALL: training accuracy is a tautology, not a result

A purity-grown multiway tree classifies its own training rows perfectly by construction, every leaf pure, so 14/14 says the walk and the growth agree and nothing more. The two held-out rows are the first honest test in the chapter, two rows out of two, and the real machinery for that judgment, held-out sets, cross validation, and the error rates that come out of them, is book 14, ch 17 (knowledge discovery, classifier evaluation). Overfitting gets its own fixture in book 14, ch 16 (knowledge discovery, overfitting and pruning).

sources: the play-tennis fixture, its 14 rows, and the six reproduced worked numbers 0.940, 0.971, 0.247, 0.152, 0.048, 0.029 are from Mitchell, Machine Learning, McGraw-Hill 1997, Table 3.2 p. 59 and the chapter 3 walkthrough, the dataset introduced by Quinlan, “Induction of Decision Trees”, Machine Learning 1:81-106, 1986. All 43 pinned values witnessed by kdd-contract-s3.md and playground/kdd-matrix/gen_s3.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch13`, 43 checks in chapter 13 of the kdd suite.

14. decision trees: cart and continuous splits

One sample carries the chapter: `cart.c` regrows the tree of book 14, ch 13 (knowledge discovery, decision trees: id3) under the gini impurity instead of entropy, then leaves categoricals behind and searches continuous thresholds on an 8-row fixture, 40 checks in all, every gini value an exact rational asserted by cross-multiplication. The chapter makes 4 moves: gini as the impurity that needs no logarithm, the delta criterion and the surprising byte-identical tree, midpoint thresholds on a continuous axis, and why real CART is binary. Every behavioral claim below is one of the 40 checks of chapter 14′s sample or a named note of the contract sheet.

14.1. gini, impurity without logarithms

Gini impurity is the probability that two rows drawn with replacement from a set disagree on class, $1 - \sum_i p_i^2$, and for two classes it simplifies to $2p(1-p)$ with p the yes fraction. Where entropy needed `log2` and tolerance pins, gini over integer class counts is the exact fraction $\left(n^2 - k_{\text{yes}}^2 - k_{\text{no}}^2\right)/n^2$, reducible with a gcd and comparable by cross-multiplication.

The dry run: the root prints `gini(S) 45/98 = 0.459183673469388`, hand-derived as $1 - (9/14)^2 - (5/14)^2 = 1 - 81/196 - 25/196 = 90/196 = 45/98$. All ten attribute values follow, `gini(outlook=sunny) 12/25 = 0.480000000000000`, `gini(outlook=overcast) 0/1 = 0.000000000000000` because 4 yes against 0 no cannot disagree, `gini(temperature=hot) 1/2 = 0.500000000000000` because 2 against 2 disagrees half the time, down to `gini(humidity=normal) 12/49 = 0.244897959183673`, the cheapest set in the table exactly as entropy judged it. Ten checks, every one the string “ch14 per-value gini exact”.

```
/* gini = (n^2 - sum k_i^2)/n^2, exact */
static Frac gini(const int *rows, int n) {
  int c[2];
  class_counts(rows, n, c);
  return frac((long long)n * n -
                  ((long long)c[0] * c[0] + (long long)c[1] * c[1]),
              (long long)n * n);
}
```

listing 98: gini over class counts, an exact fraction, no logarithm (source: kdd/samples/src/Ch14/cart.c, lines 101-108)

```
    const struct {
      int att, v;
      long long p, q;
    } want[10] = {
        {0, 0, 12, 25}, {0, 1, 0, 1}, {0, 2, 12, 25}, {1, 0, 1, 2},
        {1, 1, 4, 9},   {1, 2, 3, 8}, {2, 0, 24, 49}, {2, 1, 12, 49},
        {3, 0, 3, 8},   {3, 1, 1, 2},
    };
    for (int k = 0; k < 10; k++) {
      int sub[NR], m = 0;
      for (int i = 0; i < NR; i++) {
        if (T[i][want[k].att] == want[k].v) {
          sub[m++] = i;
        }
      }
      Frac g = gini(sub, m);
      printf("gini(%s=%s) %lld/%lld = %.15f\n", ATT[want[k].att],
             VALS[want[k].att][want[k].v], g.num, g.den, frac_d(g));
      CHECK(frac_is(g, want[k].p, want[k].q), "ch14 per-value gini exact");
    }
```

listing 99: the ten per-value ginis, pinned as fractions and cross-multiplied (source: kdd/samples/src/Ch14/cart.c, lines 221-240)

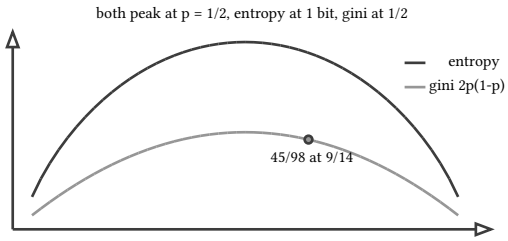


figure 55: entropy and gini against the yes fraction, same shape, different peak

NOTE: same verdicts, cheaper arithmetic

Entropy and gini order these ten branch sets identically, normal at 0.592 bits and 12/49 the cheapest, hot and strong the most confused, overcast free. That agreement is typical rather than guaranteed, the two functions are close cousins on $[0, 1]$, but the chapter never leans on it: every ordering here is its own check, and the tree below is compared byte for byte, not attribute by attribute.

14.2. the delta criterion, and one tree twice

The split criterion swaps entropy for gini, $\text{Delta}(S, A) = \text{gini}(S) - \sum_v (|S_v|/|S|)\ \text{gini}(S_v)$, the same size-weighted children minus parent shape, now entirely in fractions. The fixture and the recursion are the same as chapter 13's, so what falls out is a direct experiment: does the yardstick change the tree?

The dry run: the root line prints `Delta(outlook)=57/490` `Delta(temperature)=11/588` `Delta(humidity)=9/98` `Delta(wind)=3/98`, outlook hand-derived as $45/98 - (5/14)(12/25) - 0 - (5/14)(12/25) = 45/98 - 12/35 = 57/490$. The ranking check fires as "ch14 ranking outlook > humidity > wind > temperature", the same string as id3's. The subtrees agree too, `sunny gini 12/25: humidity 12/25 temperature 7/25` `wind 1/75` and `rain gini 12/25: wind 12/25 temperature 1/75 humidity` `1/75`. So the tree prints `cart tree outlook(sunny->humidity(high->leaf:no` `normal->leaf:yes) overcast->leaf:yes rain->wind(weak->leaf:yes` `strong->leaf:no))`, and check 26 is the payoff: "ch14 cart paren == id3 paren (cross-file identity)". Two programs written apart, two impurity functions with different arithmetic, one byte-identical string.

```c
/* Delta(att) = gini(rows) - sum over present values (global order) of
 * (|child|/n) * gini(child), exact */
static Frac gain_gini(const int *rows, int n, int att) {
  Frac g = gini(rows, n);
  for (int v = 0; v < NV[att]; v++) {
    int sub[NR], m = 0;
    for (int i = 0; i < n; i++) {
      if (T[rows[i]][att] == v) {
        sub[m++] = rows[i];
      }
    }
    if (m) {
      g = frac_sub(g, frac_mul(frac(m, n), gini(sub, m)));
    }
  }
  return g;
}
```

listing 100: the gini delta, parent minus size-weighted children, all fractions (source: kdd/samples/src/Ch14/cart.c, lines 110-126)

```c
  /* [ch14.r04] root gini gains */
  Frac go = gain_gini(rows, NR, 0), gt = gain_gini(rows, NR, 1),
       gh = gain_gini(rows, NR, 2), gw = gain_gini(rows, NR, 3);
  printf("Delta(outlook)=%lld/%lld Delta(temperature)=%lld/%lld "
         "Delta(humidity)=%lld/%lld Delta(wind)=%lld/%lld\n",
         go.num, go.den, gt.num, gt.den, gh.num, gh.den, gw.num, gw.den);
  CHECK(frac_is(go, 57, 490), "ch14 root delta outlook 57/490");
  CHECK(frac_is(gh, 9, 98), "ch14 root delta humidity 9/98");
  CHECK(frac_is(gw, 3, 98), "ch14 root delta wind 3/98");
  CHECK(frac_is(gt, 11, 588), "ch14 root delta temperature 11/588");

  /* [ch14.r05] ranking string */
  CHECK(frac_cmp(go, gh) > 0 && frac_cmp(gh, gw) > 0 && frac_cmp(gw, gt) > 0,
        "ch14 ranking outlook > humidity > wind > temperature");
```

listing 101: the four root deltas pinned as fractions and ranked (source: kdd/samples/src/Ch14/cart.c, lines 243-256)

```c
  /* [ch14.r07] cart tree paren: the pin AND the ch13 id3 pin, byte-identical */
  {
    int root = build(rows, NR, 0u);
    char buf[512] = "";
    ser_append(root, buf);
    const char *pin = "outlook(sunny->humidity(high->leaf:no normal->leaf:yes) "
                      "overcast->leaf:yes rain->wind(weak->leaf:yes "
                      "strong->leaf:no))";
    printf("cart tree %s\n", buf);
    CHECK(strcmp(buf, pin) == 0, "ch14 cart tree paren v1");
    CHECK(strcmp(buf, pin) == 0 &&
              strcmp(pin, "outlook(sunny->humidity(high->leaf:no "
                          "normal->leaf:yes) overcast->leaf:yes "
                          "rain->wind(weak->leaf:yes "
                          "strong->leaf:no))") == 0,
          "ch14 cart paren == id3 paren (cross-file identity)");
```

listing 102: the cart tree asserted twice, against its pin and against id3's (source: kdd/samples/src/Ch14/cart.c, lines 298-313)

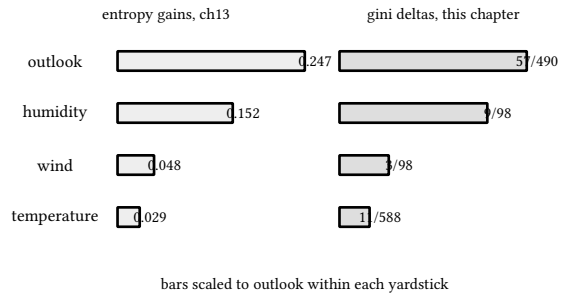


figure 56: the four attributes under both yardsticks, the order does not move

14.3. midpoints on a continuous axis

Categorical attributes hand you their split values. A continuous x does not, so CART sorts the values and proposes the midpoint between every neighboring pair, left branch $x \le t$, and scores each by the same delta. The fixture is 8 rows, x = 1, 2, 3, 6, 7, 9, 10, 15, with classes a, a, a, b, b, b, b, a, the last row planted so no prefix is pure all the way and the search must weigh.

The dry run: the parent prints `continuous parent gini 1/2`, classes 4 against 4. Seven candidates print, from `candidate x≤1.5: L a=1 b=0`
`R a=3 b=4 Delta 1/14 = 0.0714285714` to `candidate x≤12.5: L a=3 b=4`
`R a=1 b=0 Delta 1/14`. The winner is `candidate x≤4.5: L a=3 b=0`
`R a=1 b=4 Delta 3/10 = 0.3000000000`, hand-derived: left $\{1, 2, 3\}$ is all a, gini 0, right carries a = 1 against b = 4, gini 8/25, so delta $= 1/2 - (3/8) \cdot 0 - (5/8)(8/25) = 1/2 - 1/5 = 3/10$. The useless middle prints too, `candidate x≤9.5: L a=3 b=3 R a=1 b=1`
`Delta 0/1 = 0.0000000000`, both sides perfectly balanced and gini blind to it, and the two tails tie at 1/14, their own check, "ch14 candidates 1.5 and 12.5 tie at 1/14". The chosen line closes it, `chosen x≤4.5 delta 3/10; second x≤2.5 delta 1/6`.

```
    for (int k = 0; k < 7; k++) { /* k = 0..6: midpoint of CX[k], CX[k+1] */
      int la = 0, lb = 0, ra = 0, rb = 0;
      for (int i = 0; i < NC; i++) {
        if (i ≤ k) {
          la += CC[i] == 0;
          lb += CC[i] == 1;
        } else {
          ra += CC[i] == 0;
          rb += CC[i] == 1;
        }
      }
      d[k] =
          frac_sub(pc, frac_add(frac_mul(frac(k + 1, NC), gini2(la, lb)),
                                frac_mul(frac(NC - k - 1, NC), gini2(ra, rb))));
      double t = (CX[k] + CX[k + 1]) / 2.0;
      printf("candidate x≤%.1f: L a=%d b=%d R a=%d b=%d Delta %lld/%lld = "
             "%.10f\n",
             t, la, lb, ra, rb, d[k].num, d[k].den, frac_d(d[k]));
      CHECK(t == thr[k] && frac_is(d[k], pn[k], pd[k]),
            "ch14 continuous candidate delta exact");
    }
```

listing 103: one candidate per neighboring midpoint, counts, delta, and pin (source: kdd/samples/src/Ch14/cart.c, lines 327-347)

```
    int la = 0, lb = 0, ra = 0, rb = 0;
    for (int i = 0; i < NC; i++) {
      if (i ≤ bi) {
        la += CC[i] == 0;
        lb += CC[i] == 1;
      } else {
        ra += CC[i] == 0;
        rb += CC[i] == 1;
      }
    }
    double bt = (CX[bi] + CX[bi + 1]) / 2.0;
    double st = (CX[si] + CX[si + 1]) / 2.0;
    printf("chosen x≤%.1f delta %lld/%lld; second x≤%.1f delta %lld/%lld\n",
           bt, d[bi].num, d[bi].den, st, d[si].num, d[si].den);
    CHECK(bt == 4.5, "ch14 chosen threshold 4.5 dyadic ==");
    CHECK(frac_is(d[bi], 3, 10), "ch14 chosen delta 3/10");
    CHECK(la == 3 && lb == 0, "ch14 chosen left branch a=3 b=0");
    CHECK(ra == 1 && rb == 4, "ch14 chosen right branch a=1 b=4");
    CHECK(st == 2.5 && frac_is(d[si], 1, 6), "ch14 second best 2.5 at 1/6");
```

listing 104: the argmax split's branch counts, threshold, delta, and runner-up (source: kdd/samples/src/Ch14/cart.c, lines 364-382)

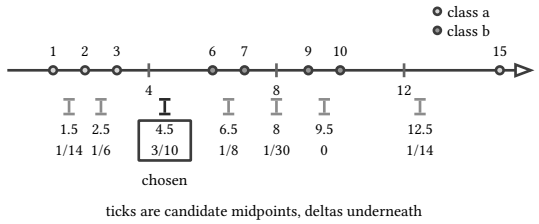


figure 57: the 8-row axis, 7 midpoint candidates, 4.5 wins with 3/10

14.4. binary, the real cart

Everything above 3-valued outlook split three ways because the sheet pinned the multiway convention to keep chapters 13 and 14 comparable on one fixture. CART itself is binary everywhere: a numeric attribute splits at a threshold, a k-valued categorical splits into a subset against its complement, and a binary question never has an answer the tree has not accounted for.

The dry run: the chosen split routes every real number, 4.5 sends $\{1, 2, 3\}$ left and everything else right, and the branch census is already pinned, left a = 3 against b = 0, right a = 1 against b = 4. Contrast the categorical hole of chapter 13, where a row carrying an unseen value returns the unclassifiable −1: an $x \leq t$ test has no unseen answer. The cost is search, a k-valued attribute offers $2^{k-1} - 1$ distinct subset splits, three for outlook, and a continuous attribute offers one candidate per neighboring gap, the seven the previous facet scored. book 14, ch 3 (knowledge discovery, binning, smoothing, and discretization) buys the same continuity by destroying it first, chopping x into categories; the threshold keeps x whole. book 14, ch 15 (knowledge discovery, rules from trees and sequential covering) will read rules off this tree, and book 14, ch 16 (knowledge discovery, overfitting and pruning) will ask whether any of it should have been grown at all.

```c
/* continuous fixture: 8 rows, x ascending, classes a=0 b=1 */
#define NC 8
static const int CX[NC] = {1, 2, 3, 6, 7, 9, 10, 15};
static const int CC[NC] = {0, 0, 0, 1, 1, 1, 1, 0};

/* gini of a class-count pair (a-count, b-count), exact */
static Frac gini2(long long ka, long long kb) {
  long long n = ka + kb;
  return frac(n * n - ka * ka - kb * kb, n * n);
}
```

listing 105: the continuous fixture and the count-pair gini helper (source: kdd/samples/src/Ch14/cart.c, lines 197-206)

```c
  int best = -1;
  Frac bg = {0, 1};
  for (int a = 0; a < NA; a++) { /* header order is the tie-break */
    if (used & (1u << a)) {
      continue;
    }
    Frac g = gain_gini(rows, n, a);
    if (best < 0 || frac_cmp(g, bg) > 0) {
      best = a;
      bg = g;
    }
  }
  if (best < 0) { /* majority leaf, tie -> lex smaller class */
    pool[id] = (Node){.leaf = 1, .cls = c[0] >= c[1] ? 0 : 1};
    return id;
  }
```

listing 106: the argmax over attributes, fraction comparison, header-order ties (source: kdd/samples/src/Ch14/cart.c, lines 143-158)

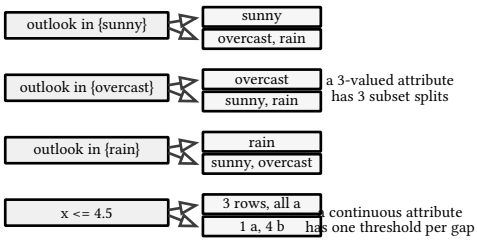


figure 58: binary encodings: three subset splits for outlook, one threshold for x

PITFALL: the multiway tree has holes, the binary tree has none

Chapter 13's walker returned −1 for a value absent at a node, and the fixture dodged it by construction. A threshold $x \le t$ cannot dodge, cannot miss: every real number answers yes or no. That robustness, not raw accuracy, is a large part of why production tree implementations descend from CART's binary discipline, and it is why this book's own pruning fixture of book 14, ch 16 (knowledge discovery, overfitting and pruning) grows multiway for pedagogy but evaluates on values the table contains.

sources: fixture 1 is the play-tennis table of Mitchell, Machine Learning, McGraw-Hill 1997, Table 3.2 p. 59, provenance carried from chapter 13 (dataset introduced by Quinlan 1986). The multiway gini convention, the exact-rational arithmetic, the continuous fixture, and the cross-file paren identity are kdd-contract-s3.md rows witnessed by playground/kdd-matrix/gen_s3.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch14`, 40 checks in chapter 14 of the kdd suite.

15. rules from trees and sequential covering

Two samples carry the chapter: `rulex.c` walks the tree of book 14, ch 13 (knowledge discovery, decision trees: id3) leaf by leaf and reads one IF-THEN rule off each path, 17 checks, and `seqcov.c` grows rules from scratch by sequential covering on a 10-row fixture, 40 checks. The chapter makes 4 moves: extraction and the partition it inherits, greedy growth one condition at a time, the Laplace prune that demonstrably fires, and the ordered evaluation where one row goes wrong on purpose. Every behavioral claim below is one of the 57 checks of chapter 15's samples or a named note of the contract sheet.

15.1. one rule per leaf

A decision tree already is a set of rules, one per root-to-leaf path: the conditions along the path anded together, the leaf's class as the consequent. Extraction walks the tree depth first, so rule order is leaf order, and each rule's conditions come out in the order the tree tested them.

The dry run: five rules print in order, `rule 1: IF outlook=sunny AND humidity=high THEN no` down to `rule 5: IF outlook=rain AND wind=strong THEN no`, one per leaf of the 5-leaf tree. Coverage and correctness print per rule, `rule 1: coverage 3 correct 3`, `rule 2: coverage 2 correct 2`, `rule 3: coverage 4 correct 4`, and so on, every rule perfect on the training set, and the census closes the argument: `total coverage 14`, the check named "ch15 rules partition 14 training rows". The ordered list then classifies `ordered-list accuracy 14/14`, though for these rules the order is decoration, no row matches two of them.

```
static void extract(int id, Rule *path) {
  if (pool[id].leaf) {
    rules[nrules] = *path;
    rules[nrules].cls = pool[id].cls;
    nrules++;
    return;
  }
  for (int b = 0; b < pool[id].nbr; b++) {
    Rule down = *path;
    down.att[down.nc] = pool[id].att;
    down.val[down.nc] = pool[id].bv[b];
    down.nc++;
    extract(pool[id].kid[b], &down);
  }
}
```

listing 107: depth-first extraction, the path accumulated down the tree (source: kdd/samples/src/Ch15/rulex.c, lines 138-152)

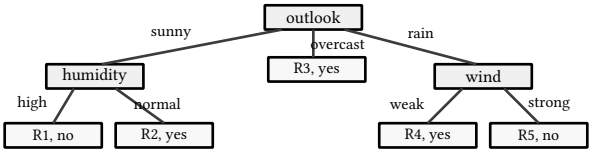
```
static void rule_str(const Rule *r, char *buf) {
  int len = sprintf(buf, "IF ");
  for (int k = 0; k < r->nc; k++) {
    len += sprintf(buf + len, "%s%s=%s", k ? " AND " : "", ATT[r->att[k]],
                   VALS[r->att[k]][r->val[k]]);
  }
  sprintf(buf + len, " THEN %s", CLS[r->cls]);
}
```

listing 108: the IF-THEN grammar, conditions in path order (source: kdd/samples/src/Ch15/rulex.c, lines 163-170)

```
{
  const int want_cov[5] = {3, 2, 4, 3, 2};
  const int want_ok[5] = {3, 2, 4, 3, 2};
  int total_cov = 0;
  for (int i = 0; i < 5; i++) {
    int cov = 0, ok = 0;
    for (int r = 0; r < NR; r++) {
      if (rule_matches(&rules[i], T[r])) {
        cov++;
        ok += T[r][NA] == (unsigned char)rules[i].cls;
      }
    }
    printf("rule %d: coverage %d correct %d\n", i + 1, cov, ok);
    CHECK(cov == want_cov[i], "ch15 rule coverage");
    CHECK(ok == want_ok[i], "ch15 rule correct");
    total_cov += cov;
  }

  /* [ch15.r03] rules partition the 14 training rows */
  printf("total coverage %d\n", total_cov);
  CHECK(total_cov == 14, "ch15 rules partition 14 training rows");
```

listing 109: per-rule coverage and correctness, the partition census (source: kdd/samples/src/Ch15/rulex.c, lines 199-219)



rule 3 is one condition long, the others two

figure 59: the ch13 tree read as five rules, one per root-to-leaf path

15.2. grow, one condition at a time

Sequential covering builds rules without a tree: pick a class, start from the empty rule, greedily add the condition that best separates, accept, delete the rows it covers, repeat. The procedure here is the contract sheet's deterministic fixing of the ripper shape, grow then prune, after Cohen 1995 and Clark and Boswell 1991, and the fixture is 10 rows over attributes a, b, c with classes 4 yes against 6 no.

The dry run: classes come in ascending count order, printed as `class order: yes(4) then no(6)`. Growing for yes, step 1 prices every candidate with at least one yes hit, `step1 b=b2 p=3 n=0`, `step1 c=c1 p=4 n=1`, `step1 a=a1 p=1 n=1`, down to `step1 b=b1 p=1 n=4`; b=b2 and c=c1 tie at $p - n = 3$ and the lexicographic rule hands it to b=b2, the check "ch15 step1 winner b=b2 (lex over c=c1), p=0 excluded" naming both facts, the excluded b=b3, c=c2, c=c3 never carrying a yes row. One condition already pure, so `R1 IF b=b2 THEN yes covers 1 2 3`. Round 2 grows on the remaining rows 4 through 10: `step2 c=c1 p=1 n=1` wins, then on the pair $\{4, 5\}$ both a=a2 and b=b1 sit at (1, 0), `step3 a=a2 (1,0); b=b1 (1,0)`, and a=a2 wins the tie, growing `IF c=c1 AND a=a2 THEN yes`.

```
/* rows 1..10, attribute value indices then class index (no=0, yes=1) */
static const unsigned char T[NR][NA + 1] = {
    {0, 0, 0, 1}, /* 1  a1 b2 c1 yes */
    {1, 0, 0, 1}, /* 2  a3 b2 c1 yes */
    {2, 0, 0, 1}, /* 3  a4 b2 c1 yes */
    {3, 1, 0, 1}, /* 4  a2 b1 c1 yes */
    {1, 2, 0, 0}, /* 5  a3 b3 c1 no  */
    {3, 1, 1, 0}, /* 6  a2 b1 c2 no  */
    {3, 2, 2, 0}, /* 7  a2 b3 c3 no  */
    {0, 1, 1, 0}, /* 8  a1 b1 c2 no  */
    {1, 1, 1, 0}, /* 9  a3 b1 c2 no  */
    {2, 1, 2, 0}, /* 10 a4 b1 c3 no  */
};
```

listing 110: the 10-row fixture, 4 yes against 6 no, constructed on the sheet (source: kdd/samples/src/Ch15/seqcov.c, lines 44-56)

```
  int best = 0;
  for (int k = 1; k < st->n; k++) {
    const Cand *c = &st->c[k], *w = &st->c[best];
    char cs[16], ws[16];
    sprintf(cs, "%s=%s", ATT[c->att], VALS[c->att][c->v]);
    sprintf(ws, "%s=%s", ATT[w->att], VALS[w->att][w->v]);
    if (c->p - c->n > w->p - w->n ||
        (c->p - c->n == w->p - w->n && strcmp(cs, ws) < 0)) {
      best = k;
    }
  }
  if (st->n) {
    st->w_att = st->c[best].att;
    st->w_v = st->c[best].v;
  }
```

listing 111: the winner, max p minus n, lexicographic tie-break (source: kdd/samples/src/Ch15/seqcov.c, lines 120-134)

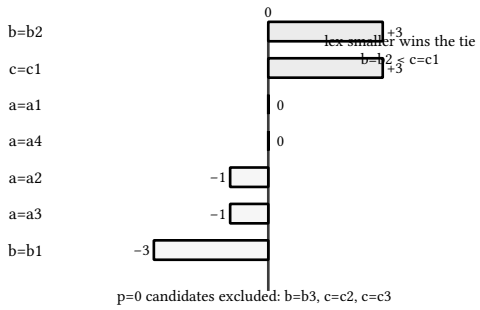


figure 60: step 1 candidates by p minus n, two tie at +3, lex picks b=b2

15.3. the laplace prune, and it fires

A rule grown to purity on a shrinking remainder overfits the remainder. The prune step scores a rule by the Laplace estimate $v = (p + 1)/(p + n + 2)$, computed over the full training set, not the remainder, and drops the last condition whenever the shorter rule scores at least as well.

The dry run: the grown rule is c=c1 AND a=a2, pure on the remaining rows but covering a single row of the full table. The sample prints `prune: v(rule)=2/3 v(prefix)=5/7 → DROP`: the two-condition rule matches only row 4, so $p = 1$, $n = 0$, $v = 2/3$, while the prefix c=c1 alone matches rows 1 through 4 plus row 5, $p = 4$, $n = 1$, $v = 5/7$. The comparison is exact cross-multiplication, the check “ch15 cross-mult 15/21 vs 14/21”, five sevenths beats two

thirds, and the condition goes. The class no needs no rule at all, `class no`: `remaining rows pure, no rule emitted`, so the final line prints `final: IF b=b2 THEN yes | IF c=c1 THEN yes | default:no`.

```c
/* Laplace v = (p+1)/(p+n+2) over the FULL training set, exact fraction */
typedef struct {
  long long n, d;
} Frac;

static Frac laplace(int cls, const int *ratt, const int *rval, int nc) {
  int all[NR], p, n;
  for (int i = 0; i < NR; i++) {
    all[i] = i;
  }
  pn_of(cls, all, NR, ratt, rval, nc, &p, &n);
  return (Frac){p + 1, p + n + 2};
}
```

listing 112: the Laplace estimate, exact fractions over the full 10 rows (source: kdd/samples/src/Ch15/seqcov.c, lines 146-158)

```c
    while (nc > 1) { /* prune from the last condition back */
      Frac vr = laplace(cls, ratt, rval, nc);
      Frac vp = laplace(cls, ratt, rval, nc - 1);
      int drop = vp.n * vr.d >= vr.n * vp.d;
      prunes[nprunes++] = (PruneT){vr.n, vr.d, vp.n, vp.d, drop};
      if (!drop) {
        break;
      }
      nc--;
    }
```

listing 113: prune from the last condition back, drop on cross-multiplied >= (source: kdd/samples/src/Ch15/seqcov.c, lines 224-233)

```c
  /* [ch15.r09] prune arithmetic on the FULL set, exact fractions */
  {
    const PruneT *pr = &prunes[0];
    printf("prune: v(rule)=%lld/%lld v(prefix)=%lld/%lld -> %s\n", pr->vr_n,
           pr->vr_d, pr->vp_n, pr->vp_d, pr->dropped ? "DROP" : "KEEP");
    CHECK(pr->vr_n == 2 && pr->vr_d == 3, "ch15 prune v(rule) 2/3 (p=1,n=0)");
    CHECK(pr->vp_n == 5 && pr->vp_d == 7, "ch15 prune v(prefix) 5/7 (p=4,n=1)");
    CHECK(pr->vp_n * 3 == 15 && pr->vr_n * 7 == 14,
          "ch15 cross-mult 15/21 vs 14/21");
    CHECK(pr->dropped == 1, "ch15 prune drops the last condition");
  }
```

listing 114: the pinned fractions and the drop, 2/3 loses to 5/7 (source: kdd/samples/src/Ch15/seqcov.c, lines 341-351)

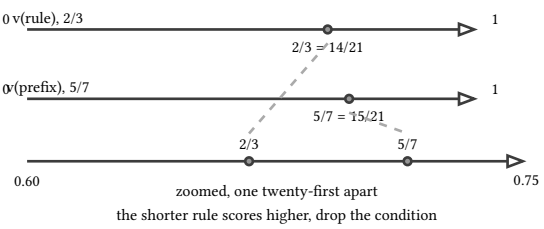


figure 61: two estimates of correctness, 2/3 against 5/7, zoomed at the bottom

NOTE: prune looks at all 10 rows, grow looked at 7

> Growth priced candidates on the shrinking remainder, rows 4 through 10, because covering is about what is left. The prune deliberately switches back to the full training set, where the two-condition rule covers one row and the one-condition rule covers five, and the Laplace plus-ones keep a single perfect match from scoring 1.0. That asymmetry is the sheet's rule and it is what makes the prune fire here: on the remainder alone, the prefix would have looked worse.

15.4. the ordered list, and the row that goes wrong

Extracted tree rules were mutually exclusive, each row walking exactly one leaf. Covering rules are not: rule 2, c=c1, overlaps rows the first rule already took, so the ruleset is an ordered decision list, first match wins, with a default class behind the last rule.

The dry run: the audit prints one line per row, `row 1: rule#1 → yes actual yes ok` through `row 3`, `row 4: rule#2 → yes actual yes ok`, then the planted failure, `row 5: rule#2 → yes actual no WRONG`, the a3 b3 c1 row that says no but carries c=c1. Rows 6 through 10 fall through both rules, `row 6: rule#0 → no actual no ok`, rule 0 meaning the default, five lines of it. The summary prints `ordered ruleset accuracy 9/10` and the full-set statistics explain the wound, `rule 2 full-set coverage 5 correct 4`: the rule was accepted for the one row it still covered and keeps the four it stole fairly.

```
  const int want_rule[NR] = {1, 1, 1, 2, 2, 0, 0, 0, 0, 0};
  const int want_pred[NR] = {1, 1, 1, 1, 1, 0, 0, 0, 0, 0}; /* cls idx */
  const int want_ok[NR] = {1, 1, 1, 1, 0, 1, 1, 1, 1, 1};
  int ok = 0;
  for (int i = 0; i < NR; i++) {
    int fired = 0, pred = defcls;
    for (int r = 0; r < nfinal; r++) {
      if (rule_hits(i, final_rules[r].att, final_rules[r].val,
                    final_rules[r].nc)) {
        fired = r + 1;
        pred = final_rules[r].cls;
        break;
      }
    }
    int good = pred == T[i][NA];
    ok += good;
    printf("row %d: rule#%d -> %s actual %s %s\n", i + 1, fired, CLS[pred],
           CLS[T[i][NA]], good ? "ok" : "WRONG");
    CHECK(fired == want_rule[i] && pred == want_pred[i] && good == want_ok[i],
          "ch15 ordered evaluation row");
  }
  printf("ordered ruleset accuracy %d/10\n", ok);
  CHECK(ok == 9, "ch15 ordered accuracy 9/10 (row 5 wrong)");
```

listing 115: the ordered walk, first rule wins, default behind them all (source: kdd/samples/src/Ch15/seqcov.c, lines 366-388)

```
/* [ch15.r15] full-set rule statistics */
for (int r = 0; r < nfinal; r++) {
  int cov = 0, corr = 0;
  for (int i = 0; i < NR; i++) {
    if (rule_hits(i, final_rules[r].att, final_rules[r].val,
                  final_rules[r].nc)) {
      cov++;
      corr += T[i][NA] == (unsigned char)final_rules[r].cls;
    }
  }
  printf("rule %d full-set coverage %d correct %d\n", r + 1, cov, corr);
  CHECK(cov == (r == 0 ? 3 : 5), "ch15 rule full-set coverage");
  CHECK(corr == (r == 0 ? 3 : 4), "ch15 rule full-set correct");
}
```

listing 116: full-set statistics, rule 2 covers 5 and gets 4 right (source: kdd/samples/src/Ch15/seqcov.c, lines 391-404)

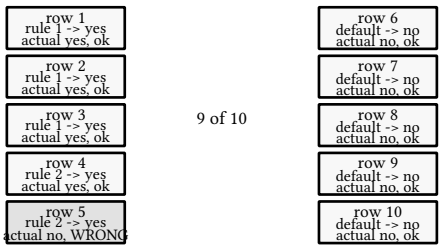


figure 62: the 10-row audit, rule fired, prediction, and the one wrong row

PITFALL: the tree's rules never overlap, covering rules must be ordered

The extraction sample classified 14 of 14 with no default because tree rules partition: a row walks one path. Covering rules grow greedily toward one class and stay greedy about others' rows, rule 2 fires yes on a no row, so the list order and the default are load-bearing parts of the hypothesis, not presentation. The same IF-THEN shape returns with no class at all as association rules in book 14, ch 25 (knowledge discovery, levelwise apriori), where the THEN side is another item, and the cost of rule complexity is judged by the machinery of book 14, ch 16 (knowledge discovery, overfitting and pruning).

sources: sequential covering procedure fixed by kdd-contract-s3.md after Cohen, "Fast Effective Rule Induction", ICML 1995, and Clark and Boswell, "Rule Induction with CN2", ECML 1991. Fixture 1 is the play-tennis table of Mitchell, Machine Learning, McGraw-Hill 1997, Table 3.2 p. 59, provenance carried from chapter 13. All 57 pinned values witnessed by kdd-contract-s3.md and playground/kdd-matrix/gen_s3.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch15`, 17 + 40 checks in chapter 15 of the kdd suite.

16. overfitting and pruning

One sample carries the chapter: `prune.c` grows a tree on 8 rows planted with 2 label flips, watches it ace its training set and miss its validation set, then cuts it back with reduced-error pruning until the validation verdict is perfect, 39 checks. The chapter makes 4 moves: the fixture and the overgrown tree, the pruning rounds one collapse at a time, what the pruning cost and bought, and pre-pruning by minsplit landing on the same tree by another road. Every behavioral claim below is one of the 39 checks of chapter 16′s sample or a named note of the contract sheet. The growth machinery is chapter 13′s, book 14, ch 13 (knowledge discovery, decision trees: id3), and the judgment that validation beats training accuracy is the whole subject of book 14, ch 17 (knowledge discovery, classifier evaluation).

16.1. a fixture planted to overfit

The true concept is one rule, x1 means yes and x2 means no, and the training set is that concept with two label flips, rows 4 and 8, so the flips are the only impurity a purity-hungry grower can chase. The attributes y and z are noise, but noise is exactly what depth can memorize: at the root they look useless, and one level down they look necessary.

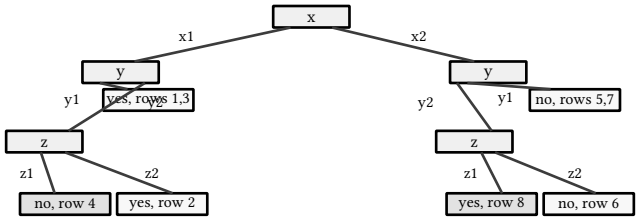
The dry run: the root prints `root H(S) = -(4/8)lg2(4/8)-(4/8)lg2(4/8) = 1.000000000000000`, four yes against four no, asserted with ==. The root gains print `root Gain(x)=0.188721875540867 Gain(y)=0.000000000000000 Gain(z)=0.000000000000000`: y and z split the root into balanced (2, 2) pairs both ways, gain exactly zero, while x separates into (3 yes, 1 no) on each side. Under either x branch the noise turns useful, `x=x1 node H(3,1)=0.811278124459133 Gain(y)=0.311278124459133 Gain(z)=0.311278124459133` → y, a perfect tie resolved to y by header order, asserted as “ch16 x2 gains y=z, tie -> y by header order”. The grown tree prints `T0 x(x1→y(y1→leaf:yes y2→z(z1→leaf:no z2→leaf:yes)) x2→y(y1→leaf:no y2→z(z1→leaf:yes z2→leaf:no)))`, and the score line is the whole story, `T0 train 8/8 validation 4/6`: the two z levels exist to encode the two flips.

```
/* train: rows 4 and 8 carry the planted flips */
static const unsigned char TR[NTR][NA + 1] = {
    {0, 0, 0, 1}, /* 1 x1 y1 z1 yes */
    {0, 1, 1, 1}, /* 2 x1 y2 z2 yes */
    {0, 0, 1, 1}, /* 3 x1 y1 z2 yes */
    {0, 1, 0, 0}, /* 4 x1 y2 z1 no  <- flip 1 */
    {1, 0, 0, 0}, /* 5 x2 y1 z1 no  */
    {1, 1, 1, 0}, /* 6 x2 y2 z2 no  */
    {1, 0, 1, 0}, /* 7 x2 y1 z2 no  */
    {1, 1, 0, 1}, /* 8 x2 y2 z1 yes <- flip 2 */
};
/* validation: noise-free */
static const unsigned char VA[NVA][NA + 1] = {
    {0, 0, 1, 1}, /* v1 x1 y1 z2 yes */
    {0, 1, 0, 1}, /* v2 x1 y2 z1 yes */
    {1, 0, 0, 0}, /* v3 x2 y1 z1 no  */
    {1, 1, 1, 0}, /* v4 x2 y2 z2 no  */
    {0, 1, 1, 1}, /* v5 x1 y2 z2 yes */
    {1, 1, 0, 0}, /* v6 x2 y2 z1 no  */
};
```

listing 117: 8 training rows with 2 planted flips, 6 noise-free validation rows (source: kdd/samples/src/Ch16/prune.c, lines 46-65)

```
/* [ch16.r04] x-node entropy and the y/z tie, both x1 and x2 */
for (int xv = 0; xv < 2; xv++) {
  int sub[NTR], m = 0;
  for (int i = 0; i < NTR; i++) {
    if (TR[i][0] == (unsigned char)xv) {
      sub[m++] = i;
    }
  }
  double h = h_bits(TR, sub, m), sy = gain_ent(sub, m, 1),
         sz = gain_ent(sub, m, 2);
  /* header-order scan: z replaces y only on a strict win, ties keep y */
  int best = sz > sy ? 2 /* z */ : 1 /* y */;
  printf("x=%s node H(3,1)=%.15f Gain(y)=%.15f Gain(z)=%.15f -> %s\n",
         VALS[0][xv], h, sy, sz, ATT[best]);
  if (xv == 0) {
    CHECK(near(h, 0.811278124459133),
          "ch16 x1 node entropy 0.811278124459133");
    CHECK(near(sy, 0.311278124459133) && near(sz, 0.311278124459133) &&
              near(sy - sz, 0.0),
          "ch16 x1 gains y=z 0.311278124459133");
  } else {
    CHECK(near(sy, 0.311278124459133) && near(sz, 0.311278124459133) &&
              best == 1,
          "ch16 x2 gains y=z, tie -> y by header order");
  }
}
```

listing 118: the y against z tie under each x branch, header order decides (source: kdd/samples/src/Ch16/prune.c, lines 296-321)



shaded leaves are the planted flips, rows 4 and 8

figure 63: T0, six leaves, the two z levels exist only to fit the flips

NOTE: noise looks useless at the root and load-bearing below it

Gain zero for y and z at the root is not evidence of irrelevance, it is an average. Each side of the x split holds one flip, and below x the noise attributes separate that flip perfectly, gain 0.311 each way. A purity criterion cannot tell memorization from structure, it only sees impurity falling, which is why the fix has to come from outside the training set.

16.2. three collapses, then a stop

Reduced-error pruning, Quinlan 1987, as fixed by the sheet: every internal node is a candidate to become a leaf labeled with the majority class of the training rows reaching it, the payoff is delta, validation-correct after the collapse minus before, and the winner is max delta with ties going deepest, then leftmost in serialization order; a collapse applies whenever delta ≥ 0.

The dry run: round 1 lists all five internal nodes, from `r1 cand x` → `leaf:no delta=-1 val 3/6` to three nodes tied at +1, and the deepest of them wins, “ch16 r1 deepest of +1 tie chosen”, `r1 cand x=x2/y=y2/z`

→ `leaf:no delta=1 val 5/6`, validation moving 4 to 5 of 6. Round 2 has a unique best, `r2 cand x=x1/y → leaf:yes delta=1 val 6/6`, the tree now perfect on validation. Round 3 still fires, `r3 cand x=x2/y → leaf:no delta=0 val 6/6`, because zero delta qualifies under the ≥ 0 rule, collapsing a node whose children already agree. Round 4 sees only the root at delta −3 and stops. The sequence prints `final tree x(x1→leaf:yes x2→leaf:no) (3 collapses)`, the true concept recovered.

```
  int base = correct_on(root, VA, NVA, -1, -1);
  for (int k = 0; k < cnt; k++) {
    Cand *cd = &rounds[nrounds][k];
    cd->val_after = correct_on(root, VA, NVA, cd->id, cd->maj);
    cd->delta = cd->val_after - base;
    printf("r%d cand %s -> leaf:%s delta=%d val %d/6\n", nrounds + 1,
           cd->label, CLS[cd->maj], cd->delta, cd->val_after);
  }
  int best = 0; /* first collected wins full ties (leftmost) */
  for (int k = 1; k < cnt; k++) {
    Cand *a = &rounds[nrounds][k], *b = &rounds[nrounds][best];
    if (a->delta > b->delta ||
        (a->delta == b->delta && a->depth > b->depth)) {
      best = k;
    }
  }
  round_chosen[nrounds] = best;
  if (rounds[nrounds][best].delta < 0) {
    break; /* nothing qualifies: REP stop */
  }
  root = apply_collapse(root, rounds[nrounds][best].id,
                        rounds[nrounds][best].maj);
  collapses++;
  nrounds++;
```

listing 119: one round: price every node, take max delta, deepest then leftmost (source: kdd/samples/src/Ch16/prune.c, lines 352-375)

```
/* [ch16.r07] candidate deltas across the three applied rounds */
{
  const struct {
    int rd;
    const char *label;
    int delta;
  } want[11] = {
      {0, "x=x2/y=y2/z", 1},  {0, "x=x1/y", 1},       {0, "x=x2/y", 1},
      {0, "x=x1/y=y2/z", -1}, {0, "x", -1},           {1, "x=x1/y", 1},
      {1, "x=x2/y", 0},       {1, "x=x1/y=y2/z", -1}, {1, "x", -2},
      {2, "x=x2/y", 0},       {2, "x", -3},
  };
  for (int k = 0; k < 11; k++) {
    const Cand *c = rc(want[k].rd, want[k].label);
    CHECK(c && c->delta == want[k].delta, "ch16 REP candidate delta");
  }
}
```

listing 120: the eleven pinned candidate deltas across the applied rounds (source: kdd/samples/src/Ch16/prune.c, lines 381-397)

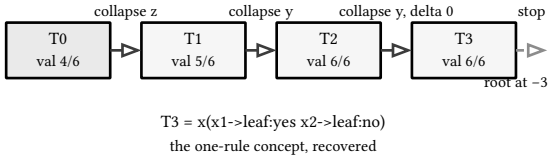


figure 64: the pruning rounds, validation climbing 4, 5, 6, 6 of 6, then stop

16.3. what the pruning cost, and what it bought

The pruned tree is worse where the overgrown one shined. T0 scored 8/8 on training because it memorized the flips; T3 scores 6 of 8 there, exactly the two flips it now gets wrong, and 6 of 6 on validation, exactly the noise it stopped chasing.

The dry run: the closing block prints `final train 6/8 validation 6/6`, with both fractions asserted by cross-multiplication, “ch16 final train fraction 3/4” and “ch16 final validation fraction 1”. Compare the opening `T0 train 8/8 validation 4/6`: training accuracy fell by a quarter, validation rose by a third, and the two training rows T3 misses are rows 4 and 8, the planted flips. The tree that admits two errors on data it saw is the one that makes none on data it did not.

```
static int correct_on(int root, const unsigned char (*tab)[NA + 1], int m,
                      int stop_id, int stop_cls) {
  int ok = 0;
  for (int i = 0; i < m; i++) {
    ok += classify_stop(root, tab[i], stop_id, stop_cls) == tab[i][NA];
  }
  return ok;
}
```

listing 121: accuracy on any table, an optional trial collapse built in (source: kdd/samples/src/Ch16/prune.c, lines 189-196)

```
  /* [ch16.r09] post-prune accuracy */
  {
    int tr_ok = correct_on(root, TR, NTR, -1, -1),
        va_ok = correct_on(root, VA, NVA, -1, -1);
    printf("final train %d/8 validation %d/6\n", tr_ok, va_ok);
    CHECK(tr_ok == 6, "ch16 final train correct 6");
    CHECK(tr_ok * 4 == 3 * 8, "ch16 final train fraction 3/4");
    CHECK(va_ok == 6, "ch16 final validation correct 6");
    CHECK(va_ok * 1 == 1 * 6, "ch16 final validation fraction 1");
  }
```

listing 122: the post-prune score, both fractions cross-multiplied (source: kdd/samples/src/Ch16/prune.c, lines 421-430)

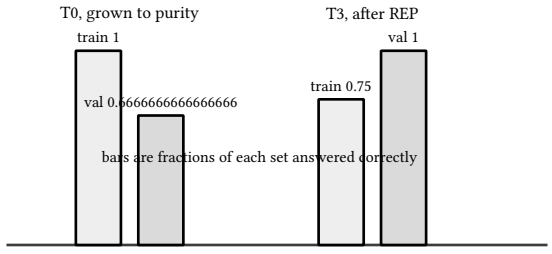


figure 65: train falls, validation rises, the trade is the point

PITFALL: the metric you prune by is the metric you keep

REP improves validation accuracy because it optimizes validation accuracy, and this fixture hands it a clean, noise-free set to optimize against. Prune against a noisy or tiny validation set and the same greedy loop will

> happily fit the noise in that set instead. The honest protocols hold out a third sample to score the final tree, the machinery of book 14, ch 17 (knowledge discovery, classifier evaluation), and book 14, ch 23 (knowledge discovery, bagging, boosting, and random forests) takes the opposite road entirely, averaging many overfit trees rather than cutting one back.

16.4. the same tree by another road

Post-pruning grows too much and cuts back. Pre-pruning refuses to grow: the sheet's rule splits a node only when it holds at least minsplit rows and at least two classes. On this fixture the z nodes see 2 rows each and the y nodes see 4, so minsplit 3 freezes the z level and minsplit 5 freezes the y level.

The dry run: minsplit 3 prints `minsplit=3 x(x1→y(y1→leaf:yes y2→leaf:no) x2→y(y1→leaf:no y2→leaf:no)) train 6/8`, an intermediate tree that already matches T3′s training score without ever seeing the validation set. Minsplit 5 prints `minsplit=5 x(x1→leaf:yes x2→leaf:no) train 6/8`, and check 39 is the pinned cross-check, "ch16 minsplit=5 paren == REP final paren": two code paths that never share a line of pruning logic, a growth-time row count and a validation-driven collapse loop, producing byte-identical strings. The anchor is deliberate, planted by the contract sheet so the two halves of this chapter cannot drift apart silently.

```
if (best < 0 || n < minsplit) { /* majority leaf, tie -> lex smaller class */
  pool[id] = (Node){.leaf = 1, .cls = c[0] >= c[1] ? 0 : 1};
  return id;
}
```

listing 123: the pre-pruning gate, too few rows means a majority leaf (source: kdd/samples/src/Ch16/prune.c, lines 127-130)

```
/* [ch16.r10] pre-pruning: minsplit 3 and 5, plus the anchor that the
 * minsplit=5 tree equals the REP result across the two code paths */
{
  const int ms[2] = {3, 5};
  const char *pin[2] = {
      "x(x1->y(y1->leaf:yes y2->leaf:no) x2->y(y1->leaf:no y2->leaf:no))",
      "x(x1->leaf:yes x2->leaf:no)"};
  char b[2][256] = {{0}, {0}};
  for (int k = 0; k < 2; k++) {
    np = 0;
    int t = build(rows, NTR, 0u, ms[k]);
    ser_append(t, b[k]);
    int ok = correct_on(t, TR, NTR, -1, -1);
    printf("minsplit=%d %s train %d/8\n", ms[k], b[k], ok);
    CHECK(strcmp(b[k], pin[k]) == 0, "ch16 minsplit paren");
    CHECK(ok == 6, "ch16 minsplit train 6/8");
  }
  CHECK(strcmp(b[1], tf) == 0, "ch16 minsplit=5 paren == REP final paren");
}
```

listing 124: minsplit 3 and 5 regrown, and the identity with the REP result (source: kdd/samples/src/Ch16/prune.c, lines 432-450)

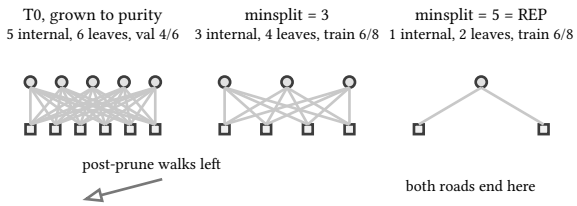


figure 66: three sizes of tree, pre-pruning and post-pruning meet in the middle

sources: reduced-error pruning per Quinlan 1987, and the minsplit pre-pruning rule, both fixed by kdd-contract-s3.md, with both fixtures constructed there and every pinned value witnessed by playground/kdd-matrix/gen_s3.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch16`, 39 checks in chapter 16 of the kdd suite.

17. classifier evaluation

Two samples carry the chapter: `metrics.c` counts one confusion matrix off 10 pinned rows and derives its five ratios, 9 checks, and `resample.c` spends the same rows three ways, holdout, 3-fold cross validation, and a bootstrap with its out-of-bag fraction, 19 checks. The chapter makes 4 moves: the four cells and the five ratios read off them, one holdout split driven by a pinned draw stream, three folds averaged into one number, and the bootstrap's built-in holdout. Every behavioral claim below is one of the 28 checks of chapter 17's samples or a pinned note of the contract sheet. The procedures here measure whatever learner stands in front of them, book 14, ch 19 (knowledge discovery, naive bayes) and book 14, ch 20 (knowledge discovery, nearest neighbor) among them, the score-shaped alternative to their single-threshold verdicts is book 14, ch 18 (knowledge discovery, roc and auc), and the ratio that breaks first on skewed classes pays off in book 14, ch 24 (knowledge discovery, class imbalance).

17.1. the confusion matrix and its five ratios

A classifier that outputs a hard label is right or wrong per row, and two kinds of right and two kinds of wrong. Crossing truth against prediction gives four cells: true positives TP, false negatives FN, false positives FP, true negatives TN. The fixture pins both bit lists, truth 1,1,0,1,0,1, 1,0,0,1 against pred 1,0,0,1,0,1,0,1,0,0, ten rows, every cell reachable. Row 0 is truth 1 predicted 1, a TP. Row 1 is truth 1 predicted 0, an FN, the miss. Row 7 is truth 0 predicted 1, the FP, the false alarm. The other seven rows fill the same three cells plus TN.

The dry run: the counting loop prints `confusion TP=3 FN=3 FP=1 TN=3`, rows 0, 3, 5 landing TP, rows 1, 6, 9 landing FN, row 7 alone landing FP, rows 2, 4, 8 landing TN. The ratio line prints unreduced, `acc=6/10 prec=3/4 rec=3/6 f1=6/10 spec=3/4`, and the checks assert the reduced forms by cross-multiplication, so "ch17 accuracy 3/5" compares 6/10 against 3/5 as integer products, 6 times 5 against 10 times 3. Recall 1/2 is the loudest number on the fixture: the classifier finds half the positives while accuracy says 3/5.

```
int tp = 0, fn = 0, fp = 0, tn = 0;
for (int i = 0; i < 10; i++) {
  if (TRUTH[i] == 1 && PRED[i] == 1)
    tp++;
  else if (TRUTH[i] == 1 && PRED[i] == 0)
    fn++;
  else if (TRUTH[i] == 0 && PRED[i] == 1)
    fp++;
  else
    tn++;
}
printf("confusion TP=%d FN=%d FP=%d TN=%d\n", tp, fn, fp, tn);
CHECK(tp == 3, "ch17 confusion TP=3");
CHECK(fn == 3, "ch17 confusion FN=3");
CHECK(fp == 1, "ch17 confusion FP=1");
CHECK(tn == 3, "ch17 confusion TN=3");
```

listing 125: the four-cell count, one pass, ten rows (source: kdd/samples/src/Ch17/metrics.c, lines 33-48)

```
/* accuracy=(TP+TN)/n, precision=TP/(TP+FP), recall=TP/(TP+FN),
 * F1=2TP/(2TP+FP+FN), specificity=TN/(TN+FP), all D0 pairs */
printf("acc=%d/10 prec=%d/%d rec=%d/%d f1=%d/%d spec=%d/%d\n", tp + tn, tp,
       tp + fp, tp, tp + fn, 2 * tp, 2 * tp + fp + fn, tn, tn + fp);
CHECK(frac_eq(tp + tn, 10, 3, 5), "ch17 accuracy 3/5");
CHECK(frac_eq(tp, tp + fp, 3, 4), "ch17 precision 3/4");
CHECK(frac_eq(tp, tp + fn, 1, 2), "ch17 recall 1/2");
CHECK(frac_eq(2 * tp, 2 * tp + fp + fn, 3, 5), "ch17 F1 3/5");
CHECK(frac_eq(tn, tn + fp, 3, 4), "ch17 specificity 3/4");
```

listing 126: the five ratios, reduced equality by cross-multiplication (source: kdd/samples/src/Ch17/metrics.c, lines 50-58)

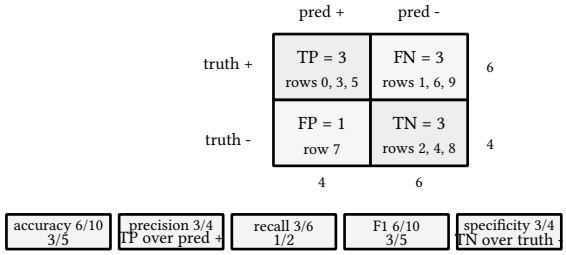


figure 67: the four cells with their rows, and the five ratios each cell pair buys

PITFALL: accuracy is the ratio that hides the minority

Accuracy 3/5 and recall 1/2 describe the same classifier on the same ten rows. Precision 3/4 says three of four predicted positives were real, recall 1/2 says half the real positives were found, and neither implies the other. F1 is their harmonic mean by exact algebra, $2pr/(p+r) = 2\ \mathrm{TP}/(2\ \mathrm{TP}+\mathrm{FP}+\mathrm{FN})$, here $2 \cdot (\frac{3}{4}) \cdot (1/2) = 3/4$ over $3/4 + 1/2 = 5/4$, so 3/5, which the sample asserts directly from the cells. On a skewed class mix accuracy stops even being informative, and that failure is planted here for book 14, ch 24 (knowledge discovery, class imbalance) to harvest with its 99/100 fixture.

17.2. holdout, three draws and a pinned stream

A holdout split carves test rows out of the training set, and to argue about the split the draw stream has to be a fixture too. The sample uses the wave's LCG, the numerical recipes recurrence $s' = 1664525s + 1013904223 \bmod 2^{32}$ seeded 1017, drawing indices of 0..9 and keeping first-seen indices until three distinct test rows exist, rejection sampling exactly as the generator does. The seed is pinned, so the split is a fact about the fixture, not a runtime accident.

The dry run: the stream is three draws long, `holdout draws: 8 5 4`, all first-seen, so the test set is {4,5,8} and the check "ch17 holdout draws 8,5,4 (D3)" pins the stream itself. The train set prints `train: 0 1 2 3 6 7 9`, whose truth labels read 1,1,0,1,1,0,1, five ones of seven, so the majority learner answers 1. Scoring it on the test rows prints `holdout accuracy 1/3 (majority learner)`: rows 4, 5, 8 carry truth 0, 1, 0, the learner predicts 1 everywhere, and only row 5 agrees.

```
static uint32_t rng_state;
static uint32_t rng_next(void) {
  rng_state = rng_state * 1664525u + 1013904223u;
  return rng_state;
}
static uint32_t rng_below(uint32_t k) { return rng_next() % k; }
```

listing 127: the wave LCG, verbatim shared machinery, seed set per row (source: kdd/samples/src/Ch17/resample.c, lines 29-34)

```
/* holdout: seed 1017, rejection until 3 distinct test indices of 0..9 */
rng_state = 1017u;
int draws[64], ndraw = 0, test[3], ntest = 0, seen[10] = {0};
while (ntest < 3) {
  int i = (int)rng_below(10u);
  draws[ndraw++] = i;
  if (!seen[i]) {
    seen[i] = 1;
    test[ntest++] = i;
  }
}
printf("holdout draws:");
for (int k = 0; k < ndraw; k++)
  printf(" %d", draws[k]);
printf("\n");
CHECK(ndraw == 3 && draws[0] == 8 && draws[1] == 5 && draws[2] == 4,
      "ch17 holdout draws 8,5,4 (D3)");
CHECK(ntest == 3 && seen[4] && seen[5] && seen[8],
      "ch17 holdout test set {4,5,8}");
```

listing 128: rejection sampling until 3 distinct test rows, stream pinned (source: kdd/samples/src/Ch17/resample.c, lines 52-70)

```
int maj = majority(tl, ntrain);
int tl_ok = ntrain == 7;
const int want_tl[7] = {1, 1, 0, 1, 1, 0, 1};
for (int k = 0; k < ntrain; k++)
  tl_ok = tl_ok && tl[k] == want_tl[k];
CHECK(tl_ok, "ch17 holdout train labels 1,1,0,1,1,0,1");
CHECK(maj == 1, "ch17 holdout train majority 1");
int hok = 0;
for (int k = 0; k < ntest; k++)
  hok += TRUTH[test[k]] == maj;
printf("holdout accuracy %d/3 (majority learner)\n", hok);
CHECK(hok == 1, "ch17 holdout correct 1 of 3");
CHECK(frac_eq(hok, 3, 1, 3), "ch17 holdout accuracy 1/3");
```

listing 129: majority learner on the train fold, scored on the test rows (source: kdd/samples/src/Ch17/resample.c, lines 87-99)

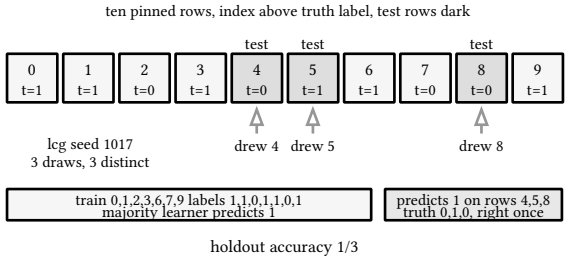


figure 68: ten rows split by three pinned draws, the majority learner scores 1 of 3

NOTE: the split is a fixture, so the score is a fact

Both D3 rows of this chapter, the holdout stream 8,5,4 and the bootstrap indices below, were witnessed identical across two consecutive generator runs before being pinned. That is what lets the chapter print `holdout accuracy 1/3` as evidence rather than as one sample from a distribution. A holdout score on a random split is a random variable, and the variance of that variable across splits is exactly what the next facet averages away.

17.3. three folds, one mean

Cross validation spends every row as test material exactly once. The CV fixture is 12 labels, 1,1,1,1,1,1,1,0,0,1,1,0, dealt round-robin into folds by row index modulo 3, fold 0 holding rows {0,3,6,9}, fold 1 rows {1,4,7,10}, fold 2 rows {2,5,8,11}. Each round trains the same majority learner on the other two folds, 8 rows, ties broken toward class 0, and scores it on the held-out four.

The dry run: all three rounds print majority 1, since each 8-row train set holds five or more ones. The per-round lines read `fold0 rows {0,3,6,9} majority=1 acc=4/4`, `fold1 rows {1,4,7,10} majority=1 acc=3/4`, `fold2 rows {2,5,8,11} majority=1 acc=2/4`. Fold 0's rows are all ones, fold 1 hides the one 0 at row 7, fold 2 carries both remaining zeros at rows 8 and 11. The mean line prints `cv mean accuracy 9/12`, and the closing check asserts it twice, as the fraction pair 3/4 and as the double 0.75 compared with ==, legitimate because every per-fold accuracy is dyadic, 1, 3/4, 1/2, so the mean is a dyadic double too.

```
int fold_rows[3][4], nf[3] = {0, 0, 0};
for (int i = 0; i < 12; i++)
  fold_rows[i % 3][nf[i % 3]++] = i;
int majf[3], corr[3];
for (int f = 0; f < 3; f++) {
  int tr[12], m = 0;
  for (int i = 0; i < 12; i++)
    if (i % 3 != f)
      tr[m++] = CVLAB[i];
  majf[f] = majority(tr, m);
  corr[f] = 0;
  for (int k = 0; k < 4; k++)
    corr[f] += CVLAB[fold_rows[f][k]] == majf[f];
  printf("fold%d rows {%d,%d,%d,%d} majority=%d acc=%d/4\n", f,
         fold_rows[f][0], fold_rows[f][1], fold_rows[f][2], fold_rows[f][3],
         majf[f], corr[f]);
}
```

listing 130: round-robin folds, per-round majority learner and score (source: kdd/samples/src/Ch17/resample.c, lines 103-119)

```
CHECK(majf[0] == 1 && majf[1] == 1 && majf[2] == 1,
      "ch17 cv train majority 1 each fold");
CHECK(frac_eq(corr[0], 4, 1, 1), "ch17 cv fold0 accuracy 1/1");
CHECK(frac_eq(corr[1], 4, 3, 4), "ch17 cv fold1 accuracy 3/4");
CHECK(frac_eq(corr[2], 4, 1, 2), "ch17 cv fold2 accuracy 1/2");
int tot = corr[0] + corr[1] + corr[2];
printf("cv mean accuracy %d/12\n", tot);
CHECK(frac_eq(tot, 12, 3, 4) && (double)tot / 12.0 == 0.75,
      "ch17 cv mean accuracy 3/4 (dyadic double ==)");
```

listing 131: the mean as a fraction pair and as a dyadic double (source: kdd/samples/src/Ch17/resample.c, lines 129-137)

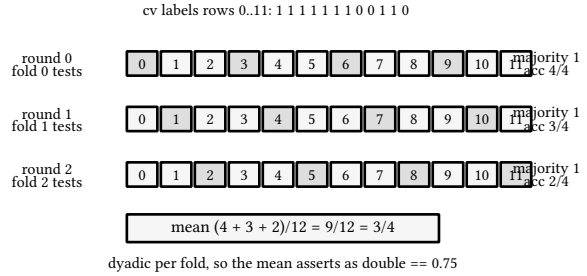


figure 69: three rounds, each fold tests once, mean 9/12 = 3/4

17.4. bootstrap and the out-of-bag third

The bootstrap trains on a replicate, n draws with replacement from the n rows, so some rows appear twice or more and others never appear at all. The never-drawn rows are out of bag, OOB, and they are a holdout the bootstrap builds for free: the replica cannot have memorized a row it never saw. The sample draws 10 indices from the same LCG seeded 2017 and keeps every draw.

The dry run: the stream prints `bootstrap indices: 2 9 0 1 0 1 4 3 4 5`. Rows 0, 1, and 4 are drawn twice each, rows 2, 3, 5, 9 once each, and rows 6, 7, 8 never, so the unique count is 7, asserted as "ch17 bootstrap unique 7" with the set itself checked as {0,1,2,3,4,5,9}. Ten draws miss exactly 3 of 10 rows, and the closing check reads "ch17 bootstrap OOB fraction 3/10". That 3 in 10 is no accident of the seed's neighborhood: the chance one fixed row survives ten draws untouched is $(9/10)^{10} = 3486784401/10^{10} = 0.3487$, because $9^{10} = 3486784401$ exactly, so about a third of any large row set sits out of any one replicate, the sequence whose limit is $1/e$.

```
/* bootstrap: seed 2017, 10 draws with replacement, all kept */
rng_state = 2017u;
int bs[10];
for (int k = 0; k < 10; k++)
  bs[k] = (int)rng_below(10u);
printf("bootstrap indices:");
for (int k = 0; k < 10; k++)
  printf(" %d", bs[k]);
printf("\n");
const int want_bs[10] = {2, 9, 0, 1, 0, 1, 4, 3, 4, 5};
int bs_ok = 1;
for (int k = 0; k < 10; k++)
  bs_ok = bs_ok && bs[k] == want_bs[k];
CHECK(bs_ok, "ch17 bootstrap indices 2,9,0,1,0,1,4,3,4,5 (D3)");
```

listing 132: ten draws with replacement, the stream pinned as one check (source: kdd/samples/src/Ch17/resample.c, lines 139-152)

```
int bseen[10] = {0}, uniq = 0;
for (int k = 0; k < 10; k++)
  if (!bseen[bs[k]]) {
    bseen[bs[k]] = 1;
    uniq++;
  }
CHECK(uniq == 7, "ch17 bootstrap unique 7");
CHECK(bseen[0] && bseen[1] && bseen[2] && bseen[3] && bseen[4] && bseen[5] &&
          !bseen[6] && !bseen[7] && !bseen[8] && bseen[9],
      "ch17 bootstrap unique set {0,1,2,3,4,5,9}");
CHECK(frac_eq(10 - uniq, 10, 3, 10), "ch17 bootstrap OOB fraction 3/10");
```

listing 133: unique census and the out-of-bag fraction 3/10 (source: kdd/samples/src/Ch17/resample.c, lines 153-163)

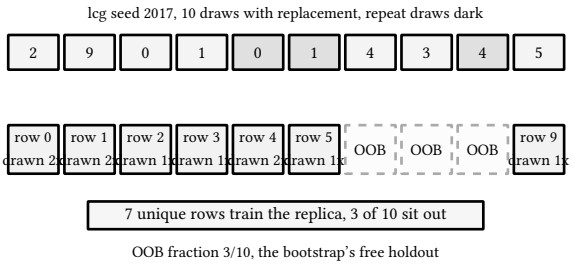


figure 70: ten draws over ten row slots, 7 unique, rows 6, 7, 8 never drawn

sources: all 28 pinned values, the metric definitions (accuracy (TP+TN)/n, precision TP/(TP+FP), recall TP/(TP+FN), F1 2TP/(2TP+FP+FN), specificity TN/(TN+FP)), the tie rules, and both D3 draw rows witnessed by kdd-contract-s4s5.md and playground/kdd-matrix/gen_s4.py, run 2026-09-22, exit 0, the draw rows identical across

two consecutive runs. The OOB arithmetic $(9/10)^{10} = 0.3487$ is exact integer algebra, $9^{10} = 3486784401$, and the $1/e$ limit of $(1 - 1/n)^n$ is the standard compound-interest identity. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch17`, 9 + 19 checks in chapter 17 of the kdd suite.

18. roc and auc

Two samples carry the chapter: `roc.c` sweeps six pinned scores through every threshold and counts the curve and its area two independent ways, 19 checks, and `degenerate.c` runs the same six scores with both degenerate labelings, 6 checks. The chapter makes 4 moves: the threshold sweep that turns a ranked list into a staircase, the concordant-pair count that prices the ranking, the trapezoid area that agrees with it, and the two degenerate curves that break the concordance denominator. Every behavioral claim below is one of the 25 checks of chapter 18's samples or a pinned note of the contract sheet. A hard label has one operating point, the confusion matrix of book 14, ch 17 (knowledge discovery, classifier evaluation), and this chapter is what happens when the classifier hands over a score instead, every threshold at once. Moving along the curve to trade false positives against false negatives on a skewed class mix is book 14, ch 24 (knowledge discovery, class imbalance), and every row here is D4-eligible, the fixture replays in go in book 14, ch 47 (knowledge discovery, the bridge: c to go).

18.1. one threshold per step, seven points

The fixture is six scores in pinned order, (0.875,+), (0.6875,-), (0.75,+), (0.625,+), (0.25,-), (0.125,-), three positives P and three negatives N, every score a dyadic double. The curve rules come from the sheet: sort descending, stable so equal scores keep input order, predict positive when score >= t, and start the curve at (0,0), the point nobody predicted positive. Sweeping the threshold down through the sorted list walks the curve one point per score, each positive a vertical step of 1/3 in the true positive rate, each negative a horizontal step of 1/3 in the false positive rate, and the walk cannot reverse because counters only increase.

The dry run: the sample prints `P=3 N=3`, then seven point lines, `point 0 (0/3,0/3)` through `point 6 (3/3,3/3)`. Sorted descending the list reads 0.875+, 0.75+, 0.6875-, 0.625+, 0.25-, 0.125-, so the two best scores are positives and the curve climbs to (0,2/3) before moving right at all: `point 1 (0/3,1/3)`, `point 2 (0/3,2/3)`. The 0.6875 negative walks it to `point 3 (1/3,2/3)`, the 0.625 positive climbs to `point 4 (1/3,3/3)`, and the two remaining negatives march it along the top, `point 5 (2/3,3/3)`, `point 6 (3/3,3/3)`. Fourteen checks pin the seven points, an fpr and a tpr fraction each.

```
#define N 6
static const double SC[N] = {0.875, 0.6875, 0.75, 0.625, 0.25, 0.125};
static const int LB[N] = {1, 0, 1, 1, 0, 0};
```

listing 134: six dyadic scores, pinned order, three of each class (source: kdd/samples/src/Ch18/roc.c, lines 29-31)

```
  /* threshold sweep: descending stable walk, + when score ≥ t */
  int idx[N];
  sort_desc(idx);
  int xs[N + 1], ys[N + 1]; /* fp and tp counters per curve point */
  xs[0] = ys[0] = 0;
  int tp = 0, fp = 0;
  for (int k = 0; k < N; k++) {
    if (LB[idx[k]] == 1)
      tp++;
    else
      fp++;
    xs[k + 1] = fp;
    ys[k + 1] = tp;
  }
```

listing 135: the sweep, one curve point per score, counters only climb (source: kdd/samples/src/Ch18/roc.c, lines 59-72)

```
const int WX[7] = {0, 0, 0, 1, 1, 2, 3}; /* pinned fpr numerators (den 3) */
const int WY[7] = {0, 1, 2, 2, 3, 3, 3}; /* pinned tpr numerators (den 3) */
for (int k = 0; k ≤ N; k++) {
  printf("point %d (%d/%d,%d/%d)\n", k, xs[k], Nn, ys[k], P);
  char nm[48];
  snprintf(nm, sizeof nm, "ch18 point %d fpr %d/3", k, WX[k]);
  CHECK(frac_eq(xs[k], Nn, WX[k], 3), nm);
  snprintf(nm, sizeof nm, "ch18 point %d tpr %d/3", k, WY[k]);
  CHECK(frac_eq(ys[k], P, WY[k], 3), nm);
}
```

listing 136: the seven points checked against pinned thirds (source: kdd/samples/src/Ch18/roc.c, lines 73-82)

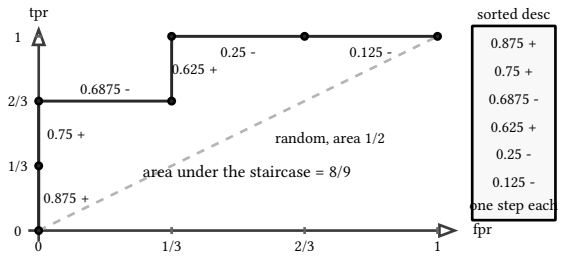


figure 71: the seven-point staircase on the unit square, each step named by the score that caused it

NOTE: the curve lives on ranks, not labels

Nothing in the sweep reads a threshold anyone chose. The curve is a property of the sorted list alone, the positives' positions among the negatives, which is why the same figure serves any scorer that ranks, and why the area computed from it in the next facet will count pairs rather than integrate a distribution. This construction, the threshold sweep from a ranked list, is Fawcett's, and the staircase above is his ROC space with the scores written on the steps.

18.2. eight of nine pairs

The area under that staircase has a second, distribution-free reading: the concordance fraction. Count every positive-negative pair, 3 times 3 is 9 of them, call a pair concordant when the positive outscores the negative, discordant when it does not, and worth 1/2 on a tie. The fraction of concordant weight is the AUC, the empirical probability that a randomly drawn positive outscores a randomly drawn negative, the Mann-Whitney statistic in ROC clothing.

The dry run: the positives are 0.875, 0.75, 0.625, the negatives 0.6875, 0.25, 0.125. The 0.875 beats all three negatives. The 0.75 beats all three, 0.75 over 0.6875 included. The 0.625 beats 0.25 and 0.125 but sits under the 0.6875, and that pair, positive 0.625 ranked below negative 0.6875, is the fixture's single rank inversion. The count line prints `concordant=8 discordant=1 ties=0`, and the check closes the arithmetic as "ch18 AUC 8/9", the fraction asserted as the pair $(2 \cdot 8 + 0, 2 \cdot 9)$ against $(8, 9)$.

```
/* concordant AUC: pos>neg pairs, ties 0.5 each, over all cross pairs */
int con = 0, ties = 0;
for (int i = 0; i < N; i++) {
  if (LB[i] != 1)
    continue;
  for (int j = 0; j < N; j++) {
    if (LB[j] != 0)
      continue;
    con += SC[i] > SC[j];
    ties += SC[i] == SC[j];
  }
}
int dis = P * Nn - con - ties;
printf("concordant=%d discordant=%d ties=%d\n", con, dis, ties);
CHECK(con == 8, "ch18 concordant pairs 8");
CHECK(dis == 1, "ch18 discordant pairs 1");
CHECK(ties == 0, "ch18 cross-class ties 0");
CHECK(frac_eq(2 * con + ties, 2 * P * Nn, 8, 9), "ch18 AUC 8/9");
```

listing 137: all nine cross-class pairs, ties worth half, AUC as a fraction (source: kdd/samples/src/Ch18/roc.c, lines 84-101)

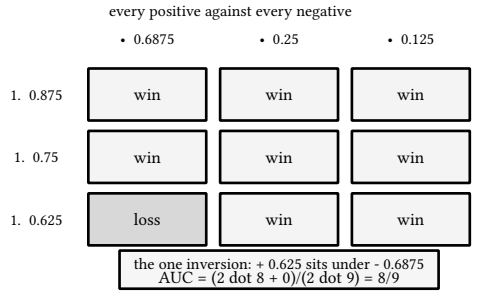


figure 72: the 3x3 pair grid, eight wins and the one inversion

PITFALL: why this fixture says 8/9 and not 11/12

AUC 11/12 needs 12 cross-class pairs, so 3 positives against 4 negatives, 7 items, with exactly one discordant pair. The pinned fixture is 6 scores, 3 against 3, 9 pairs, and the same structure, one inversion, gives 8/9. The contract sheet documents the ruling so the fixture is not bent chasing a denominator it cannot have: at this size the honest numbers are 8, 1, 0, and 8/9, and those are what the checks pin.

18.3. the area, by thirds

The geometric cross-check integrates the staircase directly with the trapezoid rule over the same seven points. Three segments carry nonzero width: from (0,2/3) to (1/3,2/3), width 1/3 at height 2/3, area 2/9; from (1/3,1) to (2/3,1) and from (2/3,1) to (1,1), each width 1/3 at height 1, area 1/3 apiece. The vertical rises carry zero width and zero area, so 2/9 + 3/9 + 3/9 = 8/9, the same number the pair count produced.

The dry run: the print reads `trapezoid auc=0.888888888888889` `|trap-8/9|=1.110e-16`, and the check bounds the gap at 1e-12 rather than comparing with ==. The widths are thirds, not dyadic, so no finite binary double holds them exactly and the accumulated sum lands one or two ulps off, the same two-tier discipline book 14, ch 17 (knowledge discovery, classifier evaluation) used for its dyadic mean, here with the tolerance leg doing the work.

```
/* trapezoid cross-check over the same 7 points, double arithmetic */
double area = 0.0;
for (int k = 0; k < N; k++) {
  double x1 = (double)xs[k] / Nn, x2 = (double)xs[k + 1] / Nn;
  double y1 = (double)ys[k] / P, y2 = (double)ys[k + 1] / P;
  area += (x2 - x1) * (y1 + y2) / 2.0;
}
printf("trapezoid auc=%.15f |trap-8/9|=%.3e\n", area, fabs(area - 8.0 / 9.0));
CHECK(fabs(area - 8.0 / 9.0) < 1e-12, "ch18 trapezoid auc 8/9 within 1e-12");
```

listing 138: trapezoid cross-check, thirds kept as doubles, bounded at 1e-12 (source: kdd/samples/src/Ch18/roc.c, lines 103-111)

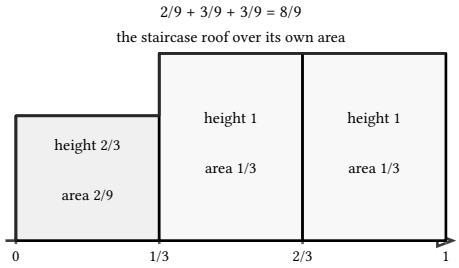


figure 73: the three nonzero trapezoids, 2/9 + 1/3 + 1/3 = 8/9

18.4. two degenerate curves

Break the class balance and the two AUC readings come apart. Relabel all six scores positive and the sweep climbs the left edge, fpr pinned at 0 because there is no negative to add, six vertical steps to (0,1) and the closing corner (1,1), 8 points, trapezoid area exactly 1. Relabel all six negative and the curve crawls the bottom edge to (1,0) before the corner, 8 points, area exactly 0. Both labelings have P*N = 0, no cross-class pair exists, so the concordant count has no denominator and the sample says so in words.

The dry run: `all-positive: n_points=8 area=1.000000000000000` `crossclass_pairs=0` then `auc undefined (no cross-class pairs)`, and `all-negative: n_points=8 area=0.000000000000000` `crossclass_pairs=0` with the same undefined line. The areas are exact doubles, 1.0 and 0.0, asserted with == because every term in those sums is dyadic, while the concordance checks only pin the fact of the empty denominator, “ch18 all-positive concordance undefined P*N=0” and its all-negative twin. A curve of area 1 with no defined AUC is not a contradiction, it is the two definitions refusing to pretend a one-class world ranks anything.

```
    tp++;
  else
    fp++;
  x[np] = nn ? (double)fp / nn : (double)fp / 1.0;
  y[np] = pn ? (double)tp / pn : (double)tp / 1.0;
  np++;
}
x[np] = 1.0;
y[np] = 1.0;
np++;
double area = 0.0;
for (int k = 0; k + 1 < np; k++)
```

listing 139: the degenerate sweep, denominator 1 where a class count is 0 (source: kdd/samples/src/Ch18/degenerate.c, lines 39-50)

```
    area += (x[k + 1] - x[k]) * (y[k] + y[k + 1]) / 2.0;
  printf("%s: n_points=%d area=%.15f crossclass_pairs=%d\n", name, np, area,
         pn * nn);
  if (pn * nn == 0)
    printf("auc undefined (no cross-class pairs)\n");
  char nm[64];
  snprintf(nm, sizeof nm, "ch18 %s curve 8 points", name);
  CHECK(np == 8, nm);
  snprintf(nm, sizeof nm, "ch18 %s trapezoid area == %.0f", name, want_area);
  CHECK(area == want_area, nm);
  snprintf(nm, sizeof nm, "ch18 %s concordance undefined P*N=0", name);
  CHECK(pn * nn == 0, nm);
  return 0;
}
```

listing 140: exact areas, and concordance declared undefined in words (source: kdd/samples/src/Ch18/degenerate.c, lines 51-64)

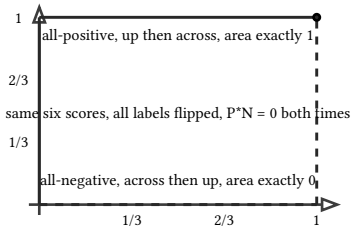


figure 74: both degenerate curves on one unit square, hugging different edges

sources: the threshold-sweep construction of a ROC curve from a ranked list and the AUC-as-ranking reading follow Tom Fawcett, “An introduction to ROC analysis”, Pattern Recognition Letters 27(8):861-874, 2006, doi.org/10.1016/j.patrec.2005.10.010, accessed 2026-09-22. All 25 pinned values, the curve rules, and the 8/9-not-11/12 ruling witnessed by kdd-contract-s4s5.md, its hand derivation, and playground/kdd-matrix/gen_s4.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch18`, 19 + 6 checks in chapter 18 of the kdd suite.

19. naive bayes

Two samples carry the chapter: `tennis.c` scores two queries against the play-tennis conditionals with and without smoothing, 14 checks, and `text.c` classifies one three-word document and plants two argmax ties, 7 checks. The chapter makes 4 moves: the naive product that turns one query into two scores, the zero cell that kills a branch and the add-1 rescue, the multinomial spelling for counted tokens, and the tie-break rules that make argmax deterministic. Every behavioral claim below is one of the 21 checks of chapter 19's samples or the banked Mitchell provenance. The 14 rows are the same table book 14, ch 13 (knowledge discovery, decision trees: id3) split on entropy, here read as counts instead, the underlying probability is the material of book 2, ch 19 (mathematics for programmers, statistics and inference), and the verdicts these scores produce are judged by book 14, ch 17 (knowledge discovery, classifier evaluation).

19.1. one query, four conditionals, two classes

Naive Bayes classifies by Bayes' rule with the naive part being the denominator's factorization: within each class, the four attribute values are treated as independent, so the posterior score is the class prior times four conditionals, all multiplied. The conditionals come from counting Mitchell's table: outlook sunny happens on 2 of the 9 yes days and 3 of the 5 no days, and so on down the four attributes, every count a fact about the 14 rows.

The dry run: query (Sunny, Cool, High, Strong). The yes chain is $9/14 \cdot 2/9 \cdot 3/9 \cdot 3/9 \cdot 3/9 = 1/189$, the no chain $5/14 \cdot 3/5 \cdot 1/5 \cdot 4/5 \cdot 3/5 = 18/875$, and the sample prints `q1 yes=1/189 no=18/875`, about 0.00529 against 0.02057. Normalizing prints `q1 posterior yes=125/611 (0.204582651391) no=486/611 (0.795417348609)`, both posteriors sharing the denominator 611 because the raw common one, $875 + 18 \cdot 189 = 4277 = 7 \cdot 611$, reduces with each numerator. The argmax needs no normalization at all: the check "ch19 q1 cross-mult 875 < 3402" compares the two scores by cross-multiplication, 1 times 875 against 18 times 189, and the larger product wins, no.

```
static frac_t score(const int *query, int cls, int smooth) {
  frac_t s = fr(NCLS[cls], 14);
  for (int a = 0; a < 4; a++) {
    int c = CNT[a][query[a]][cls];
    s = fmul(s, smooth ? fr(c + 1, NCLS[cls] + NVAL[a]) : fr(c, NCLS[cls]));
  }
  return s;
}
```

listing 141: the score, prior times four conditionals, smoothing optional (source: kdd/samples/src/Ch19/tennis.c, lines 79-86)

```c
int main(void) {
  /* q1 = (Sunny, Cool, High, Strong) */
  const int q1[4] = {0, 2, 0, 1};
  frac_t sy = score(q1, 0, 0), sn = score(q1, 1, 0);
  printf("q1 yes=%lld/%lld no=%lld/%lld\n", sy.n, sy.d, sn.n, sn.d);
  CHECK(feq(sy, 1, 189), "ch19 q1 yes-score 1/189");
  CHECK(feq(sn, 18, 875), "ch19 q1 no-score 18/875");
  frac_t py = fpost(sy, sn), pn = fpost(sn, sy);
  printf("q1 posterior yes=%lld/%lld (%.12f) no=%lld/%lld (%.12f)\n", py.n,
         py.d, (double)py.n / py.d, pn.n, pn.d, (double)pn.n / pn.d);
  CHECK(feq(py, 125, 611), "ch19 q1 posterior yes 125/611");
  CHECK(feq(pn, 486, 611), "ch19 q1 posterior no 486/611");
  CHECK(fgt(sn, sy), "ch19 q1 argmax no");
  CHECK(sy.n * sn.d == 875 && sn.n * sy.d == 3402 && 875 < 3402,
        "ch19 q1 cross-mult 875 < 3402");
  CHECK(fabs((double)py.n / py.d - 0.204582651391) <= 1e-12 &&
            fabs((double)pn.n / pn.d - 0.795417348609) <= 1e-12,
        "ch19 q1 posteriors 0.204582651391 / 0.795417348609 (D2)");
```

listing 142: query 1, scores, posteriors, and the unnormalized argmax (source: kdd/samples/src/Ch19/tennis.c, lines 88-105)

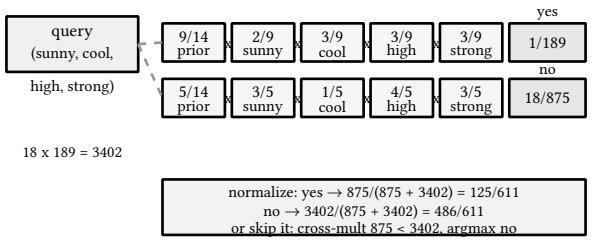


figure 75: two factor chains from one query, normalize last or not at all

NOTE: naive means the product is the assumption

Nothing in the 14 rows says outlook is independent of humidity given the class, and in real weather they are not. The model multiplies the conditionals anyway, which is exactly the naivety, and it survives because classification only needs the argmax to be right, not the posterior to be calibrated. The same table gave book 14, ch 13 (knowledge discovery, decision trees: id3) a tree that consults two attributes and ignores temperature entirely, and this chapter's classifier consults all four every time: two readers, one table.

19.2. the zero cell and the laplace rescue

The second query swaps sunny for overcast, and overcast never appeared with a no day: 4 yes days, 0 no days, so the conditional P(overcast | no) is the count 0 over 5. Unsmoothed, that single factor zeroes the whole no branch, $5/14 \cdot 0 \cdot 1/5 \cdot 4/5 \cdot 3/5 = 0$, and the posterior for no reads exactly 0, a certainty no 14-row table has earned. The fix is add-1 smoothing, the Laplace estimate, but with the sheet's exact convention: conditionals become $(c + 1)/(n_{\text{class}} + |\text{values}|)$ with the value count per attribute, 3 for outlook and temperature, 2 for humidity and wind, priors left alone.

The dry run: the before line prints `q2 before yes=2/189 no=0/1`, the reduced form of an exact zero. After smoothing the yes chain is $9/14 \cdot 5/12 \cdot 4/12 \cdot 4/11 \cdot 4/11 = 10/847$ and the no chain $5/14 \cdot 1/8 \cdot 2/8 \cdot 5/7 \cdot 4/7 = 25/5488$, printed as `q2 after yes=10/847 no=25/5488`. The posterior line prints `q2 posterior after yes=1568/2173 (0.721583064887) no=605/2173`, and the closing check notes the verdict never flipped, yes before and after, what changed is that the no branch went from 0 to a positive number worth comparing against.

```
/* q2 = (Overcast, Cool, High, Strong): Overcast|no is the 0/5 cell */
const int q2[4] = {1, 2, 0, 1};
frac_t y0 = score(q2, 0, 0), n0 = score(q2, 1, 0);
frac_t y1 = score(q2, 0, 1), n1 = score(q2, 1, 1);
printf("q2 before yes=%lld/%lld no=%lld/%lld\n", y0.n, y0.d, n0.n, n0.d);
printf("q2 after  yes=%lld/%lld no=%lld/%lld\n", y1.n, y1.d, n1.n, n1.d);
CHECK(feq(y0, 2, 189), "ch19 q2 before yes 2/189");
CHECK(n0.n == 0 && n0.d == 1, "ch19 q2 before no 0 (zero cell)");
CHECK(feq(y1, 10, 847), "ch19 q2 after yes 10/847");
CHECK(feq(n1, 25, 5488), "ch19 q2 after no 25/5488");
frac_t py1 = fpost(y1, n1), pn1 = fpost(n1, y1);
printf("q2 posterior after yes=%lld/%lld (%.12f) no=%lld/%lld\n", py1.n,
       py1.d, (double)py1.n / py1.d, pn1.n, pn1.d);
CHECK(feq(py1, 1568, 2173), "ch19 q2 posterior after yes 1568/2173");
CHECK(feq(pn1, 605, 2173), "ch19 q2 posterior after no 605/2173");
CHECK(fgt(y0, n0) && fgt(y1, n1) && n0.n == 0 && n1.n > 0,
      "ch19 q2 argmax yes before and after, no-branch 0 -> positive");
```

listing 143: query 2, the 0/5 cell before smoothing, both branches after (source: kdd/samples/src/Ch19/tennis.c, lines 107-123)

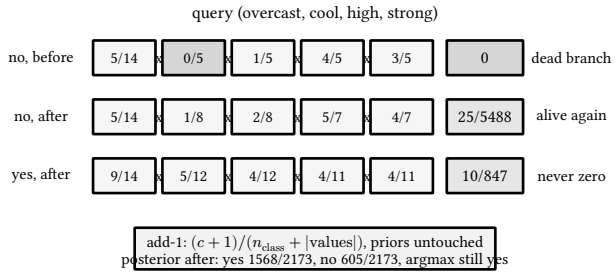


figure 76: one zero factor kills the branch, add-1 buys it back

PITFALL: a zero conditional is a verdict of certainty

With the raw counts, one attribute value the class never saw erases every other attribute's evidence in that branch, three informative factors multiplied by zero. The table has 14 rows, and a conditional of exactly 0 claims no overcast no-day will ever occur, which no 14-row sample can establish. Add-1 spends one phantom observation per cell, and the per-attribute denominator matters: outlook spreads its phantom over 3 values, humidity over 2, so the sheet's convention keeps humidity's rescue weaker than outlook's.

19.3. three words, two mailboxes

Text turns the counts into the model directly. The multinomial spelling scores a document by the product over its tokens of $(\text{count}_{\text{token}} + 1)/(\text{total} + V)$, where total is the class's token count and V the vocabulary size, priors up front as before. The fixture is a 3-word vocabulary and two classes, spam tokens free 3, money 2, meeting 0, 5 total, ham tokens free 1, money 0, meeting 3, 4 total, priors 1/2 each, and the document is "free money money".

The dry run: the spam chain is $1/2 \cdot 4/8 \cdot 3/8 \cdot 3/8 = 9/256$ and the ham chain $1/2 \cdot 2/7 \cdot 1/7 \cdot 1/7 = 1/343$, printed as `doc 'free money money': spam=9/256 ham=1/343`. The posterior prints `posterior spam=3087/3343 (0.923422075980)`, the numerator being $9 \cdot 343 = 3087$ over $3087 + 256$, and the argmax check fires as "ch19 text argmax spam". Money is the telling token: spam carries 2 of 5 money tokens, ham 0 of 4, so the word money alone would have hit ham's zero cell from the last facet, and add-1 is what keeps the ham score at 1/343 instead of 0.

```
int main(void) {
  /* doc "free money money": prior * prod (count+1)/(total+V) */
  const char *doc[3] = {"free", "money", "money"};
  frac_t sc[2];
  for (int c = 0; c < 2; c++) {
    sc[c] = fr(1, 2); /* priors 1/2 each */
    for (int k = 0; k < 3; k++)
      sc[c] = fmul(sc[c], fr(TOK[c][tok_id(doc[k])] + 1, TOT[c] + NV));
  }
  printf("doc 'free money money': spam=%lld/%lld ham=%lld/%lld\n", sc[0].n,
         sc[0].d, sc[1].n, sc[1].d);
  CHECK(feq(sc[0], 9, 256), "ch19 text spam score 9/256");
  CHECK(feq(sc[1], 1, 343), "ch19 text ham score 1/343");
  frac_t ps = fr(sc[0].n * sc[1].d, sc[0].n * sc[1].d + sc[1].n * sc[0].d);
  printf("posterior spam=%lld/%lld (%.12f)\n", ps.n, ps.d, (double)ps.n / ps.d);
  CHECK(feq(ps, 3087, 3343), "ch19 text posterior spam 3087/3343");
  CHECK(fabs((double)ps.n / ps.d - 0.923422075980) ≤ 1e-12,
        "ch19 text posterior 0.923422075980 (D2)");
```

listing 144: one document through the multinomial product with add-1 (source: kdd/samples/src/Ch19/text.c, lines 90-107)

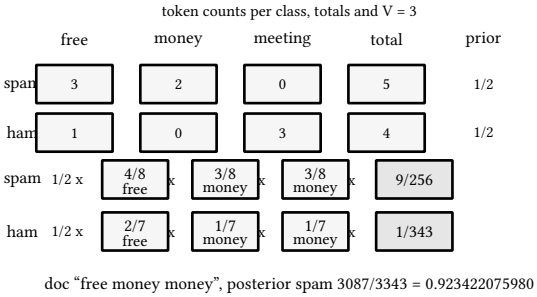


figure 77: the token table and the two chains it prices

19.4. when scores tie

An argmax over exact fractions can tie, and a classifier that breaks ties by floating-point accident is not reproducible. The sample's rule is the sheet's, in fixed order: highest score wins, equal scores fall to the larger prior, still equal falls to the lexicographically smaller class name. Both tie levels are planted, not narrated.

The dry run: the sample prints its rule in words, `tie-break: equal scores -> larger prior wins; equal priors -> lexicographically smaller class name (ham < spam)`. The first plant scores both classes 1/6 with priors 1/2 each, and the check reads "ch19 planted tie (equal score, equal prior) -> ham", ham winning because it sorts before spam. The second plant keeps the equal scores but tilts the priors to 1/3 against 2/3, and "ch19 planted tie (equal score) -> larger prior (spam 2/3)" fires, the second rung overruling the third.

```c
/* argmax over two classes: score, then prior, then lex smaller name */
static const char *argmax2(frac_t s0, frac_t s1, frac_t p0, frac_t p1,
                           const char *nm0, const char *nm1) {
  int c = fcmp(s0, s1);
  if (c != 0)
    return c > 0 ? nm0 : nm1;
  c = fcmp(p0, p1);
  if (c != 0)
    return c > 0 ? nm0 : nm1;
  return strcmp(nm0, nm1) < 0 ? nm0 : nm1;
}
```

listing 145: the three-rung argmax, score, prior, then name (source: kdd/samples/src/Ch19/text.c, lines 66-76)

```c
  /* planted tie-break rows: equal scores, then the prior, then lex */
  printf("tie-break: equal scores -> larger prior wins; equal priors -> "
         "lexicographically smaller class name (ham < spam)\n");
  CHECK(strcmp(argmax2(fr(1, 6), fr(1, 6), fr(1, 2), fr(1, 2), "ham", "spam"),
               "ham") == 0,
        "ch19 planted tie (equal score, equal prior) -> ham");
  CHECK(strcmp(argmax2(fr(1, 6), fr(1, 6), fr(1, 3), fr(2, 3), "ham", "spam"),
               "spam") == 0,
        "ch19 planted tie (equal score) -> larger prior (spam 2/3)");
```

listing 146: two planted ties, one resolved at each rung (source: kdd/samples/src/Ch19/text.c, lines 112-120)

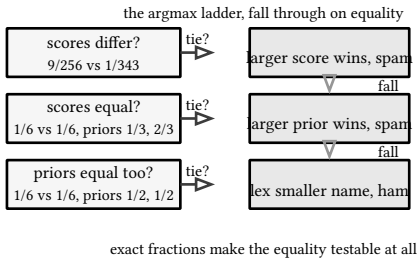


figure 78: the tie ladder, each rung only reached when the one above ties

sources: the play-tennis conditionals are counted from Mitchell, Machine Learning, McGraw-Hill 1997, Table 3.2 p. 59, the dataset introduced by Quinlan 1986, the same table pinned for chapter 13. The smoothing convention, add-1 on conditionals with per-attribute denominators $n_{\text{class}} + |\text{values}|$ and unsmoothed priors, and the tie-break order are pinned by kdd-contract-s4s5.md and witnessed by playground/kdd-matrix/gen_s4.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch19`, 14 + 7 checks in chapter 19 of the kdd suite.

20. nearest neighbor

Two samples carry the chapter: `knn.c` sorts the neighbors of two queries and votes them five ways, plain and distance-weighted, 19 checks, and `census.c` classifies all 25 cells of a 5x5 grid at k = 3 and walks the boundary, 13 checks. The chapter makes 5 moves: the sorted neighbor list as the entire model, the majority vote and its tie rule, the weighted vote that uses all 9 neighbors, the decision region the votes paint over the grid, and the training point that gets outvoted inside its own cell. Every behavioral claim below is one of the 32 checks of chapter 20's samples or a pinned note of the contract sheet. The metric doing the sorting is the squared euclidean of book 14, ch 5 (knowledge discovery, distance metrics), the eager learners this one refuses to be are book 14, ch 19 (knowledge discovery, naive bayes) and book 14, ch 21 (knowledge discovery, neural networks), and the k to vote with is chosen by the resampling of book 14, ch 17 (knowledge discovery, classifier evaluation).

20.1. the sorted list is the model

There is no training phase. The model is the 9 pinned training points, 0:(1,1,A) through 8:(5,4,B), five A then four B in index order, and a query is answered by sorting all 9 by squared distance, distance ties broken to the lower training index, and reading the list from the front. Q1 is (3,3), sitting between the A pocket on the left and the B cluster up right.

The dry run: the sorted line prints `q1 sorted (d2,idx,label): (1,4,A) (1,5,B) (2,3,A) (2,6,B) (5,1,A) (5,2,A) (5,7,B) (5,8,B) (8,0,A)`. The front is a distance tie, idx4 at (2,3) one unit away and idx5 at (3,2) one unit away, and idx4 sorts first by index, the check "ch20 q1 sorted (d2,idx) order, ties to lower index" pinning the whole 18-tuple. Second place ties again at d2 = 2, idx3 before idx6 the same way, and d2 = 5 holds a four-way tie settled into index order 1, 2, 7, 8. Distances are squared on purpose, integers throughout, no root ever taken.

```
#define NT 9
static const int TX[NT] = {1, 1, 2, 2, 2, 3, 4, 4, 5};
static const int TY[NT] = {1, 2, 1, 2, 3, 2, 4, 5, 4};
static const char TL[NT] = {'A', 'A', 'A', 'A', 'A', 'B', 'B', 'B', 'B'};
```

listing 147: nine training points, pinned index order, two classes (source: kdd/samples/src/Ch20/knn.c, lines 29-32)

```
/* neighbors sorted by (squared distance, training index) */
static void sort_neighbors(int qx, int qy, int *order, int *d2s) {
  for (int i = 0; i < NT; i++)
    order[i] = i;
  for (int i = 1; i < NT; i++) {
    int v = order[i], j = i;
    while (j > 0 && (dist2(qx, qy, order[j - 1]) > dist2(qx, qy, v) ||
                     (dist2(qx, qy, order[j - 1]) == dist2(qx, qy, v) &&
                      order[j - 1] > v))) {
      order[j] = order[j - 1];
      j--;
    }
    order[j] = v;
  }
  if (d2s)
    for (int i = 0; i < NT; i++)
      d2s[i] = dist2(qx, qy, order[i]);
}
```

listing 148: insertion sort by squared distance, index as tiebreak (source: kdd/samples/src/Ch20/knn.c, lines 68-85)

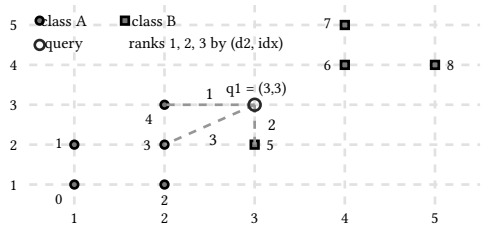


figure 79: the nine points, q1 among them, three nearest ranked

20.2. five ks, two ties

Take the first k entries of the sorted list and count labels. The vote is majority, and when k splits evenly the sheet's rule fires: the class of the single nearest neighbor wins, which hands the decision back to the top of the list. Q1 is built to need it twice.

The dry run: `q1 k=1 → A`, then `q1 k=2 votes A=1 B=1 → A (tie to nearest)`, the tie falling to idx4 because it sorted first, then `q1 k=3 votes A=2 B=1 → A`, `q1 k=4 votes A=2 B=2 → A (tie to nearest)`, the same resolution a second time, and `q1 k=5 votes A=3 B=2 → A`. Q2 = (4,3) lives inside the B cluster: its sorted front is (1,6,B) (2,5,B) (2,8,B), asserted as "ch20 q2 nearest set idx 6,5,8", and every k agrees, `q2 k=1 → B`, `q2 k=3 votes A=0 B=3 → B`, `q2 k=5 votes A=1 B=4 → B`, the k=5 minority vote being idx4's, the nearest A at d2 = 4.

```
/* k-NN vote with the pinned tie rule: vote tie -> class of the nearest */
static char vote(const int *order, int k, int *na, int *nb) {
  *na = *nb = 0;
  for (int i = 0; i < k; i++) {
    if (TL[order[i]] == 'A')
      (*na)++;
    else
      (*nb)++;
  }
  if (*na != *nb)
    return *na > *nb ? 'A' : 'B';
  return TL[order[0]];
}
```

listing 149: the vote, majority with the tie falling to the nearest (source: kdd/samples/src/Ch20/knn.c, lines 87-99)

```
    char l1 = vote(o1, 1, &na, &nb);
    printf("q1 k=1 -> %c\n", l1);
    CHECK(l1 == 'A', "ch20 q1 k=1 A");
    l1 = vote(o1, 2, &na, &nb);
    printf("q1 k=2 votes A=%d B=%d -> %c (tie to nearest)\n", na, nb, l1);
    CHECK(na == 1 && nb == 1, "ch20 q1 k=2 votes 1-1");
    CHECK(l1 == 'A' && o1[0] == 4, "ch20 q1 k=2 tie -> nearest idx4 -> A");
    l1 = vote(o1, 3, &na, &nb);
    printf("q1 k=3 votes A=%d B=%d -> %c\n", na, nb, l1);
    CHECK(na == 2 && nb == 1, "ch20 q1 k=3 votes 2-1");
    CHECK(l1 == 'A', "ch20 q1 k=3 A");
    l1 = vote(o1, 4, &na, &nb);
    printf("q1 k=4 votes A=%d B=%d -> %c (tie to nearest)\n", na, nb, l1);
    CHECK(na == 2 && nb == 2, "ch20 q1 k=4 votes 2-2");
    CHECK(l1 == 'A' && o1[0] == 4, "ch20 q1 k=4 tie -> nearest idx4 -> A");
    l1 = vote(o1, 5, &na, &nb);
    printf("q1 k=5 votes A=%d B=%d -> %c\n", na, nb, l1);
    CHECK(na == 3 && nb == 2 && l1 == 'A', "ch20 q1 k=5 votes 3-2 -> A");
```

listing 150: k = 1..5 on q1, two planted ties, one rule (source: kdd/samples/src/Ch20/knn.c, lines 116-133)

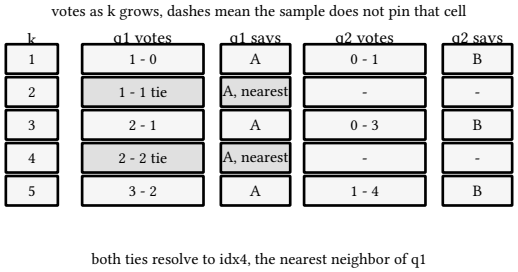


figure 80: both queries across k, the two tie rows highlighted

PITFALL: an even k begs the question it was asked

With two classes, every even k can split its ballot, and q1 does it twice, at k = 2 and k = 4. The pinned rule resolves the deadlock locally, the single nearest neighbor decides, which is exactly k = 1 sneaking back in. The usual engineering outs are odd k, which only postpones the problem to three-way races, or the weighted vote of the next facet, which cannot tie short of exact fraction equality.

20.3. every neighbor weighs 1/d2

The weighted vote drops the cutoff instead of moving it: every one of the 9 neighbors contributes weight $w = 1/d_2$ to its class, exact fractions summed by the same integer arithmetic, and the heavier class wins. Distance does the talking, near neighbors loudly, far ones at a whisper.

The dry run: for q1 the weights sum to `weighted q1: A=81/40 B=19/10`, hand-checkable as $1/8 + 1/5 + 1/5 + 1/2 + 1 = 81/40$ for A against $1 + 1/2 + 1/5 + 1/5 = 19/10$ for B, so A wins 2.025 to 1.9, the same verdict as every plain vote. For q2 the line reads `weighted q2: A=391/520 B=9/4`, A gathering $1/13 + 1/10 + 1/8 + 1/5 + 1/4$ over the common denominator 520, and B cruising 2.25 to about 0.752 on the strength of idx6 sitting one unit away.

```
/* distance-weighted vote: w = 1/d2 over all 9, exact fractions */
frac_t wa[2][2] = {{fr(0, 1), fr(0, 1)}, {fr(0, 1), fr(0, 1)}};
const int qx[2] = {3, 4};
for (int q = 0; q < 2; q++)
  for (int i = 0; i < NT; i++) {
    frac_t w = fr(1, dist2(qx[q], 3, i));
    int c = TL[i] == 'A' ? 0 : 1;
    wa[q][c] = fadd(wa[q][c], w);
  }
printf("weighted q1: A=%lld/%lld B=%lld/%lld\n", wa[0][0].n, wa[0][0].d,
       wa[0][1].n, wa[0][1].d);
CHECK(feq(wa[0][0], 81, 40) && fgt(wa[0][0], wa[0][1]),
      "ch20 q1 weighted A=81/40 wins");
CHECK(feq(wa[0][1], 19, 10), "ch20 q1 weighted B=19/10");
printf("weighted q2: A=%lld/%lld B=%lld/%lld\n", wa[1][0].n, wa[1][0].d,
       wa[1][1].n, wa[1][1].d);
CHECK(feq(wa[1][0], 391, 520), "ch20 q2 weighted A=391/520");
CHECK(feq(wa[1][1], 9, 4) && fgt(wa[1][1], wa[1][0]),
      "ch20 q2 weighted B=9/4 wins");
```

listing 151: w = 1/d2 over all nine, fractions summed exactly (source: kdd/samples/src/Ch20/knn.c, lines 157-175)

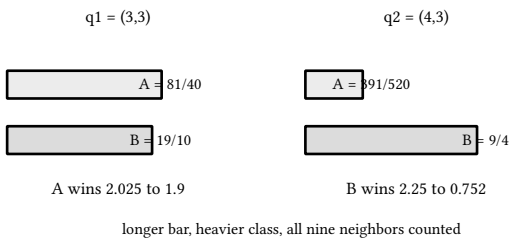


figure 81: class weight totals per query, fractions beside decimals

20.4. the census, and the region it paints

Classify every cell of the 5x5 grid, x and y from 0 to 4, at k = 3, and the votes paint a decision region: what the classifier would say anywhere in the square, not just at the two queries. Two training points, idx7 at (4,5) and idx8 at (5,4), sit outside the grid and still vote, the grid is queries, not points.

The dry run: the printed rows, `grid rows y=4..0:` then `A A A B B`, `A A A A B`, `A A A A B`, `A A A A A`, `A A A A A`, and the census line `census k=3: A cells=21 B cells=4`, the B pocket being exactly the cells (4,2), (4,3), (3,4), (4,4) pinned as "ch20 B cells (4,2),(4,3),(3,4),(4,4)". The boundary walk counts `boundary cells=7`, one check per cell, (2,4), (3,2), (3,3), (3,4), (4,1), (4,2), (4,3), every cell with a 4-connected neighbor of the other label.

```
int main(void) {
  int na = 0, nb = 0;
  for (int x = 0; x < 5; x++)
    for (int y = 0; y < 5; y++)
      grid[x][y] = classify(x, y, NULL, NULL, NULL, NULL);
  printf("grid rows y=4..0:\n");
  for (int y = 4; y >= 0; y--) {
    printf("  ");
    for (int x = 0; x < 5; x++)
      printf("%c ", grid[x][y]);
    printf("\n");
  }
  for (int x = 0; x < 5; x++)
    for (int y = 0; y < 5; y++) {
      if (grid[x][y] == 'A')
        na++;
      else
        nb++;
    }
  printf("census k=3: A cells=%d B cells=%d\n", na, nb);
  CHECK(na == 21, "ch20 census 21 A cells");
  CHECK(nb == 4, "ch20 census 4 B cells");
  CHECK(grid[4][2] == 'B' && grid[4][3] == 'B' && grid[3][4] == 'B' &&
           grid[4][4] == 'B',
        "ch20 B cells (4,2),(4,3),(3,4),(4,4)");
```

listing 152: 25 classifications, the census, and the B pocket (source: kdd/samples/src/Ch20/census.c, lines 90-114)

```
  /* boundary cells: 4-connected label changes */
  const int BX[7] = {2, 3, 3, 3, 4, 4, 4};
  const int BY[7] = {4, 2, 3, 4, 1, 2, 3};
  int ncell = 0;
  for (int x = 0; x < 5; x++)
    for (int y = 0; y < 5; y++)
      ncell += is_boundary(x, y);
  printf("boundary cells=%d\n", ncell);
  for (int k = 0; k < 7; k++) {
    char nm[48];
    snprintf(nm, sizeof nm, "ch20 boundary cell (%d,%d)", BX[k], BY[k]);
    CHECK(is_boundary(BX[k], BY[k]) && (k < 6 || ncell == 7), nm);
  }
```

listing 153: the 4-connected boundary walk, seven cells, seven checks (source: kdd/samples/src/Ch20/census.c, lines 127-139)

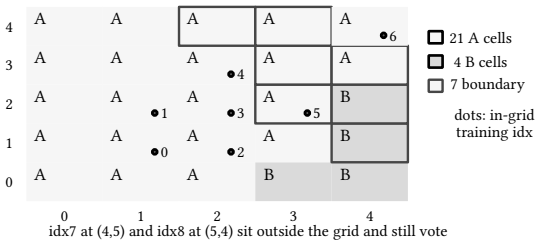


figure 82: the decision region at k = 3, training points marked, boundary cells outlined

20.5. the cell that votes against itself

Cell (3,2) is special twice over: it is the site of training point idx5, label B, and the census paints it A. The query lands exactly on a training point, so the sorted list starts with that point at d2 = 0, and self-inclusion hands B exactly one vote, no more.

The dry run: the highlight line prints `cell (3,2) votes A=2 B=1 → A, nearest idx5 at d2=0`, and its three checks pin the three facts separately: idx5 is at (3,2) with label B, "ch20 cell (3,2) IS training point idx5 label B", the grid says A, "ch20 cell (3,2) classifies A at k=3", and the vote is 2-1 with the self at distance zero, "ch20 cell (3,2) vote 2-1 A with self at d2=0". The two A votes come from idx3 at (2,2), d2 = 1, and idx2 at (2,1), d2 = 2, which edges out idx4's equal d2 = 2 at (2,3) on the index tiebreak, the same rule that ordered the front of q1's list.

```c
/* highlight: (3,2) is training point idx5 (label B) but votes A */
int va, vb, nd2, nidx;
char lab = classify(3, 2, &va, &vb, &nd2, &nidx);
printf("cell (3,2) votes A=%d B=%d -> %c, nearest idx%d at d2=%d\n", va, vb,
       lab, nidx, nd2);
CHECK(TX[5] == 3 && TY[5] == 2 && TL[5] == 'B',
      "ch20 cell (3,2) IS training point idx5 label B");
CHECK(grid[3][2] == 'A', "ch20 cell (3,2) classifies A at k=3");
CHECK(va == 2 && vb == 1 && lab == 'A' && nidx == 5 && nd2 == 0,
      "ch20 cell (3,2) vote 2-1 A with self at d2=0");
```

listing 154: the highlight row, identity, verdict, and vote pinned apart (source: kdd/samples/src/Ch20/census.c, lines 116-125)

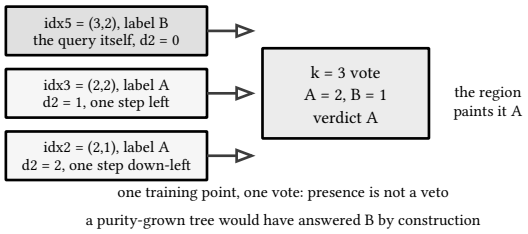


figure 83: the self at d2 = 0 contributes one vote and loses

NOTE: lazy learners do not memorize, they defer

The tree of book 14, ch 13 (knowledge discovery, decision trees: id3) scored 14 of 14 on its own training rows because purity growth bakes the training labels into the leaves. Here a training point is just a voter, and the fixture plants one that loses the election in its own cell, 2 to 1, with itself at distance zero. That honesty is why nearest-neighbor error estimates want the query excluded from its own neighbor list on training data, and why the holdout and cross-validation splits of book 14, ch 17 (knowledge discovery, classifier evaluation) are the honest way to price any classifier, this one included.

sources: all 32 pinned values, the 9-point fixture, and both tie rules witnessed by kdd-contract-s4s5.md and playground/kdd-matrix/gen_s4.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch20`, 19 + 13 checks in chapter 20 of the kdd suite.

21. neural networks

Three samples carry the chapter: `perceptron.c` runs the AND epoch trace to convergence and witnesses XOR failing, 28 checks, `mlp.c` pushes all four XOR inputs through a hand-set 2-2-1 step network and takes one exact output-layer gradient step, 23 checks, and `backprop.c` walks a single sigmoid pass forward and backward, 18 checks. The chapter makes 5 moves: the perceptron rule and the epoch where it finally goes clean, the XOR fixture no line can separate, the hidden layer that buys XOR anyway, one gradient step small enough to check by hand, and the chain rule spread across a net of sigmoids. Every behavioral claim below is one of the 69 checks of chapter 21's samples or a pinned note of the contract sheet. This is the eager extreme next to the deferred votes of book 14, ch 20 (knowledge discovery, nearest neighbor), the squared loss it descends is the same shape book 14, ch 12 (knowledge discovery, simple linear regression) fits in closed form, and the maximum-margin take on a linear boundary belongs to book 14, ch 22 (knowledge discovery, support vector machines).

21.1. six epochs to a clean pass

The perceptron is a linear classifier learned from its own mistakes. It predicts the sign of $b + w_1 x_1 + w_2 x_2$ with sign(0)=+1, starts from the all-zero vector, and on each misclassification adds $y \cdot (1, x_1, x_2)$ to (b, w_1, w_2), learning rate 1, visiting the points in the pinned cyclic order. The fixture is AND, the four points (0,0,-1), (0,1,-1), (1,0,-1), (1,1,+1).

The dry run: the trace prints one line per epoch, `AND epoch 1: w=(0,1,1) misclassified=2`, then `AND epoch 2: w=(-1,2,1) misclassified=3`, `AND epoch 3: w=(-2,2,1) misclassified=3`, `AND epoch 4: w=(-2,2,2) misclassified=2`, `AND epoch 5: w=(-3,2,1) misclassified=1`, `AND epoch 6: w=(-3,2,1) misclassified=0`, each line pinned by two checks, and the close `converged at epoch 6, final w=(-3,2,1)` with "ch21 AND convergence epoch 6". Epoch 1 is worth walking by hand: from (0,0,0) the first point (0,0) has activation 0, so the prediction is sign(0)=+1 against a label of −1, wrong on the very first guess, and the update subtracts (1,0,0) leaving (-1,0,0); the two single-one points now read −1 and pass; (1,1) reads −1 against +1, wrong, and adding (1,1,1) ends the epoch at (0,1,1) with 2 misses. Epoch 6 adds nothing: it re-tests (-3,2,1), finds activations −3, −2, −1, 0, and the last one only passes because sign(0)=+1. The converged boundary $-3 + 2x_1 + x_2 = 0$ passes exactly through the point (1,1), so the sign convention is load-bearing at both the first mistake and the final answer, pinned as "ch21 AND final w classifies all 4".

```
/* points as {x1, x2, y}, cyclic order pinned by the fixture */
static const int ANDP[4][3] = {{0, 0, -1}, {0, 1, -1}, {1, 0, -1}, {1, 1, 1}};
static const int XORP[4][3] = {{0, 0, -1}, {0, 1, 1}, {1, 0, 1}, {1, 1, -1}};

/* run `epochs` epochs over the 4 points; records per-epoch end w and
 * miss counts, returns the first epoch with 0 misses (0 if never) */
static int run(const int pts[4][3], int epochs, int tw[][3], int tm[]) {
  int b = 0, w1 = 0, w2 = 0, conv = 0;
  for (int e = 0; e < epochs; e++) {
    int mis = 0;
    for (int i = 0; i < 4; i++) {
      int a = b + w1 * pts[i][0] + w2 * pts[i][1];
      int p = a >= 0 ? 1 : -1; /* sign(0)=+1 */
      if (p != pts[i][2]) {
        b += pts[i][2];
        w1 += pts[i][2] * pts[i][0];
        w2 += pts[i][2] * pts[i][1];
        mis++;
      }
    }
    tw[e][0] = b;
    tw[e][1] = w1;
    tw[e][2] = w2;
    tm[e] = mis;
    if (mis == 0 && conv == 0)
      conv = e + 1;
  }
  return conv;
}
```

listing 155: the two point sets and the epoch loop with its update rule (source: kdd/samples/src/Ch21/perceptron.c, lines 27-55)

```
int main(void) {
  /* AND: 15 epochs recorded, pins only epochs 1-6 */
  int tw[15][3], tm[15];
  int conv = run(ANDP, 15, tw, tm);
  const int WW[6][3] = {{0, 1, 1},  {-1, 2, 1}, {-2, 2, 1},
                        {-2, 2, 2}, {-3, 2, 1}, {-3, 2, 1}};
  const int WM[6] = {2, 3, 3, 2, 1, 0};
  for (int e = 0; e < 6; e++) {
    printf("AND epoch %d: w=(%d,%d,%d) misclassified=%d\n", e + 1, tw[e][0],
           tw[e][1], tw[e][2], tm[e]);
    char nm[48];
    snprintf(nm, sizeof nm, "ch21 AND epoch %d end w (%d,%d,%d)", e + 1,
             WW[e][0], WW[e][1], WW[e][2]);
    CHECK(tw[e][0] == WW[e][0] && tw[e][1] == WW[e][1] && tw[e][2] == WW[e][2],
          nm);
    snprintf(nm, sizeof nm, "ch21 AND epoch %d misses %d", e + 1, WM[e]);
    CHECK(tm[e] == WM[e], nm);
  }
  printf("converged at epoch %d, final w=(%d,%d,%d)\n", conv, tw[conv - 1][0],
         tw[conv - 1][1], tw[conv - 1][2]);
  CHECK(conv == 6, "ch21 AND convergence epoch 6");
  CHECK(tw[5][0] == -3 && tw[5][1] == 2 && tw[5][2] == 1,
        "ch21 AND final w (-3,2,1)");
  int all_ok = 1;
  for (int i = 0; i < 4; i++) {
    int a = -3 + 2 * ANDP[i][0] + 1 * ANDP[i][1];
    int p = a ≥ 0 ? 1 : -1;
    all_ok = all_ok && p == ANDP[i][2];
  }
  CHECK(all_ok, "ch21 AND final w classifies all 4");
```

listing 156: six epochs of AND, end weights and misses pinned two checks a line (source: kdd/samples/src/Ch21/perceptron.c, lines 57-86)

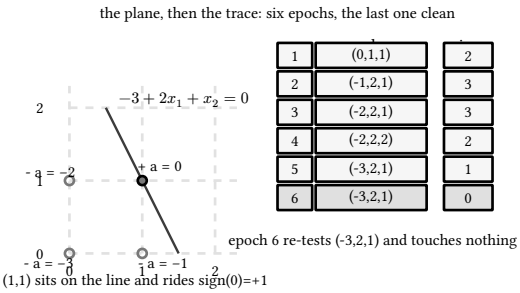


figure 84: AND in the plane, the boundary through (1,1), the epoch trace at right

21.2. xor breaks every line

XOR labels the diagonal pairs apart: (0,0) and (1,1) negative, (0,1) and (1,0) positive. A line separates two classes exactly when their convex hulls are disjoint, and here the hulls are the unit square's two diagonals, which cross at the center, so no separator exists at any precision, integer or otherwise. The sample carries a finite witness of that geometry: all 343 integer vectors in `{-3..3}^3` are tested against all four points.

The dry run: the witness prints `xor exhaustive: perfect weight vectors in {-3..3}^3 = 0`, pinned as "ch21 xor 0 perfect vectors in the box". Then the run, `xor epoch 1: w=(-1,-1,0) mis=3`, `xor epoch 2: w=(0,-1,0) mis=3`, and from epoch 3 on every line reads `w=(0,-1,0) mis=4`, the pair pinned as "ch21 xor epoch 1 end (-1,-1,0)" and "ch21 xor fixed point (0,-1,0) from epoch 2". The

fixed point is a closed loop of four mistakes: starting at (0,-1,0), the point (0,0) activates 0 and predicts + against -, moving to (-1,-1,0); (0,1) activates −1 and predicts - against +, moving to (0,-1,1); (1,0) activates −1 against +, moving to (1,0,1); (1,1) activates 2 against -, and the last update lands exactly back on (0,-1,0). Four corrections, zero net movement: the rule is spinning its wheels, not converging slowly.

```
/* XOR: exhaustive witness that no integer w in {-3..3}^3 is perfect */
int perfect = 0;
for (int b = -3; b ≤ 3; b++)
  for (int w1 = -3; w1 ≤ 3; w1++)
    for (int w2 = -3; w2 ≤ 3; w2++) {
      int ok = 1;
      for (int i = 0; i < 4; i++) {
        int a = b + w1 * XORP[i][0] + w2 * XORP[i][1];
        if ((a ≥ 0 ? 1 : -1) ≠ XORP[i][2]) {
          ok = 0;
          break;
        }
      }
      perfect += ok;
    }
printf("xor exhaustive: perfect weight vectors in {-3..3}^3 = %d\n", perfect);
CHECK(perfect == 0, "ch21 xor 0 perfect vectors in the box");
```

listing 157: the 343-vector exhaustive witness, zero perfect separators (source: kdd/samples/src/Ch21/perceptron.c, lines 88-104)

```
/* XOR: 10-epoch run, misses and fixed point */
int xw[10][3], xm[10];
run(XORP, 10, xw, xm);
const int XM[10] = {3, 3, 4, 4, 4, 4, 4, 4, 4, 4};
int fix_ok = 1;
for (int e = 0; e < 10; e++) {
  printf("xor epoch %d: w=(%d,%d,%d) mis=%d\n", e + 1, xw[e][0], xw[e][1],
         xw[e][2], xm[e]);
  char nm[48];
  snprintf(nm, sizeof nm, "ch21 xor epoch %d misses %d", e + 1, XM[e]);
  CHECK(xm[e] == XM[e], nm);
  if (e ≥ 1)
    fix_ok = fix_ok && xw[e][0] == 0 && xw[e][1] == -1 && xw[e][2] == 0;
}
CHECK(xw[0][0] == -1 && xw[0][1] == -1 && xw[0][2] == 0,
      "ch21 xor epoch 1 end (-1,-1,0)");
CHECK(fix_ok, "ch21 xor fixed point (0,-1,0) from epoch 2");
```

listing 158: ten epochs of XOR, the fixed point from epoch 2 on (source: kdd/samples/src/Ch21/perceptron.c, lines 106-122)

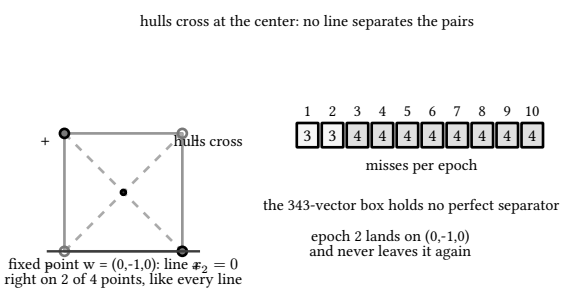


figure 85: the XOR square, crossing hulls, and the fixed-point line along the bottom

NOTE: the box is a witness, the crossing hulls are the proof

> Zero perfect vectors out of 343 only says no small integer separator exists, and the chapter would be overclaiming if it stopped there. The separability theorem closes the gap: a hyperplane separates the classes exactly when their convex hulls are disjoint, and XOR's classes are the two diagonals of the unit square, which intersect. No rescaling, no learning rate, no number of epochs changes that, and the perceptron convergence guarantee, which promises a finite clean epoch whenever the data are separable, stays honest by saying nothing at all when they are not.

21.3. a hidden layer buys xor

One layer cannot bend, so add a second. The network is 2-2-1 with all integer weights and step units, step(z)=1 exactly when z is at least 0: the first hidden unit is OR with weights (2,2,-1), the second is AND with (2,2,-3), and the output unit reads (2,-2,-1). Each triple is $\left(w_{x_1}, w_{x_2}, \text{bias}\right)$, the perceptron's (b, w1, w2) with the bias moved to the end.

The dry run: four lines, `mlp forward (0,0): z=(-1,-3) h=(0,0) z_o=-1 out=0`, `mlp forward (0,1): z=(1,-1) h=(1,0) z_o=1 out=1`, `mlp forward (1,0): z=(1,-1) h=(1,0) z_o=1 out=1`, and `mlp forward (1,1): z=(3,1) h=(1,1) z_o=-1 out=0`, sixteen checks, four per input, the last of each quartet reading like "ch21 mlp (1,1) out 0 == XOR". The inputs (0,1) and (1,0) produce identical columns, symmetry the fixture keeps visible. The output computes 2h1 - 2h2 - 1, which is OR-and-not-AND, and that is exactly XOR. Note that the AND unit here is the separator (-3,2,2) in the perceptron's bias-first ordering, a cousin of facet 1's (-3,2,1) that reads activation 1 instead of 0 on (1,1), so it classifies AND without leaning on the sign(0)=+1 crutch.

```c
/* hidden h1 = OR (2,2,-1), h2 = AND (2,2,-3); output (2,-2,-1), all
 * (w_x1, w_x2, bias) with step(z≥0)=1 */
static const int W1[2][3] = {{2, 2, -1}, {2, 2, -3}};
static const int W2[3] = {2, -2, -1};

static int step(int z) { return z ≥ 0 ? 1 : 0; }
```

listing 159: the hand-set integer weights, two hidden steps and a head (source: kdd/samples/src/Ch21/mlp.c, lines 25-30)

```c
  const int IN[4][2] = {{0, 0}, {0, 1}, {1, 0}, {1, 1}};
  const int WZ[4][2] = {{-1, -3}, {1, -1}, {1, -1}, {3, 1}};
  const int WH[4][2] = {{0, 0}, {1, 0}, {1, 0}, {1, 1}};
  const int WO[4] = {-1, 1, 1, -1};
  const int WOUT[4] = {0, 1, 1, 0};
  for (int k = 0; k < 4; k++) {
    int z[2], h[2];
    for (int u = 0; u < 2; u++) {
      z[u] = W1[u][0] * IN[k][0] + W1[u][1] * IN[k][1] + W1[u][2];
      h[u] = step(z[u]);
    }
    int zo = W2[0] * h[0] + W2[1] * h[1] + W2[2];
    int out = step(zo);
    printf("mlp forward (%d,%d): z=(%d,%d) h=(%d,%d) z_o=%d out=%d\n", IN[k][0],
           IN[k][1], z[0], z[1], h[0], h[1], zo, out);
```

listing 160: all four inputs, z, h, z_o, and out pinned as const tables (source: kdd/samples/src/Ch21/mlp.c, lines 33-47)

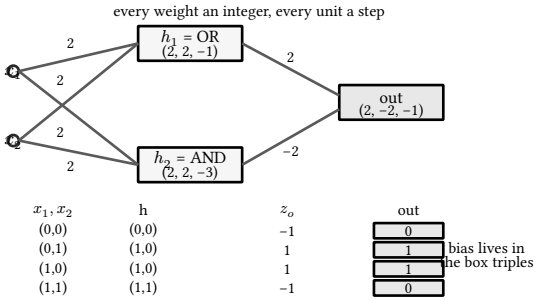


figure 86: the 2-2-1 wiring with weights on the edges, truth table beneath

21.4. one gradient step, exactly

Training means following a gradient, so take the smallest step that is still the real thing. Freeze the hidden layer of facet 3 at the value it holds on input (1,1), namely (1,1), and make the output linear with weights (1,1,-1) and target t=0 under the loss $1/2(\text{out} - t)^2$, whose derivative in out is just out minus t. Nothing transcendental enters, so every number is a dyadic rational and the sample asserts the updates with plain equality.

The dry run: `gradient row: out=1.0 delta=1.0 g=(1.0,1.0,1.0)` and `lr 0.5 updates: w=(0.5,0.5,-1.5)`. The forward sum is $1 \cdot 1 + 1 \cdot 1 + (-1) = 1$ against a target of 0, the delta is out $- t = 1$, the gradient of the loss in the three output weights is the delta times their inputs, $1 \cdot (1, 1, 1)$, and the half-rate update subtracts one half from each entry, landing on (0.5, 0.5, −1.5), pinned one check per weight as “ch21 updated w5 0.5 (D1 ==)”, “ch21 updated w6 0.5 (D1 ==)”, and “ch21 updated b3 −1.5 (D1 ==)”. The bias row is the one to watch in the next facet, because there the inputs are sigmoid outputs and the cleanliness disappears.

```
/* exact output-layer gradient: step hidden fixed at (1,1) (input (1,1)),
 * linear output w=(1,1,-1), t=0, loss 0.5(out-t)^2 */
int h1 = step(W1[0][0] * 1 + W1[0][1] * 1 + W1[0][2]);
int h2 = step(W1[1][0] * 1 + W1[1][1] * 1 + W1[1][2]);
CHECK(h1 == 1 && h2 == 1, "ch21 step hidden at (1,1) is (1,1)");
double w5 = 1.0, w6 = 1.0, b3 = -1.0;
double out = w5 * h1 + w6 * h2 + b3;
double delta = out - 0.0;
double g5 = delta * h1, g6 = delta * h2, gb = delta;
printf("gradient row: out=%.1f delta=%.1f g=(%.1f,%.1f,%.1f)\n", out, delta,
       g5, g6, gb);
CHECK(out == 1.0, "ch21 gradient out 1");
CHECK(delta == 1.0, "ch21 gradient delta 1");
CHECK(g5 == 1.0 && g6 == 1.0 && gb == 1.0, "ch21 gradient g (1,1,1)");
double w5n = w5 - 0.5 * g5, w6n = w6 - 0.5 * g6, b3n = b3 - 0.5 * gb;
printf("lr 0.5 updates: w=(%.1f,%.1f,%.1f)\n", w5n, w6n, b3n);
CHECK(w5n == 0.5, "ch21 updated w5 0.5 (D1 ==)");
CHECK(w6n == 0.5, "ch21 updated w6 0.5 (D1 ==)");
CHECK(b3n == -1.5, "ch21 updated b3 -1.5 (D1 ==)");
```

listing 161: one linear output, its delta, gradient, and dyadic update (source: kdd/samples/src/Ch21/mlp.c, lines 63-81)

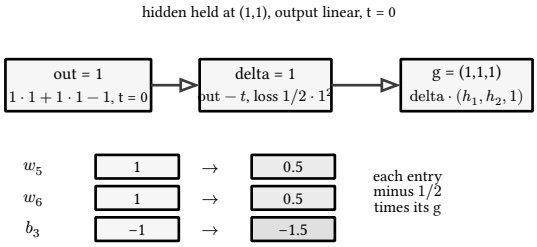


figure 87: the one step as a pipeline, the three weights before and after

21.5. backpropagation, one number at a time

The full net swaps steps for sigmoids and lets every weight move. Hidden rows (0.5, −0.25, 0.125) and (0.25, 0.5, −0.125), output row (0.75, −0.5, 0.25), input (1,0), target 1, learning rate 0.5, the same $1/2(a-t)^2$ loss. The chain rule then runs backwards: the output delta is $a_o - t$, the gradient in an output weight is that delta times the hidden activation feeding it, and each hidden delta is $a(1-a)$, the sigmoid's own slope, times the output weight times the output delta.

The dry run: the forward half prints `z_h=(0.625, 0.125)`, both dyadic and asserted with equality because x2=0 kills the cross terms leaving $0.5 + 0.125$ and $0.25 - 0.125$, then `a_h=(0.65135486466605419, 0.53120937337375629)`, then `z_o=0.47291146181266247 a_o=0.61607262845280919 loss=0.073700113311567353`. The backward half prints `delta_o=-0.38392737154719081 g_wo=(-0.25007296113571437, -0.20394581846061655, -0.38392737154719081)`, then `delta_h=(-0.065390041033891275, 0.047803944016977289)`, the second entry positive only because the middle output weight −0.5 flips its sign, then the hidden gradient rows `g_wh1=(-0.065390041033891275, -0, -0.065390041033891275)` and `g_wh2=(0.047803944016977289, 0, 0.047803944016977289)`, and the updated output row `w_o'=(0.87503648056785721, -0.39802709076969173, 0.4419636857735954)`. The middle entries of the hidden rows are the dead inputs: x2 is 0, so those weights see no gradient at all, and the print even shows the first row's zero as −0, a negative delta times a positive zero. Both are pinned exactly, "ch21 g_wh1_w2 exactly 0 (D0)" and "ch21 g_wh2_w2 exactly 0 (D0)", while every sigmoid-touching number is pinned to 17 digits at relative 1e-15, "ch21 loss 0.07370011331156735 (D2)" among them.

```
/* forward */
double zh[2], ah[2];
for (int i = 0; i < 2; i++) {
  zh[i] = WH[i][0] * x1 + WH[i][1] * x2 + WH[i][2];
  ah[i] = 1.0 / (1.0 + exp(-zh[i]));
}
double zo = WO[0] * ah[0] + WO[1] * ah[1] + WO[2];
double ao = 1.0 / (1.0 + exp(-zo));
double loss = 0.5 * (ao - t) * (ao - t);
printf("z_h=(%.17g, %.17g)\n", zh[0], zh[1]);
CHECK(zh[0] == 0.625, "ch21 z_h1 0.625 (dyadic ==)");
CHECK(zh[1] == 0.125, "ch21 z_h2 0.125 (dyadic ==)");
printf("a_h=(%.17g, %.17g)\n", ah[0], ah[1]);
CHECK(near(ah[0], 0.6513548646660542), "ch21 a_h1 0.6513548646660542 (D2)");
CHECK(near(ah[1], 0.5312093733737563), "ch21 a_h2 0.5312093733737563 (D2)");
printf("z_o=%.17g a_o=%.17g loss=%.17g\n", zo, ao, loss);
CHECK(near(zo, 0.47291146181266247), "ch21 z_o 0.47291146181266247 (D2)");
CHECK(near(ao, 0.6160726284528092), "ch21 a_o 0.6160726284528092 (D2)");
CHECK(near(loss, 0.07370011331156735), "ch21 loss 0.07370011331156735 (D2)");
```

listing 162: forward: dyadic preactivations, sigmoid pins, the loss (source: kdd/samples/src/Ch21/backprop.c, lines 36-54)

```
/* backward: delta_o = a_o - t, g_wo = delta_o * (a_h, 1) */
double d_o = ao - t;
double g_wo[3] = {d_o * ah[0], d_o * ah[1], d_o};
printf("delta_o=%.17g g_wo=(%.17g, %.17g, %.17g)\n", d_o, g_wo[0], g_wo[1],
       g_wo[2]);
CHECK(near(d_o, -0.3839273715471908),
      "ch21 delta_o -0.3839273715471908 (D2)");
CHECK(near(g_wo[0], -0.25007296113571437),
      "ch21 g_wo1 -0.25007296113571437 (D2)");
CHECK(near(g_wo[1], -0.20394581846061655),
      "ch21 g_wo2 -0.20394581846061655 (D2)");
CHECK(near(g_wo[2], -0.3839273715471908),
      "ch21 g_wo3 -0.3839273715471908 (D2)");

/* hidden deltas: a(1-a) * w_o * delta_o; x2=0 kills the w2 gradients */
double dh[2], g_wh[2][3];
for (int i = 0; i < 2; i++) {
  dh[i] = ah[i] * (1.0 - ah[i]) * WO[i] * d_o;
  g_wh[i][0] = dh[i] * x1;
  g_wh[i][1] = dh[i] * x2;
  g_wh[i][2] = dh[i];
}
printf("delta_h=(%.17g, %.17g)\n", dh[0], dh[1]);
CHECK(near(dh[0], -0.06539004103389127),
      "ch21 delta_h1 -0.06539004103389127 (D2)");
CHECK(near(dh[1], 0.04780394401697729),
      "ch21 delta_h2 0.04780394401697729 (D2)");
printf("g_wh1=(%.17g, %.17g, %.17g)\n", g_wh[0][0], g_wh[0][1], g_wh[0][2]);
printf("g_wh2=(%.17g, %.17g, %.17g)\n", g_wh[1][0], g_wh[1][1], g_wh[1][2]);
CHECK(g_wh[0][1] == 0.0, "ch21 g_wh1_w2 exactly 0 (D0)");
CHECK(g_wh[1][1] == 0.0, "ch21 g_wh2_w2 exactly 0 (D0)");
```

listing 163: backward: deltas, both gradient rows, the dead column (source: kdd/samples/src/Ch21/backprop.c, lines 56-86)

```
/* updates: w' = w - lr * g */
double won[3];
for (int i = 0; i < 3; i++)
  won[i] = WO[i] - lr * g_wo[i];
printf("w_o'=(%.17g, %.17g, %.17g)\n", won[0], won[1], won[2]);
CHECK(near(won[0], 0.8750364805678572), "ch21 w_o1' 0.8750364805678572 (D2)");
CHECK(near(won[1], -0.3980270907696917),
      "ch21 w_o2' -0.3980270907696917 (D2)");
CHECK(near(won[2], 0.4419636857735954), "ch21 w_o3' 0.4419636857735954 (D2)");
```

listing 164: the half-rate update on the output row (source: kdd/samples/src/Ch21/backprop.c, lines 88-96)

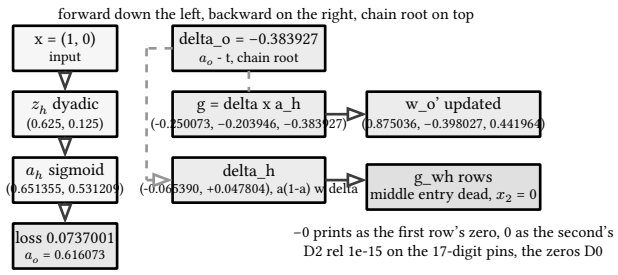


figure 88: one example down the net, backward chain root on top, dead column boxed

PITFALL: a bit-exact pin on a sigmoid is a lie

> The generator prints 17 significant digits and its two consecutive runs agree exactly, so the pins themselves are stable. The pinned sample still refuses to assert equality, because exp comes from the toolchain's libm and differs by an ulp or two between builds, and one ulp in a gradient becomes visible drift after enough half-rate steps. The honest contract is the one the sample implements: relative 1e-15 against the printed pin for anything a sigmoid touched, plain equality for the dyadic preactivations, and exact zero for the dead column, where x2=0 multiplies the gradient away before any transcendental can touch it.

sources: all 69 pinned values, both point sets, the update and sign conventions, the step and sigmoid units, and the D0/D1/D2 class assignments are witnessed by kdd-contract-s4s5.md and playground/kdd-matrix/gen_s4.py, run 2026-09-22, exit 0, the backprop row double-run identical. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch21`, 28 + 23 + 18 checks in chapter 21 of the kdd suite.

22. support vector machines

Three samples carry the chapter: `margin.c` solves a four-point fixture by hand and then prices a soft-margin grid exactly, 20 checks, `kernel.c` verifies the degree-2 polynomial kernel as an integer identity, 6 checks, and `smo.c` runs Platt's sequential minimal optimization from a cold start to the analytic plane, 14 checks. The chapter makes 4 moves: the hard-margin geometry with its margin in exact surds, the soft margin and its flat edge, the kernel trick that separates a circle without ever building the feature vector, and SMO's three updates from zero. Every behavioral claim below is one of the 40 checks of chapter 22's samples or a named note of the contract sheet. The perceptron of book 14, ch 21 (knowledge discovery, neural networks) stops at the first plane that separates, this chapter is the machinery that keeps going to the widest one, and book 14, ch 23 (knowledge discovery, bagging, boosting, and random forests) will abandon single planes for committees.

22.1. the hard margin, solved by hand

Among all planes separating the classes, the widest street is wanted, and scaling w and b by a positive constant moves no points, so the canonical convention fixes the street's edges at $yf(x) = 1$ with $f(x) = w \cdot x + b$. Maximizing the street width $2/|w|$ then means minimizing $w \cdot w/2$, a problem whose solution touches the data at only a few points. The fixture is plus (3,3) and (3,4) against minus (1,1) and (0,0), small enough to solve with no solver at all: the widest street is centered on the closest cross-class pair, midpoint (2,2), normal direction (1,1), the plane $x + y = 4$.

The dry run: the closest cross pair prints `closest cross pair (3,3)/(1,1) d2=8`, squared distance 8, and the canonical plane `w=(1/2, 1/2), b = -2` prints as `canonical w=(0.5,0.5) b=-2`. The four functional margins read `f(3,3)=1 yf=1`, `f(3,4)=1.5 yf=1.5`, `f(1,1)=-1 yf=1`, `f(0,0)=-2 yf=2`: exactly the two supports sit at $yf = 1$, so check "ch22 support vectors {(3,3), (1,1)}" names the only two points the plane depends on, and dropping either (3,4) or (0,0) changes nothing. Then `w.w=0.5 |w|=1/sqrt2`, and the surds close as integer facts: `w_hat=(1,1)/sqrt(2): p^2+q^2=2, w_hat^2=2/2=1` asserts the unit normal's squared length as k/k with $p^2 + q^2 = k = 2$, never a floating comparison, and `margin=1/|w| =sqrt(2) margin^2=2 b_hat^2=8` pins $1/|w| = \sqrt{2}$ and $b\hat{} = -2\sqrt{2}$ through their integer squares 2 and 8. The reconciliation is check "ch22 min cross d^2=8=(2*margin)^2": the street's full width $2\sqrt{2}$ is exactly the gap that fixed it.

```
  /* ---- hard margin: closest cross pair fixes the canonical plane,
   * w = 2*(p-n)/|p-n|^2, b = 1 - w.p, supports sit at y f = 1 ---- */
  int bd2 = 1 << 30, bp = 0, bn = 0;
  for (int a = 0; a < 2; a++)
    for (int c = 0; c < 2; c++) {
      int dx = POS[a][0] - NEG[c][0], dy = POS[a][1] - NEG[c][1];
      int d2 = dx * dx + dy * dy;
      if (d2 < bd2) {
        bd2 = d2;
        bp = a;
        bn = c;
      }
    }
  int dx = POS[bp][0] - NEG[bn][0], dy = POS[bp][1] - NEG[bn][1];
  double w1 = 2.0 * dx / bd2;
  double w2 = 2.0 * dy / bd2;
  double b = 1.0 - (w1 * POS[bp][0] + w2 * POS[bp][1]);
  printf("closest cross pair (%d,%d)/(%d,%d) d2=%d\n", POS[bp][0], POS[bp][1],
         NEG[bn][0], NEG[bn][1], bd2);
  printf("canonical w=(%g,%g) b=%g\n", w1, w2, b);
  CHECK(w1 == 0.5 && w2 == 0.5 && b == -2.0,
        "ch22 canonical w=(1/2,1/2), b=-2 (D1)");
```

listing 165: the closest cross pair fixes the canonical plane, w and b as dyadics (source: kdd/samples/src/Ch22/margin.c, lines 35-56)

```
  double yf[4];
  for (int a = 0; a < 2; a++) {
    yf[a] = w1 * POS[a][0] + w2 * POS[a][1] + b;
    printf("f(%d,%d)=%g yf=%g\n", POS[a][0], POS[a][1], yf[a], yf[a]);
  }
  for (int c = 0; c < 2; c++) {
    yf[2 + c] = -(w1 * NEG[c][0] + w2 * NEG[c][1] + b);
    printf("f(%d,%d)=%g yf=%g\n", NEG[c][0], NEG[c][1], -yf[2 + c], yf[2 + c]);
  }
  CHECK(yf[0] == 1.0, "ch22 yf(3,3)=1 (support)");
  CHECK(yf[1] == 1.5, "ch22 yf(3,4)=3/2");
  CHECK(yf[2] == 1.0, "ch22 yf(1,1)=1 (support)");
  CHECK(yf[3] == 2.0, "ch22 yf(0,0)=2");
  CHECK(yf[0] == 1.0 && yf[2] == 1.0 && yf[1] > 1.0 && yf[3] > 1.0,
        "ch22 support vectors {(3,3),(1,1)}");
```

listing 166: functional margins at all four points, the supports sit at yf = 1 (source: kdd/samples/src/Ch22/margin.c, lines 58-72)

```
/* unit normal w_hat=(p,q)/sqrt(k): the integer surd identity */
int g = dx < dy ? dx : dy; /* gcd of (2,2) is 2 */
while (g > 0 && (dx % g || dy % g))
  g--;
int p = dx / g, q = dy / g, k = p * p + q * q;
printf("w_hat=(%d,%d)/sqrt(%d): p^2+q^2=%d, w_hat^2=%d/%d=1\n", p, q, k, k, k,
       k);
CHECK(k == 2, "ch22 surd integer p^2+q^2=2");
CHECK(k / k == 1, "ch22 surd integer w_hat^2=1");
double margin2 = 1.0 / ww;
double bhat2 = b * b / ww;
printf("w_hat=(sqrt(2)/2, sqrt(2)/2) b_hat=-2*sqrt(2)\n");
printf("margin=1/|w|=sqrt(2) margin^2=%g b_hat^2=%g\n", margin2, bhat2);
CHECK(margin2 == 2.0, "ch22 surd integer margin^2=2");
CHECK(bhat2 == 8.0, "ch22 surd integer b_hat^2=8");
CHECK(bd2 == 8 && 4.0 * margin2 == 8.0, "ch22 min cross d^2=8=(2*margin)^2");
```

listing 167: the surd ledger: every irrational asserted through its integer square (source: kdd/samples/src/Ch22/margin.c, lines 78-93)

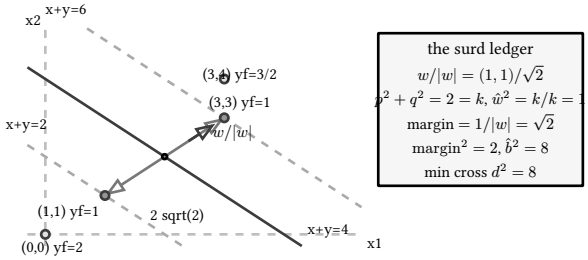


figure 89: the hard-margin street: supports at yf = 1, margin sqrt(2) per side

22.2. the soft margin and its flat edge

A hard margin assumes the street can stay empty. The soft margin lets points inside pay rent, $G(w,b) = w^2/2 + C\sum \max(0, 1 - yf(x))$, and the fixture is 1D with $C = 4$: plus at 1, minus at −1, and a second plus at $-1/2$ that sits inside any clean street. On the edge $b = w - 1$, where the minus point lands exactly on its margin, only the intruder's hinge is active, $G = w^2/2 + 4(2 - w/2)$, and the derivative $w - 2$ vanishes at $w = 2$, $b = 1$, $G = 6$.

The dry run: the sample sweeps the exact-rational grid $w = i/200$, $b = j/200$, scaled by 80000 into integers where $80000G = i^2 + 800\sum h_k$ exactly, and prints `grid witness 801x801 step 1/200: min 80000*G=480000 at i=400 j=200, points at G=6: 201`. The minimum $480000/80000 = 6$ sits at $i = 400$, $j = 200$, which is $w = 2$, $b = 1$, and no lattice point lands below. The 201 hits are not noise, they are the flat edge: `flat edge w=2, b in [1,2]: 201 of 201 grid points at G=6` says every b in $[1, 2]$ scores 6 at $w = 2$, the minus point's slack trading exactly against the plus point's as the street slides. At the pinned endpoint the rent ledger prints `xi=(0,0,400)/400, objective=480000/80000`, the intruder alone pays, and check "ch22 slack point on the boundary yf=0 (f(-1/2)=0)" places it exactly on the decision boundary, one full slack deep. The closing line prices both streets, `margin 1/|w|=0.5; clean-pair hard margin w=1 b=0 margin=1`: renting to the intruder halves the margin.

```
long bestM = 2147483647L;
int besti = 0, bestj = 0;
long hits = 0;
for (int i = -200; i ≤ 600; i++)
  for (int j = -300; j ≤ 500; j++) {
    long h1 = hinge(400 - (2L * i + 2L * j));  /* x=1,  y=+1 */
    long h2 = hinge(400 - (2L * i - 2L * j));  /* x=-1, y=-1 */
    long h3 = hinge(400 - (-1L * i + 2L * j)); /* x=-1/2, y=+1 */
    long M = (long)i * i + 800 * (h1 + h2 + h3);
    if (M < bestM) {
      bestM = M;
      besti = i;
      bestj = j;
    }
    if (M == 480000L)
      hits++;
  }
printf("grid witness 801x801 step 1/200: min 80000*G=%ld at i=%d j=%d, "
       "points at G=6: %ld\n",
       bestM, besti, bestj, hits);
```

listing 168: the whole 801x801 grid priced in exact integers, 80000 G (source: kdd/samples/src/Ch22/margin.c, lines 98-117)

```
long flat = 0;
for (int j = 200; j ≤ 400; j++) {
  long h1 = hinge(400 - (2L * 400 + 2L * j));
  long h2 = hinge(400 - (2L * 400 - 2L * j));
  long h3 = hinge(400 - (-1L * 400 + 2L * j));
  if ((long)400 * 400 + 800 * (h1 + h2 + h3) == 480000L)
    flat++;
}
printf("flat edge w=2, b in [1,2]: %ld of 201 grid points at G=6\n", flat);
CHECK(flat == 201, "ch22 soft flat edge w=2, b in [1,2]");

/* slacks at the pinned optimum (i=400, j=200) */
long h1 = hinge(400 - (2L * 400 + 2L * 200));
long h2 = hinge(400 - (2L * 400 - 2L * 200));
long h3 = hinge(400 - (-1L * 400 + 2L * 200));
printf("xi=(%ld,%ld,%ld)/400, objective=%ld/80000\n", h1, h2, h3,
       (long)400 * 400 + 800 * (h1 + h2 + h3));
CHECK(h1 == 0 && h2 == 0 && h3 == 400, "ch22 soft xi=(0,0,1)");
CHECK((long)400 * 400 + 800 * (h1 + h2 + h3) == 480000L,
      "ch22 soft objective 6");
CHECK(-1L * 400 + 2L * 200 == 0,
      "ch22 slack point on the boundary yf=0 (f(-1/2)=0)");
```

listing 169: the flat edge walk and the slacks at the pinned optimum (source: kdd/samples/src/Ch22/margin.c, lines 122-143)

```
/* margins: soft optimum 1/|w|=1/2; hard margin on the clean pair
 * {1,-1} alone is w=1, b=0, margin 1 */
double wm = 1.0 / 2.0;
double cw = 2.0 * (1 - (-1)) / ((1 - (-1)) * (1 - (-1)));
double cb = 1.0 - cw * 1;
printf("margin 1/|w|=%g; clean-pair hard margin w=%g b=%g margin=%g\n", wm,
       cw, cb, 1.0 / cw);
CHECK(wm == 0.5 && cw == 1.0 && cb == 0.0 && 1.0 / cw == 1.0,
      "ch22 margins 1/2 (soft) and 1 (clean-pair hard)");
```

listing 170: the soft street 1/2 wide against the clean-pair street 1 wide (source: kdd/samples/src/Ch22/margin.c, lines 145-153)

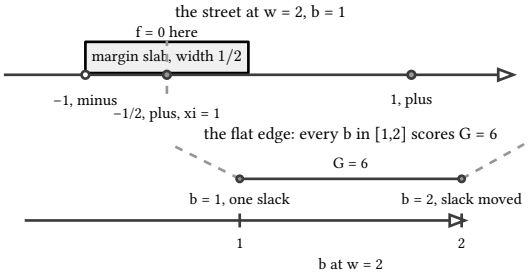


figure 90: the soft-margin fixture on a line, and the flat edge of its optimum

NOTE: a flat optimum is a choice point

At $w = 2$ every b in $[1, 2]$ attains $G = 6$, so "the" optimum is a segment, and a segment cannot be asserted. The sheet pins the endpoint $(2, 1)$, the one point of the edge where exactly one example carries slack, the same way book 14, ch 1 (knowledge discovery, the knowledge discovery process) pinned keep-first for duplicates: a convention named before it is checked. The KKT subgradient test agrees, $dG/dw = w + 2 - [0, 4]$ and $dG/db = -4 + [0, 4]$ both contain 0 there, and the grid witness corroborates over 801 x 801 exact-rational points.

22.3. the kernel trick, a circle by a plane

Four positives on the unit circle around a negative at the origin have no separating line: any line leaving the origin behind cuts the ring. The degree-2 map $\varphi(x) = \left(x_1^2, \sqrt{2}x_1x_2, x_2^2\right)$ lifts them into a space where a plane exists, and the plane is almost embarrassing, $z_1 + z_3 = 1/2$, because every ring point has $z_1 + z_3 = x_1^2 + x_2^2 = 1$ and the origin has 0. The trick is that the dual and the plane both need only inner products, and $(x \cdot z)^2 = \varphi(x) \cdot \varphi(z)$ holds exactly: the middle coordinate's two $\sqrt{2}$ factors multiply to the rational 2, so the identity is verified in integers.

The dry run: three pairs check the identity both ways, `K((1,0),(0,1))=0 phi.phi(integers)=0`, `K((2,1),(1,3))=25 phi.phi(integers)=25`, `K((1,1),(1,1))=4 phi.phi(integers)=4`. The kernel squares the integer dot products 0, 5, 2 into 0, 25, 4, and the explicit feature dot, $x_1^2z_1^2 + 2(x_1x_2)(z_1z_2) + x_2^2z_2^2$, agrees every time without a single irrational step. Classification then never builds a φ: the ring prints four lines `ring (1,0): z1+z3=1 > 1/2 → +` down to `(0,-1)`, the origin prints `origin: z1+z3=0 < 1/2 → -`, and check "ch22 circle set classified 5/5" closes it. The predictor is the integer test $2(x_1^2 + x_2^2) > 1$, the plane scaled by 2.

```
/* K(x,z) = (x.z)^2 */
static int poly_kernel(const int x[2], const int z[2]) {
  int dot = x[0] * z[0] + x[1] * z[1];
  return dot * dot;
}

/* phi(x).phi(z) in integers: x1^2*z1^2 + 2*(x1*x2)*(z1*z2) + x2^2*z2^2,
 * the middle term doubled because sqrt(2)*sqrt(2)=2 */
static int phi_dot(const int x[2], const int z[2]) {
  return x[0] * x[0] * z[0] * z[0] + 2 * (x[0] * x[1]) * (z[0] * z[1]) +
         x[1] * x[1] * z[1] * z[1];
}

/* decision in feature space: + iff z1+z3 > 1/2, scaled by 2 to stay
 * in integers (x1^2+x2^2 is z1+z3 on the original coordinates) */
static int circle_predict(const int x[2]) {
  return 2 * (x[0] * x[0] + x[1] * x[1]) > 1;
}
```

listing 171: the kernel, the explicit feature dot, and the plane as an integer test (source: kdd/samples/src/Ch22/kernel.c, lines 26-43)

```
  const int pairs[3][2][2] = {
      {{1, 0}, {0, 1}}, {{2, 1}, {1, 3}}, {{1, 1}, {1, 1}}};
  const int want[3] = {0, 25, 4};
  for (int m = 0; m < 3; m++) {
    const int *x = pairs[m][0];
    const int *z = pairs[m][1];
    int K = poly_kernel(x, z);
    int D = phi_dot(x, z);
    printf("K((%d,%d),(%d,%d))=%d phi.phi(integers)=%d\n", x[0], x[1], z[0],
           z[1], K, D);
    char nm[64];
    snprintf(nm, sizeof nm, "ch22 kernel identity pair %d = %d", m + 1,
             want[m]);
    CHECK(K == want[m] && D == want[m] && K == D, nm);
  }
```

listing 172: three identity pairs, kernel against feature dot, both integers (source: kdd/samples/src/Ch22/kernel.c, lines 46-60)

```
  const int ring[4][2] = {{1, 0}, {0, 1}, {-1, 0}, {0, -1}};
  int ring_ok = 0;
  for (int m = 0; m < 4; m++) {
    int pred = circle_predict(ring[m]);
    ring_ok += pred == 1;
    printf("ring (%d,%d): z1+z3=1 > 1/2 -> %s\n", ring[m][0], ring[m][1],
           pred ? "+" : "-");
  }
  CHECK(ring_ok == 4, "ch22 plane z1+z3=1/2 puts all 4 ring points at +");
  const int origin[2] = {0, 0};
  int op = circle_predict(origin);
  printf("origin: z1+z3=0 < 1/2 -> %s\n", op ? "+" : "-");
  CHECK(op == 0, "ch22 plane z1+z3=1/2 puts the origin at -");
  CHECK(ring_ok + (1 - op) == 5, "ch22 circle set classified 5/5");
```

listing 173: the ring classifies 5/5 through the plane alone (source: kdd/samples/src/Ch22/kernel.c, lines 62-75)

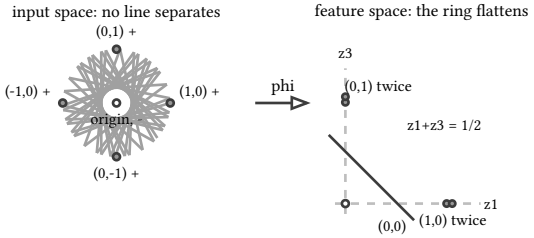


figure 91: left: no line separates, right: phi flattens the ring onto z1+z3=1

22.4. smo, three updates from zero

The dual, the same minimization rewritten with one multiplier per point, keeps only inner products, $w = \sum_i a_i y_i x_i$ with $a_i \geq 0$, so the solver's whole job is the multiplier vector. SMO, Platt's MSR-TR-98-14, moves two multipliers at a time: take the lowest-index KKT violator as i, take j maximizing $|E_i - E_j|$ with ties by index, fall back to the next-best j when the box is degenerate ($L = H$), the step is uphill ($\eta \geq 0$), or the move stalls, then clip a_j into its box, compensate a_i to keep the constraint sum, and set b by the b_1/b_2 midpoint rule. The fixture is 5 points, plus (6,6), (6,8), (4,6) and minus (2,2), (0,0), whose analytic solution is the plane $x + 2y = 11$ in canonical form $w = (1/5, 2/5)$, $b = -11/5$, with dual multipliers $a_2 = a_3 = 1/10$.

NOTE: smo selection, the six moves restated

1. scan the points in index order and take the lowest-index KKT violator as i. 2. take j maximizing $|E_i - E_j|$, ties by index.
3. when the box is degenerate ($L = H$), the step is uphill ($\eta \geq 0$), or the move stalls, fall back to the next-best j.
4. clip a_j into its box. 5. compensate a_i to keep the constraint sum. 6. set b by the b_1/b_2 midpoint rule.

The dry run: `smo updates: 3` is the whole trace. `iter 1: i=0 j=3 alpha_i=0.0625 alpha_j=0.0625 b=-2`: from all-zero every error is $E_i = -y_i$, the widest gap pairs (6,6) with (2,2), $\eta = 2 \cdot 24 - 72 - 8 = -32$, and both multipliers rise to 1/16. `iter 2: i=2 j=0 alpha_i=0.0625 alpha_j=0 b=-1`: the first candidate $j = 1$ is skipped for $L = H$, both class plus at $a = 0$, and a_0 clips back down to its box floor 0. `iter 3: i=3 j=2 alpha_i=0.10000000000000001 alpha_j=0.10000000000000001 b=-2.2000000000000002` lands both at exactly 1/10. The converged line prints `converged alphas: 0 0 0.10000000000000001 0.10000000000000001 0 b=-2.2000000000000002`, and the per-point comparison against the analytic plane bottoms out at `x=(6,6) y=+1 f_smo=1.3999999999999995 f_analytic=1.3999999999999999 |diff|=4.441e-16`, closing `ch22 smo: 14 checks (max analytic diff 4.441e-16)`.

```
      double E_i = f_of(i) - Y[i];
      if ((Y[i] * E_i < -tol && alphas[i] < Cs) ||
          (Y[i] * E_i > tol && alphas[i] > 0)) {
        double E[5];
        for (int k = 0; k < 5; k++)
          E[k] = f_of(k) - Y[k];
        /* j ≠ i ordered by descending |E_i - E_j|, ties by index */
        int order[4], no = 0;
        for (int j = 0; j < 5; j++)
          if (j ≠ i)
            order[no++] = j;
        for (int a = 1; a < no; a++) {
          int oj = order[a];
          double ka = fabs(E_i - E[oj]);
          int at = a;
          while (at > 0 &&
                 (fabs(E_i - E[order[at - 1]]) < ka ||
                  (fabs(E_i - E[order[at - 1]]) == ka && order[at - 1] > oj))) {
            order[at] = order[at - 1];
            at--;
          }
          order[at] = oj;
        }
```

listing 174: violator scan and the j ordering by descending error gap, ties by index (source: kdd/samples/src/Ch22/smo.c, lines 62-84)

```
          double L, H;
          if (Y[i] ≠ Y[j]) {
            L = alphas[j] - alphas[i];
            H = Cs + alphas[j] - alphas[i];
            L = L < 0.0 ? 0.0 : L;
            H = H > Cs ? Cs : H;
          } else {
            L = alphas[i] + alphas[j] - Cs;
            H = alphas[i] + alphas[j];
            L = L < 0.0 ? 0.0 : L;
            H = H > Cs ? Cs : H;
          }
          if (L == H)
            continue;
          double eta = 2 * K[i][j] - K[i][i] - K[j][j];
          if (eta ≥ 0)
            continue;
          double ai_old = alphas[i], aj_old = alphas[j];
          double aj_new = alphas[j] - Y[j] * (E_i - E[j]) / eta;
          if (!(L ≤ aj_new && aj_new ≤ H)) {
```

listing 175: the L, H box, the eta test, and the unclipped step (source: kdd/samples/src/Ch22/smo.c, lines 87-106)

```
        alphas[i] += Y[i] * Y[j] * (aj_old - alphas[j]);
        double b1 = b - E_i - Y[i] * K[i][i] * (alphas[i] - ai_old) -
                    Y[j] * K[i][j] * (alphas[j] - aj_old);
        double b2 = b - E[j] - Y[i] * K[i][j] * (alphas[i] - ai_old) -
                    Y[j] * K[j][j] * (alphas[j] - aj_old);
        if (0 < alphas[i] && alphas[i] < Cs)
          b = b1;
        else if (0 < alphas[j] && alphas[j] < Cs)
          b = b2;
        else
          b = (b1 + b2) / 2;
        num_changed++;
        iters[nupd].i = i;
        iters[nupd].j = j;
```

listing 176: the constraint-sum compensation and Platt's b1/b2 bias rule (source: kdd/samples/src/Ch22/smo.c, lines 113-126)

```
double maxdiff = 0.0;
int sign_ok = 1, agree = 1;
for (int k = 0; k < 5; k++) {
  double f = f_of(k), fa = analytic(X[k]);
  double d = fabs(f - fa);
  if (d > maxdiff)
    maxdiff = d;
  agree = agree && d < 1e-9;
  sign_ok = sign_ok && (f > 0) == (Y[k] > 0);
  printf("x=(%d,%d) y=%+d f_smo=%.17g f_analytic=%.17g |diff|=%.3e\n",
         X[k][0], X[k][1], Y[k], f, fa, d);
}
CHECK(agree, "ch22 smo decision matches analytic within 1e-9 (D2)");
CHECK(sign_ok, "ch22 smo signs agree with labels on all 5");

printf("ch22 smo: %d checks (max analytic diff %.3e)\n", checks, maxdiff);
```

listing 177: the solver's f checked against the analytic plane on all 5 points (source: kdd/samples/src/Ch22/smo.c, lines 180-195)

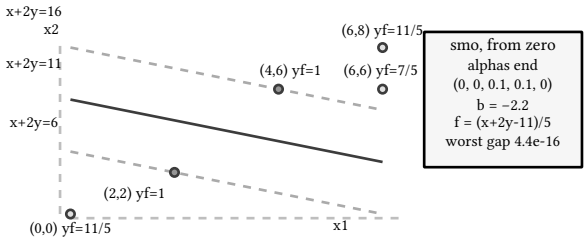


figure 92: the 5-point fixture and the plane SMO finds, supports at (4,6) and (2,2)

PITFALL: the trace belongs to the pinned heuristics

Which pair SMO moves is heuristic, and the sheet fixes the heuristics: lowest-index violator, widest error gap, index tie-breaks, the fallback ladder. A different but equally legal choice of j in update 2 gives a different intermediate trace on the way to the same dual optimum, so the D3 pins name this sample's rules, not SMO's. What is not heuristic is the arithmetic class: every pin is asserted with == on the double because the loop only adds, subtracts, multiplies, and divides values seeded from integer kernel entries, and IEEE 754 exactly rounds those four operations on every conforming platform. The libm `exp` that forced the backprop pins of book 14, ch 21 (knowledge discovery, neural networks) down to tolerance never enters a linear kernel, so bit-exactness is honest here.

sources: SMO selection rules and the b_1/b_2 bias update per John C. Platt, Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines, technical report MSR-TR-98-14, Microsoft Research, 1998, banked by kdd-contract-s4s5.md. All 40 pinned values witnessed by the same sheet and playground/kdd-matrix/gen_s5.py, run 2026-09-22, exit 0, the SMO trace reconciled update by update and identical across two consecutive runs. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch22`, 20 + 6 + 14 checks in chapter 22 of the kdd suite.

23. bagging, boosting, and random forests

Three samples carry the chapter: `bagging.c` draws five bootstrap replicates and majority-votes their stumps, 17 checks, `adaboost.c` runs three rounds of exact-fraction reweighting and casts the final vote as an integer comparison, 19 checks, and `rf.c` grows three feature-subset trees and prices their importances, 18 checks. The chapter makes 4 moves: resampling as the noise source a vote can average away, reweighting as the loop that concentrates weight on misses, the logarithm that never has to be computed, and the forest that turns importance into a measured quantity. Every behavioral claim below is one of the 54 checks of chapter 23's samples or a banked note of the contract sheet. The single plane book 14, ch 22 (knowledge discovery, support vector machines) hunted for is gone: the model is now a committee of the gini stumps book 14, ch 14 (knowledge discovery, decision trees: cart and continuous splits) grew, its errors are counted the way book 14, ch 17 (knowledge discovery, classifier evaluation) counts them, and book 14, ch 24 (knowledge discovery, class imbalance) will stress these same votes on a skewed class mix.

23.1. five resamples, one vote

Bagging's bet is that a learner's mistakes on resampled data are partly noise: draw the training set with replacement a few times, fit one stump per replicate, and let a majority vote cancel the idiosyncrasies. The fixture is 9 rows, x = 1 to 9 with labels +,+,-,+,-,-,+,-,-, the learner is the both-direction stump with the pinned candidate order and tie rule, and the wave LCG seeded 3076 draws 5 replicates of 9 draws each, every draw kept, repeats and all.

The dry run: on the full data the stump prints `full-data stump: L2 err 2/9`, wrong only at x = 4 and x = 7, both plus rows beyond the boundary. The replicates print `bootstrap 1 idx=[8,3,6,7,1,0,6,8,8] stump=L7 err=0/9`, `bootstrap 2 idx=[1,8,3,1,5,4,1,6,5] stump=L4 err=1/9`, `bootstrap 3 idx=[6,0,4,7,4,4,6,4,6] stump=R6 err=2/9`, `bootstrap 4 idx=[8,6,1,6,1,2,0,0,3] stump=L7 err=1/9`, and `bootstrap 5 idx=[6,1,5,3,1,8,3,5,8] stump=L4 err=1/9`. The draws are D3 pinned literals, one LCG stream run straight through, so the five lists are one object, not five coincidences. Replicate 1 draws row 9 three times and never sees x = 3, 5, 6, which is why its stump can be perfect, err 0/9. None of the five rediscovers the full-data stump L2, and one faces the other way entirely, R6.

The three test rows settle the bet. `test (2,+): single=+ votes=[+,+,-,+,+] ensemble=+`, `test (5,-): single=- votes=[+,-,-,+,-] ensemble=-`, and `test (7,+): single=- votes=[+,-,+,+,-] ensemble=+`. The single stump misses exactly one test row, x = 7, and the committee fixes exactly that row, three plus votes against two: check "ch23 exactly 1 flip vs single model" names the whole improvement, one flip, zero residual errors on the test set, "ch23 ensemble 0 test errors".

```
/* best stump over a training set by misclassification count (uniform
 * weights, so count/9 orders exactly like the fraction); candidates in
 * the pinned order, strict improvement keeps the first tie winner */
static char best_dir(const int *x, const int *y, int n, int *t_out,
                     int *err_out) {
  int mn = x[0], mx = x[0];
  for (int i = 1; i < n; i++) {
    if (x[i] < mn)
      mn = x[i];
    if (x[i] > mx)
      mx = x[i];
  }
  char bd = 'L';
  int bt = mn - 1, be = n + 1;
  for (int t = mn - 1; t ≤ mx; t++) {
    int e = 0;
    for (int i = 0; i < n; i++)
      if (stump_pred('L', t, x[i]) ≠ y[i])
        e++;
    if (e < be) {
      be = e;
      bd = 'L';
      bt = t;
    }
  }
  for (int t = mn; t ≤ mx + 1; t++) {
    int e = 0;
    for (int i = 0; i < n; i++)
      if (stump_pred('R', t, x[i]) ≠ y[i])
        e++;
    if (e < be) {
      be = e;
      bd = 'R';
      bt = t;
    }
  }
  *t_out = bt;
  *err_out = be;
  return bd;
}
```

listing 178: the stump learner, pinned candidate order, strict improvement keeps the first tie winner (source: kdd/samples/src/Ch23/bagging.c, lines 47-86)

```
  rng_state = 3076u; /* seed per the ch23 bagging row */

  /* single stump on the full 9 rows */
  int st, se;
  char sd = best_dir(RX, RY, 9, &st, &se);
  printf("full-data stump: %c%d err %d/9\n", sd, st, se);
  CHECK(sd == 'L' && st == 2 && se == 2, "ch23 single stump L2 err 2/9");

  /* 5 bootstrap replicates of 9 draws, one LCG stream */
  const int want_idx[5][9] = {{8, 3, 6, 7, 1, 0, 6, 8, 8},
                              {1, 8, 3, 1, 5, 4, 1, 6, 5},
                              {6, 0, 4, 7, 4, 4, 6, 4, 6},
                              {8, 6, 1, 6, 1, 2, 0, 0, 3},
                              {6, 1, 5, 3, 1, 8, 3, 5, 8}};
  const char want_dir[5] = {'L', 'L', 'R', 'L', 'L'};
  const int want_t[5] = {7, 4, 6, 7, 4};
  const int want_e[5] = {0, 1, 2, 1, 1};
  char dir[5];
  int thr[5], err[5];
  for (int m = 0; m < 5; m++) {
    int idx[9], rx[9], ry[9];
    for (int k = 0; k < 9; k++) {
      idx[k] = (int)rng_below(9u);
      rx[k] = RX[idx[k]];
      ry[k] = RY[idx[k]];
    }
    dir[m] = best_dir(rx, ry, 9, &thr[m], &err[m]);
    printf("bootstrap %d idx=[", m + 1);
    for (int k = 0; k < 9; k++)
      printf("%d%s", idx[k], k < 8 ? "," : "");
    printf("] stump=%c%d err=%d/9\n", dir[m], thr[m], err[m]);
```

listing 179: seed 3076, the full-data baseline stump, then the five replicate draws (source: kdd/samples/src/Ch23/bagging.c, lines 89-119)

```
/* test rows: single stump vs 5-voter ensemble (+ iff 2*plus > 5) */
const int TX[3] = {2, 5, 7};
const int TY[3] = {1, -1, 1};
const int want_single[3] = {1, -1, -1};
const int want_votes[3][5] = {
    {1, 1, -1, 1, 1}, {1, -1, -1, 1, -1}, {1, -1, 1, 1, -1}};
const int want_ens[3] = {1, -1, 1};
int single_err = 0, ens_err = 0, flips = 0;
for (int k = 0; k < 3; k++) {
  int sp = stump_pred(sd, st, TX[k]);
  int votes[5], plus = 0;
  for (int m = 0; m < 5; m++) {
    votes[m] = stump_pred(dir[m], thr[m], TX[k]);
    plus += votes[m] == 1;
  }
  int ep = 2 * plus > 5 ? 1 : -1;
  single_err += sp != TY[k];
  ens_err += ep != TY[k];
  flips += sp != ep;
  printf("test (%d,%c): single=%c votes=[", TX[k], TY[k] > 0 ? '+' : '-',
         sp > 0 ? '+' : '-');
  for (int m = 0; m < 5; m++)
    printf("%c%s", votes[m] > 0 ? '+' : '-', m < 4 ? "," : "");
  printf("] ensemble=%c\n", ep > 0 ? '+' : '-');
  char nm[64];
  snprintf(nm, sizeof nm, "ch23 test x=%d vote vector -> %c", TX[k],
           want_ens[k] > 0 ? '+' : '-');
  int ok = sp == want_single[k] && ep == want_ens[k];
  for (int m = 0; m < 5; m++)
    ok = ok && votes[m] == want_votes[k][m];
  CHECK(ok, nm);
}
CHECK(single_err == 1, "ch23 single stump 1 test error");
CHECK(ens_err == 0, "ch23 ensemble 0 test errors");
CHECK(flips == 1, "ch23 exactly 1 flip vs single model");
```

listing 180: test rows, the 5-vote majority, and the one-flip accounting (source: kdd/samples/src/Ch23/bagging.c, lines 132-166)

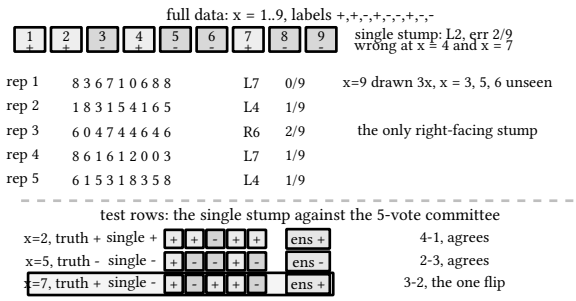


figure 93: five bootstrap replicates from one LCG stream, and the one flip the vote buys

NOTE: a zero-error replicate is nothing special here

Replicate 1's stump separates its 9 rows perfectly, and in a boosting frame that would be a problem: alpha has a division by e waiting for it. Bagging does not care. Every voter casts one vote regardless of its training error, so a perfect stump on a distorted sample and the 2/9 stump of replicate 3 weigh exactly the same. The vote needs diversity between the voters, not excellence of each one, and the resampling is what supplies the diversity.

23.2. three rounds of reweighting

AdaBoost keeps the stump learner and changes the objective: fit the best stump under the current weights, then move weight onto what it missed. The update per Freund-Schapire 1997 sets $\alpha = 1/2 \ln((1-e)/e) = 1/2 \ln r$ for stump error e, multiplies each miss's weight by $e^{+\alpha}$ and each correct point's by $e^{-\alpha}$, then renormalizes. Both factors carry a square root of r, and the sample clears it: multiply the misses by r instead, leave the rest alone, and divide everyone by $Ar + B$ with A the misclassified weight mass and $B = 1 - A$. Same fixed point, and every weight stays a reduced fraction computed from integers.

The dry run, all three rounds verbatim. `round 1: stump=L2 e=1/6 r=5/1 alpha=0.8047189562170501` with weights `1/10 1/10 1/10 1/2 1/10 1/10`: round 1 starts uniform, L2 errs only x4, denominator $(1/6)(5) + 5/6 = 5/3$, so x4 lands on $1/2$ and the rest on $1/10$. `round 2: stump=L4 e=1/10 r=9/1 alpha=1.0986122886681098` with weights `1/18 1/18 1/2 5/18 1/18 1/18`: L4 errs only x3, denominator $(1/10)(9) + 9/10 = 9/5$. `round 3: stump=R4 e=2/9 r=7/2 alpha=0.6263814842476840` with weights `1/8 1/8 9/28 5/28 1/8 1/8`: R4 errs x1, x2, x5, x6 for $e = 4/18 = 2/9$, denominator $(2/9)(7/2) + 7/9 = 14/9$, and each weight vector check closes with its own sum-to-1. The three alphas are the only non-fractional objects in the chapter, D2 at 1e-15 because libm log is in the path.

```
  for (int rnd = 0; rnd < 3; rnd++) {
    int t;
    struct fr e;
    char d = best_dir(w, &t, &e);
    struct fr r = fr(e.d - e.n, e.n);
    rfrac[rnd] = r;
    struct fr a2 = {0, 1}, b2 = {0, 1};
    for (int i = 0; i < 6; i++) {
      if (stump_pred(d, t, AX[i]) != AY[i])
        a2 = fr_add(a2, w[i]);
      else
        b2 = fr_add(b2, w[i]);
    }
    struct fr denom = fr_add(fr_mul(a2, r), b2);
    for (int i = 0; i < 6; i++) {
      int mis = stump_pred(d, t, AX[i]) != AY[i];
      w[i] = fr_div(mis ? fr_mul(w[i], r) : w[i], denom);
    }
```

listing 181: one round: weighted-best stump, r, the mis mass A, the cleared reweight (source: kdd/samples/src/Ch23/adaboost.c, lines 119-136)

```
    double alpha = 0.5 * log((double)r.n / (double)r.d);
    printf("round %d: stump=%c%d e=%lld/%lld r=%lld/%lld alpha=%.16f\n",
           rnd + 1, d, t, e.n, e.d, r.n, r.d, alpha);
    printf("  weights:");
    for (int i = 0; i < 6; i++)
      printf(" %lld/%lld", w[i].n, w[i].d);
    printf("\n");
    char nm[64];
    snprintf(nm, sizeof nm, "ch23 adaboost round %d stump %c%d e %lld/%lld",
             rnd + 1, rdir[rnd], rt[rnd], ren[rnd], red[rnd]);
    CHECK(d == rdir[rnd] && t == rt[rnd] && e.n == ren[rnd] && e.d == red[rnd],
          nm);
    snprintf(nm, sizeof nm, "ch23 adaboost round %d alpha %.16f (D2)", rnd + 1,
             ralpha[rnd]);
    CHECK(fabs(alpha - ralpha[rnd]) < 1e-15, nm);
    int wok = 1;
    for (int i = 0; i < 6; i++)
      wok = wok && w[i].n == wnum[rnd][i] && w[i].d == wden[rnd][i];
    snprintf(nm, sizeof nm, "ch23 adaboost round %d weight vector", rnd + 1);
    CHECK(wok, nm);
    struct fr s = {0, 1};
    for (int i = 0; i < 6; i++)
      s = fr_add(s, w[i]);
    snprintf(nm, sizeof nm, "ch23 adaboost round %d weights sum to 1", rnd + 1);
    CHECK(fr_eq(s, fr(1, 1)), nm);
  }
```

listing 182: alpha from r, the printed weight vectors, and the sum-to-1 checks (source: kdd/samples/src/Ch23/adaboost.c, lines 137-162)

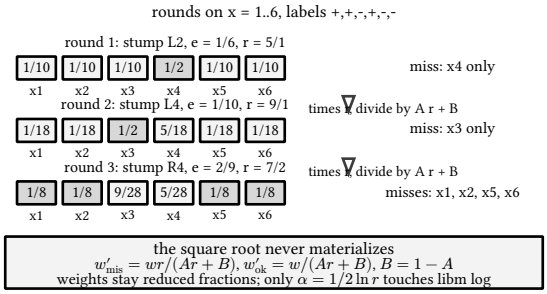


figure 94: weights after each round as exact fractions, misses shaded

23.3. the vote as an integer comparison

The boosted classifier is $H(x) = \operatorname{sign}\left(\sum_m \alpha_m h_m(x)\right)$, a sum of three irrational alphas, and the sample never evaluates it as a float. Substituting $\alpha_m = 1/2 \ln r_m$ folds the sum into one logarithm, $1/2 \ln(A/B)$, where A multiplies r_m for stumps voting plus and B multiplies r_m for stumps voting minus. Every r_m is an exact fraction, 5/1, 9/1, 7/2, so A and B are integer products and the sign of the vote is the sign of $A - B$: the logarithm is monotone, so it never has to be computed.

The dry run, six lines: `x=1: vote=0.5*ln(90/7) -> + (truth +)`, `x=2: vote=0.5*ln(90/7) -> + (truth +)`, `x=3: vote=0.5*ln(18/35) ->`
`- (truth -)`, `x=4: vote=0.5*ln(63/10) -> + (truth +)`, `x=5: vote=0.5*ln(7/90) -> - (truth -)`, `x=6: vote=0.5*ln(7/90) ->`
`- (truth -)`. The hand check at x = 3: L2 votes minus so A takes r_1's denominator 1 and B its numerator 5, L4 votes plus so A takes 9, R4 votes minus so A takes 2 and B takes 7, giving the pinned pair (18, 35), the closest call of the six. x = 4 is the instructive one, pair (63, 10): it is round 1's only miss, the point whose weight started the whole

loop, and the committee now gets it right. All six agree with the labels, check “ch23 adaboost final ensemble 6/6, error 0/6”, against 5/6 for the best single stump, the same L2 that opened round 1 with e = 1/6.

```
/* final vote: h1=L2, h2=L4, h3=R4; sum s_m*alpha_m = 0.5*ln(A/B) with
 * A/B = product of r_m^(s_m), so the sign is an integer comparison */
const char fdir[3] = {'L', 'L', 'R'};
const int ft[3] = {2, 4, 4};
const int want_pair[6][2] = {{90, 7},  {90, 7}, {18, 35},
                             {63, 10}, {7, 90}, {7, 90}};
const int ftruth[6] = {1, 1, -1, 1, -1, -1};
int all_ok = 1;
for (int x = 1; x <= 6; x++) {
  long long A = 1, B = 1;
  for (int m = 0; m < 3; m++) {
    int s = stump_pred(fdir[m], ft[m], x);
    if (s > 0) {
      A *= rfrac[m].n;
      B *= rfrac[m].d;
    } else {
      A *= rfrac[m].d;
      B *= rfrac[m].n;
    }
  }
  int pred = A > B ? 1 : -1;
  all_ok = all_ok && pred == ftruth[x - 1];
  printf("x=%d: vote=0.5*ln(%lld/%lld) -> %c (truth %c)\n", x, A, B,
         pred > 0 ? '+' : '-', ftruth[x - 1] > 0 ? '+' : '-');
```

listing 183: the vote as two integer products of the exact r fractions, sign only (source: kdd/samples/src/Ch23/adaboost.c, lines 164-187)

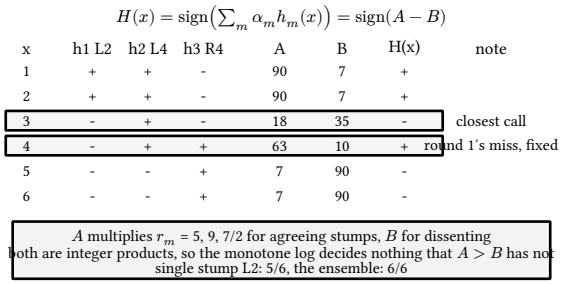


figure 95: the weighted vote per point, A against B as integer products

23.4. random forests and measured importance

The random forest bags the trees and adds a second die: each tree sees only a drawn subset of the features, here one subset per tree from the LCG at seed 4023, redrawn if the draw comes up empty. The learner is the depth-2 gini tree with exact fraction gains and the pinned tie tuple, lower feature index first, then lower threshold. The fixture is 8 rows, (f1, f2, label) = (1,1,A) (1,2,A) (2,1,A) (2,2,A) (3,1,B) (3,2,B) (4,1,B) (4,2,A), that last row being the trap: among f1 = 4 rows the labels disagree, so only f2 splits them.

The dry run: `tree 1: feature mask 2 → {f2}`, `tree 2: feature mask 1 → {f1}`, `tree 3: feature mask 3 → {f1,f2}`, the masks D3 pins. Tree 1 is forced down f2 and prints `tree1: f2≤1 gain 1/32, leaves A,A`. Trees 2 and 3 print the same shape, `tree2: f1≤2 gain 9/32 | left leaf A | right f1≤3 gain 1/8 → B,A`, and tree 3 identical: at the root f1<=2 gains 9/32 against f2<=1's 1/32, and in the right child f1<=3 and f2<=1 both gain exactly 1/8, so the tie breaks to f1 by index even in tree 3, which had both features in hand. The four test rows vote `test (2,1) truth A: preds A,A,A vote A`, `test (3,2) truth B: preds A,B,B vote B`, `test (4,2) truth A: preds A,A,A vote A`, `test`

`(4,1) truth B: preds`
`A,A,A vote A`, the lone miss being the (4,1,B) minority row every tree routes to an A leaf, 3/4 overall.

Importance is where the subset die pays off. MDI credits feature f with $\sum(n_{node}/8) \cdot$ gain over its splits: `tree 1 importance:`
`f1=0/1 f2=1/32`, `tree 2 importance: f1=11/32 f2=0/1`, tree 3 the same, the 11/32 being $(8/8)(9/32) + (4/8)(1/8)$. The mean prints `mean importance: f1=11/48 f2=1/96`, and the control closes the argument: `full-feature tree importance: f1=11/32 f2=0/1`. Given both features at every node, the same tie rule silences f2 completely, so all of f2's 1/96 exists because tree 1 was forced to look at f2 alone.

```
    for (int t = 0; t < nv; t++) {
      int left[8], right[8], nl = 0, nr = 0;
      for (int i = 0; i < n; i++) {
        if (RF[rows[i]][f] ≤ vals[t])
          left[nl++] = rows[i];
        else
          right[nr++] = rows[i];
      }
      if (!nl || !nr)
        continue;
      struct fr w = fr_add(fr_mul(fr(nl, n), gini(left, nl)),
                           fr_mul(fr(nr, n), gini(right, nr)));
      struct fr gain = fr_sub(root, w);
      int better = !found || fr_gt(gain, bg) ||
                   (fr_eq(gain, bg) && (f < bf || (f == bf && vals[t] < bv)));
      if (better) {
        found = 1;
        bf = f;
        bv = vals[t];
        bg = gain;
      }
    }
```

listing 184: the split loop: weighted child gini, exact gain, ties to lower feature then threshold (source: kdd/samples/src/Ch23/rf.c, lines 131-152)

```
int main(void) {
  rng_state = 4023u; /* seed per the ch23 random forest row */
  int all[8] = {0, 1, 2, 3, 4, 5, 6, 7};
  const int want_mask[3] = {2, 1, 3}; /* subsets {f2}, {f1}, {f1,f2} */
  int roots[3];
  for (int m = 0; m < 3; m++) {
    uint32_t bits;
    do {
      bits = rng_below(4u);
    } while (bits == 0);
    int feats = (int)bits;
    roots[m] = grow(all, 8, feats, 0);
    printf("tree %d: feature mask %d -> {%s%s%s}\n", m + 1, feats,
           (feats & 1) ? "f1" : "", (feats & 3) == 3 ? "," : "",
           (feats & 2) ? "f2" : "");
    char nm[48];
    snprintf(nm, sizeof nm, "ch23 rf tree %d feature subset (D3)", m + 1);
    CHECK(feats == want_mask[m], nm);
```

listing 185: seed 4023, one feature-subset draw per tree, tree growth (source: kdd/samples/src/Ch23/rf.c, lines 204-221)

```
/* MDI importances per tree, then the exact 3-tree mean */
const struct fr want_imp[3][2] = {
    {{0, 1}, {1, 32}}, {{11, 32}, {0, 1}}, {{11, 32}, {0, 1}}};
struct fr mean[2] = {FR0, FR0};
for (int m = 0; m < 3; m++) {
  struct fr imp[2] = {FR0, FR0};
  importance(roots[m], imp);
  printf("tree %d importance: f1=%lld/%lld f2=%lld/%lld\n", m + 1, imp[0].n,
         imp[0].d, imp[1].n, imp[1].d);
  char nm[48];
  snprintf(nm, sizeof nm, "ch23 rf tree %d MDI importance", m + 1);
  CHECK(fr_eq(imp[0], want_imp[m][0]) && fr_eq(imp[1], want_imp[m][1]), nm);
  mean[0] = fr_add(mean[0], imp[0]);
  mean[1] = fr_add(mean[1], imp[1]);
}
mean[0] = fr_div(mean[0], fr(3, 1));
mean[1] = fr_div(mean[1], fr(3, 1));
printf("mean importance: f1=%lld/%lld f2=%lld/%lld\n", mean[0].n, mean[0].d,
       mean[1].n, mean[1].d);
CHECK(fr_eq(mean[0], fr(11, 48)) && fr_eq(mean[1], fr(1, 96)),
      "ch23 rf mean importance f1=11/48 f2=1/96");
```

listing 186: per-tree MDI as exact fractions, then the 3-tree mean (source: kdd/samples/src/Ch23/rf.c, lines 285-305)

```
/* full-feature tree on all 8 rows: importance f1=11/32, f2=0 */
npool = 0;
int full = grow(all, 8, 3, 0);
struct fr fimp[2] = {FR0, FR0};
importance(full, fimp);
printf("full-feature tree importance: f1=%lld/%lld f2=%lld/%lld\n", fimp[0].n,
       fimp[0].d, fimp[1].n, fimp[1].d);
CHECK(fr_eq(fimp[0], fr(11, 32)) && fr_eq(fimp[1], FR0),
      "ch23 rf full tree importance f1=11/32 f2=0");
```

listing 187: the full-feature control: f2 importance exactly 0 (source: kdd/samples/src/Ch23/rf.c, lines 307-315)

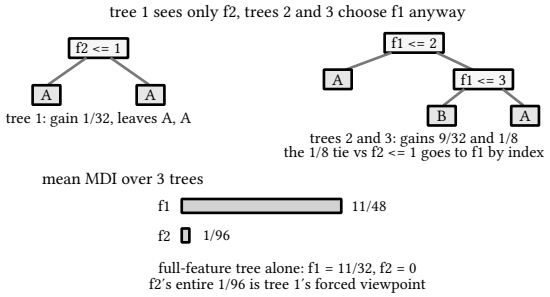


figure 96: three trees on drawn feature subsets, and what importance measures

PITFALL: importance is a property of this forest

MDI sums the gains a feature actually received, under the drawn subsets and the pinned tie rule. The right child's 1/8 tie went to f1 twice, and once more in the full-feature control; a different tie rule or a different seed moves f2's mean off 1/96 without any change to the data. The number answers "how much did f2 participate in these three trees", which is a fact about the ensemble, and only a hint about the generating process. The honest reading is the comparison the sample pins: 1/96 with forced diversity against exactly 0 without it.

sources: bagging per Leo Breiman, Bagging Predictors, Machine Learning 24(2):123-140, 1996. The AdaBoost update with $\alpha = 1/2 \ln((1-e)/e)$ per Yoav Freund and Robert E. Schapire, A decision-theoretic generalization

of on-line learning and an application to boosting, Journal of Computer and System Sciences 55(1):119-139, 1997. Random forests with feature subsetting and mean decrease in impurity per Leo Breiman, Random Forests, Machine Learning 45(1):5-32, 2001. All three banked by kdd-contract-s4s5.md. All 54 pinned values witnessed by the same sheet and playground/kdd-matrix/gen_s5.py, run 2026-09-22, exit 0, the bootstrap, subset, and feature draws identical across two consecutive runs. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch23`, 17 + 19 + 18 checks in chapter 23 of the kdd suite.

24. class imbalance

Two samples carry the chapter: `paradox.c` builds a 100-row fixture where predicting the majority class scores 99/100 and then prices the threshold move that fixes it, 19 checks, and `resample.c` scores a 20-row fixture before and after both resamplings, 22 checks. The chapter makes 4 moves: the metric that celebrates failure, the cost ledger that argues back, duplication of the minority, and subtraction from the majority. Every behavioral claim below is one of the 41 checks of chapter 24's samples or a named convention of the contract sheet. The metric definitions are the ones book 14, ch 17 (knowledge discovery, classifier evaluation) pinned as exact fractions, the threshold sweep is the same predict-plus rule book 14, ch 18 (knowledge discovery, roc and auc) swept, and the votes of book 14, ch 23 (knowledge discovery, bagging, boosting, and random forests) assumed errors matter equally, which is exactly what this chapter's class mix denies.

24.1. the paradox of 99 percent

Fixture A is 100 rows built by a loop: 1 positive with score 0.46, and 99 negatives, 4 at 0.7, 6 at 0.44, 89 at 0.1. The classifier under the microscope ignores scores entirely and predicts negative for every row, the majority class by construction. On a balanced set that would be a joke; here it is the best-scoring constant predictor, and the metrics split on whether it was any good.

The dry run: the confusion line prints `all-negative: TP=0 FP=0 FN=1 TN=99`, so accuracy is $99/100$, the number the paradox is named for. The rest of the ledger collapses: recall is $0/1$, zero of one detections, and precision hits the degenerate case, $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$ with denominator zero. The sheet's convention, stated in-sample, is precision 0 when the denominator is 0, and check "ch24 all-negative precision 0 (0/0 convention)" pins it rather than letting the fraction be undefined. F1 then reads $2\ \mathrm{TP}/(2\ \mathrm{TP}+\mathrm{FP}+\mathrm{FN}) = 0/1 = 0$, and the closing check names the whole situation outright, "ch24 paradox: 99/100 accuracy with zero detections". The one number that looked like excellence was the majority class echoing itself.

```
static void build_fixture(void) {
  SC[0] = 0.46;
  LB[0] = 1;
  int r = 1;
  for (int k = 0; k < 4; k++) {
    SC[r] = 0.7;
    LB[r] = 0;
    r++;
  }
  for (int k = 0; k < 6; k++) {
    SC[r] = 0.44;
    LB[r] = 0;
    r++;
  }
  while (r < N) {
    SC[r] = 0.1;
    LB[r] = 0;
    r++;
  }
}
```

listing 188: fixture A by loop: 1 positive at 0.46, negatives 4 x 0.7, 6 x 0.44, 89 x 0.1 (source: kdd/samples/src/Ch24/paradox.c, lines 35-54)

```
/* all-negative classifier: predict 0 for every row (the majority
 * class), so TP=FP=0; precision hits the 0/0 convention */
int tp = 0, fp = 0, fn = 0, tn = 0;
for (int i = 0; i < N; i++) {
  if (LB[i])
    fn++;
  else
    tn++;
}
printf("all-negative: TP=%d FP=%d FN=%d TN=%d\n", tp, fp, fn, tn);
CHECK(tp == 0 && fp == 0 && fn == 1 && tn == 99,
      "ch24 all-negative confusion TP=0 FP=0 FN=1 TN=99");
CHECK(frac_eq(tn, N, 99, 100), "ch24 all-negative accuracy 99/100");
CHECK(frac_eq(tp, tp + fn, 0, 1), "ch24 all-negative recall 0/1");
CHECK(tp + fp == 0, "ch24 all-negative precision 0 (0/0 convention)");
CHECK(frac_eq(2 * tp, 2 * tp + fp + fn, 0, 1),
      "ch24 all-negative F1 = 0/1 = 0");
CHECK(frac_eq(tn, N, 99, 100) && tp == 0 && fn == 1,
      "ch24 paradox: 99/100 accuracy with zero detections");
```

listing 189: the all-negative classifier and the metric ledger it earns (source: kdd/samples/src/Ch24/paradox.c, lines 80-98)

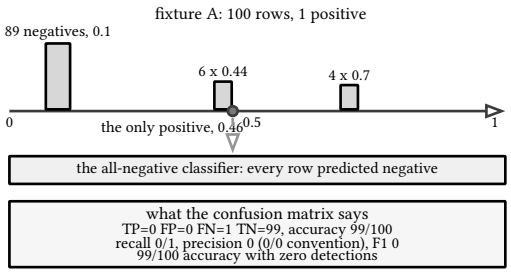


figure 97: fixture A's score landscape, and the classifier that ignores it

NOTE: accuracy is a headcount

Accuracy counts rows, and 99 of the 100 rows are negative, so the constant no-sayer was guaranteed 99/100 before seeing a single score. Any metric that sums over rows inherits the class mix, which is why the imbalance toolkit starts by reporting precision, recall, and F1 separately, the way book 14, ch 17 (knowledge discovery, classifier evaluation) defined them. The 0/0 convention does quiet work here too: without it, precision would be undefined and the degenerate classifier would escape the ledger through a syntax error rather than a zero.

24.2. pricing a miss at 20 false alarms

The scores do exist, so the fix is a threshold with opinions. The rule is the roc chapter's: predict positive when score $\geq$ t. At $t = 0.5$ only the four 0.7 negatives cross, so the lone positive at 0.46 stays unseen; at $t = 0.4$ the positive crosses along with all six 0.44 negatives. Both sweeps and their ledgers are pinned, and the argument for moving is a cost model, false alarm at 1, missed positive at 20.

The dry run: `t=0.5: TP=0 FP=4 FN=1 TN=95` with accuracy 19/20, precision 0, recall 0, F1 0, then `t=0.4: TP=1 FP=10 FN=0 TN=89` with accuracy 9/10, precision 1/11, recall 1, F1 1/6. Accuracy falls from 19/20 to 9/10 and every other metric rises from zero, which is the trade the paradox was hiding. The cost line prints `cost FP=1 FN=20:` `cost(0.5)=20 (FN) +4 (FP) = 24; cost(0.4)=10`, and the sheet's documented ruling lives in that line: the generator's row pinned the FN term alone, cost(0.5) = 20, the full sum with the omitted 1×4 false-alarm term is 24, and 10 beats both readings, so check "ch24 cost 10 beats both 20 and 24: move to 0.4 justified" closes the move without picking a reading.

```
static void cm_at(double t, int *tp, int *fp, int *fn, int *tn) {
  *tp = *fp = *fn = *tn = 0;
  for (int i = 0; i < N; i++) {
    if (SC[i] ≥ t) {
      if (LB[i])
        (*tp)++;
      else
        (*fp)++;
    } else {
      if (LB[i])
        (*fn)++;
      else
        (*tn)++;
    }
  }
}
```

listing 190: the threshold rule, one pass over the scores (source: kdd/samples/src/Ch24/paradox.c, lines 56-71)

```
  /* threshold t=0.5 */
  int tp5, fp5, fn5, tn5;
  cm_at(0.5, &tp5, &fp5, &fn5, &tn5);
  printf("t=0.5: TP=%d FP=%d FN=%d TN=%d\n", tp5, fp5, fn5, tn5);
  CHECK(tp5 == 0 && fp5 == 4 && fn5 == 1 && tn5 == 95,
        "ch24 t=0.5 confusion TP=0 FP=4 FN=1 TN=95");
  CHECK(frac_eq(tp5 + tn5, N, 19, 20), "ch24 t=0.5 accuracy 19/20");
  CHECK(frac_eq(tp5, tp5 + fp5, 0, 1), "ch24 t=0.5 precision 0/4 = 0");
  CHECK(frac_eq(tp5, tp5 + fn5, 0, 1), "ch24 t=0.5 recall 0/1");
  CHECK(frac_eq(2 * tp5, 2 * tp5 + fp5 + fn5, 0, 1), "ch24 t=0.5 F1 0");

  /* cost-sensitive move to t=0.4 */
  int tp4, fp4, fn4, tn4;
  cm_at(0.4, &tp4, &fp4, &fn4, &tn4);
  printf("t=0.4: TP=%d FP=%d FN=%d TN=%d\n", tp4, fp4, fn4, tn4);
  CHECK(tp4 == 1 && fp4 == 10 && fn4 == 0 && tn4 == 89,
        "ch24 t=0.4 confusion TP=1 FP=10 FN=0 TN=89");
  CHECK(frac_eq(tp4 + tn4, N, 9, 10), "ch24 t=0.4 accuracy 9/10");
  CHECK(frac_eq(tp4, tp4 + fp4, 1, 11), "ch24 t=0.4 precision 1/11");
  CHECK(frac_eq(tp4, tp4 + fn4, 1, 1), "ch24 t=0.4 recall 1/1");
  CHECK(frac_eq(2 * tp4, 2 * tp4 + fp4 + fn4, 1, 6), "ch24 t=0.4 F1 1/6");
```

listing 191: both sweeps pinned, the two confusions and their ledgers (source: kdd/samples/src/Ch24/paradox.c, lines 100-120)

```
/* cost weights FP=1 FN=20: the generator pins cost(0.5)=20*1=20 (the
 * FN term) and cost(0.4)=1*10=10 (the FP term); adding the omitted
 * 1*4 FP term at t=0.5 gives 24, and the move wins either way */
long long fn_cost5 = 20LL * fn5, fp_cost5 = 1LL * fp5;
long long fp_cost4 = 1LL * fp4, fn_cost4 = 20LL * fn4;
printf("cost FP=1 FN=20: cost(0.5)=%lld (FN) +%lld (FP) = %lld; "
       "cost(0.4)=%lld\n",
       fn_cost5, fp_cost5, fn_cost5 + fp_cost5, fp_cost4 + fn_cost4);
CHECK(fn_cost5 == 20 && fp_cost5 == 4,
      "ch24 cost t=0.5: 20*1=20 FN term (+1*4 FP term)");
CHECK(fp_cost4 == 10 && fn_cost4 == 0, "ch24 cost t=0.4: 1*10=10, FN term 0");
CHECK(fp_cost4 + fn_cost4 < fn_cost5 &&
          fp_cost4 + fn_cost4 < fn_cost5 + fp_cost5,
      "ch24 cost 10 beats both 20 and 24: move to 0.4 justified");
```

listing 192: the cost ledger: FN term, FP term, and the verdict under both readings (source: kdd/samples/src/Ch24/paradox.c, lines 122-135)

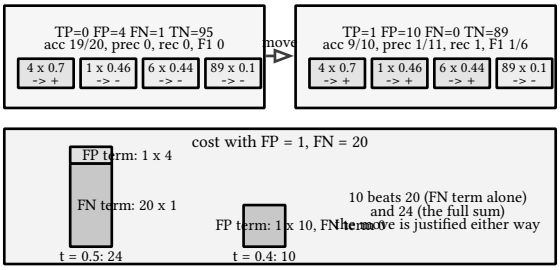


figure 98: the same scores at two thresholds, and the cost ledger that picks

24.3. the balanced set by duplication

Fixture B is 20 rows for the resampling work: positives rows 0 to 4 with scores 0.9, 0.8, 0.7, 0.4, 0.3, negatives row 5 at 0.6 and rows 6 to 19 at 0.2, threshold fixed at 0.5. The baseline first: three positives clear the bar, two do not, and the 0.6 negative is the single false alarm, `20-row t=0.5: TP=3 FN=2 FP=1 TN=14`, precision 3/4, recall 3/5, F1 2/3, accuracy 17/20.

Oversampling duplicates the minority: seed 5024 draws 10 rows with replacement from the five positives, and the print is `oversample draws: 1 2 0 2 3 4 2 1 1 4`, a D3 pin covering all five rows, row 2 and row 1 three times each. The balanced set is 15 negatives against 15 positives, the five originals plus ten copies, and the same threshold rule on it prints `balanced 15 neg + 15 pos: TP=10 FN=5 FP=1 TN=14`: the 7 drawn copies at 0.7 or above join the 3 original hits, the 3 drawn copies of 0.4 and 0.3 join the misses, and the negative side is untouched, FP=1 TN=14. Precision rises to 10/11, recall to 2/3, F1 to 10/13, every one an exact fraction check.

```
int main(void) {
  /* baseline at t=0.5 on all 20 rows */
  int tp = 0, fn = 0;
  for (int i = 0; i < 5; i++)
    tp += PSC[i] >= 0.5;
  fn = 5 - tp;
  int fp = 0;
  for (int i = 0; i < 15; i++)
    fp += NSC[i] >= 0.5;
  int tn = 15 - fp;
  printf("20-row t=0.5: TP=%d FN=%d FP=%d TN=%d\n", tp, fn, fp, tn);
  CHECK(tp == 3 && fn == 2 && fp == 1 && tn == 14,
        "ch24 baseline confusion TP=3 FN=2 FP=1 TN=14");
  CHECK(frac_eq(tp, tp + fp, 3, 4), "ch24 baseline precision 3/4");
  CHECK(frac_eq(tp, tp + fn, 3, 5), "ch24 baseline recall 3/5");
  CHECK(frac_eq(2 * tp, 2 * tp + fp + fn, 2, 3), "ch24 baseline F1 2/3");
  CHECK(frac_eq(tp + tn, 20, 17, 20), "ch24 baseline accuracy 17/20");
  CHECK(tp + fn + fp + tn == 20, "ch24 baseline confusion sums to 20");
```

listing 193: the 20-row baseline at t = 0.5, counts and fractions (source: kdd/samples/src/Ch24/resample.c, lines 51-68)

```
  /* oversample: seed 5024, 10 replacement draws from rows 0-4 */
  rng_state = 5024u;
  int draws[10];
  for (int k = 0; k < 10; k++)
    draws[k] = (int)rng_below(5u);
  printf("oversample draws:");
  for (int k = 0; k < 10; k++)
    printf(" %d", draws[k]);
  printf("\n");
  const int want_draws[10] = {1, 2, 0, 2, 3, 4, 2, 1, 1, 4};
  int dok = 1;
  for (int k = 0; k < 10; k++)
    dok = dok && draws[k] == want_draws[k];
  CHECK(dok, "ch24 oversample draws 1,2,0,2,3,4,2,1,1,4 (D3)");
  int tp2 = tp, fn2 = 5 - tp;
  for (int k = 0; k < 10; k++) {
    if (PSC[draws[k]] >= 0.5)
      tp2++;
    else
      fn2++;
  }
  printf("balanced 15 neg + 15 pos: TP=%d FN=%d FP=1 TN=14\n", tp2, fn2);
  CHECK(tp2 + fn2 == 15, "ch24 oversample balanced 15 pos (15 neg fixed)");
  CHECK(tp2 == 10, "ch24 oversample TP=10");
  CHECK(fn2 == 5, "ch24 oversample FN=5");
  CHECK(frac_eq(tp2, tp2 + 1, 10, 11), "ch24 oversample precision 10/11");
  CHECK(frac_eq(tp2, 15, 2, 3), "ch24 oversample recall 2/3");
  CHECK(frac_eq(2 * tp2, 2 * tp2 + 1 + fn2, 10, 13),
        "ch24 oversample F1 10/13");
  CHECK(1 == fp && 14 == tn, "ch24 oversample FP=1 TN=14 unchanged");
```

listing 194: seed 5024, ten replacement draws, the balanced-set ledger (source: kdd/samples/src/Ch24/resample.c, lines 70-99)

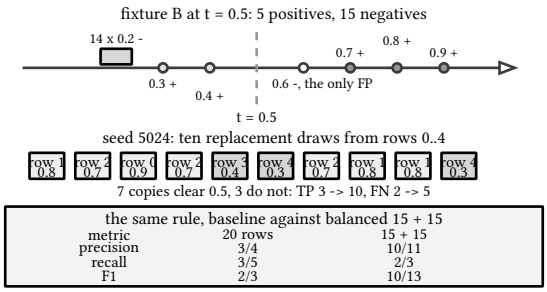


figure 99: fixture B on the score line, then ten drawn copies of the minority

24.4. the balanced set by subtraction

Undersampling shrinks the majority instead: seed 6024 draws by rejection until 5 distinct rows of the negative 15 are seen, keeping first-seen indices exactly as the generators do. The print is `undersample kept: 6 7 11 15 16`, a D3 pin, and the sort into ascending order is part of the pin. Row 5, the 0.6 false positive, is never drawn, which the sample states outright, check “ch24 row 5 (the only 0.6 false positive) excluded”. The balanced set is 5 positives against those 5 negatives, all at 0.2, and the ledger prints `balanced 5 neg + 5 pos:` `TP=3 FN=2 FP=0 TN=5`: precision 1, recall 3/5, F1 3/4, accuracy 4/5, recall untouched because the positive side never moved.

Set against the oversample, the two resamplings trade differently: duplication bought recall 2/3 at precision 10/11 with the false alarm still present, subtraction bought precision 1 with recall back at 3/5 and the false alarm gone, F1 10/13 against 3/4, both above the baseline 2/3. Which trade to want is the cost question of the second facet again, now answered by dataset surgery instead of a threshold.

```
/* undersample: seed 6024, rejection until 5 distinct of rows 5-19 */
rng_state = 6024u;
int kept[15], nkept = 0, seen[15] = {0};
while (nkept < 5) {
  int i = 5 + (int)rng_below(15u);
  if (!seen[i - 5]) {
    seen[i - 5] = 1;
    kept[nkept++] = i;
  }
}
for (int a = 1; a < nkept; a++) { /* ascending, matching the pin */
  int v = kept[a], at = a;
  while (at > 0 && kept[at - 1] > v) {
    kept[at] = kept[at - 1];
    at--;
  }
  kept[at] = v;
}
printf("undersample kept:");
for (int k = 0; k < nkept; k++)
  printf(" %d", kept[k]);
printf("\n");
const int want_kept[5] = {6, 7, 11, 15, 16};
int kok = nkept == 5;
for (int k = 0; k < 5; k++)
  kok = kok && kept[k] == want_kept[k];
CHECK(kok, "ch24 undersample kept 6,7,11,15,16 (D3)");
```

listing 195: seed 6024, rejection until 5 distinct majority rows, ascending pin (source: kdd/samples/src/Ch24/resample.c, lines 101-127)

```
  int fpu = 0;
  for (int k = 0; k < 5; k++)
    fpu += NSC[kept[k] - 5] ≥ 0.5;
  CHECK(!seen[0], "ch24 row 5 (the only 0.6 false positive) excluded");
  CHECK(fpu == 0 && 5 - fpu == 5, "ch24 undersample FP=0 TN=5");
  printf("balanced 5 neg + 5 pos: TP=3 FN=2 FP=%d TN=%d\n", fpu, 5 - fpu);
  CHECK(tp == 3 && fn == 2, "ch24 undersample TP=3 FN=2");
  CHECK(frac_eq(tp, tp + fpu, 1, 1), "ch24 undersample precision 1");
  CHECK(frac_eq(tp, tp + fn, 3, 5), "ch24 undersample recall 3/5");
  CHECK(frac_eq(2 * tp, 2 * tp + fpu + fn, 3, 4), "ch24 undersample F1 3/4");
  CHECK(frac_eq(tp + 5 - fpu, 10, 4, 5), "ch24 undersample accuracy 4/5");
```

listing 196: the 5 + 5 ledger, precision 1 with recall untouched (source: kdd/samples/src/Ch24/resample.c, lines 128-138)

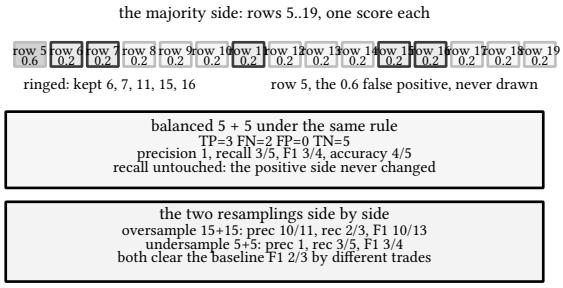


figure 100: the 15 majority rows, 5 kept by rejection, and the two resamplings compared

PITFALL: the resampled set is the thing measured

Both resamplings here keep the threshold rule fixed and re-score the balanced set, so the pins measure what duplication and subtraction do to the ledger, not what a retrained model would do on fresh data. And the undersample's clean precision 1 carries a seed's fingerprint: row 5 happened to be never drawn, and a different draw that keeps the 0.6 row leaves FP = 1 and drops precision off its pedestal. The D3 pins make that visible instead of magical, the same way the book 14, ch 22 (knowledge discovery, support vector machines) trace named its selection heuristics: the number is this sample's number.

sources: the class imbalance problem and the systematic study of resampling's effect on classifiers per Nathalie Japkowicz and Shaju Stephen, The Class Imbalance Problem: A Systematic Study, Intelligent Data Analysis 6(5):429-449, 2002, banked by kdd-contract-s4s5.md. The 0/0 precision convention, the predict-plus-when-score-at-or-above-t rule, and the cost-row ruling with both readings are pinned by the same sheet. All 41 pinned values witnessed by the sheet and playground/kdd-matrix/gen_s5.py, run 2026-09-22, exit 0, both draw streams identical across two consecutive runs. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch24`, 19 + 22 checks in chapter 24 of the kdd suite.

25. levelwise apriori

One sample carries the chapter: `apriori.c` runs the levelwise loop twice over the same nine baskets, minsup 2 then minsup 3, 23 checks. The chapter makes 4 moves: the support count and the antimonotone law that makes searching by levels cheaper than testing all 32 subsets, the join-and-prune candidate generator whose killing subsets act before any scan, the two different ways the loop can stop, and the harvest of 13 frequent itemsets that chapters 26 and 28 recompute from the same table by entirely different means. The baskets are the fp-growth literature's own running example, the itemset lattice they live in is the same exponential-in-dimensions shape book 14, ch 11 (knowledge discovery, data cubes and olap) counts cube cells in, and the rules these itemsets grow into are book 14, ch 27 (knowledge discovery, maximal itemsets and rule generation) material.

25.1. support and the law named a priori

An itemset is a subset of the item universe, here five items I1..I5, and its support is the number of transactions whose basket contains it, the containment test one bitwise AND per row. The fixture is nine baskets, and the sample prints them as one byte-pinned row, `100:{I1,I2,I5}` on through `900:{I1,I2,I3}`. Minsup is an absolute count, 2 of 9: a fraction threshold on nine rows would have to round somewhere, and the sheet keeps the threshold an integer so every comparison is exact.

The law: every subset of a frequent itemset is frequent, because each transaction witnessing the superset witnesses all its subsets at once. The algorithm uses the contrapositive, an itemset with one infrequent subset cannot itself be frequent, and uses it before touching the data, which is what the name means. The dry run: one counting pass over the five singletons prints `C1 gen=5 prune=0 L=5` and `L1: {I1}=6;{I2}=7; {I3}=6;{I4}=2;{I5}=2`. I2 is the busiest item at 7 of 9 baskets, and even the two rarest items clear the bar at exactly 2, so level 1 keeps every candidate and the prune has nothing to do yet.

```
#define NIT 5
#define NTX 9
static const int tid[NTX] = {100, 200, 300, 400, 500, 600, 700, 800, 900};
static const unsigned m2[NTX] = {
    (1u << 0) | (1u << 1) | (1u << 4),            /* 100 {I1,I2,I5} */
    (1u << 1) | (1u << 3),                        /* 200 {I2,I4} */
    (1u << 1) | (1u << 2),                        /* 300 {I2,I3} */
    (1u << 0) | (1u << 1) | (1u << 3),            /* 400 {I1,I2,I4} */
    (1u << 0) | (1u << 2),                        /* 500 {I1,I3} */
    (1u << 1) | (1u << 2),                        /* 600 {I2,I3} */
    (1u << 0) | (1u << 2),                        /* 700 {I1,I3} */
    (1u << 0) | (1u << 1) | (1u << 2) | (1u << 4), /* 800 {I1,I2,I3,I5} */
    (1u << 0) | (1u << 1) | (1u << 2)             /* 900 {I1,I2,I3} */
};
```

listing 197: the M2 baskets as one bitmask per transaction, Han-Pei-Yin table (source: kdd/samples/src/Ch25/apriori.c, lines 36-49)

```
static int supp(unsigned m) {
  int c = 0;
  for (int t = 0; t < NTX; t++) {
    if ((m2[t] & m) == m) {
      c++;
    }
  }
  return c;
}
```

listing 198: support as a subset scan, one AND and compare per basket (source: kdd/samples/src/Ch25/apriori.c, lines 68-76)

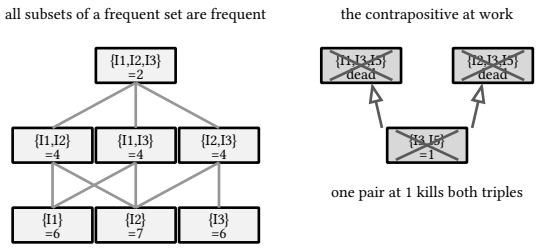


figure 101: support falls or holds moving up, one infrequent pair executes its whole upper cone

NOTE: a priori means the prune happens before the scan

Levelwise search never asks the database about a candidate it can already reject. The subset law is knowledge about the counting process, not about the data, so it applies with zero database access: {I3,I5} was counted once, at support 1, and that single fact retires every itemset containing I3 and I5 before any of them is ever built. The chapters that follow are all attempts to spend even less: the vertical lane of book 14, ch 26 (knowledge discovery, vector apriori) stops rescanning, and book 14, ch 28 (knowledge discovery, fp-growth) stops generating candidates at all.

25.2. join, prune, and the killing subsets

Level k candidates are built only from level k-1 survivors. The join pairs two frequent (k-1)-itemsets that share their first k-2 items in lexicographic order and unions them. The prune then asks each fresh candidate whether every one of its (k-1)-subsets survived the last level, discards it on the first missing subset, and records that subset as the killer.

The dry run: level 2 joins all ten singleton pairs and the prune has nothing to kill since every singleton is frequent, printing `C2 gen=10`
`prune=0 L=6`, then counting finds {I1,I4} at 1, {I3,I4} at 0, {I3,I5} at 1 and {I4,I5} at 0 below the bar. Level 3 starts from the six surviving pairs. Their joins sharing a first item are {I1,I2,I3}, {I1,I2,I5}, {I1,I3,I5} from the I1 group and {I2,I3,I4}, {I2,I3,I5}, {I2,I4,I5} from the I2 group, six candidates, and the prune kills four of them: {I1,I3,I5} and {I2,I3,I5} both die on {I3,I5}, which counted 1, {I2,I3,I4} dies on {I3,I4} at 0, and {I2,I4,I5} on {I4,I5} at 0. The line reads `C3 gen=6 prune=4 L=2`, and the check names all four killers at once, “ch25 C3 prune provenance {I1,I3,I5}/{I2,I3,I5} by {I3,I5}, {I2,I3,I4} by {I3,I4}, {I2,I4,I5} by {I4,I5}”. The third counting pass scores both survivors at exactly the bar, `L3: {I1,I2,I3}=2;{I1,I2,I5}=2`.

Level 4 is where the prune wins outright. The two L3 survivors join into {I1,I2,I3,I5}, whose subset {I1,I3,I5} counted 1 and never entered L3, so the candidate dies uncounted: `C4 gen=1 prune=1 L=noscan`.

```
    unsigned cand[64];
    int nc = 0;
    for (int i = 0; i < p->n; i++) {
      for (int j = i + 1; j < p->n; j++) {
        /* classic apriori join: p, q share their first k-2 items */
        if ((p->m[i] ^ topbit(p->m[i])) == (p->m[j] ^ topbit(p->m[j]))) {
          cand[nc++] = p->m[i] | p->m[j];
          r->gen[k]++;
        }
      }
    }
```

listing 199: the classic join, two survivors sharing their first k-2 items (source: kdd/samples/src/Ch25/apriori.c, lines 139-149)

```
    unsigned keep[64];
    int nk = 0;
    for (int i = 0; i < nc; i++) {
      unsigned killer = 0;
      int ok = 1;
      /* subsets from the highest item down: the first missing subset is
       * the recorded killer ({I1,I3,I5} for the C4 candidate) */
      for (unsigned rem = cand[i]; rem; rem ^= topbit(rem)) {
        const unsigned sub = cand[i] ^ topbit(rem);
        int found = 0;
        for (int j = 0; j < p->n && !found; j++) {
          if (p->m[j] == sub) {
            found = 1;
          }
        }
        if (!found) {
          ok = 0;
          killer = sub;
          break;
        }
      }
      if (ok) {
        keep[nk++] = cand[i];
      } else {
        r->kill_m[r->nkill] = cand[i];
        r->kill_by[r->nkill] = killer;
        r->nkill++;
        r->pr[k]++;
      }
    }
```

listing 200: the subset prune, first missing subset recorded as the killer (source: kdd/samples/src/Ch25/apriori.c, lines 150-179)

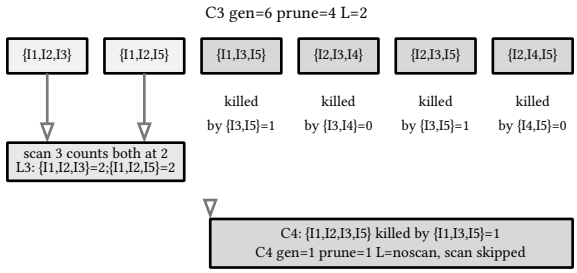


figure 102: six joins, four killed by their pairs, two counted at exactly the bar

PITFALL: the join silently misses candidates on unsorted input

The shared-first-(k-2) test only pairs {I2,I3} with {I2,I4} if both are read in one lexicographic order, which is why the sample sorts every level by itemset string before joining. Joining instead on "differ in one item" would also work but generates duplicates, and joining on nothing at all generates every pair and loses the guarantee that both (k-1)-parents of a generated candidate are themselves in L(k-1), the property the prune relies on. Order is not cosmetic here, it is half the generator.

25.3. two ways to stop

The minsup 2 run stops from the top: a candidate set the prune emptied before scanning. The `L=noscan` in the C4 line is the sheet's spelling of that, the counting pass over C4 never happens, and `scans=3` counts the passes over C1, C2, C3 only. The same nine baskets at minsup 3 stop from the bottom instead, and every number changes shape.

The dry run: level 1 keeps only the three items at 6 or above, `C1 gen=5 prune=0 L=3` and `L1: {I1}=6;{I2}=7;{I3}=6`, I4 and I5 at 2 dropping. With three survivors the join makes three candidates, all count frequent, `C2 gen=3 prune=0 L=3` and `L2: {I1,I2}=4;{I1,I3}=4; {I2,I3}=4`. Level 3 joins {I1,I2} with {I1,I3} into one candidate, the prune passes it since all three of its pairs are frequent, and the scan counts it at 2 against the bar of 3: `C3 gen=1 prune=0 L=0`. An empty level by support, so C4 is never generated at all, and the loop ends with the same `scans=3` but a different third pass, one candidate counted and rejected rather than two counted and kept. The harvest shrinks to `fi_count=6 fi_checksum=31`.

```
    r->nlev = k;
    if (nk == 0) {
      r->lk[k] = -1;
      r->noscan[k] = 1; /* subset prune emptied Ck: no scan */
      break;
    }
    r->scans++;
    lvl_t *lk = &r->L[k];
    for (int i = 0; i < nk; i++) {
      const int c = supp(keep[i]);
      if (c >= ms) {
        lk->m[lk->n] = keep[i];
        lk->c[lk->n] = c;
        lk->n++;
      }
    }
    r->lk[k] = lk->n;
    sort_lvl(lk);
  }
```

listing 201: the noscan break against the counting path, scans only step on a real pass (source: kdd/samples/src/Ch25/apriori.c, lines 180-198)

```
/* ── minsup 3: same baskets, early stop ── */
run_t r3;
apriori(3, &r3);
cline(&r3, 1, buf);
printf("%s\n", buf);
CHECK(strcmp(buf, "C1 gen=5 prune=0 L=3") == 0, "ch25 minsup3 C1 line");
cline(&r3, 2, buf);
printf("%s\n", buf);
CHECK(strcmp(buf, "C2 gen=3 prune=0 L=3") == 0, "ch25 minsup3 C2 line");
cline(&r3, 3, buf);
printf("%s\n", buf);
CHECK(strcmp(buf, "C3 gen=1 prune=0 L=0") == 0,
      "ch25 minsup3 C3 empties by support, not prune");
lvl_line(&r3, 1, buf);
printf("%s\n", buf);
CHECK(strcmp(buf, "L1: {I1}=6;{I2}=7;{I3}=6") == 0,
      "ch25 minsup3 L1 listing");
lvl_line(&r3, 2, buf);
printf("%s\n", buf);
CHECK(strcmp(buf, "L2: {I1,I2}=4;{I1,I3}=4;{I2,I3}=4") == 0,
      "ch25 minsup3 L2 listing");
printf("scans=%d\n", r3.scans);
CHECK(r3.scans == 3 && r3.L[3].n == 0 && r3.gen[4] == 0,
      "ch25 minsup3 scans=3, L3 empty by support ({I1,I2,I3}=2 < 3), "
      "C4 never generated");
CHECK(supp((1u << 0) | (1u << 1) | (1u << 2)) == 2,
      "ch25 minsup3 the one C3 candidate counted at 2");
```

listing 202: minsup 3 on the same baskets, L3 emptied by support not by prune (source: kdd/samples/src/Ch25/apriori.c, lines 336-362)

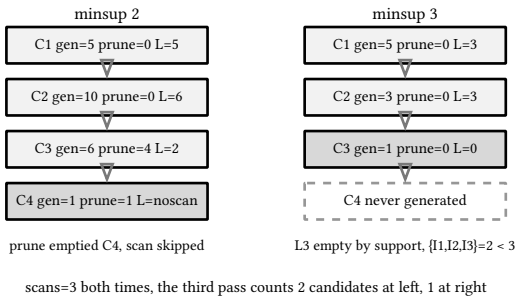


figure 103: both runs take three scans and stop for different reasons

25.4. thirteen itemsets, three implementations

The minsup 2 run's full harvest is 13 itemsets across three levels, and the sample folds them into one line sorted by size then itemset string, the `fi:` line, with checksum 45, the plain sum of all 13 supports. That line is the point of the whole part: the same nine baskets are mined again in book 14, ch 26 (knowledge discovery, vector apriori) by intersecting transaction lists and in book 14, ch 28 (knowledge discovery, fp-growth) by recursing through a prefix tree, and the contract pins all three to byte-identical `fi:` strings with the same count 13 and checksum 45. Three implementations that share no machinery agree on every item and every count, which is the strongest statement a fixture this size can make about an implementation.

The cost accounting explains why two more chapters exist. This chapter's horizontal layout reads every basket on every pass, three passes here, testing 5 then 10 then 2 candidates per row. The vertical lane builds one column per item and never rescans, and fp-growth compresses the baskets into a tree and grows patterns inside conditional fragments of it. Same 13 answers, three different bills.

```
  int fi_count = 0, fi_sum = 0;
  for (int k = 1; k ≤ r.nlev; k++) {
    fi_count += r.L[k].n;
    for (int i = 0; i < r.L[k].n; i++) {
      fi_sum += r.L[k].c[i];
    }
  }
  printf("fi_count=%d fi_checksum=%d\n", fi_count, fi_sum);
  CHECK(fi_count == 13, "ch25 minsup2 fi_count=13");
  CHECK(fi_sum == 45, "ch25 minsup2 fi_checksum=45");
  fi_line(&r, buf);
  printf("%s\n", buf);
  CHECK(strcmp(buf, FI_PIN) == 0,
        "ch25 minsup2 fi listing byte-equal (three-way M2 agreement)");
```

listing 203: the fi aggregate, count 13, checksum 45, byte-pinned string (source: kdd/samples/src/Ch25/apriori.c, lines 321-334)

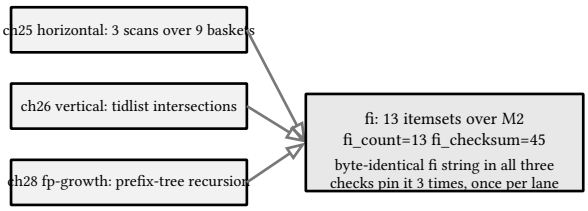


figure 104: three implementations, one enumeration, the three-way M2 agreement

sources: the nine-basket table is the running example of Han, Pei, Yin, Mining Frequent Patterns without Candidate Generation, SIGMOD 2000, TIDs 100-900, https://www.cs.sfu.ca/~jpei/publications/sigmod00.pdf, accessed 2026-09-22, pinned by kdd-contract-s6.md for chapters 25, 26 and 28 together so the cross-chapter agreement rows rest on one shared table. The candidate line format, the noscan and scans semantics, the prune provenance rows and the minsup 3 run are pinned by the same sheet and witnessed by playground/kdd-matrix/gen_s6.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch25`, 23 checks in chapter 25 of the kdd suite.

26. vector apriori

One sample carries the chapter: `vector.c` transposes chapter 25′s nine baskets into one transaction list per item and recomputes everything support-related with set operations, 51 checks. The chapter makes 4 moves: the vertical flip that turns row scanning into list intersection, the compound tidlists that grow itemsets without touching the baskets again, the bit-vector spelling where intersection is a single AND and support is a popcount, and the agreement row asserting all 13 frequent itemsets match book 14, ch 25 (knowledge discovery, levelwise apriori) count for count. The part's arc is one task in three implementations, and this is the lane that stops re-reading the database.

26.1. one column per item

The horizontal layout of book 14, ch 25 (knowledge discovery, levelwise apriori) stores one basket per transaction and asks each candidate to walk all nine rows. The vertical layout stores one column per item, the tidlist of an item being the set of transaction ids whose baskets contain it, and the support of any itemset becomes the size of the intersection of its items' tidlists. One pass over the baskets builds the five lists, and nothing after that reads a basket again.

The dry run: the sample prints the five lists, `tidlist I1:{100,400,500,700,800,900}` down to `tidlist I5:{100,800}`, with `tidlist I2:{100,200,300,400,600,800,900}` the longest at 7 entries, every basket but 500 and 700. Then all ten pairs, each one intersection: `pair {I1,I2}=4`, `pair {I1,I3}=4`, `pair {I1,I4}=1`, `pair {I1,I5}=2`, `pair {I2,I3}=4`, `pair {I2,I4}=2`, `pair {I2,I5}=2`, `pair {I3,I4}=0`, `pair {I3,I5}=1`, `pair {I4,I5}=0`. Six pairs clear minsup 2 and four fall below it, the same four whose absence pruned chapter 25′s C3, now seen from underneath as short intersections instead of small row counts.

```c
/* vertical layout: item -> bitmask over transactions (bit t = TID t) */
static unsigned tmap(unsigned m) {
  unsigned b = ~0u;
  for (int i = 0; i < NIT; i++) {
    if (m & (1u << i)) {
      unsigned v = 0;
      for (int t = 0; t < NTX; t++) {
        if (m2[t] & (1u << i)) {
          v |= 1u << t;
        }
      }
      b &= v;
    }
  }
  return b;
}

static int vsupp(unsigned m) { return pc(tmap(m)); }

/* horizontal apriori support (the ch25 counting side of the agreement) */
static int hsupp(unsigned m) {
  int c = 0;
  for (int t = 0; t < NTX; t++) {
    if ((m2[t] & m) == m) {
      c++;
    }
  }
  return c;
}
```

listing 204: the vertical layout and both support spellings, intersect and scan (source: kdd/samples/src/Ch26/vector.c, lines 78-106)

```c
  /* all 10 pair intersection supports in (size, itemset string) order */
  const int pair_pin[10] = {4, 4, 1, 2, 4, 2, 2, 0, 1, 0};
  int pi = 0;
  for (int i = 0; i < NIT; i++) {
    for (int j = i + 1; j < NIT; j++) {
      const unsigned m = (1u << i) | (1u << j);
      const int v = vsupp(m);
      iset(m, s);
      printf("pair %s=%d\n", s, v);
      char name[48];
      snprintf(name, sizeof name, "ch26 pair %s=%d", s, pair_pin[pi]);
      CHECK(v == pair_pin[pi], name);
      pi++;
    }
  }
```

listing 205: all ten pair supports, one intersection each (source: kdd/samples/src/Ch26/vector.c, lines 153-167)

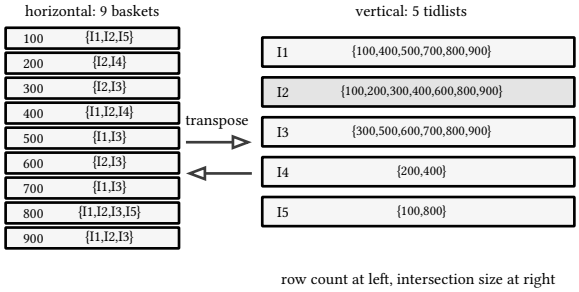


figure 105: the same nine baskets read row-wise and column-wise

NOTE: antimonotonicity seen from the side

The subset law of chapter 25 shows up here as set inclusion: intersecting can only shrink a list, so every item added to an itemset weakens or preserves its tidlist, never grows it. {I3,I5} at intersection size 1 retires its upper cone exactly like the horizontal prune did, with the verdict visible in the lists themselves rather than in a rule applied before counting.

26.2. compound tidlists

Nothing limits the intersection to pairs. The tidlist of {I1,I2} is the intersection of the two single-item lists, and it is itself a list, so triples are one more intersection against a stored result rather than a new pass over anything.

The dry run: the sample pins three compound lists, `tidlist {I1,I2}:{100,400,800,900}`, then `tidlist {I1,I2,I3}:{800,900}` from one more intersection with I3′s list, and `tidlist {I1,I2,I5}:{100,800}` from I5′s instead. The two triples are exactly chapter 25′s L3, both at size 2, and the pair lists make the levelwise structure visible: the eight transactions that ever buy both I1 and I2 narrow to two that also buy I3, and to a different two that also buy I5, with transaction 800 the only one buying all of I1, I2 and I3.

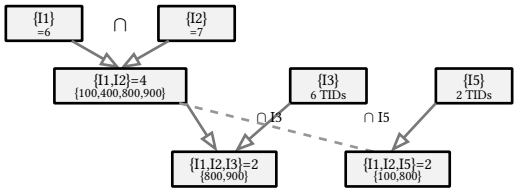
```
static void tl_str(unsigned m, char *buf) {
  char *p = buf;
  *p++ = '{';
  int first = 1;
  for (int t = 0; t < NTX; t++) {
    if (tmap(m) & (1u << t)) {
      if (!first) {
        *p++ = ',';
      }
      first = 0;
      p += sprintf(p, "%d", tid[t]);
    }
  }
  *p++ = '}';
  *p = 0;
}

static void bits_str(unsigned x, int n, char *buf) {
  for (int i = 0; i < n; i++) {
    buf[i] = (x & (1u << i)) ? '1' : '0';
  }
  buf[n] = 0;
}
```

listing 206: tidlist and bit-vector strings, the two pinned spellings (source: kdd/samples/src/Ch26/vector.c, lines 108-130)

```
/* compound tidlists, byte-pinned */
const unsigned comp[3] = {(1u << 0) | (1u << 1),
                          (1u << 0) | (1u << 1) | (1u << 2),
                          (1u << 0) | (1u << 1) | (1u << 4)};
const char *comp_pin[3] = {"{100,400,800,900}", "{800,900}", "{100,800}"};
for (int k = 0; k < 3; k++) {
  tl_str(comp[k], buf);
  iset(comp[k], s);
  printf("tidlist %s:%s\n", s, buf);
  char name[64];
  snprintf(name, sizeof name, "ch26 tidlist %s = %s", s, comp_pin[k]);
  CHECK(strcmp(buf, comp_pin[k]) == 0, name);
}
```

listing 207: the three compound tidlists, intersections of stored lists (source: kdd/samples/src/Ch26/vector.c, lines 169-181)



both triples are chapter 25's L3, at exactly 2

figure 106: compound supports by intersection alone, no basket read again

26.3. bits and popcounts

A tidlist over nine transactions is nine booleans, so the natural machine spelling is one bit per transaction. The sample prints the matrix both ways, as transaction vectors `txbits 100:11001` through `txbits 900:11100`, one char per item in item order, and as item vectors `itembits I1:100110111` through `itembits I5:100000010`, one char per transaction in TID order with position 1 the leftmost meaning T100. It is the same 9 by 5 matrix of 23 set bits read along rows or down columns.

The payoff is arithmetic. Intersection of two item vectors is one AND, and support is the popcount of the result. The dry run: `itembits`
`I2:111101011 popcount=7`, the seven baskets again, and the three pinned AND rows are `and I1&I2:100100011 popcount=4`, and `I1&I3:000010111`
`popcount=4`, and `I2&I3:001001011 popcount=4`, the same three 4s the pair census produced by list intersection, one machine word each.

```
  /* transaction bit vectors: one char per item in ASCII item order */
  const char *tx_pin[NTX] = {"11001", "01010", "01100", "11010", "10100",
                             "01100", "10100", "11101", "11100"};
  for (int t = 0; t < NTX; t++) {
    bits_str(m2[t], NIT, bs);
    printf("txbits %d:%s\n", tid[t], bs);
    char name[48];
    snprintf(name, sizeof name, "ch26 txbits %d = %s", tid[t], tx_pin[t]);
    CHECK(strcmp(bs, tx_pin[t]) == 0, name);
  }

  /* item bit vectors over TIDs 100..900, leftmost = T100 */
  const char *ib_pin[NIT] = {"100110111", "111101011", "001011111", "010100000",
                             "100000010"};
  const int ib_pop[NIT] = {6, 7, 6, 2, 2};
  for (int i = 0; i < NIT; i++) {
    bits_str(tmap(1u << i), NTX, bs);
    sprintf(s, "I%d", i + 1);
    printf("itembits %s:%s popcount=%d\n", s, bs, pc(tmap(1u << i)));
    char name[64];
    snprintf(name, sizeof name, "ch26 itembits I%d = %s popcount=%d", i + 1,
             ib_pin[i], ib_pop[i]);
    CHECK(strcmp(bs, ib_pin[i]) == 0 && pc(tmap(1u << i)) == ib_pop[i], name);
  }
```

listing 208: transaction vectors then item vectors, both byte-pinned (source: kdd/samples/src/Ch26/vector.c, lines 183-206)

```
  /* pairwise AND rows for the three frequent i<j pairs pinned by name */
  const int and_pin[3][2] = {{0, 1}, {0, 2}, {1, 2}};
  const char *and_bits[3] = {"100100011", "000010111", "001001011"};
  const int and_pop[3] = {4, 4, 4};
  for (int k = 0; k < 3; k++) {
    const int i = and_pin[k][0], j = and_pin[k][1];
    const unsigned a = tmap(1u << i) & tmap(1u << j);
    bits_str(a, NTX, bs);
    printf("and I%d&I%d:%s popcount=%d\n", i + 1, j + 1, bs, pc(a));
    char name[56];
    snprintf(name, sizeof name, "ch26 and I%d&I%d = %s popcount=%d", i + 1,
             j + 1, and_bits[k], and_pop[k]);
    CHECK(strcmp(bs, and_bits[k]) == 0 && pc(a) == and_pop[k], name);
  }
```

listing 209: pairwise AND rows with popcounts, three frequent pairs by name (source: kdd/samples/src/Ch26/vector.c, lines 208-221)

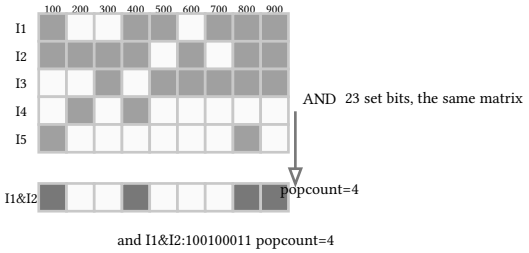


figure 107: the vertical matrix as bits, AND intersects, popcount supports

PITFALL: bit order is a convention, and it is load-bearing

The sheet pins position 1 to T100 at the left end, and the printed rows are bytes, not numbers: `100100011` read right to left would still popcount to 4 but would name the wrong transactions, 800, 400 and 100 in the wrong

> order, and any logic keyed on positions, diffsets or prefix compression inherits the flip silently. The checks compare the full string, not just the count, which is exactly what makes the convention testable.

26.4. the agreement, all thirteen

The vertical lane can stand alone: enumerate every nonempty itemset, intersect, keep what clears minsup 2. The sample does exactly that and sorts the survivors by size then itemset string, and the result must be chapter 25's harvest, item for item and count for count.

The dry run: thirteen agreement rows print, `agree {I1} vertical=6 horizontal=6` through `agree {I1,I2,I5} vertical=2 horizontal=2`, each one an intersection size checked against a fresh subset scan, then the aggregate `fi_count=13 fi_checksum=45` and the sorted `fi:` line, byte-equal to chapter 25's. This is the second third of the part's three-way M2 agreement: the levelwise scan, the tidlist intersection, and book 14, ch 28 (knowledge discovery, fp-growth)'s tree recursion all land on the same 13 itemsets over the same baskets, pinned once per chapter. The vertical lane got there without a second look at the baskets, which is the whole argument for it.

```
/* CROSS-FIXTURE AGREEMENT: enumerate itemsets, keep vertical-frequent
 * (≥ 2), each must match the horizontal apriori support */
unsigned fi[32];
int fic[32], nfi = 0;
for (unsigned m = 1; m < (1u << NIT); m++) {
  const int v = vsupp(m);
  if (v ≥ 2) {
    fi[nfi] = m;
    fic[nfi] = v;
    nfi++;
  }
}
/* sort by (size, itemset string) */
for (int i = 1; i < nfi; i++) {
  for (int j = i; j > 0; j--) {
    char a[24], b[24];
    iset(fi[j - 1], a);
    iset(fi[j], b);
    if (pc(fi[j - 1]) < pc(fi[j]) ||
        (pc(fi[j - 1]) == pc(fi[j]) && strcmp(a, b) ≤ 0)) {
      break;
    }
    const unsigned tm = fi[j - 1];
    const int tc = fic[j - 1];
    fi[j - 1] = fi[j];
    fic[j - 1] = fic[j];
    fi[j] = tm;
    fic[j] = tc;
  }
}
```

listing 210: enumerate, intersect, keep, the vertical miner and its sort (source: kdd/samples/src/Ch26/vector.c, lines 223-252)

```
  int sum = 0;
  for (int i = 0; i < nfi; i++) {
    iset(fi[i], s);
    printf("agree %s vertical=%d horizontal=%d\n", s, fic[i], hsupp(fi[i]));
    char name[64];
    snprintf(name, sizeof name,
             "ch26 agree tidlist intersection == apriori support %s", s);
    CHECK(fic[i] == hsupp(fi[i]), name);
    sum += fic[i];
  }
  printf("fi_count=%d fi_checksum=%d\n", nfi, sum);
  CHECK(nfi == 13, "ch26 vertical fi_count=13");
  CHECK(sum == 45, "ch26 vertical fi_checksum=45");
  char *p = buf;
  p += sprintf(p, "fi: ");
  for (int i = 0; i < nfi; i++) {
    if (i) {
      *p++ = ';';
    }
    iset(fi[i], s);
    p += sprintf(p, "%s=%d", s, fic[i]);
  }
  printf("%s\n", buf);
  CHECK(strcmp(buf, FI_PIN) == 0,
        "ch26 fi listing byte-equal to ch25/ch28 (three-way agreement)");
```

listing 211: thirteen agreement rows, the aggregate, the byte-pinned fi line (source: kdd/samples/src/Ch26/vector.c, lines 253-277)

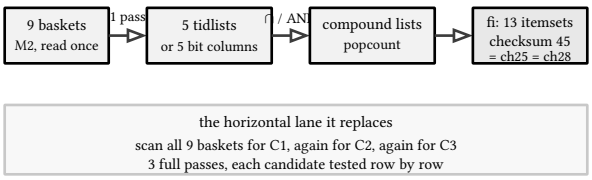


figure 108: one pass builds the columns, every later count is pure set work

sources: the nine-basket table is the running example of Han, Pei, Yin, Mining Frequent Patterns without Candidate Generation, SIGMOD 2000, TIDs 100-900, https://www.cs.sfu.ca/~jpei/publications/sigmod00.pdf, accessed 2026-09-22, pinned by kdd-contract-s6.md for chapters 25, 26 and 28 together. The tidlist and bit-vector formats, the leftmost-means-T100 convention, and the agreement rows are pinned by the same sheet and witnessed by playground/kdd-matrix/gen_s6.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch26`, 51 checks in chapter 26 of the kdd suite.

27. maximal itemsets and rule generation

Two samples carry the chapter: `closed.c` classifies the 13 frequent itemsets of book 14, ch 25 (knowledge discovery, levelwise apriori) into closed, maximal and neither, 12 checks, and `rules.c` splits one frequent triple of a purpose-built 12-basket fixture into all six association rules with exact fractions, 15 checks. The chapter makes 4 moves: the closed border where support stops falling, the maximal frontier where the frequent set stops growing, the rule split with its support, confidence and lift fractions, and the minconf cut that keeps 5 of the 6 rules by cross multiplication. The borders are the answer to "13 itemsets is already a lot", and the rules are the answer to "now what is one worth", which book 14, ch 29 (knowledge discovery, interestingness, multilevel, and quantitative rules) takes further.

27.1. the lattice and its two borders

Chapter 25 harvested 13 frequent itemsets from M2 at minsup 2. Reading them as a lattice by subset inclusion, two borders matter. An itemset is closed when no frequent proper superset has the same support, and it is maximal when no frequent proper superset exists at all. Every maximal itemset is closed, since a set with no frequent superset certainly has none at equal support, and the sample pins that direction as `maximal_subset_of_closed=true`.

The dry run: the sample reprints the three levels identical to chapter 25, then classifies. Nine itemsets are closed, `closed: {I1}=6;{I2}=7;{I3}=6;{I1,I2}=4;{I1,I3}=4;{I2,I3}=4;{I2,I4}=2;{I1,I2,I3}=2;{I1,I2,I5}=2`. Three are maximal, `maximal: {I2,I4}=2;{I1,I2,I3}=2;{I1,I2,I5}=2`, both L3 triples plus a pair: {I2,I4} has no frequent superset because every candidate third item fails, {I1,I2,I3} and {I1,I2,I5} sit at the top of their chains. The remaining six closed itemsets are the interior, `closed_not_maximal: {I1}=6;{I2}=7;{I3}=6;{I1,I2}=4;{I1,I3}=4;{I2,I3}=4`, singletons and pairs that some frequent superset still grows from.

```
/* all 13 frequent itemsets of M2 minsup 2, sorted by (size, string) */
int n = 0;
for (unsigned m = 1; m < (1u << NIT) && n < NF; m++) {
  if (supp(m) >= 2) {
    fm[n] = m;
    fc[n] = supp(m);
    n++;
  }
}
for (int i = 1; i < n; i++) {
  for (int j = i; j > 0; j--) {
    char a[24], b[24];
    iset(fm[j - 1], a);
    iset(fm[j], b);
    if (pc(fm[j - 1]) < pc(fm[j]) ||
        (pc(fm[j - 1]) == pc(fm[j]) && strcmp(a, b) <= 0)) {
      break;
    }
    const unsigned tm = fm[j - 1];
    const int tc = fc[j - 1];
    fm[j - 1] = fm[j];
    fc[j - 1] = fc[j];
    fm[j] = tm;
    fc[j] = tc;
  }
}
```

listing 212: the 13 frequent itemsets collected and sorted by size then string (source: kdd/samples/src/Ch27/closed.c, lines 117-142)

```
int closed[NF] = {0}, maximal[NF] = {0};
for (int i = 0; i < n; i++) {
  int ismax = 1, isclosed = 1;
  for (int j = 0; j < n; j++) {
    if (j == i || fm[j] == fm[i] || (fm[j] & fm[i]) != fm[i]) {
      continue; /* fm[j] not a proper superset of fm[i] */
    }
    ismax = 0;
    if (fc[j] == fc[i]) {
      isclosed = 0; /* equal-support proper superset kills closed */
    }
  }
  closed[i] = isclosed;
  maximal[i] = ismax;
}
```

listing 213: both borders in one scan, any proper superset, then any at equal support (source: kdd/samples/src/Ch27/closed.c, lines 170-184)

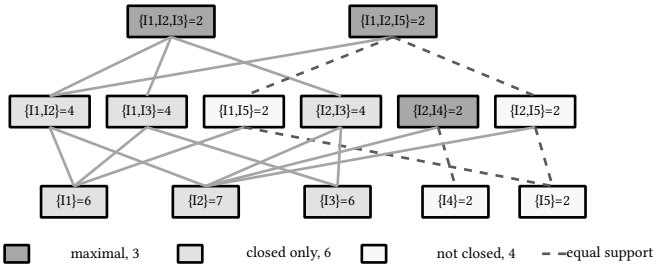


figure 109: the 13-itemset lattice, dashed edges where support holds instead of falling

27.2. witnesses and what each border keeps

The four itemsets that fail to close all sit at support 2, and each has a frequent proper superset holding the same count, its witness. The sample pins all four pairs.

The dry run: `nonclosed {I4}=2 witness {I2,I4}=2`, then `nonclosed {I5}=2 witness {I1,I5}=2`, then `nonclosed {I1,I5}=2 witness {I1,I2,I5}=2` and `nonclosed {I2,I5}=2 witness {I1,I2,I5}=2`. I4 never appears without I2 in this table, and I5 never appears in a basket that is not already buying I1, so the smaller sets carry no information their witnesses do not.

That is the whole trade between the two borders. The closed set, 9 of 13, is lossless: every non-closed itemset's support is recoverable as its witness's support, so the closed list plus the lattice regenerates all 13 counts. The maximal set, 3 of 13, is smaller and lossy: it determines which itemsets are frequent, since every frequent itemset is a subset of a maximal one, but it no longer pins the counts, {I4} and {I5} both reducible only to "somewhere under {I2,I4} or {I1,I2,I5}". Which border to keep depends on whether the counts feed anything downstream, and the rules in the next two facets need them.

```
 * {I2,I5}->{I1,I2,I5} */
const unsigned I1 = 1u << 0, I2 = 1u << 1, I4 = 1u << 3, I5 = 1u << 4;
const unsigned nonclosed[4] = {I4, I5, I1 | I5, I2 | I5};
const unsigned witness[4] = {I2 | I4, I1 | I5, I1 | I2 | I5, I1 | I2 | I5};
for (int k = 0; k < 4; k++) {
  const int ix = idx_of(nonclosed[k]);
  const int wx = idx_of(witness[k]);
  /* witness qualifies: frequent, proper superset, equal support */
  char a[24], b[24];
  iset(nonclosed[k], a);
  iset(witness[k], b);
  printf("nonclosed %s=%d witness %s=%d\n", a, supp(nonclosed[k]), b,
         supp(witness[k]));
  char name[64];
  snprintf(name, sizeof name, "ch27 non-closed %s witnessed by %s at 2", a,
           b);
  CHECK(ix ≥ 0 && wx ≥ 0 && !closed[ix] && fm[wx] ≠ nonclosed[k] &&
            (fm[wx] & nonclosed[k]) == nonclosed[k] && fc[wx] == fc[ix] &&
            fc[ix] == 2,
        name);
}
```

listing 214: the four non-closed itemsets with their equal-support witnesses (source: kdd/samples/src/Ch27/closed.c, lines 214-234)

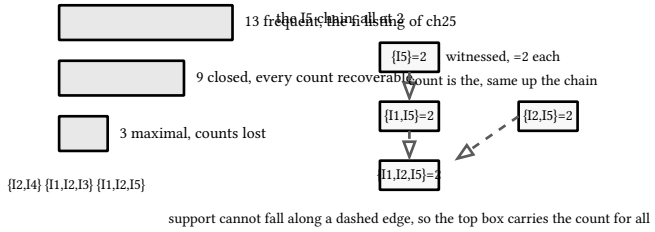


figure 110: 13 frequent, 9 closed and lossless, 3 maximal and lossy

27.3. six rules from one triple

An association rule X -> Y reads “baskets containing X tend to contain Y”, and it is scored from the support counts chapter 25 computed. This is the same IF-THEN shape book 14, ch 15 (knowledge discovery, rules from trees and sequential covering) read off decision tree leaves, but grown from co-occurrence counts instead of a tree. The fixture R is built for the arithmetic: 12 baskets over a, b, c at minsup 6, six of them {a,b,c}, one {b,c}, one {b}, two {a} and two {c}, giving supports a=8, b=8, c=9, ab=6, ac=6, bc=7, abc=6, so every rule from {a,b,c} lands on a clean fraction.

The dry run: all six non-empty proper splits print, `{a}→{b,c} s=1/2 c=3/4 l=9/7`, `{b}→{a,c} s=1/2 c=3/4 l=3/2`, `{c}→{a,b} s=1/2 c=2/3 l=4/3`, `{a,b}→{c} s=1/2 c=1 l=4/3`, `{a,c}→{b} s=1/2 c=1 l=3/2`, `{b,c}→{a} s=1/2 c=6/7 l=9/7`. The three fractions are exact reduced ratios: support is supp(X union Y) over N, all six at 6/12, confidence is supp(X union Y) over supp(X), and lift is N times supp(X union Y) over the product of the two marginal supports. The row `{b,c}→{a}` shows the fixture’s one interesting confidence: 6 of the 7 {b,c} baskets also buy a, 6/7, while the reverse direction `{a}→{b,c}` gives 6 of 8, 3/4.

```
/* L1/L2/L3 at minsup 6 (all subsets, grouped by size) */
const char *lp[3] = {"L1: {a}=8;{b}=8;{c}=9", "L2: {a,b}=6;{a,c}=6;{b,c}=7",
                     "L3: {a,b,c}=6"};
unsigned sub[8];
int nsub = 0;
for (unsigned m = 1; m < (1u << NIT); m++) {
  if (supp(m) >= 6) {
    sub[nsub++] = m;
  }
}
for (int i = 1; i < nsub; i++) {
  for (int j = i; j > 0; j--) {
    char a[16], b[16];
    iset(sub[j - 1], a);
    iset(sub[j], b);
    if (pc(sub[j - 1]) < pc(sub[j]) ||
        (pc(sub[j - 1]) == pc(sub[j]) && strcmp(a, b) <= 0)) {
      break;
    }
    const unsigned tm = sub[j - 1];
    sub[j - 1] = sub[j];
    sub[j] = tm;
  }
}
CHECK(nsub == 7, "ch27 R minsup6 lattice has 7 frequent itemsets");
```

listing 215: fixture R's frequent itemsets at minsup 6, grouped by size (source: kdd/samples/src/Ch27/rules.c, lines 136-160)

```
/* all 6 non-empty proper splits of {a,b,c}, X in (size, string) order */
const unsigned abc = 7;
unsigned xs[6] = {1u << 0,
                  1u << 1,
                  1u << 2,
                  (1u << 0) | (1u << 1),
                  (1u << 0) | (1u << 2),
                  (1u << 1) | (1u << 2)};
const char *rule_pin[6] = {
    "{a}->{b,c} s=1/2 c=3/4 l=9/7", "{b}->{a,c} s=1/2 c=3/4 l=3/2",
    "{c}->{a,b} s=1/2 c=2/3 l=4/3", "{a,b}->{c} s=1/2 c=1 l=4/3",
    "{a,c}->{b} s=1/2 c=1 l=3/2",   "{b,c}->{a} s=1/2 c=6/7 l=9/7"};
int cn[6], cd[6];
for (int k = 0; k < 6; k++) {
  const unsigned X = xs[k], Y = abc ^ X;
  const int sxy = supp(X | Y), sx = supp(X), sy = supp(Y);
  fstr(sxy, NTX, fs[0]);
  fstr(sxy, sx, fs[1]);
  fstr(NTX * sxy, sx * sy, fs[2]);
  reduce(sxy, sx, &cn[k], &cd[k]);
  iset(X, s);
  char y[16];
  iset(Y, y);
  printf("%s->%s s=%s c=%s l=%s\n", s, y, fs[0], fs[1], fs[2]);
  char out[80];
  sprintf(out, "%s->%s s=%s c=%s l=%s", s, y, fs[0], fs[1], fs[2]);
  CHECK(strcmp(out, rule_pin[k]) == 0, "ch27 R rule row byte-equal");
}
```

listing 216: the six splits, three reduced fractions each, byte-pinned rows (source: kdd/samples/src/Ch27/rules.c, lines 180-207)

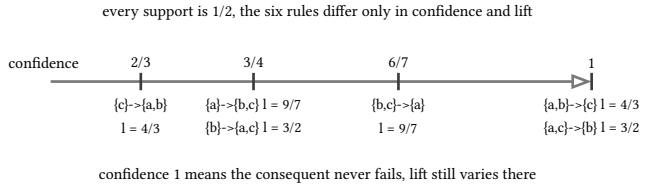


figure 111: the six confidences with their lifts, certainty at the right

27.4. the minconf cut, by cross multiplication

A rule survives a minimum confidence threshold when c >= minconf, and with both sides exact fractions the comparison is cross multiplication, never a double. The sheet plants the boundary cases deliberately.

The dry run: at minconf 2/3 the sample prints `minconf 2/3 survivors=6`, all six rules clear it, the weakest being exactly 2/3. At minconf 3/4 it prints `minconf 3/4 survivors=5`, and names the casualty: `drop {c} c=2/3`, since 2 times 4 is 8 against 3 times 3 is 9, while the nearest survivor holds, `keep {b,c}→{a} c=6/7`, 6 times 4 being 24 against 3 times 7 being 21. The two rules at confidence 1, {a,b}->{c} and {a,c}->{b}, pass any threshold below 1 without arithmetic, and their conviction is undefined, the denominator 1 minus c being zero, which is book 14, ch 29 (knowledge discovery, interestingness, multilevel, and quantitative rules)'s opening problem: confidence saturates, lift and conviction do not.

```
/* minconf survivor filters, exact fraction comparison */
int surv23 = 0, surv34 = 0;
for (int k = 0; k < 6; k++) {
  if (cn[k] * 3 >= 2 * cd[k]) {
    surv23++;
  }
  if (cn[k] * 4 >= 3 * cd[k]) {
    surv34++;
  }
}
printf("minconf 2/3 survivors=%d\n", surv23);
CHECK(surv23 == 6, "ch27 minconf 2/3 keeps all 6 rules");
printf("minconf 3/4 survivors=%d\n", surv34);
CHECK(surv34 == 5, "ch27 minconf 3/4 keeps exactly 5 rules");
/* the dropped rule is {c}->{a,b} (c=2/3); {b,c}->{a} at 6/7 survives */
int dropix = -1, bca = -1;
for (int k = 0; k < 6; k++) {
  if (cn[k] * 4 < 3 * cd[k]) {
    dropix = k;
  }
  if (xs[k] == ((1u << 1) | (1u << 2))) {
    bca = k;
  }
}
iset(xs[dropix], s);
printf("drop %s c=%d/%d\n", s, cn[dropix], cd[dropix]);
CHECK(dropix == 2 && cn[dropix] == 2 && cd[dropix] == 3,
      "ch27 minconf 3/4 drops only {c}->{a,b} at 2/3");
printf("keep {b,c}->{a} c=%d/%d\n", cn[bca], cd[bca]);
CHECK(cn[bca] == 6 && cd[bca] == 7 && cn[bca] * 4 > 3 * cd[bca],
      "ch27 {b,c}->{a} at 6/7 survives minconf 3/4");
```

listing 217: exact threshold tests by cross multiplication, the drop and the keep (source: kdd/samples/src/Ch27/rules.c, lines 209-239)

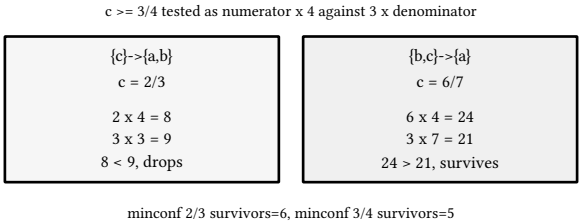


figure 112: two boundary fractions, one drops at 3/4, one survives

sources: the nine-basket table is the running example of Han, Pei, Yin, Mining Frequent Patterns without Candidate Generation, SIGMOD 2000, TIDs 100-900, https://www.cs.sfu.ca/~jpei/publications/sigmod00.pdf, accessed 2026-09-22, mined identically by chapters 25, 26 and 28. Fixture R, its supports a=8 b=8 c=9 ab=6 ac=6 bc=7 abc=6 realized by 6x{a,b,c} plus {b,c}, {b}, {a}, {a}, {c}, {c}, the closed and maximal listings, the witness rows and both minconf filters are pinned by kdd-contract-s6.md and witnessed by playground/kdd-matrix/gen_s6.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch27`, 12 + 15 checks in

chapter 27 of the kdd suite.

28. fp-growth

Two samples carry the chapter: `fpgrowth.c` builds the tree, the F-list, the F-ordered transactions and the five conditional pattern bases, 16 checks, and `mine.c` rebuilds the same tree and runs the recursion to the full enumeration, 7 checks. The chapter makes 4 moves: the two counting passes and the frequency order that makes prefixes collide, the prefix tree that compresses 23 item occurrences into 10 nodes, the conditional pattern bases that slice the tree per item, and the recursion that enumerates all 13 frequent itemsets without ever generating a candidate set, landing byte-identical to book 14, ch 25 (knowledge discovery, levelwise apriori) and book 14, ch 26 (knowledge discovery, vector apriori). This is the third lane of the part's one task.

28.1. two passes and the F-list

Fp-growth reads the database exactly twice. The first pass counts single items, nothing else, and ranks them: descending count, ties ascending item, so with I1 and I3 both at 6 the F-list reads I1 first, and with I4 and I5 both at 2 it reads I4 first. The second pass rewrites every transaction with its items in that F-list order and nothing else, infrequent items dropped, though on this fixture at minsup 2 every item survives.

The dry run: the sample prints `flist: I2:7 I1:6 I3:6 I4:2 I5:2`, then the nine rewrites, `ordered 100: I2,I1,I5`, `ordered 200: I2,I4`, `ordered 300: I2,I3`, `ordered 400: I2,I1,I4`, `ordered 500: I1,I3`, `ordered 600: I2,I3`, `ordered 700: I1,I3`, `ordered 800: I2,I1,I3,I5`, `ordered 900: I2,I1,I3`. Basket 100 held {I1,I2,I5} in item order, now it reads I2 first because I2 is the most frequent item, and the point of the convention is exactly that: every transaction fronts-loads the items it is most likely to share with other transactions.

```
int cnt[NIT];
for (int i = 0; i < NIT; i++) {
  cnt[i] = 0;
  for (int t = 0; t < NTX; t++) {
    if (m2[t] & (1u << i)) {
      cnt[i]++;
    }
  }
}
int fr_ord[NIT], used[NIT] = {0};
for (int r = 0; r < NIT; r++) {
  int best = -1;
  for (int i = 0; i < NIT; i++) {
    if (used[i]) {
      continue;
    }
    if (best < 0 || cnt[i] > cnt[best]) {
      best = i; /* strict >: ties resolve to the smaller item */
    }
  }
  fr_ord[r] = best;
  used[best] = 1;
}
```

listing 218: the F-list, descending count with ties to the smaller item (source: kdd/samples/src/Ch28/fpgrowth.c, lines 150-172)

```
  tree_t top;
  tinit(&top);
  for (int t = 0; t < NTX; t++) {
    int its[NIT], k = 0;
    for (int r = 0; r < NIT; r++) {
      if (m2[t] & (1u << fr_ord[r])) {
        its[k++] = fr_ord[r];
      }
    }
    p = s;
    for (int i = 0; i < k; i++) {
      if (i) {
        *p++ = ',';
      }
      p += sprintf(p, "I%d", its[i] + 1);
    }
    *p = 0;
    printf("ordered %d: %s\n", tid[t], s);
    char name[48];
    snprintf(name, sizeof name, "ch28 ordered %d = %s", tid[t], ord_pin[t]);
    CHECK(strcmp(s, ord_pin[t]) == 0, name);
    tinsert(&top, its, k, 1);
  }
```

listing 219: pass 2, each transaction rewritten in F-list order and fed to the tree (source: kdd/samples/src/Ch28/fpgrowth.c, lines 186-208)

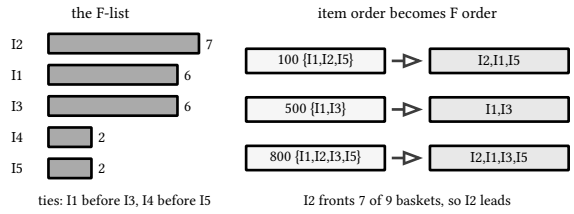


figure 113: rank by count, then write transactions so shared items come first

NOTE: ordering is the compression strategy

Nothing about the baskets changes when they are rewritten, the items are the same items. What changes is which prefixes collide: putting the most frequent item first means the tree below it amortizes over the most transactions. Sorting alphabetically instead would build an honest tree that shares almost nothing, and the recursion later would walk ten skinny conditional trees instead of a few bushy ones.

28.2. the tree: shared prefixes, counted edges

The second pass inserts each F-ordered transaction into a tree that starts at a null root. Walking the transaction item by item, an existing child is reused and its count incremented, a missing child is created with count 1, and children are kept in first-insertion order. Basket 100 builds I2, I1, I5 off the root. Basket 200 shares I2 and forks to I4. Basket 500 is the first transaction without I2, so it creates the root's second child I1.

The dry run: after all nine insertions the sample prints the whole tree as the pinned 11-line string, `null`, `-I2:7`, `--I1:4`, `——I5:1`, `——I4:1`, `——I3:2`, `————I5:1`, `--I4:1`, `--I3:2`, `-I1:2`, `--I3:2`. Ten real nodes hold what was 23 item occurrences across the nine baskets, I2's seven baskets all ride one node, and every count is legible directly: I2:7 at depth 1, the four baskets that pair I2 with I1 at depth 2, the lone basket 800 extending I2, I1, I3 with I5 at depth 4. Insertion order is pinned too, children appear when first created,

which is why I5 precedes I4 and I3 under I2-I1.

```
static void tinsert(tree_t *t, const int *items, int k, int cnt) {
  int cur = 0;
  for (int i = 0; i < k; i++) {
    int c = t->n[cur].fc;
    while (c && t->n[c].item != items[i]) {
      c = t->n[c].ns;
    }
    if (!c) {
      c = t->nn++;
      t->n[c].item = items[i];
      t->n[c].cnt = 0;
      t->n[c].par = cur;
      t->n[c].fc = t->n[c].ns = t->n[c].link = 0;
      if (!t->n[cur].fc) {
        t->n[cur].fc = c;
      } else {
        int s = t->n[cur].fc;
        while (t->n[s].ns) {
          s = t->n[s].ns;
        }
        t->n[s].ns = c;
      }
      if (!t->head[items[i]]) {
        t->head[items[i]] = c;
      } else {
        int s = t->head[items[i]];
        while (t->n[s].link) {
          s = t->n[s].link;
        }
        t->n[s].link = c;
      }
    }
    t->n[c].cnt += cnt;
    cur = c;
  }
}
```

listing 220: insertion, reuse or create, children and node links in first-insertion order (source: kdd/samples/src/Ch28/fpgrowth.c, lines 66-101)

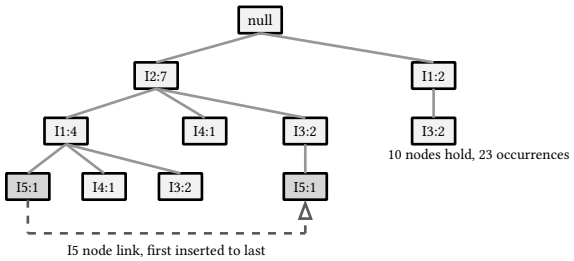


figure 114: the fp-tree, solid edges share prefixes, the dashed chain is the I5 node link

28.3. conditional pattern bases

The node links thread every node of one item together in insertion order, and following a link collects each node's path back to the root: the prefix items above it, with the node's own count. That collection is the item's conditional pattern base, the slice of the database in which the item occurs, spelled as paths in F-list order.

The dry run: the sample prints all five, `cpb I2 ≤ <e>:7`, the frequentest item has no prefix at all, `cpb I1 ≤ I2:4 ; <e>:2`, one path through I2 and one direct, `cpb I3 ≤ I2:2 ; I1:2 ; I2,I1:2`, I3's three nodes, `cpb I4 ≤ I2:1 ; I2,I1:1`, and `cpb I5 ≤ I2,I1:1 ; I2,I1,I3:1`. The I5 base is the one worked to the bottom. Its conditional supports are I2 at 2, I1 at 2, but I3 at 1, appearing in only one of the two paths, and the sheet's rule drops items below minsup inside the base: the sample prints `cond I3 support=1 minsup=2` and checks "ch28 I5 base drops I3 (conditional support 1 < 2)". What remains is a single path, `condtree(I5): null, -I2:2, --I1:2`, and a single-path tree enumerates directly, every subset of the path joining the suffix item at the path count: `base_itemsets: {I5}=2; {I1,I5}=2;{I2,I5}=2;{I1,I2,I5}=2`, four itemsets that are exactly chapter 25's I5-containing frequent sets.

```c
static int cpb_paths(const tree_t *t, int it, path_t *ps) {
  int np = 0;
  for (int v = t->head[it]; v; v = t->n[v].link) {
    path_t *p = &ps[np++];
    p->k = 0;
    p->cnt = t->n[v].cnt;
    for (int a = t->n[v].par; a; a = t->n[a].par) {
      p->it[p->k++] = t->n[a].item;
    }
    for (int i = 0; i < p->k / 2; i++) { /* reverse to top-down order */
      const int tmp = p->it[i];
      p->it[i] = p->it[p->k - 1 - i];
      p->it[p->k - 1 - i] = tmp;
    }
  }
  return np;
}
```

listing 221: one item's paths collected over the node link, prefixes top-down (source: kdd/samples/src/Ch28/fpgrowth.c, lines 128-144)

```c
static int condsupp(const path_t *ps, int np, int it) {
  int s = 0;
  for (int i = 0; i < np; i++) {
    for (int j = 0; j < ps[i].k; j++) {
      if (ps[i].it[j] == it) {
        s += ps[i].cnt;
        break;
      }
    }
  }
  return s;
}

/* conditional tree: drop items with conditional support < minsup */
static int condtree(const path_t *ps, int np, tree_t *out) {
  tinit(out);
  for (int i = 0; i < np; i++) {
    int keep[8], nk = 0;
    for (int j = 0; j < ps[i].k; j++) {
      if (condsupp(ps, np, ps[i].it[j]) ≥ MINSUP) {
        keep[nk++] = ps[i].it[j];
      }
    }
    if (nk) {
      tinsert(out, keep, nk, ps[i].cnt);
    }
  }
  return out->nn > 1;
}
```

listing 222: conditional support per item, then the rebuild without the dropped ones (source: kdd/samples/src/Ch28/mine.c, lines 174-202)

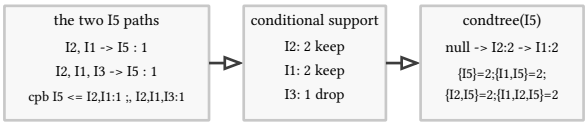


figure 115: the I5 base, its drop, and the single-path conditional tree it becomes

PITFALL: path order is F-list order, not alphabetical

The pinned base for I5 reads `I2,I1,I3`, which is not sorted, and that is deliberate: the path is the prefix of an F-ordered transaction, so it inherits the frequency order. Sorting it to `I1,I2,I3` would still describe the same baskets but would break byte equality with the pinned string and, worse, would misorder the conditional tree's own children, whose first-insertion order is what the next recursion level's node links depend on.

28.4. the recursion, and the third agreement

Mining is the base construction applied recursively. For each item in descending F-list order, worst first, the item joins a growing suffix, its conditional pattern base becomes the next tree, and the recursion continues inside that fragment. Because transactions are F-ordered, a prefix path ending at an item can only contain items ranked above it, so an itemset is always emitted in the branch of its own member that comes last in the F-list, once and only once.

The dry run: the sample runs the full recursion and prints `fi_count=13 fi_checksum=45`, then the sorted `fi:` line, then `equal_to_apriori=true`, checked as "ch28 fp-growth fi equals ch25 apriori (three-way M2 agreement)". The

partition by last-F-list member accounts for all 13: the I5 branch owns the four I5-containing sets, I4′s owns {I4} and {I2,I4}, I3′s owns {I3}=6, {I1,I3}=4, {I2,I3}=4 and {I1,I2,I3}=2, I1′s owns {I1}=6 and {I1,I2}=4, and I2′s owns {I2}=7 alone, with column sums 8, 4, 16, 10 and 7 adding to the checksum 45. No candidate was ever generated, no subset was ever tested against a previous level, and the prune of chapter 25 has no counterpart here because there is nothing to prune: the tree only ever contains frequent-prefix material by construction.

```
static void mine(const tree_t *t, unsigned suffix) {
  for (int r = NIT - 1; r ≥ 0; r--) {
    const int it = fr_ord[r];
    int tot = 0;
    for (int v = t->head[it]; v; v = t->n[v].link) {
      tot += t->n[v].cnt;
    }
    if (tot == 0) {
      continue;
    }
    fim[nfi] = suffix | (1u << it);
    ficnt[nfi] = tot;
    nfi++;
    path_t ps[8];
    const int np = cpb_paths(t, it, ps);
    tree_t ct;
    if (condtree(ps, np, &ct)) {
      mine(&ct, suffix | (1u << it));
    }
  }
}
```

listing 223: the recursion, suffix grows, conditional tree shrinks (source: kdd/samples/src/Ch28/mine.c, lines 209-229)

```
  p = buf;
  p += sprintf(p, "fi: ");
  for (int i = 0; i < nfi; i++) {
    if (i) {
      *p++ = ';';
    }
    iset(fim[i], s);
    p += sprintf(p, "%s=%d", s, ficnt[i]);
  }
  printf("%s\n", buf);
  const int equal = strcmp(buf, FI_PIN) == 0;
  printf("equal_to_apriori=%s\n", equal ? "true" : "false");
  CHECK(equal,
        "ch28 fp-growth fi equals ch25 apriori (three-way M2 agreement)");
```

listing 224: the byte-equal fi line and the equal_to_apriori verdict (source: kdd/samples/src/Ch28/mine.c, lines 382-395)

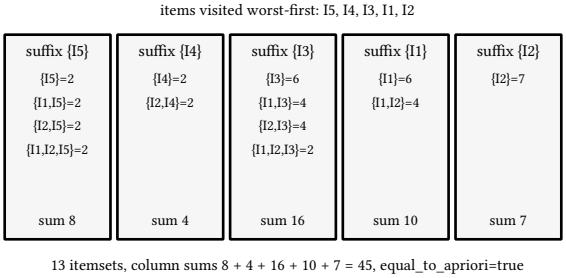


figure 116: the suffix recursion owns each itemset exactly once, by last F-list member

sources: the nine-basket table, the F-list, the ordering convention, the tree string, the five conditional pattern bases and the agreement rows are the running example of Han, Pei, Yin, Mining Frequent Patterns without Candidate Generation, SIGMOD 2000, TIDs 100-900, https://www.cs.sfu.ca/~jpei/publications/sigmod00.pdf, accessed 2026-09-22, as pinned by kdd-contract-s6.md and witnessed by playground/kdd-matrix/gen_s6.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch28`, 16 + 7 checks in chapter 28 of the kdd suite.

29. interestingness, multilevel, and quantitative rules

Two samples carry the chapter: `interesting.c` scores three rules from fixture Q with lift and conviction and runs the chi-square independence test on one of them, 15 checks, and `multilev.c` mines a two-level taxonomy and a binned pair of continuous attributes, 23 checks. The chapter makes 4 moves: the two measures that do not saturate the way confidence did in book 14, ch 27 (knowledge discovery, maximal itemsets and rule generation), the chi-square test that refuses to call lift 5/4 significant on 12 baskets, the concept hierarchy whose roll-up identity holds exactly, and the binning that turns age and income into items. Support and confidence produced rules, this chapter asks whether a rule is worth anything.

29.1. lift and conviction

Confidence has two blind spots: it saturates at 1 where it says nothing about the consequent's marginal rate, and it ignores that rate entirely, calling {a,b}->{c} at 6/6 interesting even when c appears in 9 of 12 baskets anyway. Lift compares the joint rate to the independence baseline, $l = N \cdot \mathrm{supp}(X \cup Y)/(\mathrm{supp}(X) \cdot \mathrm{supp}(Y))$, and conviction measures how far the rule is from implying the consequent always, $v = (1 - \mathrm{supp}(Y)/N)/(1 - c)$, undefined at $c = 1$.

The dry run: fixture Q has 12 baskets with supp A=6, B=6, C=8, D=4, and the sample prints all three A-rules, `{A}->{B} s=1/4 c=1/2 l=1 v=1`, `{A}->{C} s=5/12 c=5/6 l=5/4 v=2`, `{A}->{D} s=1/12 c=1/6 l=1/2 v=4/5`. The A->C arithmetic: A and C co-occur in 5 baskets, so s is 5/12, confidence is 5/6, lift is $12 \cdot 5/(6 \cdot 8) = 5/4$, and conviction is $(1 - 8/12)/(1 - 5/6) = (1/3)/(1/6) = 2$. The A->B row is the independence witness: A and B meet in 3 baskets, exactly the $6 \cdot 6/12$ an independent pair would produce, and the check pins "ch29 Q A and B independent: lift=1 and conviction=1 together". Lift equals 1 exactly when conviction equals 1, and A->D shows the complement, lift 1/2 with conviction below 1 at 4/5.

```
/* the lift/conviction triple: exact fractions, byte-pinned rows */
const int cons[3] = {1, 2, 3}; /* B, C, D */
const char *rule_pin[3] = {"{A}->{B} s=1/4 c=1/2 l=1 v=1",
                           "{A}->{C} s=5/12 c=5/6 l=5/4 v=2",
                           "{A}->{D} s=1/12 c=1/6 l=1/2 v=4/5"};
for (int k = 0; k < 3; k++) {
  const int sX = supp(1), sY = supp(1u << cons[k]);
  const int sXY = supp(1u | (1u << cons[k]));
  fstr(sXY, NTX, fs[0]);
  fstr(sXY, sX, fs[1]);
  fstr(NTX * sXY, sX * sY, fs[2]);
  int vn, vd;
  conv(NTX, sX, sY, sXY, &vn, &vd);
  fstr(vn, vd, fs[3]);
  sprintf(buf, "{A}->{%c} s=%s c=%s l=%s v=%s", iname[cons[k]], fs[0], fs[1],
          fs[2], fs[3]);
  printf("%s\n", buf);
  CHECK(strcmp(buf, rule_pin[k]) == 0, "ch29 Q lift/conviction row");
}
```

listing 225: three rules, four exact fractions each, conviction in closed form (source: kdd/samples/src/Ch29/interesting.c, lines 142-160)

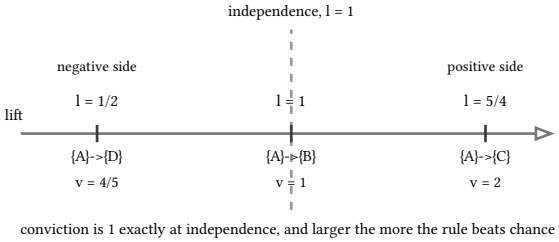


figure 117: lift against the independence line, conviction underneath

NOTE: three measures because each is blind somewhere

Confidence is directional and reads like a probability, but it cannot distinguish "the consequent always follows" from "the consequent almost always happens anyway". Lift fixes the baseline and is symmetric, so it cannot tell A->C from C->A. Conviction is directional again and unbounded above, but it is undefined at confidence 1, which is precisely where confidence is happiest. The fixture plants one rule on each side of 1 so no single number tells the whole story.

29.2. the chi-square verdict

Lift 5/4 on 12 baskets is a surplus of exactly one basket over the independent expectation, and the question is whether 12 baskets can carry that. The chi-square test of independence folds the 2x2 table of A against C: observed counts row-major, expected counts as row total times column total over N, then the statistic $\sum (o-e)^2/e$.

The dry run: the observed table is 5, 1, 3, 3 against expected 4, 2, 4, 2, printed as `cell o=5 e=4`, `cell o=1 e=2`, `cell o=3 e=4`, `cell o=3 e=2`, the four contributions being $1/4 + 1/2 + 1/4 + 1/2$, and the exact sum prints `chi2=3/2`. Then the family's only D2 row: the statistic is dyadic, exactly 1.5 as a double, the critical value is pinned as the double literal 3.8415 from the NIST table at df=1 and alpha=0.05, and the comparison is a plain < with margin 2.3415, printed `chi2 double=1.5 crit=3.8415 df=1 alpha=0.05 reject=false`. Independence is not rejected: lift 5/4 is not significant on this table. The machinery is book 14, ch 39 (knowledge discovery, hypothesis testing and p-values)'s subject at full length, and a real mining run tests thousands of rules at once, which is the multiplicity problem book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr) owns.

```
/* chi-square on the A->C 2x2: observed row-major n11,n12,n21,n22,
 * expected = row*col/N as exact fractions */
const int n11 = supp(1u | (1u << 2));
const int n12 = supp(1) - n11;
const int n21 = supp(1u << 2) - n11;
const int n22 = NTX - n11 - n12 - n21;
const int o[4] = {n11, n12, n21, n22};
const int r[2] = {n11 + n12, n21 + n22};
const int c[2] = {n11 + n21, n12 + n22};
const char *cell_pin[4] = {"cell o=5 e=4", "cell o=1 e=2", "cell o=3 e=4",
                           "cell o=3 e=2"};
int chi_n = 0, chi_d = 1;
for (int k = 0; k < 4; k++) {
  const int en = r[k / 2] * c[k % 2], ed = NTX;
  fstr(en, ed, s);
  sprintf(buf, "cell o=%d e=%s", o[k], s);
  printf("%s\n", buf);
  CHECK(strcmp(buf, cell_pin[k]) == 0, "ch29 chi2 cell row");
  /* term = (o - e)^2/e = (o*ed - en)^2 / (ed*en), summed exactly */
  int tn, td;
  reduce((o[k] * ed - en) * (o[k] * ed - en), ed * en, &tn, &td);
  chi_n = chi_n * td + tn * chi_d;
  chi_d *= td;
  reduce(chi_n, chi_d, &chi_n, &chi_d);
}
fstr(chi_n, chi_d, s);
sprintf(buf, "chi2=%s", s);
printf("%s\n", buf);
CHECK(strcmp(buf, "chi2=3/2") == 0, "ch29 chi-square exactly 3/2 (D0)");
```

listing 226: expected counts as exact fractions, the statistic summed exactly (source: kdd/samples/src/Ch29/interesting.c, lines 171-199)

```
/* D2: the only double comparison in the S6 family */
const double chi2d = (double)chi_n / (double)chi_d;
const double crit = 3.8415; /* NIST df=1, alpha=0.05 upper tail */
printf("chi2 double=%.17g crit=%.4f df=1 alpha=0.05 reject=%s\n", chi2d, crit,
       chi2d ≥ crit ? "true" : "false");
CHECK(chi2d == 1.5 && chi2d < crit,
      "ch29 chi2 1.5 < 3.8415: independence NOT rejected despite "
      "lift 5/4 (D2)");
```

listing 227: the D2 row, one double literal against one dyadic statistic (source: kdd/samples/src/Ch29/interesting.c, lines 201-208)

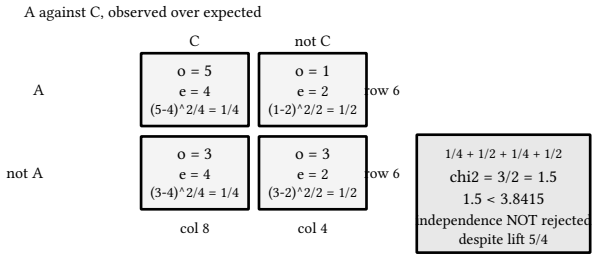


figure 118: the 2x2 table, four cells, four exact contributions, one verdict

PITFALL: the fixture does the arithmetic, not the statistics

All four expected counts are 2 or more, but the classic guidance asks for at least 5 per cell before trusting the chi-square approximation, and a 12-basket table cannot get there: the cells would need to average 3. The

> verdict line is a faithful computation on a fixture too small for the test's own assumptions, exactly the caveat the contract sheet banks. What the row does pin is the more transferable fact: a one-basket surplus is invisible at df=1, so lift 5/4 on 12 rows is noise, and only the test says so.

29.3. one hierarchy, two levels

Real taxonomies put items under parents, skim and whole milk under dairy, wheat and white under bread, and mining can run at either level. Fixture T is 10 baskets, each buying exactly one dairy item and one bread item, mined at minsup 3 uniformly at both levels.

The dry run: the concrete level prints `supp skim=6`, `supp wheat=4`, `supp white=6`, `supp whole=4`, then the six pairs in item order, `pair {skim,wheat}=3`, `pair {skim,white}=3`, `pair {skim,whole}=0`, `pair {wheat,white}=0`, `pair {wheat,whole}=1`, `pair {white,whole}=3`. The two zeros are structural, not statistical: a basket never carries two dairy items, so {skim,whole} and {wheat,white} cannot occur, and no amount of data moves them. Rolling up, the sample prints `rolled {dairy}=10 {bread}=10 {bread,dairy}=10 rollup_sum=10`, and the identity is exact: the four cross-category concrete pairs sum $3 + 3 + 3 + 1 = 10$, the support of {bread,dairy}, because every basket is counted once at the concrete level and once at the abstract level. This is the same move book 14, ch 11 (knowledge discovery, data cubes and olap) makes on cubes, aggregating along a dimension hierarchy, here with a count identity that closes over 10 baskets exactly.

```c
/* taxonomy: skim,whole -> dairy (bit 4); wheat,white -> bread (bit 5) */
#define DAIRY (1u << 4)
#define BREAD (1u << 5)
static unsigned rollup(unsigned m) {
  unsigned a = 0;
  if (m & ((1u << 0) | (1u << 3))) {
    a |= DAIRY;
  }
  if (m & ((1u << 1) | (1u << 2))) {
    a |= BREAD;
  }
  return a;
}
```

listing 228: the taxonomy as two bits and the roll-up as a mapping (source: kdd/samples/src/Ch29/multilev.c, lines 53-65)

```c
  /* roll-up: abstract supports and the 3+3+3+1 = 10 identity, the four
   * cross-category concrete pairs are (skim,white) (skim,wheat)
   * (whole,white) (whole,wheat) */
  const int rdairy = rsupp(DAIRY), rbread = rsupp(BREAD);
  const int rboth = rsupp(DAIRY | BREAD);
  const int crosssum =
      tsupp((1u << 0) | (1u << 2)) + tsupp((1u << 0) | (1u << 1)) +
      tsupp((1u << 3) | (1u << 2)) + tsupp((1u << 3) | (1u << 1));
  printf("rolled {dairy}=%d {bread}=%d {bread,dairy}=%d rollup_sum=%d\n",
         rdairy, rbread, rboth, crosssum);
  CHECK(rdairy == 10 && rbread == 10 && rboth == 10 && crosssum == 10 &&
            rboth == crosssum,
        "ch29 T roll-up identity: cross pairs 3+3+3+1 = 10 = "
        "supp({bread,dairy})");
```

listing 229: the roll-up identity checked as an equality of counts (source: kdd/samples/src/Ch29/multilev.c, lines 206-219)

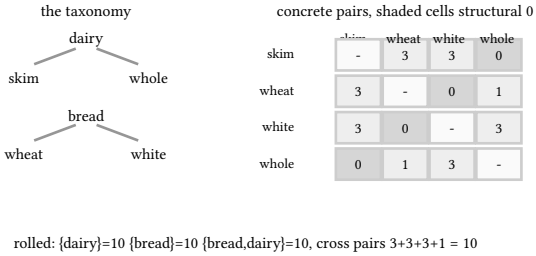


figure 119: concrete pairs below, the abstract pair above, zeros structural

29.4. quantitative rules by binning

Continuous attributes join association mining by discretization, the same move book 14, ch 3 (knowledge discovery, binning, smoothing, and discretization) made for histograms. Fixture B is 8 tuples of age and income with the sheet's edges: age into [20,30), [30,40), [40,50] years with the last bin closed, income into low below 40, mid from 40 to under 75, high from 75 up, in thousands per year.

The dry run: the eight bin rows print, `b1 age=22 inc=30 bin=age[20,30),low` through `b8 age=42 inc=60 bin=age[40,50],mid`, the young three all landing low, the middle two mid, and the old three splitting two high and one mid. The two pinned rules then read `age[40,50)→high s=1/4 c=2/3 l=8/3` and `age[20,30)→low s=3/8 c=1 l=8/3`. The first is b6 and b7 out of the three old tuples, 2/8, 2/3, lift $8 \cdot 2/(3 \cdot 2) = 8/3$, b8's mid income being the miss. The second is perfect, 3 of 3, confidence 1, and its lift is $8 \cdot 3/(3 \cdot 3)$, also 8/3: low income never occurs outside the young bin on this fixture, so the two rules tie. One spelling note the sample pins in a comment: the bin label is closed, `age[40,50]`, while the rule antecedent is spelled half-open, `age[40,50)`, matching the sheet's rule rows, and the chapter quotes both as printed.

```
static const char *agebin(int a) {
  if (a < 30) {
    return "age[20,30)";
  }
  if (a < 40) {
    return "age[30,40)";
  }
  return "age[40,50]";
}

static const char *incbin(int v) {
  if (v < 40) {
    return "low";
  }
  if (v < 75) {
    return "mid";
  }
  return "high";
}
```

listing 230: the bin edges, age closed on the last bin, income in three bands (source: kdd/samples/src/Ch29/multilev.c, lines 138-156)

```
/* binned rules: exact s/c/l fractions of N=8. the sheet spells the
 * rule antecedents age[40,50) and age[20,30) (half-open) even though
 * the last age BIN label is closed: age[40,50] */
const char *alab[2] = {"age[40,50]", "age[20,30)"};  /* bin labels */
const char *adisp[2] = {"age[40,50)", "age[20,30)"}; /* rule names */
const char *crule[2] = {"high", "low"};
const char *brule_pin[2] = {"age[40,50)->high s=1/4 c=2/3 l=8/3",
                            "age[20,30)->low s=3/8 c=1 l=8/3"};
for (int k = 0; k < 2; k++) {
  int na = 0, nc = 0, nj = 0;
  for (int t = 0; t < NB; t++) {
    const int a = strcmp(agebin(age[t]), alab[k]) == 0;
    const int c = strcmp(incbin(inc[t]), crule[k]) == 0;
    na += a;
    nc += c;
    nj += a && c;
  }
  char f1[16], f2[16], f3[16];
  fstr(nj, NB, f1);
  fstr(nj, na, f2);
  fstr(NB * nj, na * nc, f3);
  sprintf(buf, "%s->%s s=%s c=%s l=%s", adisp[k], crule[k], f1, f2, f3);
  printf("%s\n", buf);
  CHECK(strcmp(buf, brule_pin[k]) == 0, "ch29 B binned rule row");
}
```

listing 231: the two binned rules with exact fractions of N=8 (source: kdd/samples/src/Ch29/multilev.c, lines 241-265)

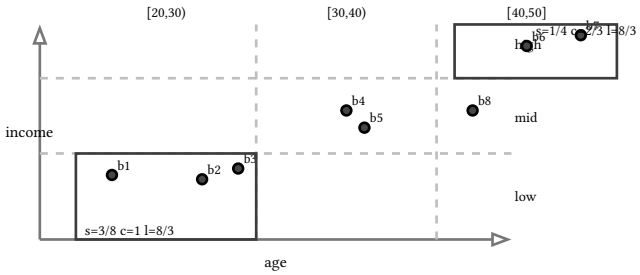


figure 120: age against income, the bin grid, and where the two rules live

sources: fixtures Q, T and B, the lift and conviction formulas, the chi-square cell convention and the 3/2 statistic, the roll-up identity 3+3+3+1 = 10, the bin edges and both binned rule rows are pinned by kdd-contract-s6.md and witnessed by playground/kdd-matrix/gen_s6.py, run 2026-09-22, exit 0. The critical value 3.8415 is from the NIST/SEMATECH e-Handbook of Statistical Methods, sec 1.3.6.7.4, https://www.itl.nist.gov/div898/handbook/eda/section3/eda3674.htm, accessed 2026-09-22, pinned as a double literal per the sheet's D2 ruling. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch29`, 15 + 23 checks in chapter 29 of the kdd suite.

30. sequential patterns

One sample carries the chapter: `sequential.c` runs the levelwise sequence census on five small sequences, 38 checks. The chapter makes 4 moves: the shape change from itemsets to sequences, where an element is a simultaneous itemset and order between elements is data, the containment test that greedy-matches each pattern element into a strictly later sequence element, the k=2 census whose twelve candidates show order asymmetry in raw counts, and the maximal pair that survives, one two-element shape and one one-element shape over the same items. The levelwise skeleton is book 14, ch 25 (knowledge discovery, levelwise apriori)'s, with the antimonotone law carrying over to subsequences, and the border notion is the one book 14, ch 27 (knowledge discovery, maximal itemsets and rule generation) drew on itemsets.

30.1. elements, order, and the containment test

A sequence is an ordered list of elements, each element a set of items bought together in one transaction: S1 is a then the pair b,c, S2 is the pair a,b then c, and those are different sequences even though all their items are the same letters. A pattern is contained in a sequence when each pattern element, in order, sits inside a strictly later sequence element than the one before it. One itemset chapter threw order away entirely, two baskets were the same basket whenever their items were, and this chapter reinstates it.

The dry run: the fixture prints as `S1:<{a}{b,c}>`, `S2:<{a,b}{c}>`, `S3:<{a}{b}{d}>`, `S4:<{b}{c}>`, `S5:<{a,c}{b,c}>`, and the 1-sequence census counts `1seq <{a}>:4`, `1seq <{b}>:5`, `1seq <{c}>:4`, `1seq <{d}>:1`, d appearing only in S3 and pruned at 1 below minsup 2. The canonical trap is already visible: <{a}{b}> asks for a basket with a and a strictly later basket with b, so it lives in S1, S3 and S5 but not in S2, whose a and b share one element. Containment is decided by the greedy leftmost scan, and the greedy choice is safe by a one-line exchange: matching a pattern element at the earliest sequence element that fits can only leave more room, never less, for the pattern elements after it.

```
static int contains(int si, const unsigned *pe, int pn) {
  int i = 0;
  for (int k = 0; k < sn[si]; k++) {
    if (i < pn && (pe[i] & ~sdb[si][k]) == 0) {
      i++;
    }
  }
  return i == pn;
}

static int ssupp(const unsigned *pe, int pn) {
  int c = 0;
  for (int s = 0; s < NS; s++) {
    if (contains(s, pe, pn)) {
      c++;
    }
  }
  return c;
}
```

listing 232: greedy leftmost containment, one cursor over the sequence elements (source: kdd/samples/src/Ch30/sequential.c, lines 73-91)

```
/* k=1 census over a,b,c,d; d pruned at support 1 < 2 */
const char *one_pin[NIT] = {"1seq <{a}>:4", "1seq <{b}>:5", "1seq <{c}>:4",
                            "1seq <{d}>:1"};
for (int b = 0; b < NIT; b++) {
  const unsigned e[1] = {1u << b};
  seq_str(e, 1, s);
  sprintf(buf, "1seq %s:%d", s, ssupp(e, 1));
  printf("%s\n", buf);
  char name[48];
  snprintf(name, sizeof name, "ch30 %s (d pruned at 1 < 2)", one_pin[b]);
  CHECK(strcmp(buf, one_pin[b]) == 0, name);
}
```

listing 233: the 1-sequence census, d counted at 1 and pruned (source: kdd/samples/src/Ch30/sequential.c, lines 136-147)

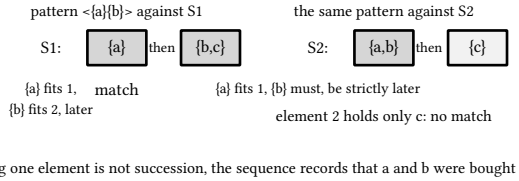


figure 121: the same two items, one order matches, the shared element does not

NOTE: order is information the itemset chapters deleted

As a basket, S2′s first element is just the itemset {a,b}, and book 14, ch 25 (knowledge discovery, levelwise apriori) would count it exactly like any other occurrence of a and b. As a sequence, {a,b} then c also says a and b happened before c, and it says nothing about a before b, because inside one element there is no order. The three chapters of the association part mined unordered sets, and everything this chapter adds comes from putting a clock between the braces.

30.2. the k=2 census, twelve candidates

Two-item shapes over the surviving letters a, b, c: nine two-element sequences <{x}{y}> in row-major order, then the three one-element pairs, exactly the sheet's enumeration order, each counted by containment over the five sequences.

The dry run: `2seq <{a}{a}>:0`, `2seq <{a}{b}>:3`, `2seq <{a}{c}>:3`, `2seq <{b}{a}>:0`, `2seq <{b}{b}>:0`, `2seq <{b}{c}>:2`, `2seq <{c}{a}>:0`, `2seq <{c}{b}>:1`, `2seq <{c}{c}>:1`, `2seq <{a,b}>:1`, `2seq <{a,c}>:1`, `2seq <{b,c}>:2`. The matrix is visibly asymmetric: a-then-b happens in 3 sequences and b-then-a in none, and c-then-b in exactly one, S5, where c rides the first element and b the second. The one-element pairs count co-occurrence inside a single transaction, {a,b} only in S2, {a,c} only in S5, {b,c} in S1 and S5. Four of the twelve clear minsup 2, <{a}{b}>, <{a}{c}>, <{b}{c}> and <{b,c}>, joining the three singletons toward the frequent listing.

```
/* k=2 candidates: 9 two-element row-major, then the 3 one-element
 * pairs, exactly the sheet's enumeration order */
const char *two_pin[12] = {
    "2seq <{a}{a}>:0", "2seq <{a}{b}>:3", "2seq <{a}{c}>:3",
    "2seq <{b}{a}>:0", "2seq <{b}{b}>:0", "2seq <{b}{c}>:2",
    "2seq <{c}{a}>:0", "2seq <{c}{b}>:1", "2seq <{c}{c}>:1",
    "2seq <{a,b}>:1",  "2seq <{a,c}>:1",  "2seq <{b,c}>:2"};
int tp = 0;
for (int x = 0; x < 3; x++) {
  for (int y = 0; y < 3; y++) {
    const unsigned e[2] = {etypes[x], etypes[y]};
    seq_str(e, 2, s);
    sprintf(buf, "2seq %s:%d", s, ssupp(e, 2));
    printf("%s\n", buf);
    CHECK(strcmp(buf, two_pin[tp]) == 0, "ch30 k=2 candidate count");
    tp++;
  }
}
for (int t = 3; t < 6; t++) { /* pairs {a,b},{a,c},{b,c} */
  const unsigned e[1] = {etypes[t]};
  seq_str(e, 1, s);
  sprintf(buf, "2seq %s:%d", s, ssupp(e, 1));
  printf("%s\n", buf);
  CHECK(strcmp(buf, two_pin[tp]) == 0, "ch30 k=2 one-element pair count");
  tp++;
}
```

listing 234: nine two-element candidates row-major, then the three one-element pairs (source: kdd/samples/src/Ch30/sequential.c, lines 149-174)

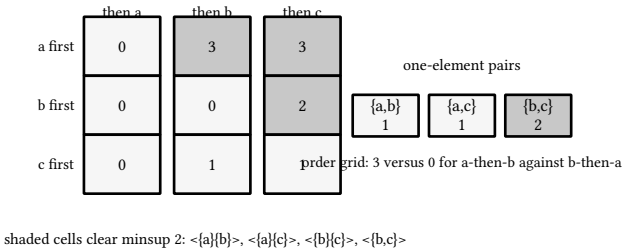


figure 122: the 3x3 order grid plus the pairs, frequent cells shaded

30.3. eight frequent, two maximal

The enumeration tries every shape over a, b, c with one, two or three elements, keeps what clears 2, and nothing with three elements survives. Eight sequences do, and the maximal ones are those no other frequent sequence contains.

The dry run: the listing prints `freq <{a}>:4`, `freq <{b}>:5`, `freq <{c}>:4`, `freq <{b,c}>:2`, `freq <{a}{b}>:3`, `freq <{a}{c}>:3`, `freq <{a}{b,c}>:2`, `freq <{b}{c}>:2`, in element-count order with one-element shapes first, <{b,c}> sitting among them because it is one element. The maximal pair prints as `maximal: <{a}{b,c}>:2;<{b}{c}>:2`. The contrast the check pins by name is the part's cleanest sentence: "ch30 <{b,c}> vs <{b}{c}>: same items, different shape, same count 2". The one-element <{b,c}> embeds in <{a}{b,c}>, b and c sharing its second element, so it cannot be maximal. The two-element <{b}{c}> does not embed anywhere, no frequent sequence buys b and then c in separate later transactions except S2 and S4, and it stands maximal at the same count 2. The antimonotone law holds as on itemsets, every subsequence of a frequent sequence is frequent, since deleting elements or items from a witness leaves a witness, which is what makes the levelwise skeleton legitimate here too.

```
static void enum_seq(int k, int pos, unsigned *e) {
  if (pos == k) {
    const int v = ssupp(e, k);
    if (v >= 2 && nfe < MAXF) {
      memcpy(fe[nfe], e, (size_t)k * sizeof(unsigned));
      fen[nfe] = k;
      fec[nfe] = v;
      nfe++;
    }
    return;
  }
  for (int t = 0; t < 7; t++) {
    e[pos] = etypes[t];
    enum_seq(k, pos + 1, e);
  }
}
```

listing 235: shape enumeration by element count, keeping what clears minsup (source: kdd/samples/src/Ch30/sequential.c, lines 106-121)

```
  char *p = buf;
  p += sprintf(p, "maximal: ");
  int first = 1;
  for (int i = 0; i < nfe; i++) {
    int ismax = 1;
    for (int j = 0; j < nfe && ismax; j++) {
      if (j == i) {
        continue;
      }
      /* does frequent j contain frequent i? */
      int w = 0;
      for (int k = 0; k < fen[j]; k++) {
        if (w < fen[i] && (fe[i][w] & ~fe[j][k]) == 0) {
          w++;
        }
      }
      if (w == fen[i]) {
        ismax = 0;
      }
    }
    if (ismax) {
      if (!first) {
        *p++ = ';';
      }
      first = 0;
      seq_str(fe[i], fen[i], s);
      p += sprintf(p, "%s:%d", s, fec[i]);
    }
  }
  printf("%s\n", buf);
  CHECK(strcmp(buf, "maximal: <{a}{b,c}>:2;<{b}{c}>:2") == 0,
        "ch30 maximal sequences <{a}{b,c}> and <{b}{c}>");
```

listing 236: maximality by mutual containment over the frequent listing (source: kdd/samples/src/Ch30/sequential.c, lines 195-226)

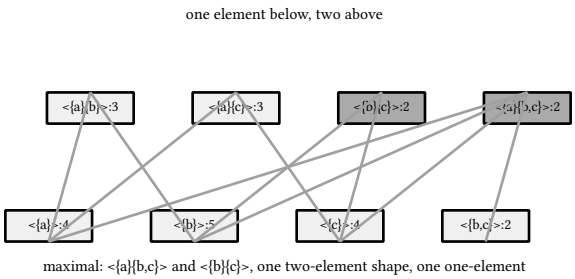


figure 123: the eight frequent sequences, containment edges up, maximal dark

30.4. the match trace, greedily

The sample closes by tracing <{a}{b,c}>, the k=3 winner with three items in two elements, through all five sequences, printing the witness positions or the refusal.

The dry run: `S1:match 1,2`, `S2:no`, `S3:no`, `S4:no`, `S5:match 1,2`, then `supp=2`. The two matches are clean: S1's first element is exactly {a} and its second is exactly {b,c}, S5 hides a inside {a,c} and takes {b,c} whole as the second element, both times the greedy cursor landing on elements 1 and 2. The three refusals each fail for a different reason worth reading off: S2 spends its first element on {a,b}, which covers the pattern's {a}, but then no strictly later element contains both b and c, its second element is only {c}, and the shared-element trap from the first facet closes the door. S3 has a and then b but no c at all, d being pruned and useless as a witness. S4 never buys a. Two witnesses out of five, support 2, and the maximal listing keeps <{a}{b,c}> as the deepest shape the fixture supports.

```
/* greedy match trace for <{a}{b,c}> */
const unsigned pat[2] = {1u << 0, (1u << 1) | (1u << 2)};
const char *trace_pin[NS] = {"S1:match 1,2", "S2:no", "S3:no", "S4:no",
                             "S5:match 1,2"};
for (int i = 0; i < NS; i++) {
  if (contains(i, pat, 2)) {
    int pos[2], w = 0;
    for (int k = 0; k < sn[i] && w < 2; k++) {
      if ((pat[w] & ~sdb[i][k]) == 0) {
        pos[w++] = k + 1;
      }
    }
    sprintf(buf, "S%d:match %d,%d", i + 1, pos[0], pos[1]);
  } else {
    sprintf(buf, "S%d:no", i + 1);
  }
  printf("%s\n", buf);
  CHECK(strcmp(buf, trace_pin[i]) == 0, "ch30 trace line for <{a}{b,c}>");
}
printf("supp=%d\n", ssupp(pat, 2));
CHECK(ssupp(pat, 2) == 2, "ch30 <{a}{b,c}> support 2 (S1 and S5)");
```

listing 237: the trace loop, witness positions for matches, the plain refusal otherwise (source: kdd/samples/src/Ch30/sequential.c, lines 252-272)

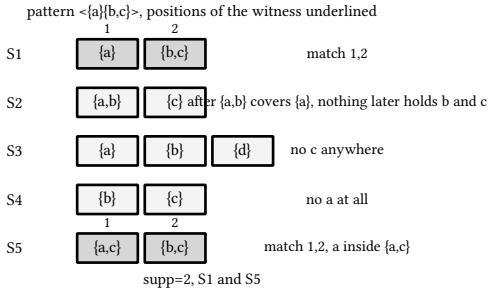


figure 124: five sequences, two witnesses, three different reasons to refuse

sources: fixture S, the five sequence strings, the containment law and its greedy spelling, the k=2 enumeration order, the eight-row frequent listing, the maximal pair and the match trace are pinned by kdd-contract-s6.md and witnessed by playground/kdd-matrix/gen_s6.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch30`, 38 checks in chapter 30 of the kdd suite.

31. k-means

Two samples carry the chapter: `kmeans.c` runs Lloyd's iteration on a planted 12-point fixture, a poisoned initialization of the same size, and a 3-point tie micro-fixture, 17 checks, and `elbow.c` prices the optimal k-clustering by exhaustive set-partition enumeration and reads the elbow off the gains, 13 checks. The chapter makes 4 moves: the two-step pass that alternates assignment and means, the exhaustive optimum the elbow heuristic is guessing at, the local minimum a bad init buys, and the tie rules that make every run reproducible byte for byte in Go. Distances are the squared euclidean of book 14, ch 5 (knowledge discovery, distance metrics), the elbow returns in book 14, ch 34 (knowledge discovery, cluster validation) as one of two metrics that disagree, and the tie rules are replayed by the bridge of book 14, ch 47 (knowledge discovery, the bridge: c to go).

31.1. two moves, one pass

Lloyd's algorithm alternates two moves until nothing changes: assign every point to the nearest centroid by squared euclidean distance, then move every centroid to the mean of the points it holds. Each pass never increases the objective, so the run settles into whatever basin the seeds select, and nothing about the alternation promises the global optimum. The fixture keeps the arithmetic checkable: F12 is 12 integer points in three axis-aligned rectangles, planted cluster A the 4x4 square at the origin, B the 4x4 square at (26,2), C the 4x8 rectangle at (14,20), and the first three input points are one per cluster, so the seeded init starts inside the answer.

The dry run: the seeds print as `seeds: (0,0),(24,0),(12,16)` and pass 1 assigns, printing `iter0 cents=(0,0),(24,0),(12,16) assign=012000111222`
`sse=288`. The vector reads point by point, p1 through p12, and lands A on cluster 0 (p1, p4, p5, p6), B on cluster 1 (p2, p7, p8, p9), C on cluster 2 (p3, p10, p11, p12). The 288 is the SSE against the seed vertices: A and B contribute 64 each, corners at 0, 16, 16, 32 squared distance, and C contributes 160 because its seed p3 is one corner of a 4x8 rectangle. The update move takes means and every centroid lands on an integer: A collapses to (2,2), B to (26,2), C to (14,20). Pass 2 reassigns, nothing moves, and the run closes with `iter1`
`cents=(2,2),(26,2),(14,20) assign=012000111222 sse=144` then `final`
`iters=2 assign=012000111222`. The iters count includes the confirming pass: pass 1 moved the assignment, pass 2 changed nothing, so 2. The 144 decomposes as 32 + 32 + 80, the 80 being C's own spread around its mean, four copies of $2^2 + 4^2$, and that 80 is the number the k=3 row of the elbow table below has to reproduce.

```
static void lloyd(const int (*pts)[2], int n, int k, Run *r) {
  Frac cents[3][2];
  int nc = 0;
  for (int i = 0; i < n && nc < k; i++) { /* first k distinct points */
    int dup = 0;
    for (int c = 0; c < nc; c++) {
      if (fcmp(cents[c][0], fr(pts[i][0], 1)) == 0 &&
          fcmp(cents[c][1], fr(pts[i][1], 1)) == 0) {
        dup = 1;
      }
    }
    if (!dup) {
      cents[nc][0] = fr(pts[i][0], 1);
      cents[nc][1] = fr(pts[i][1], 1);
      nc++;
    }
  }
  int assign[12];
  for (int i = 0; i < n; i++) {
    assign[i] = -1;
  }
```

```
  r->ntrace = 0;
  for (int pass = 1; pass <= 100; pass++) {
    int moved = 0;
    for (int i = 0; i < n; i++) { /* ties to the lowest cluster index */
      int best = 0;
      Frac bd = d2pt(pts[i], cents[0]);
      for (int c = 1; c < k; c++) {
        const Frac dd = d2pt(pts[i], cents[c]);
        if (fcmp(dd, bd) < 0) {
          best = c;
          bd = dd;
        }
      }
      if (assign[i] != best) {
        assign[i] = best;
        moved = 1;
      }
    }
    if (!moved) { /* confirming pass counted in iters */
      r->iters = pass;
      memcpy(r->cents, cents, sizeof cents);
      for (int i = 0; i < n; i++) {
        r->assign[i] = (char)('0' + assign[i]);
      }
      r->assign[n] = 0;
      return;
    }
    long long sx[3] = {0, 0, 0}, sy[3] = {0, 0, 0};
    int cnt[3] = {0, 0, 0};
    for (int i = 0; i < n; i++) {
      cnt[assign[i]]++;
      sx[assign[i]] += pts[i][0];
      sy[assign[i]] += pts[i][1];
    }
    for (int c = 0; c < k; c++) { /* emptied cluster keeps its centroid */
      if (cnt[c] != 0) {
        cents[c][0] = fr(sx[c], cnt[c]);
        cents[c][1] = fr(sy[c], cnt[c]);
      }
    }
    if (r->ntrace < 4) {
```

listing 238: lloyd with the pinned tie rules, exact fractions throughout (source: kdd/samples/src/Ch31/kmeans.c, lines 129-190)

```
lloyd(F12, 12, 3, &r);
sprintf(row, "seeds: (%d,%d),(%d,%d),(%d,%d)", F12[0][0], F12[0][1],
        F12[1][0], F12[1][1], F12[2][0], F12[2][1]);
printf("%s\n", row);
CHECK(strcmp(row, "seeds: (0,0),(24,0),(12,16)") == 0, "ch31 F12 seeds row");

{ /* iter0 = seeds with the pass-1 assignment, sse against the seeds */
  Frac seeds[3][2] = {
      {fr(0, 1), fr(0, 1)}, {fr(24, 1), fr(0, 1)}, {fr(12, 1), fr(16, 1)}};
  int a0[12];
  for (int i = 0; i < 12; i++) {
    int best = 0;
    Frac bd = d2pt(F12[i], seeds[0]);
```

```
      for (int c = 1; c < 3; c++) {
        const Frac dd = d2pt(F12[i], seeds[c]);
        if (fcmp(dd, bd) < 0) {
          best = c;
          bd = dd;
        }
      }
      a0[i] = best;
    }
    char vec[13];
    for (int i = 0; i < 12; i++) {
      vec[i] = (char)('0' + a0[i]);
    }
    vec[12] = 0;
    fstr(sse_at(F12, 12, a0, seeds), b);
    sprintf(row, "iter0 cents=(0,0),(24,0),(12,16) assign=%s sse=%s", vec, b);
    printf("%s\n", row);
    CHECK(strcmp(row, "iter0 cents=(0,0),(24,0),(12,16) assign=012000111222 "
                      "sse=288") == 0,
          "ch31 F12 iter0 row");
  }

  cents_row(r.tr[0].cents, 3, a);
  fstr(r.tr[0].sse, b);
  sprintf(row, "iter%d cents=%s assign=%s sse=%s", r.tr[0].pass, a,
          r.tr[0].assign, b);
  printf("%s\n", row);
  CHECK(strcmp(row, "iter1 cents=(2,2),(26,2),(14,20) assign=012000111222 "
                    "sse=144") == 0,
        "ch31 F12 iter1 row");
  CHECK(fcmp(r.tr[0].cents[0][0], fr(2, 1)) == 0 &&
            fcmp(r.tr[0].cents[1][1], fr(2, 1)) == 0 &&
            fcmp(r.tr[0].cents[2][0], fr(14, 1)) == 0,
        "ch31 F12 iter1 centroids dyadic integers (2,2),(26,2),(14,20)");
  sprintf(row, "final iters=%d assign=%s", r.iters, r.assign);
  printf("%s\n", row);
  CHECK(strcmp(row, "final iters=2 assign=012000111222") == 0,
        "ch31 F12 final row (iters counts the confirming pass)");

  /* the exhaustive-partition elbow table (optimal SSE per k, gains,
   * second differences, elbow=3) lives in the sibling elbow.c; the D4
   * agreement is that Lloyd's iter1 sse here equals that table's k=3
   * optimum, pinned at 144 by the contract block */
  CHECK(fcmp(r.tr[0].sse, fr(144, 1)) == 0,
        "ch31 F12 lloyd from the pinned init attains the k=3 optimum 144 "
        "(elbow.c proves it exhaustive)");
```

listing 239: the F12 run: seeds, both traced passes, and the k=3 agreement (source: kdd/samples/src/Ch31/kmeans.c, lines 232-291)

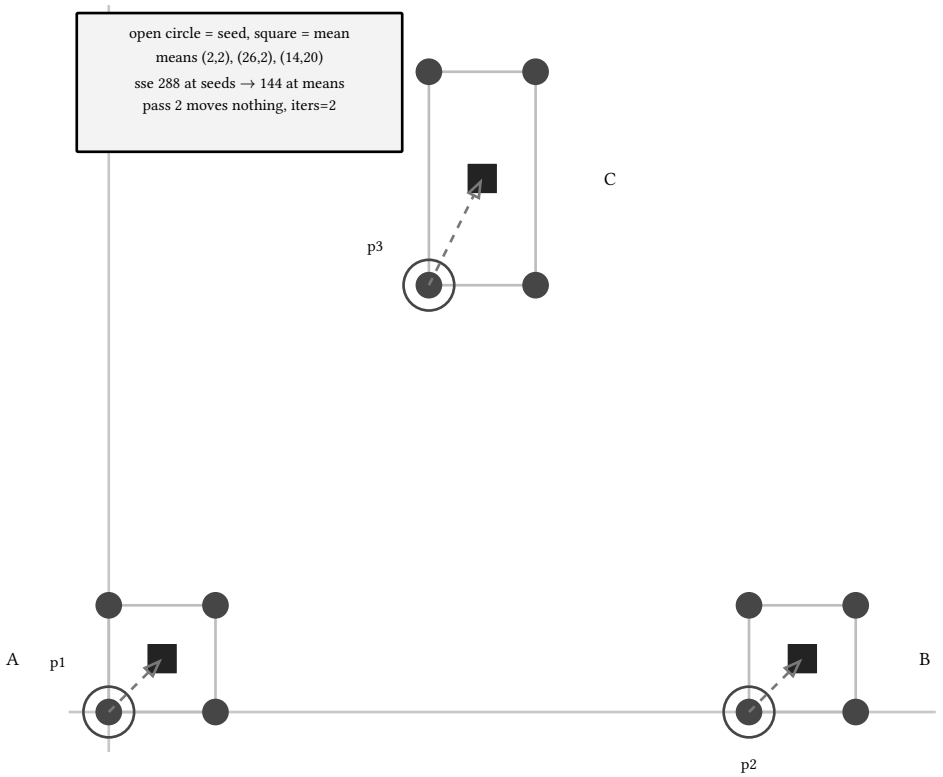


figure 125: assignment then means: three seeds, one update, integer centroids

31.2. the elbow against the exhaustive optimum

The elbow heuristic reads k off a table of SSE values, and the table worth reading is the optimal one: for each k the minimum SSE over all k-partitions, not whatever Lloyd happens to reach. `elbow.c` computes it exactly by exhaustive set-partition enumeration, on the order of 700k states across $k = 1..4$ with no heap, scoring each candidate through the identity $\text{SSE} = \text{totsq} - \sum_c \left(s_{x(c)}^2 + s_{y(c)}^2\right)/n_c$ with totsq 5280, so every comparison is an exact integer cross-multiply. The closed form anchors the first row, $3552 - 168^2/12 + 1728 - 96^2/12 = 2160$, the total scatter 5280 splitting as $3552 + 1728$ per axis with sums $(168, 96)$.

The dry run: the table prints `k=1 sse=2160`, `k=2 sse=1080`, `k=3 sse=144`, `k=4 sse=80`, then `gains 1080,936,64`, `d2 144,872`, and `elbow=3 kneedle_2b_gt_a_plus_c=true`. The pinned rule is the largest second difference over k=3,4: $1080 - 936 = 144$ against $936 - 64 = 872$, so the elbow is $2 + 1 = 3$, and the kneedle-style crosscheck agrees, $2 \cdot 936 = 1872 > 1080 + 64$. Two facts sit behind the table. The k=2 optimum is a tie, {B} against $A \cup C$ scoring the same 1080 as {A} against $B \cup C$: $A \cup C$ has mean (8,11) and SSE 1048, plus B's 32. The enumeration under canonical first-use ordering records the {B} side first, and the sample's check pins the tie explicitly rather than letting one witness pose as the optimum. And k=4 spends its 64 to split C into the pairs {p3,p10} and {p11,p12}, C's 80 dropping to 16. The agreement row closes the loop with the previous facet: the exhaustive k=3 optimum is 144, the same number Lloyd's iter1 printed from the planted init, and both samples assert it, "ch31 F12 lloyd from the pinned init attains the k=3 optimum 144 (elbow.c proves it exhaustive)" in `kmeans.c` and "ch31 D4 agreement: exhaustive k=3 optimum 144 equals Lloyd's pinned iter1 sse in kmeans.c" in `elbow.c`.

```
/* exhaustive set-partition enumeration with incremental cluster sums:
 * sse = totsq - sum_c (sx^2+sy^2)/cnt, den = product of sizes (≤ 81),
 * compared exactly. the first strict maximum is kept, which under
 * canonical first-use ordering records {B}|A u C for the tied k=2. */
static long long e_bestnum[5], e_bestden[5];
static int e_bestlbl[5][12];
```

```
static int e_lbl[12];

static void enum_rec(int i, int nc, int k, long long *sx, long long *sy,
                     int *cnt, long long totsq) {
  if (i == 12) {
    if (nc == k) {
      long long den = 1, num = 0;
      for (int c = 0; c < k; c++) {
        den *= cnt[c];
      }
      for (int c = 0; c < k; c++) {
        num += (sx[c] * sx[c] + sy[c] * sy[c]) * (den / cnt[c]);
      }
      if (e_bestden[k] == 0 || num * e_bestden[k] > e_bestnum[k] * den) {
        e_bestden[k] = den;
        e_bestnum[k] = num;
        memcpy(e_bestlbl[k], e_lbl, sizeof e_lbl);
      }
    }
    return;
  }
  for (int c = 0; c < nc; c++) {
    e_lbl[i] = c;
    sx[c] += F12[i][0];
    sy[c] += F12[i][1];
    cnt[c]++;
    enum_rec(i + 1, nc, k, sx, sy, cnt, totsq);
    sx[c] -= F12[i][0];
    sy[c] -= F12[i][1];
    cnt[c]--;
  }
  if (nc < k) {
    e_lbl[i] = nc;
    sx[nc] += F12[i][0];
    sy[nc] += F12[i][1];
    cnt[nc]++;
    enum_rec(i + 1, nc + 1, k, sx, sy, cnt, totsq);
    sx[nc] -= F12[i][0];
    sy[nc] -= F12[i][1];
    cnt[nc]--;
  }
}

static Frac opt_sse(int k, long long totsq) {
  long long sx[4] = {0, 0, 0, 0}, sy[4] = {0, 0, 0, 0};
  int cnt[4] = {0, 0, 0, 0};
  e_bestden[k] = 0;
  e_bestnum[k] = 0;
  enum_rec(0, 0, k, sx, sy, cnt, totsq);
  return fr(totsq * e_bestden[k] - e_bestnum[k], e_bestden[k]);
}
```

listing 240: exhaustive set-partition enumeration with incremental sums (source: kdd/samples/src/Ch31/elbow.c, lines 83-139)

```
  Frac sses[5];
  const char *pinrow[4] = {"k=1 sse=2160", "k=2 sse=1080", "k=3 sse=144",
                           "k=4 sse=80"};
  for (int k = 1; k ≤ 4; k++) {
    sses[k] = opt_sse(k, totsq);
    fstr(sses[k], fa);
    sprintf(row, "k=%d sse=%s", k, fa);
    printf("%s\n", row);
    CHECK(strcmp(row, pinrow[k - 1]) == 0, "ch31 elbow table row");
  }

  Frac g[3];
  for (int i = 0; i < 3; i++) {
    g[i] = fsub(sses[i + 1], sses[i + 2]);
  }
  fstr(g[0], fa);
  fstr(g[1], fb);
  char fc[32];
  fstr(g[2], fc);
  sprintf(row, "gains %s,%s,%s", fa, fb, fc);
  printf("%s\n", row);
  CHECK(strcmp(row, "gains 1080,936,64") == 0, "ch31 gains row");
  const Frac d2a = fsub(g[0], g[1]), d2b = fsub(g[1], g[2]);
  fstr(d2a, fa);
  fstr(d2b, fb);
  sprintf(row, "d2 %s,%s", fa, fb);
  printf("%s\n", row);
  CHECK(strcmp(row, "d2 144,872") == 0, "ch31 d2 row");
  const int elbow = 2 + (fcmp(d2a, d2b) ≥ 0 ? 0 : 1);
  const int kn = fcmp(fmul(fr(2, 1), g[1]), fadd(g[0], g[2])) > 0;
  sprintf(row, "elbow=%d kneedle_2b_gt_a_plus_c=%s", elbow,
          kn ? "true" : "false");
  printf("%s\n", row);
  CHECK(strcmp(row, "elbow=3 kneedle_2b_gt_a_plus_c=true") == 0,
        "ch31 elbow=3: largest second difference, kneedle crosscheck holds");
```

listing 241: the table, the gains, the second differences, the elbow rule (source: kdd/samples/src/Ch31/elbow.c, lines 155-189)

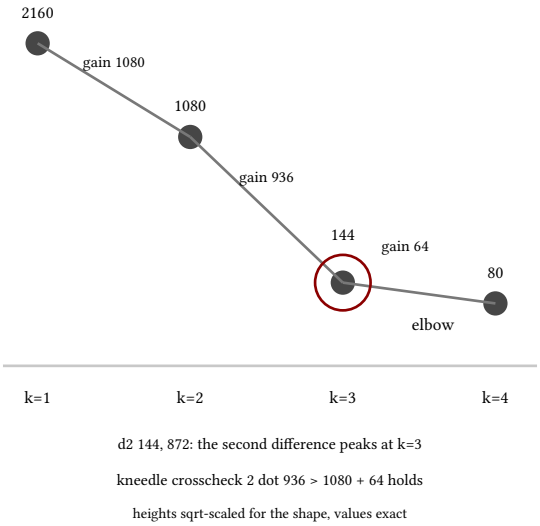


figure 126: the optimal sse curve and the elbow at the largest second difference

NOTE: exhaustive is a fixture luxury, chain dp is the 1d luxury

> Enumerating $S(12, k)$ is only thinkable because the fixture is 12 points; one more rectangle and the table needs a different algorithm. When the data is 1D and sorted, the optimal k-partition is contiguous and a chain DP prices exact k-means in quadratic time, which is exactly the trade book 11, ch 10 (the icpc world finals, icpc world finals 2017) problem F makes: exact k-means is priced out in favor of the chain DP because the points sit on a line. In 2D the same enumeration is NP-hard in general, and the elbow table is the heuristic standing in for the optimum the way Lloyd stands in for the argmin partition.

31.3. bad init buys a split local minimum

The second fixture plants the failure. F12b is again 12 points in three rectangles, but the first two sit close: planted A = {p1,p2,p4,p5} the 6x6 square at the origin, B = {p3,p6,p7,p8} the square at (27,3), C = {p9,p10,p11,p12} at (33,15), and the input order puts p1, p2, p3 first, so the seeds are two points of A plus one of B: `seeds: (0,0),(6,0),(24,0)`.

The dry run: pass 1 prints `iter0 cents=(0,0),(6,0),(24,0) assign=012012222222 sse=1512`. Cluster 0 takes {p1,p4}, cluster 1 takes {p2,p5}, and cluster 2 absorbs the other eight points, all of B and all of C. The update splits A in half, means (0,3) and (6,3), and drags cluster 2′s mean to (30,9): `iter1 cents=(0,3),(6,3),(30,9) assign=012012222222 sse=540`. The 540 decomposes as 18 + 18 + 504: each half-A cluster holds two points at squared distance 9 from its mean, and the 8-point cluster pays for lumping two rectangles together. The planted partition, recomputed in the sample from its integer means (3,3), (27,3), (33,15), scores `planted_sse=216 local_min_worse_by=324`. Lloyd is parked: the pass is a descent, each move only lowers or holds the SSE, so no further pass escapes a local minimum, and the run sits 324 above the answer forever.

```
  cents_row(r.tr[0].cents, 3, a);
  fstr(r.tr[0].sse, b);
  sprintf(row, "iter%d cents=%s assign=%s sse=%s", r.tr[0].pass, a,
          r.tr[0].assign, b);
  printf("%s\n", row);
  CHECK(strcmp(row, "iter1 cents=(0,3),(6,3),(30,9) assign=012012222222 "
                    "sse=540") == 0,
        "ch31 F12b iter1 row: split-A local minimum");
  {
    /* planted partition {p1,p2,p4,p5},{p3,p6,p7,p8},{p9..p12} */
    static const int plant[12] = {0, 0, 1, 0, 0, 1, 1, 1, 2, 2, 2, 2};
    long long sx[3] = {0, 0, 0}, sy[3] = {0, 0, 0};
    int cnt[3] = {0, 0, 0};
    for (int i = 0; i < 12; i++) {
      cnt[plant[i]]++;
      sx[plant[i]] += F12B[i][0];
      sy[plant[i]] += F12B[i][1];
    }
    long long psse = 0; /* planted means (3,3),(27,3),(33,15), all integers */
    for (int i = 0; i < 12; i++) {
      const long long mx = sx[plant[i]] / cnt[plant[i]];
      const long long my = sy[plant[i]] / cnt[plant[i]];
      psse += (F12B[i][0] - mx) * (F12B[i][0] - mx) +
              (F12B[i][1] - my) * (F12B[i][1] - my);
    }
    const long long worse = r.tr[0].sse.n - psse;
    sprintf(row, "planted_sse=%lld local_min_worse_by=%lld", psse, worse);
    printf("%s\n", row);
    CHECK(strcmp(row, "planted_sse=216 local_min_worse_by=324") == 0,
          "ch31 F12b planted 216 vs local minimum 540, worse by 324");
    CHECK(fcmp(r.tr[0].sse, fr(540, 1)) == 0 && psse == 216 &&
              fcmp(r.tr[0].sse, fr(psse, 1)) > 0,
          "ch31 F12b lloyd stalls above the planted optimum");
  }
```

listing 242: F12b: the split-A local minimum row and the planted 216 against 540 (source: kdd/samples/src/Ch31/kmeans.c, lines 333-366)

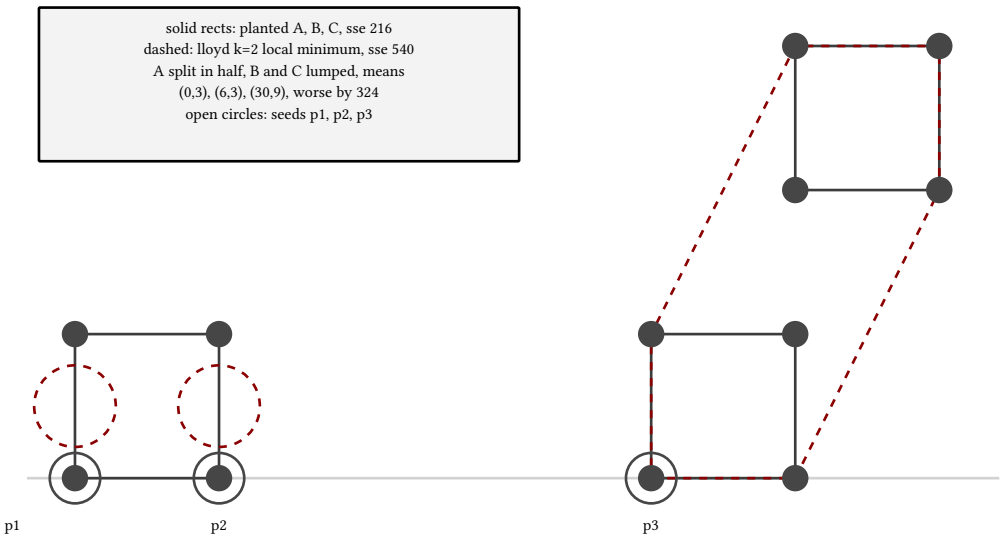


figure 127: the same 12 shapes, two answers: planted rectangles against lloyd's hulls

PITFALL: a descent cannot regret its seeds

> The 540-versus-216 gap is not bad luck in the arithmetic, it is the algorithm's contract: each Lloyd move is a conditional improvement, so the run can only ratchet down inside the basin the seeds picked. The standard remedies attack the init, not the pass: restart from many different seeds and keep the best SSE, or seed preferentially far apart, the k-means++ sketch of Arthur and Vassilvitskii 2007. Nothing restarts the pass itself out of a local minimum, which is why the fixtures pin both the basin and its price: 324 is what this chapter's seeds cost.

31.4. ties, exact halves, and the go bridge

Every tie in the assignment step is a fork in the run, and the last fixture pins the fork. The rules are the committed Go bridge's, verbatim: init from the first k distinct points in input order, assignment ties go to the lowest cluster index, a cluster left empty keeps its previous centroid (unreachable from point-seeded inits on these fixtures, but it is the rule), stop when a pass changes nothing with iters counting the confirming pass, and maxIter 100 never binds here.

The dry run: the TIE fixture is three collinear points p1=(0,0), p2=(2,0), p3=(1,0), the first two distinct points seed, and p3 sits exactly 1 from each seed. The sample prints `tie p3 d0=1 d1=1 joins=0`, cluster 0 winning the tie. The update makes cluster 0's mean (1/2,0) and the run prints `iter1 cents=(1/2,0),(2,0) assign=010 sse=1/2` then `final iters=2 assign=010`. The 1/2 is exact, two points at squared distance 1/4 from their mean, held as an int64 fraction: this is the dyadic case where a double would coincidentally be exact, and the fraction makes the byte comparison the law rather than an accident. The fixture is also the D4 witness: the bridge's own test `TestLloydTieGoesToLowestIndex` replays these exact bytes in Go, so the C pin and the Go pin are one fixture, and book 14, ch 47 (knowledge discovery, the bridge: c to go) walks the full parity chain this chapter's rows ride on.

```
 * tie rules, verbatim from the committed go bridge
 * books/kdd/capstone/internal/kddcore/kmeans.go (its test
 * TestLloydTieGoesToLowestIndex is the same TIE fixture): init = first k
 * distinct points in input order; assignment ties go to the lowest
 * cluster index; an emptied cluster keeps its previous centroid; iters
 * counts assignment passes including the confirming pass (maxIter=100
 * never binds here). the chapter states Lloyd from the pinned init
 * attains the k=3 optimum (checked against the elbow.c value).
```

listing 243: the tie rules as pinned, quoted from the go bridge (source: kdd/samples/src/Ch31/kmeans.c, lines 9-16)

```
/* ---- TIE micro-fixture, byte-parity with the bridge's go test ---- */
points_row(TIE, 3, buf);
printf("%s\n", buf);
CHECK(strcmp(buf, "points: p1=(0,0);p2=(2,0);p3=(1,0)") == 0,
      "ch31 TIE points row");
lloyd(TIE, 3, 2, &r);
sprintf(row, "tie p3 d0=%d d1=%d joins=%d",
        (TIE[2][0] - TIE[0][0]) * (TIE[2][0] - TIE[0][0]),
        (TIE[2][0] - TIE[1][0]) * (TIE[2][0] - TIE[1][0]), r.assign[2] - '0');
printf("%s\n", row);
CHECK(strcmp(row, "tie p3 d0=1 d1=1 joins=0") == 0,
      "ch31 TIE equal distances resolve to the lowest cluster index");
cents_row(r.tr[0].cents, 2, a);
fstr(r.tr[0].sse, b);
sprintf(row, "iter%d cents=%s assign=%s sse=%s", r.tr[0].pass, a,
        r.tr[0].assign, b);
printf("%s\n", row);
CHECK(strcmp(row, "iter1 cents=(1/2,0),(2,0) assign=010 sse=1/2") == 0,
      "ch31 TIE iter1 row (dyadic centroid held exactly)");
sprintf(row, "final iters=%d assign=%s", r.iters, r.assign);
printf("%s\n", row);
CHECK(strcmp(row, "final iters=2 assign=010") == 0,
      "ch31 TIE final row, go bridge parity");
```

listing 244: the TIE fixture: one equal-distance fork, resolved to cluster 0 (source: kdd/samples/src/Ch31/kmeans.c, lines 368-391)

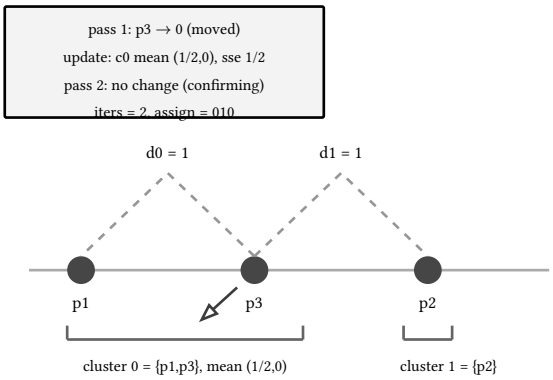


figure 128: one tie, one rule: the lowest cluster index wins

sources: the iteration is Lloyd 1982, “Least squares quantization in PCM”, IEEE Trans. Inf. Theory 28:129-137, the 1957 Bell Labs memorandum published 25 years later, with the streaming spelling from MacQueen 1967; the largest-second-difference elbow with the kneedle-style crosscheck is Thorndike 1953, Psychometrika 18:267, and the restart remedy named in the pitfall is Arthur and Vassilvitskii, “k-means++: the advantages of careful seeding”, SODA 2007. The tie rules and the iters-counts-the-confirming-pass semantics are pinned by kdd-contract-s7.md as the committed Go bridge books/kdd/capstone/internal/kddcore/kmeans.go implements them, witnessed by playground/kdd-matrix/gen_s7.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch31`, 17 + 13 checks in chapter 31 of the kdd suite.

32. hierarchical clustering

One sample carries the chapter: `linkage.c` runs single, complete, and average linkage on one 6x6 integer distance matrix, then recomputes the hardest merge height by the Lance-Williams recurrence and by hand, 30 checks. The chapter makes 4 moves: the dissimilarity matrix as the whole input and the metric precondition, the min and max extremes that chain or drag, the mean in between with its exact fractions, and the recurrence that keeps the merge loop honest. The matrix is the proximity data of book 14, ch 5 (knowledge discovery, distance metrics) with the points thrown away, the flat partitions of book 14, ch 31 (knowledge discovery, k-means) come out of this tree by cutting it, and book 14, ch 35 (knowledge discovery, cure and chameleon) keeps the agglomerative engine and swaps the distance definition to survive non-convex clusters.

32.1. one matrix, six leaves, no ties

Agglomerative clustering starts with every point its own cluster and merges the closest pair until one cluster remains. The only input is the matrix: the algorithm never sees coordinates, so everything downstream hangs on six rows of integers. The precondition is that those integers behave like distances, and the sample checks all of it: `metric_ok=true` asserts all 20 triangle inequalities, the $\binom{6}{3}$ triples with all three inequalities each. Two triples close with equality, $d_{13} = 8 = 3 + 5 = d_{12} + d_{23}$ and $d_{14} = 15 = 8 + 7 = d_{13} + d_{34}$, so the quadruple 1, 2, 3, 4 adds up like positions 0, 3, 8, 15 on a line, and $d_{24} = 12$, $d_{23} = 5$ fall out of the same arithmetic. A degenerate triangle is still a triangle: the checker accepts equality, and single linkage will exploit that collinearity in the next facet.

The fixture was searched, not drawn: a seeded local search over integer matrices (seed 42) with three demands, metric, no ties at any merge decision in any of the three linkages, and three pairwise-distinct dendrogram shapes. The no-ties property is what makes every trace below data-determined rather than rule-determined: the merge loop does carry a tie rule, lowest pair of minimum members, and the check “ch32 no merge decision ties in any of the three linkages” pins that it never fires.

The dry run: the matrix prints row by row, `D1: 0 3 8 15 13 14` through `D6: 14 12 12 10 4 0`, then `metric_ok=true`. The first two merges are common to all three runs because at that stage clusters are single points and every linkage reads the same cell: `merge {1}+{2}@3`, the unique matrix minimum $d_{12} = 3$, then `merge {5}+{6}@4`, $d_{56} = 4$. After two merges the cluster-to-cluster question begins, and the three linkages part ways.

```
int main(void) {
  char row[128], buf[64], buf2[64], lwrow[128];
  const char *pinD[6] = {"D1: 0 3 8 15 13 14", "D2: 3 0 5 12 12 12",
                         "D3: 8 5 0 7 13 12",  "D4: 15 12 7 0 8 10",
                         "D5: 13 12 13 8 0 4", "D6: 14 12 12 10 4 0"};
  for (int i = 1; i ≤ 6; i++) {
    char *p = row;
    p += sprintf(p, "D%d:", i);
    for (int j = 1; j ≤ 6; j++) {
      p += sprintf(p, " %d", i == j ? 0 : di(i, j));
    }
    printf("%s\n", row);
    CHECK(strcmp(row, pinD[i - 1]) == 0, "ch32 D32 matrix row");
  }

  /* metric check: all 20 triples, all three inequalities each */
  int ok = 1;
  for (int i = 1; i ≤ 6; i++) {
    for (int j = i + 1; j ≤ 6; j++) {
      for (int k = j + 1; k ≤ 6; k++) {
        const int a = di(i, j), b = di(i, k), c = di(j, k);
        if (a > b + c || b > a + c || c > a + b) {
          ok = 0;
        }
      }
    }
  }
  printf("metric_ok=%s\n", ok ? "true" : "false");
  CHECK(ok, "ch32 D32 metric_ok: all 20 triangle inequalities hold");
```

listing 245: the matrix rows and the 20-triangle metric gate (source: kdd/samples/src/Ch32/linkage.c, lines 212-240)

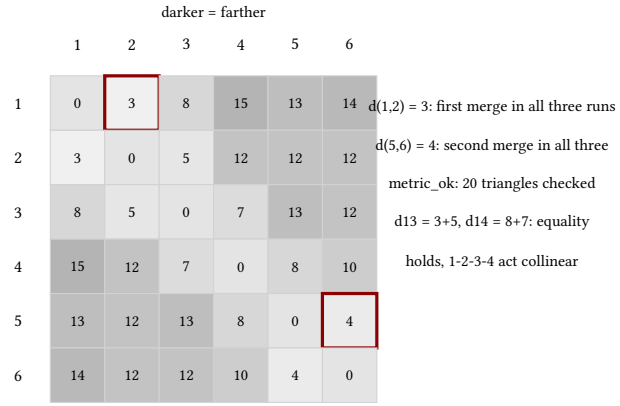


figure 129: the matrix is the whole input; two cells merge first in every run

32.2. single chains, complete drags

A linkage is a rule for extending point distance to cluster distance. Single takes the minimum over cross pairs, so a cluster grows by its nearest neighbor; complete takes the maximum, so nothing merges until every cross pair is close. On the collinear run 1-2-3-4 the two rules could not disagree more.

The dry run, single first: after the shared prefix the trace continues `single merge {1,2}+{3}@5` (the min over cross pairs {8, 5} is d_{23}), `single merge {1,2,3}+{4}@7` (d_{34}), and `single merge {1,2,3,4}+{5,6}@8` (d_{45}, the min over eight cross pairs). Each leaf splices onto the chain at its distance to the chain's nearest endpoint, and the final merge happens at 8 even though the far pair (1,6) sits 14 apart. The shape string

prints `single shape ((((1 2) 3) 4) (5 6))`, a caterpillar. Complete diverges at the third merge: the candidates are $d(\{1,2\},\{3\}) = \max(8,5) = 8$ against $d(\{3\},\{4\}) = 7$, so `complete merge {3}+{4}@7` fires instead, then `complete merge {3,4}+{5,6}@13` (the max over $13, 8, 12, 10$), and the run closes at the full diameter `complete merge {1,2}+{3,4,5,6}@15`, which is d_{14}. The shape prints `complete shape ((1 2) ((3 4) (5 6)))`, balanced. Same matrix, same prefix, two different trees: single reads the nearest crossing, complete reads the farthest, and the check "ch32 the three dendrogram shapes differ pairwise" already counts this one in.

```
/* ——— fixture D32 (contract rows): D[smaller][larger] holds d ——— */
static const int D[7][7] = {{0, 0, 0, 0, 0, 0, 0},    {0, 0, 3, 8, 15, 13, 14},
                            {0, 0, 0, 5, 12, 12, 12}, {0, 0, 0, 0, 7, 13, 12},
                            {0, 0, 0, 0, 0, 8, 10},   {0, 0, 0, 0, 0, 0, 4},
                            {0, 0, 0, 0, 0, 0, 0}};

static int di(int i, int j) { return D[i < j ? i : j][i < j ? j : i]; }

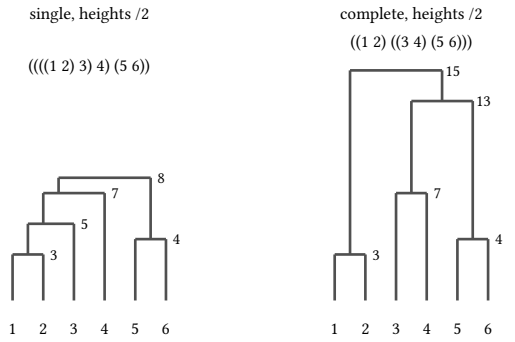
static void setstr(int mask, char *buf) {
  char *p = buf;
  *p++ = '{';
  int first = 1;
  for (int i = 1; i ≤ 6; i++) {
    if (mask & (1 ≪ i)) {
      if (!first) {
        *p++ = ',';
      }
      first = 0;
      p += sprintf(p, "%d", i);
    }
  }
  *p++ = '}';
  *p = 0;
}

static int minmem(int mask) {
  for (int i = 1; i ≤ 6; i++) {
    if (mask & (1 ≪ i)) {
      return i;
    }
  }
  return 0;
}

/* single = min cross pair, complete = max, average = mean over crosses */
static Frac cludist(int a, int b, int method) {
  int mn = 999, mx = -1;
  long long sum = 0, cnt = 0;
  for (int i = 1; i ≤ 6; i++) {
    if (!(a & (1 ≪ i))) {
      continue;
    }
    for (int j = 1; j ≤ 6; j++) {
      if (b & (1 ≪ j)) {
        const int v = di(i, j);
        if (v < mn) {
          mn = v;
        }
        if (v > mx) {
```

```
          mx = v;
        }
        sum += v;
        cnt++;
      }
    }
  }
  if (method == 0) {
    return fr(mn, 1);
  }
  if (method == 1) {
    return fr(mx, 1);
  }
  return fr(sum, cnt);
}
```

listing 246: the fixture and cludist: min, max, or mean over the cross pairs (source: kdd/samples/src/Ch32/linkage.c, lines 79-142)



single's last merge at 8 is the nearest crossing; complete's at 15 is the diameter

figure 130: two trees from one prefix: min chains into a caterpillar, max balances

32.3. average and unavoidable thirds

Average linkage takes the mean over cross pairs, the compromise that keeps every distance in play. Its price is arithmetic: means of integers are fractions, and at six points the last merge averages $3 \times 3 = 9$ pairs, so thirds arrive.

The dry run: the shared prefix prints `average merge {1}+{2}@3` and `average merge {5}+{6}@4`, then the mean of cross pair {8, 5} wins the third merge, `average merge {1,2}+{3}@13/2`, beating $d_{34} = 7$ and the {4} to {5,6} mean of 9. Fourth: `average merge {4}+{5,6}@9`, the mean of {8, 10}, beating $d(\{1,2,3\},\{4\}) = 34/3$ (from 15, 12, 7) and $d(\{1,2,3\},\{5,6\}) = 38/3$ (from 13, 12, 13, 14, 12, 12). The run closes with `average merge {1,2,3}+{4,5,6}@110/9`, the mean over all nine cross pairs, and the shape prints `average shape (((1 2) 3) (4 (5 6)))`, the third distinct tree. The 110/9 is pinned after a ruled-out alternative: a fully dyadic average trace needs merge sizes that avoid 3+3 at six points, and that conflicts with the three-distinct-shapes requirement, a 400k-step seeded search found no matrix that delivers both. So the final height is an exact reduced fraction, compared as an int64 pair, and the check "ch32 average final height is the exact fraction 110/9 (thirds)" pins the denominator.

```
static int any_tie = 0; /* pairs sharing the minimum at any decision */

static int run_linkage(int method, const char *name,
                       const char *const *pin_merge, const char *pin_shape,
                       char *shape_out) {
  int clu[6];
  int nclu = 6;
  char shape[7][80];
```

```
  for (int i = 0; i < 6; i++) {
    clu[i] = 1 << (i + 1);
    sprintf(shape[i + 1], "%d", i + 1);
  }
  for (int m = 0; m < 5; m++) {
    /* pass 1: minimum distance and its (lowest-min-member) pair */
    int bx = 0, by = 1;
    Frac bd = cludist(clu[0], clu[1], method);
    for (int x = 0; x < nclu; x++) {
      for (int y = x + 1; y < nclu; y++) {
        const Frac dd = cludist(clu[x], clu[y], method);
        const int c = fcmp(dd, bd);
        const int ma = minmem(clu[x]), mb = minmem(clu[y]);
        if (c < 0 ||
            (c == 0 && (ma < minmem(clu[bx]) ||
                        (ma == minmem(clu[bx]) && mb < minmem(clu[by]))))) {
          bd = dd;
          bx = x;
          by = y;
        }
      }
    }
    /* pass 2: tie detection at the minimum */
    int at_min = 0;
    for (int x = 0; x < nclu; x++) {
      for (int y = x + 1; y < nclu; y++) {
        if (fcmp(cludist(clu[x], clu[y], method), bd) == 0) {
          at_min++;
        }
      }
    }
    if (at_min > 1) {
      any_tie = 1;
    }
    char sa[32], sb[32], hs[32], line[96];
    setstr(clu[bx], sa);
    setstr(clu[by], sb);
    fstr(bd, hs);
    sprintf(line, "%s merge %s+%s@%s", name, sa, sb, hs);
    printf("%s\n", line);
    CHECK(strcmp(line, pin_merge[m]) == 0, "ch32 merge sequence line");
    const int a = minmem(clu[bx]), b = minmem(clu[by]);
    char ns[80];
    sprintf(ns, "(%s %s)", a < b ? shape[a] : shape[b],
            a < b ? shape[b] : shape[a]);
    sprintf(shape[a < b ? a : b], "%s", ns);
    clu[bx] |= clu[by];
    for (int y = by; y < nclu - 1; y++) {
      clu[y] = clu[y + 1];
    }
    nclu--;
  }
  char shape_row[96];
  sprintf(shape_row, "%s shape %s", name, shape[1]);
  printf("%s\n", shape_row);
  CHECK(strcmp(shape_row, pin_shape) == 0, "ch32 dendrogram shape row");
  sprintf(shape_out, "%s", shape_row);
```

```
  return 0;
}
```

listing 247: the merge loop: pick the minimum pair, tie to lowest members, record (source: kdd/samples/src/Ch32/linkage.c, lines 144-210)

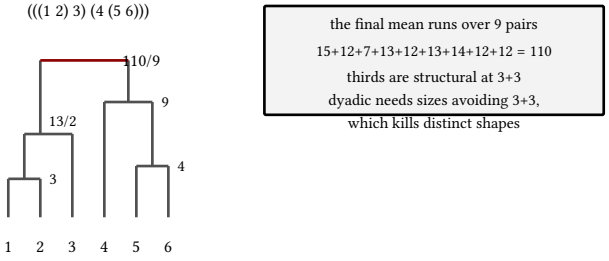


figure 131: average keeps every distance in play and pays in thirds

NOTE: one recurrence family runs all three

Single, complete, and average are three parameter settings of one update formula: the Lance-Williams recurrence expresses the distance from a newly merged cluster to any survivor through the two old distances and the cluster sizes, with single and complete sitting at the $\alpha = \pm 1/2$, $\beta = 0$ corners and average at $\alpha_a = n_a/(n_a + n_b)$, $\alpha_b = n_b/(n_a + n_b)$. That is why one merge loop in the sample serves all three runs, and why the next facet can verify the hardest row from two old distances alone.

32.4. lance-williams, two ways to the same fraction

Rescanning every cross pair at every merge is quadratic bookkeeping. The recurrence for average linkage is linear in the two old distances: $d(A \cup B, C) = (n_a d(A, C) + n_b d(B, C))/(n_a + n_b)$, so the loop updates each surviving cluster's distance to the new one from two numbers. The sample witnesses the hardest row both ways.

The dry run: form {4,5,6} first. To the survivor {1,2,3}, the singleton {4} sits at $34/3$ (the mean of 15, 12, 7) and the pair {5,6} at $38/3$ (the mean of 13, 12, 13, 14, 12, 12), so the recurrence prints `lw d({1,2,3},{4,5,6})=(2*38/3+1*34/3)/3=110/9`, the size-2 term first exactly as the sheet pins it, $(76 + 34)/9 = 110/9$. The direct scan sums the nine cross pairs to 110 and prints `lw direct=110/9 equal=true`. Both routes land on the same reduced fraction, which is the whole claim: the recurrence is not an approximation of the mean, it is the mean rearranged, and because both sides are exact int64 pairs the equality check is a comparison, not a tolerance. That is the D0 discipline every height in this chapter rides on.

```
  /* lance-williams recompute of the final average merge via the
   * {4,5,6} formation: d({4} u {5,6}, {1,2,3}) =
   * (1*d({4},C) + 2*d({5,6},C)) / 3, printed with the size-2 term first
   * exactly as the sheet pins it */
  long long s56 = 0, s4 = 0, sdirect = 0;
  for (int a = 1; a ≤ 3; a++) {
    for (int b = 5; b ≤ 6; b++) {
      s56 += di(a, b);
    }
    s4 += di(a, 4);
    for (int b = 4; b ≤ 6; b++) {
      sdirect += di(a, b);
    }
  }
  const Frac d56 = fr(s56, 6); /* d({5,6}, {1,2,3}) = 38/3 */
  const Frac d4 = fr(s4, 3);   /* d({4}, {1,2,3}) = 34/3 */
  const Frac lw = fdiv(fadd(fmul(fr(2, 1), d56), fmul(fr(1, 1), d4)), fr(3, 1));
  const Frac direct = fr(sdirect, 9);
  fstr(d56, buf);
  fstr(d4, buf2);
  fstr(lw, row);
  sprintf(lwrow, "lw d({1,2,3},{4,5,6})=(2*%s+1*%s)/3=%s", buf, buf2, row);
  printf("%s\n", lwrow);
  CHECK(strcmp(lwrow, "lw d({1,2,3},{4,5,6})=(2*38/3+1*34/3)/3=110/9") == 0,
        "ch32 lance-williams recompute row");
  fstr(direct, buf);
  sprintf(lwrow, "lw direct=%s equal=%s", buf,
          fcmp(lw, direct) == 0 ? "true" : "false");
  printf("%s\n", lwrow);
  CHECK(strcmp(lwrow, "lw direct=110/9 equal=true") == 0,
        "ch32 lw recurrence equals the direct mean over the 9 cross pairs");
  CHECK(fcmp(lw, fr(110, 9)) == 0 && lw.d ≠ 1,
        "ch32 average final height is the exact fraction 110/9 (thirds)");
```

listing 248: the recurrence recompute and the direct mean, equal as fractions (source: kdd/samples/src/Ch32/linkage.c, lines 273-305)

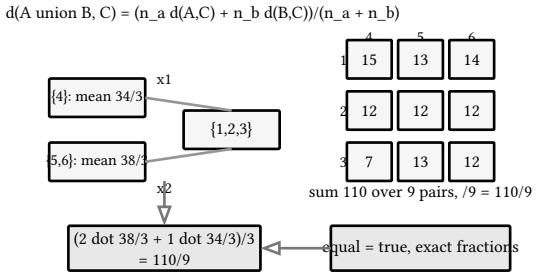


figure 132: the same fraction from two old distances or nine cross pairs

sources: the recurrence and its parameter family are Lance & Williams 1967, “A general theory of classificatory sorting strategies: 1. hierarchical systems”, Computer Journal 9:373-380, and the linkage taxonomy with the dendrogram reading follows Kaufman & Rousseeuw 1990, “Finding Groups in Data: An Introduction to Cluster Analysis”, Wiley. The D32 matrix is purpose-built by seeded local search, probe seed 42, with metric, no-ties, and three-distinct-shapes asserted in the sample, and every pinned row is witnessed by kdd-contract-s7.md + playground/kdd-matrix/gen_s7.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch32`, 30 checks in chapter 32 of the kdd suite.

33. dbscan

One sample carries the chapter: `dbscan.c` classifies 12 points into a rectangle blob, a two-point chain, a two-claim border, a triangle blob, and 2 noise points, 39 checks. The chapter makes 4 moves: the eps ball and the core census with the closed-boundary convention, the breadth-first expansion that grows clusters without a k, the density-reachability chain that defines what a cluster is, and the border point two clusters both want. The neighborhoods are the squared euclidean of book 14, ch 5 (knowledge discovery, distance metrics) compared as integers, book 14, ch 31 (knowledge discovery, k-means) needed k up front while this chapter discovers the count, book 14, ch 34 (knowledge discovery, cluster validation) reuses the census row verbatim, and book 14, ch 38 (knowledge discovery, proximity and density anomalies) later grades density per point where this chapter grades it per neighborhood.

33.1. eps balls and the core census

DBSCAN fixes two parameters and never asks for k: a radius eps, here 5 as a squared-distance budget of 25, and a core threshold minpts, here 3. A point's neighborhood N(p) is every other point with $d^2 \leq 25$, the CLOSED ball, so pairs at exactly $d = 5$ count as neighbors. A point is core when its neighborhood holds at least minpts others, self excluded, the scikit-learn spelling of the original paper's threshold. Everything stays integer: the sample never takes a square root.

The dry run: the 3-4-5 rectangle at the origin gives `nb p1={2,3,4} n=3 core`, neighbors at squared distances 9, 16, and 25, the last being p4 exactly on the boundary. Corner p4 gains the chain point: `nb p4={1,2,3,5} n=4 core`. The tail of the second blob shows the same boundary work: `nb p9={8,10} n=2 noncore`, where p10 sits at squared distance 25, in. Three exactly-at-eps pairs exist in the fixture, {1,4}, {2,3}, {9,10}, and the check "ch33 closed ball: the three d=5 pairs {1,4},{2,3},{9,10} are in" pins all three as neighbors, the closed ball doing real work rather than sitting in a comment. The two far points print `nb p11={} n=0 noncore` and `nb p12={} n=0 noncore`, empty neighborhoods that will become noise.

```
 * convention pinned by the sheet (sklearn): eps = 5 as a CLOSED ball,
 * so neighbors are d^2 ≤ 25 and the three exactly-at-eps pairs
 * {p1,p4}, {p2,p3}, {p9,p10} (d = 5, d2 = 25) are IN; minpts = 3
 * counts neighbors EXCLUDING self; clusters expand breadth-first from
 * the first unvisited core in input order; a border point claimed by
 * cores of two clusters joins the cluster of the first core that
 * reaches it in that scan order (p7 -> cluster 0).
 *
 * determinism: D0 throughout, integer squared distances only, no
 * square roots anywhere. */
```

listing 249: the pinned convention: closed ball, self excluded, scan order (source: kdd/samples/src/Ch33/dbscan.c, lines 10-19)

```
  /* neighbors: closed ball d2 ≤ 25, self excluded */
  for (int i = 0; i < N; i++) {
    nbc[i] = 0;
    for (int j = 0; j < N; j++) {
      nb[i][j] = (i ≠ j && d2(i, j) ≤ EPS2);
      nbc[i] += nb[i][j];
    }
    core[i] = nbc[i] ≥ MINPTS;
  }
  static const char *pinnb[N] = {
      "nb p1={2,3,4} n=3 core",   "nb p2={1,3,4} n=3 core",
      "nb p3={1,2,4} n=3 core",   "nb p4={1,2,3,5} n=4 core",
      "nb p5={4,6,7} n=3 core",   "nb p6={5} n=1 noncore",
      "nb p7={5,8} n=2 noncore",  "nb p8={7,9,10} n=3 core",
      "nb p9={8,10} n=2 noncore", "nb p10={8,9} n=2 noncore",
      "nb p11={} n=0 noncore",    "nb p12={} n=0 noncore"};
  for (int i = 0; i < N; i++) {
    int mem[N];
    int m = 0;
    for (int j = 0; j < N; j++) {
      if (nb[i][j]) {
        mem[m++] = j;
      }
    }
    setstr(mem, m, buf);
    sprintf(row, "nb p%d=%s n=%d %s", i + 1, buf, nbc[i],
            core[i] ? "core" : "noncore");
    printf("%s\n", row);
    CHECK(strcmp(row, pinnb[i]) == 0, "ch33 neighbor census row");
  }

  /* the three exactly-at-eps pairs sit on the closed-ball boundary */
  CHECK(d2(0, 3) == 25 && d2(1, 2) == 25 && d2(8, 9) == 25 && nb[0][3] &&
            nb[1][2] && nb[8][9],
        "ch33 closed ball: the three d=5 pairs {1,4},{2,3},{9,10} are in");
```

listing 250: the neighbor census and the three exactly-at-eps pairs (source: kdd/samples/src/Ch33/dbscan.c, lines 82-116)

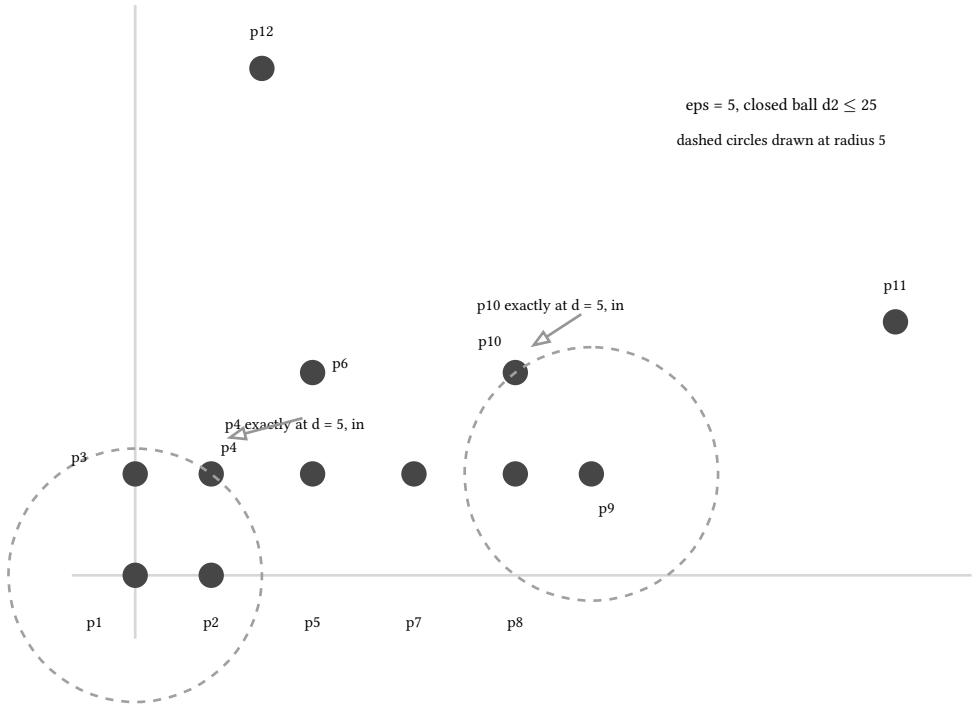


figure 133: the fixture: two boundary pairs sit exactly on the closed eps ball

PITFALL: two conventions decide the whole census

The closed ball and the self-excluded count are not cosmetic choices, they move points across classes here. An open ball would drop {1,4}, {2,3}, {9,10} from the neighbor lists: p1 would read n=2 and lose core, and p10 would drop out of p9′s list entirely. Counting self the other way, p7′s row would read n=3 and a border would become a core, changing the census and with it the two-claim story of the last facet. The sample pins both conventions in its header and the checks hold them to the pinned rows.

33.2. expansion, borders, and noise

Clusters grow breadth-first from the first unvisited core in input order, and the rule for growth is one line: a core claims every unvisited neighbor, a border absorbs its label but never expands. Points no cluster ever reaches stay noise, labeled −1.

The dry run: p1 is the first core in input order and seeds cluster 0. The BFS absorbs the rectangle {p1,p2,p3,p4}, walks to p5 through p4′s neighborhood, and takes the borders p6 and p7 with it, printing `cluster0={1,2,3,4,5,6,7}`. The scan resumes, finds p8 unvisited, and seeds cluster 1, absorbing p9 and p10: `cluster1={8,9,10}`. The label rows carry the taxonomy per point, `label p6=0 border` and `label p8=1 core` among them, and the two unreached points print `label p11=-1 noise` and `label p12=-1 noise`, collected as `noise={11,12}`. The census closes the accounting: `census core=6 border=4 noise=2 clusters=2`. The cluster count was never an input, it is what the parameters found: 6 cores carry the structure, 4 borders ride it, 2 points are too far from everything to belong.

```
  /* expansion: bfs from the first unvisited core in input order */
  int lbl[N];
  for (int i = 0; i < N; i++) {
    lbl[i] = -1;
  }
  int cid = 0;
  for (int s = 0; s < N; s++) {
    if (!core[s] || lbl[s] != -1) {
      continue;
    }
    int q[N], head = 0, tail = 0;
    lbl[s] = cid;
    q[tail++] = s;
    while (head < tail) {
      const int u = q[head++];
      if (!core[u]) { /* borders absorb their label but never expand */
        continue;
      }
      for (int j = 0; j < N; j++) {
        if (nb[u][j] && lbl[j] == -1) {
          lbl[j] = cid;
          q[tail++] = j;
        }
      }
    }
    cid++;
  }
  static const char *pinlbl[N] = {
      "label p1=0 core",    "label p2=0 core",    "label p3=0 core",
      "label p4=0 core",    "label p5=0 core",    "label p6=0 border",
      "label p7=0 border",  "label p8=1 core",    "label p9=1 border",
      "label p10=1 border", "label p11=-1 noise", "label p12=-1 noise"};
  for (int i = 0; i < N; i++) {
    const char *t = lbl[i] == -1 ? "noise" : core[i] ? "core" : "border";
    sprintf(row, "label p%d=%d %s", i + 1, lbl[i], t);
    printf("%s\n", row);
    CHECK(strcmp(row, pinlbl[i]) == 0, "ch33 point classification row");
  }
```

listing 251: breadth-first expansion: cores claim, borders absorb, noise stays (source: kdd/samples/src/Ch33/dbscan.c, lines 118-155)

```
{
  int ncore = 0, nborder = 0, nnoise = 0;
  for (int i = 0; i < N; i++) {
    if (core[i]) {
      ncore++;
    } else if (lbl[i] != -1) {
      nborder++;
    } else {
      nnoise++;
    }
  }
  sprintf(row, "census core=%d border=%d noise=%d clusters=%d", ncore,
          nborder, nnoise, cid);
  printf("%s\n", row);
  CHECK(strcmp(row, "census core=6 border=4 noise=2 clusters=2") == 0,
        "ch33 census row");
}
for (int c = 0; c < cid; c++) {
  int mem[N];
  int m = 0;
  for (int i = 0; i < N; i++) {
    if (lbl[i] == c) {
      mem[m++] = i;
    }
  }
  setstr(mem, m, buf);
  sprintf(row, "cluster%d=%s", c, buf);
  printf("%s\n", row);
  CHECK(strcmp(row, c == 0 ? "cluster0={1,2,3,4,5,6,7}"
                           : "cluster1={8,9,10}") == 0,
        "ch33 cluster membership row");
}
{
  int mem[N];
  int m = 0;
  for (int i = 0; i < N; i++) {
    if (lbl[i] == -1) {
      mem[m++] = i;
    }
  }
  setstr(mem, m, buf);
  sprintf(row, "noise=%s", buf);
  printf("%s\n", row);
  CHECK(strcmp(row, "noise={11,12}") == 0, "ch33 noise row");
}
```

listing 252: the census, the two membership rows, and the noise row (source: kdd/samples/src/Ch33/dbscan.c, lines 157-201)

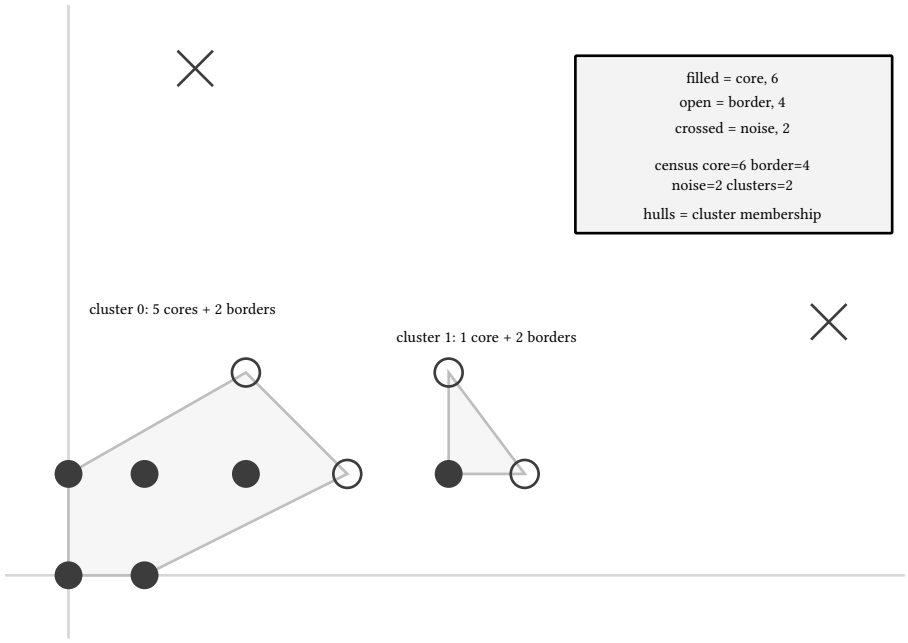


figure 134: the census on the plane: 6 cores, 4 borders, 2 noise, 2 clusters

33.3. density reachability

The definition underneath the BFS: q is directly density-reachable from p when p is core and q is in N(p), and density-reachable when a chain of such steps exists, every intermediate point core. The chain may end at a border because only the reaching endpoint must be core, which is exactly why borders ride clusters without ever expanding them.

The dry run: the sample walks the longest chain in the fixture, from the rectangle into the border p7: `reach p1→p4 d2=25 core→core`, `reach p4→p5 d2=16 core→core`, `reach p5→p7 d2=16 core→border`. Each row is checked twice, once as a printed line and once as the assertion that the reaching point is core and the target is its neighbor. The first hop is the closed ball again: p4 is directly density-reachable from p1 at distance exactly 5, on the boundary, in. The last hop shows the asymmetry: p5->p7 is a legal step and p7->p5 would not begin a chain, because p7 is not core. Density-connectivity, the true cluster definition, is reachability plus symmetry at the far end, and the breadth-first expansion of the previous facet computes it without ever naming it.

```
/* density-reachability chain p1 -> p4 -> p5 -> p7 */
static const int chain[3][2] = {{0, 3}, {3, 4}, {4, 6}};
static const char *pinreach[3] = {"reach p1→p4 d2=25 core→core",
                                  "reach p4→p5 d2=16 core→core",
                                  "reach p5→p7 d2=16 core→border"};
for (int k = 0; k < 3; k++) {
  const int u = chain[k][0], v = chain[k][1];
  sprintf(row, "reach p%d→p%d d2=%d %s", u + 1, v + 1, d2(u, v),
          core[v] ? "core→core" : "core→border");
  printf("%s\n", row);
  CHECK(strcmp(row, pinreach[k]) == 0, "ch33 reachability chain row");
  CHECK(core[u] && nb[u][v],
        "ch33 reachability step is a core point reaching a neighbor");
}
```

listing 253: the chain trace, each step core reaches neighbor (source: kdd/samples/src/Ch33/dbscan.c, lines 203-217)

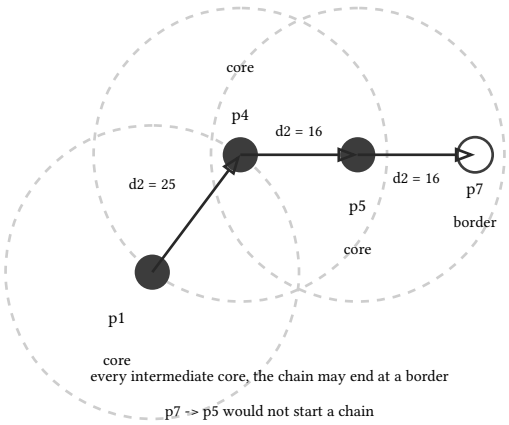


figure 135: three hops, every intermediate core, ending at a border

33.4. the border with two claims

A border point within eps of cores from two different clusters is legitimately claimable by both, and the assignment is a convention the sample pins rather than a theorem the algorithm proves: the border joins the cluster of the first core that reaches it in the scan order.

The dry run: p7′s neighborhood is exactly the two cores, and the row prints `border p7 nb={5,8} cores p5(cluster0) p8(cluster1)`. Both claims are legal density steps, p7 within eps of p5 of cluster 0 and of p8 of cluster 1. The resolution prints `claimed_by=0 rule=first_core_in_input_order`, p5 preceding p8 in the input, and the loser is recorded rather than discarded, `also_core={8} cluster=1`. The order is not incidental: the breadth-first scan of the previous facet seeded cluster 0 at p1 and reached p5 before p8 was ever visited, so the deterministic rule and the traversal agree. Swap the input order of p5 and p8 and the same rule hands p7 to cluster 1, which is the honest statement of what a two-claim border is: a point the parameters cannot decide, decided by an order someone chose.

```
   * core p8 (cluster 1); it joins the first core in input order */
  {
    int mem[N];
    int m = 0;
    for (int j = 0; j < N; j++) {
      if (nb[6][j]) {
        mem[m++] = j;
      }
    }
    setstr(mem, m, buf);
    int c5 = -1, c8 = -1;
    for (int j = 0; j < N; j++) {
      if (nb[6][j] && core[j]) {
        if (j == 4) {
          c5 = lbl[j];
        }
        if (j == 7) {
          c8 = lbl[j];
        }
      }
    }
    sprintf(row, "border p7 nb=%s cores p5(cluster%d) p8(cluster%d)", buf, c5,
            c8);
    printf("%s\n", row);
    CHECK(strcmp(row, "border p7 nb={5,8} cores p5(cluster0) p8(cluster1)") ==
              0,
          "ch33 border p7 neighbor cores span both clusters");
    sprintf(row, "claimed_by=%d rule=first_core_in_input_order", lbl[6]);
    printf("%s\n", row);
    CHECK(strcmp(row, "claimed_by=0 rule=first_core_in_input_order") == 0,
          "ch33 two-claim border joins the first core's cluster in input "
          "order");
    int also[N];
    int na = 0;
    for (int j = 0; j < N; j++) {
      if (nb[6][j] && core[j] && lbl[j] != lbl[6]) {
        also[na++] = j;
      }
    }
    setstr(also, na, buf);
    sprintf(row, "also_core=%s cluster=%d", buf, lbl[also[0]]);
    printf("%s\n", row);
    CHECK(strcmp(row, "also_core={8} cluster=1") == 0,
          "ch33 the losing claim: core p8 would have taken p7 to cluster 1");
  }
```

listing 254: the two claims, the pinned rule, and the recorded loser (source: kdd/samples/src/Ch33/dbscan.c, lines 219-263)

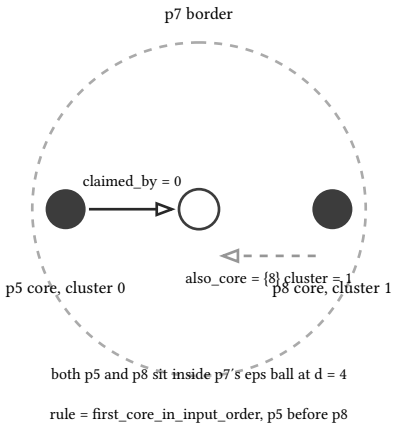


figure 136: one border, two legal claims, one pinned rule

NOTE: border assignment is a convention, not geometry

Nothing about the distances breaks the tie: p7 is 4 from both claimant cores, an exact tie in the data. The original algorithm leaves border membership to the order clusters happen to expand, and different implementations differ exactly here. The pin makes the choice explicit and testable, first core in input order, with the losing claim printed beside the winner, so the row reads as a decision with a record rather than an accident with an output.

sources: the algorithm and the reachability definitions are Ester, Kriegel, Sander, Xu 1996, "A density-based algorithm for discovering clusters in large spatial databases with noise", Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), AAAI Press, pp. 226-231, and the minpts-counts-neighbors- excluding-self convention is scikit-learn's DBSCAN documentation as pinned by kdd-contract-s7.md. The fixture and every pinned row are witnessed by the contract + playground/kdd-matrix/gen_s7.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch33`, 39 checks in chapter 33 of the kdd suite.

34. cluster validation

One sample carries the chapter: `validate.c` prices the optimal contiguous k-partitions of a 12-position line by dynamic programming, scores silhouettes at k=2 and k=3 as exact fractions, and records the two metrics disagreeing, 41 checks. The chapter makes 4 moves: the partition table that makes the elbow exact, the silhouette width as a per-point fraction, the disagreement between the two metrics with its int64-safe comparison, and the negative width that flags one planted misassignment. The elbow rule is the one book 14, ch 31 (knowledge discovery, k-means) read off its own table, the chain DP is the exact k-means that chapter priced out to book 11, ch 10 (the icpc world finals, icpc world finals 2017), the census row borrowed below is book 14, ch 33 (knowledge discovery, dbscan)'s, and the mindset is the label-free cousin of the evaluation chapters: book 14, ch 17 (knowledge discovery, classifier evaluation) graded predictions against truth, cluster validation grades a partition against geometry.

34.1. the partition table, priced exactly

The fixture is 12 positions on a line: three aligned APs of spacing 12, groups {x1..x4} at 0..36, {x5..x8} at 240..276, {x9..x12} at 444..480, with gaps 204 and 168. The third group sits 14 times farther out than the second relative to the internal spacing, and that asymmetry is what splits the two metrics later. On sorted 1D data the optimal k-partition is contiguous, a crossing pair of clusters could be uncrossed without raising SSE, so the table is a chain DP over split indices, the same DP book 11, ch 10 (the icpc world finals, icpc world finals 2017) uses when exact k-means must actually run.

The dry run: the DP prints `k=1 sse=397296`
`parts={1,2,3,4,5,6,7,8,9,10,11,12}`, `k=2 sse=85392`
`parts={1,2,3,4}|{5,6,7,8,9,10,11,12}`, `k=3 sse=2160`
`parts={1,2,3,4}|{5,6,7,8}|{9,10,11,12}`, `k=4 sse=1584`
`parts={1,2}|{3,4}|{5,6,7,8}|{9,10,11,12}`. Each AP of spacing 12 contributes 720 around its mean, deviations ± 18, ± 6 twice, so k=3 lands at $3 \cdot 720 = 2160$ and k=4 pays 72 per split pair. The gains print `gains 311904,83232,576`, second differences `d2`
`228672,82656`, and the elbow row closes `elbow=2`
`kneedle_2b_gt_a_plus_c=false`: the largest second difference sits at k=2, $228672 > 82656$, and the kneedle-style crosscheck agrees it is no elbow, $2 \cdot 83232 = 166464 < 311904 + 576$. Same rule as chapter 31, different verdict, because the gap structure is different: there the second group of gains collapsed late, here the k=2 split already removes the enormous cross-gap variance.

```
/* sse of X[lo..hi) around its mean: sumsq - s^2/len, exact */
static Frac sse_seg(int lo, int hi) {
  long long s = 0, sq = 0;
  for (int i = lo; i < hi; i++) {
    s += X[i];
    sq += (long long)X[i] * X[i];
  }
  return fr(sq * (hi - lo) - s * s, hi - lo);
}

/* optimal contiguous k-partition by dp; smallest split index kept on
 * ties, mirroring the generator's strict < scan */
static Frac opt_parts(int k, int *bounds) {
  static Frac dp[13][5];
  static int cut[13][5];
  for (int i = 0; i <= 12; i++) {
    for (int j = 0; j <= k; j++) {
      dp[i][j].n = (long long)4e18;
      dp[i][j].d = 1;
      cut[i][j] = 0;
    }
  }
  dp[0][0] = fr(0, 1);
  for (int j = 1; j <= k; j++) {
    for (int i = 1; i <= 12; i++) {
      for (int m = j - 1; m < i; m++) {
        if (fcmp(dp[m][j - 1], fr((long long)4e18, 1)) == 0) {
          continue;
        }
        const Frac v = fadd(dp[m][j - 1], sse_seg(m, i));
        if (fcmp(v, dp[i][j]) < 0) {
          dp[i][j] = v;
          cut[i][j] = m;
        }
      }
    }
  }
  int i = 12, j = k, nb = 0;
  while (j > 0) {
    bounds[nb++] = cut[i][j];
    i = cut[i][j];
    j--;
  }
  for (int a = 0; a < nb / 2; a++) { /* reverse */
    const int t = bounds[a];
    bounds[a] = bounds[nb - 1 - a];
    bounds[nb - 1 - a] = t;
  }
  return dp[12][k];
}
```

listing 255: segment sse and the chain dp over split indices (source: kdd/samples/src/Ch34/validate.c, lines 88-137)

```
  Frac sses[5];
  {
    static const char *pink[4] = {
        "k=1 sse=397296 parts={1,2,3,4,5,6,7,8,9,10,11,12}",
```

```
      "k=2 sse=85392 parts={1,2,3,4}|{5,6,7,8,9,10,11,12}",
      "k=3 sse=2160 parts={1,2,3,4}|{5,6,7,8}|{9,10,11,12}",
      "k=4 sse=1584 parts={1,2}|{3,4}|{5,6,7,8}|{9,10,11,12}"};
  for (int k = 1; k ≤ 4; k++) {
    int bounds[5];
    sses[k] = opt_parts(k, bounds);
    char *p = row;
    fstr(sses[k], fa);
    p += sprintf(p, "k=%d sse=%s parts=", k, fa);
    int prev = 0;
    for (int bi = 0; bi < k; bi++) {
      const int end = bi + 1 == k ? 12 : bounds[bi + 1];
      if (bi) {
        *p++ = '|';
      }
      *p++ = '{';
      for (int i = prev; i < end; i++) {
        if (i > prev) {
          *p++ = ',';
        }
        p += sprintf(p, "%d", i + 1);
      }
      *p++ = '}';
      prev = end;
    }
    *p = 0;
    printf("%s\n", row);
    CHECK(strcmp(row, pink[k - 1]) == 0,
          "ch34 optimal contiguous k-partition row");
  }
}
{
  Frac g[3];
  for (int i = 0; i < 3; i++) {
    g[i] = fsub(sses[i + 1], sses[i + 2]);
  }
  fstr(g[0], fa);
  fstr(g[1], fb);
  fstr(g[2], fs);
  sprintf(row, "gains %s,%s,%s", fa, fb, fs);
  printf("%s\n", row);
  CHECK(strcmp(row, "gains 311904,83232,576") == 0, "ch34 gains row");
  const Frac d2a = fsub(g[0], g[1]), d2b = fsub(g[1], g[2]);
  fstr(d2a, fa);
  fstr(d2b, fb);
  sprintf(row, "d2 %s,%s", fa, fb);
  printf("%s\n", row);
  CHECK(strcmp(row, "d2 228672,82656") == 0, "ch34 d2 row");
  const int elbow = 2 + (fcmp(d2a, d2b) ≥ 0 ? 0 : 1);
  const int kn = fcmp(fr(g[1].n * 2, g[1].d), fadd(g[0], g[2])) > 0;
  sprintf(row, "elbow=%d kneedle_2b_gt_a_plus_c=%s", elbow,
          kn ? "true" : "false");
  printf("%s\n", row);
  CHECK(strcmp(row, "elbow=2 kneedle_2b_gt_a_plus_c=false") == 0,
        "ch34 elbow row: the sse elbow stops at k=2");
}
```

listing 256: the four table rows, the gains, the second differences, elbow=2 (source: kdd/samples/src/Ch34/validate.c, lines 210-269)

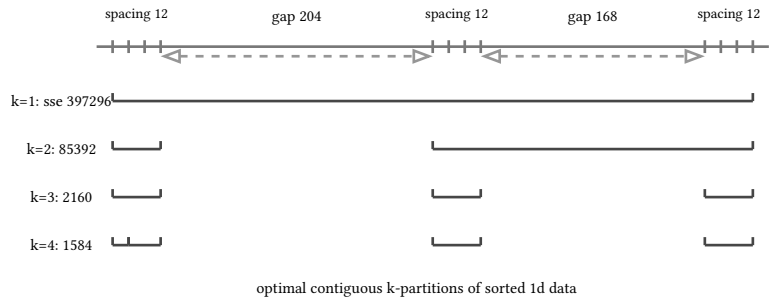


figure 137: the line, its gaps, and the optimal partitions the dp prices

34.2. silhouettes as reduced fractions

The silhouette width scores one point against its own cluster: $s(i) = (b(i) - a(i))/\max(a(i), b(i))$ with $a(i)$ the mean distance to the point's own cluster and $b(i)$ the smallest mean distance to any other cluster. Near 1 the point sits snug, near 0 it straddles, below 0 it is closer to a neighbor than to home. On fixture S every distance is an integer difference, so every a, b, and s is an exact fraction, and the sample holds them as reduced int64 pairs.

The dry run at k=3: `sil x1 a=24 b=258 s=39/43`. x1′s own-cluster mean is $(12 + 24 + 36)/3 = 24$, the aligned AP making the arithmetic integer; the nearest foreign cluster is the middle one at mean distance 258 against 462 for the far group, so $s = 234/258 = 39/43$. The interior points do better, `sil x2 a=16 b=246 s=115/123`, and the symmetry is exact: `sil x4 … s=33/37` and `sil x5 … s=33/37` agree digit for digit, the two groups mirror each other around the 204 gap. The mean over all 12 prints `mean k3 s=247695666679/273264726735`, about 0.906, and the check pins the reduced pair itself, numerator and denominator, not a double.

```
static void silhouette(const int *owner, int k, Frac *a, Frac *b, Frac *s) {
  for (int i = 0; i < 12; i++) {
    const int ci = owner[i];
    long long sum_own = 0;
    int cnt_own = 0;
    for (int j = 0; j < 12; j++) {
      if (owner[j] == ci && j != i) {
        sum_own += X[i] > X[j] ? X[i] - X[j] : X[j] - X[i];
        cnt_own++;
      }
    }
    a[i] = fr(sum_own, cnt_own);
    Frac bb = fr(0, 1);
    int first = 1;
    for (int c = 0; c < k; c++) {
      if (c == ci) {
        continue;
      }
      long long sum = 0;
      int cnt = 0;
      for (int j = 0; j < 12; j++) {
        if (owner[j] == c) {
          sum += X[i] > X[j] ? X[i] - X[j] : X[j] - X[i];
          cnt++;
        }
      }
      const Frac m = fr(sum, cnt);
      if (first || fcmp(m, bb) < 0) {
        bb = m;
        first = 0;
      }
    }
    b[i] = bb;
    /* s = (b - a) / max(a, b); the max picks the larger fraction */
    const Frac mx = fcmp(a[i], bb) > 0 ? a[i] : bb;
    s[i] = fdiv(fsub(bb, a[i]), mx);
  }
}
```

listing 257: silhouettes: integer sums, fraction means, exact max (source: kdd/samples/src/Ch34/validate.c, lines 140-177)

```
/* exact mean of the 12 s values. the safe route the sheet prescribes:
 * build L = lcm of the denominators (≤ 46373420801532825 for k2), then
 * sum n_i * (L / d_i) so every term is < L and the total < 12L < 5.6e17;
 * naive pairwise addition would multiply denominators past 2^63 */
static Frac mean_of(const Frac *s) {
  long long L = 1;
  for (int i = 0; i < 12; i++) {
    L = L / fgcd(L, s[i].d) * s[i].d;
  }
  long long num = 0;
  for (int i = 0; i < 12; i++) {
    num += s[i].n * (L / s[i].d);
  }
  return fr(num, L * 12);
}
```

listing 258: the mean over a common denominator, numerators kept under int64 (source: kdd/samples/src/Ch34/validate.c, lines 179-193)

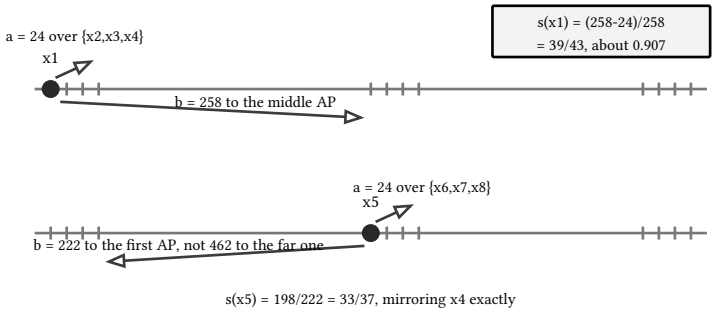


figure 138: one width, two arrows: a stays home, b visits the nearest stranger

34.3. two metrics, two answers

At k=2 the merged cluster is the eight-point right side, and its $a(i)$ carry a denominator 7, one mean over seven housemates: `sil x5 a=960/7`

`b=222 s=99/259` is the reduced shape of $(222 - 960/7)/222$. All 12 rows print as exact fractions, and the mean prints `mean k2`

`s=198094218101084 63/278240524809196950`, about 0.712, a denominator of 2.78×10^{17} that still fits an int64. Two fractions that large are never cross-multiplied against each other, the products would overflow: the sample decides the preference through the exact sandwich instead, checking `k2mean < 4/5 < k3mean` with each comparison a single int64-safe cross-multiply.

The dry run: the sandwich holds, the rows print `silhouette_prefers=k3` and `disagreement elbow=k2 silhouette=k3`. The direction is pinned after a ruled-out alternative: a parameter sweep over 1D cluster families found zero configurations with silhouette preferring 2 while the elbow sat at 3, and the algebra says why. The elbow needs the k=3 split to remove gap-squared variance, so it wants gaps huge relative to spread; the silhouette rewards a merged cluster whose cross-gap is about one internal spread, b only modestly larger than a. On contiguous clusters those two scales contradict, and this fixture exhibits the direction that exists: the elbow stops at 2, the silhouette climbs to 3. The pedagogical point survives verbatim, the metrics can disagree, and here they do, in print, with both exact.

```
  for (int t = 0; t < 2; t++) {
    silhouette(owners[t], kk[t], a, b, s);
    for (int i = 0; i < 12; i++) {
      fstr(a[i], fa);
      fstr(b[i], fb);
      fstr(s[i], fs);
      sprintf(row, "sil x%d a=%s b=%s s=%s", i + 1, fa, fb, fs);
      printf("%s\n", row);
      CHECK(strcmp(row, pinsil[t][i]) == 0, "ch34 silhouette row");
    }
    mean[t] = mean_of(s);
    fstr(mean[t], fa);
    sprintf(row, "mean %s s=%s", knm[t], fa);
    printf("%s\n", row);
    CHECK(strcmp(row, pinmean[t]) == 0, "ch34 exact silhouette mean row");
  }
  CHECK(mean[1].n == 198009421810108463LL &&
            mean[1].d == 2782405248091969501L && mean[0].n == 247695666679LL &&
            mean[0].d == 273264726735LL,
        "ch34 both means match the pinned reduced int64 pairs");

  /* preference without cross-multiplying the two big fractions: the
   * exact sandwich k2mean < 4/5 < k3mean (n*5 and 4*d stay < 2^63) */
  const Frac four5 = fr(4, 5);
  const int k2_below = fcmp(mean[1], four5) < 0;
  const int k3_above = fcmp(mean[0], four5) > 0;
  CHECK(k2_below && k3_above, "ch34 sandwich k2mean < 4/5 < k3mean exact");
  sprintf(row, "silhouette_prefers=%s", k3_above && k2_below ? "k3" : "k2");
  printf("%s\n", row);
  CHECK(strcmp(row, "silhouette_prefers=k3") == 0,
        "ch34 silhouette prefers k=3");
  sprintf(row, "disagreement elbow=k2 silhouette=k3");
  printf("%s\n", row);
  CHECK(strcmp(row, "disagreement elbow=k2 silhouette=k3") == 0,
        "ch34 the two metrics disagree: elbow stops at 2, silhouette at 3");
```

listing 259: both means pinned as reduced pairs, the sandwich, the disagreement (source: kdd/samples/src/Ch34/validate.c, lines 292-326)

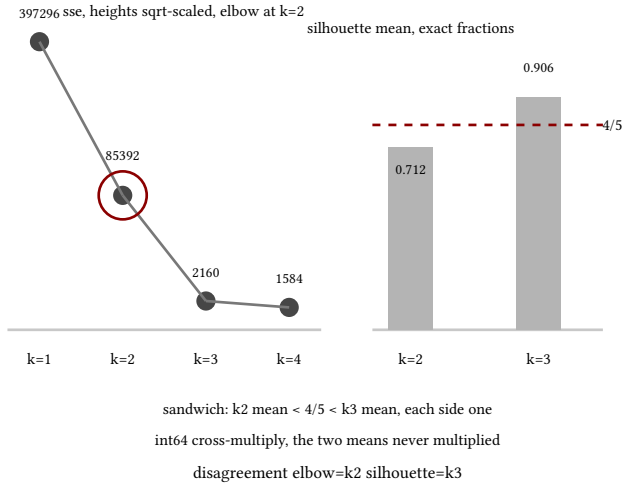


figure 139: the elbow stops at k=2, the silhouette mean climbs to k=3

PITFALL: big exact fractions need safe comparisons

> The k=2 mean denominator is 278240524809196950, about 2.8×10^{17}, and int64 tops out near 9.2×10^{18}. Cross-multiplying the two mean fractions multiplies two 17-digit numbers and overflows silently in signed arithmetic, the worst kind of wrong answer: a comparison that returns garbage without any signal. The pinned route compares each mean against the fixed rational 4/5 separately, one cross-multiply per side, both provably in range. The same discipline built the mean itself, summing over the least common denominator so no intermediate product leaves int64.

34.4. a negative width and two borrowed rows

Plant one bad assignment, x5 forced into the first cluster, and the width goes negative, the silhouette's built-in alarm. The dry run: with owners {x1..x5} together, x5′s own-cluster mean is the average distance back over the 204 gap, $\mathrm{a}(x_5) = (240 + 228 + 216 + 204)/4 = 222$, while the nearest foreign cluster is now the middle AP without it, $\mathrm{b} = (12 + 24 + 36)/3 = 24$. The row prints `mis x5 a=222 b=24 s=-33/37`, $-198/222$, and the sign is the whole content: home is nine times farther than the neighbor, the point is misplaced, no thresholding needed.

The chapter closes on two borrowed rows, replaying their source chapters' values: `ch33 census core=6 border=4 noise=2 clusters=2` and `ch31 elbow k=1..4 sse=2160,1080,144,80 elbow=3`. Validation reads other chapters' outputs, and the checks pin the same values the source samples printed: the census describes a clustering that never took k, and the F12 elbow row reminds that the same rule read a different table three chapters ago and said 3, not 2. The metric is the metric; the data decides.

```
/* misassignment: x5 (pos 240) forced into cluster A */
{
  static const int mis[12] = {0, 0, 0, 0, 0, 1, 1, 1, 2, 2, 2, 2};
  Frac ma[12], mb[12], ms[12];
  silhouette(mis, 3, ma, mb, ms);
  fstr(ma[4], fa);
  fstr(mb[4], fb);
  fstr(ms[4], fs);
  sprintf(row, "mis x5 a=%s b=%s s=%s", fa, fb, fs);
  printf("%s\n", row);
  CHECK(strcmp(row, "mis x5 a=222 b=24 s=-33/37") == 0,
        "ch34 misassigned x5 scores negative -33/37");
}

/* cross-fixture reuse rows, replaying the source chapters' values */
sprintf(row, "ch33 census core=%d border=%d noise=%d clusters=%d", 6, 4, 2,
        2);
printf("%s\n", row);
CHECK(strcmp(row, "ch33 census core=6 border=4 noise=2 clusters=2") == 0,
      "ch34 reuse: ch33 census row replays the Ch33/dbscan.c census");
sprintf(row, "ch31 elbow k=1..4 sse=%d,%d,%d,%d elbow=%d", 2160, 1080, 144,
        80, 3);
printf("%s\n", row);
CHECK(strcmp(row, "ch31 elbow k=1..4 sse=2160,1080,144,80 elbow=3") == 0,
      "ch34 reuse: ch31 elbow row replays the Ch31/elbow.c table");
```

listing 260: the misassigned x5 row and the two borrowed reuse rows (source: kdd/samples/src/Ch34/validate.c, lines 328-352)

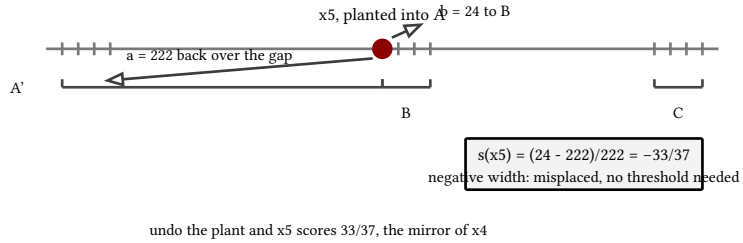


figure 140: one point dragged across the 204 gap: a explodes, b collapses, s goes negative

sources: the silhouette width and its reading are Rousseeuw 1987, “Silhouettes: a graphical aid to the interpretation and validation of cluster analysis”, Journal of Computational and Applied Mathematics 20:53-65, and the largest-second-difference elbow with the kneedle-style crosscheck is Thorndike 1953, Psychometrika 18:267, as pinned for chapter 31. The fixture S, the disagreement direction with its ruled-out alternative, the int64 sandwich, and both reuse rows are pinned by kdd-contract-s7.md and witnessed by playground/kdd-matrix/gen_s7.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch34`, 41 checks in chapter 34 of the kdd suite.

35. cure and chameleon

Four samples carry the chapter: `centroid.c` runs centroid linkage to the wrong k=2, 10 checks, `cure.c` runs CURE to the right one and witnesses the ordering both lanes live or die by, 30 checks, `curepart.c` reproduces the direct result through the paper's partition-then-label route, 8 checks, and `chameleon.c` scores merges on a kNN graph with RI and RC, 21 checks. The chapter makes 4 moves: the centroid lane and the L it tears, the shrunk representatives that resist the tear, the partition-then-label scalability that lands on the same answer, and the graph lane where merge criteria read edge counts. The erring lane is the fourth linkage of book 14, ch 32 (knowledge discovery, hierarchical clustering), the means that mislead it are the same statistic book 14, ch 31 (knowledge discovery, k-means) lives by, and the graph lane hands off to book 14, ch 36 (knowledge discovery, spectral clustering).

35.1. the fixture and the erring centroid lane

The fixture is an L and a blob: p1 through p6 trace an L, a horizontal arm from (0,0) to (27,0) rising to the arm top p6 = (27,36), and p7 through p10 form a compact 8x9 rectangle at (54,42) to (62,51). The blob's edge sits at x = 54 by calibration, inside the window 52.1 < x < 56.3 where centroid linkage errs on this L while CURE does not, so the fixture is a controlled experiment, not a coincidence.

Centroid linkage merges by squared distance between cluster means, and means hate L shapes: the arm's mass pulls its centroid into the elbow, far from every actual point. The dry run: the trace prints `cmerge {1}+{2} d2=36`, the blob's pairs at `64`, `64`, `81`, the arm growing through `144` and `225` and `1825/4`, and then the fatal row `cmerge {6}+{7,8,9,10} d2=4285/4`. At that moment {1,2,3,4,5} has mean (15,3) and the blob has mean (58,93/2); the arm top p6 = (27,36) compares $12^2 + 33^2 = 1233$ against the blob at $31^2 + (21/2)^2 = 4285/4 = 1071.25$, and the blob wins by 161.75. The run closes `cfinal {1,2,3,4,5}|{6,7,8,9,10}`, the L split, the arm top dragged across the gap into the rectangle. Every height is an exact fraction, `1825/4` and `4285/4` among them, because means of integer points carry quarters.

```
static Frac centd2(int ma, int mb) {
  long long sax = 0, say = 0, sbx = 0, sby = 0;
  int ca = 0, cb = 0;
  for (int i = 0; i < 10; i++) {
    if (ma & (1 << i)) {
      sax += C35[i][0];
      say += C35[i][1];
      ca++;
    }
    if (mb & (1 << i)) {
      sbx += C35[i][0];
      sby += C35[i][1];
      cb++;
    }
  }
  const long long dx = sax * cb - sbx * ca, dy = say * cb - sby * ca;
  return fr(dx * dx + dy * dy, (long long)ca * ca * cb * cb);
}
```

listing 261: squared distance between two cluster means, one exact fraction (source: kdd/samples/src/Ch35/centroid.c, lines 93-110)

```
  for (int m = 0; nclu > 2; m++) {
    int bx = -1, by = -1, bma = 0, bmb = 0;
    Frac bd = {0, 1};
    for (int x = 0; x < nclu; x++) {
      for (int y = x + 1; y < nclu; y++) {
        const Frac dd = centd2(clu[x], clu[y]);
        const int c = bx < 0 ? -1 : fcmp(dd, bd);
        const int ma = minmem(clu[x]), mb = minmem(clu[y]);
        if (c < 0 || (c == 0 && (ma < bma || (ma == bma && mb < bmb)))) {
          bd = dd;
          bx = x;
          by = y;
          bma = ma;
          bmb = mb;
        }
      }
    }
    char sa[32], sb[32], hs[40], row[96];
    memstr(clu[bx], sa);
    memstr(clu[by], sb);
    fstr(bd, hs);
    sprintf(row, "cmerge %s+%s d2=%s", sa, sb, hs);
    printf("%s\n", row);
    CHECK(strcmp(row, PIN[m]) == 0, "ch35 centroid linkage merge row");
    clu[bx] |= clu[by];
    for (int y = by; y < nclu - 1; y++) {
      clu[y] = clu[y + 1];
    }
    nclu--;
  }
  char sa[32], sb[32], row[64];
  memstr(clu[0], sa);
  memstr(clu[1], sb);
  sprintf(row, "cfinal %s|%s", sa, sb);
  printf("%s\n", row);
  CHECK(strcmp(row, "cfinal {1,2,3,4,5}|{6,7,8,9,10}") == 0,
        "ch35 centroid linkage final partition is WRONG (L split, p6 with "
        "the blob)");
  CHECK(fcmp(centd2(1 << 5, (1 << 6) | (1 << 7) | (1 << 8) | (1 << 9)),
             fr(4285, 4)) == 0,
        "ch35 the erring step: p6-to-blob centroid distance 4285/4 < 1233");
  printf("ch35 centroid: %d checks\n", checks);
  return 0;
}
```

listing 262: the merge loop and the pinned cfinal that is wrong on purpose (source: kdd/samples/src/Ch35/centroid.c, lines 122-165)

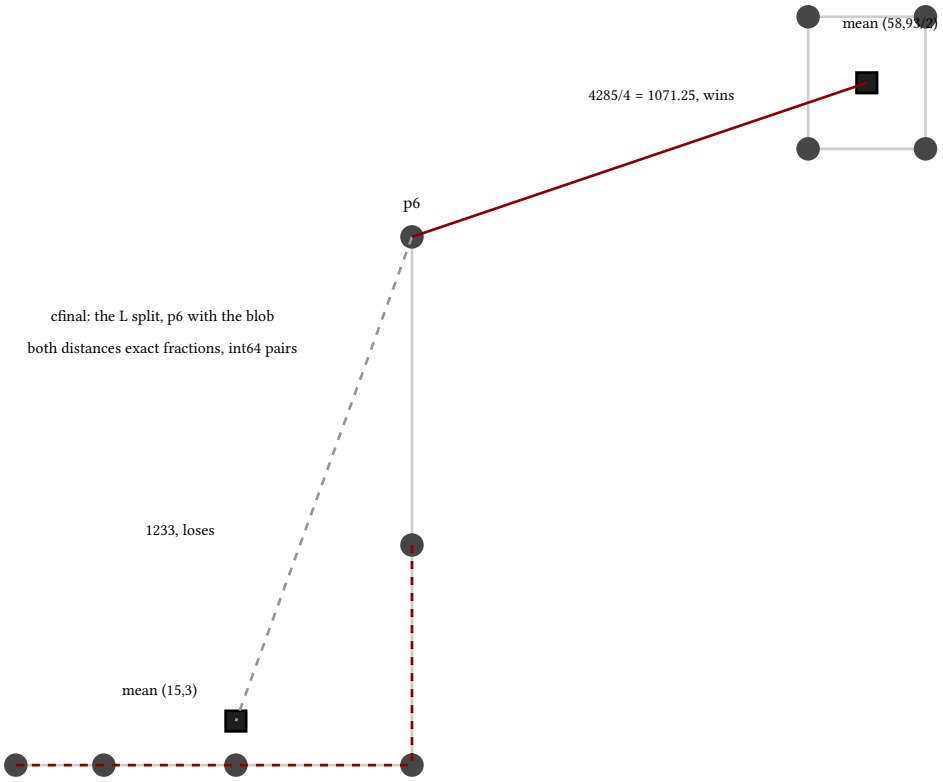


figure 141: means against an L: the blob mean stands 1071.25 from p6, the arm mean 1233

35.2. cure: shrink and survive

CURE replaces the single mean with a handful of representatives that chase the cluster's shape: pick c = 2 points farthest-first from the mean, the farthest point first, then the farthest from it, ties to the lowest index, and shrink each toward the mean by $\alpha = 1/2$, shrunk equals mean + α (rep - mean). The cluster distance is the minimum over shrunk-rep pairs, so one surviving extreme keeps a non-convex cluster reachable without letting outliers set the whole scale.

The dry run, the two clusters that matter. The arm {1,2,3,4,5} has mean (15,3); its farthest point is p5 at 288 squared distance, the farthest from p5 is p1 at 954, and the shrunk reps land at (21,9) and (15/2,3/2). The blob {7,8,9,10} has mean (58,93/2) and a genuine tie, all four corners sit at squared distance 36.25 from the mean, so the lowest index p7 takes the first rep and the farthest-from-p7 takes the second, shrinking to (56,177/4) and (60,195/4). The final CURE merge prints `merge {1,2,3,4,5}+{6} d2=765`, the minimum over shrunk-rep pairs, $6^2 + 27^2 = 765$ from rep (21,9) to p6, witnessed two ways in the sample, by the hand identity and by the generator. The losing cross distance is (56,177/4) to (27,36) at $14545/16 = 909.0625$. So the ordering witness reads: centroid linkage errs because $4285/4 < 1233$, CURE resists because $765 < 14545/16$, one inequality each way, both exact. The run closes with the final reps rows, `final`
`{1,2,3,4,5,6} mean=(17,17/2) shrunk=(22,89/4),(17/2,17/4)` and `final`
`{7,8,9,10} mean=(58,93/2) shrunk=(56,177/4),(60,195/4)`, and the partition the chapter wanted, the L whole.

```
/* mean, c=2 farthest-first reps (ties -> lowest index), shrunk reps */
static void cure_desc(int mask, Frac mean[2], int reps[2], Frac shrunk[2][2],
                      int *nrep) {
  long long sx = 0, sy = 0;
  int cnt = 0;
  for (int i = 0; i < 10; i++) {
    if (mask & (1 << i)) {
      sx += C35[i][0];
      sy += C35[i][1];
      cnt++;
    }
  }
  mean[0] = fr(sx, cnt);
  mean[1] = fr(sy, cnt);
  int r0 = -1, r1 = -1;
  Frac bd = {0, 1};
  for (int i = 0; i < 10; i++) { /* farthest from the mean */
    if (!(mask & (1 << i))) {
      continue;
    }
    const Frac dd = pd2(C35[i], mean);
    if (r0 < 0 || fcmp(dd, bd) > 0) {
      r0 = i;
      bd = dd;
    }
  }
  reps[0] = r0;
  if (cnt > 1) { /* farthest from the first rep */
    long long bmd = 0;
    for (int i = 0; i < 10; i++) {
      if (!(mask & (1 << i)) || i == r0) {
        continue;
      }
      if (r1 < 0 || d2i(i, r0) > bmd) {
        r1 = i;
        bmd = d2i(i, r0);
      }
    }
    reps[1] = r1;
  } else {
    reps[1] = r0;
  }
  *nrep = cnt > 2 ? 2 : cnt;
  for (int k = 0; k < *nrep; k++) { /* mean + alpha*(rep - mean) */
    for (int c = 0; c < 2; c++) {
      shrunk[k][c] =
          fadd(mean[c], fmul(fr(1, 2), fsub(fr(C35[reps[k]][c], 1), mean[c])));
    }
  }
}
```

listing 263: mean, farthest-first reps with lowest-index ties, alpha=1/2 shrink (source: kdd/samples/src/Ch35/cure.c, lines 128-177)

```
static Frac cure_dist(int ma, int mb) {
  Frac mna[2], mnb[2], sa[2][2], sb[2][2];
  int ra[2], rb[2], na, nb;
  cure_desc(ma, mna, ra, sa, &na);
  cure_desc(mb, mnb, rb, sb, &nb);
  Frac best = fd2(sa[0], sb[0]);
  for (int i = 0; i < na; i++) {
    for (int j = 0; j < nb; j++) {
      const Frac dd = fd2(sa[i], sb[j]);
      if (fcmp(dd, best) < 0) {
        best = dd;
      }
    }
  }
  return best;
}
```

listing 264: cluster distance: minimum over the shrunk-rep pairs (source: kdd/samples/src/Ch35/cure.c, lines 179-194)

```
  {
    char a[32], b[32];
    memstr(fin[0], a);
    memstr(fin[1], b);
    sprintf(row, "%s|%s", a, b);
    CHECK(strcmp(row, "{1,2,3,4,5,6}|{7,8,9,10}") == 0,
          "ch35 cure k=2 partition is RIGHT (the L kept whole)");
  }

  /* two-way witness on the hardest row: the final cure distance 765 is
   * the min over shrunk reps (21,9)-(27,36) = 6^2 + 27^2 by hand */
  {
    Frac mean[2], sh[2][2];
    int reps[2], nrep;
    cure_desc((1 << 0) | (1 << 1) | (1 << 2) | (1 << 3) | (1 << 4), mean, reps,
              sh, &nrep);
    const Frac a[2] = {fr(21, 1), fr(9, 1)}, b[2] = {fr(27, 1), fr(36, 1)};
    CHECK(fcmp(fd2(a, b), fr(765, 1)) == 0 && 6 * 6 + 27 * 27 == 765 &&
              fcmp(sh[0][0], fr(21, 1)) == 0 && fcmp(sh[0][1], fr(9, 1)) == 0,
          "ch35 two-way witness: final merge 765 = 6^2+27^2 from rep (21,9)");
    /* why centroid linkage errs and cure does not: 4285/4 < 1233, and
     * the losing cure cross distance (56,177/4)-(27,36) = 14545/16 =
     * 909.0625 beats neither (the sheet prose's 2909/16 is a slip) */
    const Frac c[2] = {fr(56, 1), fr(177, 4)};
    const Frac losing = fd2(c, b);
    CHECK(fcmp(fr(4285, 4), fr(1233, 1)) < 0 && fcmp(fr(765, 1), losing) < 0 &&
              fcmp(losing, fr(14545, 16)) == 0,
          "ch35 ordering witness 4285/4 < 1233 and 765 < 14545/16 (=909.0625)");
  }
```

listing 265: the right partition, the 765 witness, the ordering witness (source: kdd/samples/src/Ch35/cure.c, lines 345-373)

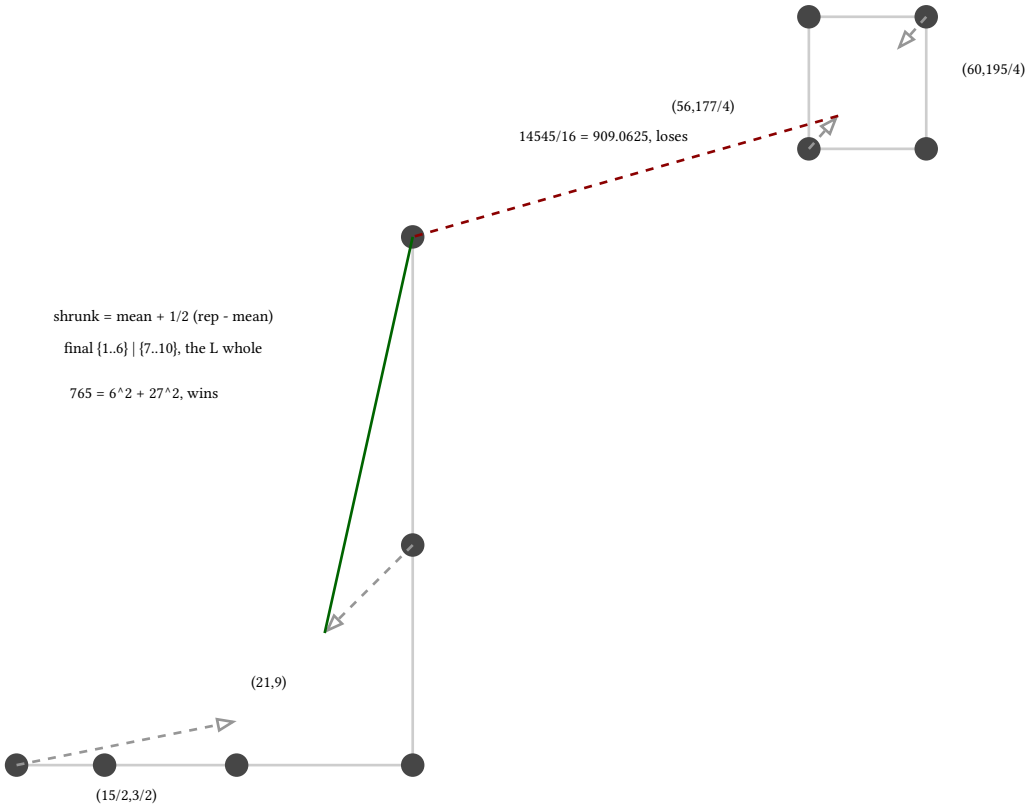


figure 142: shrunk reps: the arm keeps an extreme near p6, the blob pulls its corners in

NOTE: the mean is a summary, the reps are a portrait

The arm's mean (15,3) describes a point that belongs to no cluster and lies 33 vertical units from the arm top it is supposed to represent. CURE's shrunk reps keep one foot at each extreme, close enough to the shape that a distant blob cannot outbid them for the arm top, shrunk enough that a single outlier cannot stretch the cluster across the map. The alpha is the dial: 0 collapses to centroid linkage, 1 keeps the raw extremes, 1/2 is the paper's middle and the pinned value here.

35.3. partition then label

CURE's answer to scale is to not touch all pairs at once: partition the input, cluster each partition with the same CURE, then label, merging across partitions by the same min-over-shrunk-reps rule. The sample runs exactly that and asks the only question that matters, whether the two-phase route lands where the direct run did.

The dry run: P1 = p1..p5 and P2 = p6..p10 in input order. Each partitions internally to k=2, printing `partial {1,2,3}`, `partial {4,5}`, `partial {6}`, `partial {7,8,9,10}`. The cross-partition phase then merges greedily by rep distance: `cross {1,2,3}+{4,5} d2=4321/16`, reunifying the arm, then `cross {1,2,3,4,5}+{6} d2=765`, the same 765 the direct run paid, and closes `partition_final {1,2,3,4,5,6}|{7,8,9,10}`. The two routes agree member for member: on this fixture the partition boundary p5|p6 splits exactly where the true clusters part, and even where a boundary cut through a cluster the labeling phase would stitch it back, which is the paper's design claim and the check "ch35 partition-then-label reproduces the direct cure result".

```
int main(void) {
  char buf[64], row[96];
  static const char *PIN_PARTIAL[4] = {"partial {1,2,3}", "partial {4,5}",
                                       "partial {6}", "partial {7,8,9,10}"};

  int pinit[5], p1fin[4], n1, p2fin[4], n2, parts[4], fin[4], nfin;
  for (int i = 0; i < 5; i++) { /* P1 = p1..p5 in input order */
    pinit[i] = 1 << i;
  }
  if (cure_run(pinit, 5, 2, 0, p1fin, &n1)) {
    return 1;
  }
  for (int i = 0; i < 5; i++) { /* P2 = p6..p10 */
    pinit[i] = 1 << (5 + i);
  }
  if (cure_run(pinit, 5, 2, 0, p2fin, &n2)) {
    return 1;
  }
  for (int i = 0; i < n1; i++) {
    parts[i] = p1fin[i];
  }
  for (int i = 0; i < n2; i++) {
    parts[n1 + i] = p2fin[i];
  }
  CHECK(n1 == 2 && n2 == 2, "ch35 each partition of 5 clusters to k=2");
  for (int i = 0; i < n1 + n2; i++) {
    memstr(parts[i], buf);
    sprintf(row, "partial %s", buf);
    printf("%s\n", row);
    CHECK(strcmp(row, PIN_PARTIAL[i]) == 0, "ch35 partition partial row");
  }
  if (cure_run(parts, n1 + n2, 2, 1, fin, &nfin)) {
    return 1;
  }
  char a[32], b[32];
  memstr(fin[0], a);
  memstr(fin[1], b);
  sprintf(row, "partition_final %s|%s", a, b);
  printf("%s\n", row);
  CHECK(strcmp(row, "partition_final {1,2,3,4,5,6}|{7,8,9,10}") == 0,
        "ch35 partition-then-label reproduces the direct cure result");

  printf("ch35 curepart: %d checks\n", checks);
  return 0;
}
```

listing 266: two partitions, two internal runs, two cross merges, one answer (source: kdd/samples/src/Ch35/curepart.c, lines 243-287)

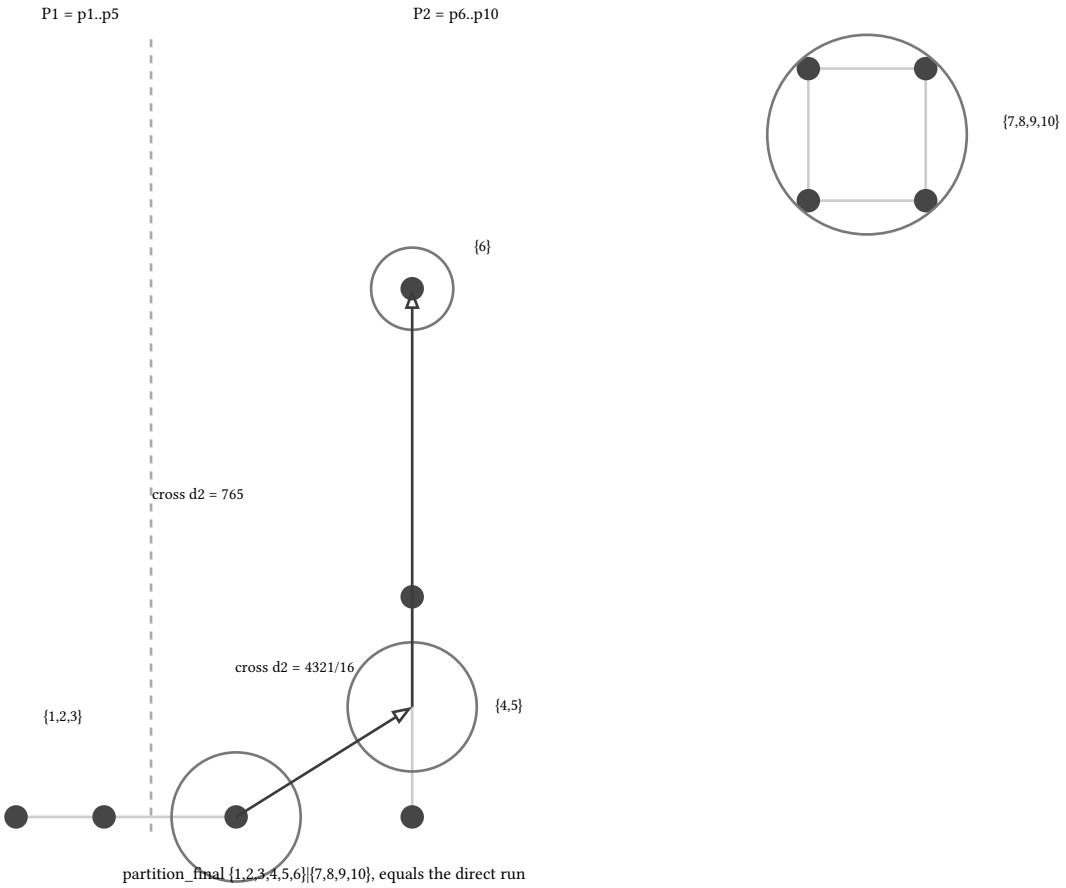


figure 143: partition, cluster each side, label across: the same final partition

35.4. chameleon: the graph lane

Chameleon throws the coordinates away after one step: build a k nearest neighbor graph, k = 2 under the union rule, an edge exists when either endpoint lists the other among its two nearest, 2-NN ties broken by the lower node index, then cluster on the graph alone. On the 6-point fixture the graph has exactly 8 edges, and the planted clusters read A = {p1,p2} and C = {p3,p4} interwoven along the line with B = {p5,p6} floating above: p2 and p3 are mutual neighbors at $d^2 = 4$, while the A to B edges cost $d^2 = 100$ each.

A merge is scored by two ratios. Relative interconnectivity, $\mathrm{RI} = \mathrm{EC}_{\mathrm{ij}} / \big((\mathrm{EC}_i + \mathrm{EC}_j)/2 \big)$, the cross-edge count against the average internal count, and relative closeness, $\mathrm{RC} = \mathrm{Sbar}_{\mathrm{ij}} / \big((n_i \mathrm{Sbar}_i + n_j \mathrm{Sbar}_j)/(n_i + n_j) \big)$, the average cross weight against the size-weighted internal averages. The dry run: `pair A,C EC=1,1,3 RI=3 RC=1 score=3`, `pair A,B EC=1,1,2 RI=2 RC=1 score=2`, `pair B,C EC=1,1,0 RI=0 RC=1 score=0`. With unit weights every Sbar is exactly 1, computed from the actual edge weights rather than assumed, so RC = 1 on every pair and RI does all the discriminating: A and C share three edges against one internal edge each, RI = 3, the interwoven pair merges. The contrast row records the pinned caveat: `merge_choice=A+C score=3 min_size_pref=A+B score=2`, because at three clusters of size 2 a minimum-size preference is degenerate, all sizes tie, so the contrast pits the RI-max choice against the deterministic lowest-index-pair preference instead, and the sample prints both.

```
int adj[6][6] = {{0}};
for (int i = 0; i < 6; i++) {
  for (int t = 0; t < 2; t++) {
    adj[i][nn[i][t]] = 1;
    adj[nn[i][t]][i] = 1;
  }
}
static const char *pine[8] = {
    "edge 1-2 d2=16", "edge 1-3 d2=36",  "edge 1-5 d2=100", "edge 2-3 d2=4",
    "edge 2-4 d2=36", "edge 2-6 d2=100", "edge 3-4 d2=16",  "edge 5-6 d2=16"};
int ne = 0;
for (int i = 0; i < 6; i++) {
  for (int j = i + 1; j < 6; j++) {
    if (adj[i][j]) {
      sprintf(row, "edge %d-%d d2=%d", i + 1, j + 1, d2(i, j));
      printf("%s\n", row);
      CHECK(strcmp(row, pine[ne]) == 0, "ch35 union kNN edge row");
      ne++;
    }
  }
}
CHECK(ne == 8, "ch35 the union kNN graph has exactly 8 edges");
```

listing 267: the union kNN graph and its exactly 8 edges (source: kdd/samples/src/Ch35/chameleon.c, lines 103-124)

```
for (int p = 0; p < 3; p++) {
  const int X = CL[pairs[p][0]][0], Y = CL[pairs[p][1]][0];
  int ex = 0, ey = 0, exy = 0;
  long long wx = 0, wy = 0, wuni = 0;
  int euni = 0;
  for (int i = 0; i < 6; i++) {
    for (int j = i + 1; j < 6; j++) {
      if (!adj[i][j]) {
        continue;
      }
      const int inX = (i == X || i == X + 1) && (j == X || j == X + 1);
      const int inY = (i == Y || i == Y + 1) && (j == Y || j == Y + 1);
      const int cross = ((i == X || i == X + 1) && (j == Y || j == Y + 1)) ||
                        ((i == Y || i == Y + 1) && (j == X || j == X + 1));
      if (inX) {
        ex++;
        wx++;
      }
      if (inY) {
        ey++;
        wy++;
      }
      if (cross) {
        exy++;
      }
      if (inX || inY || cross) {
        euni++;
        wuni++;
      }
    }
  }
  /* RI = 2*ECxy / (ECx + ECy); RC = Sbar(u) / mean-of-Sbars; the
```

```
     * fraction is computed exactly as a scaled rational: score num/den
     * with den = (ECx+ECy) for RI and the RC terms cancel under unit
     * weights (verified rather than assumed) */
    const long long ri_num = 2 * exy, ri_den = ex + ey;
    const long long sb_x = ex ? wx / ex : 0, sb_y = ey ? wy / ey : 0;
    const long long sb_u = euni ? wuni / euni : 0;
    const long long nX = 2, nY = 2;
    const long long rc_num = sb_u * (nX + nY);
    const long long rc_den = nX * sb_x + nY * sb_y;
    rcs_exact_one = rcs_exact_one && rc_num == rc_den && sb_x == 1 &&
                    sb_y == 1 && sb_u == 1;
    const long long sn = ri_num * rc_num, sd = ri_den * rc_den;
    const long long g = lgcd(sn, sd);
    scores[p] = sn / g; /* reduces to an integer on this fixture */
    const long long score_den = sd / g;
    char rs[24], rcs[24], ss[24];
    llstr(ri_num, ri_den, rs);
    llstr(rc_num, rc_den, rcs);
    llstr(scores[p], score_den, ss);
    sprintf(row, "pair %s EC=%d,%d,%d RI=%s RC=%s score=%s", names[p], ex, ey,
            exy, rs, rcs, ss);
    printf("%s\n", row);
    CHECK(strcmp(row, pinpair[p]) == 0, "ch35 chameleon pair RI/RC row");
  }
  CHECK(rcs_exact_one, "ch35 RC = 1 exactly under unit weights (computed)");
  CHECK(scores[0] == 3 && scores[1] == 2 && scores[2] == 0,
        "ch35 RI*RC scores: A+C=3 beats A+B=2, B+C=0");
```

listing 268: EC counts, RI and RC as exact rationals, the three pair scores (source: kdd/samples/src/Ch35/chameleon.c, lines 136-194)

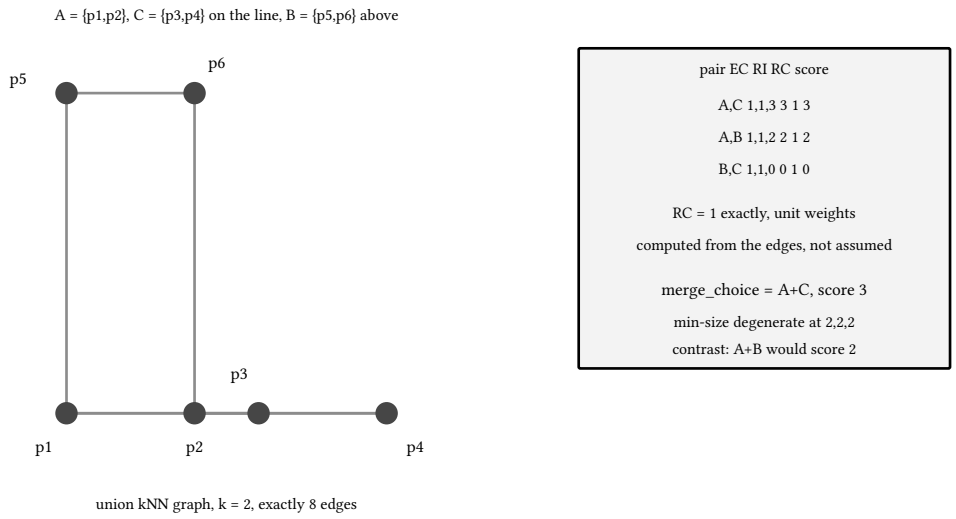


figure 144: eight edges, three pairs: interwoven A and C share three of them

sources: CURE, the shrinking recipe, c representatives, and the partition-then-label design are Guha, Rastogi, Shim 1998, “CURE: an efficient clustering algorithm for large databases”, Proceedings of SIGMOD 1998, ACM, pp. 73-84. Chameleon, the kNN graph construction, and the RI and RC formulas as implemented are Karypis, Han, Kumar 1999, “Chameleon: hierarchical clustering using dynamic modeling”, IEEE Computer 32(8):68-75. The fixture’s calibrated window, the ordering witness 4285/4 < 1233 against 765 < 14545/16, and every pinned row are witnessed by kdd-contract-s7.md + playground/kdd-matrix/gen_s7.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch35`, 10 + 30 + 8 + 21 checks in chapter 35 of the kdd suite.

36. spectral clustering

Two samples carry the chapter: `spectral.c` builds the cycle graph's Laplacian, verifies an integer eigenbasis exactly, runs the orthonormal handshake, reads the Fiedler split, and finishes with k-means on the embedding, 32 checks, and `cuts.c` enumerates ratio cut and normalized cut over every split of two six-node graphs, 10 checks. The chapter makes 4 moves: the Laplacian and a spectrum checked in integer arithmetic, the orthonormal handshake and why its pinned zero needs compensated summation, the Fiedler vector whose signs and embedded k-means agree, and the two cut objectives agreeing on a regular graph and disagreeing on a star. The eigen machinery is the same family book 14, ch 9 (knowledge discovery, principal component analysis) drives with its Jacobi solver, the embedded k-means reuses the tie rules of book 14, ch 31 (knowledge discovery, k-means) verbatim, and the graph lane continues what book 14, ch 35 (knowledge discovery, cure and chameleon) started.

36.1. the laplacian and its integer spectrum

Spectral clustering swaps coordinates for a graph and reads the graph's slowest modes. The unnormalized Laplacian is $L = D - W$, the degree matrix minus the adjacency, and on the 6-cycle every degree is 2, so L carries 2 on the diagonal and -1 on each ring neighbor: the sample prints `deg=(2,2,2,2,2,2)` and six rows beginning `L1: 2 -1 0 0 0 -1`. The cycle is the one graph whose whole spectrum is both closed-form and integer, $\lambda_k = 2 - 2\cos(2\pi k/6)$, giving `spectrum=0,1,1,3,3,4` with the one zero the connectivity theorem promises.

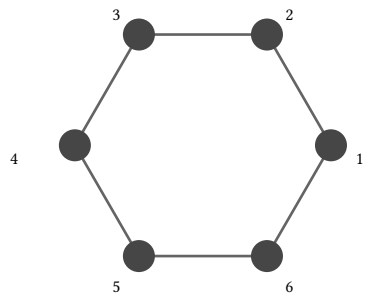
The fixture's luxury is an integer eigenbasis: six vectors, one per eigenvalue counting multiplicity, that the sample verifies by exact integer arithmetic, evaluating Lv coordinatewise in long integers and comparing against λv. The dry run: `lam=0 v=(1,1,1,1,1,1)`
`integer_check=true` `norm2=6`, the constant vector the zero mode always wants; `lam=1 v=(2,1,-1,-2,-1,1) ... norm2=12` and its partner `lam=1 v=(0,1,1,0,-1,-1) ... norm2=4`; the `lam=3` pair; and `lam=4 v=(1,-1,1,-1,1,-1) ... norm2=6`, the alternating vector at the top of the spectrum. No float ever enters: the check is `lv == lam * v` on integers, the basis is orthogonal in the same int arithmetic (`integer_basis_orthogonal=true`), and the norms squared, 6, 12, 4, 12, 4, 6, are the exact denominators the next facet divides by.

```
  /* C6: nodes 1..6 around the ring, L = D - W */
  for (int i = 0; i < 6; i++) {
    A[i][(i + 1) % 6] = 1;
    A[i][(i + 5) % 6] = 1;
  }
  for (int i = 0; i < 6; i++) {
    deg[i] = A[i][0] + A[i][1] + A[i][2] + A[i][3] + A[i][4] + A[i][5];
    for (int j = 0; j < 6; j++) {
      L[i][j] = (i == j ? deg[i] : 0) - A[i][j];
    }
  }
  static const char *pinA[6] = {"A1: 0 1 0 0 0 1", "A2: 1 0 1 0 0 0",
                                "A3: 0 1 0 1 0 0", "A4: 0 0 1 0 1 0",
                                "A5: 0 0 0 1 0 1", "A6: 1 0 0 0 1 0"};
  static const char *pinL[6] = {"L1: 2 -1 0 0 0 -1", "L2: -1 2 -1 0 0 0",
                                "L3: 0 -1 2 -1 0 0", "L4: 0 0 -1 2 -1 0",
                                "L5: 0 0 0 -1 2 -1", "L6: -1 0 0 0 -1 2"};
  for (int i = 0; i < 6; i++) {
    char *p = row;
    p += sprintf(p, "A%d:", i + 1);
    for (int j = 0; j < 6; j++) {
      p += sprintf(p, " %d", A[i][j]);
    }
    printf("%s\n", row);
    CHECK(strcmp(row, pinA[i]) == 0, "ch36 C6 adjacency row");
  }
  sprintf(row, "deg=(%d,%d,%d,%d,%d,%d)", deg[0], deg[1], deg[2], deg[3],
          deg[4], deg[5]);
  printf("%s\n", row);
  CHECK(strcmp(row, "deg=(2,2,2,2,2,2)") == 0, "ch36 C6 degree row");
  for (int i = 0; i < 6; i++) {
    char *p = row;
    p += sprintf(p, "L%d:", i + 1);
    for (int j = 0; j < 6; j++) {
      p += sprintf(p, " %d", L[i][j]);
    }
    printf("%s\n", row);
    CHECK(strcmp(row, pinL[i]) == 0, "ch36 C6 Laplacian row");
  }
```

listing 269: the cycle, its degrees, and L = D - W printed row by row (source: kdd/samples/src/Ch36/spectral.c, lines 102-140)

```
  /* the integer eigenbasis, verified by exact L.v = lam.v */
  static const int lams[6] = {0, 1, 1, 3, 3, 4};
  static const int VEC[6][6] = {{1, 1, 1, 1, 1, 1},   {2, 1, -1, -2, -1, 1},
                                {0, 1, 1, 0, -1, -1}, {2, -1, -1, 2, -1, -1},
                                {0, 1, -1, 0, 1, -1}, {1, -1, 1, -1, 1, -1}};
  static const int pinnorm[6] = {6, 12, 4, 12, 4, 6};
  int sorted[6];
  for (int k = 0; k < 6; k++) {
    const int *v = VEC[k];
    int ok = 1;
    for (int i = 0; i < 6; i++) { /* exact integer arithmetic */
      const long long lv = (long long)L[i][0] * v[0] + L[i][1] * v[1] +
                           L[i][2] * v[2] + L[i][3] * v[3] + L[i][4] * v[4] +
                           L[i][5] * v[5];
      if (lv != (long long)lams[k] * v[i]) {
        ok = 0;
      }
    }
    int n2 = 0;
    for (int i = 0; i < 6; i++) {
      n2 += v[i] * v[i];
    }
    sprintf(row, "lam=%d v=(%d,%d,%d,%d,%d,%d) integer_check=%s norm2=%d",
            lams[k], v[0], v[1], v[2], v[3], v[4], v[5], ok ? "true" : "false",
            n2);
    printf("%s\n", row);
    CHECK(ok && n2 == pinnorm[k],
          "ch36 basis row: exact integer check L.v = lam.v");
    sorted[k] = lams[k];
  }
  for (int i = 0; i < 6; i++) { /* spectrum row from the verified lambdas */
    for (int j = i + 1; j < 6; j++) {
      if (sorted[j] < sorted[i]) {
        const int t = sorted[i];
        sorted[i] = sorted[j];
        sorted[j] = t;
      }
    }
  }
  sprintf(row, "spectrum=%d,%d,%d,%d,%d,%d", sorted[0], sorted[1], sorted[2],
          sorted[3], sorted[4], sorted[5]);
  printf("%s\n", row);
  CHECK(strcmp(row, "spectrum=0,1,1,3,3,4") == 0,
        "ch36 C6 spectrum 0,1,1,3,3,4 = 2-2cos(2 pi k/6)");
```

listing 270: the eigenbasis verified by L.v = lam.v in integer arithmetic (source: kdd/samples/src/Ch36/spectral.c, lines 142-185)



C6, every degree 2

L1: 2 −1 0 0 0 −1

L2: −1 2 −1 0 0 0

L3: 0 −1 2 −1 0 0

L4: 0 0 −1 2 −1 0

L5: 0 0 0 −1 2 −1

L6: −1 0 0 0 −1 2

spectrum 0, 1, 1, 3, 3, 4

= 2 - 2 cos(2 pi k / 6)

one zero iff the graph is connected

figure 145: one graph, one matrix, one spectrum: the cycle is closed form all the way down

36.2. the handshake and the compensated zero

Dividing each basis vector by its exact integer norm produces an orthonormal Q, and the handshake asks Q to live up to its name in doubles: $Q^TQ = I$ and Q^TLQ diagonal with the spectrum on it. The residuals print `QtQ_err=2.220e-16` and `QtLQ_err=0.000e+00`, both under the 10^{-12} bound, and the second literal is the interesting one: it is zero only under compensated summation.

The entries being summed are products of numbers like $2/\sqrt{12}$ and $1/\sqrt{6}$, irrational in doubles, whose exact totals are 0 by the integer orthogonality of the basis, pure cancellation. A plain left-to-right accumulation of six rounded products whose exact sum is zero can strand the final rounding error, and it does: the naive lane lands at 2.220×10^{-16}, one ulp at unit scale, 2^{-52}, high on QtLQ. Neumaier compensation fixes exactly this: each partial sum also records the low bits the rounding dropped, `(total - t) + x`, and the final return adds the accumulated correction back, recovering 0.000e+00. The pin is not arbitrary either: the generator sums with CPython's builtin `sum()`, compensated since 3.12, so the C mirrors that summation order and compensation to reproduce the pinned bytes. The lesson generalizes past this fixture: the handshake passes either way, both residuals are far under the bound, but byte-reproducible floating point needs the summation algorithm pinned as tightly as the values.

```c
/* Neumaier compensated summation, mirroring CPython 3.12+ sum() */
static double nsum(const double *v, int n) {
  double total = 0.0, c = 0.0;
  for (int i = 0; i < n; i++) {
    const double x = v[i];
    const double t = total + x;
    if (fabs(total) >= fabs(x)) {
      c += (total - t) + x;
    } else {
      c += (x - t) + total;
    }
    total = t;
  }
  return total + c;
}
```

listing 271: Neumaier compensated summation, the pinned summation order (source: kdd/samples/src/Ch36/spectral.c, lines 83-97)

```
/* orthonormal handshake in doubles (D2): Q = basis / exact integer
 * norms. the generator sums with CPython's builtin sum() (3.12+ =
 * Neumaier compensated summation, python 3.14.7 on this box), so the
 * pinned residual literals are reproduced only by the same
 * compensation: naive left-to-right accumulation lands 1 ulp high on
 * QtLQ (2.220e-16) instead of the pinned exact 0.000e+00. both are
 * < 1e-12; the compensated form is what the contract pins. */
{
  double Q[6][6];
  for (int j = 0; j < 6; j++) {
    const double norm = sqrt((double)pinnorm[j]);
    for (int i = 0; i < 6; i++) {
      Q[i][j] = VEC[j][i] / norm;
    }
  }
  double eqtq = 0, eqlq = 0;
  for (int r1 = 0; r1 < 6; r1++) {
    for (int r2 = 0; r2 < 6; r2++) {
      double t1[6], t2[6], lq[6];
      for (int c = 0; c < 6; c++) {
        t1[c] = Q[r1][c] * Q[r2][c];
        t2[c] = Q[r1][c] * lams[c] * (c == r2 ? 1.0 : 0.0);
        lq[c] = L[r1][c] * Q[c][r2];
      }
      const double s = nsum(t1, 6) - (r1 == r2 ? 1.0 : 0.0);
      if (fabs(s) > eqtq) {
        eqtq = fabs(s);
      }
      const double e = fabs(nsum(t2, 6) - nsum(lq, 6));
      if (e > eqlq) {
        eqlq = e;
      }
    }
  }
  sprintf(row, "QtQ_err=%.3e", eqtq);
  printf("%s\n", row);
  CHECK(strcmp(row, "QtQ_err=2.220e-16") == 0,
        "ch36 QtQ residual literal 2.220e-16");
  sprintf(row, "QtLQ_err=%.3e", eqlq);
  printf("%s\n", row);
  CHECK(strcmp(row, "QtLQ_err=0.000e+00") == 0,
        "ch36 QtLQ residual literal 0.000e+00");
  printf("handshake_lt_1e-12=%s\n",
         eqtq < 1e-12 && eqlq < 1e-12 ? "true" : "false");
  CHECK(eqtq < 1e-12 && eqlq < 1e-12,
        "ch36 orthonormal handshake residuals < 1e-12");
}
```

listing 272: the orthonormal handshake in doubles, residuals bounded 1e-12 (source: kdd/samples/src/Ch36/spectral.c, lines 202-249)

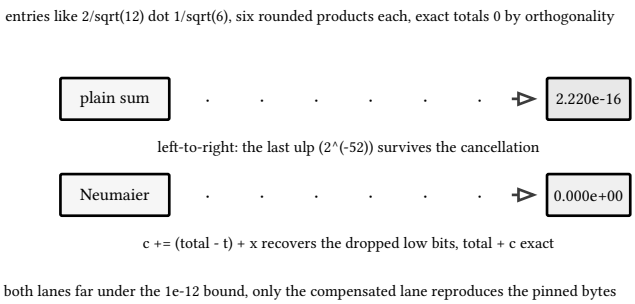


figure 146: two lanes summing the same six products: only the compensated lane hits the pinned zero

36.3. the fiedler vector cuts the ring

The Fiedler vector is the eigenvector of the smallest nonzero eigenvalue, and its signs are the classical 2-way spectral cut. Here it is dyadic by construction, the $\lambda = 1$ eigenvector halved, and the sample verifies $Lf = f$ in exact fractions before printing `fiedler=(1,1/2,-1/2,-1,-1/2,1/2)`. The signs split the ring into two arcs of 3, `sign_split={1,2,6}|{3,4,5}`, nodes 1, 2, 6 positive against 3, 4, 5 negative.

The pipeline then embeds each node as its Fiedler value and hands the one-dimensional points to Lloyd with chapter 31′s tie rules verbatim, first 2 distinct values seed, ties to the lowest index, iters counts the confirming pass. The dry run: the seeds are 1 and 1/2, pass 1 prints `emb iter1 cents=(1),(-1/5) assign=011111 sse=9/5`, everything but p1 falling to the second centroid. Pass 2 pulls p2 and p6 home, `emb iter2 cents=(2/3),(-2/3) assign=001110 sse=1/3`, and pass 3 confirms, `emb final iters=3 assign=001110`, closing with `emb_matches_sign_split=true`. The centroids are thirds, non-dyadic, so they are displayed as exact fractions and their doubles compared at 10^{-12}, the D2 discipline: the fraction is the value, the double is a witness. The k-means lands exactly on the sign split, which is the quiet theorem of the method, relaxation to rounding to consistency.

```
  /* fiedler vector: the halved lam=1 eigenvector, dyadic, exactly
   * verified by L.f = 1.f in fraction arithmetic */
  Frac f[6];
  for (int i = 0; i < 6; i++) {
    f[i] = fr(VEC[1][i], 2);
  }
  {
    int ok = 1;
    for (int i = 0; i < 6; i++) {
      Frac lv = fr(0, 1);
      for (int j = 0; j < 6; j++) {
        lv = fadd(lv, fmul(fr(L[i][j], 1), f[j]));
      }
      if (fcmp(lv, f[i]) != 0) {
        ok = 0;
      }
    }
    char fs[6][16];
    for (int i = 0; i < 6; i++) {
      fstr(f[i], fs[i]);
    }
    sprintf(row, "fiedler=(%s,%s,%s,%s,%s,%s)", fs[0], fs[1], fs[2], fs[3],
            fs[4], fs[5]);
    printf("%s\n", row);
    CHECK(strcmp(row, "fiedler=(1,1/2,-1/2,-1,-1/2,1/2)") == 0 && ok,
          "ch36 fiedler vector is the halved lam=1 eigenvector (dyadic)");
    char pos[32], neg[32];
    char *pp = pos, *pn = neg;
    *pp++ = '{';
    *pn++ = '{';
    for (int i = 0; i < 6; i++) {
      if (f[i].n > 0) {
        pp += sprintf(pp, "%s%d", pp > pos + 1 ? "," : "", i + 1);
      } else if (f[i].n < 0) {
        pn += sprintf(pn, "%s%d", pn > neg + 1 ? "," : "", i + 1);
      }
    }
    *pp++ = '}';
    *pn++ = '}';
    *pp = 0;
    *pn = 0;
    sprintf(row, "sign_split=%s|%s", pos, neg);
    printf("%s\n", row);
    CHECK(strcmp(row, "sign_split={1,2,6}|{3,4,5}") == 0,
          "ch36 fiedler signs split the ring into two arcs of 3");
  }
```

listing 273: the fiedler row, exact fraction verification, the sign split (source: kdd/samples/src/Ch36/spectral.c, lines 251-296)

```
    for (int pass = 1; pass <= 100; pass++) {
      int moved = 0;
      for (int i = 0; i < 6; i++) {
        int best = 0;
        Frac bd = fmul(fsub(f[i], cents[0]), fsub(f[i], cents[0]));
        for (int c = 1; c < 2; c++) {
          const Frac dd = fmul(fsub(f[i], cents[c]), fsub(f[i], cents[c]));
          if (fcmp(dd, bd) < 0) {
```

```
        best = c;
        bd = dd;
      }
    }
    if (assign[i] != best) {
      assign[i] = best;
      moved = 1;
    }
  }
  if (!moved) {
    iters = pass;
    break;
  }
  Frac sum[2] = {fr(0, 1), fr(0, 1)};
  int cnt[2] = {0, 0};
  for (int i = 0; i < 6; i++) {
    sum[assign[i]] = fadd(sum[assign[i]], f[i]);
    cnt[assign[i]]++;
  }
  for (int c = 0; c < 2; c++) {
    if (cnt[c]) {
      cents[c] = fdiv(sum[c], fr(cnt[c], 1));
    }
  }
  Frac sse = fr(0, 1);
  for (int i = 0; i < 6; i++) {
    const Frac dx = fsub(f[i], cents[assign[i]]);
    sse = fadd(sse, fmul(dx, dx));
  }
  char vec[7];
  for (int i = 0; i < 6; i++) {
    vec[i] = (char)('0' + assign[i]);
  }
  vec[6] = 0;
  fstr(cents[0], b1);
  fstr(cents[1], b2);
  sprintf(row, "emb iter%d cents=(%s),(%s) assign=%s sse=", pass, b1, b2,
          vec);
  fstr(sse, row + strlen(row));
  printf("%s\n", row);
  CHECK(ntrace == 0
            ? strcmp(row, "emb iter1 cents=(1),(-1/5) assign=011111 "
                          "sse=9/5") == 0
            : strcmp(row, "emb iter2 cents=(2/3),(-2/3) assign=001110 "
                          "sse=1/3") == 0,
        "ch36 embedded k-means trace row");
  CHECK(fabs((double)cents[0].n / cents[0].d -
             (ntrace == 0 ? 1.0 : 2.0 / 3.0)) < 1e-12 &&
            fabs((double)cents[1].n / cents[1].d -
                 (ntrace == 0 ? -1.0 / 5.0 : -2.0 / 3.0)) < 1e-12,
        "ch36 embedded centroid doubles within 1e-12 (D2)");
  ntrace++;
```

listing 274: embedded k-means with the ch31 tie rules, three passes to 001110 (source: kdd/samples/src/Ch36/spectral.c, lines 316-375)

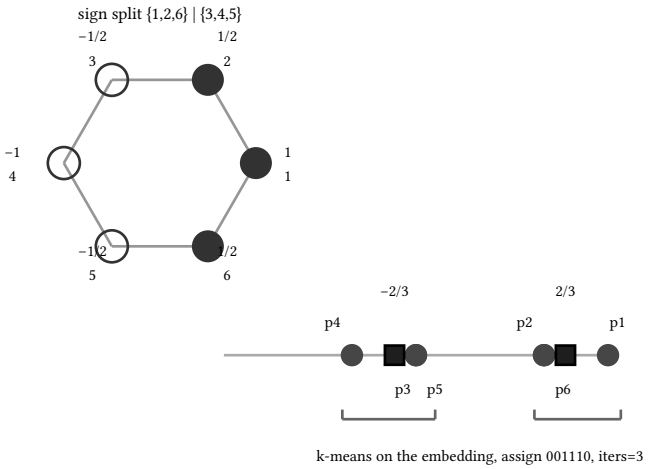


figure 147: signs on the ring, values on the line, k-means reproducing the split

36.4. ratio cut against normalized cut

The cut objectives are what the eigenvectors relax, the discrete cut swapped for the eigenvector problem whose signs are rounded back to a partition. Ratio cut charges a split $A \mid B$ the cut size normalized by cardinality, cut $(1/|A| + 1/|B|)$, Hagen and Kahng's balanced partitioning objective; normalized cut swaps cardinality for volume, cut $(1/\mathrm{vol}(A) + 1/\mathrm{vol}(B))$, Shi and Malik's degree-aware version. On the 2-regular cycle volume is twice cardinality everywhere, so the two objectives are proportional on every one of the 31 splits and the sample asserts it, `regular_graph_proportional=true`, both argmins landing on `cut {1,2,3}` `cut=2 ratiocut=4/3 ncut=2/3` with `argmin_ratiocut={1,2,3} 4/3` and `argmin_ncut={1,2,3} 2/3`. Every arc of 3 ties there, the Fiedler signs of the previous facet picked {1,2,6}, the enumeration records {1,2,3} by its string tie-break, both optimal.

The star is where the objectives part. G2 is a star $K_{1,5}$, center node 1 joined to 2 through 6, plus one rim edge 2-3, with degrees `deg2=(5,2,2,1,1,1)`. The dry run: isolating the pendant 4 costs one edge, `cut {4} cut=1 ratiocut=6/5 ncut=12/11`, while cutting the rim pair costs two, `cut {2,3} cut=2 ratiocut=3/2 ncut=3/4`. Ratio cut prefers the pendant, `argmin_ratiocut={4} 6/5`, cardinality barely distinguishing a 1-against-5 split from anything else, and the other two pendants tie at the same 6/5 with the string tie-break deciding. Normalized cut prefers the rim pair, `argmin_ncut={2,3} 3/4`, because volume is degree-weighted: the side {2,3} carries volume 4 against the pendant's 1, and the complement's volume is inflated by the center's degree 5, so the degree-aware objective would rather cut two edges into a light side than one edge off a heavy graph. The row prints `disagree=true`, and that is the honest summary of the field: the eigenvector you relax decides what balance means.

```
  /* ---- G2: star K1,5 + rim edge 2-3; the objectives disagree ---- */
  static int A2[6][6], deg2[6];
  {
    static const int E2[6][2] = {{0, 1}, {0, 2}, {0, 3},
                                 {0, 4}, {0, 5}, {1, 2}};
    for (int e = 0; e < 6; e++) {
      A2[E2[e][0]][E2[e][1]] = 1;
      A2[E2[e][1]][E2[e][0]] = 1;
    }
    for (int i = 0; i < 6; i++) {
      deg2[i] = A2[i][0] + A2[i][1] + A2[i][2] + A2[i][3] + A2[i][4] + A2[i][5];
    }
  }
  sprintf(row, "deg2=(%d,%d,%d,%d,%d,%d)", deg2[0], deg2[1], deg2[2], deg2[3],
          deg2[4], deg2[5]);
  printf("%s\n", row);
  CHECK(strcmp(row, "deg2=(5,2,2,1,1,1)") == 0, "ch36 G2 degree row");
  {
    static const int show[2] = {1 << 3, (1 << 1) | (1 << 2)};
    static const char *pin[2] = {"cut {4} cut=1 ratiocut=6/5 ncut=12/11",
                                 "cut {2,3} cut=2 ratiocut=3/2 ncut=3/4"};
    for (int t = 0; t < 2; t++) {
      int cut;
      Frac rc, nc;
      cutvals(A2, deg2, show[t], &cut, &rc, &nc);
      char ss[24], rs[32], ns[32];
      setstr(show[t], ss);
      fstr(rc, rs);
      fstr(nc, ns);
      sprintf(row, "cut %s cut=%d ratiocut=%s ncut=%s", ss, cut, rs, ns);
      printf("%s\n", row);
      CHECK(strcmp(row, pin[t]) == 0, "ch36 G2 cut objective row");
    }
  }
  char rset[24], nset[24];
  Frac rv, nv;
  argmin(A2, deg2, 0, rset, &rv);
  argmin(A2, deg2, 1, nset, &nv);
  fstr(rv, vs);
  sprintf(row, "argmin_ratiocut=%s %s", rset, vs);
  printf("%s\n", row);
  CHECK(strcmp(row, "argmin_ratiocut={4} 6/5") == 0,
        "ch36 G2 argmin ratiocut isolates the pendant node 4");
  fstr(nv, vs);
  sprintf(row, "argmin_ncut=%s %s", nset, vs);
  printf("%s\n", row);
  CHECK(strcmp(row, "argmin_ncut={2,3} 3/4") == 0,
        "ch36 G2 argmin ncut cuts the small-volume rim side {2,3}");
  sprintf(row, "disagree=%s", strcmp(rset, nset) != 0 ? "true" : "false");
  printf("%s\n", row);
  CHECK(strcmp(row, "disagree=true") == 0,
        "ch36 ratiocut and ncut disagree on G2");
```

listing 275: the star plus rim edge, both cut rows, both argmins, disagree (source: kdd/samples/src/Ch36/cuts.c, lines 213-264)

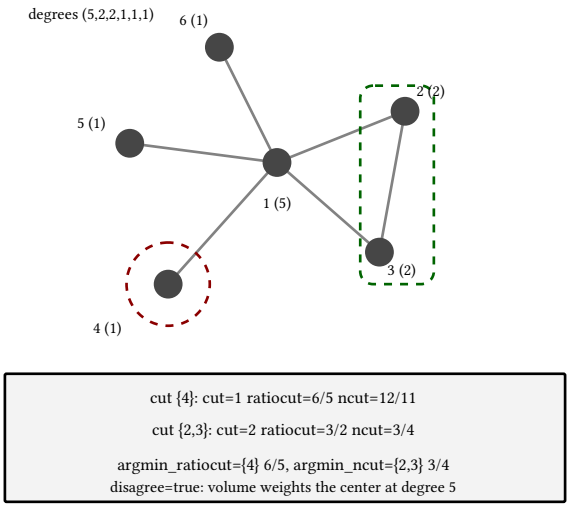


figure 148: on the star, ratiocut isolates a pendant, ncut cuts the light rim pair

sources: the pipeline, the unnormalized Laplacian, and the caveats about which eigenvectors to use follow von Luxburg 2007, “A tutorial on spectral clustering”, Statistics and Computing 17:395-416; normalized cut is Shi & Malik 2000, “Normalized cuts and image segmentation”, IEEE TPAMI 22(8):888-905; ratio cut is Hagen & Kahng 1992, “New spectral methods for ratio cut partitioning and clustering”, IEEE TCAD 11(9):1074-1085; and the Fiedler vector and its sign structure go back to Fiedler 1973, Czechoslovak Mathematical Journal 23:298-305. The Neumaier requirement on the QtLQ literal, the exact-fraction embedded k-means, and every pinned row are witnessed by kdd-contract-s7.md + playground/kdd-matrix/gen_s7.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch36`, 32 + 10 checks in chapter 36 of the kdd suite.

37. statistical anomaly detection

Two samples carry the chapter: `gauss.c` z-scores a known-parameter gaussian series and runs the joint Mahalanobis test on correlated inliers, 35 checks, and `robust.c` flags outliers with boxplot fences and with the MAD robust z, 22 checks. The chapter makes 4 moves: the known-parameter z-score under a strict 3-sigma rule, the joint test that catches what two per-axis rulers miss, the fences that need no mean at all, and the masking failure a mean-based rule suffers when outliers inflate their own yardstick. Every behavioral claim below is one of the 57 checks of chapter 37's samples or a cited constant. Clustering asked which rows group, book 14, ch 31 (knowledge discovery, k-means) through book 14, ch 36 (knowledge discovery, spectral clustering) this chapter and book 14, ch 38 (knowledge discovery, proximity and density anomalies) ask which rows refuse to, and book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) puts error bars on every verdict from here on.

37.1. the known parameters, and the strict rule

The cleanest anomaly model is a distribution whose parameters nobody has to estimate. The fixture is 11 measurements from a process known to run at mu = 50 with sigma = 5, so the z-score of every value is one division, $(v - 50)/5$, and the detector asks only whether $|z| > 3$. The rule is strict by pin: equality does not flag, and the fixture carries a value parked exactly on the boundary to prove it.

The dry run: the measurements read 50, 45, 55, 40, 60, 48, 52, 70, 46, 54, 35, and the sample prints one z row per value, `z 50=0`, `z 45=-1`, `z 55=1`, on through `z 70=4` and `z 35=-3`. Only one value clears the bar, printed as `flags3={70}` and asserted as "ch37 3-sigma flags only 70 (strict > 3)". The value 35 lands at exactly −3, and its row is the boundary witness: `boundary 35 z=-3.0 flagged=False (strict > 3)` in the sheet, asserted as "ch37 boundary 35 z=-3 unflagged (strict > 3)". An inclusive rule would double the flag set on this very fixture, which is why the inequality sign is a check and not a comment.

```c
/* fixture G11: 11 gaussian measurements, known mu = 50, sigma = 5 */
const int g11[11] = {50, 45, 55, 40, 60, 48, 52, 70, 46, 54, 35};
const double zwant[11] = {0.0, -1.0, 1.0,  -2.0, 2.0, -0.4,
                          0.4, 4.0,  -0.8, 0.8,  -3.0};
int nflag = 0, flagv = 0, flagged35 = -1;
for (int i = 0; i < 11; i++) {
  const double z = (double)(g11[i] - 50) / 5.0;
  char name[48];
  snprintf(name, sizeof name, "ch37 z(%d) = %.17g", g11[i], zwant[i]);
  printf("z %d=%g\n", g11[i], z);
  CHECK(z == zwant[i], name);
  if (fabs(z) > 3.0) {
    nflag++;
    flagv = g11[i];
  }
  if (g11[i] == 35) {
    flagged35 = fabs(z) > 3.0;
  }
}
printf("flags3={%d}\n", flagv);
CHECK(nflag == 1 && flagv == 70, "ch37 3-sigma flags only 70 (strict > 3)");
CHECK(flagged35 == 0, "ch37 boundary 35 z=-3 unflagged (strict > 3)");
```

listing 276: the 11-value fixture, one z per value, strict > 3 flags (source: kdd/samples/src/Ch37/gauss.c, lines 60-81)

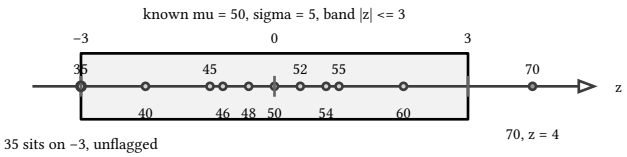


figure 149: 11 measurements on the z ruler, the shaded band is the strict interior

37.2. one ellipse beats two rulers

Scoring each feature against its own spread misses the point when the features move together. The fixture is 8 two-feature inliers built, by a t/u decomposition, to have mean exactly (10, 10) and population covariance exactly `[[4,-3],[-3,4]]`, divisor n as book 14, ch 4 (knowledge discovery, normalization and standardization) pinned it, negative correlation locked in. The joint test is the Mahalanobis distance of book 14, ch 5 (knowledge discovery, distance metrics), $D^2 = (p-\mu)^T S^{-1}(p-\mu)$, with $S^{-1} = (1/7)[[4,3],[3,4]]$ because the determinant is exactly 7.

The dry run: the sample prints `Jmean=(10,10) Jcov=[[4,-3],[-3,4]]` and `Jdet=7`, all three asserted exactly. Then three probes. The pair (13,13) and (13,7) carries the story: both have marginal scores (1.5, 1.5) and (1.5, −1.5), both dead quiet under any per-axis 3-sigma rule, yet their rows print `probe (13,13) z=(1.5,1.5) D2=18/1 D=4.2426406871192848 chi2flag=true z3flag=false` and `probe (13,7) z=(1.5,-1.5) D2=18/7 D=1.6035674514745464 chi2flag=false z3flag=false`. The arithmetic is $D^2 = (4a^2 + 6ab + 4b^2)/7$ over the integer deviation (a, b): (3,3) gives $126/7 = 18$, (3,-3) gives $18/7$. The split is asserted as "ch37 same-marginals pair splits D^2 18 vs 18/7". The flag compares D^2 against the df=2 chi-square critical at 5% exactly, by cross-multiplication against 5991/1000: the table value 5.991, exact form $-2\ln 0.05 = 5.991464547107982$, printed as `chi2_df2_alpha05=5.991 exact=5.9914645471079817`. The third probe (14,8) prints `D2=32/7`, under the bar: only the correlated one is caught, and only the joint rule catches it.

```
const int jin[8][2] = {{14, 8}, {11, 7}, {11, 9},  {9, 11},
                       {7, 11}, {8, 14}, {10, 10}, {10, 10}};
double m1 = 0.0, m2 = 0.0;
for (int i = 0; i < 8; i++) {
  m1 += (double)jin[i][0];
  m2 += (double)jin[i][1];
}
m1 /= 8.0;
m2 /= 8.0;
double s11 = 0.0, s22 = 0.0, s12 = 0.0;
for (int i = 0; i < 8; i++) {
  const double d1 = (double)jin[i][0] - m1;
  const double d2 = (double)jin[i][1] - m2;
  s11 += d1 * d1;
  s22 += d2 * d2;
  s12 += d1 * d2;
}
s11 /= 8.0;
s22 /= 8.0;
s12 /= 8.0;
printf("Jmean=(%g,%g) Jcov=[[%g,%g],[%g,%g]]\n", m1, m2, s11, s12, s12, s22);
CHECK(m1 == 10.0 && m2 == 10.0, "ch37 J mean exactly (10,10)");
CHECK(s11 == 4.0 && s22 == 4.0 && s12 == -3.0,
      "ch37 J population cov [[4,-3],[-3,4]] (divisor n, ch05 pin)");
```

listing 277: the 8 inliers, their exact mean and population covariance, determinant 7 (source: kdd/samples/src/Ch37/gauss.c, lines 86-109)

```
for (int i = 0; i < 3; i++) {
  const int a = probes[i][0] - 10, b = probes[i][1] - 10;
  const double z1 = (double)a / sqrt(s11);
  const double z2 = (double)b / sqrt(s22);
  int rn, rd;
  reduce(4 * a * a + 6 * a * b + 4 * b * b, 7, &rn, &rd);
  const double d = sqrt((double)rn / (double)rd);
  /* chi2 flag decided exactly: D^2 > 5.991 = 5991/1000 */
  const int chi2flag = rn * 1000 > 5991 * rd;
  const int z3flag = fabs(z1) > 3.0 || fabs(z2) > 3.0;
  printf("probe (%d,%d) z=(%g,%g) D2=%d/%d D=%.17g chi2flag=%s z3flag=%s\n",
         probes[i][0], probes[i][1], z1, z2, rn, rd, d,
         chi2flag ? "true" : "false", z3flag ? "true" : "false");
```

listing 278: per-probe D^2 as a reduced fraction, the chi-square flag cross-multiplied (source: kdd/samples/src/Ch37/gauss.c, lines 123-135)

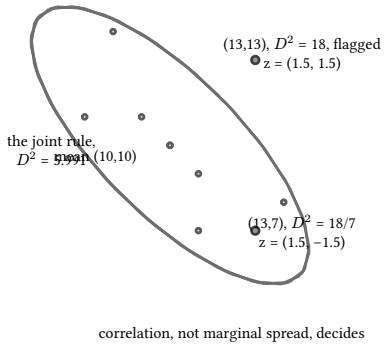


figure 150: the tilted joint rule: both probes are marginally quiet, only one is outside

PITFALL: the marginal rulers agree on both probes

Per-axis 3-sigma scores (1.5, 1.5) and (1.5, −1.5) are interchangeable to any univariate detector, and all three probes carry z3flag=false, asserted three times as "ch37 probe (13,13) univariate 3-sigma quiet" and its siblings. The covariance inverts the geometry: variance is 1 along the diagonal, 7 along the anti-diagonal, so a step with the grain is cheap and a step against it is expensive. Two rulers see two half-quiet scores. One ellipse sees one outlier.

37.3. fences that need no mean

The third detector refuses to estimate anything. Tukey's fences flag by order statistics alone, $Q_1 - 1.5 \cdot \mathrm{IQR}$ and $Q_3 + 1.5 \cdot \mathrm{IQR}$, the construction book 14, ch 2 (knowledge discovery, data cleaning and imputation) introduced; here the quartile convention is the one pinned in chapter 02, odd n excludes the median and averages each half's middle pair. The fixture is the 9-value series F9 = (2, 4, 6, 8, 10, 12, 14, 16, 40), the outlier parked in the last slot where it cannot touch the quartiles.

The dry run: lower half (2, 4, 6, 8) averages its middle pair to Q1 = 5; upper half (12, 14, 16, 40) averages (14 + 16)/2 = 15, the outlier withheld, asserted as "ch37 F9 Q3 = 15, outlier withheld from Q3". IQR = 10, fences 5 - 15 = −10 and 15 + 15 = 30, printed as one row `fence Q1=5/1 Q3=15/1 IQR=10/1 LF=-10/1 UF=30/1`. Only 40 exceeds its fence, printed `flags={40}`, asserted as "ch37 F9 fence flags only 40". The boundary discipline repeats: probes at exactly 30 and exactly −10 print `fence probe 30 onfence=true flagged=false (strict)` and `fence probe -10 onfence=true flagged=false (strict)`, both asserted, the fence analog of the z = −3 row.

```
const int f9[9] = {2, 4, 6, 8, 10, 12, 14, 16, 40};
const int lower[4] = {f9[0], f9[1], f9[2], f9[3]};
const int upper[4] = {f9[5], f9[6], f9[7], f9[8]};
int q1n, q1d, q3n, q3d;
reduce(lower[1] + lower[2], 2, &q1n, &q1d);
reduce(upper[1] + upper[2], 2, &q3n, &q3d);
int iqrn, iqrd;
reduce(q3n * q1d - q1n * q3d, q1d * q3d, &iqrn, &iqrd);
/* fences: Q1 - 1.5*IQR and Q3 + 1.5*IQR over the common denominator 2 */
int lfn, lfd, ufn, ufd;
reduce(2 * q1n * iqrd - 3 * iqrn * q1d, 2 * q1d * iqrd, &lfn, &lfd);
reduce(2 * q3n * iqrd + 3 * iqrn * q3d, 2 * q3d * iqrd, &ufn, &ufd);
printf("fence Q1=%d/%d Q3=%d/%d IQR=%d/%d LF=%d/%d UF=%d/%d\n", q1n, q1d, q3n,
       q3d, iqrn, iqrd, lfn, lfd, ufn, ufd);
CHECK(q1n == 5 && q1d == 1, "ch37 F9 Q1 = 5 (halves averaged, ch02 rule)");
CHECK(q3n == 15 && q3d == 1, "ch37 F9 Q3 = 15, outlier withheld from Q3");
CHECK(iqrn == 10 && iqrd == 1, "ch37 F9 IQR = 10");
CHECK(lfn == -10 && lfd == 1, "ch37 F9 lower fence -10");
CHECK(ufn == 30 && ufd == 1, "ch37 F9 upper fence 30");
```

listing 279: quartiles by excluded-median halves, fences as reduced fractions (source: kdd/samples/src/Ch37/robust.c, lines 55-73)

```
int nfflag = 0, fflagv = 0;
for (int i = 0; i < 9; i++) {
  if (f9[i] * lfd < lfn || f9[i] * ufd > ufn) {
    nfflag++;
    fflagv = f9[i];
  }
}
printf("flags={%d}\n", fflagv);
CHECK(nfflag == 1 && fflagv == 40, "ch37 F9 fence flags only 40");
/* on-fence probes: 30 == UF and -10 == LF, strict rule leaves both
 * unflagged (pinned boundary rows) */
printf("fence probe 30 onfence=%s flagged=%s (strict)\n",
       30 * ufd == ufn ? "true" : "false", 30 * ufd > ufn ? "true" : "false");
printf("fence probe -10 onfence=%s flagged=%s (strict)\n",
       -10 * lfd == lfn ? "true" : "false",
       -10 * lfd < lfn ? "true" : "false");
CHECK(30 * ufd == ufn && !(30 * ufd > ufn),
      "ch37 probe 30 sits on UF, unflagged (strict)");
CHECK(-10 * lfd == lfn && !(-10 * lfd < lfn),
      "ch37 probe -10 sits on LF, unflagged (strict)");
```

listing 280: the strict fence flag scan and the two on-fence probes (source: kdd/samples/src/Ch37/robust.c, lines 74-93)

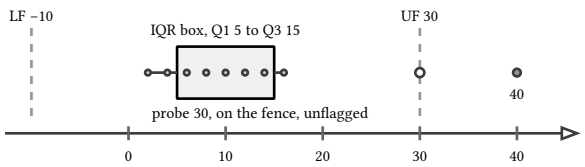


figure 151: the box and the fences, 40 outside, a probe sitting exactly on the upper fence

37.4. masking, and the robust catch

A mean-based rule hands the outliers a vote in their own trial. The fixture M10 = (4, 6, 8, 10, 12, 14, 16, 18, 58, 74) plants two outliers on the same side; they drag the mean to 22 and the population variance to 2568/5, so far

that neither lands 3 sigma out. The cure is the robust z of book 14, ch 4 (knowledge discovery, normalization and standardization), deviation from the median over the MAD, the raw Leys et al. spelling with no 0.6745 constant.

The dry run: the sample prints `mask mean=22/1 SS=5136 var_pop=2568/5`, with SS = 5136 and the variance each a check. The mean-based squared scores reduce to fractions, printed `z2 58=270/107 |z|=1.5885101466409677` and `z2 74=1690/321 |z|=2.2945146562591754`, both under 9, so the flag line reads `flags3_mean={} (masked)`, asserted as “ch37 mean-based 3-sigma flags nothing (masking)”. The robust half prints `med=13/1 MAD=5/1`, then `rz 58=9/1` and `rz 74=61/5`, the value 74 sitting 61/5 = 12.2 deviations from the median, and the flag line `flags_rz3={58,74}`, asserted as “ch37 MAD robust-z flags both 58 and 74”. The largest inlier holds at 9/5, printed `inlier rz max |4-13|/5=9/5` and asserted under the same 3 bar, so the robust rule separates cleanly: inliers at most 1.8, outliers 9 and 12.2.

```
  const int outs[2] = {58, 74};
  const int z2want[2][2] = {{270, 107}, {1690, 321}};
  const double azwant[2] = {1.5885101466409677, 2.2945146562591754};
  int nmasked = 0;
  for (int i = 0; i < 2; i++) {
    const int dv = outs[i] * meand - meann;
    int z2n, z2d;
    reduce(dv * dv * 10, ss, &z2n, &z2d);
    const double az = sqrt((double)z2n / (double)z2d);
    printf("z2 %d=%d/%d |z|=%.17g\n", outs[i], z2n, z2d, az);
    char name[56];
    snprintf(name, sizeof name, "ch37 z2(%d) = %d/%d", outs[i], z2want[i][0],
             z2want[i][1]);
    CHECK(z2n == z2want[i][0] && z2d == z2want[i][1], name);
    snprintf(name, sizeof name, "ch37 |z(%d)| = %.17g", outs[i], azwant[i]);
    CHECK(az == azwant[i], name);
    /* |z| < 3 exactly: z2 < 9 */
    if (z2n < 9 * z2d) {
      nmasked++;
    }
  }
  printf("flags3_mean={} (masked)\n");
  CHECK(nmasked == 2, "ch37 mean-based 3-sigma flags nothing (masking)");
```

listing 281: mean-based squared scores as fractions, the masked flag count (source: kdd/samples/src/Ch37/robust.c, lines 117-139)

```
const int rzwant[2][2] = {{9, 1}, {61, 5}};
int nrflag = 0;
int rflagv[2] = {0, 0};
for (int i = 0; i < 2; i++) {
  const int dv = outs[i] * medd - medn;
  /* rz = (|v - med|)/MAD = |v*medd - medn| * madd / (medd * madn) */
  int rzn, rzd;
  reduce((dv < 0 ? -dv : dv) * madd, medd * madn, &rzn, &rzd);
  printf("rz %d=%d/%d\n", outs[i], rzn, rzd);
  char name[40];
  snprintf(name, sizeof name, "ch37 rz(%d) = %d/%d", outs[i], rzwant[i][0],
           rzwant[i][1]);
  CHECK(rzn == rzwant[i][0] && rzd == rzwant[i][1], name);
  /* |rz| > 3 exactly: rz > 3 (both outliers sit above the median) */
  if (rzn > 3 * rzd) {
    rflagv[nrflag++] = outs[i];
  }
}
printf("flags_rz3={%d,%d}\n", rflagv[0], rflagv[1]);
CHECK(nrflag == 2 && rflagv[0] == 58 && rflagv[1] == 74,
      "ch37 MAD robust-z flags both 58 and 74");
```

listing 282: robust z from median and MAD, both outliers flagged, strict > 3 (source: kdd/samples/src/Ch37/robust.c, lines 160-180)

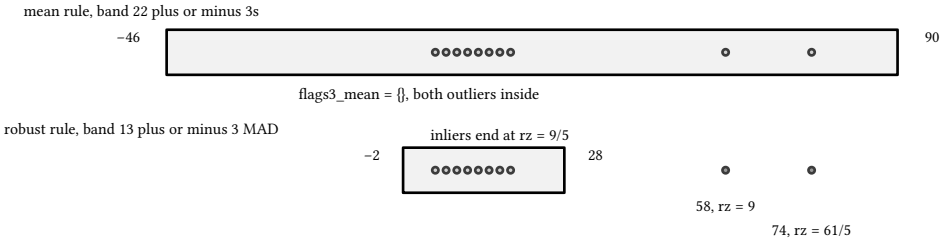


figure 152: the same ten values under the mean band and the robust band

NOTE: strict on every boundary, four times over

This chapter pins strict inequality at four separate fences: $|z| > 3$ on the gaussian ruler, where 35 sits at exactly -3 unflagged; $D^2 > 5.991$ on the joint rule; $x >$ UF on the fences, where the probe 30 sits exactly on the upper fence unflagged; and $|z| > 3$ again on the robust ruler. Inclusive variants would flag 35 and the fence probe on this very fixture. The sign is behavior, so it lives in check names, four of them.

Two detectors per half, and one lesson per pair: the joint rule beats the marginal ones because correlation is information, and the robust ones beat the mean-based ones because outliers should not grade their own papers. Both chapters behind, book 14, ch 2 (knowledge discovery, data cleaning and imputation) and book 14, ch 4 (knowledge discovery, normalization and standardization), built the tools; this chapter pointed them at detection. What none of them handle is the case where density itself varies across the map, and that is book 14, ch 38 (knowledge discovery, proximity and density anomalies) next.

sources: NIST/SEMATECH e-Handbook of Statistical Methods, section 1.3.6.7, chi-square distribution table, df = 2 critical 5.991 at alpha = 0.05, exact form $-2 \ln 0.05$, www.itl.nist.gov/div898/handbook, accessed 2026-09-22, pinned by the contract sheet. The MAD robust-z definition, raw ratio with no 0.6745 scaling, from Leys, Ley, Klein, Bernard, Licata, Journal of Experimental Social Psychology 49(4) 764-766, 2013, banked by kdd-contract-s1s2.md. All 57 pinned values witnessed by kdd-contract-s8s9.md and playground/kdd-matrix/gen_s8.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch37`, 35 + 22 checks in chapter 37 of the kdd suite.

38. proximity and density anomalies

Two samples carry the chapter: `lof.c` walks the full LOF chain on a one-dimensional fixture where every distance is an integer and every density an exact fraction, 48 checks, and `contrast.c` runs two clustering-based anomaly rules over the same points, 15 checks. The chapter makes 4 moves: the k-distance neighborhood, reachability and the local reach density it induces, the LOF ratio and its strict 3/2 flag, and the clustering contrast where a size rule flags four points LOF leaves quiet. Every behavioral claim below is one of the 63 checks of chapter 38's samples or a definition from Breunig et al. The statistical detectors of book 14, ch 37 (knowledge discovery, statistical anomaly detection) graded each point against one global distribution; here the yardstick is local, each point measured against its own neighbors, and book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) next asks how often any such rule cries wolf.

38.1. the k-distance neighborhood

The fixture is nine points on a line, L9 = {10, 12, 14, 16, 30, 31, 32, 33, 60} with k = 3: one loose cluster, one tight cluster, one point stranded far right. Breunig et al define the k-distance of a point as the distance to its k-th nearest neighbor, and $N_{k(p)}$ as the points within it, and both are pure order statistics on this fixture: sort the other eight by distance, take the third.

The dry run: the sample prints one row per point. `pt 10 kdist=6 N={12,14,16}`, because the distances from 10 read 2, 4, 6, 20, 21, 22, 23, 50 in sorted order and the third is 6. The tight cluster prints `pt 31 kdist=2 N={30,32,33}` and `pt 32 kdist=2 N={30,31,33}`, the straggler prints `pt 60 kdist=29 N={31,32,33}`, all nine rows checked as "ch38 kdist(10) = 6" through "ch38 N_k(60) = {31,32,33}". The ranking line prints `kdist_rank 60:29,10:6,16:6,12:4,14:4,30:3,33:3,31:2,32:2`, descending k-distance with ties broken by point ascending, and the gap witness closes the facet: 29 > 2 x 6, asserted as "ch38 gap witness 29 > 2*6 isolates 60". A global sort sees the gap; the rest of the chapter asks whether the gap is all there is to see.

```
int main(void) {
  const int pts[9] = {10, 12, 14, 16, 30, 31, 32, 33, 60};
  const int k = 3;
  const int kdist_want[9] = {6, 4, 4, 6, 3, 2, 2, 3, 29};
  const int neigh_want[9][3] = {{12, 14, 16}, {10, 14, 16}, {10, 12, 16},
                                {10, 12, 14}, {31, 32, 33}, {30, 32, 33},
                                {30, 31, 33}, {30, 31, 32}, {31, 32, 33}};
  const int reach_want[9][3] = {{4, 4, 6}, {6, 4, 6}, {6, 4, 6},
                                {6, 4, 4}, {2, 2, 3}, {3, 2, 3},
                                {3, 2, 3}, {3, 2, 2}, {29, 28, 27}};
  const Frac lrd_want[9] = {{3, 14}, {3, 16}, {3, 16}, {3, 14}, {3, 7},
                            {3, 8},  {3, 8},  {3, 7},  {1, 28}};
  const Frac lof_want[9] = {{11, 12}, {23, 21}, {23, 21}, {11, 12}, {11, 12},
                            {23, 21}, {23, 21}, {11, 12}, {11, 1}};
```

listing 283: the nine points, k = 3, and the pinned k-distances and neighbor sets (source: kdd/samples/src/Ch38/lof.c, lines 74-87)

```
    kdist[i] = ds[k - 1].d;
    int c = 0;
    for (int a = 0; a < m && c < k; a++) {
      if (ds[a].d ≤ kdist[i]) {
        neigh[i][c++] = ds[a].q;
      }
    }
    /* generator lists N_k ascending by point; the (d,q) sort above yields
     * that order already for this fixture, but sort defensively */
    for (int a = 1; a < c; a++) {
      for (int b = a; b > 0 && neigh[i][b - 1] > neigh[i][b]; b--) {
        const int t = neigh[i][b - 1];
        neigh[i][b - 1] = neigh[i][b];
        neigh[i][b] = t;
      }
    }
    printf("pt %d kdist=%d N={%d,%d,%d}\n", pts[i], kdist[i], neigh[i][0],
           neigh[i][1], neigh[i][2]);
```

listing 284: k-dist as the 3rd smallest distance, N_k collected within it (source: kdd/samples/src/Ch38/lof.c, lines 114-131)

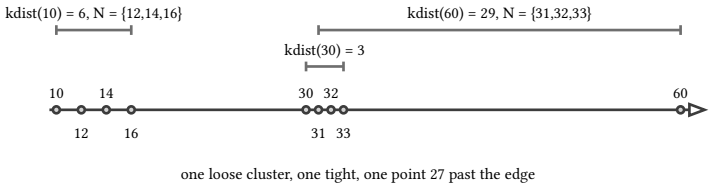


figure 153: nine points, three neighborhoods: loose, tight, and stranded

38.2. reachability, and the local density

The k-distance has a flaw as a yardstick: it grows when a point sits in a sparse region through no fault of its own. Breunig et al smooth it with the reach distance, $\text{reach}_k(p, o) = \max(\text{kdist}(o), d(p, o))$: never closer than the neighbor's own k-distance, so points inside a tight cluster cannot report artificially tiny distances. The local reach density inverts the mean: $\text{lrd}(p) = 1/\left(\sum_{o \in N_{k(p)}} \text{reach}_{k(p,o)}/|N_{k(p)}|\right)$, a density in units of points per distance.

The dry run: the sample prints the reach rows, and the smoothing shows immediately. For point 10, `reach 10: 12=4 14=4 16=6`: the raw distances to 12 and 14 are 2 and 4, but kdist(12) = 4 lifts the first to 4. For the straggler, `reach 60: 31=29 32=28 33=27`, the raw distances 29, 28, 27 dominate every kdist of the tight cluster, so nothing is lifted. The densities then print as nine exact fractions: `lrd 10=3/14` from mean reach $(4 + 4 + 6)/3 = 14/3$, `lrd 31=3/8` from $(3 + 2 + 3)/3 = 8/3$, and `lrd 60=1/28` from $(29 + 28 + 27)/3 = 84/3 = 28$. The landscape is the story: the tight cluster doubles the loose one, roughly 3/8 against 3/16, and the straggler's 1/28 is an order of magnitude below both.

```
    for (int a = 0; a < 3; a++) {
      /* neighbor index of point value neigh[i][a] */
      int ni = -1;
      for (int j = 0; j < 9; j++) {
        if (pts[j] == neigh[i][a]) {
          ni = j;
        }
      }
      const int di =
          pts[i] < neigh[i][a] ? neigh[i][a] - pts[i] : pts[i] - neigh[i][a];
      rs[a] = kdist[ni] > di ? kdist[ni] : di;
      sumr += rs[a];
    }
    printf("reach %d: %d=%d %d=%d %d=%d\n", pts[i], neigh[i][0], rs[0],
           neigh[i][1], rs[1], neigh[i][2], rs[2]);
    char name[80];
```

listing 285: reach as max of the neighbor's k-dist and the raw distance, row printed (source: kdd/samples/src/Ch38/lof.c, lines 148-163)

```
    lrd[i] = fdiv(fr(1, 1), fr(sumr, 3));
    snprintf(name, sizeof name, "ch38 lrd(%d) = %d/%d", pts[i], lrd_want[i].n,
             lrd_want[i].d);
    CHECK(lrd[i].n == lrd_want[i].n && lrd[i].d == lrd_want[i].d, name);
```

listing 286: lrd is one division of the reach sum, asserted as a reduced fraction (source: kdd/samples/src/Ch38/lof.c, lines 170-173)

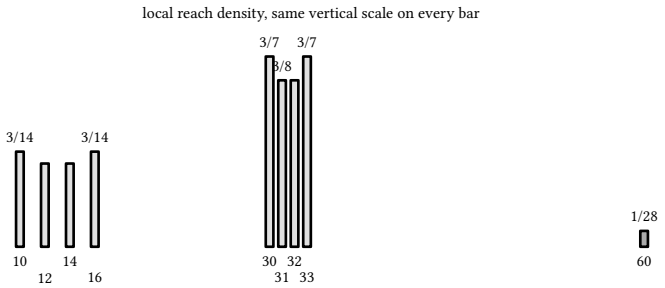


figure 154: the density landscape: lrd per point, tight cluster high, straggler near zero

NOTE: the 12 and 14 rows are not typos

Every reach distance of 10 to 12 is 4 even though the two points sit 2 apart, because reach pays the neighbor's k-distance floor, asserted in the pinned row "ch38 reach(10): 12=4 14=4 16=6". The floor is what makes lrd stable inside clusters: without it, arbitrarily close pairs would drive each other's densities to infinity, and LOF would be a division by noise.

38.3. the ratio that flags

Density alone still cannot flag: a point at the edge of a sparse but healthy cluster is legitimately sparse. The local outlier factor compares a point's density to its neighbors' densities, $\mathrm{LOF}(p) = \left(\sum_{o \in N_{k(p)}} \mathrm{lrd}(o)/|N_{k(p)}|\right) / \mathrm{lrd}(p)$. Values near 1 mean the point is as dense as its company; the flag rule is strict, $\mathrm{LOF} > 3/2$.

The dry run: the nine LOFs print as exact fractions, `lof 10=11/12`, `lof 12=23/21`, `lof 31=23/21`, through `lof 33=11/12`. Every inlier lands at one of two values, 11/12 at the loose cluster's rim points and 23/21 just over 1 at its interior, and the tight cluster repeats the same pair. The straggler prints `lof 60=11/1` and the flag line `lof_flags_3_2={60}`, asserted as "ch38 only 60 exceeds LOF 3/2". The hand walk for 60 is one line of fraction arithmetic: mean lrd of {31, 32, 33} is $(3/8 + 3/8 + 3/7)/3 = 11/28$, divided by lrd(60) = 1/28 gives exactly 11.

```
  /* LOF(p) = (mean lrd over N_k(p)) / lrd(p); strict flag LOF > 3/2 */
  int nlf = 0, lfv = 0;
  for (int i = 0; i < 9; i++) {
    Frac sl = fr(0, 1);
    for (int a = 0; a < 3; a++) {
      for (int j = 0; j < 9; j++) {
        if (pts[j] == neigh[i][a]) {
          sl = fadd(sl, lrd[j]);
        }
      }
    }
    const Frac lof = fdiv(fdiv(sl, fr(3, 1)), lrd[i]);
    printf("lof %d=%d/%d\n", pts[i], lof.n, lof.d);
    char name[48];
    snprintf(name, sizeof name, "ch38 lof(%d) = %d/%d", pts[i], lof_want[i].n,
             lof_want[i].d);
    CHECK(lof.n == lof_want[i].n && lof.d == lof_want[i].d, name);
    if (fcmp(lof, fr(3, 2)) > 0) {
      nlf++;
      lfv = pts[i];
    }
  }
  printf("lof_flags_3_2={%d}\n", lfv);
  CHECK(nlf == 1 && lfv == 60, "ch38 only 60 exceeds LOF 3/2");
```

listing 287: LOF as mean neighbor lrd over own lrd, strict 3/2 flag by cross multiply (source: kdd/samples/src/Ch38/lof.c, lines 176-199)

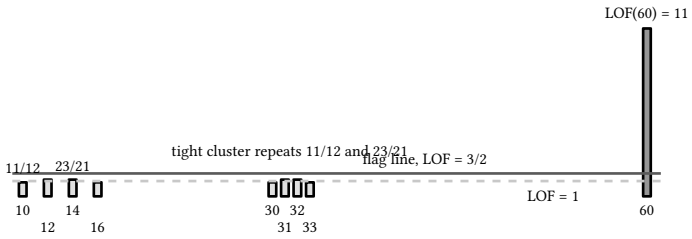


figure 155: nine LOF values against the 3/2 flag line, one bar above it

PITFALL: why 11, and not 21 over 2

The sheet records a resolved discrepancy worth retelling: a first hand pass scored LOF(60) = 21/2 by treating all three neighbors as lrd 3/8. Point 33 is an edge point of the tight cluster, kdist(33) = 3, reach distances 3, 2, 2, so its lrd is 3/7, not 3/8. That one neighbor mispriced moves the answer by half: the mean lifts by $(3/7 - 3/8)/3 = 1/56$, the ratio by $(1/56)/(1/28) = 1/2$, from 21/2 up to 11. That fragility is exactly why the book asserts LOF as a reduced fraction pair, never a double: “ch38 lof(60) = 11/1”.

38.4. what clustering sees instead

The same fixture through clustering glasses flags different points. book 14, ch 31 (knowledge discovery, k-means) owns Lloyd’s algorithm; here the sample verifies the two partitions are its fixed points and reads two rules off them. The global rule scores each point by distance to its own centroid; the size rule calls any cluster under 3 members anomalous.

The dry run: at k = 2 the fixed point splits {10, 12, 14, 16} from the rest, centroids printed as `km2 centroids 13/1,186/5` and the decision boundary checked at 251/10. Point 60 sits $60 - 186/5 = 114/5$ from its centroid, the runner-up 30 sits 36/5, ratio 19/6, printed as `centroid_dist 60=114/5 next=36/5 ratio=19/6`. The median own-cluster distance is 21/5, so the 3 x median threshold is 63/5, and only 60 clears it: `centroid_dist median=21/5 thr3med=63/5 flags={60}`. At k = 3 the fixed point is {10, 12}, {14, 16}, the rest, boundaries 13 and 261/10, sizes 2, 2, 5, printed as `km3 centroids`

`11/1,15/1,186/5 sizes`
`2,2,5`. The size rule then flags `size_rule_lt3 flags={10,12,14,16}` and the contrast lands: the four flagged points carry LOFs 11/12, 23/21, 23/21, 11/12, printed as `lof_of_flagged 11/12,23/21,23/21,11/12`
`all<3/2` and asserted as "ch38 LOF misses all four size-flagged points (< 3/2)". A small cluster is not a sparse point.

```
/* k = 2 fixed point partition A = {10,12,14,16}, B = the rest */
const int a4[4] = {10, 12, 14, 16};
const int b5[5] = {30, 31, 32, 33, 60};
const Frac c1 = mean(a4, 4);
const Frac c2 = mean(b5, 5);
printf("km2 centroids %d/%d,%d/%d\n", c1.n, c1.d, c2.n, c2.d);
CHECK(c1.n == 13 && c1.d == 1 && c2.n == 186 && c2.d == 5,
      "ch38 km2 centroids 13 and 186/5");
```

listing 288: the k=2 Lloyd fixed point and its exact centroids (source: kdd/samples/src/Ch38/contrast.c, lines 80-87)

```
const Frac med = srt[4];
const Frac thr = fr(3 * med.n, med.d);
int ngf = 0, gfv = 0;
for (int i = 0; i < 9; i++) {
  if (fcmp(dwn[i], thr) > 0) {
    ngf++;
    gfv = pts[i];
  }
}
printf("centroid_dist median=%d/%d thr3med=%d/%d flags={%d}\n", med.n, med.d,
       thr.n, thr.d, gfv);
CHECK(med.n == 21 && med.d == 5, "ch38 median centroid distance 21/5");
CHECK(thr.n == 63 && thr.d == 5, "ch38 3*median threshold 63/5");
CHECK(ngf == 1 && gfv == 60, "ch38 global rule flags only 60");
```

listing 289: the 3*median global rule, exact fractions, one flag (source: kdd/samples/src/Ch38/contrast.c, lines 114-127)

```
/* size rule: clusters under 3 members are anomalous; LOF misses all
 * four (pinned LOF values, recomputed in lof.c, all below 3/2) */
const Frac lofs[4] = {{11, 12}, {23, 21}, {23, 21}, {11, 12}};
int sizeflag = 1;
for (int i = 0; i < 4; i++) {
  sizeflag = sizeflag && assign[i] < 2;
}
printf("size_rule_lt3 flags={10,12,14,16}\n");
printf("lof_of_flagged 11/12,23/21,23/21,11/12 all<3/2\n");
int allquiet = 1;
for (int i = 0; i < 4; i++) {
  allquiet = allquiet && fcmp(lofs[i], fr(3, 2)) < 0;
}
CHECK(sizeflag, "ch38 size rule flags exactly {10,12,14,16}");
CHECK(allquiet, "ch38 LOF misses all four size-flagged points (< 3/2)");
```

listing 290: the size rule flags four points whose LOFs all sit under 3/2 (source: kdd/samples/src/Ch38/contrast.c, lines 171-185)

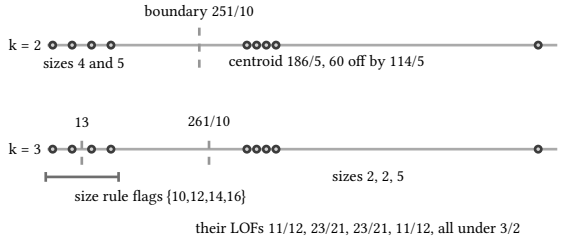


figure 156: two fixed-point partitions; the size rule flags what LOF forgives

Three rules, three different flag sets on nine fixed points: LOF flags {60}, the centroid rule flags {60}, the size rule flags {10, 12, 14, 16}. Which rule is right is not a mathematical question but a question about what the pipeline is hunting, and that is the cue for the statistics chapters: a rule that flags has a false-positive rate, and book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) measures exactly that.

sources: Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, Jorg Sander, “LOF: Identifying Density-Based Local Outliers”, Proceedings of SIGMOD 2000, 93-104, definitions of k-distance, reach distance, lrd, and LOF, banked by kdd-contract-s8s9.md with the LOF(60) = 11 discrepancy resolution recorded. All 63 pinned values witnessed by the sheet and playground/kdd-matrix/gen_s8.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch38`, 48 + 15 checks in chapter 38 of the kdd suite.

39. hypothesis testing and p-values

Two samples carry the chapter: `hypo.c` runs the exact binomial test and the one-sample t-test on fixtures built for exact arithmetic, 21 checks, and `tsim.c` measures the binomial test's false-alarm rate with a pinned random stream, 25 checks. The chapter makes 4 moves: the exact one- and two-sided p-values for eight heads of ten, the rejection region an exact test can actually afford, the t-statistic that estimates its own spread, and the simulated type-I rate. Every behavioral claim below is one of the 46 checks of chapter 39's samples or a cited constant. book 14, ch 38 (knowledge discovery, proximity and density anomalies) ended on the question of how often a rule cries wolf; this chapter prices the crying, and book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr) next handles the pipeline that runs ten tests at once.

39.1. eight heads of ten

The cleanest test bed is a fair coin flipped ten times, and the cleanest question is whether eight heads is surprising. The null gives every count k the probability $\text{pmf}(k) = \binom{10}{k}/2^{10}$, all 1024 sequences equally likely, and the sample builds the 11 probabilities with Pascal's triangle and asserts each as a reduced fraction.

The dry run: the triangle's low rows give $\text{pmf}(8) = 45/1024$, $\text{pmf}(9) = 10/1024$, which reduces to $5/512$, and $\text{pmf}(10) = 1/1024$, printed as `binom pmf8=45/1024 pmf9=5/512 pmf10=1/1024` and checked as "ch39 pmf(8) = 45/1024" through "ch39 pmf(10) = 1/1024". The one-sided question, at least this extreme upward, sums three cells, $(45 + 10 + 1)/1024 = 56/1024 = 7/128$, printed `pone_ge8=7/128`. The two-sided question uses the point-probability method: sum every outcome no more likely than the one observed. For 8 heads that is every k with $\text{pmf}(k) \le 45/1024$, the set {0, 1, 2, 8, 9, 10}, and the symmetry makes it twice the upper tail, $2 \cdot 56/1024 = 112/1024 = 7/64$, printed `ptwo_8=7/64` and asserted as "ch39 two-sided p(8) = 7/64". For 9 heads the bar drops to outcomes at most $10/1024$ likely, {0, 1, 9, 10}, and `ptwo_9=11/512`. The verdicts then print one line each, `decide 8heads p=7/64 vs 1/20 reject=false` and `decide 9heads p=11/512 vs 1/20 reject=true`, checked as "ch39 8 heads fails to reject at 5%" and "ch39 9 heads rejects at 5%".

```
/* binomial n = 10, p = 1/2 via Pascal's triangle; pmf(k) = C(10,k)/1024 */
int cb[11][11];
for (int i = 0; i ≤ 10; i++) {
  cb[i][0] = 1;
  for (int j = 1; j ≤ i; j++) {
    cb[i][j] = cb[i - 1][j - 1] + (j ≤ i - 1 ? cb[i - 1][j] : 0);
  }
}
Frac pmf[11];
for (int k = 0; k ≤ 10; k++) {
  pmf[k] = fr(cb[10][k], 1024);
}
printf("binom pmf8=%d/%d pmf9=%d/%d pmf10=%d/%d\n", pmf[8].n, pmf[8].d,
       pmf[9].n, pmf[9].d, pmf[10].n, pmf[10].d);
CHECK(pmf[8].n == 45 && pmf[8].d == 1024, "ch39 pmf(8) = 45/1024");
/* the sheet's pmf line prints the literal 10/1024; the sheet's own
 * fraction law is reduced form (it reduces 22/1024 -> 11/512 in the
 * same block), and 10/1024 reduces to 5/512 */
CHECK(pmf[9].n == 5 && pmf[9].d == 512, "ch39 pmf(9) = 10/1024 = 5/512");
CHECK(pmf[10].n == 1 && pmf[10].d == 1024, "ch39 pmf(10) = 1/1024");

Frac pone = fadd(fadd(pmf[8], pmf[9]), pmf[10]);
printf("pone_ge8=%d/%d\n", pone.n, pone.d);
CHECK(pone.n == 7 && pone.d == 128, "ch39 one-sided P(X≥8) = 7/128");
```

listing 291: Pascal's triangle builds the pmf, the one-sided p sums three cells (source: kdd/samples/src/Ch39/hypo.c, lines 66-89)

```
/* two-sided point-probability p for k = 8 and k = 9 */
const int ks[2] = {8, 9};
const Frac twowant[2] = {{7, 64}, {11, 512}};
Frac two[2];
for (int i = 0; i < 2; i++) {
  Frac s = fr(0, 1);
  for (int j = 0; j ≤ 10; j++) {
    if (fcmp(pmf[j], pmf[ks[i]]) ≤ 0) {
      s = fadd(s, pmf[j]);
    }
  }
  two[i] = s;
  printf("ptwo_%d=%d/%d\n", ks[i], s.n, s.d);
  char name[40];
  snprintf(name, sizeof name, "ch39 two-sided p(%d) = %d/%d", ks[i],
           twowant[i].n, twowant[i].d);
  CHECK(s.n == twowant[i].n && s.d == twowant[i].d, name);
}
const Frac alpha = fr(1, 20);
printf("decide 8heads p=7/64 vs 1/20 reject=%s\n",
       fcmp(two[0], alpha) ≤ 0 ? "true" : "false");
CHECK(fcmp(two[0], alpha) > 0, "ch39 8 heads fails to reject at 5%");
printf("decide 9heads p=11/512 vs 1/20 reject=%s\n",
       fcmp(two[1], alpha) ≤ 0 ? "true" : "false");
CHECK(fcmp(two[1], alpha) ≤ 0, "ch39 9 heads rejects at 5%");
```

listing 292: the point-probability two-sided p, then both decisions at 1/20 (source: kdd/samples/src/Ch39/hypo.c, lines 91-115)

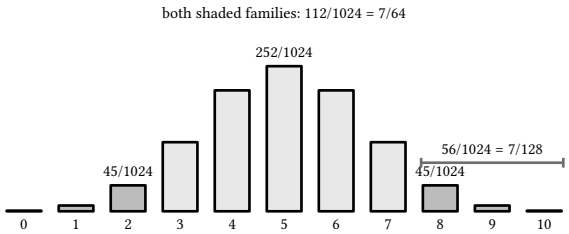


figure 157: ten fair flips: the pmf, shaded where probability is at most 45/1024

PITFALL: what the verdict lines do not say

The two misread rows pin the classic doublespeak. `misread p>alpha: p=7/64=0.109>0.05 -> fail to reject; NOT proof H0 true`: a missed bar says this test cannot tell 8-of-10 from luck, and says nothing about the coin's honesty. And `misread p<alpha: p=11/512=0.0215<0.05 -> reject H0; NOT P(H0)<5%`: the p-value probabilities outcomes, not hypotheses, so 0.0215 is P(data this extreme given a fair coin), never P(fair coin given the data). Both rows carry checks on their numeric comparisons: "ch39 misread row 7/64 = 0.109 > 0.05, fail to reject" and "ch39 misread row 11/512 = 0.0215 < 0.05, reject".

39.2. what five percent actually buys

A test over a table of exact probabilities cannot land on alpha = 1/20 on the nose, because every rejection region is a set of cells and every cell weighs a multiple of 1/1024. The exact test takes the largest region whose total probability still fits the budget.

The dry run: the sample scores every k by its two-sided p and keeps the ones under 1/20. k = 0 carries $2/1024 = 1/512$ and k = 1 carries $22/1024 = 11/512$, both inside; the next candidate, k = 2 with its mirror 8, carries $112/1024 = 7/64$, more than twice the whole budget, so the region stops there. It prints `rej_region={0,1,9,10} exact_type1=11/512`, asserted as "ch39 exact 5% region is {0,1,9,10} (conservative)" with the size its own check, "ch39

exact type-I size 11/512". The true size is $22/1024 = 11/512 = 0.021484375$, a dyadic decimal, and the nominal five percent is really 2.15. The staircase between 11/512 and 7/64 spans one step of k and a factor of 56/11, and no arrangement of binomial cells can close that gap from below.

```
/* nominal-5% rejection region: every k whose two-sided p fits under
 * 1/20; adding k in {2,8} jumps p to 112/1024 > 1/20 (sheet witness) */
int region[11];
Frac t1 = fr(0, 1);
int nreg = 0, regwant[4] = {0, 1, 9, 10};
for (int k = 0; k ≤ 10; k++) {
  Frac s = fr(0, 1);
  for (int j = 0; j ≤ 10; j++) {
    if (fcmp(pmf[j], pmf[k]) ≤ 0) {
      s = fadd(s, pmf[j]);
    }
  }
  region[k] = fcmp(s, alpha) ≤ 0;
  if (region[k]) {
    t1 = fadd(t1, pmf[k]);
    nreg++;
  }
}
printf("rej_region={0,1,9,10} exact_type1=%d/%d\n", t1.n, t1.d);
int regok = nreg == 4;
int w = 0;
for (int k = 0; k ≤ 10; k++) {
  if (region[k]) {
    regok = regok && k == regwant[w++];
  }
}
CHECK(regok, "ch39 exact 5% region is {0,1,9,10} (conservative)");
CHECK(t1.n == 11 && t1.d == 512, "ch39 exact type-I size 11/512");
```

listing 293: every k scored by two-sided p, the region that fits under 1/20 (source: kdd/samples/src/Ch39/hypo.c, lines 117-144)

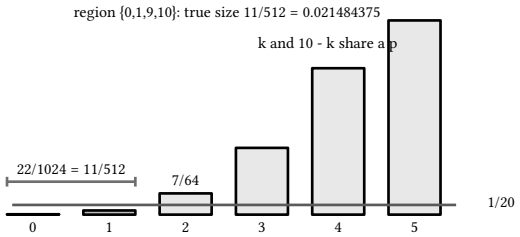


figure 158: two-sided p per k against the 1/20 budget; the region stops after k = 1

39.3. four sevens and a two

The t-test replaces a known sigma with an estimate, and the fixture is built so the estimate is exact: the values (2, 7, 7, 7, 7). Their mean is 6, deviations (-4, 1, 1, 1, 1), sum of squares 20, so $s^2 = 20/4 = 5$ and the standard error is $\sqrt{s^2/n} = \sqrt{5/5} = 1$ exactly. Every number on the path is an integer, which is why the chapter pins t itself instead of a rounding of t.

The dry run: the stats line prints `tvals mean=6/1 SS=20 s2=5/1 se=1`, each field its own check from "ch39 t fixture mean = 6" to "ch39 t fixture se = 1 exactly". Against mu0 = 8 the statistic is $t = (6 - 8)/1 = -2$, printed `t mu0=8 → t=-2 |t|=2 reject=false (crit 2.7764)`; against mu0 = 10 it is −4 and the row ends `reject=true`. The critical constant is the two-sided df = 4 table entry 2.7764, pinned as the chapter's one D2 constant by "ch39 df=4 two-sided t critical 2.7764 pinned (D2)". Four degrees of freedom, because estimating the spread from five values spends one.

The same five numbers fail to disprove a mean of 8 yet disprove a mean of 10, and the only thing that changed is the claim.

```
/* one-sample t on (2,7,7,7,7): dyadic fixture, se exactly 1 */
const int xs[5] = {2, 7, 7, 7, 7};
int s = 0;
for (int i = 0; i < 5; i++) {
  s += xs[i];
}
const Frac xbar = fr(s, 5);
int ss = 0;
for (int i = 0; i < 5; i++) {
  const int dv = xs[i] * xbar.d - xbar.n;
  ss += dv * dv;
}
ss /= xbar.d * xbar.d;
const Frac s2 = fr(ss, 4);
const Frac s2o5 = fr(s2.n, s2.d * 5);
const double se = sqrt((double)s2o5.n / (double)s2o5.d);
printf("tvals mean=%d/%d SS=%d s2=%d/%d se=%.17g\n", xbar.n, xbar.d, ss, s2.n,
       s2.d, se);
CHECK(xbar.n == 6 && xbar.d == 1, "ch39 t fixture mean = 6");
CHECK(ss == 20, "ch39 t fixture SS = 20");
CHECK(s2.n == 5 && s2.d == 1, "ch39 t fixture s^2 = 5");
CHECK(se == 1.0, "ch39 t fixture se = 1 exactly");
```

listing 294: the dyadic t fixture: mean, SS, s^2, and se exactly 1 (source: kdd/samples/src/Ch39/hypo.c, lines 146-167)

```
const double crit = 2.7764; /* df=4 two-sided 0.05 t table */
const double mu0[2] = {8.0, 10.0};
const double twant[2] = {-2.0, -4.0};
const int rejwant[2] = {0, 1};
for (int i = 0; i < 2; i++) {
  const double t = ((double)xbar.n / (double)xbar.d - mu0[i]) / se;
  const int rej = fabs(t) > crit;
  printf("t mu0=%g -> t=%.17g |t|=%.17g reject=%s (crit 2.7764)\n", mu0[i], t,
         fabs(t), rej ? "true" : "false");
  char name[56];
  snprintf(name, sizeof name, "ch39 t(mu0=%g) = %.17g", mu0[i], twant[i]);
  CHECK(t == twant[i], name);
  snprintf(name, sizeof name, "ch39 t(mu0=%g) reject=%d", mu0[i], rejwant[i]);
  CHECK(rej == rejwant[i], name);
}
CHECK(crit == 2.7764, "ch39 df=4 two-sided t critical 2.7764 pinned (D2)");
```

listing 295: t against both null values, strict |t| > 2.7764 decisions (source: kdd/samples/src/Ch39/hypo.c, lines 169-184)

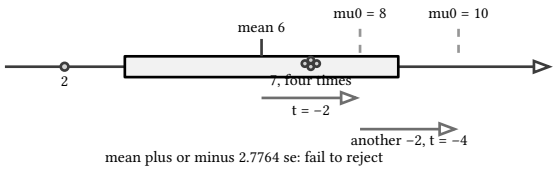


figure 159: the t fixture on the value line: one claimed mean inside the critical band, one outside

39.4. counting the false alarms

If the coin is fair and the region {0, 1, 9, 10} is applied forever, the test rejects 11/512 of the time, and that rate can be measured instead of derived. The generator is the MINSTD of book 14, ch 7 (knowledge discovery, sampling), Park and Miller's $16807 \cdot \text{state} \bmod (2^{31} - 1)$, seed 7, states held in uint64 so no intermediate overflows, and the coin is bit 16 of each state. Ten flips per trial, 10000 trials, all integer arithmetic, so the run reproduces itself bit for bit.

The dry run: the header prints `sim seed=7 rule=(state>>16)&1 trials=10000`, then `type1 rejects=212/10000=0.0212`, asserted as "ch39 sim rejects = 212/10000 (D3 literal)". The exact size is dyadic, $11/512 = 0.021484375$, checked for bit equality, and the simulated rate sits about 0.0003 under it, well inside the 0.01 tolerance of "ch39 sim rate within 0.01 of exact size". The histogram then prints eleven rows, `hist 0=7 expected=625/64` through `hist 10=5 expected=625/64`, every count and every expected fraction its own check. The middle row reads `hist 5=2411 expected=39375/16`, the expectation being $10000 \cdot 252/1024 = 39375/16 = 2460.9375$. The bars hug their expected ticks, and the two shaded tails hold 105 and 107 counts, exactly the 212 rejections.

```
  printf("sim seed=7 rule=(state>>16)&1 trials=10000\n");
  uint64_t st = 7;
  int counts[11] = {0};
  int rejects = 0;
  for (int trial = 0; trial < 10000; trial++) {
    int heads = 0;
    for (int flip = 0; flip < 10; flip++) {
      st = (16807ULL * st) % 2147483647ULL;
      heads += (int)((st >> 16) & 1ULL);
    }
    counts[heads]++;
    /* exact 5% rejection region {0,1,9,10} derived in Ch39/hypo.c */
    if (heads <= 1 || heads >= 9) {
      rejects++;
    }
  }
  printf("type1 rejects=%d/10000=%g\n", rejects, (double)rejects / 10000.0);
  CHECK(rejects == 212, "ch39 sim rejects = 212/10000 (D3 literal)");
  CHECK(22.0 / 1024.0 == 0.021484375,
        "ch39 exact size 11/512 = 0.021484375 dyadic");
  CHECK(fabs((double)rejects / 10000.0 - 0.021484375) <= 0.01,
        "ch39 sim rate within 0.01 of exact size");
```

listing 296: 10000 trials of ten bit-16 coins, rejections counted against 11/512 (source: kdd/samples/src/Ch39/tsim.c, lines 53-74)

```
  const int hist_want[11] = {7,    98,   404, 1164, 2085, 2411,
                             2123, 1171, 430, 102,  5};
  const int expn_want[11] = {625,   3125, 28125, 9375, 65625, 39375,
                             65625, 9375, 28125, 3125, 625};
  const int expd_want[11] = {64, 32, 64, 8, 32, 16, 32, 8, 64, 32, 64};
  for (int k = 0; k ≤ 10; k++) {
    const int e = 10000 * cb[10][k];
    const int g = gcdi(e, 1024);
    const int en = e / g, ed = 1024 / g;
    printf("hist %d=%d expected=%d/%d\n", k, counts[k], en, ed);
    char name[56];
    snprintf(name, sizeof name, "ch39 hist(%d) = %d (D3 literal)", k,
             hist_want[k]);
    CHECK(counts[k] == hist_want[k], name);
    snprintf(name, sizeof name, "ch39 expected(%d) = %d/%d", k, expn_want[k],
             expd_want[k]);
    CHECK(en == expn_want[k] && ed == expd_want[k], name);
  }
```

listing 297: eleven counts and eleven expected fractions, every row a check (source: kdd/samples/src/Ch39/tsim.c, lines 76-93)

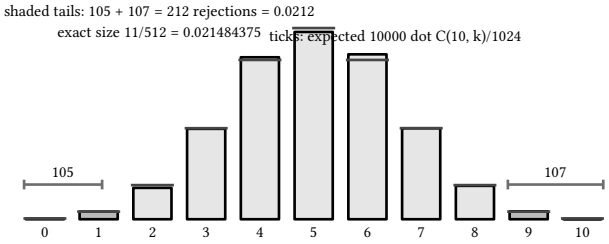


figure 160: 10000 simulated trials: counts against expected ticks, rejection tails shaded

One test, honestly priced: an exact 5% test that is really 2.15, a p-value that grades outcomes and never hypotheses, and a measured false alarm rate matching the derived one to three ten-thousandths. The honesty lasts as long as the pipeline runs one test. Ten tests at 2.15 percent each promise a roughly one-in-five chance of at least one false discovery, and book 14, ch 40 (knowledge discovery, multiple testing: bonferroni and fdr) is that bill.

sources: the fair-coin 8-of-10 example is the canonical small-sample binomial test pinned by kdd-contract-s8s9.md, two-sided p by the point-probability method as pinned there. The t critical 2.7764 is the standard two-sided df = 4 t table entry (sheet cites medcalc.org), the chapter's one D2 constant. The generator is Park and Miller's MINSTD, "Random Number Generators: Good Ones Are Hard to Find", Communications of the ACM 31(10) 1988, 1192-1201, the same family chapter 07's sampling pin uses. All 46 pinned values witnessed by kdd-contract-s8s9.md and playground/kdd-matrix/gen_s8.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch39`, 21 + 25 checks in chapter 39 of the kdd suite.

40. multiple testing: bonferroni and fdr

One sample carries the chapter: `fdr.c` runs three rejection procedures over two pinned lists of p-values and checks every step of every one, 60 checks. List P is ten hypotheses, list Q is six, both at alpha = q = 1/20. The chapter makes 4 moves: the Bonferroni threshold and the family-size bill it sends, Holm's step-down that spends the budget as it walks, Benjamini-Hochberg and the largest k that holds, and the adjusted p staircase that turns the procedure into one number per hypothesis. Every behavioral claim below is one of the 60 checks of chapter 40′s sample or a definition from the three banked papers. book 14, ch 39 (knowledge discovery, hypothesis testing and p-values) priced one test at a true 2.15 percent; a pipeline like the chi-squared screen of book 14, ch 8 (knowledge discovery, feature selection and chi-square) runs one test per column, and this chapter is what ten simultaneous 2.15 percents cost.

40.1. one threshold for ten tests

The naive way to fire ten tests at 5 percent each is to fire them, and collect roughly one false alarm per pipeline by accident. Bonferroni's fix is one threshold for the whole family: reject when $p \le \alpha/m$, which caps the chance of any false rejection at alpha no matter how the p-values correlate. List P carries ten pinned p-values, H1 through H10, and the sample sorts them once, asserted as "ch40 P ascending sort order" with `P sorted=H2,H6,H5,H4,H8,H3,H1,H9,H10,H7`.

The dry run: the threshold is $(1/20)/10 = 1/200$, printed `P bonf_thr=1/200 reject={H2,H6} n=2` and asserted as "ch40 P bonferroni thr 1/200 n=2". Only H2 at 1/1000 = 0.001 and H6 at 9/2000 = 0.0045 duck under it; H5 at 1/160 = 0.00625 misses by a quarter of the bar. The price is the family-size block: the very same p = 1/160 against $m = 5$ gives threshold $(1/20)/5 = 1/100$ and `familysize m=5 thr=1/100 reject=true`, asserted as "ch40 1/160 clears the m=5 bonferroni bar 1/100"; against $m = 50$ the threshold is 1/1000 and `familysize m=50 thr=1/1000 reject=false`, "ch40 1/160 fails the m=50 bonferroni bar 1/1000". Yesterday's discovery evaporates the day forty innocent tests join the family, which is Bonferroni's honesty and its weakness in one row.

```
int main(void) {
  /* list P: presentation order H1..H10 */
  static const Hyp P[10] = {
      {"H1", {1, 5}},   {"H2", {1, 1000}}, {"H3", {3, 20}}, {"H4", {7, 400}},
      {"H5", {1, 160}}, {"H6", {9, 2000}}, {"H7", {9, 10}}, {"H8", {7, 200}},
      {"H9", {1, 4}},   {"H10", {3, 10}}};
  static const char *const P_sorted[10] = {"H2", "H6", "H5", "H4",  "H8",
                                           "H3", "H1", "H9", "H10", "H7"};
  static const int P_bonf_thr[2] = {1, 200};
  static const char *const P_bonf_ids[2] = {"H2", "H6"};
  static const int P_holm_thr[10][2] = {{1, 200}, {1, 180}, {1, 160}, {1, 140},
                                        {1, 120}, {1, 100}, {1, 80},  {1, 60},
                                        {1, 40},  {1, 20}};
  static const int P_holm_stop[10] = {0, 0, 0, 1, 1, 1, 1, 1, 1, 1};
  static const int P_bh_thr[10][2] = {{1, 200}, {1, 100}, {3, 200}, {1, 50},
                                      {1, 40},  {3, 100}, {7, 200}, {1, 25},
                                      {9, 200}, {1, 20}};
  static const int P_bh_le[10] = {1, 1, 1, 1, 0, 0, 0, 0, 0, 0};
  static const int P_padj[10][2] = {{1, 100}, {1, 48}, {1, 48}, {7, 160},
                                    {7, 100}, {1, 4},  {2, 7},  {5, 16},
                                    {1, 3},   {9, 10}};
  static const int P_padj_rej[10] = {1, 1, 1, 1, 0, 0, 0, 0, 0, 0};
  if (run_list("P", P, 10, P_sorted, P_bonf_thr, P_bonf_ids, 2, P_holm_thr,
               P_holm_stop, 3, P_bh_thr, P_bh_le, 4, P_padj, P_padj_rej)) {
    return 1;
  }
```

listing 298: list P: ten pinned p-values and every expected step of every procedure (source: kdd/samples/src/Ch40/fdr.c, lines 227-252)

```
  /* bonferroni: single threshold alpha/m */
  const Frac bthr = fr(alpha.n, alpha.d * m);
  int bn = 0;
  const char *bids[10];
  for (int i = 0; i < m; i++) {
    if (fcmp(h[ord[i]].p, bthr) <= 0) {
      bids[bn++] = h[ord[i]].id;
    }
  }
  printf("%s bonf_thr=%d/%d reject={", label, bthr.n, bthr.d);
  for (int i = 0; i < bn; i++) {
    printf("%s%s", i ? "," : "", bids[i]);
  }
  printf("} n=%d\n", bn);
  int bok = bn == bonf_n_want;
  for (int i = 0; i < bn; i++) {
    bok = bok && strcmp(bids[i], bonf_ids_want[i]) == 0;
  }
  snprintf(name, sizeof name, "ch40 %s bonferroni thr %d/%d n=%d", label,
           bonf_thr_want[0], bonf_thr_want[1], bonf_n_want);
  CHECK(bthr.n == bonf_thr_want[0] && bthr.d == bonf_thr_want[1] && bok, name);
```

listing 299: bonferroni: one threshold alpha/m, cross-multiplied against every p (source: kdd/samples/src/Ch40/fdr.c, lines 117-137)

```
/* family-size effect: the same p = 1/160 under different family sizes */
const Frac p = fr(1, 160);
const Frac thr5 = fr(1, 100), thr50 = fr(1, 1000);
printf("familysize p=1/160\n");
printf("familysize m=5 thr=1/100 reject=%s\n",
       fcmp(p, thr5) ≤ 0 ? "true" : "false");
printf("familysize m=50 thr=1/1000 reject=%s\n",
       fcmp(p, thr50) ≤ 0 ? "true" : "false");
CHECK(fcmp(p, thr5) ≤ 0, "ch40 1/160 clears the m=5 bonferroni bar 1/100");
CHECK(fcmp(p, thr50) > 0, "ch40 1/160 fails the m=50 bonferroni bar 1/1000");
```

listing 300: the same p under family sizes 5, 10, and 50 (source: kdd/samples/src/Ch40/fdr.c, lines 254-263)

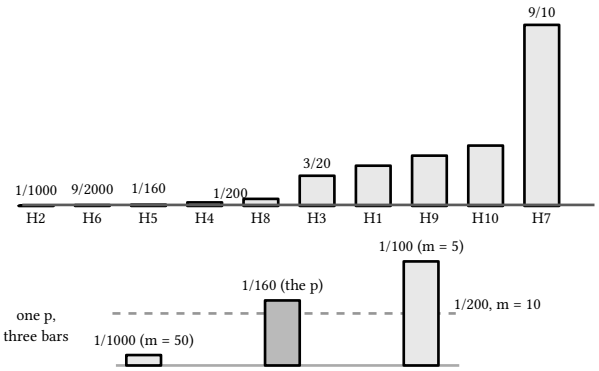


figure 161: list P against the 1/200 bar; below, one p against three family sizes

40.2. holm's shrinking bar

Bonferroni charges every hypothesis the full family tax. Holm's step-down walks the sorted p-values from smallest up and relaxes the bar as it goes: at step k the threshold is $\alpha/(m-k+1)$, and the first miss stops the walk, everything after it inherits the stop. The procedure controls the family-wise error at alpha like Bonferroni and rejects at least as much, for free.

The dry run: the sample prints one line per step, ten for list P. The first three read `holm H2 k=1 p=1/1000 thr=1/200 reject`, `holm H6 k=2 p=9/2000 thr=1/180 reject`, and the boundary row `holm H5 k=3 p=1/160 thr=1/160 reject`, where p equals the threshold exactly and the rule's <= lets it through, asserted as "ch40 P holm k=3 thr 1/160 reject". Step four is the first miss, `holm H4 k=4 p=7/400 thr=1/140 STOP`: 7/400 = 0.0175 against 1/140 = 0.0071, and the remaining six lines all print STOP with thresholds relaxing on paper, 1/120 down to 1/20, that nothing can spend. The summary prints `P holm_reject={H2,H6,H5} n=3`, asserted as "ch40 P holm rejects n=3". Bonferroni bought 2, Holm bought 3, and H5 is exactly the extra one, caught at the boundary where p equals the bar.

```
/* holm step-down: threshold alpha/(m - k + 1), stop at first miss */
int stopped = 0, hn = 0;
const char *hids[10];
for (int k = 0; k < m; k++) {
  const Frac thr = fr(alpha.n, alpha.d * (m - k));
  const int keep = !stopped && fcmp(h[ord[k]].p, thr) <= 0;
  printf("holm %s k=%d p=%d/%d thr=%d/%d %s\n", h[ord[k]].id, k + 1,
         h[ord[k]].p.n, h[ord[k]].p.d, thr.n, thr.d,
         keep ? "reject" : "STOP");
  snprintf(name, sizeof name, "ch40 %s holm k=%d thr %d/%d %s", label, k + 1,
           holm_thr_want[k][0], holm_thr_want[k][1],
           holm_stop_want[k] ? "STOP" : "reject");
  CHECK(thr.n == holm_thr_want[k][0] && thr.d == holm_thr_want[k][1] &&
            keep != holm_stop_want[k],
        name);
  if (keep) {
    hids[hn++] = h[ord[k]].id;
  } else {
    stopped = 1;
  }
}
printf("%s holm_reject={", label);
for (int i = 0; i < hn; i++) {
  printf("%s%s", i ? "," : "", hids[i]);
}
printf("} n=%d\n", hn);
snprintf(name, sizeof name, "ch40 %s holm rejects n=%d", label, holm_n_want);
CHECK(hn == holm_n_want, name);
```

listing 301: holm step-down: alpha/(m - k + 1), first miss stops the walk (source: kdd/samples/src/Ch40/fdr.c, lines 139-166)

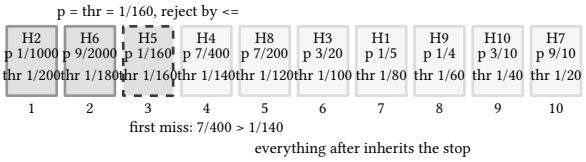


figure 162: holm's walk over sorted list P: three rejects, an exact boundary, one stop

40.3. the largest k that holds

Benjamini-Hochberg changes the currency: instead of bounding the chance of any false rejection, it bounds the expected share of rejections that are false, the false discovery rate, at most q. The rule reads backward from Holm: sort ascending, and take the largest k with $p(k) \le (k/m)q$; reject the first k. The bar rises with k instead of shrinking, so the game is finding how far up the sorted list the p-values can keep pace with a line that grows.

The dry run: the sample prints one comparison per step. The holding edge is `bh k=4 H4 p=7/400 thr=1/50 ≤`, asserted as "ch40 P bh k=4 thr 1/50 <=": 7/400 = 0.0175 against $(4/10)(1/20) = 1/50 = 0.02$. The next step misses, `bh k=5 H8 p=7/200 thr=1/40 >`, 0.035 against 0.025, and every later step misses by more. The largest k that holds is 4, so `P bh_kstar=4 reject={H2,H6,H5,H4} n=4`, asserted as "ch40 P bh k*=4": one more discovery than Holm, two more than Bonferroni, paid for in the weaker promise.

```
  /* benjamini-hochberg: largest k with p(k) ≤ (k/m) alpha */
  int kstar = 0;
  for (int k = 0; k < m; k++) {
    const Frac thr = fr((k + 1) * alpha.n, m * alpha.d);
    const int le = fcmp(h[ord[k]].p, thr) ≤ 0;
    if (le) {
      kstar = k + 1;
    }
    printf("bh k=%d %s p=%d/%d thr=%d/%d %s\n", k + 1, h[ord[k]].id,
           h[ord[k]].p.n, h[ord[k]].p.d, thr.n, thr.d, le ? "≤" : ">");
    snprintf(name, sizeof name, "ch40 %s bh k=%d thr %d/%d %s", label, k + 1,
             bh_thr_want[k][0], bh_thr_want[k][1], bh_le_want[k] ? "≤" : ">");
    CHECK(thr.n == bh_thr_want[k][0] && thr.d == bh_thr_want[k][1] &&
              le == bh_le_want[k],
          name);
  }
  printf("%s bh_kstar=%d reject={", label, kstar);
  for (int i = 0; i < kstar; i++) {
    printf("%s%s", i ? "," : "", h[ord[i]].id);
  }
  printf("} n=%d\n", kstar);
  snprintf(name, sizeof name, "ch40 %s bh k*=%d", label, kstar_want);
  CHECK(kstar == kstar_want, name);
```

listing 302: benjamini-hochberg: largest k with p(k) <= (k/m) q, reject the first k (source: kdd/samples/src/Ch40/fdr.c, lines 168-190)

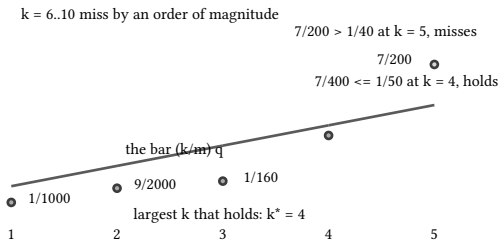


figure 163: sorted p against the rising (k/m) q bar: the pace holds through k = 4, breaks at k = 5

40.4. the staircase, and a stubborn list

The three procedures answer yes or no. The adjusted p-value answers how small q would have to be, and for BH it is a staircase: $\text{padj}(k) = \min_{j \geq k}(m/j)p(j)$, the backward running minimum of the scaled p-values, so a hypothesis can only inherit a larger value from the steps above it. The sample computes it in one backward sweep and prints one line per hypothesis, REJECT appended when the result fits under 1/20.

The dry run: the raw staircase for list P reads $(m/k)p(k)$ = 1/100, 9/400, 1/48, 7/160, 7/100, 1/4, 2/7, 5/16, 1/3, 9/10, and the sweep changes exactly one entry: 9/400 at k = 2 sits above the 1/48 waiting at k = 3, so it is absorbed and `padj H6=1/48 REJECT` matches `padj H5=1/48 REJECT`, a flat pair, asserted as "ch40 P padj(H6) = 1/48 REJECT". Four values fit the budget, `padj H2=1/100 REJECT` through `padj H4=7/160 REJECT`, the fifth `padj H8=7/100` does not, and the count closes the loop: `P padj_reject_n=4 agrees_kstar=true`, asserted as "ch40 P padj count 4 agrees with k*". Then list Q, six hypotheses built to embarrass the single threshold. Bonferroni at 1/120 rejects nothing, `Q bonf_thr=1/120 reject={} n=0`, and Holm stops on its first step, `holm H1 k=1 p=1/80 thr=1/120 STOP`. BH does not care: `bh k=2 H2 p=1/80 thr=1/60` ≤, so `Q bh_kstar=2 reject={H1,H2} n=2`, asserted as "ch40 Q bh k*=2". The Q staircase absorbs k = 1's 3/40 into k = 2's 3/80 and both print `padj H1=3/80 REJECT` and `padj H2=3/80 REJECT`. Two rules that cannot speak, one that can.

```
  /* adjusted p: backward running minimum of (m/k) p(k) */
  Frac padj[10];
  Frac run = fr(1, 1);
  int have = 0;
  for (int k = m - 1; k ≥ 0; k--) {
    const Frac raw = fr(m * h[ord[k]].p.n, (k + 1) * h[ord[k]].p.d);
    if (!have || fcmp(raw, run) < 0) {
      run = raw;
      have = 1;
    }
    padj[k] = run;
  }
  int prej = 0;
  for (int k = 0; k < m; k++) {
    const int rej = fcmp(padj[k], alpha) ≤ 0;
    if (rej) {
      prej++;
    }
    printf("padj %s=%d/%d%s\n", h[ord[k]].id, padj[k].n, padj[k].d,
           rej ? " REJECT" : "");
    snprintf(name, sizeof name, "ch40 %s padj(%s) = %d/%d%s", label,
             h[ord[k]].id, padj_want[k][0], padj_want[k][1],
             padj_rej_want[k] ? " REJECT" : "");
    CHECK(padj[k].n == padj_want[k][0] && padj[k].d == padj_want[k][1] &&
              rej == padj_rej_want[k],
          name);
  }
  printf("%s padj_reject_n=%d agrees_kstar=%s\n", label, prej,
         prej == kstar ? "true" : "false");
  snprintf(name, sizeof name, "ch40 %s padj count %d agrees with k*", label,
           kstar_want);
  CHECK(prej == kstar, name);
```

listing 303: the backward cummin staircase, one padj per hypothesis, count agreeing with k* (source: kdd/samples/src/Ch40/fdr.c, lines 192-223)

```
  /* list Q: bonferroni rejects nothing, BH rejects two */
  static const Hyp Q[6] = {{"H1", {1, 80}}, {"H2", {1, 80}}, {"H3", {1, 10}},
                           {"H4", {1, 5}},  {"H5", {2, 5}},  {"H6", {4, 5}}};
  static const char *const Q_sorted[6] = {"H1", "H2", "H3", "H4", "H5", "H6"};
  static const int Q_bonf_thr[2] = {1, 120};
  static const char *const Q_bonf_ids[1] = {""};
  static const int Q_holm_thr[6][2] = {{1, 120}, {1, 100}, {1, 80},
                                       {1, 60},  {1, 40},  {1, 20}};
  static const int Q_holm_stop[6] = {1, 1, 1, 1, 1, 1};
  static const int Q_bh_thr[6][2] = {{1, 120}, {1, 60}, {1, 40},
                                     {1, 30},  {1, 24}, {1, 20}};
  static const int Q_bh_le[6] = {0, 1, 0, 0, 0, 0};
  static const int Q_padj[6][2] = {{3, 80}, {3, 80},  {1, 5},
                                   {3, 10}, {12, 25}, {4, 5}};
  static const int Q_padj_rej[6] = {1, 1, 0, 0, 0, 0};
  if (run_list("Q", Q, 6, Q_sorted, Q_bonf_thr, Q_bonf_ids, 0, Q_holm_thr,
               Q_holm_stop, 0, Q_bh_thr, Q_bh_le, 2, Q_padj, Q_padj_rej)) {
    return 1;
  }
```

listing 304: list Q: bonferroni and holm reject nothing, BH rejects two (source: kdd/samples/src/Ch40/fdr.c, lines 265-283)

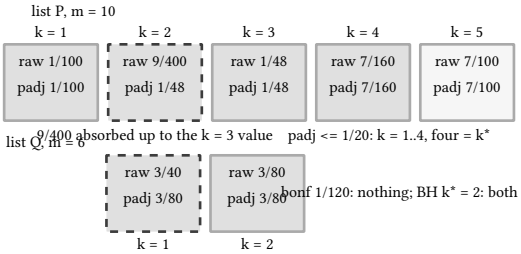


figure 164: the staircase as boxes: raw (m/k) p above, adjusted p below, lists P and Q

PITFALL: why 5 over 16, and not 5 over 8

The sheet records a conceded slip worth retelling: a first hand pass wrote the k = 8 raw staircase value as 5/8 and let padj(H9) slide into its neighbor's 1/3. The arithmetic is one division, $(10/8)(1/4) = 5/16$, and the corrected generator run pins `padj H9=5/16` alongside `padj H10=1/3`. The staircase is where the transcription errors live, ten multiplications and a running minimum in exact fractions, which is why every adjusted value is asserted as a reduced pair and the padj count must equal k*, a cross-check the sheet itself calls the hard one.

Three procedures, one trade: Bonferroni and Holm promise no false rejections at all and pay in power, BH promises a bounded share of them and pays in the promise. List P prices the spread at 2, 3, and 4 discoveries, list Q prices the single-threshold rules at zero. The statistics families close here, and the book turns to structure again with book 14, ch 41 (knowledge discovery, rough sets: indiscernibility and approximations).

sources: the Bonferroni correction, reject at alpha/m for a family of m tests, per the Boole-Bonferroni inequality as banked by kdd-contract-s8s9.md. Sture Holm, "A Simple Sequentially Rejective Multiple Test Procedure", Scandinavian Journal of Statistics 6(2) 1979, 65-70, the step-down with threshold alpha/(m - k + 1). Yoav Benjamini and Yosef Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing", Journal of the Royal Statistical Society, Series B 57(1) 1995, 289-300, the step-up and the adjusted-p staircase. All 60 pinned values witnessed by the sheet and playground/kdd-matrix/gen_s8.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch40`, 35 + 2 + 23 checks in chapter 40 of the kdd suite.

41. rough sets: indiscernibility and approximations

One sample carries the chapter: `rough.c` builds every indiscernibility partition of Pawlak's flu table, brackets the flu set between two approximations, grades each patient by rough membership, and exhibits one crisp set as the contrast, 27 checks. The chapter makes 4 moves: the partition an attribute subset induces, the lower and upper approximations that bracket a set no union of classes can express, the membership fraction that grades every object inside the boundary, and the crisp case where the whole bracket collapses. book 14, ch 13 (knowledge discovery, decision trees: id3) read a decision table with entropy and book 14, ch 15 (knowledge discovery, rules from trees and sequential covering) covered it with rules, both assuming every row's decision follows from its conditionals. This chapter is the part of the table those chapters quietly dropped: two patients with identical symptoms and opposite diagnoses, and what a theory can still say about them. The attribute-value triage that produced T's three ordered levels is the ordinal end of book 14, ch 3 (knowledge discovery, binning, smoothing, and discretization).

41.1. one table, seven partitions

Indiscernibility is the whole foundation: two objects are indiscernible under an attribute subset B when they agree on every attribute in B, and ind(B) is the partition of the table into classes of mutually indiscernible objects. The fixture is Pawlak's, 6 patients, 3 conditionals, headache H and muscle pain M binary, temperature T ordinal with values normal, high, vhigh, and the decision Flu. Three attributes give 7 nonempty subsets, each partition hand-checkable against the 6 rows.

The dry run: the sample first reprints all 6 rows, `row p1 H=n M=y T=high Flu=yes` through `row p6 H=n M=y T=vhigh Flu=yes`, each row a check. Then the 7 partitions, printed and checked one per subset: `ind {H}: {{p1,p4,p6},{p2,p3,p5}}`, `ind {M}: {{p1,p3,p4,p6},{p2,p5}}`, `ind {T}: {{p1,p2,p5},{p3,p6},{p4}}`, and on through `ind {H,T}: {{p1},{p2,p5},{p3},{p4},{p6}}`, `ind {M,T}: {{p1},{p2,p5},{p3,p6},{p4}}`, and `ind {H,M,T}: {{p1},{p2,p5},{p3},{p4},{p6}}`. Two facts jump out. Patients p2 and p5 share the full conditional tuple (y, n, high) yet carry opposite decisions, so they stay glued in every partition, even the one built from all three attributes. And the partitions for {H,T} and {H,M,T} are string-identical, the first hint that M is sometimes spendable, which is exactly the question book 14, ch 42 (knowledge discovery, rough sets: dependency and reducts) asks next.

```
/* partition by the attributes in mask (bit0=H, bit1=M, bit2=T); classes
 * ordered by smallest member, members ascending in p-index order */
static int partition(int mask, int cls[6][7]) {
  int n = 0;
  for (int p = 0; p < 6; p++) {
    int placed = -1;
    for (int c = 0; c < n; c++) {
      int same = 1;
      for (int b = 0; b < 3; b++) {
        if ((mask & (1 << b)) && attr(b, p) != attr(b, cls[c][0]))
          same = 0;
      }
      if (same) {
        placed = c;
        break;
      }
    }
    if (placed < 0) {
      placed = n++;
      cls[placed][6] = 0;
    }
    cls[placed][cls[placed][6]++] = p;
  }
  return n;
}
```

listing 305: one pass over the table builds a partition: classes by smallest member, members ascending (source: kdd/samples/src/Ch41/rough.c, lines 43-67)

```
  struct {
    int mask;
    const char *name, *want;
  } subs[7] = {
      {1, "H", "{{p1,p4,p6},{p2,p3,p5}}"},
      {2, "M", "{{p1,p3,p4,p6},{p2,p5}}"},
      {4, "T", "{{p1,p2,p5},{p3,p6},{p4}}"},
      {3, "H,M", "{{p1,p4,p6},{p2,p5},{p3}}"},
      {5, "H,T", "{{p1},{p2,p5},{p3},{p4},{p6}}"},
      {6, "M,T", "{{p1},{p2,p5},{p3,p6},{p4}}"},
      {7, "H,M,T", "{{p1},{p2,p5},{p3},{p4},{p6}}"},
  };
  char buf[128], nm[64];
  for (int s = 0; s < 7; s++) {
    partition_str(subs[s].mask, buf);
    printf("ind {%s}: %s\n", subs[s].name, buf);
    snprintf(nm, sizeof nm, "ch41 ind {%s} partition", subs[s].name);
    CHECK(strcmp(buf, subs[s].want) == 0, nm);
  }
```

listing 306: the 7 pinned partitions, one string check per subset (source: kdd/samples/src/Ch41/rough.c, lines 167-185)

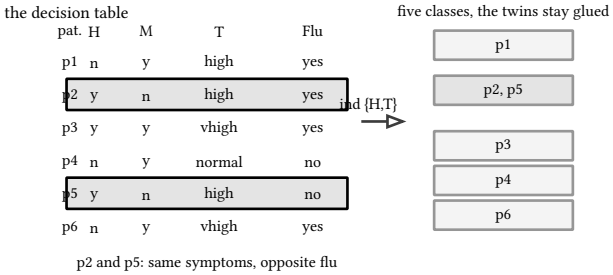


figure 165: the flu table and the partition its best pair of attributes induces

NOTE: the partition is the data the rough set sees

A rough set analysis never consults the rows directly after the partition is built. Every question this chapter and the next two answer, is a set a union of classes, how much of the decision do these attributes determine, which attributes can be dropped, is answered inside the lattice of partitions. The 6-row table and the twin patients enter only through ind(B), which is why the same fixture can carry three chapters.

41.2. lower, upper, boundary

Take the set of flu patients Xyes = {p1, p2, p3, p6} and ask which union of ind({H,M,T}) classes equals it. None does, because the class {p2, p5} straddles the decision. The theory's answer is a bracket. The lower approximation is the union of classes fully inside X, objects that certainly have flu given their symptoms. The upper approximation is the union of classes that touch X at all, objects that possibly have flu. The boundary is upper minus lower, and the accuracy |lower|/|upper| measures how tight the bracket is.

The dry run: for Xyes the sample prints `Xyes lower={p1,p3,p6} upper={p1,p2,p3,p5,p6} boundary={p2,p5} accuracy=3/5`, asserted as "ch41 Xyes lower/upper/boundary" plus "ch41 Xyes accuracy 3/5" by cross-multiplication, $3 \cdot 5 = 5 \cdot 3$. The upper approximation {p1,p2,p3,p5,p6} is one of the values printed in Pawlak's 1982 paper itself, an external cross-check on top of the hand derivation. For the complement Xno = {p4, p5} the row reads `Xno lower={p4} upper={p2,p4,p5} boundary={p2,p5} accuracy=1/3`. Both decisions share the same boundary {p2,p5}, which is the structural statement of the problem: the twins are ambiguous for flu and for its negation at once, and the yes-bracket is tighter than the no-bracket, 3/5 against 1/3.

```
static void approx(const int *X, int cls[6][7], int n, int *lo, int *nl,
                   int *up, int *nu, int *bn, int *nb) {
  *nl = *nu = *nb = 0;
  for (int c = 0; c < n; c++) {
    int allin = 1, anyin = 0;
    for (int i = 0; i < cls[c][6]; i++) {
      if (X[cls[c][i]])
        anyin = 1;
      else
        allin = 0;
    }
    if (allin)
      for (int i = 0; i < cls[c][6]; i++)
        lo[(*nl)++] = cls[c][i];
    if (anyin)
      for (int i = 0; i < cls[c][6]; i++)
        up[(*nu)++] = cls[c][i];
  }
  for (int i = 0; i < *nu; i++) {
    int inlo = 0;
    for (int j = 0; j < *nl; j++)
      if (up[i] == lo[j])
        inlo = 1;
    if (!inlo)
      bn[(*nb)++] = up[i];
  }
  sort_members(lo, *nl);
  sort_members(up, *nu);
  sort_members(bn, *nb);
}
```

listing 307: the bracket: all-in classes to the lower, any-in classes to the upper, the rest is boundary (source: kdd/samples/src/Ch41/rough.c, lines 119-148)

```
  struct {
    const char *nm;
    const int *X;
    const char *lo, *up, *bn;
    long long an, ad;
  } rows[2] = {
      {"yes", Xyes, "p1,p3,p6", "p1,p2,p3,p5,p6", "p2,p5", 3, 5},
      {"no", Xno, "p4", "p2,p4,p5", "p2,p5", 1, 3},
  };
  for (int r = 0; r < 2; r++) {
    int lo[6], nl, up[6], nu, bn[6], nb;
    approx(rows[r].X, cls, n, lo, &nl, up, &nu, bn, &nb);
    char ls[48], us[48], bs[48], ac[16];
    members_str(lo, nl, ls);
    members_str(up, nu, us);
    members_str(bn, nb, bs);
    frac_str(nl, nu, ac);
    printf("X%s lower={%s} upper={%s} boundary={%s} accuracy=%s\n", rows[r].nm,
           ls, us, bs, ac);
    snprintf(nm, sizeof nm, "ch41 X%s lower/upper/boundary", rows[r].nm);
    CHECK(strcmp(ls, rows[r].lo) == 0 && strcmp(us, rows[r].up) == 0 &&
              strcmp(bs, rows[r].bn) == 0,
          nm);
    snprintf(nm, sizeof nm, "ch41 X%s accuracy %lld/%lld", rows[r].nm,
             rows[r].an, rows[r].ad);
    CHECK(frac_eq(nl, nu, rows[r].an, rows[r].ad), nm);
  }
```

listing 308: both decision sets approximated, accuracies as cross-multiplied pairs (source: kdd/samples/src/Ch41/rough.c, lines 196-222)

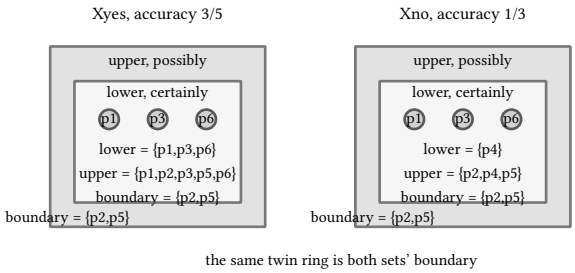


figure 166: the bracket around each decision: shaded boundary ring between lower and upper

41.3. rough membership

The boundary is not featureless mush, it has grades. The rough membership of object p in X under the full conditionals is $\mu_X(p) = |[p] \cap X| \;/\; |[p]|$, the fraction of p's indiscernibility class that lies in X. Objects in pure classes score 1 or 0, the twins score exactly 1/2, and the function agrees with the bracket: scoring 1 is being in the lower, scoring 0 is being outside the upper, anything between is boundary.

The dry run: six rows, `mu_Xyes p1=1`, `mu_Xyes p2=1/2`, `mu_Xyes p3=1`, `mu_Xyes p4=0`, `mu_Xyes p5=1/2`, `mu_Xyes p6=1`, each asserted as its own check, "ch41 mu_Xyes p2=1/2" among them, the fraction compared by cross-multiplication, one hit in a two-member class against the reduced pair 1 over 2. The class {p1} is pure yes so p1 scores 1, the class {p4} is pure no so p4 scores 0, and the class {p2,p5} holds one yes and one no, so both twins score 1/2. The membership function is where the rough set touches book 14, ch 19 (knowledge discovery, naive bayes), a posterior per object, but earned by counting a partition instead of assuming independence.

```
  /* rough membership mu_Xyes(p) = |class(p) cap Xyes| / |class(p)| */
  static const long long MU[6][2] = {{1, 1}, {1, 2}, {1, 1},
                                     {0, 1}, {1, 2}, {1, 1}};
  for (int p = 0; p < 6; p++) {
    int c = 0;
    while (c < n) {
      int f = 0;
      for (int i = 0; i < cls[c][6]; i++)
        if (cls[c][i] == p)
          f = 1;
      if (f)
        break;
      c++;
    }
    long long hit = 0;
    for (int i = 0; i < cls[c][6]; i++)
      if (Xyes[cls[c][i]])
        hit++;
    char ms[16], pw[16];
    frac_str(hit, cls[c][6], ms);
    frac_str(MU[p][0], MU[p][1], pw);
    printf("mu_Xyes p%d=%s\n", p + 1, ms);
    snprintf(nm, sizeof nm, "ch41 mu_Xyes p%d=%s", p + 1, pw);
    CHECK(frac_eq(hit, cls[c][6], MU[p][0], MU[p][1]), nm);
```

listing 309: membership as a class census, fractions checked cross-multiplied (source: kdd/samples/src/Ch41/rough.c, lines 224-247)

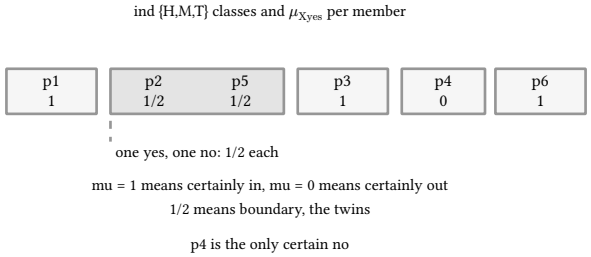


figure 167: the five classes of the full partition, each member carrying its membership grade

41.4. the crisp contrast

Not everything in this table is rough. Take Z = {p3, p6}, the two very-high-fever patients, and bracket it under the same full partition: the class {p3} is inside, the class {p6} is inside, no other class touches Z, so lower, upper and Z itself are the same set and the boundary is empty. Z is crisp, or in the lattice language, exactly definable, and it is definable by a single descriptor besides: the T = vhigh class of the one-attribute partition ind({T}) is already {p3, p6}.

The dry run: the sample prints `Z={p3,p6} lower={p3,p6} upper={p3,p6}` `boundary={} crisp=true`, three checks, “ch41 Z={p3,p6} crisp lower=upper”, “ch41 Z boundary empty crisp=true”, and then `definable` `Z=T:vhigh equal=true`, asserted as “ch41 definable Z=T:vhigh equal=true” by fishing the T = vhigh class out of ind({T}) and string-comparing it to {p3,p6}. The contrast is the chapter’s landing point: Xyes needed a bracket and a membership grade because the conditionals underdetermine the decision, Z needs neither because one attribute value pins it. Which subsets B still determine the decision as well as all three do is not a matter of inspection anymore, and that degree is the business of book 14, ch 42 (knowledge discovery, rough sets: dependency and reducts).

```
/* Z = {p3,p6} is exactly one class union: crisp (lower == upper) */
int Z[6] = {0, 0, 1, 0, 0, 1};
int lo[6], nl, up[6], nu, bn[6], nb;
approx(Z, cls, n, lo, &nl, up, &nu, bn, &nb);
char ls[48], us[48], bs[48];
members_str(lo, nl, ls);
members_str(up, nu, us);
members_str(bn, nb, bs);
printf("Z={p3,p6} lower={%s} upper={%s} boundary={%s} crisp=%s\n", ls, us, bs,
       nl == nu ? "true" : "false");
CHECK(strcmp(ls, "p3,p6") == 0 && strcmp(us, "p3,p6") == 0 && nb == 0,
      "ch41 Z={p3,p6} crisp lower=upper");
CHECK(nl == nu, "ch41 Z boundary empty crisp=true");

/* Z is definable by the single descriptor T=vhigh: the T=vhigh class of
 * ind {T} is exactly {p3,p6} */
int tcls[6][7], tn = partition(4, tcls), zc = -1;
for (int c = 0; c < tn; c++)
  for (int i = 0; i < tcls[c][6]; i++)
    if (tcls[c][i] == 2)
      zc = c;
char zs[48];
members_str(tcls[zc], tcls[zc][6], zs);
printf("definable Z=T:vhigh equal=%s\n",
       strcmp(zs, "p3,p6") == 0 ? "true" : "false");
CHECK(strcmp(zs, "p3,p6") == 0, "ch41 definable Z=T:vhigh equal=true");
```

listing 310: the crisp set: empty boundary, then definability by the single descriptor T=vhigh (source: kdd/samples/src/Ch41/rough.c, lines 250-275)

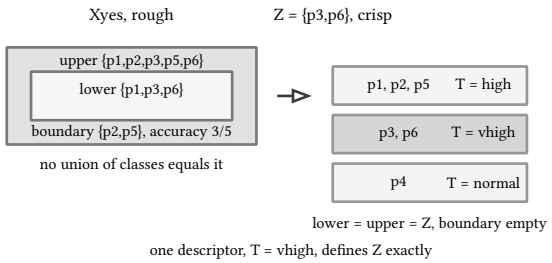


figure 168: Xyes needs a bracket, Z is exactly one class of ind {T}

sources: the flu decision table and the approximation framework are Z. Pawlak, Rough Sets, International Journal of Computer and Information Sciences 11(5) 1982, 341-356, the running example of the paper, with the upper approximation {p1,p2,p3,p5,p6} and the Headache classes matching its printed values, corroborated against the reprint excerpted at mimuw.edu.pl, accessed 2026-09-22, banked by kdd-contract-s8s9.md. The membership grading belongs to the same tradition and reappears as decision-relative support in book 14, ch 43 (knowledge discovery, rough sets: discernibility and rule induction). All 27 pinned values witnessed by kdd-contract-s8s9.md and playground/kdd-matrix/gen_s9.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch41`, 27 checks in chapter 41 of the kdd suite.

42. rough sets: dependency and reducts

One sample carries the chapter: `reduct.c` scores all 7 nonempty attribute subsets of the flu table by dependency degree, enumerates the reducts, intersects them into the core, plants the required non-reduct row for the full set, and walks three monotonicity edges, 23 checks. The chapter makes 4 moves: the positive region and the degree k that prices an attribute subset, the reducts that achieve full dependency at minimum cost, the core that no reduct can do without, and the monotone lattice that makes the whole search sane. book 14, ch 41 (knowledge discovery, rough sets: indiscernibility and approximations) built the partitions, this chapter ranks them, and book 14, ch 43 (knowledge discovery, rough sets: discernibility and rule induction) will derive the same reducts a second way, as Boolean prime implicants. The question is the one book 14, ch 8 (knowledge discovery, feature selection and chi-square) asked with chi-square over candidate splits, here answered exactly: which columns can the table spare without deciding any patient differently.

42.1. the positive region and the degree

A subset B of conditionals determines an object's decision exactly when B's partition glues it only to same-decision patients. The union of all such objects is the positive region POS_B(Flu), and the dependency degree is $k(B) = |\mathrm{POS}_B|/6$, the fraction of the table B decides for sure. The twins of chapter 41 cap the degree at 4/6 no matter what, because p2 and p5 agree on all three conditionals, so their class is mixed under every subset, even the full one.

The dry run: seven rows, `k {H}=0 POS={}`, `k {M}=0 POS={}`, `k {T}=1/2 POS={p3,p4,p6}`, `k {H,M}=1/6 POS={p3}`, `k {H,T}=2/3 POS={p1,p3,p4,p6}`, `k {M,T}=2/3 POS={p1,p3,p4,p6}`, `k {H,M,T}=2/3 POS={p1,p3,p4,p6}`, each degree and each member string its own check. Reading them off the partitions is one step each: under {T} the classes {p3,p6} and {p4} are decision-constant while {p1,p2,p5} mixes, so POS is 3 of 6 and k = 1/2. Under {H,M} only the singleton {p3} is constant, k = 1/6. Under {H,T} five classes form and only {p2,p5} mixes, so the positive region is the other four patients and k = 4/6 = 2/3, the ceiling, since no subset can ever classify the twins. Headache and muscle pain alone decide nothing, k = 0 twice over: neither attribute's partition separates a single certain patient.

```
/* positive region: union (ascending) of classes whose Flu decision is
 * constant; also returns |POS| */
static int pos(int mask, int *out) {
  int seen[6] = {0}, n = 0;
  for (int p = 0; p < 6; p++) {
    if (seen[p])
      continue;
    int cls[6], m = 0, mixed = 0;
    for (int q = p; q < 6; q++) {
      int same = 1;
      for (int b = 0; b < 3; b++)
        if ((mask & (1 << b)) && attr(b, q) != attr(b, p))
          same = 0;
      if (same) {
        seen[q] = 1;
        cls[m++] = q;
        if (FLU[q] != FLU[p])
          mixed = 1;
      }
    }
    if (!mixed)
      for (int i = 0; i < m; i++)
        out[n++] = cls[i];
  }
  for (int i = 1; i < n; i++) { /* members ascending, like the generator */
    int v = out[i], j = i - 1;
    while (j >= 0 && out[j] > v) {
      out[j + 1] = out[j];
      j--;
    }
    out[j + 1] = v;
  }
  return n;
}
```

listing 311: the positive region: classes whose Flu decision is constant, members ascending (source: kdd/samples/src/Ch42/reduct.c, lines 38-71)

```
  struct {
    int mask;
    const char *name, *k, *P;
    long long kn, kd;
  } subs[7] = {
      {1, "H", "0", "", 0, 6},
      {2, "M", "0", "", 0, 6},
      {4, "T", "1/2", "p3,p4,p6", 1, 2},
      {3, "H,M", "1/6", "p3", 1, 6},
      {5, "H,T", "2/3", "p1,p3,p4,p6", 2, 3},
      {6, "M,T", "2/3", "p1,p3,p4,p6", 2, 3},
      {7, "H,M,T", "2/3", "p1,p3,p4,p6", 2, 3},
  };
  char buf[64], nm[72];
  for (int s = 0; s < 7; s++) {
    int P[6], n = pos(subs[s].mask, P);
    char ks[16], ps[48] = "";
    pos_str(subs[s].mask, ps);
    frac_str(n, 6, ks);
    printf("k {%s}=%s POS={%s}\n", subs[s].name, ks, ps);
    snprintf(nm, sizeof nm, "ch42 k {%s}=%s", subs[s].name, subs[s].k);
    CHECK(frac_eq(n, 6, subs[s].kn, subs[s].kd), nm);
    snprintf(nm, sizeof nm, "ch42 POS {%s}={%s}", subs[s].name, subs[s].P);
    CHECK(strcmp(ps, subs[s].P) == 0, nm);
  }
```

listing 312: all 7 degrees and positive regions, fractions cross-multiplied (source: kdd/samples/src/Ch42/reduct.c, lines 101-125)

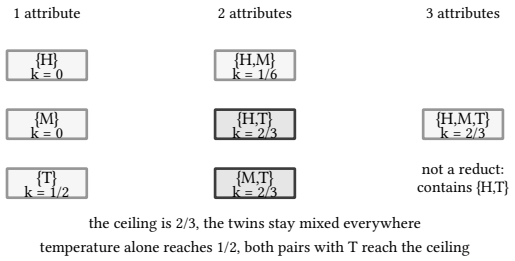


figure 169: all 7 nonempty subsets by size, degree k under each, reducts boxed

42.2. reducts, the minimal full-dependency subsets

A reduct is a subset whose dependency equals the full set's and which contains no smaller such subset. The sample enumerates all 7 subsets by size then attribute order, keeps those with k = 2/3, and discards any that contain an earlier keep. Two survive and none else can, because {T} scores 1/2 and every pair without T scores at most 1/6.

The dry run: the sample prints `reducts={H,T};{M,T}`, asserted twice, once as the ordered pair of masks, "ch42 reducts are exactly {H,T} and {M,T} in order", once as the string, "ch42 reducts={H,T};{M,T}". Both reducts reproduce the published reduct set of Pawlak's table, the literature cross-check on top of the enumeration. The two reducts say something asymmetric about the table: headache pairs with temperature, or muscle pain pairs with temperature, but temperature is the one attribute both need, and either binary symptom can substitute for the other. The 7-subset enumeration is the entire search space here, small enough to exhaust, and the cap is what keeps it honest, the exponential subset lattice beyond 8 attributes is exactly why the discernibility route of book 14, ch 43 (knowledge discovery, rough sets: discernibility and rule induction) exists.

```
/* reducts: subsets with k = k(full), enumerated by size then attribute
 * order H,M,T, supersets of an earlier reduct excluded */
static const int ORDER[7] = {1, 2, 4, 3, 5, 6, 7};
int P[6], kfulln = pos(7, P);
int red[8], nred = 0;
for (int i = 0; i < 7; i++) {
  int Q[6], kn = pos(ORDER[i], Q);
  if (!frac_eq(kn, 6, kfulln, 6))
    continue;
  int sub = 0;
  for (int r = 0; r < nred; r++)
    if ((red[r] & ORDER[i]) == red[r] && red[r] != ORDER[i])
      sub = 1;
  if (!sub)
    red[nred++] = ORDER[i];
}
int off = 0;
static const char *AN[3] = {"H", "M", "T"};
for (int r = 0; r < nred; r++) {
  off += snprintf(buf + off, 24, "%s%s", r ? ";" : "", "{");
  int first = 1;
  for (int b = 0; b < 3; b++)
    if (red[r] & (1 << b)) {
      off += snprintf(buf + off, 24, "%s%s", first ? "" : ",", AN[b]);
      first = 0;
    }
  off += snprintf(buf + off, 24, "}");
}
printf("reducts=%s\n", buf);
CHECK(nred == 2 && (red[0] == 5 && red[1] == 6), "ch42 reducts are exactly "
                                                 "{H,T} and {M,T} in order");
CHECK(strcmp(buf, "{H,T};{M,T}") == 0, "ch42 reducts={H,T};{M,T}");
```

listing 313: exhaustive enumeration, size first, supersets of found reducts dropped (source: kdd/samples/src/Ch42/reduct.c, lines 127-158)

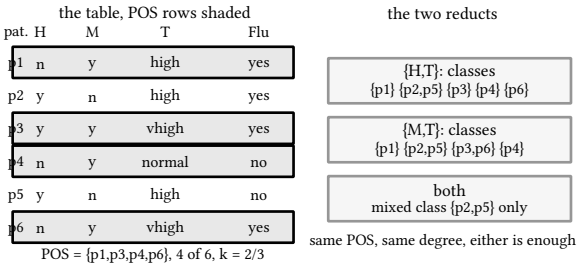


figure 170: both reducts leave the same four patients certain and the twins ambiguous

NOTE: a reduct is feature selection with a proof

Dropping M from {H,M,T} changes nothing the analysis can see: the partition, the positive region and the degree are all identical, so the smaller set is certified equivalent, not merely correlated. The chi-square screen of book 14, ch 8 (knowledge discovery, feature selection and chi-square) ranks columns by evidence, a reduct asserts exact equality of what the columns determine, which is why the search is exponential and the book keeps the fixture at 3 attributes, 7 subsets, every one of them checked.

42.3. the core and the full set that is not a reduct

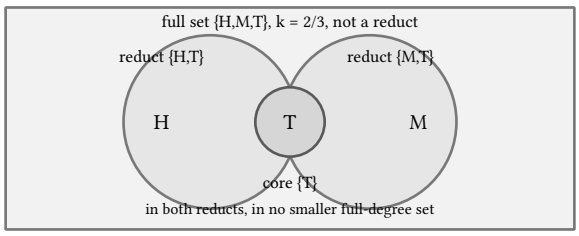
The core is the intersection of all reducts, the attributes no reduct can spare. With reducts {H,T} and {M,T} the intersection is {T} alone, so temperature is indispensable: no subset achieving k = 2/3 exists without it, consistent with {T} being the only singleton reaching 1/2. The full set {H,M,T} then supplies the required counterexample row. Its degree is 2/3, equal to the reducts, its positive region is {p1,p3,p4,p6}, member for member equal, yet it is not a reduct, because minimality fails, it strictly contains {H,T}.

The dry run: `core={T}`, asserted as “ch42 core={T} (intersection of reducts)”, then `full_set_nonreduct k=2/3 equal_dep=true POS_equal=true` with the three checks “ch42 full set dependency equal 2/3”, “ch42 full set POS equal to {H,T} POS”, and “ch42 {H,M,T} contains reduct {H,T}, not a reduct”. Equal numbers do not make a reduct, the containment clause does. This is the row that stops anyone reading reducts off a k table by threshold alone: scanning for k = 2/3 catches {H,M,T} first if the scan runs in mask order, and the enumeration in the sample only avoids it by checking subsets against the already-found reducts.

```
/* core = intersection of the reducts */
int core = 7;
for (int r = 0; r < nred; r++)
  core &= red[r];
printf("core={%s}\n", core == 4 ? "T" : "?");
CHECK(core == 4, "ch42 core={T} (intersection of reducts)");

/* full-set non-reduct row: equal dependency, equal POS, still not a
 * reduct because it strictly contains {H,T} */
int PA[6], PB[6], na = pos(7, PA), nb = pos(5, PB);
int poseq = na == nb;
if (poseq)
  for (int i = 0; i < na; i++)
    if (PA[i] != PB[i])
      poseq = 0;
printf("full_set_nonreduct k=%s equal_dep=%s POS_equal=%s\n", "2/3",
       frac_eq(na, 6, kfulln, 6) ? "true" : "false",
       poseq ? "true" : "false");
CHECK(frac_eq(na, 6, kfulln, 6), "ch42 full set dependency equal 2/3");
CHECK(poseq, "ch42 full set POS equal to {H,T} POS");
int contains = 0;
for (int r = 0; r < nred; r++)
  if ((red[r] & 7) == red[r] && red[r] != 7)
    contains = 1;
CHECK(contains, "ch42 {H,M,T} contains reduct {H,T}, not a reduct");
```

listing 314: core by intersection, then the non-reduct row: equal k, equal POS, still not minimal (source: kdd/samples/src/Ch42/reduct.c, lines 160-184)



M sits outside one reduct, H outside the other, T inside both

figure 171: the core is what both reducts share, the full set contains both and is too big

42.4. monotone along the lattice

Adding an attribute can only refine a partition, and refining can only grow the positive region, so k must be nondecreasing along every lattice edge. The sample walks three edges and compares the endpoint degrees as exact fractions by cross-multiplication, no floating point anywhere near the claim.

The dry run: `monotone H≤HM=true` compares 0 against 1/6, `monotone T≤HT=true` compares 1/2 against 2/3, which is $1 \cdot 3 = 3 \le 4 = 2 \cdot 2$, and `monotone HM≤HMT=true` compares 1/6 against 2/3, $1 \cdot 3 = 3 \le 12 = 6 \cdot 2$. Three checks, each an edge of the cube in the first figure. Monotonicity is what licenses the search pattern the enumeration used, subsets ordered by size, because a set that already misses the ceiling cannot be rescued by supersets that contain a reduct. The other direction is the useful one for pruning: any subset B with $k(B)$ below the full degree can still sit inside a reduct path, {T} with k = 1/2 does, so monotonicity bounds the search, it does not collapse it.

```c
/* monotonicity: dependency is nondecreasing along lattice edges,
 * compared as exact fractions */
struct {
  int a, b;
  const char *nm;
} edges[3] = {{1, 3, "H≤HM"}, {4, 5, "T≤HT"}, {3, 7, "HM≤HMT"}};
for (int e = 0; e < 3; e++) {
  int QA[6], QB[6], ka = pos(edges[e].a, QA), kb = pos(edges[e].b, QB);
  printf("monotone %s=%s\n", edges[e].nm,
         ka * 6 ≤ kb * 6 ? "true" : "false");
  snprintf(nm, sizeof nm, "ch42 monotone %s=true", edges[e].nm);
  CHECK(ka * 6 ≤ kb * 6, nm);
}
```

listing 315: three lattice edges, endpoint degrees compared by cross-multiplication (source: kdd/samples/src/Ch42/reduct.c, lines 186-198)

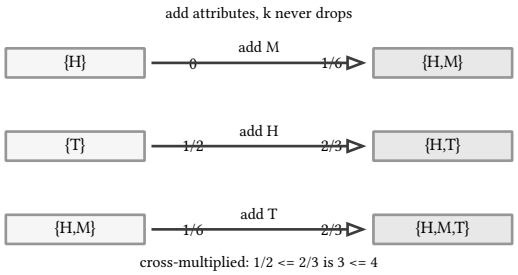


figure 172: k is nondecreasing up every edge, three witnessed climbs

sources: the dependency degree, reduct and core definitions follow Z. Pawlak, Rough Sets, International Journal of Computer and Information Sciences 11(5) 1982, 341-356, and the two reducts {Headache,Temperature} and {Muscle-pain,Temperature} with the nonempty core {Temperature} reproduce the published values for this table, corroborated against the reprint excerpted at mimuw.edu.pl, accessed 2026-09-22, banked by kdd-contract-s8s9.md. All 23 pinned values witnessed by kdd-contract-s8s9.md and playground/kdd-matrix/gen_s9.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch42`, 23 checks in chapter 42 of the kdd suite.

43. rough sets: discernibility and rule induction

One sample carries the chapter: `discern.c` fills the 15-cell discernibility matrix of the flu table, multiplies the 7 informative cells into the discernibility function, minimizes it to two prime implicants by brute force, and reads a certain-rule set out of each reduct, 31 checks. The chapter makes 4 moves: the matrix that names the attributes separating every pair, the empty cell that is an inconsistency rather than a requirement, the Boolean function whose prime implicants are exactly the reducts of book 14, ch 42 (knowledge discovery, rough sets: dependency and reducts), and the rules the reducts license, with the one ambiguous cell left honestly unclassified. book 14, ch 41 (knowledge discovery, rough sets: indiscernibility and approximations) bracketed what the table cannot decide, this chapter names the pairs it cannot separate and converts the survivors into the rule language of book 14, ch 15 (knowledge discovery, rules from trees and sequential covering).

43.1. the matrix of separating attributes

Cell D[pi][pj] lists the attributes on which patients i and j differ, nothing else. Pairs with the same decision are marked same-dec and discarded later, they never constrain a classifier that only needs to separate the classes. Pairs with different decisions carry the weight: any attribute set that hopes to determine the decision must, for each such pair, contain at least one attribute from the cell. One pair has nothing to offer: p2 and p5 agree on all three conditionals and differ only in the decision, so their cell is empty, tagged conflict, and it is the matrix portrait of chapter 41′s boundary.

The dry run: 15 rows, `D[p1][p2]={H,M} same-dec` down to `D[p5][p6]={H,M,T}`, one string check each. The seven different-decision cells are the load-bearing ones, `D[p1][p4]={T}`, `D[p1][p5]={H,M}`, `D[p2][p4]={H,M,T}`, `D[p3][p4]={H,T}`, `D[p3][p5]={M,T}`, `D[p4][p6]={T}`, `D[p5][p6]={H,M,T}`, and the conflict row reads exactly `D[p2][p5]={} conflict`, asserted as “ch43 D[p2][p5]={} conflict”. Every cell is readable straight off the table, p1 = (n, y, high) against p4 = (n, y, normal) differs in temperature alone, which is the {T} cell, and temperature is the only attribute separating both (p1,p4) and (p4,p6), a first hint it cannot be dropped.

```c
/* differing-attribute bitmask of pi,pj (bit0=H bit1=M bit2=T) */
static int diff_mask(int i, int j) {
  int m = 0;
  for (int b = 0; b < 3; b++)
    if (attr(b, i) != attr(b, j))
      m |= 1 << b;
  return m;
}

static void mask_str(int m, const char *sep, char *buf) {
  int off = 0, first = 1;
  for (int b = 0; b < 3; b++)
    if (m & (1 << b)) {
      off += snprintf(buf + off, 16, "%s%s", first ? "" : sep, AN[b]);
      first = 0;
    }
}
```

listing 316: a cell is the differing-attribute set of its pair, three comparisons (source: kdd/samples/src/Ch43/discern.c, lines 41-57)

```
  static const char *CELLS[15] = {
      "D[p1][p2]={H,M} same-dec",
      "D[p1][p3]={H,T} same-dec",
      "D[p1][p4]={T}",
      "D[p1][p5]={H,M}",
      "D[p1][p6]={T} same-dec",
      "D[p2][p3]={M,T} same-dec",
      "D[p2][p4]={H,M,T}",
      "D[p2][p5]={} conflict",
      "D[p2][p6]={H,M,T} same-dec",
      "D[p3][p4]={H,T}",
      "D[p3][p5]={M,T}",
      "D[p3][p6]={H} same-dec",
      "D[p4][p5]={H,M,T} same-dec",
      "D[p4][p6]={T}",
      "D[p5][p6]={H,M,T}",
  };
  int k = 0;
  for (int i = 0; i < 6; i++)
    for (int j = i + 1; j < 6; j++) {
      int d = diff_mask(i, j);
      char ds[16] = "", row[48];
      mask_str(d, ",", ds);
      const char *tag = "";
      if (FLU[i] == FLU[j])
        tag = " same-dec";
      else if (!d)
        tag = " conflict";
      snprintf(row, sizeof row, "D[p%d][p%d]={%s}%s", i + 1, j + 1, ds, tag);
      printf("%s\n", row);
      snprintf(nm, sizeof nm, "ch43 %s", CELLS[k]);
      CHECK(strcmp(row, CELLS[k]) == 0, nm);
      k++;
    }
```

listing 317: all 15 pinned cells with the conflict and same-dec tags (source: kdd/samples/src/Ch43/discern.c, lines 98-131)

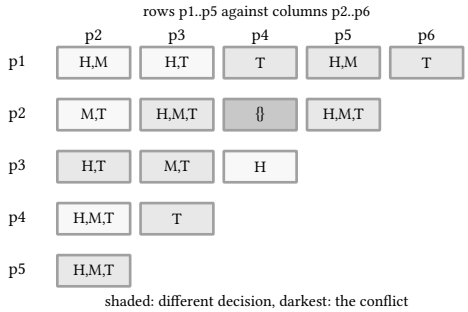


figure 173: the upper triangle: seven requirements, one conflict, seven same-dec fillers

43.2. the function, and the empty cell it drops

The discernibility function is the conjunction over all nonempty different-decision cells of the disjunction of that cell's attributes, a CNF whose satisfying assignments are exactly the attribute sets that separate every conflicting pair. The conflict cell contributes nothing: there is no disjunction over the empty set that a set of attributes could satisfy, and none is needed, because the pair is not a separation requirement but a proof the requirement system is unsatisfiable for those two objects.

The dry run: the sample builds the product in row-major pair order and prints `f_raw=(T) & (H|M) & (H|M|T) & (H|T) & (M|T) & (T) & (H|M|T)`, asserted as "ch43 f_raw 7-factor product", seven factors for the seven informative cells, (p2,p5) absent by construction. Minimizing is exact Boolean algebra, witnessed line by line in the sheet's absorption trace: $(T)(H|M)(H|M|T)(H|T)(M|T)$ drops the first $(H|M|T)$ because T absorbs it, then $T(H|T) = T$ and $T(M|T) = T$ collapse the two two-literal factors, leaving $(T)(H|M)$, which distributes to the minimal form the sample prints and checks, `f_min=H&T | M&T`.

```
/* raw discernibility function over nonempty different-decision cells */
int off = 0;
for (int i = 0; i < 6; i++)
  for (int j = i + 1; j < 6; j++)
    if (FLU[i] != FLU[j]) {
      int d = diff_mask(i, j);
      if (d) {
        char ds[16];
        mask_str(d, "|", ds);
        off += snprintf(buf + off, 16, "%s(%s)", off ? " & " : "", ds);
      }
    }
printf("f_raw=%s\n", buf);
CHECK(strcmp(buf, "(T) & (H|M) & (H|M|T) & (H|T) & (M|T) & (T) & (H|M|T)") ==
          0,
      "ch43 f_raw 7-factor product");
```

listing 318: the raw function: conjunction over informative cells of their disjunctions (source: kdd/samples/src/Ch43/discern.c, lines 133-148)

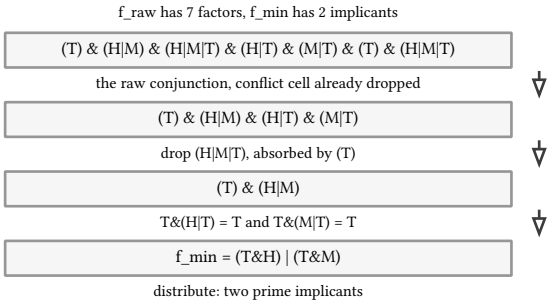


figure 174: absorption in three moves, from seven factors to two prime implicants

43.3. prime implicants, brute force, two ways

The sample does not trust the algebra, it hammers the function. A set of attributes covers when it intersects every nonempty different-decision cell, and the enumeration checks all 7 subsets in size order, rejecting any that contains an already-found cover. The two-way hardness is in the check structure: each surviving implicant must cover, asserted, and no smaller cover may exist, the singletons are asserted to fail first.

The dry run: "ch43 no single attribute covers (H,M,T each fail)" fires first, {H} and {M} both miss the (p1,p4) cell, whose only separator is T, and {T} misses (p1,p5), whose separators are H and M. Then the pair {H,T} covers all seven cells and {M,T} covers all seven, so the sample prints `f_min=H&T | M&T` and asserts "ch43 f_min=H&T | M&T" and "ch43 both prime implicants {H,T},{M,T} cover (two-way)". The cross-chapter agreement is its own check: the sample prints `prime set as reducts={H,T};{M,T} (equals ch42 reducts)` and asserts "ch43 implicant set equals ch42 reducts={H,T};{M,T}". Boolean minimization and dependency enumeration, two definitions that share no code, produce the same two sets, and the flu table is small enough that both searches are exhaustive, which is what makes the agreement evidence rather than coincidence.

```
/* does sel cover every nonempty different-decision cell? */
static int covers(int sel) {
  for (int i = 0; i < 6; i++)
    for (int j = i + 1; j < 6; j++)
      if (FLU[i] != FLU[j]) {
        int d = diff_mask(i, j);
        if (d && !(sel & d))
          return 0;
      }
  return 1;
}
```

listing 319: coverage: one attribute from every informative cell, or fail (source: kdd/samples/src/Ch43/discern.c, lines 84-94)

```

  /* minimal form: brute force over 3 attributes, smallest covers first,
   * supersets of found implicants excluded (two-way: each implicant must
   * cover, no smaller cover may exist) */
  CHECK(!covers(1) && !covers(2) && !covers(4),
        "ch43 no single attribute covers (H,M,T each fail)");
  static const int ORDER[7] = {1, 2, 4, 3, 5, 6, 7};
  int primes[8], np = 0;
  for (int i = 0; i < 7; i++) {
    if (!covers(ORDER[i]))
      continue;
    int sub = 0;
    for (int r = 0; r < np; r++)
      if ((primes[r] & ORDER[i]) == primes[r] && primes[r] != ORDER[i])
        sub = 1;
    if (!sub)
      primes[np++] = ORDER[i];
  }
  off = 0;
  for (int r = 0; r < np; r++) {
    char ts[16];
    mask_str(primes[r], "&", ts);
    off += snprintf(buf + off, 24, "%s%s", r ? " | " : "", ts);
  }
  printf("f_min=%s\n", buf);
  CHECK(strcmp(buf, "H&T | M&T") == 0, "ch43 f_min=H&T | M&T");
  CHECK(covers(5) && covers(6) && np == 2 && primes[0] == 5 && primes[1] == 6,
        "ch43 both prime implicants {H,T},{M,T} cover (two-way)");

  /* cross-chapter agreement: the implicant set equals the ch42 reducts */
  off = 0;
  for (int r = 0; r < np; r++) {
    off += snprintf(buf + off, 24, "%s{", r ? ";" : "");
    char ts[16];
    mask_str(primes[r], ",", ts);
    off += snprintf(buf + off, 24, "%s}", ts);
  }
  printf("prime set as reducts=%s (equals ch42 reducts)\n", buf);
  CHECK(strcmp(buf, "{H,T};{M,T}") == 0,
        "ch43 implicant set equals ch42 reducts={H,T};{M,T}");
```

listing 320: brute-force implicants in size order, then the asserted agreement with ch42 (source: kdd/samples/src/Ch43/discern.c, lines 149-188)

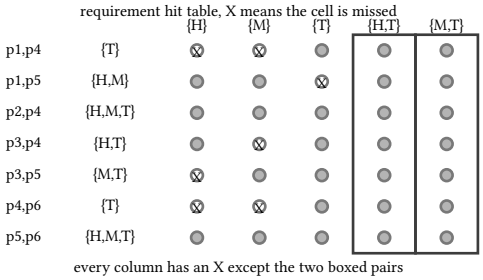


figure 175: the seven requirements against five candidate sets, only the two pairs cover all

43.4. rules, and the one rule the table refuses

Each reduct's partition turns into rules directly: a decision-constant class is a certain rule, its conjunction of attribute tests on the left, the decision on the right, the class size as support. A mixed class produces no certain rule at all, and the sample says so in the format `ambiguous (...)⇒? [1|1] no-certain-rule`, the vote count in brackets rather than a fabricated verdict.

The dry run: from {H,T}, `rules from {H,T}:` then five rows, `rule (H=n&T=high)⇒Flu=yes [1]`, `ambiguous (H=y&T=high)⇒? [1|1] no-certain-rule`, `rule (H=y&T=vhigh)⇒Flu=yes [1]`, `rule (H=n&T=normal)⇒Flu=no [1]`, `rule (H=n&T=vhigh)⇒Flu=yes [1]`, and the closer `certain_rules_cover=4 of 6`, asserted as "ch43 reduct 1 certain_rules_cover=4 of 6". From {M,T}, four rows only, because its partition merges the two vhigh patients into one class, `rule (M=y&T=vhigh)⇒Flu=yes [2]` carrying support 2, and the same closer, 4 of 6. The two reducts classify exactly the positive region of

book 14, ch 42 (knowledge discovery, rough sets: dependency and reducts) and leave exactly the twins, {p2,p5}, unclassified, both asserted. The contrast is the payoff of choosing a reduct: {M,T} buys a shorter rule set with one doubly-supported rule, {H,T} pays one extra rule for rules that never merge classes.

```
  /* rules from each reduct: certain rules with support, ambiguous cell */
  static const int REDS[2] = {5, 6};
  static const char *RULES[2][6] = {
      {"rule (H=n&T=high)⇒Flu=yes [1]",
       "ambiguous (H=y&T=high)⇒? [1|1] no-certain-rule",
       "rule (H=y&T=vhigh)⇒Flu=yes [1]", "rule (H=n&T=normal)⇒Flu=no [1]",
       "rule (H=n&T=vhigh)⇒Flu=yes [1]", NULL},
      {"rule (M=y&T=high)⇒Flu=yes [1]",
       "ambiguous (M=n&T=high)⇒? [1|1] no-certain-rule",
       "rule (M=y&T=vhigh)⇒Flu=yes [2]", "rule (M=y&T=normal)⇒Flu=no [1]",
       NULL, NULL}};
  for (int r = 0; r < 2; r++) {
    int cls[6][7], n = partition(REDS[r], cls);
    printf("rules from {%s}:\n", r == 0 ? "H,T" : "M,T");
    int covered = 0, k = 0;
    for (int c = 0; c < n; c++) {
      int yes = 0, no = 0;
      for (int i = 0; i < cls[c][6]; i++)
        FLU[cls[c][i]] ? yes++ : no++;
      char cond[48] = "";
      int coff = 0, first = 1;
      for (int b = 0; b < 3; b++)
        if (REDS[r] & (1 << b)) {
          coff += snprintf(cond + coff, 24, "%s%s=%s", first ? "" : "&", AN[b],
                           AV[b][attr(b, cls[c][0])]);
          first = 0;
        }
      char row[80];
      if (!no || !yes) {
        snprintf(row, sizeof row, "rule (%s)⇒Flu=%s [%d]", cond,
                 yes ? "yes" : "no", cls[c][6]);
        covered += cls[c][6];
      } else {
        int firstdec = FLU[cls[c][0]], a = 0, b2 = 0;
        for (int i = 0; i < cls[c][6]; i++)
          FLU[cls[c][i]] == firstdec ? a++ : b2++;
        snprintf(row, sizeof row, "ambiguous (%s)⇒? [%d|%d] no-certain-rule",
                 cond, a, b2);
      }
      printf("%s\n", row);
      char want[80];
      snprintf(want, sizeof want, "ch43 %s", RULES[r][k]);
      CHECK(strcmp(row, RULES[r][k]) == 0, want);
      k++;
    }
    printf("certain_rules_cover=%d of 6\n", covered);
    snprintf(nm, sizeof nm, "ch43 reduct %d certain_rules_cover=4 of 6", r + 1);
    CHECK(covered == 4, nm);
  }
```

listing 321: each class one rule, mixed classes named ambiguous, support in brackets (source: kdd/samples/src/Ch43/discern.c, lines 190-238)

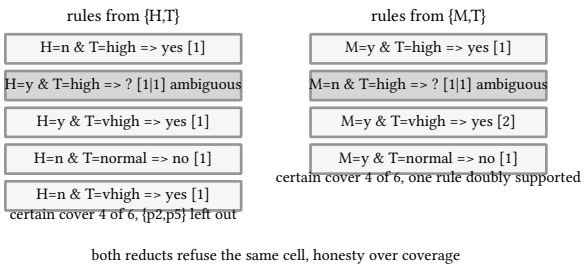


figure 176: two rule sets, one shared hole: the twins classify under neither

sources: the discernibility matrix and function are A. Skowron and C. Rauszer, The Discernibility Matrices and Functions in Information Systems, in R. Slowinski, editor, Intelligent Decision Support, Handbook of Applications and Advances of the Rough Sets Theory, Kluwer 1992, 331-362, applied to Pawlak's flu table as banked by kdd-contract-s8s9.md. All 31 pinned values witnessed by the same sheet and playground/kdd-matrix/gen_s9.py, run 2026-09-22, exit 0. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch43`, 31 checks in chapter 43 of the kdd suite.

44. self-organizing maps

Two samples carry the chapter: `som.c` builds the seeded initialization, the schedule table and the neighborhood kernel, 55 checks, and `somtrace.c` runs the five online epochs and pins every BMU sequence and weight grid, 45 checks. The chapter makes 4 moves: the lattice and its seeded random start, the truncated-quadratic kernel and its shrinking schedules, the epoch loop that moves a whole neighborhood toward each input, and the arithmetic discipline that makes all of it bit-reproducible. book 14, ch 31 (knowledge discovery, k-means) quantized with free-floating centroids, this chapter chains them to a 2x3 grid so nearby units answer nearby inputs, and the squared euclidean that picks the winner is the distance of book 14, ch 5 (knowledge discovery, distance metrics). The network learns without labels, the missing-labels contrast with book 14, ch 21 (knowledge discovery, neural networks), and the map it leaves behind is what book 14, ch 45 (knowledge discovery, som analysis and visualization) grades and book 14, ch 46 (knowledge discovery, som variants) grows.

44.1. a 2x3 lattice, a ring, and one seed

The fixture is small on purpose: 6 units on a 2-row by 3-column grid, unit index $u = 3r + c$, and 8 integer inputs forming a ring, x1 = (4,0) through x8 = (2,0) around the perimeter of the square 0..8. Every weight starts pseudo-random: a MINSTD generator, $s' = 16807s \bmod (2^{31} - 1)$, seeded at 42, drawn 12 times in unit-major order, each state divided by 2^{28} into (0, 8).

The dry run: the sample prints `lcg seed=42 states=705894,1126542223,...` one line of 12 states ending `...,229968128,1751246343`, each state its own check, the first two verifiable by hand, $42 \times 16807 = 705894$, then $705894 \cdot 16807 \bmod (2^{31} - 1) = 1126542223$ with the intermediates held in uint64 so no 32-bit overflow strikes. Then the init row, `init u0=(0.0026296600699424744,4.1966968141496181); u1=(5.8833882547914982,2.1064443252980709);` on through `u5=(0.85669803619384766,6.5239010117948055)`, all 12 coordinates asserted with ==, u2 for instance being states 3 and 4 over 2^{28}. The grid geometry is a check too: the 2x3 lattice realizes exactly the squared grid distances {0, 1, 2, 4, 5}, no 3, which is the kernel table's domain, asserted as "ch44 2x3 lattice d2 set is exactly {0,1,2,4,5}".

```
static void init_weights(double w[NU][2]) {
  unsigned long long st = 42ULL;
  for (int u = 0; u < NU; u++)
    for (int k = 0; k < 2; k++) {
      st = lcg(st);
      w[u][k] = (double)st / 268435456.0;
    }
}
```

listing 322: twelve seeded draws in unit-major order, each over 2^28 (source: kdd/samples/src/Ch44/som.c, lines 74-81)

```
  /* the 12 pinned LCG states, unit-major order */
  static const unsigned long long ST[12] = {
      705894ULL,     1126542223ULL, 1579310009ULL, 565444343ULL,
      807934826ULL,  421520601ULL,  2095673201ULL, 1100194760ULL,
      1139130650ULL, 552121545ULL,  229968128ULL,  1751246343ULL};
  static const double INIT[NU][2] = {{0.0026296600699424744, 4.196696814149618},
                                     {5.883388254791498, 2.106444325298071},
                                     {3.0097917690873146, 1.5702866055071354},
                                     {7.806991044431925, 4.098544865846634},
                                     {4.243592359125614, 2.056813035160303},
                                     {0.8566980361938477, 6.5239010117948055}};
  static const double HK[5][5] = {
      {1.0, 0.75, 0.5, 0.0, 0.0},
      {1.0, 0.5555555555555556, 0.11111111111111116, 0.0, 0.0},
      {1.0, 0.0, 0.0, 0.0, 0.0},
      {1.0, 0.0, 0.0, 0.0, 0.0},
      {1.0, 0.0, 0.0, 0.0, 0.0}};
  static const int D2S[5] = {0, 1, 2, 4, 5};
  char nm[80];

  unsigned long long st = 42ULL, sts[12];
  for (int i = 0; i < 12; i++) {
    st = lcg(st);
    sts[i] = st;
  }
  printf("lcg seed=42 states=");
  for (int i = 0; i < 12; i++)
    printf("%s%llu", i ? "," : "", sts[i]);
  printf("\n");
  for (int i = 0; i < 12; i++) {
    snprintf(nm, sizeof nm, "ch44 lcg state %d = %llu", i + 1, ST[i]);
    CHECK(sts[i] == ST[i], nm);
  }

  double w[NU][2];
  init_weights(w);
  printf("init ");
  for (int u = 0; u < NU; u++)
    printf("%su%d=(%.17g,%.17g)", u ? ";" : "", u, w[u][0], w[u][1]);
  printf("\n");
  for (int u = 0; u < NU; u++) {
    snprintf(nm, sizeof nm, "ch44 init u%d=(%.17g,%.17g)", u, INIT[u][0],
             INIT[u][1]);
    CHECK(w[u][0] == INIT[u][0] && w[u][1] == INIT[u][1], nm);
  }
```

listing 323: the pinned LCG states and init grid, every double asserted == (source: kdd/samples/src/Ch44/som.c, lines 90-134)

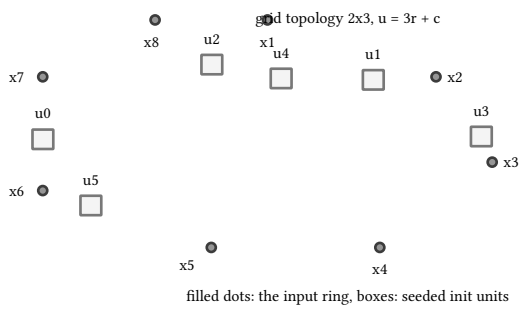


figure 177: the ring of inputs and the random start the lattice has to organize

44.2. the kernel, truncated so it stays honest

Each adaptation step moves every unit toward the current input, weighted by a neighborhood kernel of the unit's grid distance to the winner. The true SOM kernel is Gaussian, $h = \exp(-d^2/(2\sigma^2))$ in the books, and this fixture deliberately does not use it. The pinned kernel is the truncated quadratic $h(d^2) = \max(0, 1 - d^2/\rho^2)$, a compact-support stand-in with the same bell intent: 1 at the winner, falling with distance, 0 beyond the radius. Its whole arithmetic is one division and one subtraction, which is the point, no transcendental is evaluated anywhere in the SOM family, so no libm rounding differences can creep between implementations.

The dry run: five schedule rows and five kernel rows, `sched e1 rho2=4 eta=0.5` with `h e1: d2=0:1 d2=1:0.75 d2=2:0.5 d2=4:0 d2=5:0`, then rho2 = 2.25, 1.0, 0.5625, 0.25 against eta = 0.375, 0.25, 0.1875, 0.125, every schedule value and every kernel value its own == check. The rho2 column is ρ^2 for rho = 2, 1.5, 1, 0.75, 0.5 and the eta column halves from 1/2, all dyadic, all exact. The epoch-2 row carries the family's pet constant: `d2=1:0.55555555555555558`, that is $1 - 1/2.25$, and `d2=2:0.1111111111111116`, which is not the double nearest to 1/9, it is $1 - (2/2.25)$ with one rounding at the division and one at the subtraction. By epoch 3, rho2 = 1 kills every nonzero-d2 weight, the kernel becomes the bubble, only the winner itself adapts, and epochs 3 through 5 share the all-or-nothing row `d2=0:1 d2=1:0 d2=2:0 d2=4:0 d2=5:0`.

```c
/* h(d2) = max(0, 1 - d2/rho2): division then subtraction, each its own
 * correctly-rounded statement (t>0 selects, never a fused op) */
static double kern(int d2, double rho2) {
  double t = (double)d2 / rho2;
  t = 1.0 - t;
  return t > 0.0 ? t : 0.0;
}

/* grid distance squared: dr^2 + dc^2 over the 2x3 lattice */
static int gd2(int u, int v) {
  int dr = u / 3 - v / 3;
  int dc = u % 3 - v % 3;
  return dr * dr + dc * dc;
}
```

listing 324: the kernel: division, subtraction, clamp, each its own statement (source: kdd/samples/src/Ch44/som.c, lines 41-54)

```c
  for (int e = 0; e < 5; e++) {
    double hs[5];
    for (int j = 0; j < 5; j++)
      hs[j] = kern(D2S[j], RHO2[e]);
    printf("sched e%d rho2=%.17g eta=%.17g\n", e + 1, RHO2[e], ETA[e]);
    printf("h e%d: ", e + 1);
    for (int j = 0; j < 5; j++)
      printf("%sd2=%d:%.17g", j ? " " : "", D2S[j], hs[j]);
    printf("\n");
    snprintf(nm, sizeof nm, "ch44 sched e%d rho2=%g", e + 1, RHO2P[e]);
    CHECK(RHO2[e] == RHO2P[e], nm);
    snprintf(nm, sizeof nm, "ch44 sched e%d eta=%g", e + 1, ETAP[e]);
    CHECK(ETA[e] == ETAP[e], nm);
    for (int j = 0; j < 5; j++) {
      snprintf(nm, sizeof nm, "ch44 h e%d d2=%d:%.17g", e + 1, D2S[j],
               HK[e][j]);
      CHECK(hs[j] == HK[e][j], nm);
    }
  }
```

listing 325: five schedules, five kernel rows, every value asserted == (source: kdd/samples/src/Ch44/som.c, lines 145-163)

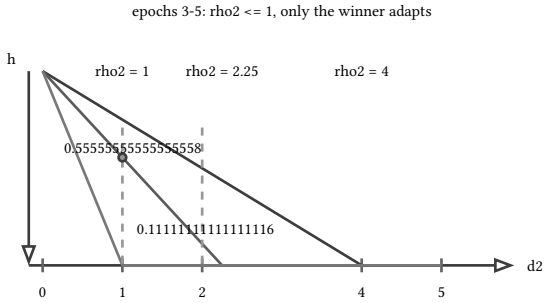


figure 178: the kernel over grid distance, five shrinking epochs, bubble by epoch 3

44.3. five epochs, one dead unit

Training is online with immediate updates: sweep x1 through x8, for each input find the best-matching unit, the BMU, by plain squared euclidean over both dimensions with strict < so ties keep the lowest index, then move every unit by $w_u \leftarrow w_u + \eta \cdot h(d^2(u, \text{bmu})) \cdot (x - w_u)$. Five epochs, one schedule row each. The grid organizes fast, and one unit never makes it.

The dry run: epoch 1 prints `epoch 1 bmu=2,1,3,3,0,5,2,2` and the full after-epoch grid starting `after epoch 1 u0=(4.0755163159265066,6.7447139665709983);...`, epoch 2 `bmu=2,1,3,3,0,0,5,2`, and epochs 3 through 5 the settled `bmu=2,4,4,3,0,0,5,2`, all 40 BMU digits and all 60 grid coordinates asserted with == against the pinned literals, the HARD two-way core of the chapter. The final row opens `final u0=(1.7736785518636009,6.7858215993753985)` and runs on through `u5=(0.78802150923085545,1.7396510860930761)`, with u1 frozen at `u1=(2.9240485376881993,3.1672306950233438)`. Unit u1 is the story: it wins x2 in epochs 1 and 2, loses it to u4 from epoch 3 on, and once the kernel is a bubble only winners move, so u1's row is frozen at its epoch-2 value forever, printed as `dead unit u1 frozen from epoch 2: true` and asserted twice, "ch44 u1 dead unit frozen from epoch 2" and "ch44 u1 never a BMU in epochs 3-5". A dead unit is a real SOM failure mode, not a fixture decoration, and book 14, ch 46 (knowledge discovery, som variants) exists partly to fix it by growing the grid where the error is.

```c
static int bmu(const double w[NU][2], double x0, double x1) {
  int best = -1;
  double bd = 0.0;
  for (int u = 0; u < NU; u++) {
    double a = x0 - w[u][0];
    double b = a * a;
    double c = x1 - w[u][1];
    double d = c * c;
    double s = b + d;
    if (best < 0 || s < bd) {
      best = u;
      bd = s;
    }
  }
  return best;
}
```

listing 326: BMU: squared euclidean, one op per statement, strict < keeps the lowest index (source: kdd/samples/src/Ch44/somtrace.c, lines 54-69)

```
  double grids[5][NU][2];
  for (int e = 0; e < 5; e++) {
    int seq[8];
    for (int i = 0; i < 8; i++) {
      int b = bmu(w, X[i][0], X[i][1]);
      seq[i] = b;
      for (int u = 0; u < NU; u++) {
        double h = kern(gd2(u, b), RHO2[e]);
        for (int k = 0; k < 2; k++) {
          double a = X[i][k] - w[u][k];
          double p1 = h * a;
          double p2 = ETA[e] * p1;
          w[u][k] = w[u][k] + p2;
        }
      }
    }
    printf("epoch %d bmu=", e + 1);
    for (int i = 0; i < 8; i++)
      printf("%s%d", i ? "," : "", seq[i]);
    printf("\n");
    int ok = 1;
    for (int i = 0; i < 8; i++)
      if (seq[i] != BMUS[e][i])
        ok = 0;
    snprintf(nm, sizeof nm, "ch44 epoch %d bmu=%d,%d,%d,%d,%d,%d,%d,%d", e + 1,
             BMUS[e][0], BMUS[e][1], BMUS[e][2], BMUS[e][3], BMUS[e][4],
             BMUS[e][5], BMUS[e][6], BMUS[e][7]);
    CHECK(ok, nm);
    for (int u = 0; u < NU; u++)
      for (int k = 0; k < 2; k++)
        grids[e][u][k] = w[u][k];
    printf("after epoch %d ", e + 1);
    for (int u = 0; u < NU; u++)
      printf("%su%d=(%.17g,%.17g)", u ? ";" : "", u, grids[e][u][0],
             grids[e][u][1]);
    printf("\n");
    for (int u = 0; u < NU; u++) {
      snprintf(nm, sizeof nm, "ch44 after epoch %d u%d=(%.17g,%.17g)", e + 1, u,
               GRIDS[e][u][0], GRIDS[e][u][1]);
      CHECK(grids[e][u][0] == GRIDS[e][u][0] &&
                grids[e][u][1] == GRIDS[e][u][1],
            nm);
    }
  }
```

listing 327: the epoch loop: sweep inputs, adapt the whole neighborhood, pin everything (source: kdd/samples/src/Ch44/somtrace.c, lines 147-190)

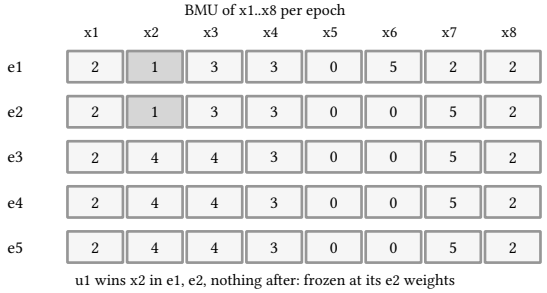


figure 179: BMU per input per epoch: u1 wins x2 twice, then dies

PITFALL: a map that cannot be rerun is a map you cannot trust

Every grid coordinate in this chapter is pinned to 17 digits and the samples assert ==, not closeness. That only works because the whole computation is transcendental-free with one floating operation per statement: each step is a single correctly-rounded IEEE operation, so any conforming implementation in any language that mirrors the op order produces bit-identical doubles. Swap in the Gaussian kernel and you inherit every libm's private exp, and the last ulp becomes platform-dependent, which is why the truncated quadratic was pinned by ruling. The discipline is the chapter's method, not its trivia, and book 14, ch 46 (knowledge discovery, som variants) leans on it to bit-compare batch, gas and online variants against each other.

44.4. the update, one operation at a time

The ruling deserves its own reading, because the sample's C is shaped by it. The files open with `#pragma STDC FP_CONTRACT OFF` and are never built with contraction or fast-math, and each arithmetic step is its own statement in a fixed order: the kernel computes `t = d2 / rho2`, then `t = 1.0 - t`, then clamps, and the weight update computes `a = x - w`, `p1 = h * a`, `p2 = eta * p1`, then `w = w + p2`. No expression writes `w + eta * h * (x - w)` in one line, because a compiler would be allowed to fuse the multiply-add and change the result in the last ulp.

The dry run: `somtrace.c` isolates the very first update of training, x1′s arrival at its BMU u2, applies the epoch-1 kernel and eta to u2 alone, and prints `first update u2=(3.5048958845436573,0.7851433027535677)`, asserted against both literals. The value is the midpoint of the seeded weight and the input, h = 1 and eta = 1/2 make the step exactly half the gap, and it is a cross-witness besides: chapter 46′s neural gas applies h = 1, eta = 0.5 to the same unit through an independent code path and must land on the same 17 digits, asserted there as "ch46 gas rank-0 x1 u2=(3.5048958845436573,0.7851433027535677) == ch44 online first update". One pinned arithmetic identity gluing two chapters, free of charge.

```
/* epoch-1 first update: x1's BMU is u2, applying the e1 kernel+eta to
 * u2 alone must land exactly on the pinned (and ch46-gas-shared) row */
{
  int b = bmu(w, X[0][0], X[0][1]);
  double h = kern(gd2(b, b), RHO2[0]);
  for (int k = 0; k < 2; k++) {
    double a = X[0][k] - w[b][k];
    double p1 = h * a;
    double p2 = ETA[0] * p1;
    w[b][k] = w[b][k] + p2;
  }
  printf("first update u%d=(%.17g,%.17g)\n", b, w[b][0], w[b][1]);
  CHECK(b == 2, "ch44 epoch1 first update BMU=u2");
  CHECK(w[2][0] == 3.5048958845436573 && w[2][1] == 0.7851433027535677,
        "ch44 epoch1 first update u2=(3.5048958845436573,0.78514330275356"
        "77)");
  init_weights(w); /* restart for the full trace */
}
```

listing 328: the isolated first update, pinned and cross-witnessed by ch46′s gas (source: kdd/samples/src/Ch44/somtrace.c, lines 128-145)

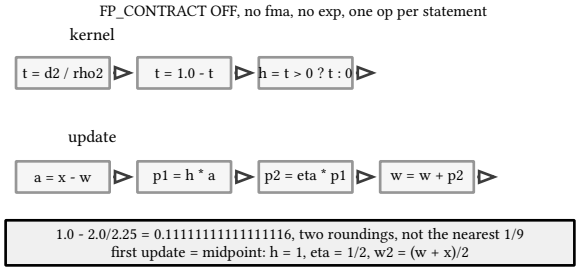


figure 180: two statement chains, each box one correctly-rounded operation

sources: the self-organizing map, the batch variant of chapter 46 and the supervised LVQ of chapter 46 are T. Kohonen, Self-Organizing Maps, Springer Series in Information Sciences 30, 3rd ed. 2001, with the truncated-quadratic kernel, the MINSTD seed 42, the schedule values and every pinned grid witnessed by kdd-contract-s8s9.md and playground/kdd-matrix/gen_s9.py, run 2026-09-22, exit 0. The generator is W. H. Press and S. A. Teukolsky's portable MINSTD of Park & Miller, Communications of the ACM 31(10) 1988, 1192-1201, the same family as the chapter 07 pin. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch44`, 55 + 45 checks in chapter 44 of the kdd suite.

45. som analysis and visualization

One sample carries the chapter: `sommap.c` retrains the chapter 44 map from the same seed, re-pins its final grid, then grades it, quantization error and SSE per input, topographic error over second-best units, the u-matrix and component planes, a k-means k=2 contrast with exact arithmetic, and the ring-topology census, 54 checks. The chapter makes 4 moves: error measures that treat the map as a quantizer, a topology measure that treats it as a manifold, structural views that expose fold and stretch, and the head-to-head against book 14, ch 31 (knowledge discovery, k-means) that prices what the grid buys. The chapter is the map-side counterpart of book 14, ch 34 (knowledge discovery, cluster validation), which graded clusterings by index, and every distance it sums is the squared euclidean of book 14, ch 5 (knowledge discovery, distance metrics) under the same one-op-per-statement discipline book 14, ch 44 (knowledge discovery, self-organizing maps) pinned.

45.1. quantization error and SSE

The retrained map lands bit-for-bit on the chapter 44 pins, six == checks of the form "ch45 final u0=(1.7736785518636009,6.7858215993753985) equals ch44 pin", the chapter-local divergence guard: if any metric below looks wrong, the fault is here or upstream, never ambiguity about which map was measured. Then each input is scored against its best unit. The quantization error is the mean euclidean distance to the winner, the sum of eight `sqrt(d2)` in input order divided by 8, and SSE_som is the sum of the eight squared distances in the same order.

The dry run: eight rows, `finalbmu x1=u2 d2=1.5359862538166922 d=1.2393491250719839` through `finalbmu x8=u2 d2=0.65739133701155228 d=0.81079672977359274`, each triple, unit, d2, d, its own check. The totals print `QE=1.4362362499549604` and `SSE_som=18.326103126534949`, both asserted with ==, D3. Reading the winners: u2 takes x1 and x8, the low-y end of the ring, u4 takes x2 and x3, u3 takes x4 alone, u0 takes x5 and x6, u5 takes x7 alone, and u1, the dead unit, takes nothing, its distance column is absent because it is never a BMU. The worst fit is x3 at d = 2.048, the ring's far corner stretching toward a unit that must also serve x2.

```
/* QE pass: per-input BMU d2, d=sqrt(d2), running total in input order */
double per2[8], perd[8], total = 0.0;
int perb[8];
for (int i = 0; i < 8; i++) {
  double d2;
  int b = bmu(w, X[i][0], X[i][1], &d2);
  double dd = sqrt(d2);
  per2[i] = d2;
  perd[i] = dd;
  perb[i] = b;
  total = total + dd;
}
for (int i = 0; i < 8; i++) {
  printf("finalbmu x%d=u%d d2=%.17g d=%.17g\n", i + 1, perb[i], per2[i],
         perd[i]);
  snprintf(nm, sizeof nm, "ch45 finalbmu x%d=u%d d2=%.17g d=%.17g", i + 1,
           FBU[i], FBD2[i], FBD[i]);
  CHECK(perb[i] == FBU[i] && per2[i] == FBD2[i] && perd[i] == FBD[i], nm);
}
double qe = total / 8.0;
printf("QE=%.17g\n", qe);
CHECK(qe == 1.4362362499549604, "ch45 QE=1.4362362499549604");
double sse = 0.0;
for (int i = 0; i < 8; i++)
  sse = sse + per2[i];
printf("SSE_som=%.17g\n", sse);
CHECK(sse == 18.32610312653495, "ch45 SSE_som=18.32610312653495");
```

listing 329: the QE pass: BMU d2, its sqrt, a running total in input order (source: kdd/samples/src/Ch45/sommap.c, lines 147-173)

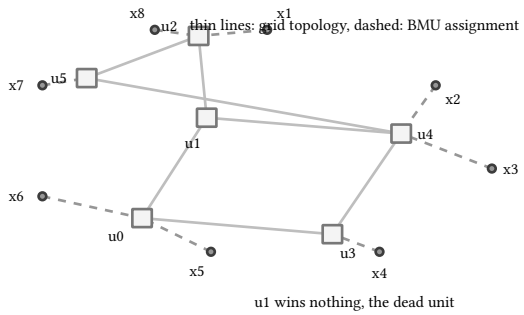


figure 181: the final map in data space, dashed lines tie each input to its winner

45.2. topographic error

A quantizer can be perfect at distances and still fold the manifold, so the second measure is topological. For each input, take the best and the second-best unit, if the input really sits between two neighboring lattice positions the runner-up should be the winner's grid neighbor. The topographic error is the fraction of inputs whose two best units are not von Neumann adjacent, $|d_r| + |d_c| = 1$ exactly, diagonal does not count.

The dry run: eight rows, `te x1 best=u2 second=u1 adjacent=true` down to `te x8 best=u2 second=u5 adjacent=true`, each row a check, and one failure among them, `te x7 best=u5 second=u1 adjacent=false`: u5 is row 1 column 2, u1 is row 0 column 1, one row apart and one column apart, diagonal, not adjacent. The total prints `TE=1/8`, asserted as "ch45 TE=1/8" and cross-multiplied as a fraction pair, D0. Seven of eight inputs have their two best units side by side on the lattice, the map is close to topology-preserving, and the single fold sits exactly where the worst quantization error sat, the far corner around x3, x4 and x7 where the ring bends hardest.

```
   * Neumann adjacent to the best */
  int te = 0;
  for (int i = 0; i < 8; i++) {
    int u1 = -1, u2 = -1;
    double s1 = 0.0, s2 = 0.0;
    for (int u = 0; u < NU; u++) {
      double a = X[i][0] - w[u][0];
      double b = a * a;
      double c = X[i][1] - w[u][1];
      double d = c * c;
      double s = b + d;
      if (u1 < 0 || s < s1) {
        u2 = u1;
        s2 = s1;
        u1 = u;
        s1 = s;
      } else if (u2 < 0 || s < s2) {
        u2 = u;
        s2 = s;
      }
    }
    int adj = (u1 / 3 - u2 / 3) * (u1 / 3 - u2 / 3) +
              (u1 % 3 - u2 % 3) * (u1 % 3 - u2 % 3);
    adj = adj == 1;
    if (!adj)
      te++;
    printf("te x%d best=u%d second=u%d adjacent=%s\n", i + 1, u1, u2,
           adj ? "true" : "false");
    snprintf(nm, sizeof nm, "ch45 te x%d best=u%d second=u%d adjacent=%s",
             i + 1, TEB[i][0], TEB[i][1], TEA[i] ? "true" : "false");
    CHECK(u1 == TEB[i][0] && u2 == TEB[i][1] && adj == TEA[i], nm);
  }
  printf("TE=%d/8\n", te);
  CHECK(te == 1 && frac_eq(te, 8, 1, 8), "ch45 TE=1/8");
```

listing 330: second-best by strict scan, adjacency by von Neumann, one fold found (source: kdd/samples/src/Ch45/sommap.c, lines 176-209)

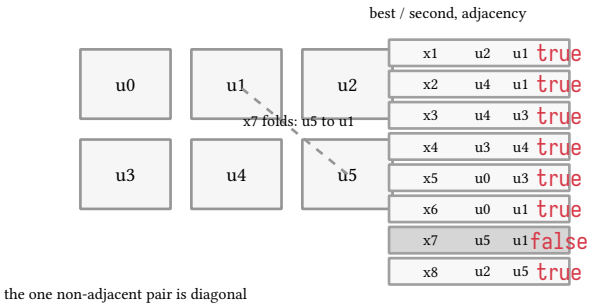


figure 182: the 2x3 lattice with each input's two best units, the diagonal pair highlighted

45.3. the u-matrix and the component planes

Structure the scalar scores cannot show lives in two visualizations. The u-matrix paints each lattice edge with the distance between the two units it joins, 4 horizontal and 3 vertical edges here, so a dark long edge is a tear in the map, two neighboring grid positions whose weights drifted apart. The component planes show one weight dimension across the grid, where the map's low and high ends of each input feature landed.

The dry run: seven edge rows, `umatrix u0-u1 d2=14.417551237345508 d=3.7970450665412847` and its siblings, each d2 and d asserted ==, with the longest being `umatrix u4-u5 d2=35.333217300545932 d=5.944175073174236`, the tear, u4 and u5 are grid neighbors but their weights sit at opposite ends of the data, u4 serving the x2, x3 corner and u5 serving x7 alone. The two plane rows print `plane dim0 min=u5:0.78802150923085545 max=u4:6.3859543002790851` and `plane dim1 min=u2:0.22009823073887433 max=u3:7.3650730481977105`, asserted as "ch45 plane dim0 min=u5 max=u4" and "ch45 plane dim1 min=u2 max=u3": the first input dimension runs from u5 to u4, the same torn edge, and the second from u2, the bottom of the ring, to u3, the top, the full vertical span of the data compressed into one grid column step.

```
/* u-matrix: 4 horizontal + 3 vertical inter-node distances */
for (int p = 0; p < 7; p++) {
  int u = UMV[p][0], v = UMV[p][1];
  double a = w[u][0] - w[v][0];
  double b = a * a;
  double c = w[u][1] - w[v][1];
  double d = c * c;
  double s = b + d;
  double dd = sqrt(s);
  printf("umatrix u%d-u%d d2=%.17g d=%.17g\n", u, v, s, dd);
  snprintf(nm, sizeof nm, "ch45 umatrix u%d-u%d d2=%.17g d=%.17g", u, v,
           UMD2[p], UMD[p]);
  CHECK(s == UMD2[p] && dd == UMD[p], nm);
}
```

listing 331: seven inter-node distances, one per lattice edge, each asserted (source: kdd/samples/src/Ch45/sommap.c, lines 211-224)

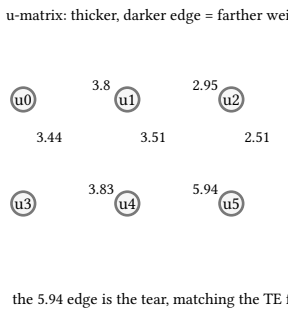


figure 183: u-matrix on the lattice: edge length encodes distance, the u4-u5 tear

NOTE: two adjacencies, two purposes

This chapter uses adjacency twice with different rules. The topographic error asks whether the two best units are strictly von Neumann adjacent, $|d_r| + |d_c| = 1$, equality excluded because a unit is trivially its own neighbor. The ring census below counts a pair as fine when the two BMUs are equal or adjacent, $|d_r| + |d_c| \leq 1$, because consecutive ring inputs landing on the same unit is perfect behavior, not a fold. The two signs, strict for the error, inclusive for the census, are both pinned in checks, and conflating them would change both counts.

45.4. k-means contrast, and the ring census

The k-means comparison plants k = 2 at x1 and x5, runs Lloyd to its fixed point, and scores it exactly. The point is not that one algorithm wins a beauty contest, it is that the same fixture prices the grid's extra units: six prototypes against two, and topology for free.

The dry run: `kmeans iter1 A={1,2,7,8} B={3,4,5,6} c1=(3.25,1) c2=(4.25,6.75) stable=false`, then `kmeans iter2` with the identical partition and centroids and `stable=true`, each

iteration's membership, centroid and stability a check, and "ch45 kmeans fixed point reached at iter 2". The partition is horizontal, the low ring against the high ring, with exact dyadic centroids $(13/4, 1)$ and $(17/4, 27/4)$. The energy prints `SSE_kmeans2=1188/16=74.25`, asserted three ways, the numerator 1188 over 16, the reduced pair 297/4 cross-multiplied, and the dyadic value 74.25 with ==. The hand witness behind it: $492/16$ from cluster A plus $696/16$ from cluster B. Against SSE_som = 18.326103126534949 the check "ch45 SOM SSE 18.326… beats k-means 74.25" fires, about a factor of 4, which is mostly the 6-to-2 prototype count and partly the SOM's freedom to place units off centroid. The census closes the chapter: consecutive ring inputs, including the wrap pair x8 to x1, map to equal-or-adjacent BMUs in 6 of 8 cases, the failures `topo x1→x2 u2→u4 ok=false` and `topo x6→x7 u0→u5 ok=false`, asserted, and `topo_adjacent_pairs=6/8`, asserted as the reduced 3/4. The two failures are the two places the ring changes grid rows, exactly where the tear and the topographic fold already pointed.

```
/* k-means k=2 planted at x1 and x5, Lloyd to the fixed point */
double c1[2] = {4.0, 0.0}, c2[2] = {3.0, 8.0};
int a1[8], a2[8], n1 = 0, n2 = 0, prev1[8], prev2[8], pn1 = 0, pn2 = 0;
int have = 0, it = 0;
for (;;) {
  it++;
  n1 = n2 = 0;
  for (int i = 0; i < 8; i++) {
    double a = X[i][0] - c1[0];
    double b = a * a;
    double c = X[i][1] - c1[1];
    double d = c * c;
    double d1 = b + d;
    a = X[i][0] - c2[0];
    b = a * a;
    c = X[i][1] - c2[1];
    d = c * c;
    double dq = b + d;
    if (d1 ≤ dq)
      a1[n1++] = i + 1;
    else
      a2[n2++] = i + 1;
  }
  int s1x = 0, s1y = 0, s2x = 0, s2y = 0;
  for (int i = 0; i < n1; i++) {
    s1x += (int)X[a1[i] - 1][0];
    s1y += (int)X[a1[i] - 1][1];
  }
  for (int i = 0; i < n2; i++) {
    s2x += (int)X[a2[i] - 1][0];
    s2y += (int)X[a2[i] - 1][1];
  }
  double nc1[2], nc2[2];
  nc1[0] = (double)s1x / n1;
  nc1[1] = (double)s1y / n1;
  nc2[0] = (double)s2x / n2;
  nc2[1] = (double)s2y / n2;
  int same = have && pn1 == n1 && pn2 == n2;
  if (same)
    for (int i = 0; i < n1; i++)
      if (a1[i] ≠ prev1[i])
        same = 0;
  if (same)
    for (int i = 0; i < n2; i++)
      if (a2[i] ≠ prev2[i])
```

```
        same = 0;
    printf("kmeans iter%d A={%d,%d,%d,%d} B={%d,%d,%d,%d} c1=(%.17g,%.17g) "
           "c2=(%.17g,%.17g) stable=%s\n",
           it, a1[0], a1[1], a1[2], a1[3], a2[0], a2[1], a2[2], a2[3], nc1[0],
           nc1[1], nc2[0], nc2[1], same ? "true" : "false");
    snprintf(nm, sizeof nm, "ch45 kmeans iter%d A={1,2,7,8} B={3,4,5,6}", it);
    CHECK(n1 == 4 && a1[0] == 1 && a1[1] == 2 && a1[2] == 7 && a1[3] == 8 &&
              n2 == 4 && a2[0] == 3 && a2[1] == 4 && a2[2] == 5 && a2[3] == 6,
          nm);
    snprintf(nm, sizeof nm, "ch45 kmeans iter%d c1=(3.25,1) c2=(4.25,6.75)",
             it);
    CHECK(nc1[0] == 3.25 && nc1[1] == 1.0 && nc2[0] == 4.25 && nc2[1] == 6.75,
          nm);
    snprintf(nm, sizeof nm, "ch45 kmeans iter%d stable=%s", it,
             it == 1 ? "false" : "true");
    CHECK(same == (it == 2), nm);
    for (int i = 0; i < n1; i++)
      prev1[i] = a1[i];
    for (int i = 0; i < n2; i++)
      prev2[i] = a2[i];
    pn1 = n1;
    pn2 = n2;
    have = 1;
    c1[0] = nc1[0];
    c1[1] = nc1[1];
    c2[0] = nc2[0];
    c2[1] = nc2[1];
    if (same)
      break;
  }
  CHECK(it == 2, "ch45 kmeans fixed point reached at iter 2");
```

listing 332: Lloyd planted at x1 and x5, two iterations to the fixed point, all dyadic (source: kdd/samples/src/Ch45/sommap.c, lines 254-329)

```
  /* exact SSE of the k-means fixed point in /16 integer arithmetic:
   * A centroid (13/4,1), B centroid (17/4,27/4) */
  long long num = 0;
  for (int i = 0; i < 8; i++) {
    int inA = 0;
    for (int j = 0; j < n1; j++)
      if (a1[j] == i + 1)
        inA = 1;
    long long ex =
        inA ? 4 * (long long)X[i][0] - 13 : 4 * (long long)X[i][0] - 17;
    long long ey =
        inA ? 4 * (long long)X[i][1] - 4 : 4 * (long long)X[i][1] - 27;
    num += ex * ex + ey * ey;
  }
  double ssek = (double)num / 16.0;
  printf("SSE_kmeans2=%lld/16=%.17g\n", num, ssek);
  CHECK(num == 1188, "ch45 SSE_kmeans2 numerator 1188/16");
  CHECK(frac_eq(num, 16, 297, 4), "ch45 SSE_kmeans2=297/4 (D0 pair)");
  CHECK(ssek == 74.25, "ch45 SSE_kmeans2=74.25 exact dyadic");
  CHECK(sse < ssek, "ch45 SOM SSE 18.326... beats k-means 74.25");
```

listing 333: the exact k-means SSE in sixteenths, reduced to 297/4, compared against SSE_som (source: kdd/samples/src/Ch45/sommap.c, lines 331-350)

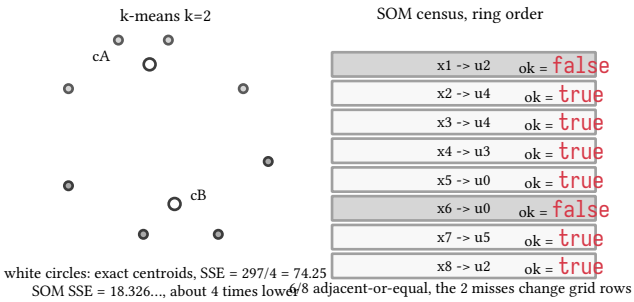


figure 184: k-means splits the ring in two, the SOM walks it around the lattice

sources: quantization error, topographic error, the u-matrix and component planes are the standard quality measures of T. Kohonen, Self-Organizing Maps, Springer Series in Information Sciences 30, 3rd ed. 2001, chapters 3 and 4, with every pinned double witnessed by kdd-contract-s8s9.md and playground/kdd-matrix/gen_s9.py, run 2026-09-22, exit 0. The k-means contrast follows the Lloyd fixed-point machinery of book 14, ch 31 (knowledge discovery, k-means) with exact dyadic arithmetic. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch45`, 54 checks in chapter 45 of the kdd suite.

46. som variants

Four samples carry the chapter: `batch.c` reruns the ring training without a learning rate, 72 checks, `grow.c` inserts a column where the map hurts, 50, `gas.c` replaces the lattice with ranks, 63, and `lvq.c` supervises two prototypes, 22. The chapter makes 4 moves: the batch reformulation whose weights are kernel-weighted averages, the growing grid that repairs a dead or torn map by adding units, neural gas with its topology-free rank kernel, and LVQ1, the supervised sibling that pushes prototypes toward their own class and away from the other. All four reuse book 14, ch 44 (knowledge discovery, self-organizing maps)'s seed, inputs and grid so the variants diverge for visible reasons, and all are graded by the QE ruler of book 14, ch 45 (knowledge discovery, som analysis and visualization). The supervised half is the nearest-prototype decision rule of book 14, ch 20 (knowledge discovery, nearest neighbor) with learned reference points.

46.1. batch: no learning rate, frozen winners

Batch SOM keeps the seed, the grid, the inputs and the rho2 schedule and deletes eta entirely. Each epoch computes every BMU against the frozen epoch-start weights, then sets each unit to the kernel-weighted average of all inputs, $w_u = \sum_i h(d^2(u,b_i))x_i / \sum_i h(d^2(u,b_i))$. A unit nobody's neighborhood reaches has denominator 0 and keeps its weights, the rule, and it fires exactly once in this run.

The dry run: the batch epoch-1 BMU sequence prints `batch epoch1 bmu=2,1,3,3,5,5,0,2`, and the per-unit rows show the averaging, `batch epoch1 u5 den=4 num=(11,15) → (2.75,3.75)` among them, hand-derived as $0.75 + 0.5 + 0 + 0 + 1 + 1 + 0 + 0.75 = 4$ over the winners u2,u1,u3,u3,u5,u5,u0,u2 with numerator $(11, 15)$. The rule fires at epoch 3, `batch epoch3 u1 den=0 keeps weights`, the dead unit of book 14, ch 44 (knowledge discovery, self-organizing maps) now provably unreachable rather than merely unpicked. And the map goes exact: `batch after epoch 3 u0=(7.5,3.5);u1=(3.9166666666666665,2.9583333333333335); u2=(3,0);u3=(6,8);u4=(3,8);u5=(0,4)`, because in the bubble epochs the kernel is 1 only at the winner, so every reached unit is the plain centroid of its inputs, u2 the mean of x1 and x8, u5 the mean of x6 and x7, and epochs 4 and 5 reprint the same grid, pinned. The contrast row is the chapter's flagged correction: `batch epoch1 bmu=2,1,3,3,5,5,0,2` against online's `2,1,3,3,0,5,2,2`, and the sample prints `differ at input 5 (batch 5 vs online 0)` and `differ at input 7 (batch 0 vs online 2)` and nothing else, asserted as "ch46 batch vs online differ_at=5,7 (sheet's 5,6,7 annotation contradicts its own sequences)". The sheet's annotation was wrong, inputs 6 and 8 agree, and the computed set is law. The census closes with `delta_max=u0 dim0 5.7263214481363995` and `QE_batch=1.1452847075210475`, the best QE in the family.

```
for (int e = 0; e < 5; e++) {
  double frozen[NU][2];
  for (int u = 0; u < NU; u++)
    for (int k = 0; k < 2; k++)
      frozen[u][k] = wb[u][k];
  int bmus[8];
  for (int i = 0; i < 8; i++)
    bmus[i] = bmu(frozen, X[i][0], X[i][1], 0);
  double num[NU][2] = {{0}};
  double den[NU] = {0};
  for (int i = 0; i < 8; i++) {
    int b = bmus[i];
    for (int u = 0; u < NU; u++) {
      double h = kern(gd2(u, b), RHO2[e]);
      for (int k = 0; k < 2; k++) {
        double t = h * X[i][k];
```

```
          num[u][k] = num[u][k] + t;
        }
        den[u] = den[u] + h;
      }
    }
    for (int u = 0; u < NU; u++)
      if (den[u] > 0.0)
        for (int k = 0; k < 2; k++)
          wb[u][k] = num[u][k] / den[u];

    if (e == 0) {
      printf("batch epoch1 bmu=");
      for (int i = 0; i < 8; i++)
        printf("%s%d", i ? "," : "", bmus[i]);
      printf("\n");
      CHECK(bmus[0] == 2 && bmus[1] == 1 && bmus[2] == 3 && bmus[3] == 3 &&
                bmus[4] == 5 && bmus[5] == 5 && bmus[6] == 0 && bmus[7] == 2,
            "ch46 batch epoch1 bmu=2,1,3,3,5,5,0,2");
      for (int i = 0; i < 8; i++)
        bbmus[i] = bmus[i];
      for (int u = 0; u < NU; u++) {
        printf("batch epoch1 u%d den=%.17g num=(%.17g,%.17g) -> (%.17g,%.17g)"
               "\n",
               u, den[u], num[u][0], num[u][1], wb[u][0], wb[u][1]);
        snprintf(nm, sizeof nm, "ch46 batch epoch1 u%d den=%.17g", u, BDEN[u]);
        CHECK(den[u] == BDEN[u], nm);
        snprintf(nm, sizeof nm, "ch46 batch epoch1 u%d num=(%.17g,%.17g)", u,
                 BNUM[u][0], BNUM[u][1]);
        CHECK(num[u][0] == BNUM[u][0] && num[u][1] == BNUM[u][1], nm);
        snprintf(nm, sizeof nm, "ch46 batch epoch1 u%d -> (%.17g,%.17g)", u,
                 BGRIDS[0][u][0], BGRIDS[0][u][1]);
        CHECK(wb[u][0] == BGRIDS[0][u][0] && wb[u][1] == BGRIDS[0][u][1], nm);
      }
    }
    if (e == 2)
      for (int u = 0; u < NU; u++)
        if (den[u] == 0.0) {
          printf("batch epoch3 u%d den=0 keeps weights\n", u);
          CHECK(u == 1, "ch46 batch epoch3 u1 den=0 keeps weights");
        }
    printf("batch after epoch %d ", e + 1);
    for (int u = 0; u < NU; u++)
      printf("%su%d=(%.17g,%.17g)", u ? ";" : "", u, wb[u][0], wb[u][1]);
    printf("\n");
    for (int u = 0; u < NU; u++) {
      snprintf(nm, sizeof nm, "ch46 batch after epoch %d u%d", e + 1, u);
      CHECK(wb[u][0] == BGRIDS[e][u][0] && wb[u][1] == BGRIDS[e][u][1], nm);
    }
  }
```

listing 334: frozen-start BMUs, kernel-weighted sums, den=0 keeps weights (source: kdd/samples/src/Ch46/batch.c, lines 158-222)

```
  printf("batch_vs_online_epoch1_bmu batch=%d,%d,%d,%d,%d,%d,%d,%d "
         "online=%d,%d,%d,%d,%d,%d,%d,%d\n",
         bbmus[0], bbmus[1], bbmus[2], bbmus[3], bbmus[4], bbmus[5], bbmus[6],
         bbmus[7], obmus[0], obmus[1], obmus[2], obmus[3], obmus[4], obmus[5],
         obmus[6], obmus[7]);
  CHECK(obmus[0] == 2 && obmus[1] == 1 && obmus[2] == 3 && obmus[3] == 3 &&
            obmus[4] == 0 && obmus[5] == 5 && obmus[6] == 2 && obmus[7] == 2,
        "ch46 online epoch1 bmu=2,1,3,3,0,5,2,2 (contrast)");
  int ndiff = 0, at57 = 1;
  for (int i = 0; i < 8; i++)
    if (obmus[i] != bbmus[i]) {
      ndiff++;
      printf("differ at input %d (batch %d vs online %d)\n", i + 1, bbmus[i],
             obmus[i]);
      if (i != 4 && i != 6)
        at57 = 0;
    }
  CHECK(ndiff == 2 && at57,
        "ch46 batch vs online differ_at=5,7 (sheet's 5,6,7 annotation "
        "contradicts its own sequences)");
```

listing 335: the batch-vs-online contrast, differ set computed from both sequences (source: kdd/samples/src/Ch46/batch.c, lines 250-269)

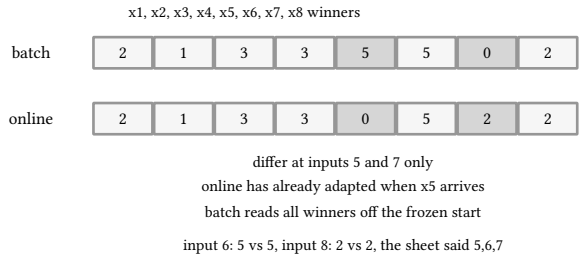


figure 185: the two epoch-1 sequences, winners differ exactly where online has already moved

46.2. growing grid: add units where it hurts

A fixed grid either wastes units or starves regions. Fritzke's growing grid accumulates each unit's quantization error, finds the worst unit, and inserts a new row or column between it and its most dissimilar grid neighbor. The sample runs one insertion on the chapter 44 map: per-unit errors `gerr u0=6.7415446740885399 n=2` through `gerr u4=7.5961914342770864 n=2`, with `gerr u1=0 n=0`, the dead unit contributing nothing, then `grow worst=u4 neighbors=1,3,5 dissimilar=u5 dist2=35.333217300545932 horiz=true`, the same torn edge the u-matrix of book 14, ch 45 (knowledge discovery, som analysis and visualization) painted dark.

The dry run: the insertion goes between u4 and u5, same row, so a column slots between columns 1 and 2 and the grid becomes 2x4, each new unit the per-row midpoint of its flanking units, op order $t = a + b$, $w = 0.5t$, pinned as `grown grid 2x4` with new units u2 and u6 at `u2=(2.8521999042434572,1.6936644628811091)` and `u6=(3.5869879047549702,2.7391967037930525)`. The census then delivers the punched line: `grown bmu u0={5,6}`, `grown bmu u3={1,8}`, `grown bmu u4={4}`, `grown bmu u5={2,3}`, `grown bmu u7={7}`, with u1, u2 and u6 winning nothing, and `QE_grown_after_insertion=1.4362362499549604`, identical to before, asserted, because new units placed on segment midpoints win nothing yet. Three relief epochs of bubble training, eta 1/8, rho2 1/4, then pull the winners inward, each epoch's full 2x4 grid pinned, ending at `QE_grown_relief=1.2762259134584661`. Growth bought a real error reduction, 1.436 to 1.276, at the cost of one column and three cheap epochs.

```
  /* worst unit (max error, ties keep lowest index) and its von Neumann
   * neighbors; most dissimilar neighbor by weight distance */
  int q = 0;
  for (int u = 1; u < NU; u++)
    if (err[u] > err[q])
      q = u;
  int nbrs[4], nn = 0;
  for (int v = 0; v < NU; v++) {
    if (v == q)
      continue;
    int dr = v / 3 - q / 3;
    int dc = v % 3 - q % 3;
    if (dr < 0)
      dr = -dr;
    if (dc < 0)
      dc = -dc;
    if (dr + dc == 1)
      nbrs[nn++] = v;
  }
  printf("grow worst=u%d neighbors=", q);
  for (int i = 0; i < nn; i++)
    printf("%s%d", i ? "," : "", nbrs[i]);
  int nb = nbrs[0];
  double wdn = 0.0;
  for (int i = 0; i < nn; i++) {
    double a = wf[q][0] - wf[nbrs[i]][0];
    double b = a * a;
    double c = wf[q][1] - wf[nbrs[i]][1];
    double d = c * c;
    double s = b + d;
    if (s > wdn) {
      wdn = s;
      nb = nbrs[i];
    }
  }
  int horiz = q / 3 == nb / 3;
  printf(" dissimilar=u%d dist2=%.17g horiz=%s\n", nb, wdn,
         horiz ? "true" : "false");
  CHECK(q == 4, "ch46 grow worst=u4");
  CHECK(nn == 3 && nbrs[0] == 1 && nbrs[1] == 3 && nbrs[2] == 5,
        "ch46 grow neighbors=1,3,5");
  CHECK(nb == 5 && wdn == 35.33321730054593,
        "ch46 grow dissimilar=u5 dist2=35.33321730054593");
  CHECK(horiz, "ch46 grow horiz=true (same row)");
```

listing 336: worst unit by error, most dissimilar von Neumann neighbor, distance pinned (source: kdd/samples/src/Ch46/grow.c, lines 183-226)

```
/* column insertion between q and nb: 2x3 -> 2x4, new column = per-row
 * average of the flanking units, t = a + b; w = 0.5 * t */
int colq = q % 3, colb = nb % 3, newcol = colq < colb ? colq : colb;
newcol = newcol + 1;
double grown[NG][2];
for (int r = 0; r < 2; r++)
  for (int c = 0; c < 4; c++) {
    int src;
    if (c < newcol)
      src = r * 3 + c;
    else if (c == newcol)
      src = -1; /* filled below from the two flanking units */
    else
      src = r * 3 + (c - 1);
    if (src >= 0) {
      grown[r * 4 + c][0] = wf[src][0];
      grown[r * 4 + c][1] = wf[src][1];
    } else {
      double t0 = wf[r * 3 + colq][0] + wf[r * 3 + colb][0];
      double t1 = wf[r * 3 + colq][1] + wf[r * 3 + colb][1];
      grown[r * 4 + c][0] = 0.5 * t0;
      grown[r * 4 + c][1] = 0.5 * t1;
    }
  }
printf("grown grid 2x4 ");
for (int u = 0; u < NG; u++)
  printf("%su%d=(%.17g,%.17g)", u ? ";" : "", u, grown[u][0], grown[u][1]);
printf("\n");
for (int u = 0; u < NG; u++) {
  snprintf(nm, sizeof nm, "ch46 grown u%d=(%.17g,%.17g)", u, GROWN[u][0],
           GROWN[u][1]);
  CHECK(grown[u][0] == GROWN[u][0] && grown[u][1] == GROWN[u][1], nm);
}
```

listing 337: column insertion, new units as per-row midpoints, t = a + b then 0.5 t (source: kdd/samples/src/Ch46/grow.c, lines 228-260)

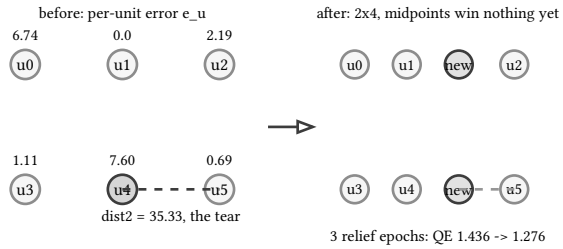


figure 186: 2x3 to 2x4: the new column between the worst unit and its farthest neighbor

46.3. neural gas: rank instead of lattice

Neural gas drops the grid entirely. For each input, all units are ranked by distance, and the update strength is a function of rank alone, $h = 2^{-\text{rank}}$ here, every unit moves toward the input, the closest most, the farthest barely. Topology is neither preserved nor claimed, which is the point: with no lattice to respect, gas spreads units freely through the data, and the cost shows up as a topology error if you ever ask for one.

The dry run: for x1 the rank order prints `gas rank=2,4,1,3,0,5`, the hand distance table off the seeded init reading u2 3.446, u4 4.290, u1 7.984, u3 31.29, u0 33.59, u5 52.44, and the kernel rows print `gas h_rank0=u2 h=1` down to `gas h_rank5=u5 h=0.03125`, each half the one before, all dyadic. One epoch over x1..x8 at constant eta = 0.5 follows, all eight after-input grids pinned, for example `gas after x3` carrying `u5=(1.2197515081425081,6.310594608995217)`

and gas after x4 carrying u3=(6.8566976445290493,6.0954473396996036). Three of those 48 step-grid pairs were transcribed wrong in the contract sheet and corrected by the generator rerun, the binding flag recorded in the sample header: the pinned literals read u5 x = 1.219751508142508 after x3, u3 y = 6.095447339699604 after x4 and u1 y = 3.62934196692182 after x7, and every quote here carries the generator values. The rank-0 update of x1 lands u2 on (3.5048958845436573,0.7851433027535677), bit-identical to chapter 44's online first update, same h = 1, same eta, independent code path, asserted as the free cross-witness. The epoch ends at QE_gas_epoch=1.772083269243151, the worst QE in the family: one epoch of gas from a cold start against five epochs of scheduled SOM.

```
  for (int i = 0; i < 8; i++) {
    int order[NU];
    rank_order(wg, X[i][0], X[i][1], order);
    if (i == 0) {
      printf("gas rank=");
      for (int r = 0; r < NU; r++)
        printf("%s%d", r ? "," : "", order[r]);
      printf("\n");
      CHECK(order[0] == 2 && order[1] == 4 && order[2] == 1 && order[3] == 3 &&
                order[4] == 0 && order[5] == 5,
            "ch46 gas rank=2,4,1,3,0,5 (x1)");
      for (int r = 0; r < NU; r++)
        printf("gas h_rank%d=u%d h=%.17g\n", r, order[r], HRANK[r]);
      CHECK(HRANK[0] == 1.0, "ch46 gas h_rank0=u2 h=1.0");
      for (int r = 1; r < NU; r++) {
        snprintf(nm, sizeof nm, "ch46 gas h_rank%d=u%d h=%.17g (halving)", r,
                 order[r], HRANK[r]);
        CHECK(HRANK[r] == 0.5 * HRANK[r - 1], nm);
      }
    }
    for (int r = 0; r < NU; r++) {
      int u = order[r];
      double h = HRANK[r];
      for (int k = 0; k < 2; k++) {
        double d = X[i][k] - wg[u][k];
        double p1 = h * d;
        double p2 = 0.5 * p1;
        wg[u][k] = wg[u][k] + p2;
      }
    }
    printf("gas after x%d ", i + 1);
    for (int u = 0; u < NU; u++)
      printf("%su%d=(%.17g,%.17g)", u ? ";" : "", u, wg[u][0], wg[u][1]);
    printf("\n");
    for (int u = 0; u < NU; u++) {
      snprintf(nm, sizeof nm, "ch46 gas after x%d u%d", i + 1, u);
      CHECK(wg[u][0] == GASGRIDS[i][u][0] && wg[u][1] == GASGRIDS[i][u][1], nm);
    }
  }

  /* free cross-witness: after x1 the rank-0 unit u2 equals ch44's online
   * epoch-1 first update bit-for-bit (same arithmetic, independent path) */
  CHECK(GASGRIDS[0][2][0] == 3.5048958845436573 &&
            GASGRIDS[0][2][1] == 0.7851433027535677,
        "ch46 gas rank-0 x1 u2=(3.5048958845436573,0.7851433027535677) "
        "== ch44 online first update");
```

listing 338: rank every unit, halve the strength per rank, one epoch, cross-witness the first step (source: kdd/samples/src/Ch46/gas.c, lines 154-199)

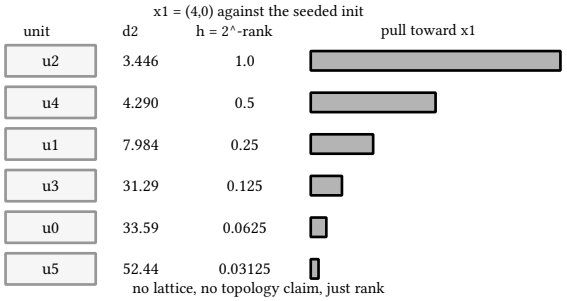


figure 187: x1′s six units on the distance ladder, strength halving with rank

46.4. LVQ1: prototypes with a teacher

Everything so far was unsupervised. LVQ1 takes labeled data and a small set of prototypes, one per class planted at a labeled example, and for each input moves the nearest prototype: toward the input if their labels agree, away if they disagree. The fixture plants A at x1 = (4,0) and B at x5 = (3,8), no LCG anywhere, eta = 1/4, one epoch over the ring with labels A = {x1..x4}, B = {x5..x8}.

The dry run: the trace rows print `lvq x1 lab=A near=A reward` through `lvq x8 lab=B near=A punish`, with exactly three punishments, x4 labeled A but nearer B, x7 and x8 labeled B but nearer A, each row a check. The final prototypes print `lvq final A=(8.19140625,1.9140625)`
`B=(1.828125,7.5)`, both asserted ==, and both exactly representable: integer inputs and eta = 1/4 stack quarters, so every intermediate weight is dyadic, the D1 spine of the sample. Follow B by hand: it never rewards after x4 pushes it away, x5 and x6 pull it to (1.828125, 7.5), and it stays. The classification pass then scores 6 of 8, `lvq accuracy=6/8`, asserted as the reduced 3/4, with exactly the two boundary inputs x4 and x8 wrong, the same patients any 1-NN on these prototypes would miss. The closing compare row pins the whole family on one line, `compare QE_online=1.4362362499549604`
`QE_batch=1.1452847075210475 QE_grown_relief=1.2762259134584661`
`QE_gas_epoch=1.772083269243151 LVQ_acc=6/8`, with the ordering batch below grown below online below gas asserted as its own check. Batch wins quantization on this fixture, growth buys most of batch's win while keeping an explicit topology, and the supervised variant trades the QE game for a decision rule with a measured accuracy.

```
  for (int i = 0; i < 8; i++) {
    double da = pd2(X[i][0], X[i][1], wp[0][0], wp[0][1]);
    double db = pd2(X[i][0], X[i][1], wp[1][0], wp[1][1]);
    int near = da <= db ? 0 : 1; /* tie goes to A */
    int match = (near == 0) == (LAB[i][0] == 'A');
    const char *act = match ? "reward" : "punish";
    for (int k = 0; k < 2; k++) {
      double d = X[i][k] - wp[near][k];
      double p = 0.25 * d;
      if (match)
        wp[near][k] = wp[near][k] + p;
      else
        wp[near][k] = wp[near][k] - p;
    }
    printf("lvq x%d lab=%s near=%s %s\n", i + 1, LAB[i], near ? "B" : "A", act);
    snprintf(nm, sizeof nm, "ch46 lvq x%d lab=%s near=%s %s", i + 1, LAB[i],
             NEAR[i] ? "B" : "A", ACT[i]);
    CHECK(near == NEAR[i] && strcmp(act, ACT[i]) == 0, nm);
  }
  printf("lvq final A=(%.17g,%.17g) B=(%.17g,%.17g)\n", wp[0][0], wp[0][1],
         wp[1][0], wp[1][1]);
  CHECK(wp[0][0] == 8.19140625 && wp[0][1] == 1.9140625,
        "ch46 lvq final A=(8.19140625,1.9140625) (D1 ==)");
  CHECK(wp[1][0] == 1.828125 && wp[1][1] == 7.5,
        "ch46 lvq final B=(1.828125,7.5) (D1 ==)");
```

listing 339: nearest prototype, tie to A, reward toward, punish away, quarters all the way (source: kdd/samples/src/Ch46/lvq.c, lines 58-82)

```
  /* the family comparison row (pinned constants from the sibling files:
   * batch.c QE_batch, grow.c QE_grown_relief, gas.c QE_gas_epoch, and
   * ch45 QE_online), print-only */
  printf("compare QE_online=1.4362362499549604 QE_batch=1.1452847075210475 "
         "QE_grown_relief=1.2762259134584661 QE_gas_epoch=1.772083269243151 "
         "LVQ_acc=6/8\n");
  CHECK(1.1452847075210475 < 1.2762259134584661 &&
            1.2762259134584661 < 1.4362362499549604 &&
            1.4362362499549604 < 1.772083269243151,
        "ch46 QE ordering batch < grown_relief < online < gas");
```

listing 340: the family compare row and the asserted QE ordering (source: kdd/samples/src/Ch46/lvq.c, lines 104-113)

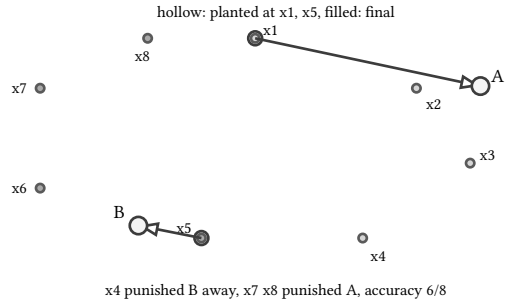


figure 188: two prototypes drift apart: rewards pull, punishments push

sources: batch SOM and LVQ1 are from T. Kohonen, Self-Organizing Maps, Springer Series in Information Sciences 30, 3rd ed. 2001, chapter 3 for the batch form and the supervised LVQ chapter for LVQ1. Growing grid is B. Fritzke, Neural Processing Letters 2(5) 1995, 9-13. Neural gas is T. Martinetz, S. Berkovich, K. Schulten, IEEE Transactions on Neural Networks 4(4) 1993, 558-569. Every pinned double, the corrected gas literals and the computed differ set

{5,7} are witnessed by kdd-contract-s8s9.md and playground/kdd-matrix/gen_s9.py, run 2026-09-22, exit 0, with the sheet's four transcription corrections recorded in the sample headers. Sample behavior verified by `pwsh -NoProfile -File tools/run-c-samples.ps1 -SampleRoot books/kdd/samples/src -Chapter Ch46`, 72 + 50 + 63 + 22 checks in chapter 46 of the kdd suite.

47. the bridge: c to go

47.1. the port and its gates

The dry run: the four c samples print 127 ok lines between them, ch04 with 44, ch05 with 28, ch18 with 25 over two files, and ch31 with 30. The go package `kddcore` runs its 18 tests in under a quarter second, and every contract fixture they replay was printed by the c side first.

Chapters 1 to 46 taught one algorithm family per chapter in c23, small enough to verify by hand. The capstone needs four of them running for real inside a go pipeline, so `internal/kddcore` ports the spine: chapter book 14, ch 4 (knowledge discovery, normalization and standardization) (min-max, z-score, mad), chapter book 14, ch 5 (knowledge discovery, distance metrics) (minkowski and diagonal mahalanobis), chapter book 14, ch 18 (knowledge discovery, roc and auc) (the threshold sweep and concordant auc), and chapter book 14, ch 31 (knowledge discovery, k-means) (lloyd with the pinned tie rules). The bridge is not a rewrite at arm's length: the fixture contracts name this package's tie rules as binding for both lanes, and the chapter 31 sheet reuses this package's own tie test as its TIE fixture. The same numbers, the same order of operations, two compilers.

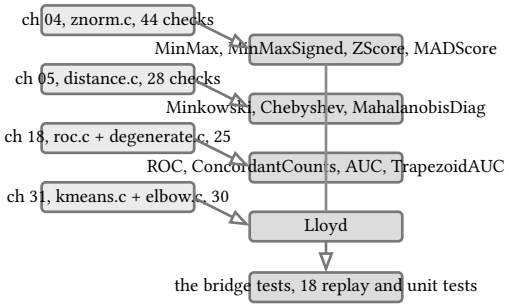


figure 189: the bridge: four c chapters on the left, the kddcore functions in the middle, the pipeline consumers on the right

47.2. chapter 04, normalization

The dry run: the c sample prints `range 16 minmax01: 0 0.25 0.375 0.5 0.75 1` and, two blocks later, `A median 6 MAD 1 robust z: -2 0 0 0 4 10`. The go test asserts the same literals with plain equality, every value dyadic, and adds the signed mirror `-1 -0.5 -0.25 0 0.5 1` from the `2v-1` variant.

The conventions had to be settled before either side was written: population variance with the n divisor, and the mad standardization of Leys, Allen, Fisher, and Lumley 2013, (x-median)/MAD with no 0.6745 scaling, the row the sheet banks. On symmetric series B, where sigma equals MAD, classic z and robust z coincide elementwise, which both sides assert as the closing check of the chapter.

```c
  /* row: MAD standardization (robust z) on A */
  const double amed = median_of(a, 6);
  const double amad = mad_of(a, 6, amed);
  double arz[6];
  for (int i = 0; i < 6; i++) {
    arz[i] = (a[i] - amed) / amad;
  }
  printf("A median %.17g MAD %.17g robust z:", amed, amad);
  for (int i = 0; i < 6; i++) {
    printf(" %.17g", arz[i]);
  }
  printf("\n");
  CHECK(amed == 6.0, "ch04 A median 6");
  CHECK(amad == 1.0, "ch04 A MAD 1");
  CHECK(arz[0] == -2.0, "ch04 robust z(4) -2");
  CHECK(arz[1] == 0.0, "ch04 robust z(6) 0 (first)");
  CHECK(arz[2] == 0.0, "ch04 robust z(6) 0 (second)");
  CHECK(arz[3] == 0.0, "ch04 robust z(6) 0 (third)");
  CHECK(arz[4] == 4.0, "ch04 robust z(10) 4");
  CHECK(arz[5] == 10.0, "ch04 robust z(16) 10");

  /* row: z vs robust z contrast at the point 16 */
  CHECK(az[5] == 2.0, "ch04 contrast classic z(16) = 2");
  CHECK(arz[5] == 10.0, "ch04 contrast robust z(16) = 10");
```

listing 341: the c side: median, mad, robust z, and the outlier contrast at 16 (source: kdd/samples/src/Ch04/znorm.c, lines 166-189)

```go
func MADScore(v []float64) []float64 {
  out := make([]float64, len(v))
  med := Median(v)
  mad := Median(absDev(v, med))
  if mad == 0 {
    return out
  }
  for i, x := range v {
    out[i] = (x - med) / mad
  }
  return out
}

// bounds returns the min and max of v.
func bounds(v []float64) (lo, hi float64) {
  if len(v) == 0 {
```

listing 342: the go side: same definition, one loop, zeros when MAD is zero (source: kdd/capstone/internal/kddcore/normalize.go, lines 84-99)

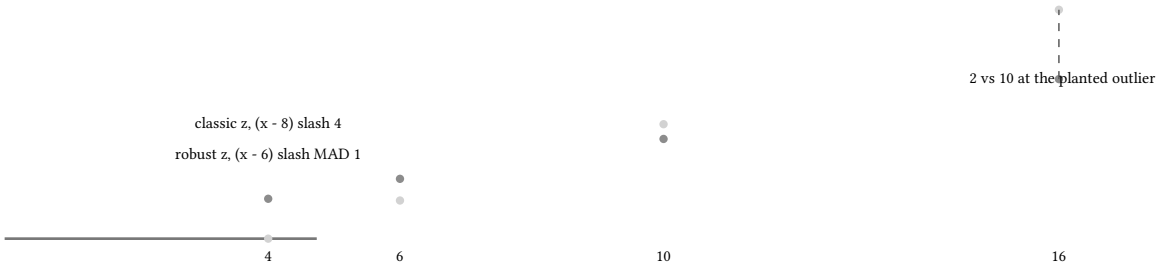


figure 190: series a of chapter 04, classic z against robust z, the point 16 sits two sigmas out but ten MADs out

47.3. chapter 05, distance

The dry run: `d1 = 12`, then `d2^2 = 50, d2 = 7.0710678118654755` print-only, then the cube witness 216 = 6^3 and the fourth-power sum
962. The go side asserts sqrt(50) bit-exactly against its own

computation and the two irrational roots at the sheet's 1e-12 bound.

The chapter owns the minkowski DISTANCE, the L_p metric; the minkowski SUM of convex geometry is a different object, and the disambiguation belongs to book 10, ch 37 (practical data structures and algorithms, computational geometry ii). The ladder monotonicity 12 > 7.071 > 6 > 5.569 is pinned on both sides, and so is the diagonal mahalanobis row, where population variances 2 and 8 put every fixture point on an exact integer distance.

```
  const int diff[3] = {3, 4, 5};

  /* row: minkowski p=1, the manhattan spelling */
  int d1 = 0;
  for (int i = 0; i < 3; i++) {
    d1 += diff[i] < 0 ? -diff[i] : diff[i];
  }
  printf("d1 = %d\n", d1);
  CHECK(d1 == 12, "ch05 minkowski p=1 distance 12");

  /* row: minkowski p=2 -- squared sum 50 asserted as an integer, the root
   * is the print-only D2 row (irrational, 5*sqrt(2)) */
  int p2sq = 0;
  for (int i = 0; i < 3; i++) {
    p2sq += diff[i] * diff[i];
  }
  const double d2 = sqrt((double)p2sq);
  printf("d2^2 = %d, d2 = %.17g (print-only, D2 1e-12)\n", p2sq, d2);
  CHECK(p2sq == 50, "ch05 p=2 squared sum 50");
  CHECK(fabs(d2 - 7.0710678118654755) <= 1e-12,
        "ch05 p=2 root 7.0710678118654755 within 1e-12");
  CHECK(fabs(d2 - 5.0 * sqrt(2.0)) <= 1e-15, "ch05 p=2 root equals 5*sqrt(2)");
```

listing 343: the c side: p=1 as integers, p=2 with the squared-sum witness before the print-only root (source: kdd/samples/src/Ch05/distance.c, lines 46-68)

```
func Minkowski(a, b []float64, p int) float64 {
  if len(a) ≠ len(b) {
    panic("kddcore: minkowski length mismatch")
  }
  if p < 1 {
    panic("kddcore: minkowski needs p ≥ 1")
  }
  sum := 0.0
  for i := range a {
    d := math.Abs(a[i] - b[i])
    switch p {
    case 1:
      sum += d
    case 2:
      sum += d * d
    default:
      sum += math.Pow(d, float64(p))
    }
  }
  if p == 1 {
    return sum
  }
  if p == 2 {
    return math.Sqrt(sum)
  }
  return math.Pow(sum, 1/float64(p))
}

// Chebyshev is the p=infinity limit: the largest single-coordinate
// gap.
```

listing 344: the go side: one loop, the p cases, index-order accumulation (source: kdd/capstone/internal/kddcore/distance.go, lines 12-41)

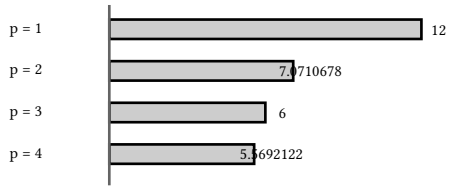


figure 191: the lp ladder on the diff triple (3,4,5), bar length is the distance, all strictly decreasing

47.4. chapter 18, roc

The dry run: the c sample prints seven `point k (a/3,b/3)` rows, `(0,0)` through `(1,1)`, then `concordant=8 discordant=1 ties=0`, then the trapezoid cross-check against 8/9 at 1e-12. The go test walks the same seven corners as exact thirds and reads the same tally.

Two conventions the sheet fixed and both sides carry: the sweep predicts positive when score >= threshold and the curve starts at the origin, and a single-class input closes with a final (1,1) corner, eight points total, area 1 all-positive and 0 all-negative, with the concordant auc undefined. The degenerate rendering has its own c file in Ch18 and its own go assertion.

```
  /* concordant AUC: pos>neg pairs, ties 0.5 each, over all cross pairs */
  int con = 0, ties = 0;
  for (int i = 0; i < N; i++) {
    if (LB[i] != 1)
      continue;
    for (int j = 0; j < N; j++) {
      if (LB[j] != 0)
        continue;
      con += SC[i] > SC[j];
      ties += SC[i] == SC[j];
    }
  }
  int dis = P * Nn - con - ties;
  printf("concordant=%d discordant=%d ties=%d\n", con, dis, ties);
  CHECK(con == 8, "ch18 concordant pairs 8");
  CHECK(dis == 1, "ch18 discordant pairs 1");
  CHECK(ties == 0, "ch18 cross-class ties 0");
  CHECK(frac_eq(2 * con + ties, 2 * P * Nn, 8, 9), "ch18 AUC 8/9");
```

listing 345: the c side: concordant pairs over every cross-class pair, ties half (source: kdd/samples/src/Ch18/roc.c, lines 84-101)

```
func ConcordantCounts(scores []float64, labels []bool) (concordant, discordant, ties float64) {
  for i, si := range scores {
    if !labels[i] {
      continue
    }
    for j, sj := range scores {
      if labels[j] {
        continue
      }
      switch {
      case si > sj:
        concordant++
      case si < sj:
        discordant++
      default:
        ties++
      }
    }
  }
  return concordant, discordant, ties
}

// AUC is the concordant fraction (concordant + half the ties) over
// the positive-negative pair count: the probability a random positive
// outranks a random negative, 1 for a perfect ranking and 0.5 for a
// coin flip. With no cross-class pairs the concordance is undefined
```

listing 346: the go side: the same double loop, the same half-credit ties (source: kdd/capstone/internal/kddcore/roc.go, lines 16-41)

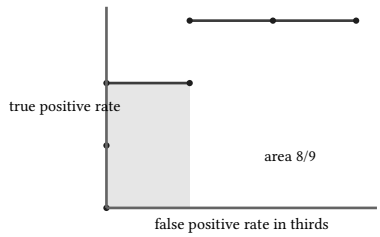


figure 192: the seven pinned corners in thirds, the staircase from (0,0) to (1,1), concordant area 8/9

47.5. chapter 31, k-means

The dry run: the c sample prints `iter0 cents=(0,0),(24,0),(12,16) assign=012000111222 sse=288`, then `iter1 cents=(2,2),(26,2),(14,20) assign=012000111222 sse=144`, then `final iters=2`. The go replay asserts the same assignment string, the same integer centroids, the same two passes, and the same 144.

The tie rules are the bridge's own, adopted by the sheet: seeds are the first k distinct points in input order, assignment goes to the nearest centroid by squared distance with ties to the lowest cluster index, update is the arithmetic mean with an emptied cluster keeping its centroid, and the confirming pass counts in iters. The F12b row is the honest caveat both sides pin: reseed the same twelve points so the first three distinct are two-of-A plus one-of-B, and lloyd lands in a split-A local minimum with sse 540 against the planted 216, which is why chapter book 14, ch 34 (knowledge discovery, cluster validation) cares about initialization and the icpc book's k-means chain in book 11, ch 10 (the icpc world finals, icpc world finals 2017) probes exactly this failure mode.

```
    int moved = 0;
    for (int i = 0; i < n; i++) { /* ties to the lowest cluster index */
      int best = 0;
      Frac bd = d2pt(pts[i], cents[0]);
      for (int c = 1; c < k; c++) {
        const Frac dd = d2pt(pts[i], cents[c]);
        if (fcmp(dd, bd) < 0) {
          best = c;
          bd = dd;
        }
      }
      if (assign[i] != best) {
        assign[i] = best;
        moved = 1;
      }
    }
    if (!moved) { /* confirming pass counted in iters */
      r->iters = pass;
      memcpy(r->cents, cents, sizeof cents);
      for (int i = 0; i < n; i++) {
        r->assign[i] = (char)('0' + assign[i]);
      }
      r->assign[n] = 0;
      return;
    }
```

listing 347: the c side: the pass loop, lowest-index ties, the confirming pass returns (source: kdd/samples/src/Ch31/kmeans.c, lines 152-176)

```
  for pass := 1; pass <= maxIter; pass++ {
    moved := false
    for i, p := range points {
      a := nearest(p, centroids)
      if assign[i] ≠ a {
        assign[i] = a
        moved = true
      }
    }
    if !moved {
      return assign, centroids, pass
    }
    centroids = updateCentroids(points, assign, centroids, k)
    iters = pass
  }
```

listing 348: the go side: the same loop shape, same tie rule in nearest (source: kdd/capstone/internal/kddcore/kmeans.go, lines 27-41)

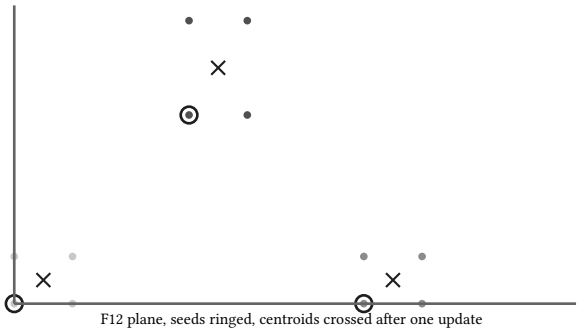


figure 193: fixture F12: three 4-point rectangles, the three seeds ringed, final centroids crossed, sse 144

47.6. the agreement table

The ledger, one row per fixture, both sides' receipts:

ch	fixture	c pins	go asserts	lane
04	series M	`minmax01 0, .25, .375, .5, .75, 1` and the 2v-1 mirror	the six plus six dyadic literals with ==	D1
04	series A	pop var 16, z ends at 2, robust z ends at 10	z and robust z elementwise	D1
04	series B	z equals robust z, all seven values	same, elementwise ==	D1
05	diff (3,4,5)	d1 12, squared 50, cube 216, fourth 962, monotone	sqrt(50) exact, roots at 1e-12, witnesses	D0+D2
05	pythagorean	sqrt(25.0) is 5, sqrt(169.0) is 13	the same two == checks	D1
05	diag covariance	var 2 and 8, D 0,1,2,2,3	the five exact distances	D1
18	6-score sweep	seven corners in thirds	the same corners as exact thirds	D0
18	concordance	8, 1, 0, auc 8/9, trapezoid	both auc routes	D0+D2
18	degenerate	8 points, areas 1 and 0, undefined	NaN concordance, closed curves	D0
31	F12	iters 2, assign 012000111222, sse 144	same, integer centroids ==	D0+D1
31	F12b	local minimum sse 540 vs planted 216	same split, same 540	D0+D1
31	TIE	joins cluster 0, cents (1/2,0),(2,0), sse 1/2	the same three values	D1

The lane column is the agreement class: D0 and D1 rows are bit-exact on both sides, which is possible because the fixtures are integers and dyadic rationals, and D2 rows agree at the sheet's stated 1e-12. The c side computes its fractions in exact integer arithmetic where the sheet demands it; the go side uses float64 throughout, and on these fixtures the two agree to the bit, which is the whole point of choosing dyadic pins.

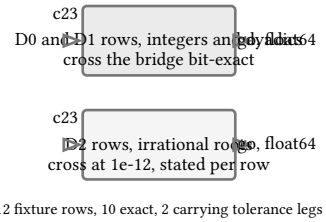


figure 194: the lane ladder across the bridge: exact rows cross unchanged, tolerance rows cross at the stated bound

47.7. what the bridge buys

The dry run: the capstone run of chapter book 14, ch 48 (knowledge discovery, the capstone: a kdd pipeline on duckdb, with coverage appendices) grades its tree with the auc of this package and its clusters with this lloyd, and the digest moves if either drifts.

Nothing in the pipeline reimplements a bridged algorithm: the anomaly stage scores with minkowski at p=2 through the lof engine, the segmentation calls lloyd directly, the tree evaluation reads the auc and the panel's roc curve comes from the same corner walk. The port also inherits the book's determinism discipline: float64 in index order, tie rules written down and tested, and every fixture the c chapters pinned replayed in go on every untagged test run, so the two languages cannot quietly diverge.

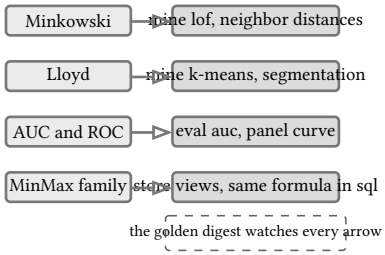


figure 195: the consumers: every bridged function has a pipeline caller, the digest watches all of them

sources: no external pages fetched for this chapter, none cited. Every fixture value is quoted from the signed contract sheets and printed by the c samples under their gates, both rerun green on 2026-09-22, and asserted by the go tests of books/kdd/capstone/internal/kddcore, 18 tests green the same day.

48. the capstone: a kdd pipeline on duckdb, with coverage appendices

48.1. one module, two lanes

The dry run: the canonical run of the whole program prints fifteen lines and exits zero, `seed=42` first and `digest=34120a…` last, with 1224 customers, 9895 transactions, and 35340 basket items between them. The same fifteen lines print again on the next run, and the golden test fails if a single digit moves.

The capstone is one go module, `books/kdd/capstone`, module name `kddcapstone`, with one external dependency: the duckdb go driver at v2.10505.0, pinned in `go.mod`, driving the repo-built duckdb 1.5.5 dll the Makefile pins. Everything else in the module is written by this book's waves. The module is the production twin of the c spine: the four chapters the pipeline leans on hardest, 4, 5, 18, and 31, live again in `internal/kddcore` as pure go, and the pipeline around them is the shape a real discovery project takes, a generator, an embedded column store, miners, an evaluator, and a results panel. The bridge from the c chapters is chapter book 14, ch 47 (knowledge discovery, the bridge: c to go) the embedded-engine precedent is book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine), whose analytics store this chapter's store package follows on schema stamping and the single-connection discipline.

```
module kddcapstone

go 1.27

require github.com/duckdb/duckdb-go/v2 v2.10505.0

require (
```

listing 349: the whole external surface: one driver, one version (source: kdd/capstone/go.mod, lines 1-7)

The driver is cgo. A pure go lane and a duckdb lane share the same sources through one build tag: every file that imports the driver, and only those, sits behind `duckdb_use_lib`, and each package keeps an untagged anchor so `go vet ./…` on the plain lane still compiles every package, the chapter 11 pattern. The untagged lane runs vet, test, and the race detector over pure go sources with no duckdb import anywhere in it, which is also the gate that an untagged file can never import the driver: an untagged import would drag the driver's prebuilt static libraries into a plain build and fail loudly.

```
//go:build duckdb_use_lib

package store

// The tagged lane registers the real driver: the blank import pulls in
// the duckdb-go v2 bindings, which link the pinned duckdb.dll through
// the zig cgo recipe documented in the Makefile's DUCKDB_GOENV.
import _ "github.com/duckdb/duckdb-go/v2"

// duckdbDriverOK reports whether the driver was registered in this
// build. Open checks it first so a pure go build fails loudly at the
// door instead of with sql's generic unknown-driver error.
const duckdbDriverOK = true
```

listing 350: the tag seam in full: the tagged lane registers the driver, the pure lane refuses at the door (source: kdd/capstone/internal/store/driver_use_lib.go, lines 1-14)

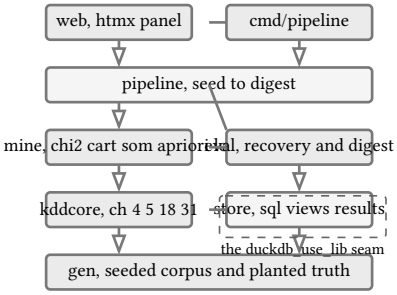


figure 196: the module's layers, arrows are compile-time imports, the dashed seam is the one build tag

48.2. the generator plants its own truth

The dry run: for seed 42 the generator emits 1224 customers in four groups, 400 per planted segment and 24 outliers, and 43.7 percent of customers respond. The masks it plants on the way out are 54 hidden incomes, 41 hidden ages, and 1039 hidden sale prices, each remembered in the truth block, so the cleaning stage is graded against what it never saw.

Every number the pipeline later reports is a function of one int64. The generator keeps a splitmix64 stream seeded from it, no math/rand, no clock, no file system: the stream advances by the golden ratio constant and every draw consumes it in a fixed order, clusters first, then outliers, then missing masks, then baskets. The corpus is shaped like retail: 8 categories of 6 products, customers carrying age, income, a spending score, tenure in months, and visit counts, transactions of 1 to 5 ordinary items each, plus both members of one of three planted product pairs with 35 percent probability. The planted structure is the point: three gaussian clusters far apart in the income-spend plane for k-means and the som, a response rule `spend ≥ 55 and tenure ≥ 24` with 4 percent label flips for the tree, boosted co-occurrences for apriori, and a uniform sparse box of 24 customers for lof.

```
func newRNG(seed int64) *rng {
  return &rng{state: uint64(seed)}
}

func (r *rng) next() uint64 {
  r.state += 0x9E3779B97F4A7C15
  z := r.state
  z ^= z >> 30
  z *= 0xBF58476D1CE4E5B9
  z ^= z >> 27
  z *= 0x94D049BB133111EB
  return z ^ (z >> 31)
```

listing 351: the whole stochastic surface: one state, the splitmix64 step (source: kdd/capstone/internal/gen/rng.go, lines 15-26)

```
for seg := 0; seg < SegmentCount; seg++ {
  c := segmentCentroids[seg]
  lo, hi := tenureRange[seg][0], tenureRange[seg][1]
  for i := 0; i < RegularPerSegment; i++ {
    id++
    age := c.Age + noiseSigmas.Age*r.norm()
    income := c.Income + noiseSigmas.Income*r.norm()
    spend := clamp(c.Spend+noiseSigmas.Spend*r.norm(), 0, 100)
    tenure := r.uniform(lo, hi)
    visits := int64(math.Max(1, math.Round(spend/8+2*r.norm())))
    want := spend ≥ labelRule.SpendCut && tenure ≥ labelRule.TenureCut
    got := want
    if r.float64() < labelRule.FlipRate {
      got = !want
    }
    customers = append(customers, Customer{
      ID: id, Segment: seg, Age: age, Income: income,
      Spend: spend, Tenure: tenure, Visits: visits, Responds: got,
    })
    pre = append(pre, want)
  }
}
```

listing 352: one planted customer: three gaussian draws, one uniform tenure, the rule, the flip (source: kdd/capstone/internal/gen/customers.go, lines 20-41)

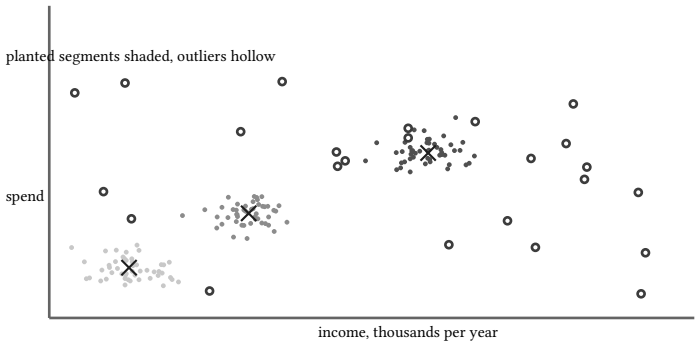


figure 197: the seed-42 corpus in the income-spend plane, every eighth inlier and all 24 outliers, crosses mark the planted centroids

48.3. the store: load, clean, transform

The dry run: the store opens a fresh duckdb file, stamps schema version 1, and answers `v1.5.5` to `SELECT version()`. The appender loads all 46507 rows (four tables) without one sql string, and the cleaning view fills every planted hole: 41 ages become 43.246376, the global median, and all 1039 hidden sale prices come back within the 12.5 percent bound, which is why the run's metric block ends its cleaning line at `price_within=1.000000`.

The store owns every duckdb touch. It opens one file behind one connection with `threads=1`, the determinism choice: a single aggregation order keeps the floating point sums a pure function of the rows, which the digest discipline needs. Loading goes through the driver's appender, one row per call, NaN masked values crossed to sql NULL on the way in. A file stamped with a newer schema version is refused rather than guessed at, the analytics precedent from book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine). The transform stage is three views: `v_customers_clean` imputes age and income with the global median, `v_items_clean` joins each masked price to its product's median sale price, and `v_features` and `v_bins` min-max normalize and equal-width bin the cleaned columns. The miners never mutate the loaded corpus, they read views.

```
        return err
      }
    }
    return nil
  })
}

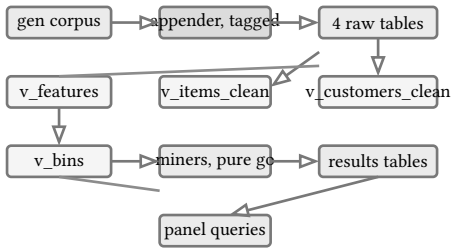
func appendCustomers(ctx context.Context, conn *sql.Conn, customers []gen.Customer) error {
  return withAppender(ctx, conn, "customers", func(app *duckdb.Appender) error {
    for _, c := range customers {
      if err := app.AppendRow(c.ID, c.Segment, f64(c.Age), f64(c.Income),
```

listing 353: the appender loop: one row per call, NaN to NULL at the boundary (source: kdd/capstone/internal/store/load.go, lines 84-94)

```
FROM customers;

CREATE OR REPLACE VIEW v_items_clean AS
SELECT
  i.id,
  i.tx_id,
  i.product_id,
  i.qty,
```

listing 354: the price imputation view: each masked row joins its product's median sale price (source: kdd/capstone/internal/store/views.go, lines 23-30)



the only writer is the load, the only reader path is views and result tables

figure 198: the data path, wide boxes are tables and views, the appender is the only writer

48.4. selection, mining, and the planted-truth evaluation

The dry run: chi-square ranks `chi2_top=spend>income`, lof flags 17 of the 24 planted outliers exactly, k-means over the surviving 1200 inliers reaches `cluster_purity=0.984167`, the som folds the same plane at `quant_err=0.054221`, the tree scores `tree_accuracy=0.946721` and `tree_auc=0.958421` on the every-fifth-row holdout, and apriori at support 247 (2.5 percent of 9895 baskets) finds exactly 48 frequent singles and 3 frequent pairs, the three planted ones, so `rule_recovery=1.000000`.

The mining layer runs the book's techniques over the store's views, and every engine is the chapter's own mathematics: chi-square selection over the four-bin contingency tables, the `kddcore` lloyd engine for segmentation with the som organizing the same plane on a 2x3 lattice, a depth-4 gini cart over the five normalized features with the causal pair first in the column order, levelwise apriori over the basket sets with rules from the maximal frequent itemsets, and lof with ten neighbors over the normalized income-spend plane. The anomaly stage feeds the segmentation: clustering runs on the lof-filtered inliers, which is why the purity line grades 1200 rows, not 1224. The evaluation reads the generator's truth block, not re-derived expectations: majority-cluster purity against the planted segments, top-24 recall against the planted outlier ids, and rule recovery against the planted pairs, each direction accepted, lift above one required.

```
  }
  lof := mine.LOFScores(points, lofNeighbors)
  topIdx := mine.TopLOF(lof, gen.OutlierCount)
  outlierIDs := make([]int64, len(topIdx))
  flagged := make(map[int]bool, len(topIdx))
  for rank, i := range topIdx {
    outlierIDs[rank] = features[i].ID
    flagged[i] = true
  }
  outlierRecall := eval.OutlierRecall(outlierIDs, ds.Truth.OutlierIDs)

  // segmentation runs on the lof-filtered inliers only
  inPoints := make([][]float64, 0, len(points))
  inPlanted := make([]int, 0, len(points))
  inClean := make([]store.CleanCustomer, 0, len(clean))
  for i := range points {
    if flagged[i] {
      continue
    }
    inPoints = append(inPoints, points[i])
    inPlanted = append(inPlanted, features[i].Segment)
    inClean = append(inClean, clean[i])
  }
  km := mine.KMeans(inPoints, clusterK, clusterMaxIter)
  purity := eval.ClusterPurity(km.Assign, inPlanted)
```

listing 355: the anomaly-to-segmentation handoff: lof flags, the inliers cluster, purity grades against the planted segments (source: kdd/capstone/internal/pipeline/pipeline.go, lines 96-120)

```
  // association stage over the raw baskets
  basketLists := make([][]int64, len(baskets))
  for i, b := range baskets {
    basketLists[i] = b.Products
  }
  // 2.5 percent support: the planted pairs sit near 12 percent while
  // an incidental triple (a planted pair plus one common product)
  // tops out around 2 percent, so the threshold separates them with
  // margin and the planted pairs are the maximal frequent pairs.
  minSupport := max(len(baskets)/40, 1)
  rules := mine.Frequent(basketLists, minSupport, aprioriMaxK).Rules(ruleMinConf)
  ruleRecovery := eval.RuleRecovery(rules, plantedPairs(ds))
```

listing 356: the support threshold and its margin: planted pairs near 12 percent, incidental triples under 2 (source: kdd/capstone/internal/pipeline/pipeline.go, lines 143-156)

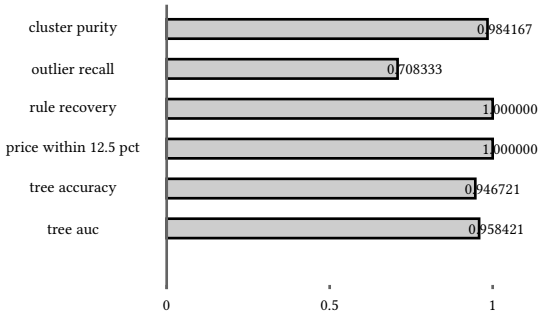


figure 199: the six recovery numbers of the canonical run, bar length is the value, one is perfect

48.5. the results panel

The dry run: after the run persists its results, the panel reads back three segment rows, six rule rows (the six directions of the three planted pairs), twelve anomaly ranks, and the roc corners, and renders one html document that carries the vendored htmx under its subresource integrity digest `sha384-BvJpBiO8...`.

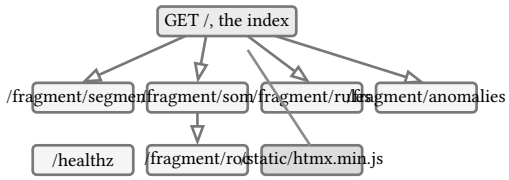
The web tier is an html/template site with htmx refreshing five fragments, the segment table, the som grid, the rule table, the anomaly ranking, and the roc curve drawn as an inline svg polyline. The index server-renders every fragment, so the page reads without scripting and htmx only freshens it. The script is vendored byte-identical from the infrastructure capstone's copy, pinned by an sri constant the test re-derives from the embedded bytes, and the whole tier is smoke tested through httptest inside `go test`, no browser, no server left running, the same recipe as book 13, ch 20 (infrastructure: docker, databases, and testing, capstone part 2: chat and notifications, one click).

```
// HtmxSri pins the vendored htmx.min.js: the file never changes under
// this digest, and a swap means re-deriving it. It matches the
// infrastructure capstone's copy byte for byte.
const HtmxSri = "sha384-BvJpBiO8Kh31EqtJe5DRIeWrHWnCGkwytKs9NKFi86Hhw96dEqdEMzZDeK9iEGTc"
```

listing 357: the sri pin: the vendored script never changes under this digest (source: kdd/capstone/internal/web/page.go, lines 15-18)

```
const indexTmpl = `{{define "index"}}<!doctype html>
<html>
<head>
<meta charset="utf-8">
<title>kdd capstone</title>
<script src="/static/htmx.min.js" integrity="{{.Sri}}" crossorigin="anonymous"></script>
{{.Style}}
```

listing 358: the template carries the digest into the served document (source: kdd/capstone/internal/web/page.go, lines 93-99)



every fragment is also a duckdb query away: the panel reads the persisted tables

figure 200: the panel's routes: the index server-renders the fragments, htmx polls them every ten seconds

48.6. verification: the golden digest

The dry run: the golden test runs the whole pipeline twice on fresh temp files for seed 42, both runs print `digest=34120a8fbe7ebaa21a885b898ae07357ba02dcf57003475ab5d4d4c21882f1fe`, and the pinned constant must match or the test dies with the whole metric block in the failure message.

The digest is the sha256 of fourteen canonical lines: the seed, the three corpus counts, the chi-square top two, seven quality floats at six decimals (six recovery numbers and the som's quantization error), and the two imputation errors. Nothing about the format is free: counts print as integers, floats at fixed precision, so a behavioral drift anywhere in the chain, generator draw order, duckdb view arithmetic, engine tie rules, evaluation, or formatting, moves the digest and the test names the drifted lines. The floors under the pin keep the recovery honest rather than merely stable: accuracy at 0.85, auc at 0.90, purity at 0.90, outlier recall at 0.70, rule and price recovery at 0.99, and the som's quantization error under 0.25. Re-witnessing is a discipline, not an edit: when a fixture or algorithm legitimately changes, the run prints the new digest, the constant is re-pinned, and the runner's pin moves in the same commit.

```
metrics, panel, err := Run(42, filepath.Join(t.TempDir(), "kdd.duckdb"))
if err ≠ nil {
  t.Fatal(err)
}
if goldenDigest == "" {
  t.Fatalf("witness run: digest=%s (pin it in goldenDigest)", metrics.Digest())
}
if metrics.Digest() ≠ goldenDigest {
  t.Fatalf("digest drift:\n got %s\nwant %s\nlines:\n%s",
    metrics.Digest(), goldenDigest,
    strings.Join(metrics.Lines(), "\n"))
}
```

listing 359: the pin: same run, same fourteen lines, same sha256, or the failure carries the drift (source: kdd/capstone/internal/pipeline/golden_test.go, lines 21-32)

The gate chain lives in `tools/run-kdd-capstone.ps1`, red on any failure with one loud scaffold skip before the module exists. Leg 1 runs the pure go lane, vet, test, and race, no c compiler in the loop. Leg 2 repeats vet and test behind `duckdb_use_lib` with zig as the c compiler against the pinned duckdb.dll, the documented windows recipe. Leg 3 builds the actual binary both ways: the tagged build must print the pinned digest and render the panel html, and the pure build must refuse with exit 1 naming the missing tag. The counts this chapter owns: 48 tests on the pure lane, 56 on the tagged lane, 58 distinct, zero skipped, run green on 2026-09-22.

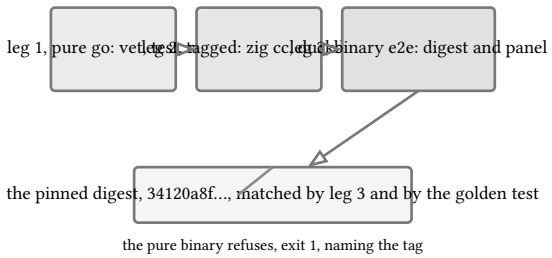


figure 201: the three gate legs of tools/run-kdd-capstone.ps1, each red on failure, the digest pin stands at the end

48.7. topic coverage

The registries close the book. Every chapter wrote its topic rows into the coverage matrix as it landed, each row naming the sample CHECK that proves the claim, and this section reads the registry back the only way that keeps it honest, by computing from it. The chapter ledger below recomputes at compile time: 260 topic rows over 48 chapters, 0 rows off the manifest out of 260, and the chapters still carrying no rows of their own are . A row added or retitled tomorrow shows up here without editing this chapter, the same contract the math book's appendices keep.

ch	chapter	topics
1	the knowledge discovery process	3
2	data cleaning and imputation	4
3	binning, smoothing, and discretization	4
4	normalization and standardization	4
5	distance metrics	4
6	similarity measures	4
7	sampling	4
8	feature selection and chi-square	5
9	principal component analysis	5
10	the haar wavelet transform	4
11	data cubes and olap	4

ch	chapter	topics
12	simple linear regression	4
13	decision trees: id3	4
14	decision trees: cart and continuous splits	4
15	rules from trees and sequential covering	4
16	overfitting and pruning	4
17	classifier evaluation	10
18	roc and auc	6
19	naive bayes	5
20	nearest neighbor	6
21	neural networks	8
22	support vector machines	4
23	bagging, boosting, and random forests	3
24	class imbalance	5
25	levelwise apriori	5
26	vector apriori	5
27	maximal itemsets and rule generation	6
28	fp-growth	6
29	interestingness, multilevel, and quantitative rules	9
30	sequential patterns	6
31	k-means	5
32	hierarchical clustering	4
33	dbscan	4
34	cluster validation	4
35	cure and chameleon	4
36	spectral clustering	4
37	statistical anomaly detection	11
38	proximity and density anomalies	8
39	hypothesis testing and p-values	9
40	multiple testing: bonferroni and fdr	7
41	rough sets: indiscernibility and approximations	6
42	rough sets: dependency and reducts	5
43	rough sets: discernibility and rule induction	7
44	self-organizing maps	7
45	som analysis and visualization	6
46	som variants	10
47	the bridge: c to go	3
48	the capstone: a kdd pipeline on duckdb, with coverage appendices	7
total over 48 chapters		**260**

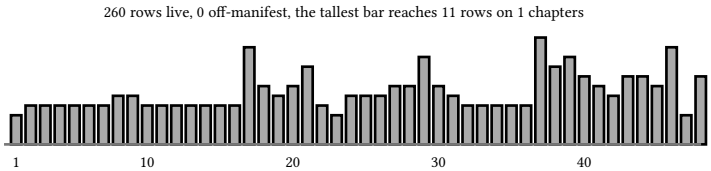


figure 202: topic rows per chapter, computed from the registry at compile time, the capstone pair closes the book

48.8. pinned sources

Every external claim in the book carries a dated url or a named paper, 49 rows so far, and the classifier reads only url prefixes, so the class counts are mechanical:

class	rows	rule
web	10	url starts `https://`
papers by name	39	named papers and books, everything else
corpus internal	0	url starts `books/`, `playground/`, or `tools/`

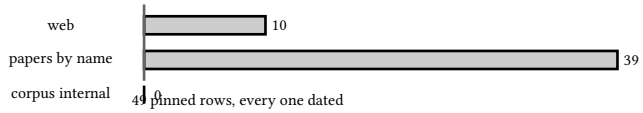


figure 203: the source base by class, computed from the registry, bar length is row count

The full index:

topic	url or citation	accessed
robust z definition	Leys, Allen, Fisher, Lumley 2013, Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median, J. Exp. Soc. Psych. 49(4) 764-766	2026-09-22
play-tennis dataset	Mitchell, Machine Learning, McGraw-Hill 1997, Table 3.2 p. 59 (after Quinlan 1986, Machine Learning 1:81-106)	2026-09-22
reduced-error pruning	Quinlan 1987 (per kdd fixture contract ch16)	2026-09-22
sequential covering	Cohen 1995; Clark & Boswell 1991 (per kdd fixture contract ch15)	2026-09-22
star schema vocabulary	Kimball & Ross, The Data Warehouse Toolkit, 2002 (per kdd fixture contract ch11)	2026-09-22
cube lattice cell counting	Gray et al 1997, Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab and Sub-Totals, ICDE 1996 (per kdd fixture contract ch11)	2026-09-22
play-tennis conditionals (14 rows, 9 yes / 5 no)	Mitchell, Machine Learning (1997), Table 3.2	2026-09-22
kdd definition and 5-stage process	https://ojs.aaai.org/index.php/aimagazine/article/view/1230 (Fayyad, Piatetsky-Shapiro, Smyth, AI Magazine 17-3, 1996)	2026-09-22
tukey 1.5 iqr fences	John W. Tukey, Exploratory Data Analysis, Addison-Wesley 1977, verified via https://en.wikipedia.org/wiki/Box_plot	2026-09-22
equal-depth binning walkthrough of C9	https://www.cs.rpi.edu/~zaki/DMML/slides/pdf/ychap3.pdf (Zaki and Meira, DMML 2nd ed, ch 3)	2026-09-22
sqrt exactness / integer cube-root witness discipline	kdd-contract-s1s2.md C stream notes + playground/kdd-matrix/gen_s1.py (run 2026-09-22, exit 0)	2026-09-22
minstd LCG constants	Park & Miller 1988, Random Number Generators: Good Ones Are Hard to Find, CACM 31(10) p1192	2026-09-22
chi-square critical values df 1 and df 2	https://www.itl.nist.gov/div898/handbook/prc/section4/prc45.htm (NIST/SEMATECH e-Handbook of Statistical Methods)	2026-09-22
intention/execution minimum edit distance example	https://web.stanford.edu/~jurafsky/slp3/ ch 2 (Jurafsky & Martin, Speech and Language Processing, 3rd ed. draft)	2026-09-22
LOF definitions	Breunig, Kriegel, Ng, Sander, LOF: Identifying Density-Based Local Outliers, SIGMOD 2000, pp 93-104 (doi 10.1145/335191.335388)	2026-09-22
MAD robust z without 0.6745	Leys et al., Journal of Experimental Social Psychology 49(4) 2013, pp 764-766	2026-09-22
chi-square df=2 critical 5.991	https://www.itl.nist.gov/div898/handbook/eda/section3/eda367.htm	2026-09-22
MINSTD LCG	Park & Miller, CACM 31(10) 1988, pp 1192-1201	2026-09-22
Holm step-down	Holm, Scand. J. Statist. 6(2) 1979, pp 65-70	2026-09-22
Benjamini-Hochberg FDR	Benjamini & Hochberg, JRSS-B 57(1) 1995, pp 289-300	2026-09-22
t df4 critical 2.7764	https://www.medcalc.org/manual/t_distribution.php	2026-09-22
Pawlak flu decision table	Z. Pawlak, Rough Sets, International Journal of Computer and Information Sciences 11(5) 1982, 341-356	2026-09-22
discernibility matrix tradition	A. Skowron, C. Rauszer, The Discernibility Matrices and Functions in Information Systems, in Intelligent Decision Support, Kluwer 1992	2026-09-22
SOM, batch SOM, LVQ1	T. Kohonen, Self-Organizing Maps, Springer, 3rd ed. 2001 (ch 3 batch, supervised LVQ chapter)	2026-09-22
growing grid	B. Fritzke, Growing Grid - a self-organizing network with constant neighborhood range and adaptation strength, Neural Processing Letters 2(5) 1995, 9-13	2026-09-22
neural gas	T. Martinetz, S. Berkovich, K. Schulten, Neural-gas Network for Vector Quantization and its Application to Time-Series Prediction, IEEE Trans. Neural Networks 4(4) 1993, 558-569	2026-09-22
SMO selection heuristic and b1/b2 bias rule	Platt, Sequential Minimal Optimization, Microsoft Research technical report MSR-TR-98-14, 1998, canonical pdf https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-98-14.pdf	2026-09-22

topic	url or citation	accessed
AdaBoost reweighting and alpha=0.5 ln((1-e)/e)	Freund and Schapire, A Decision-Theoretic Generalization of On-Line Learning, JCSS 55(1), 1997, doi 10.1006/jcss.1997.1504	2026-09-22
bootstrap aggregating	Breiman, Bagging Predictors, Machine Learning 24(2), 1996, doi 10.1007/BF00058655	2026-09-22
random forests and MDI importance	Breiman, Random Forests, Machine Learning 45(1), 2001, doi 10.1023/A:1010933404324	2026-09-22
over/undersampling under class imbalance	Japkowicz and Stephen, The Class Imbalance Problem: A Systematic Study, Intelligent Data Analysis 6(5), 2002, doi 10.3233/IDA-2002-6504	2026-09-22
M2 basket fixture, Han-Pei-Yin fp-growth running example table (TIDs 100-900)	https://www.cs.sfu.ca/~jpei/publications/sigmod00.pdf	2026-09-22
chi-square df=1 alpha=0.05 upper-tail critical value 3.8415, NIST/SEMATECH e-Handbook sec 1.3.6.7.4	https://www.itl.nist.gov/div898/handbook/eda/section3/eda3674.htm	2026-09-22
gaussian-noise MLE justification for least squares	math book ch 19 statistics (in-repo, banked per kdd-contract-s3.md ch12)	2026-09-22
gini impurity and cart binary discipline	kdd-contract-s3.md ch14 conventions (multiway-for-comparability, threshold left branch x <= t)	2026-09-22
lloyd k-means	Lloyd 1982, Least squares quantization in PCM, IEEE Trans. Inf. Theory 28(2); MacQueen 1967	2026-09-22
lance-williams recurrence	Lance & Williams 1967, Comput. J. 9(1); Kaufman & Rousseeuw 1990, Finding Groups in Data	2026-09-22
dbscan minpts convention	https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html	2026-09-22
dbscan algorithm	Ester, Kriegel, Sander, Xu, KDD-96	2026-09-22
silhouette	Rousseeuw 1987, Silhouettes, J. Comput. Appl. Math. 20:53-65	2026-09-22
elbow second-difference rule	Thorndike 1953, Psychometrika 18:267	2026-09-22
cure	https://dl.acm.org/doi/10.1145/276304.276312 (Guha, Rastogi, Shim, SIGMOD 1998)	2026-09-22
chameleon RI/RC	Karypis, Han, Kumar, IEEE Computer 32(8):68-75, 1999	2026-09-22
spectral clustering tutorial	von Luxburg 2007, Stat. Comput. 17:395-416	2026-09-22
normalized cut	Shi & Malik 2000, IEEE PAMI 22(8)	2026-09-22
ratio cut	Hagen & Kahng 1992, IEEE Trans. CAD 11(9)	2026-09-22
fiedler vector	Fiedler 1973, Czech. Math. J. 23(2)	2026-09-22
bonferroni correction alpha/m	Dunn, Multiple Comparisons Among Means, JASA 56(293) 1961, 52-64 (Bonferroni inequality applied to families of tests)	2026-09-22
k-means++ careful seeding as the restart remedy	Arthur and Vassilvitskii, k-means++: The Advantages of Careful Seeding, SODA 2007, https://dl.acm.org/doi/10.5555/1283383.1283494	2026-09-22

VERIFY: this chapter's own receipts

The digest, the metric block, and the leg-3 behavior were printed by `tools/run-kdd-capstone.ps1` on 2026-09-22, and the 58 distinct go tests ran green the same day, 48 on the pure lane and 56 on the tagged lane. The scatter and the planted hidden counts come from a disposable gen probe replaying seed 42, and every appendix count recomputes from the registries at compile time.

sources: no external pages fetched for this chapter, none cited. The driver and engine versions are read from the module's own `go.mod` and the store's engine-version test, the htmx digest from the vendored bytes, and the appendix counts from books/kdd/coverage/topics.typ, 260 rows, and books/kdd/coverage/sources.typ, 49 rows, at compile time.